

Cylindrical Algebraic Decomposition Based Methods in Satisfiability Modulo Non-linear Real Arithmetic

Überarbeitete Version
vom 6. Mai 2026
der Dissertation
zur Erlangung des akademischen Grades
eines Doktors der Naturwissenschaften

von

Jasper Nalbach, Master of Science

aus Bonn, Deutschland

Berichter: Universitätsprofessorin Dr. Dr. h.c. Erika Ábrahám
Universitätsprofessor Dr. Dr. h.c. James H. Davenport

Tag der mündlichen Prüfung: 12. November 2025

Abstract

The first-order theory of the reals admits an expressive language with a variety of potential applications. The cylindrical algebraic decomposition method is a powerful tool from real algebra for answering different kinds of problems related to the theory of the reals, among them satisfiability of quantifier-free formulas, validity of sentences, and quantifier elimination. It decomposes the real space into finitely many subsets in which certain properties of the polynomials occurring in the input are invariant. The resulting decomposition is then used to conclude about properties of the input formula. Despite its doubly exponential complexity bound, up to today, the cylindrical algebraic decomposition method has been the foundation of most algorithms implemented in state-of-the-art tools.

Satisfiability-modulo-theories solving is a technology for checking the satisfiability of quantifier-free first-order logic formulas over one or more theories. Modern solvers support a wide range of theories and are applied to real-world problems. In the past decade, these solvers have been extended to support also real arithmetic formulas, based on ideas from the cylindrical algebraic decomposition. Instead of computing a whole decomposition eagerly, these adaptations follow an exploration-guided approach: by guessing sample points, relevant conflicts are detected and generalized using technology from the cylindrical algebraic decomposition, yielding a coarser decomposition than what the original method would compute. Although the theoretical complexity bound is not improved, these methods allow room for heuristics that guide the search process and may improve practical efficiency on real-world problems.

This thesis concerns a variety of such exploration-guided algorithms. We identify common parts of these algorithms to unify their presentation and transfer ideas between these methods, further improving their practical efficiency. Further, we transfer the exploration-guided approach to quantifier elimination by extending these algorithms accordingly.

The cylindrical algebraic decomposition implicitly decomposes the space by computing projection polynomials. The set of projection polynomials can be reduced in exploration-guided algorithms, as we only need to compute a decomposition tailored towards a given conflict. As this projection set is determined according to more complex rules, this thesis formulates the projection operation as a proof system. This makes the algorithms extensible for further optimizations, increases the trust into their implementation, and defines interfaces for fine-grained heuristics. Further, it is a first step towards certificates witnessing the correctness of a solver's run.

Based on the proof system, this thesis contributes a range of improvements to the projection operation, and leads to new notions that contribute to the understanding of the theory behind the cylindrical algebraic decomposition. We further examine some heuristics that improve the practical efficiency of exploration-guided algorithms.

The methods presented in this thesis are experimentally evaluated based on an implementation in the SMT solver SMT-RAT.

Zusammenfassung

Die Theorie erster Ordnung der reellen Zahlen bietet eine ausdrucksstarke Sprache mit einer Vielzahl von Anwendungsmöglichkeiten. Die Methode der zylindrischen algebraischen Zerlegung ist ein leistungsfähiges Werkzeug aus der reellen Algebra zur Beantwortung verschiedener Arten von Problemen im Zusammenhang mit der Theorie der reellen Zahlen, darunter die Erfüllbarkeit von quantorenfreien Formeln, die Gültigkeit von Sätzen und die Eliminierung von Quantoren. Sie zerlegt den reellen Raum in endlich viele Teilmengen, in denen bestimmte Eigenschaften der Polynome aus der Eingabe-Formel invariant sind. Die sich daraus ergebende Zerlegung wird dann verwendet, um auf die Eigenschaften der Eingabe-Formel zu schließen. Trotz seiner doppelt exponentiellen Komplexitätsgrenze ist die Methode der zylindrischen algebraischen Zerlegung bis heute die Grundlage aller Algorithmen, die in modernen Werkzeugen implementiert sind.

Satisfiability-modulo-theories Solving ist eine Technik zur Überprüfung der Erfüllbarkeit von quantorenfreien Formeln der Logik erster Ordnung über eine oder mehrere Theorien. Moderne Solver unterstützen eine breite Palette von Theorien und werden bei realen Problemen eingesetzt. Im letzten Jahrzehnt wurden diese Solver so erweitert, dass sie auch reelle arithmetische Formeln unterstützen, die auf Ideen aus der zylindrischen algebraischen Zerlegung basieren. Anstatt eine vollständige Zerlegung zu berechnen, folgen diese Anpassungen einem explorationsgeleiteten Ansatz: Durch das Raten von Punkten werden relevante Konflikte erkannt und mithilfe der Technik der zylindrischen algebraischen Zerlegung verallgemeinert, was zu einer gröberen Zerlegung führt als die, die die ursprüngliche Methode berechnen würde. Obwohl die theoretische Komplexitätsgrenze nicht verbessert wird, lassen diese Methoden Raum für Heuristiken, die den Suchprozess leiten und die praktische Effizienz bei realen Problemen verbessern können.

Diese Arbeit befasst sich mit einer Vielzahl von solchen explorationsgeleiteten Algorithmen. Wir identifizieren gemeinsame Teile dieser Algorithmen, um ihre Darstellung zu vereinheitlichen und Ideen zwischen diesen Methoden zu übertragen, um ihre praktische Effizienz weiter zu verbessern. Außerdem übertragen wir den explorationsgeleiteten Ansatz auf die Quantorenelimination, indem wir diese Algorithmen entsprechend erweitern.

Bei der zylindrischen algebraischen Zerlegung wird der Raum implizit durch die Berechnung von Projektionspolynomen zerlegt. Die Menge der Projektionspolynome kann in explorationsgeleiteten Algorithmen reduziert werden, da wir nur eine auf einen bestimmten Konflikt zugeschnittene Zerlegung berechnen müssen. Da diese Projektionsmenge nach komplexeren Regeln bestimmt wird, formuliert diese Arbeit die Projektionsoperation als Beweissystem. Dies macht die Algorithmen erweiterbar für weitere Optimierungen, erhöht das Vertrauen in ihre Implementierung und definiert Schnittstellen für feinkörnige Heuristiken. Darüber hinaus ist es ein erster Schritt in Richtung von Zertifikaten, die die Korrektheit der Ausführung eines Solvers bezeugen.

Auf der Grundlage des Beweissystems trägt diese Arbeit zu einer Reihe von Verbesserungen der Projektionsoperation bei und führt zu neuen Begriffen, die zum Verständnis der Theorie hinter der zylindrischen algebraischen Zerlegung beitragen. Darüber hinaus untersuchen wir einige Heuristiken, die die praktische Effizienz von explorationsgeleiteten Algorithmen verbessern.

Die in dieser Arbeit vorgestellten Methoden werden anhand einer Implementierung im SMT-Solver SMT-RAT experimentell evaluiert.

Declaration of Authorship

I, Jasper Nalbach declare that this thesis and the work presented in it are my own and has been generated by me as the result of my own original research. I do solemnly swear that:

1. This work was done wholly or mainly while in candidature for the doctoral degree at this faculty and university;
2. Where any part of this thesis has previously been submitted for a degree or any other qualification at this university or any other institution, this has been clearly stated;
3. Where I have consulted the published work of others or myself, this is always clearly attributed;
4. Where I have quoted from the work of others or myself, the source is always given. This thesis is entirely my own work, with the exception of such quotations;
5. I have acknowledged all major sources of assistance;
6. Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself;
7. Parts of this work have been published before. A detailed list can be found in Section 1.2.

Aachen, May 6, 2026

Acknowledgements

First of all, I would like to thank my supervisor, Erika Ábrahám, for sparking my interest in this exciting and diverse topic. Her encouragement during my studies and the opportunity to work on this subject have been invaluable. She introduced me quite early to the scientific community, I still remember giving my Bachelor talk in front of dozens of international researchers. Secondly, I extend my gratitude to Gereon Kremer, who — as my advisor during my time as a student assistant — infected me with his enthusiasm. I appreciate all the discussions and insights he provided.

I would also like to thank my collaborators for the fruitful discussions: Chris Brown for taking the time to explain the details of the CAD projection and providing proofs; Matthew England for his insightful views on CAD; and James H. Davenport who served as living library, examined this thesis, and shared stories during lunch breaks. Thanks to Pascal Fontaine for introducing me to the mathematicians in Liège: Lucas Michel, from whom I learned a great deal during our visit with James and Matthew, and who — along with Pierre Mathonet and Naïm Zenaïdi — worked on theoretical results required for this thesis and rigorously checked our findings. Last but not least, I thank my colleagues: Valentin Promies for numerous discussions in the office, Lina Gerlach for feedback on parts of this thesis, and Emma Ahrens for brainstorming ideas related to a proof. Thanks to Lina Blume for comments that were incorporated in the revised version.

I am also grateful for the RTG UnRAVeL program. Thanks to my thesis students who contributed to the results presented here, particularly Philippe Specht, Philip Kroll, Philipp Bär, Paul Wagner, Carsten Perkampus, and Alexej Kolbin.

I appreciate the colleagues who accompanied me throughout my PhD journey: Rebecca, with whom I managed the teaching load during COVID times and enjoyed chats via Zoom; László, Nicolai, Lina, József, and Valentin who became friends through kicker matches and train travels to Romania for hikes, weddings, and chimney cakes. The conversations with various individuals from the i2 chair have also been enjoyable.

I thank my parents and grandparents for their support and instilling integrity within me. I thank my family and friends for their support during difficult times. Thanks also go out to the computer science faculty at RWTH Aachen University and fellow students for enriching my experience while studying a variety of theoretical subjects.

Finally, I am thankful for living in a society that enables researchers to pursue their interests. The freedom of science is not something we can take for granted — especially considering the rise of right-wing parties and oligarchies in Western democracies today. Thanks to everyone believing in humanity and the activists in times of climate change and human rights crisis.

Contents

I Introduction and Preliminaries

1	Introduction	3
1.1	Related Work	4
1.2	Contributions	14
2	Preliminaries	23
2.1	Fundamentals and Notation	23
2.2	Real Algebra	25
2.3	Real Projective Space	32
2.4	Mathematical Logic	33
2.5	Satisfiability and Satisfiability-Modulo-Theories	37
2.6	Real Algebraic Number Computations	45

II CAD-based Algorithms

3	CAD and Exploration-guided Algorithms	55
3.1	Introduction	56
3.2	Cylindrical Algebraic Decomposition	57
3.3	Cylindrical Algebraic Cells	84
3.4	Cylindrical Algebraic Covering	92
3.5	Non-uniform CAD	104
3.6	Computing Implicants	114
3.7	Variants of the CAIC Algorithm	117
4	Proof System for CAD	123
4.1	Introduction	124
4.2	McCallum's Projection	127
4.3	Equational Constraint Projection	146
4.4	Sign- and Order-invariance for Nullified Polynomials	148
4.5	Merging Adjacent Cells using Local Delineability	149
4.6	Weak Orderings and Closing Interval Bounds	160
4.7	Compound Cell Bounds	171
4.8	Projective Delineability	176
4.9	Handling Indexed Root Expressions	188

III Practical Aspects

5	Projection Heuristics for the Proof System	193
5.1	Basic Heuristics for Cells	194
5.2	Basic Heuristics for Coverings	200
5.3	Compound Bounds Heuristic for Cells and Coverings	202
5.4	Under-Approximation of Cells	202
5.5	Combinatorial Optimization Approach for Single Cells	206
6	Variants of Cylindrical Algebraic Covering	211
6.1	Incremental CAIC for CDCL(T)	211
6.2	Exploration Heuristic for CAIC	212
7	Implementation and Experiments	215
7.1	Implementation	215
7.2	Benchmarks and State of the Art	220
7.3	Evaluation of Algorithms	225
7.4	Evaluation of Proof System	231
7.5	Summary	235
8	Conclusion	237
8.1	Future Work	238

Bibliography	241
---------------------	------------

Part I

Introduction and Preliminaries

Chapter 1

Introduction

The first-order logic over the theory of the reals admits a highly expressive language. Questions of interest are (1) the satisfiability of its quantifier-free fragment, for example, a solution of

$$\bigwedge_{i=1}^2 (0 \leq x_i - 1 \wedge x_i + 1 \leq 5 \wedge 0 \leq y_i - 1 \wedge y_i + 1 \leq 5) \wedge (x_1 - x_2)^2 + (y_1 - y_2)^2 \geq 4$$

encodes the centres (x_1, y_1) and (x_2, y_2) of two (non-overlapping) unit circles that fit into a 5×5 square (and if this is not possible, the formula is not satisfiable); (2) the validity of sentences, for example,

$$\forall z. 2 \leq z \wedge z \leq 6 \rightarrow$$

$$\exists x_1. \exists x_2. \exists y_1. \exists y_2. \left(\bigwedge_{i=1}^2 (0 \leq x_i - 1 \wedge x_i + 1 \leq z \wedge 0 \leq y_i - 1 \wedge y_i + 1 \leq z) \right. \\ \left. \wedge (x_1 - x_2)^2 + (y_1 - y_2)^2 \geq 4 \right)$$

is valid if and only if for each square with side lengths between 2 and 6 there exist two (non-overlapping) unit circles; and (3) the elimination of quantifiers, that is, we ask for an equivalent formula not containing the quantified variables, for example, the solution formula of

$$\exists x_1. \exists x_2. \exists y_1. \exists y_2. \left(\bigwedge_{i=1}^2 (0 \leq x_i - 1 \wedge x_i + 1 \leq z \wedge 0 \leq y_i - 1 \wedge y_i + 1 \leq z) \right. \\ \left. \wedge (x_1 - x_2)^2 + (y_1 - y_2)^2 \geq 4 \right)$$

is $z \geq 2 + \sqrt{2}$ and its solutions describe all possible side lengths of a square such that two unit circles fit into it.

Due to its high expressiveness, the language has many potential applications. However, the *cylindrical algebraic decomposition (CAD)* method [Col75] introduced by Collins in the 1970's has doubly exponential worst case complexity in the number of variables (that is, if the input formula has 5 input variables, then the computation steps required by the CAD method are already in the order of magnitude of $2^{2^5} = 4\,294\,967\,296!$). Nevertheless, most algorithms implemented in state-of-the-art tools for answering these questions are based on the CAD method.

While for many practical applications, approximate solutions are sufficient and thus more efficient (but inexact) numerical methods are suitable, still, there is a need for symbolic algorithms whenever exact answers are required. Real quantifier elimination is applied in chemistry, geometry, life sciences, physics, planning [Stu17] and verification [Pla17]. For example, it has been used to disprove a conjecture in biology [RS21] where numeric methods failed to do so.

While the CAD method was the subject of active research in computer algebra since its discovery, in the nineties, *satisfiability* (SAT) and *satisfiability-modulo-theories* (SMT) solving emerged from the computer science community. The former is about checking the satisfiability of propositional logic formulas, and the latter about the satisfiability of the quantifier-free fragment of first-order logic modulo a given theory. While these solvers are quite general, their main motivation stems from verification [KS16, Chapter 12] — the typical example justifying the necessity of formal verification is the explosion of the Ariane 5 rocket in 1996 due to a software error.

SAT solvers use an *exploration-guided* approach, consisting of “guessing” variable assignments, propagating the consequences until a conflict occurs, and learning from these conflicts to guide the further solving process. This idea was transferred to the CAD in 2012 by Jovanović and de Moura [Jd12] for SMT solving. This work was highly influential, and its ideas lead to further algorithms like the *non-uniform CAD* (NuCAD) [Bro15] and the *cylindrical algebraic covering* (CAIC) [ÁDE⁺21]. The latter emerged from joint work of members from the computer algebra and satisfiability checking communities, facilitated by the *SC-square project* [ÁAB⁺16].

Since recently, SAT and SMT solvers deliver certificates proving the correctness of their answers, to increase trust into the results without the need of trusting the (huge) codebase of the solvers themselves. While there is already good support for a variety of theories, there is no feasible approach for the reals yet.

The work presented in this thesis is a child of SC-square. We give a formulation of the exploration-guided algorithms NLSAT, NuCAD, and CAIC in a unified way to show their common ideas, and how these ideas are transferred to quantifier elimination. We develop a modular framework for improving also the (computationally expensive) operations that stem from the CAD underpinning these algorithms. Our work also includes a step towards certificates for real algebra. Besides the theoretical contributions, we give an evaluation and comparison of the presented algorithms.

1.1 Related Work

We give a brief overview of important literature related to this thesis. We start with some milestones for real algebra. As this thesis has strong focus on the CAD method, we elaborate the origins and modern variants of the method. For a broader overview, we also include other (incomplete) methods that are used in practice. Finally, as the work reported in this thesis aims to improve the CAD for SMT solving, we examine the influential literature from that field as well.

1.1.1 Real Algebra

The word *algebra* stems from the Arabic word *Al-Jabr*, which is taken from the title of the influential book [Khwh20] of the Persian polymath Al-Khwarizmi published in 820

and means “reunion, resetting of broken parts”. While arithmetic studies operations like addition, subtraction, multiplication, and division on a given domain of numbers, algebra is its generalization to variables. *Real algebra* is the algebra over the *real numbers*, and is thus about solving systems of polynomial equalities and inequalities for real solutions.

Although algebra exists since the Babylonians, as any other branch of mathematics, it was expressed in natural language. The emergence of *mathematical logic* during the 19th century lead to a formalization of mathematics. Instead of saying “for each real number, its square is greater or equal to zero”, we can write the *statement* $\forall x. x^2 \geq 0$ which we interpret over the theory of the reals and decide upon its *truth*. This process took until the beginning of the 20th century, leading among others to the perhaps most influential work *Principia Mathematica* [WR10].

In the first half of the 20th century, remarkable results in *metamathematics* — the study about mathematics using mathematics — were achieved: Some issues could not be resolved to unify the entire of mathematics in a single foundation, which was the goal of a project by Hilbert known as *Hilbert’s program*. Together with Ackermann, he also posed the *Entscheidungsproblem* asking whether there exists a complete algorithm that can decide whether a given mathematical statement is universally valid (i.e. whether it is true or false) — a *decision procedure* for mathematics. Both problems were proven to be unsolvable: Hilbert’s program was proved to be impossible by Gödel’s *incompleteness theorems* [Göd31] in 1931 — it is impossible to formalize all mathematical true statements in a single formal language. The negative answer for the Entscheidungsproblem was later given by Church [Chu36a] and Turing [Tur37] in 1936.

Although there does not exist a universal decision procedure, there exist decision procedures for certain theories. Regarding algebra, the first result is on the decidability of *linear algebra* over the reals where we restrict the operations to addition and subtraction: The *Fourier-Motzkin elimination* was first proposed by Fourier [Fou25; Fou26] in 1826 and rediscovered by Motzkin [Mot36] in 1936. Presburger [Pre30] proved that *linear algebra* over the integers (also called *Presburger arithmetic*) is decidable in 1930. However, the general algebra over the integers was proven to be undecidable by Church [Chu36b] and Rosser [Ros36] in 1936. The question whether the algebra over the reals is decidable was finally answered by Tarski [Tar51] in the 1930’s (although published in 1951) and Seidenberg [Sei54] in 1954.

As an indicator for practical usability of a decision procedure, we look at its time and space complexity: How many computation steps respectively memory cells does the algorithm need to decide upon the truth of a given statement, depending on the length of the statement’s encoding? Tarski’s method has a non-elementary time complexity, that is, its complexity cannot be bounded by a finite term of the form $2^{2^{\dots^{2^n}}}$, and is thus impractical to solve interesting formulas. The first feasible algorithm was given by Collins [Col75] in the 1970s — called the *cylindrical algebraic decomposition* method — which has “only” doubly exponential time and space complexity in the number of variables. As the name says, it computes a decomposition of the real space where each part retains that a set of polynomials do not change their sign. Like Tarski’s method, it gives rise to a *quantifier elimination* procedure: Given a formula where variables either occur *universally* quantified (“for all real numbers x ”, $\forall x. \varphi(x)$), *existentially* quantified (“there is a real number x ”, $\exists x. \varphi(x)$), or as *parameter* (not quantified, $\varphi(x)$), it computes an equivalent formula that contains the parameters but no quantifiers; for

example, $\forall y. \exists z. xz = y$ is equivalent to $x \neq 0$.

The CAD was followed by a family of methods based on the *critical point theory*, which improve its theoretical complexity: Ben-Or, Kozen, and Reif [BKR84] give an algorithm in exponential space and parallel exponential time for quantifier elimination which influenced many of the following works, and was strengthened later in [MP90; Can93]. Grigor'ev and Vorobjov [GV88] give an algorithm for formulas with existentially quantified variables only (*existential theory of the reals* (ER)) in sub-exponential time in the number of variables. Canny [Can88] proved that ER can be solved in polynomial space (but his algorithm was not exponential time in the number of variables). Renegar [Ren88] gave an algorithm for ER that is in polynomial space and singly exponential in the number of variables. Grigor'ev [Gri88] gave an algorithm for formulas without parameters that has time complexity singly exponential in the number of variables, but doubly exponential in the number of quantifier alternations. This was extended for quantifier elimination by Heintz et al. [HRS90] and Renegar [Ren92c; Ren92b; Ren92a]. This was further improved by Basu, Pollack, and Roy [BPR96]. We refer to [BPR06] for an elaborate description of these methods. There is also recent research on critical point methods [LS21].

The work by Chen et al. [CMX⁺09] computes a decomposition of the complex space and translates this to a CAD of the real space. It can answer the same questions as Collins' CAD and has the same complexity.

The above algorithms prove upper bounds on the complexity of real quantifier elimination. Davenport and Heintz [DH88] and Weispfenning [Wei88] proved independently that they are already “close” to the theoretically possible, as real quantifier elimination requires in the worst case doubly exponential space and time in the number of quantifier alternations. Weispfenning's proof even applied to linear formulas. Brown and Davenport [BD07] showed that there are formulas for which the CAD is doubly exponential — independently of the order in which the variables are eliminated. (This result does not follow from [DH88], as in quantifier elimination, the variable ordering cannot be chosen freely.) Although there are theoretically more efficient methods than the CAD — particularly for deciding ER — it is (up to our knowledge) the most effective complete method that is implemented in practical tools for deciding questions about real algebra. Hong [Hon91] gives a comparison of Collins [Col75], Renegar [Ren92a], and Grigor'ev and Vorobjov [GV88], pointing out that the coefficients of the complexity bounds of the latter two are that high that their running time is worse than the CAD on inputs of reasonable size. Further, as pointed out in [BD07], the CAD has various advantages, such as generating simple quantifier elimination results or answering some questions directly (such as the Piano Mover's problem [SS83]).

1.1.2 Improvements of CAD

For a historical introduction of the CAD, we refer to the introduction of [Kre20], here we focus on relevant results.

The most important part of the CAD algorithm is the *projection* phase where a variable is eliminated from a set of polynomials, obtaining a set of polynomials with one variable less. This projection is iteratively applied until all variables are eliminated; the decomposition is then computed using the varieties of the projection polynomials (called the *lifting* phase). Thus, the choice of the projection polynomials (defined by a

projection operator) is crucial for the computational effort for the CAD for two reasons: Firstly, it is expensive to compute, and secondly, it has an impact on the coarseness of the resulting decomposition.

Collins [Col75] introduced the first projection operator (and published a corrected version of his proof in [Col98]). The number of polynomials in the projection was reduced in subsequent research (interestingly, the titles of all of these papers contain variants of the terms “improvement”, “projection operator” and “cylindrical algebraic decomposition”). Hong [Hon90] removed some redundancies in Collins’ projection. McCallum [McC85; McC88; McC98] invented a much lighter projection operator based on a new theory; however, it cannot be applied in certain cases. Brown [Bro01] improved McCallum’s operator, still leaving it incomplete. Lazard [Laz94] proposed a new projection operator which is similarly (but less) small as McCallum’s, but its proof contained a flaw, and was later proven by McCallum et al. [MH16; MPP19]. Brown and McCallum [BM20] improved Lazard’s operator to be equal to the Brown-McCallum operator on cases where it does not fail. However, Lazard’s projection has the disadvantage that the lifting phase of the algorithm needs to be adapted non-trivially in certain cases.

Furthermore, the CAD projection has been altered for special cases. If we are only interested in cells where some polynomials are equal to zero, then we can omit certain projection polynomials. McCallum introduced the concept of the *equational constraint* [McC99; McC01] projection for his projection operator, which was later transferred to Lazard’s theory by Nair [Nai21]. Work by England et al. [EBD15; ED16] deals with the implications on the complexity bounds when using equational constraints. Work by McCallum and Brown [MB09] proposes deeper changes of the CAD theory and algorithm for dealing with multiple equational constraints. Recently, the *geometric CAD* [Che25] was introduced, a generalization of the CAD with improvements particularly on inputs with multiple equational constraints.

1.1.3 Applications and Variants of CAD

The CAD often computes a decomposition which is finer than needed for answering certain questions. Over the years, a variety of modifications to the original CAD algorithm had been developed which eliminate redundant computations.

Arnon [Arn81; ACM84; Arn88] developed an algorithm that computes clusters of *adjacent* parts of the decomposition during its construction, potentially saving expensive computations which then are only computed for the clusters and not all parts. This was implemented in the **SAC-2** [Col85] computer algebra system. Collins and Hong [CH91] developed the *partial CAD*, reducing the amount of projection polynomials based on the sign conditions of the input polynomials, i.e. the resulting decomposition retains not the sign of the input polynomials, but the truth of the input formula. Hong [Hon92] developed a method for reducing the size of *solution formulas* for quantifier elimination. Hong’s work was implemented in **QEPCAD** and was further developed by Brown [Bro99], resulting in an implementation called **QEPCAD B** [Bro03]. Bradford et al. [BDE⁺13a] introduced the *truth-table invariant CAD* that incorporates the Boolean structure of the input to improve the applicability of the equational constraints. Strzeboński [Str14b] handles Boolean structure by a divide-and-conquer approach.

Besides the traditional algorithm, there are adaptations answering simpler questions or accepting restricted inputs. McCallum introduced an *open CAD* [McC93] algorithm,

which only computes the open cells of the CAD, sufficient for deciding on purely existentially quantified formulas where all relation symbols are strict. Strzeboński [Str00] simplified the projection operator for this algorithm. Seidl and Sturm [SS03] introduced a projection operator which reduced the number of projection polynomials based on additional assumptions (which are part of the output). Iwane et al. [IYA⁺13; IYA14] introduced several numeric checks that allow to reduce some projection and lifting steps. Another interesting task is to determine which cells of a CAD are *adjacent* to another (one definition says their union is connected). This was studied by Schwartz and Sharir [SS83] in the context of the *Piano Mover's* problem. A more efficient method was proposed by McCallum and Collins [MC02] and Strzeboński [Str17]. While Arnon [Arn81; ACM84; Arn88] was the first to exploit adjacencies for his clustering-based algorithm, this topic was not part of active research in the past decades; del Río and England [dE22b] argue that clustering could reduce the complexity of the lifting phase from doubly to singly exponential.

The CAD is implemented in the computer algebra systems **Redlog** [DS97b; SS03], **Mathematica** [Str00], and **Maple** [CMX⁺09; IYA⁺09], as well as in the theorem prover **Rocq** [Mah07] which is not actually verified.

In the past decade, CAD technology has been adapted for ER, driven by the rise of *SMT solvers* which we will introduce below. This problem is generally simpler, as if the formula holds, we only need to find a *satisfying* real point that evaluates it to true. The *incremental CAD* algorithm by Kremer and Ábrahám [KÁ20] implemented in **SMT-RAT** [CKJ⁺15] builds the CAD by adding each polynomial individually to the projection and lifting phase, potentially allowing to terminate the computation if a witness for the formula is found. Jovanović and de Moura [Jd12] introduced the *NLSAT* algorithm which was first implemented in **z3** and later in **yices2** [Dut14] and **SMT-RAT** [Kre20]. This algorithm introduced a novel *exploration-guided* technique: In the CAD, we compute the projection of the polynomials in the input formula and then check all cells during the lifting phase to find a satisfying point. NLSAT used ideas from *satisfiability checking* (more details below): It starts by guessing a (partial) real point, partially evaluates the input formula under this point, and propagates truth values of sub-formulas. In case the given point cannot satisfy the formula, we use CAD technology to generate a cell which generalizes this *conflict*. This generalization step uses a *local projection* which has been further refined by Brown and Košta [Bro13; BK15], as well as Strzeboński [Str14a]. Brown [Bro15; Bro17] used these ideas to compute a *non-uniform CAD* (*NuCAD*) which reduces projection polynomials at the cost of a different arrangement of the cells, implemented in the **Tarski** [VB18] computer algebra system. Ábrahám et al. [ÁDE⁺21] introduced the *cylindrical algebraic covering* (*CAIC*) method which is based on similar ideas to compute a covering of cells (instead of a decomposition) and still retaining the relevant information of the input formula; this has been implemented in the **SMT-RAT** and **cvc5** [KÁE⁺21] SMT solvers.

Besides the development on the CAD theory and CAD-based algorithms, recently another subject on the CAD has been become actively researched: The choice of the ordering in which the variables are eliminated has a significant impact on the effort required for the projection, as shown by [BD07]. Traditionally, these choices were made using hand-crafted heuristics [Bro04; DSS04; BDE⁺13a; WEB⁺14; LXZ⁺21; dE22a; EBC⁺14], and recently using machine learning [HEW⁺14; HEW⁺19; DE19; CZC20; FE20; PdE⁺24; FE24; dE24].

1.1.4 Computational Aspects of CAD

The CAD algorithm relies on many sub-algorithms for algebraic computations. Each of them has a crucial impact on the performance of the overall algorithm. For an overview of these algorithms, we refer to [Mis93].

The most crucial ingredient is the computation of a resultant of two polynomials. They can be computed as the determinant of the Sylvester matrix (or alternatively the *Bézout matrix*); there are algorithms for more efficient computations of determinants, see e.g. [EP05]. There also exist specialized algorithms, such as the *subresultant algorithm* or *subresultant pseudo-remainder sequences* due to Brown [BT71; Bro78] and Collins [Col67]; a currently widespread variant stems from [Duc00]. Further, there are *modular algorithms* (see e.g. [Eme10]) due to Collins [Col71] which compute resultants modulo some prime numbers and reconstruct the result using the Chinese Remainder Theorem. See also [Loo82b] for an overview about the different algorithms. Commercial tools such as **Maple** [Map24] and **Mathematica** [Wei24] implement various algorithms and select the appropriate one based on the input polynomial.

Throughout the CAD algorithm, we often need to compute the roots of univariate polynomials. This process is called *real root isolation*. The first method was the *continued fraction method* introduced by Vincent [Vin36]. Collins and Akritas [CA76] introduced the *bisection method*, which uses either *Descartes' rule of signs* [Des37] or *Sturm's theorem* [Stu29]. The problem of real root isolation is an active research area; both methods were improved in later work: Continued fractions in e.g. [ABS94; TE06], bisection in e.g. [RZ04; SM16; EMT08]. A comparison is given in [AS05]. For a broader overview, we refer to e.g. [SM16].

Another important ingredient is the representation of *real algebraic numbers*, which enables exact computations with polynomial varieties. The *interval representation* was first introduced in [Rub74]. A summary of algorithms for the elementary operations such as addition and multiplication can be found in [Loo82a]. Complexity results are provided in [SS83, Appendix B]. We further recommend [Mis93] as a reference and [Kre20, Section 2.5] for mathematical details. We note that there is an alternative representation, known as the *sign representation* which was introduced in [CR88], is based on *Thom's Lemma*, and was later improved in [MP90]. We also refer to [BPR06].

For CAD implementations, we need an algorithm for polynomial factorization of univariate and multivariate polynomials. For an introduction, we refer to a textbook like [DST93]. A common method is the *multivariate Hensel lifting*, developed by Yun [Yun74], improved by Wang [Wan78], and more recently by Monagan and Tuncer [MT16; MT18]. We refer to the latter and to [Kal03] for a more detailed bibliography.

Besides commercial computer algebra systems such as **Maple** or **Mathematica**, there are also open-source implementations of the computer algebra algorithms required for implementing the CAD method. In this work, we use **CoCoALib** [AB10] and **LibPoly** [JD17].

1.1.5 Other Methods for Real Algebra

Apart from the CAD method, there are other methods used in practice for fragments of real algebra. Due to the doubly exponential complexity of the CAD method, in particular SMT solvers use a combination of incomplete methods to supplement the CAD method.

Linear Real Algebra When restricting to *linear* real algebra, there exist various methods: *Gaussian elimination* for eliminating variables using equations, *Fourier-Motzkin elimination* [Fou26; Mot36] for eliminating variables using inequations (thus suitable for quantifier elimination), the *simplex method* by Dantzig [Dan51; Dan90] for linear programming (and thus deciding the linear version of ER), which was preceded by Kantorovich’s method for problems of special type [Kan60; vR85]. Note that the Fourier-Motzkin method has doubly exponential complexity, whereas the simplex method has singly exponential complexity. However, there is the polynomial-time *Ellipsoid method* [BGT81] which decides the linear version of ER and, to our knowledge, is not used in practice. The author was involved in the development of the *FMplex* method [NPÁ⁺23; PNÁ⁺25; PÁ24], which decides the linear version of ER and is suitable for quantifier elimination; in contrast to the Fourier-Motzkin elimination, it has a singly exponential worst-case complexity.

In the context of SMT solving, an adaptation of the simplex method for deciding ER is the state-of-the-art [Dd06; KBD13].

Linearization Especially in SMT solving, linearization techniques have been developed which encode an over- or under-approximation of the solution space of a real-algebraic formula using a linear real-algebraic formula. The *incremental linearization* [CGI⁺18] technique replaces all multiplications by variables and encodes lemmas about these multiplications incrementally guided by counter-examples. Similarly, the work in [BKK⁺19] splits the input formula in linear and non-linear parts and incrementally encodes lemmas about the non-linear monomials. The *subtropical satisfiability* [Stu15; FOS⁺17] method encodes a special class of solutions. The work in [MFK⁺16] used Handelman’s Theorem for the approximation of polynomials.

Virtual Term Substitution The *virtual (term) substitution* method is a quantifier elimination procedure which arose from Weispfenning’s complexity lower bound of real algebra [Wei88]. It was first introduced for linear formulas by Loos and Weispfenning [LW93] and later extended to quadratic [Wei97] and cubic [Wei94] formulas by Weispfenning. Although it is only applicable on formulas of low degree, in contrast to the CAD, it has only singly exponential complexity if the number of quantifier alternations is bounded. It has been extended to formulas of arbitrary (but bounded) degree by Košta and Sturm [KS15]. Note that due to the bounded degree, the method still remains incomplete. We refer to [Koš16] for details on the virtual substitution.

We note that there exist several variants [Stu99; Dol00] of the virtual substitution; in particular, there is an analogy [DW00] to CAD’s local projection. The method is implemented in various tools such as *Redlog* [DS97b], *QEPCAD B* [Bro03], and *z3* [Bj010]. Further, there are adaptations for deciding ER in *SMT-RAT* [CÁ11; CKJ⁺15] and *Redlog* [KKS14]. Oftentimes, the virtual substitution is used to complement the CAD on subformulas that are out of reach otherwise.

Gröbner Bases *Gröbner bases* are canonical representations of a polynomial ideal and are an important tool for reasoning about algebraic varieties, thus for solving systems of polynomial equations; it can be seen as a kind of “Gaussian elimination for polynomials”. The problem was unsolved for decades, and eventually Gröbner gave it to his student Buchberger [Buc65], who named the canonical representation after

his supervisor when he realized the significance of his task [Unc22]. The algorithm to compute Gröbner bases is referred to as *Buchberger’s algorithm*.

Despite its doubly exponential complexity (as the CAD), the use of Gröbner bases might be beneficial for computing with equations, as it does not compute projections but reasons about common roots of polynomials directly. It is thus used for preconditioning inputs to the CAD [BH91; WBD12; HED⁺16]. Note that although ER problems can be encoded as a system of polynomial equations, Gröbner bases are not directly applicable for solving ER, as it reasons about common complex roots and thus additional checks for real roots are necessary [JÁG⁺12].

In the context of quantifier elimination, *comprehensive Gröbner bases* [Wei92] are a stronger version of Gröbner bases which give a more “useful” description of the parameter space. Although it may be used for real quantifier elimination, it is most effective for eliminating equational constraints; thus a combination of comprehensive Gröbner bases and the CAD is implemented in CGSQE/SyNRAC [IYA14; FIS16].

Discriminant Varieties The *discriminant variety* [LR07] is a generalization of discriminants of univariate polynomials and describes the critical points of a set of polynomials. It may be used to eliminate several equalities from a set of equalities and inequalities, by eliminating some variables in a block.

Interval Constraint Propagation In the context of ER, given initial boxes (i.e. intervals for all variables), *interval constraint propagation (ICP)* [BG06] iteratively shrinks and splits these boxes such that all solutions contained in the initial boxes are contained in the refined boxes. It thus computes a safe over-approximation of the solution space within the input. The refinement steps make use of *interval arithmetic*: Instead of evaluating polynomials using real numbers, we evaluate on intervals; the result of such an evaluation is an interval which contains all possible values if the variables are assigned to values from their intervals.

ICP is successfully applied in SMT solving, and implemented in dReal [GKC13], iSAT [LSC⁺13], raSAT tung2016, and SMT-RAT [CKJ⁺15]. The method is incomplete, as it does not always terminate; the derived bounds however can be used e.g. for preconditioning inputs to the CAD [LSC⁺13].

Preprocessing and Further Incomplete Methods As mentioned, many of the above methods can be used to precondition inputs for the CAD. Although practical experience shows that simple transformations have a significant impact on the feasibility of the CAD, there do not exist widely applied techniques specifically for real-algebraic formulas. Still, there is work by Dolzmann and Sturm [DS97a], Brown [Bro09; Bro12], Brown and Strzeboński [BS10], Brown and Vale-Enriquez [BV20], and Iwane and Anai [IA17].

Recently, Li et al. [LXZ23] applied *local search* on ER which might find satisfying real points efficiently for ER problems. Cimatti et al. [CGL⁺22; Lip24] applied *unconstrained non-linear optimization* for finding approximative solutions (which could be exploited by complete methods).

We should note that there exist related numerical methods, i.e. the *Newton method* for finding (approximative) roots of polynomials. Such methods might be used to initialize exact methods, such as done in [Kre13]. Although not directly related to any method,

at this point we should refer to the field of *numerical algebraic geometry*; an overview is given e.g. in [HS17].

1.1.6 Satisfiability and Satisfiability-Modulo-Theories

As mentioned, modern CAD algorithms have been influenced by ideas from SAT and SMT solving. We thus give a brief overview on the important literature from SAT and SMT solving. For an extensive introduction, we refer to [Bie09] and [KS16] respectively.

SAT solving is about solving the *Boolean satisfiability problem (SAT)*, where we aim to find a solution for propositional logic formulas which are built from variables which can be assigned to either *true* or *false* and Boolean connectives like “and” (written as $\wedge$), “or” (written as $\vee$), or “not” (written as $\neg$), e.g. $a \wedge (b \vee c)$ states that a is true and at least one of b or c is true. It was the first problem to be proven to be NP-complete by Cook [Coo71] and Levin [Tra84], meaning that all problems that can be verified in polynomial time can be reduced to SAT. It is thus a powerful language; however, there is no algorithm known that solves SAT in polynomial time (which we would call an *efficient* algorithm in theoretical computer science); otherwise “P=NP” would hold, which contradicts the usual (yet unproven) assumption in computer science. Still, modern SAT solvers can solve practically relevant instances in reasonable amount of time.

The *Davis-Putnam* algorithm [DP60] is a complete algorithm for SAT which applies three rules alternatingly: eliminating variables, *propagation* of implications (i.e. if a and $\neg a \vee b$ hold, then b holds), and *resolution rule* for eliminating a variable from the formula. This was soon modified to the *Davis-Putnam-Loveland-Logemann (DPLL)* algorithm [DLL62], which replaced the resolution rule by a splitting (or *decision*) rule, which takes a variable and assigns a truth value to it: If the resulting formula turns out to be unsatisfiable, the other truth value is assigned; if the second formula is unsatisfiable as well, then the formula is unsatisfiable. This adaptation greatly reduces the memory necessary for storing the formulas and was motivated by the limited computing resources available at that time. In fact, the step from *inference* to *search* was an important shift. Borrowing ideas from CSP solving (a related problem), the **GRASP** SAT solver [MS96; MS99] combined the inference and search paradigm by extending DPLL for using the resolution rule to derive new clauses; these clauses encode which decisions are relevant for the unsatisfiability of the current formula, and thus can be used to revert multiple decisions (called *non-chronological backtracking*). **GRASP** also introduced the concept of *clause-learning*, that is, derived clauses are added to the input formula to facilitate future propagation and resolution (clauses are also deleted regularly according to some heuristics). The **chaff** SAT solver [MMZ⁺01] introduced a *conflict-driven* heuristic for making the decisions, which focusses on variables that occurred in recent conflicts. These ingredients build the base of modern *conflict-driven clause-learning (CDCL)* SAT solvers, which are developed up to today, improving data structures and heuristics. We note that there are other SAT solver architectures, which are less competitive than CDCL. Mentionable CDCL solvers are **MiniSAT** [ES04], **Glucose** [AS09], as well as the SAT solvers by Biere **PicoSAT** [Bie08], **Lingeling** [Bie10], **Kissat**, and **CaDiCaL** [BFF⁺20]. Annual SAT Competitions [FHI⁺21] take place since the early nineties, leading to standardization of input formats, benchmarks, and development of new ideas.

There were early ideas in the nineties to integrate DPLL for reasoning about richer logics, in particular modal logics [AG93; ACG00]. Soon after the first efficient SAT solvers

arrived, this was transferred to checking the satisfiability of quantifier-free first-order logic formulas modulo one or more theories, resulting in the field called *satisfiability-modulo-theories (SMT)*. Early SMT solvers were **ICS-SAT** [FOR⁺01], **CVC** [BDS02], **MathSAT** [ABC⁺02], **DLSAT** [MNA⁺02], and **VeriFun** [FJO⁺03]. These solvers were mostly built modular in order to support multiple theories, resulting in the *CDCL(T) framework* [Tin02; GHN⁺04; NO05; NOT06; Seb07] (sometimes called *DPLL(T)*) which builds the basis of most state-of-the-art SMT solvers. Theories supported by SMT solvers include *equality logic with uninterpreted functions* [Sho78], *bit vectors* [BDL98], *arrays* [BB08], and *linear arithmetic* (over the reals and integers) [Dd06]. Also, combinations of these theories are possible [NO79].

The CDCL(T) framework does perform a conflict-driven search on the Boolean structure of the problem, while a theory solver checks consistency of partial Boolean solutions with the underlying theory. For arithmetic theories, algorithms were developed which transfer the conflict-driven search to the theory as well [KTV09; MKS09; Cot10; JBd13]. Particularly successful was the *NLSAT* [Jd12] algorithm for non-linear real arithmetic, which is still state-of-the-art for this theory. Shortly after, the *model-constructing satisfiability calculus (MCSAT)* [dJ13; JBd13] was developed, which generalizes the ideas from NLSAT for other theories. Besides linear arithmetic, it was also extended to non-linear integer arithmetic [Jov17] and bit vectors [GJD20]. There also exist various extensions [BGS17; JD21] of MCSAT for related tasks.

In the last decade, checking quantified formulas has gained interest in the SMT community. The most common techniques are based on *quantifier instantiation* [dB07; NPR⁺21; BJ15; Gd09] (for an overview, we refer to the related work section in [NPR⁺21]), which are however incomplete for non-linear real arithmetic. The recent *QSMA* algorithm [BGV23; BGV25] is complete and successful also for non-linear real arithmetic. Another extension to SMT is *optimization-modulo-theories (OMT)* [NO06] which asks for an optimal solution of a formula w.r.t. an objective function; for non-linear arithmetic, an incomplete method based on incremental linearization exists [ST15].

For the task of solving non-linear real arithmetic, current SMT solvers are

- ▶ **z3** [dB08; BN24] which implements the NLSAT algorithm, quantifier instantiation techniques, and optimization techniques,
- ▶ **yices2** [Dut14] providing a modular MCSAT implementation with support for non-linear real arithmetic,
- ▶ **yicesQS** [BGV23] implementing the QSMA algorithm and builds on top of **yices2**,
- ▶ **cvc5** [BBB⁺22] which implements CDCL(T) with CAIC as theory solver, as well as quantifier instantiation techniques,
- ▶ **MathSAT5** [CGS⁺13] implements incremental linearization,
- ▶ **OptiMathSAT** [ST15] builds on top of **MathSAT5** to support optimization,
- ▶ **veriT+raSAT+Redlog** [BCD⁺09] uses the CDCL(T) framework, the ICP implementation from **raSAT** [TVO16] and the CAD implementation from **Redlog**, and
- ▶ **SMT-RAT** [CKJ⁺15; Kre20] which implements the CDCL(T) framework and combines simplex, ICP, VS, and CAD for theory solving, as well as the MCSAT framework

combining Fourier-Motzkin, ICP, VS and NLSAT. During the work on this thesis, **SMT-RAT** was extended with single cell construction, CAIC, and quantifier elimination.

Similarly to the SAT community, annual *SMT Competitions* [WCD⁺19] take place. The *SMT-LIB standard* [BFT17] provides a rich input language, and the *SMT-LIB benchmark library* [PSB⁺24] provides a collection of benchmarks.

Last but not least, we should mention that also tools designed for different purposes might be used to solve SMT problems: **Vampire** [KV13] is a higher-order logic theorem prover which can be applied also on arithmetic problems, **Ultimate Eliminator** [BDH⁺22] is a general quantifier elimination tool, **AProVE** [GBE⁺14] is a system for automated termination proofs and has support for non-linear integer arithmetic.

1.1.7 Proofs

Proof assistants like **Rocq** (formerly called **Coq**) [BC04], **Isabelle** [Pau94], and **Lean** [dU21] are designed for formalizing mathematical proofs into low-level steps such that they can be verified by a computer. These assistants consist of a small kernel that is compiled to machine code, and a large library written in a language of the proof assistant: the idea is that the kernel is kept small and maintainable in order to exclude bugs and increase trust; the kernel then verifies the library. Modern proof assistants allow writing algorithms in their language, find some proofs fully automatically, and provide an online repository for downloading existing formalizations.

The SAT and SMT solvers described above can automatically verify the correctness of logical formulas, theorems, and (software) systems. However, these tools may contain bugs themselves, mitigating the trust into their answers. For increasing the trust, there are two approaches: Implementing the tool directly in a proof assistant (a *verified implementation*) which can be trusted. However, SAT and SMT solvers are complex and huge software products, and their performance is crucial for many applications, making a verified implementation infeasible; an alternative is to let the solvers produce proofs for their answers which can then be verified by a proof assistant.

SAT solvers nowadays produce proofs [WHH14] which can be externally verified by another program. Similar developments are work-in-progress in SMT solvers [BFT11; SFB⁺21]; the **cvc5** solver supports proof generation for a wide range of theories [BRK⁺22] except non-linear arithmetic.

Regarding real algebra, [MH05] proposes a complete proof-producing method for quantifier elimination, but suffers from the complexity of the method. A verified implementation of the (incomplete) virtual substitution and the (relatively inefficient) method by Ben-Or, Kozen, and Reif [BKR84] are presented in [SCM⁺21; Kos23]. A verification of CAD seems intractable at the moment due to the nature of the proofs (an attempt has been done in [Mah07]). Thus, generating proofs for real algebra is a yet unsolved task.

1.2 Contributions

The work presented in this thesis contributes towards the understanding, improvement, and applicability of exploration-guided algorithms based on the CAD.

In Chapter 2, we introduce the notation and required mathematical foundations, as well as the common architecture of SAT and SMT solvers and algorithms for the algebraic operations used in CAD implementations.

Unified Presentation of Exploration-guided Algorithms for SMT and Quantifier Elimination

The exploration-guided algorithms NLSAT, NuCAD, CAIC, as well as the single cell construction evolved over time and use projection operations with subtle differences in their original publications. Further, their presentation is based on different notations.

In Chapter 3, we first introduce the CAD and its existing variants. We give a unified formulation of the exploration-guided algorithms, sharing projection primitives wherever possible in Sections 3.3 to 3.5. Central to the projection is our *levelwise single cell construction* introduced in [NÁS⁺24], which is based on prior work on the single cell construction [Bro13; BK15], and uses ideas from the projection used in CAIC [ÁDE⁺21]. Compared to the original version of CAIC [ÁDE⁺21], our work improves the projection operation, as presented in [NK25]. Further, using the same observations, we are able to improve the projection of NLSAT [Jd12]. As NuCAD [Bro15; Bro17] is directly based on the single cell construction, it benefits from all the following improvements to the projection as well.

All algorithms in Chapter 3 directly support Boolean structure in the input formula without the need of a SAT solver by the use of *implicants*, following our work in [KN23; NK25]. This enables us to extend the CAIC for quantifier elimination [NK25]. Although [Bro17] already extends NuCAD for quantifier elimination, transferring our approach from CAIC to NuCAD yields a presumably simpler extension for quantifier elimination, as presented in [NÁ25].

We conclude Chapter 3 by various optimizations to the algorithms, among other things, a variant of implicant calculation in Section 3.7.2 (as described in [NK25] based on ideas from the Bachelor’s thesis of Kroll [Kro23]), and an adaptation of the idea of open CAD to the CAIC algorithm in Section 3.7.3 (as described in [BNÁ⁺23]).

Formulation of the Projection Operation as Proof System

The exploration-guided algorithms admit a smaller projection compared to the CAD, as only specific conflicts need to be explained. Prior work on the CAD theory admits optimizations to the projection in some special cases (like the equational constraints projection [McC99], as already exploited in [BK15]). This motivates the formulation of the projection as a proof system as first introduced in our work in [NÁS⁺24] and of which we present an evolved version in Chapter 4. We define properties of polynomials to be maintained on the resulting decomposition, and rules that lead these properties back to sets of “smaller” properties. The proof system does not fix how the rules are to be instantiated, but delegates these choices to a heuristic. On the one hand, this cleanly separates the heuristic optimizations from the correctness of the algorithm. On the other hand, while the proof system is a coarse-grained framework for projections, the algorithms that use it may be seen as exploration strategies.

The proof system makes the complex projection easier to maintain, and thus increases the trustworthiness of its implementation. Further, by storing the trace of the instantiations of the proof rules, the algorithm may certify that its result is correct (if we trust the

correctness of the proof system). While this still leaves gaps in the trust chain, it is a first step towards certificates for real algebra.

Improvements to the Projection Operation

As the proof system is designed to structure the projection into well-defined properties, it enables fine-grained optimizations.

The proof system is based on McCallum’s projection operator which is incomplete, as it cannot deal with nullifications; in Section 4.4, we introduce a rule that fills this gap.

Further, in [NÁ24], we developed techniques to merge adjacent cells during single cell construction. In Sections 4.5 to 4.7, we embed this work into the proof system (yielding a different presentation than in the paper), further generalize the work, and elaborate on details that exceeded the scope of the paper.

The CAD projection consists of resultants, discriminants and (leading) coefficients. While for the former two, rigorous characterizations in the context of the CAD projection can be given, this was not possible for leading coefficients. During our work on [NÁS⁺24], this question arose. By intensive collaboration, we finally succeeded to answer this question in [MNM⁺24]. Its exploitation to single cell construction is presented in [NMÁ⁺25], and we give the embedding into the proof system in Section 4.8 (again, the presentation greatly differs from the one in the paper).

Heuristics for Projection and Exploration

Alongside the proof system, we present heuristics for its instantiation in Chapter 5, including the basic heuristics already presented in [NÁS⁺24], a heuristic based on combinatorial optimization from the Bachelor’s thesis of Perkampus [Per24], and approximation heuristics [PNÁ25] from the Master’s respectively Bachelor’s theses of Promies [Pro22] and Wagner [Wag23].

Regarding the CALC algorithm, in Chapter 6, we investigate variants influencing the order in which the search space is explored. This includes an incremental version of CALC for the CDCL(T) framework from the Bachelor’s thesis of Kroll [Kro20], and a variant that avoid computations involving non-rational numbers from the Bachelor’s thesis of Kolbin [Kol25].

Evaluation of Exploration-guided Algorithms

We provide an implementation and evaluation of all described techniques in Chapter 7. We evaluate state-of-the-art solvers, compare the exploration-guided algorithms, and investigate the gains of the optimizations to the projection.

Finally, we conclude the thesis in Chapter 8 and give an outlook on possible future research directions.

1.2.1 Relevant Publications

This thesis is based on prior publications co-authored by the author. Their presentation has been adapted for this thesis, put into context, and further illustrations and details were added. At the beginning of each chapter, we attribute its origins and outline the contributions beyond these publications.

In the following, we list the relevant publications for this thesis, and the contributions by the author. We emphasize that these publications are result of broad collaborations and were not possible otherwise. The ideas, questions, and feedback from all co-authors was invaluable for each publication.

Furthermore, the following list provides abbreviated references. Please refer to the bibliography at the end of this thesis for details.

- [NÁS⁺24] Jasper Nalbach, Erika Ábrahám, Philippe Specht, Christopher W. Brown, James H. Davenport, and Matthew England. “Levelwise construction of a single cylindrical algebraic cell”. In: *Journal of Symbolic Computation* (2024)

In this paper, we introduce the levelwise single cell construction, the proof system, and present a first experimental evaluation based on basic heuristics.

The author developed the proof system, wrote large parts of the paper, and conducted the experimental evaluation based on the implementation from the Master’s thesis of Specht [Spe20] (whereas this thesis used a modular re-implementation of the proof system). Particularly the technical proofs in the appendix were provided by the collaborators.

- [KN23] Gereon Kremer and Jasper Nalbach. “Cylindrical Algebraic Coverings for Quantifiers”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2022)* 2023

This short paper introduces the extension of the CAIC method for quantifier elimination.

The author co-developed the algorithms. Both authors contributed equally.

- [NK25] Jasper Nalbach and Gereon Kremer. “Extensions of the Cylindrical Algebraic Covering Method for Quantifiers”. In: *Mathematics in Computer Science (accepted, in print)* (2025)

This paper extends the work in [KN23] by elaborating details of the algorithms, solution formula generation, implicant calculation, a modification of CAIC that splits the formula based on the quantifier structure (inspired by ideas from [Kro23]), an embedding into the proof system presented in [NÁS⁺24], and an implementation as well as an experimental evaluation.

The author is the main contributor and wrote most parts of this paper that go beyond [KN23] as well as most parts of the implementation (some parts were implemented by Kroll in his Bachelor’s thesis [Kro23]).

- [NÁ25] Jasper Nalbach and Erika Ábrahám. “A Variant of Non-uniform Cylindrical Algebraic Decomposition for Real Quantifier Elimination”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025) (accepted, in print)* 2025

This paper extends the NuCAD by Brown [Bro15] to a complete algorithm and quantifier elimination by transferring ideas from [KN23; NK25] (as an alternative to [Bro17]).

The author is the main contributor and wrote most parts of the paper.

- [NÁ24] Jasper Nalbach and Erika Ábrahám. “Merging Adjacent Cells During Single Cell Construction”. In: *Computer Algebra in Scientific Computing (CASC 2024)* 2024

In this work, we detect when adjacent cells may be merged during single cell construction, based on the proof system from [NÁS⁺24].

The author is the main contributor and wrote most parts of the paper.

- [MNM⁺24] Lucas Michel, Jasper Nalbach, Pierre Mathonet, Naïm Zénaïdi, Christopher W. Brown, Erika Ábrahám, James H. Davenport, and Matthew England. “On Projective Delineability”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2024)* 2024

In this work, the theoretical foundations of *projective delineability* are introduced, a new notion that describes the role of the leading coefficient in the CAD projection.

The idea originates from Brown during our work on [NÁS⁺24], and proofs were provided by Michel, Mathonet, and Zénaïdi. The author contributed the motivating example and discussed how the statements are applied in single cell construction.

- [NMÁ⁺25] Jasper Nalbach, Lucas Michel, Erika Ábrahám, Christopher Brown, James H. Davenport, Matthew England, Pierre Mathonet, and Naïm Zénaïdi. “Projective Delineability for Single Cell Construction”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025)* (accepted, in print) 2025

In this paper, we apply the statements from [MNM⁺24] to reduce the amount of leading coefficients added to the projection during the single cell construction, based on the proof system from [NÁS⁺24].

The author is the main contributor and wrote the adaptation of the algorithms and its explanations, the implementation, and the evaluation. The technical proofs were provided by the collaborators.

- [BNÁ⁺23] Philipp Bär, Jasper Nalbach, Erika Ábrahám, and Christopher W. Brown. “Exploiting Strict Constraints in the Cylindrical Algebraic Covering”. In: *Satisfiability Modulo Theories (SMT 2023)* 2023

This work presents an adaptation of the CAIC algorithms for ideas of the open CAD.

The work is based on an idea by Brown. The author contributed major parts to the foundations and theoretical parts, while the implementation is based on the Bachelor’s thesis of Bär [Bär22].

- [PNÁ25] Valentin Promies, Jasper Nalbach, and Erika Ábrahám. “More is Less: Adding Polynomials for Faster Explanations in NLSAT”. in: *Automated Deduction (CADE-30)* 2025

This paper presents approximation techniques during single cell construction, based on the proof system in [NÁS⁺24]. The work is part of the Master’s respectively Bachelor’s theses of Promies [Pro22] and Wagner

[Wag23]. While the former is about finding linear bounds, the latter uses piece-wise linear functions which relies on the work in [NÁ24].

The author was an advisor for both theses projects, took part in discussions, and provided the basic implementation of the single cell construction as well as the underpinning theory for dealing with piece-wise linear functions during single cell construction.

1.2.2 Further Publications

The author was co-author of further publications that are not part of this thesis.

[Nal17] Jasper Nalbach. “Embedding the virtual substitution in the MCSAT framework”. Bachelor’s thesis. RWTH Aachen University, 2017

[ÁNK17] Erika Ábrahám, Jasper Nalbach, and Gereon Kremer. “Embedding the Virtual Substitution Method in the Model Constructing Satisfiability Calculus Framework (work-in-progress paper)”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2017)* 2017

The above two publications are about the adaptation of the virtual substitution [Wei97] method for MCSAT.

[NKÁ19] Jasper Nalbach, Gereon Kremer, and Erika Ábrahám. “On variable orderings in MCSAT for non-linear real arithmetic”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2019)* 2019

This paper reports on different variable orderings for NLSAT [Jd12], including an adaptation of the VSIDS heuristic for non-linear arithmetic.

[Nal20] Jasper Nalbach. “A novel adaption of the Simplex algorithm for linear real arithmetic”. Master’s Thesis. RWTH Aachen University, 2020

This thesis explored a novel embedding of the simplex algorithm [Dan51; Dan90] in CDCL(T). Although the results did not improve running times, they helped our understanding of linear arithmetic problems, leading to the following publications.

[NÁK21] Jasper Nalbach, Erika Ábrahám, and Gereon Kremer. “Extending the Fundamental Theorem of Linear Programming for Strict Inequalities”. In: *Symbolic and Algebraic Computation (ISSAC 2021)* 2021

We give a proof of a common approach to extend linear real arithmetic methods (like the simplex method) for dealing with strict inequalities, which contributes to the understanding of the algorithmic changes.

[NPÁ⁺23] Jasper Nalbach, Valentin Promies, Erika Ábrahám, and Paul Kobialka. “FMplex: A Novel Method for Solving Linear Real Arithmetic Problems”. In: *Games, Automata, Logics, and Formal Verification (GandALF 2023)* 2023

[PNÁ⁺25] Valentin Promies, Jasper Nalbach, Erika Ábrahám, and Paul Kobialka. “FMplex: Exploring a Bridge between Fourier-Motzkin and Simplex”. In: *Logical Methods in Computer Science* (2025)

In the above two works, we introduce *FMplex* — a modification of Fourier-Motzkin [Fou25; Fou26; Mot36] for satisfiability checking that is singly exponential by dividing the input polytope into multiple ones, and show its relation to the simplex method. It was first implemented in the Master’s thesis of Kobialka [Kob21]. It led to a competitive singly exponential polyhedral projection algorithm by Promies [PÁ24].

- [NÁ23] Jasper Nalbach and Erika Ábrahám. “Subtropical Satisfiability for SMT Solving”. In: *NASA Formal Methods (NFM 2023)* 2023
- We present an alternative embedding of the subtropical satisfiability [Stu15; FOS⁺17] method into SMT solvers which does not rely on CDCL(T), and thus may be integrated also with non-CDCL(T)-based solvers.
- [ÁNP23] Erika Ábrahám, Jasper Nalbach, and Valentin Promies. “Automated Exercise Generation for Satisfiability Checking”. In: *Formal Methods Teaching (FMTea 2023)* 2023
- This work reports on the automated generation of exercises in lectures on SAT and SMT solving.

1.2.3 Advised Theses

The author advised several thesis projects related to linear and non-linear real arithmetic. Some of them are already mentioned above.

- [Spe20] Philippe Specht. “A level-wise variant of single cell construction in cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2020
- [Kro20] Philip Kroll. “Efficient data structures for cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2020
- [Bär22] Philipp Bär. “Exploiting strict constraints in the computations of cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2022
- [Kro23] Philip Kroll. “Implementation of cylindrical algebraic coverings for quantifier elimination”. Master’s Thesis. RWTH Aachen University, 2023
- [Kol25] Alexej Kolbin. “Traversal heuristics for cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2025
- [Pro22] Valentin Promies. “Underapproximating cell bounds in MCSAT using low-degree polynomials”. Master’s Thesis. RWTH Aachen University, 2022
- [Wag23] Paul Tristan Wagner. “Piecewise linear under-approximation of cell boundaries in MCSAT”. Bachelor’s thesis. RWTH Aachen University, 2023
- [Per24] Carsten Peter Perkampus. “Speed up single cell construction via combinatorial optimization”. Bachelor’s thesis. RWTH Aachen University, 2024

- [Hei20] Daniel Alexander Heinen. “Purging spurious samples in the cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2020
- [Cov23] Kristian Cović. “Heuristical variable ordering in the cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2023
- [Spe23] Philippe Specht. “Improving incremental linearization for satisfiability modulo non-linear real arithmetic checking”. Master’s Thesis. RWTH Aachen University, 2023
- [Pon21] André Leon Poncelet. “Conflict generalization for quadratic real-arithmetic constraints in SMT-RAT”. Bachelor’s thesis. RWTH Aachen University, 2021
- [Har22] Max Harder. “Generating coverings using virtual substitution for explanations in mcSAT”. Bachelor’s thesis. RWTH Aachen University, 2022
- [Ali20] Fabian Alieff. “Simplex heuristics in SMT solving”. Bachelor’s thesis. RWTH Aachen University, 2020
- [Kob21] Paul Kobialka. “Connecting simplex and Fourier-Motzkin into a novel quantifier elimination method for linear real algebra”. Master’s Thesis. RWTH Aachen University, 2021
- [Hil22] Kai Hilgers. “An FMplex-inspired simplex heuristics”. Bachelor’s thesis. RWTH Aachen University, 2022
- [Ste22] Svenja Stein. “An incremental adaption of the FMplex method for solving linear real algebraic formulas”. Bachelor’s thesis. RWTH Aachen University, 2022

A good portion of the above theses are related to the single cell construction and cylindrical algebraic coverings algorithms, as [Spe20; Hei20; Kro20; Bär22; Kro23; Kol25], or focus on heuristics for single cell construction using the proof system, namely [Pro22; Wag23; Per24]. Further projects explored ideas [Hei20] in the context of CAD or implemented and extended state-of-the-art approaches [Cov23; Spe23] for non-linear real arithmetic. Two theses [Pon21; Har22] transferred ideas from the covering algorithm to virtual substitution in MCSAT, continuing the work in [Nal17; ÁNK17]. Another chunk is related to the simplex and FMplex algorithms [Ali20; Kob21; Hil22; Ste22].

Chapter 2

Preliminaries

We introduce the notation used throughout this work as well as the necessary foundations of real algebra and SMT solving.

Origins Parts of the notation were already developed in [NÁS⁺24]. For detailed attributions, we refer to the references in the corresponding sections.

2.1 Fundamentals and Notation

2.1.1 Numbers

Let $\mathbb{N}$ denote the set of all *natural* numbers (naturals) including 0, $\mathbb{N}_{>0} = \mathbb{N} \setminus \{0\}$ denote the set of positive naturals, $\mathbb{Z}$ denote the set of *integer* numbers (integers), $\mathbb{Q}$ denote the set of all *rational* numbers (rationals), and $\mathbb{R}$ denote the set of all *real* numbers (reals).

For $i, j \in \mathbb{N}$ with $i \leq j$, we define the set of integers $[i..j] = \{i, \dots, j\}$, and $[i] = [1..i]$.

The *sign* of $r \in \mathbb{R}$, denoted $\text{sgn}(r)$, is defined to be 1 if $r > 0$, -1 if $r < 0$, and 0 otherwise.

2.1.2 Sets

We denote the *empty set* using $\emptyset$. Let A and B be *sets*, then $A \cup B$ denotes their *union*, $A \cap B$ denotes their *intersection*, $A \setminus B$ the *set difference*, $A \times B$ denotes their *Cartesian product*, $A^0 = \{()\}$, and $A^i = A^{i-1} \times A$ for $i \in \mathbb{N}_{>0}$. Furthermore, $A \subseteq B$ ($A \subset B$) denotes that B (strictly) includes A . We denote the *power set* of A as 2^A .

Let A be a set, $i, j \in \mathbb{N}_{>0}$, $j \leq i$. We call $r = (r_1, \dots, r_i) \in A^i$ a *vector*, and denote by r_j the j -th *component* of r , by $r_{[j]}$ the vector $(r_1, \dots, r_j)$, and by $r_{[0]}$ the vector $()$. Given $i, j \in \mathbb{N}_{>0}$ with $j < i$, we define the *projection* of a set $R \subseteq A^i$ onto A^j by $R \downarrow_{[j]} = \{(r_1, \dots, r_j) \mid (r_1, \dots, r_i) \in R\}$ and call A^i an *extension* of A^j . Let $R \subseteq A^i$, then we denote by $\text{Ext}(R)$ the set of all $R' \subseteq A^{i+1}$ such that $R' \downarrow_{[i]} = R$.

2.1.3 Relations

Let A be a set. The relation $\preceq \subseteq A^2$ is a (*partial*) *order* on A if it is reflexive ($a \preceq a$ for all $a \in A$), antisymmetric (if $a \preceq b$ and $b \preceq a$ then $a = b$ for all $a, b \in A$), and transitive (if $a \preceq b$ and $b \preceq c$ then $a \preceq c$ for all $a, b, c \in A$). The relation $\prec \subseteq A^2$ is a

strict (partial) order on A if it is irreflexive (not $a \prec a$ for all $a \in A$), asymmetric (if $a \prec b$ then not $b \prec a$ for all $a, b \in A$), and transitive. The relation $\preceq \subseteq A^2$ is called a *total order on A* if it is a partial order on A and strongly connected ($a \preceq b$ or $b \preceq a$ for all $a, b \in A$). The relation $\prec \subseteq A^2$ is called a *strict total order on A* if it is a strict partial order on A and connected (if $a \neq b$ then $a \prec b$ or $b \prec a$ for all $a, b \in A$). The relation $\approx \subseteq A^2$ is an *equivalence relation on A* if it is reflexive, symmetric (if $a \approx b$ then $b \approx a$ for all $a, b \in A$), and transitive. We denote the *transitive closure of a relation $\sim$* as $\sim^t$; the *reflexive and transitive closure of a relation $\sim$* as $\sim^{rt}$; and the *reflexive, transitive, and symmetric closure of a relation $\sim$* as $\sim^{rts}$. The *domain* of a relation $\sim \subseteq A^2$ is denoted by $\text{dom}(\sim) = \{a, b \mid a, b \in \sim\}$.

A *cyclic ordering* $X \subset A^3$ on a set A is a ternary relation that is cyclic ($(a, b, c) \in X \Rightarrow (b, c, a) \in X$), asymmetric ($(a, b, c) \in X \Rightarrow (c, b, a) \notin X$), transitive ($(a, b, c) \in X \wedge (a, c, d) \in X \Rightarrow (a, b, d) \in X$) and connected ($a \neq b, b \neq c, a \neq c \Rightarrow (a, b, c) \in X \vee (c, b, a) \in X$).

2.1.4 Functions

Let $f : D \rightarrow E$ be a *function*, then the *domain* D of f is denoted by $\text{dom}(f)$, and the *restriction of f to $A \subseteq D$* is denoted by $f|_A$ (i.e. $\text{dom}(f|_A) = A$). Let $f : D \hookrightarrow E$ be a *partial function*, then the domain $\text{dom}(f) \subseteq D$ of f is the maximal set such that $f : \text{dom}(f) \rightarrow E$. Let $f, g : D \rightarrow E$ and let $<$ be a total order on E . We write $f < g$ if $f(d) < g(d)$ for all $d \in D$ and $f \leq g$ if $f(d) \leq g(d)$ for all $d \in D$. Let $D' \subseteq D$, then the *graph of f on D'* is the set $\{(d, e) \mid d \in D', f(d) = e\}$.

2.1.5 Tuples

For a *tuple* $t = (a, b, c, \dots)$ we denote by $t.a, t.b, t.c, \dots$ the corresponding tuple entries. Assume $k \in \mathbb{N}_{>0}$, some set D , and an order $<$ on D . The *lexicographical ordering* $<$ on D^k is defined such that for all $t, t' \in D^k$, $t < t'$ if and only if there is an $i \in [k]$ such that $t_i < t'_i$ and $t_j = t'_j$ for all $j \in [i - 1]$.

2.1.6 Real Space

Let $i \in \mathbb{N}_{>0}$. The space $\mathbb{R}^i$ is called *real space*. For $r \in \mathbb{R}^i$ we define the *Euclidean norm* as $\|r\| = \sqrt{r_1^2 + \dots + r_i^2}$. Given $\varepsilon \in \mathbb{R}$, $\varepsilon > 0$ and $r \in \mathbb{R}^i$, the ε -*ball around r* is the set $B_\varepsilon(r) := \{r' \in \mathbb{R}^i \mid \|r - r'\| \leq \varepsilon\}$. A set $R \subseteq \mathbb{R}^i$ is called *open* if for all $r \in R$ there is an $\varepsilon \in \mathbb{R}$, $\varepsilon > 0$ such that $B_\varepsilon(r) \subseteq R$. R is called *closed* if $\mathbb{R}^i \setminus R$ is open. The *closure* $\text{cl}(R)$ of R is the smallest closed superset of R , its *interior* $\text{int}(R)$ is its largest open subset, and its *boundary* $\text{bound}(R)$ is its closure without its interior. R is called *(path) connected* if for any $r, r' \in R$ there is a path from r to r' (that is a continuous function $f : [0, 1] \rightarrow R$ such that $f(0) = r$ and $f(1) = r'$). R is called *simply connected* if it is connected and any loop in R (that is a continuous function $f : [0, 1] \rightarrow R$ such that $f(0) = f(1)$) can be continuously contracted to a point (we omit a more formal definition here; intuitively, R has no “holes”). A *cell* is a non-empty connected subset of $\mathbb{R}^i$ for some $i \in \mathbb{N}_{>0}$.¹ For a point $r \in \mathbb{R}^i$, a *neighbourhood of r* is an open set $N \subseteq \mathbb{R}^i$

¹Note that in topology, a cell is a subset of $\mathbb{R}^i$ for some $i \in \mathbb{N}_{>0}$ that is homeomorphic to an open sphere. We use the above definition as this is the one used in CAD literature. Indeed, the difference in the definition matters e.g. for the technique described in Section 4.6.

such that $r \in N$. A function $f : R \rightarrow \mathbb{R}$ is *continuous on R* if for every $r \in R$ and for any neighbourhood $N' \subseteq \mathbb{R}$ of $f(r)$ there exists a neighbourhood $N \subseteq R$ of r such that for each $r' \in N$ it holds $f(r') \in N'$.

For $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$ a connected set, and $f, g : R \rightarrow \mathbb{R}$ continuous functions, we define the sets $R \times (f, g) = \{(r, r_{i+1}) \mid r \in R, f(r) < r_{i+1} < g(r)\}$, $R \times [f, g] = \{(r, r_{i+1}) \mid r \in R, f(r) \leq r_{i+1} \leq g(r)\}$, $R \times (-\infty, g) = \{(r, r_{i+1}) \mid r \in R, r_{i+1} < g(r)\}$, $R \times (-\infty, \infty) = R \times \mathbb{R}$, and analogously we define $R \times (f, \infty)$, $R \times (-\infty, g]$, $R \times [f, \infty)$, $R \times [f, g]$, and $R \times (f, g]$. We set $R \times f = R \times [f, f]$. Note that if $f \leq g$ (on R), then these sets are connected sets (as a continuous image of a connected set is connected).

2.2 Real Algebra

We introduce some common definitions and terminology related to polynomials. For further reading, we refer to [CLO15; BPR96].

2.2.1 Polynomials

A *polynomial* is built from a set of *variables* and coefficients from a domain K (which is required to be a commutative ring) using addition and multiplication. In this work, we mostly choose $K = \mathbb{Q}$.

Definition 2.1 (Polynomial)

Let X be a finite set of variables, and K be a commutative ring.

- A *monomial m in X* is a product $\prod_{x \in X} x^{e_x}$ where $e_x \in \mathbb{N}$. The *total degree of m* is $\text{tdeg}(m) = \sum_{x \in X} e_x$. The *degree of m* is $\deg(m) = \max_{x \in X} e_x$.
- A *term in X over K* is a product $c \cdot m$ where $c \in K$ is a *coefficient* and m is a monomial in X .
- A *polynomial p in X over K* is a sum $\sum_{i=1}^k c_i \cdot m_i$ where $k \in \mathbb{N}_{>0}$ and $c_i \cdot m_i$ are terms in X over K such that their monomials are pairwise different. The *total degree of p* is $\text{tdeg}(p) = \max\{\text{tdeg}(m_i) \mid i \in [k], c_i \neq 0\}$ (and $\text{tdeg}(p) = 0$ if $p = 0$). The *degree of p* is $\deg(p) = \max\{\deg(m_i) \mid i \in [k], c_i \neq 0\}$ (and $\deg(p) = 0$ if $p = 0$). Given an ordering on all monomials, the *leading term of p w.r.t. the monomial ordering* is the term with the largest monomial w.r.t. that ordering and non-zero coefficient. The set $K[X]$ contains all polynomials in X over K (the *polynomial ring in X*).
- A polynomial is *linear* if its total degree is at most 1, and *non-linear* otherwise. A polynomial is *univariate* if it is a polynomial in a single variable x , *constant* if it is an element of K , and *multivariate* otherwise.
- The *leading coefficient* of a univariate polynomial p is the coefficient of the non-zero term of highest degree in p . A univariate polynomial is *monic* if its leading coefficient is equal to 1.

We can view multivariate polynomials as univariate polynomials over a polynomial ring:

Definition 2.2 (Univariate Representation)

Let X be a finite set of variables, K be a commutative ring, $p \in K[X]$, and $x \in X$.

- The *univariate representation of p w.r.t. x* is the polynomial $c_0 \cdot x^0 + \dots + c_k \cdot x^k$ in x with coefficients $c_0, \dots, c_k \in K[X \setminus \{x\}]$ such that $c_k \neq 0$. We denote the set of such polynomials as $K[X \setminus \{x\}][x]$.
- The *coefficients of p w.r.t. x* are $\text{coeff}_x(p) = \{c_i \mid i \in [0..k], c_i \neq 0\}$. The *leading coefficient of p w.r.t. x* is $\text{ldcf}_x(p) = c_k$. The *trailing coefficient of p w.r.t. x* is c_0 .
- The *degree of p in x* is $\deg_x(p) = k$.

We recall some well-known properties of polynomials from $\mathbb{Q}[X]$.

Definition 2.3 (Factors)

Let X be a finite set of variables.

- A polynomial $p \in \mathbb{Q}[X] \setminus \{0\}$ is *irreducible* if for all polynomials $q, q' \in \mathbb{Q}[X]$ such that $p = q \cdot q'$ it holds that either $q \in \mathbb{Q}$ or $q' \in \mathbb{Q}$.
- A polynomial $q \in \mathbb{Q}[X]$ is a *factor* of $p \in \mathbb{Q}[X] \setminus \{0\}$ if there exists a polynomial $q' \in \mathbb{Q}[X]$ such that $p = q \cdot q'$. Assuming any ordering on monomials, $\text{factors}(p)$ is the set of all irreducible factors $q \notin \mathbb{Q}$ of p whose leading term with respect to the monomial ordering has coefficient one.
- A polynomial $p \in \mathbb{Q}[X] \setminus \{0\}$ is *square-free* if there does not exist any $q \in \mathbb{Q}[X] \setminus \mathbb{Q}$ such that q^2 is a factor of p .
- Two polynomials $p, q \in \mathbb{Q}[X] \setminus \{0\}$ are called *coprime* if there does not exist any $r \in \mathbb{Q}[X] \setminus \mathbb{Q}$ such that $r \in \text{factors}(p) \cap \text{factors}(q)$.

Note that an irreducible polynomial is always square-free; and that two different irreducible polynomials whose leading term with respect to the monomial ordering has coefficient one are always coprime, in particular for $p \in \mathbb{Q}[X] \setminus \{0\}$ and any $q, q' \in \text{factors}(p)$, q and q' are coprime if and only if $q \neq q'$.

For studying the geometric properties, it is useful to fix a set of variables $x_1, \dots, x_n$ for some $n \in \mathbb{N}_{>0}$ and an ordering $x_1 \prec x_2 \prec \dots \prec x_n$. In particular, this allows to associate a point $(r_1, \dots, r_n) \in \mathbb{R}^n$ with an *assignment* of the variables $x_1, \dots, x_n$.

Definition 2.4 (Level)

A polynomial $p \in \mathbb{Q}[x_1, \dots, x_n]$ is of level i (denoted as $\text{level}(p) = i$) if x_i is the largest variable w.r.t. $\prec$ such that $\deg_{x_i}(p) > 0$.

For $p \in \mathbb{Q}[x_1, \dots, x_n]$, we write $p(x_1, \dots, x_i)$ to indicate that i is the largest index from $[0..n]$ with $\deg_{x_i}(p) > 0$. For $j \in [i]$ and $r = (r_1, \dots, r_j) \in \mathbb{R}^j$ we write $p(r, x_{j+1}, \dots, x_i)$ for the polynomial p after substituting $r_1, \dots, r_j$ for $x_1, \dots, x_j$ in p and indicating the remaining variables in p .

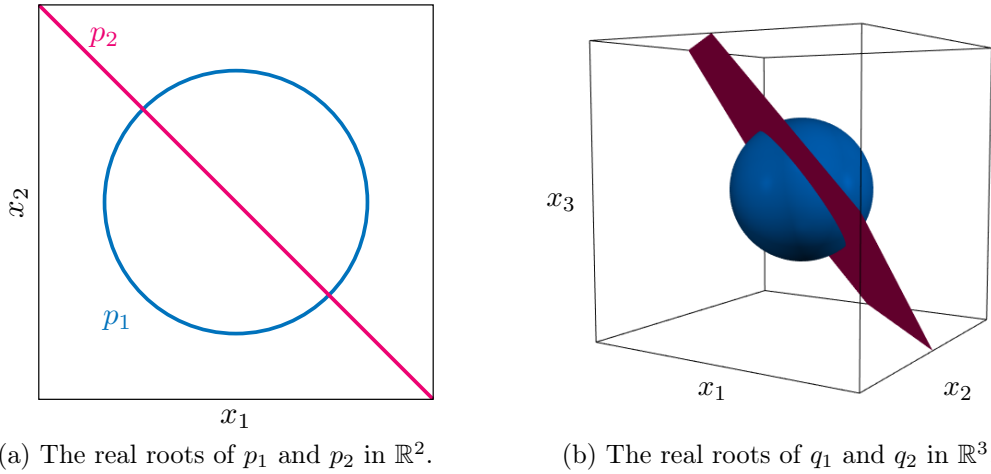


Figure 2.1: Varieties of the polynomials from Example 2.1.

Definition 2.5 (Polynomial Function and Real Roots)

Let $p \in \mathbb{Q}[x_1, \dots, x_n]$.

- The *polynomial function* associated to p is $p : \mathbb{R}^n \rightarrow \mathbb{R}, r \mapsto p(r)$.
- Let $r \in \mathbb{R}^n$. If $p(r) = 0$, then r is a (real) *root* of p , and p *vanishes* at r . The (real) *variety* of p is the set of all real roots of p and is denoted as $\text{realRoots}(p)$.
- Let $i = \text{level}(p)$, and $r \in \mathbb{R}^{i-1}$. Then p is *nullified* on r if $p(r, x_i) = 0$.

Univariate polynomials $p \in \mathbb{Q}[x] \setminus \{0\}$ have finitely many real roots bounded by $|\text{realRoots}(p)| \leq \deg_x(p)$; but this property does not necessarily hold for multivariate polynomials.

Example 2.1

Consider the polynomials $p_1 = x_1^2 + x_2^2 - 1$ and $p_2 = x_1 + x_2$ whose varieties are depicted in Figure 2.1a and the polynomials $q_1 = x_1^2 + x_2^2 + x_3^2 - 1$ and $q_2 = x_1 + x_2 + x_3$ whose varieties are depicted in Figure 2.1b.

For univariate polynomials, we define the multiplicity of its roots:

Definition 2.6 (Multiplicity)

Let x be a variable, $p \in \mathbb{Q}[x] \setminus \{0\}$, and $r \in \text{realRoots}(p)$. The root r of p is of *multiplicity* k if there is a $q \in \mathbb{R}[x]$ such that $q(r) \neq 0$ and $p(x) = (x - r)^k q(x)$.

Let $p \in \mathbb{Q}[x_1, \dots, x_n]$, $\alpha \in \mathbb{N}^n$, and $k = \sum_{i \in [n]} \alpha_i$. Then $\frac{\partial^k}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p$ denotes the *partial derivative* of p of order α ; its *total order* is k . This allows us to define the order of multivariate polynomials:

Definition 2.7 (Order)

Let $p \in \mathbb{Q}[x_1, \dots, x_n]$, and $r \in \mathbb{R}^n$.

The *order of p at r* is

$$\text{ord}_r(p) = \min \left(\left\{ k \in \mathbb{N} \mid \exists \alpha \in \mathbb{N}^n. \left(\sum_{i \in [n]} \alpha_i = k \right) \wedge \left(\frac{\partial^k}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p \right)(r) \neq 0 \right\} \cup \{\infty\} \right).$$

Finally, we define some *invariance properties* of polynomials on a given cell:

A polynomial $p \in \mathbb{Q}[x_1, \dots, x_n]$ is *sign-invariant* on a set $R \subseteq \mathbb{R}^n$ if $\text{sgn}(p(r)) = \text{sgn}(p(r'))$ for all $r, r' \in R$. A set of polynomials $P \subseteq \mathbb{Q}[x_1, \dots, x_n]$ is *sign-invariant* on $R \subseteq \mathbb{R}^n$ if all $p \in P$ are sign-invariant on R . Similarly, p is *order-invariant* on R if $\text{ord}_r(p) = \text{ord}_{r'}(p)$ for all $r, r' \in R$. Note that if p has no root in R , then $\text{ord}_r(p) = 0$ for all $r \in R$ and p is trivially order-invariant on R . By the intermediate value theorem, if p is order-invariant on some connected set R , then p is also sign-invariant on R .

A polynomial $p \in \mathbb{Q}[x_1, \dots, x_{n-1}][x_n]$ is *degree-invariant* on a set $R \subseteq \mathbb{R}^{n-1}$ if $\deg_{x_n}(p(r)) = \deg_{x_n}(p(r'))$ for all $r, r' \in R$.

2.2.2 Algebraic Closure and Real Algebraic Numbers

There are irrational numbers in $\mathbb{R}$ which do not appear as solutions of polynomial equations over $\mathbb{Q}$, such as π . In general, such irrational numbers cannot be finitely represented, thus, reasoning with them on a computer is impossible. However, the set of *real algebraic numbers* ($\mathbb{R} \cap \bar{\mathbb{Q}}$) suffices for representing such solutions, and can be finitely represented on a computer, as they are a subset of the set of *algebraic numbers*

$$\bar{\mathbb{Q}} = \{r \in \mathbb{C} \mid \exists p \in \mathbb{Q}[x] \setminus \{0\}. p(r) = 0\}$$

and thus can be represented using a polynomial and an index of its (real) root.

Definition 2.8 (Algebraic Closure)

Let x be a variable, and F be a field.

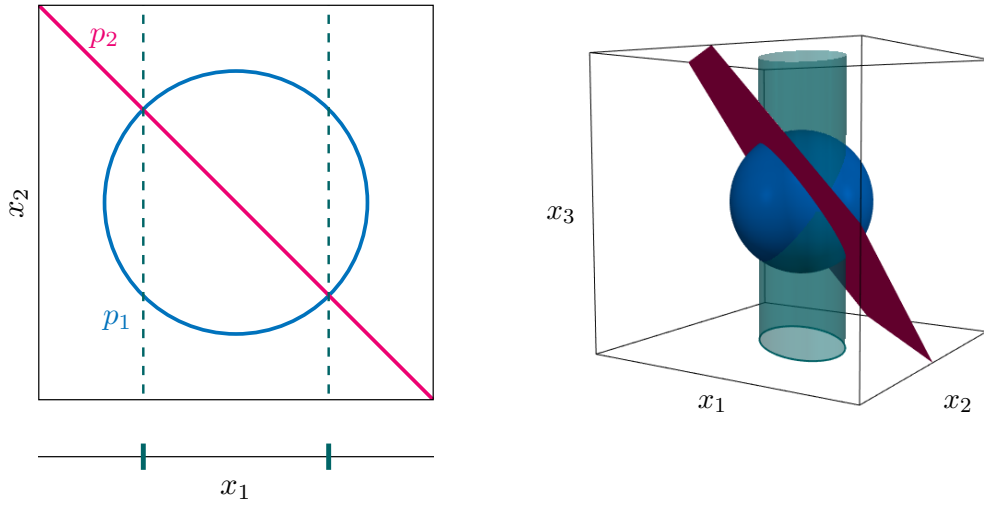
- F is *algebraically closed* if every non-constant polynomial in $F[x]$ has a root in F .
- The *algebraic closure* $\bar{F}$ of a field F is an algebraically closed field such that $\bar{F} \supseteq F$ and for each $a \in \bar{F}$ there is a $p \in F[x] \setminus \{0\}$ such that $p(a) = 0$.

Let F be an algebraically closed field, then the roots of polynomials over F are in F . The only irreducible univariate polynomials over F are of degree one, and every univariate polynomial over F is a product of linear polynomials and a constant.

The algebraic closure of $\mathbb{Q}$ are the algebraic numbers $\bar{\mathbb{Q}}$. The algebraic closure of $\mathbb{R}$ are the complex numbers $\mathbb{C}$.

2.2.3 Resultants and Discriminants

A central tool for the CAD are resultants and discriminants. We introduce their definition and important properties.



(a) The real roots of the resultant of p_1 and p_2 w.r.t. x_2 depicted in $\mathbb{R}$ (on the line below). (b) The real roots of the resultant of q_1 and q_2 w.r.t. x_3 depicted in $\mathbb{R}^2$ (on the x_1 - x_2 -plane).

Figure 2.2: Resultants of the polynomials from Example 2.2.

Definition 2.9 (Resultant)

Let X be a set of variables, $x \notin X$ be a variable, $p, q \in \mathbb{Q}[X][x]$, $d = \deg_x(p) > 0$, and $e = \deg_x(q) > 0$. Assume $p = a_d x^d + \dots + a_0 x^0$ and $q = b_e x^e + \dots + b_0 x^0$. The (univariate) resultant of p and q w.r.t. x is the determinant of the *Sylvester matrix* (of dimension $(d+e) \times (d+e)$):

$$\text{res}_x(p, q) = \det \begin{pmatrix} a_d & \dots & a_0 & & & \\ & a_d & \dots & a_0 & & \\ & & \ddots & \dots & \ddots & \\ & & & a_d & \dots & a_0 \\ b_e & & & & b_0 & \\ & b_e & & & b_0 & \\ & & \ddots & & & \ddots \\ & & & b_e & \dots & b_0 \end{pmatrix}.$$

Given two polynomials $p, q \in \mathbb{Q}[X][x]$, the degree (in all variables) of their resultant depends on the degrees of p and q bounded by $\deg(\text{res}_x(p, q)) \leq 2 \max(\deg(p), \deg(q))^2$ [ED16].

The following characterization shows the usefulness of resultants:

Lemma 2.1 (Characterization of Resultants [BPR06, Theorem 4.16])

Let x be a variable, $p, q \in \mathbb{Q}[x]$, $d = \deg_x(p) > 0$, and $e = \deg_x(q) > 0$. Assume $p = a_d x^d + \dots + a_0 x^0$ and $q = b_e x^e + \dots + b_0 x^0$.

Let $\lambda_1, \dots, \lambda_d \in \mathbb{C}$ be the complex roots of p and $\mu_1, \dots, \mu_e \in \mathbb{C}$ be the complex roots of q counted with their multiplicities.

Then $\text{res}_x(p, q) = a_d^e b_e^d \prod_{i \in [d], j \in [e]} (\lambda_i - \mu_j)$.

This characterization considers the *complex* roots of p and q . As $\mathbb{C}$ is an algebraically closed field, both polynomials factor into linear polynomials. The resultant of two univariate polynomials is zero if and only if they have a common root. Now consider a set of variables X and $x \notin X$ and two polynomials $p, q \in \mathbb{Q}[X][x]$. Then their resultant vanishes at points exactly where p and q have a common complex root in x . As $\mathbb{R} \subset \mathbb{C}$, this also covers the real roots. Thus, the resultant is used to identify intersections of the variety graphs of two polynomials.

Example 2.2

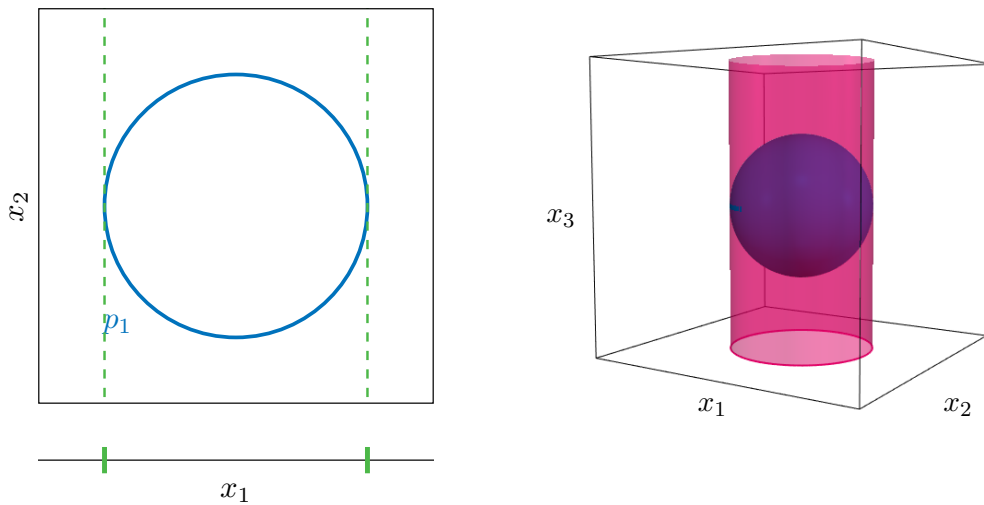
Consider the polynomials $p_1 = x_1^2 + x_2^2 - 1$, $p_2 = x_1 + x_2$ and their resultant $\text{res}_{x_2}(p_1, p_2) = 2x_1^2 - 1$ whose varieties are depicted in Figure 2.2a and the polynomials $q_1 = x_1^2 + x_2^2 + x_3^2 - 1$, $q_2 = x_1 + x_2 + x_3$ and their resultant $\text{res}_{x_3}(q_1, q_2) = 2x_1^2 + 2x_2x_1 + 2x_2^2 - 1$ whose varieties are depicted in Figure 2.2b.

Definition 2.10 (Discriminant)

Let X be a set of variables, $x \notin X$ be a variable, $p \in \mathbb{Q}[X][x]$, and $d = \deg_x(p) > 0$. The (*univariate*) *discriminant* of p w.r.t. x is the resultant of p and its partial derivative w.r.t. x :

$$\text{disc}_x(p) = (-1)^{\frac{d \cdot (d-1)}{2}} \cdot \frac{\text{res}_x(p, \frac{\partial}{\partial x} p)}{\text{lcf}_x(p)}.$$

The discriminant of a polynomial p with respect to a variable x is a polynomial that vanishes at those points where p has a multiple root. Thus, the discriminant is used to identify self-intersections and endpoints of the variety graphs of polynomials.



(a) The real roots of the discriminant of p_1 w.r.t. x_2 depicted in $\mathbb{R}$ (on the line below). (b) The real roots of the discriminant of q_1 w.r.t. x_3 depicted in $\mathbb{R}^2$ (on the x_1 - x_2 -plane).

Figure 2.3: Discriminants of the polynomials from Example 2.3.

Example 2.3

Consider the polynomial $p_1 = x_1^2 + x_2^2 - 1$ and its discriminant $\text{disc}_{x_2}(p_1) = -4(x_1^2 - 1)$ whose varieties are depicted in Figure 2.3a and the polynomial $q_1 = x_1^2 + x_2^2 + x_3^2 - 1$ and its discriminant $\text{disc}_{x_3}(q_1) = -4(x_1^2 + x_2^2 - 1)$ whose varieties are depicted in Figure 2.3b.

2.2.4 Semi-Algebraic Sets and Analytic Submanifolds

A set $R \subseteq \mathbb{R}^n$ is called *semi-algebraic* if it is of the form $\{r \in \mathbb{R}^n \mid p(r) = 0\}$ for some $p \in \mathbb{Q}[X]$, or is of the form $\{r \in \mathbb{R}^n \mid p(r) < 0\}$ for some $p \in \mathbb{Q}[X]$, or is a finite union of intersections of such sets.

For the CAD projection by McCallum [McC85] used in large parts of this thesis, in the following, we introduce analytic submanifolds. For more details, we refer to [McC85].

A function is *analytic* if it is “as smooth as it could be”: Given an open set $R \subseteq \mathbb{R}^n$, a function $f : R \rightarrow \mathbb{R}$ is called *analytic* if for every $r \in R$ there exists a neighbourhood N_r of r and $a_\alpha \in \mathbb{R}$ for $\alpha \in \mathbb{N}^n$ such that the *power series*

$$\sum_{\alpha \in \mathbb{N}^n} a_\alpha (x_1 - r_1)^{\alpha_1} \cdots (x_n - r_n)^{\alpha_n}$$

is absolutely convergent to f in N_r . An *(analytic) submanifold of dimension i* is a set $R \subseteq \mathbb{R}^n$ that “looks locally like $\mathbb{R}^i$ ”. All non-empty open subsets of $\mathbb{R}^n$ are submanifolds of dimension n , but also lower-dimensional submanifolds are possible, if they can be mapped to an open set of dimension n using an analytic function. Formally, this is expressed by the following property. A non-empty set $R \subseteq \mathbb{R}^n$ is an *(analytic) submanifold of dimension i* ($i \in [n]$) if and only if for every $r \in R$ there is a neighbourhood $N_r \subseteq \mathbb{R}^n$ of r , an open $V \subseteq \mathbb{R}^n$, and a *coordinate system* $\Phi : N_r \rightarrow V$ about r — which is a homeomorphism such that $\Phi_1, \dots, \Phi_n : N_r \rightarrow \mathbb{R}$ and $(\Phi^{-1})_1, \dots, (\Phi^{-1})_n : V \rightarrow \mathbb{R}$ are analytic functions and $\Phi(r) = 0$ — such that $R \cap N_r = \{r' \in N_r \mid \Phi_{i+1}(r') = 0 \wedge \dots \wedge \Phi_n(r') = 0\}$ [McC85, Theorem 2.2.1]. We can extend the notion of analytic functions to be defined on analytic submanifolds $R \subseteq \mathbb{R}^n$ of dimension i using these homeomorphisms to show that a function $f : R \rightarrow \mathbb{R}$ “looks locally like an analytic function from $\mathbb{R}^i$ to $\mathbb{R}$ ”.

Given an analytic submanifold $R \subseteq \mathbb{R}^n$ and analytic functions $f, g : R \rightarrow \mathbb{R}$ with $f < g$, the sets $R \times (f, g)$ and $R \times f$ are analytic submanifolds [McC85, Theorem 2.2.3. and Theorem 2.2.4].

2.2.5 Thom’s Lemma

Another important statement is the following theorem (see also [Mis93, pp. 325–326]):

Theorem 2.1 (Thom’s Lemma)

Let $p \in \mathbb{Q}[x]$, $d = \deg_x(p)$, and $s \in \{-1, 0, 1\}^{d+1}$. Then $\{r \in \mathbb{R} \mid \forall k \in [0..d]. \text{sgn}((\frac{\partial^k}{\partial x^k} p)(r)) = s_{k+1}\}$ is either empty or connected.

2.3 Real Projective Space

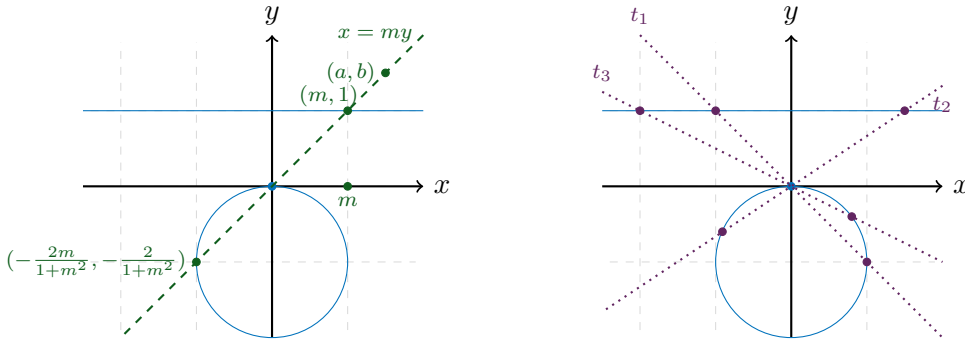
We briefly introduce foundations on the real projective space required later in this thesis; for further reading, we refer to e.g. [Hat01; MNM⁺24].

The *real projective line* $\mathbb{P}$ is defined as the quotient set of $\mathbb{R}^2 \setminus \{(0, 0)\}$ by the equivalence relation of proportionality of the coordinates. We denote the quotient map by $\pi : \mathbb{R}^2 \setminus \{(0, 0)\} \rightarrow \mathbb{P}$ and the equivalence class of $(x, y) \in \mathbb{R}^2 \setminus \{(0, 0)\}$ by $(x : y) = \pi(x, y)$. Intuitively, $\mathbb{P}$ is the set of lines in $\mathbb{R}^2$ passing through the origin $(0, 0)$, and such a line is identified by any point on it.

The real line $\mathbb{R}$ is usually identified with an open and dense subset of $\mathbb{P}$ as follows: the continuous injective map $\varphi : \mathbb{R} \rightarrow \mathbb{P} : x \mapsto (x : 1)$ maps each element of $\mathbb{R}$ to a line in $\mathbb{R}^2 \setminus \{(0, 0)\}$; the only point of $\mathbb{P}$ not contained in the image of φ is $(1 : 0)$, which we also denote by ∞ . The inverse map $\varphi^{-1} : \mathbb{P} \setminus \{\infty\} \rightarrow \mathbb{R} : (x : y) \mapsto \frac{x}{y}$ is also continuous.

For visualizing $\mathbb{P}$, we identify an element $(m : 1) \in \mathbb{P}$ with the intersection point $(-\frac{2m}{1+m^2}, -\frac{2}{1+m^2})$ of $x = my$ with the unit circle around $(0, -1)$ and identify ∞ with $(0, 0)$, as depicted in Figure 2.4a. Thus, we can identify $\mathbb{P}$ with a circle.

We cannot use a linear order on $\mathbb{P}$; however, $\mathbb{P}$ has a *cyclic ordering* that extends the usual order on $\mathbb{R}$, as intuitively given in Figure 2.4b using the identification with the circle. We write $[t_1, t_2, t_3]$, $[t_2, t_3, t_1]$, and $[t_3, t_1, t_2]$ if $t_1, t_2, t_3 \in \mathbb{R}$ and $t_1 < t_2 < t_3$, and $[t_1, t_2, \infty]$, $[t_2, \infty, t_1]$ and $[\infty, t_1, t_2]$ if $t_1, t_2 \in \mathbb{R}$ and $t_1 < t_2$. Thus, intuitively, for distinct $t_1, t_2, t_3 \in \mathbb{P}$, we use $[t_1, t_2, t_3]$ to denote that “after t_1 , one reaches t_2 before t_3 ” in that cyclic ordering on $\mathbb{P}$. We use $[t_1, \dots, t_k]$ for $k > 3$ to denote $\forall j < j' < j'' \in [k]. [t_j, t_{j'}, t_{j''}]$.



(a) Embedding of $\mathbb{R}$ into $\mathbb{P}$, and an identification with a circle. The equivalence class $(a : b)$ is identified with the dashed green line $x = my$ (determined by the vector (a, b)), the real number $m := \varphi^{-1}(a : b)$ (consider the intersection with $y = 1$), and a point on the unit circle at $(0, -1)$. (b) Visualization of the cyclic ordering $[t_1, t_2, t_3]$.

Figure 2.4: Embedding of the real line into the real projective line.

2.4 Mathematical Logic

We recall definitions from mathematical logic to formally state the problems that the algorithms from this thesis solve. For further reading, we refer to, e.g., [Bur98].

We start with recalling *propositional logic* which reasons about *Boolean* predicates that can take the values *false* and *true*:

Definition 2.11 (Propositional Logic)

- Let τ be the countable set of *propositional variables*.
- Let $\mathbb{B} = \{\text{false}, \text{true}\}$ be the set of *Boolean constants*.
- The set PL of *propositional formulas* consists of $\mathbb{B} \subset \text{PL}$; $\tau \subset \text{PL}$; for each $\varphi \in \text{PL}$, its *negation* $\neg\varphi \in \text{PL}$; for each $\varphi, \psi \in \text{PL}$, their *conjunction* $\varphi \wedge \psi \in \text{PL}$ and their *disjunction* $\varphi \vee \psi \in \text{PL}$.
- Formulas in PL that are variables or constants are called *atoms*. Atoms and their negations are called *literals*.
- An *interpretation* is a function $\mathfrak{I} : \sigma \rightarrow \mathbb{B}$ for a finite $\sigma \subseteq \tau$. For $\varphi \in \text{PL}$ and an interpretation $\mathfrak{I}$ that assigns all variables in φ , $\llbracket \varphi \rrbracket^{\mathfrak{I}} \in \mathbb{B}$ is the *truth value of φ under $\mathfrak{I}$* which is defined according to the usual semantics. If $\llbracket \varphi \rrbracket^{\mathfrak{I}} = \text{true}$, then $\mathfrak{I}$ *satisfies* (or is a *solution* of) φ , and we write $\mathfrak{I} \models \varphi$.

Definition 2.12 (Boolean Satisfiability Problem (SAT))

The *Boolean satisfiability problem (SAT)* is to decide whether for a given formula $\varphi \in \text{PL}$, there exists an interpretation $\mathfrak{I}$ such that $\mathfrak{I} \models \varphi$.

SAT was the first problem to be proven to be NP-complete by Cook [Coo71] and Levin [Tra84], that is, all problems that can be verified in polynomial time can be reduced to SAT.

While SAT is already powerful, it does not allow expressing statements about mathematical theories such as real algebra. We thus define the stronger first-order logic:

Definition 2.13 (First-Order Logic)

- A *signature* $\tau = (f_1, f_2, \dots, P_1, P_2, \dots)$ is a set of *function symbols* $f_1, f_2, \dots$ and *relation symbols* $P_1, P_2, \dots$, each of them having a fixed *arity*. Function symbols of zero arity are called *constant symbols*.
- Let VAR be a countable set of *variables*.
- The set $T(\tau)$ of τ -*terms* consists of $\text{VAR} \subset T(\tau)$; and for all $t_1, \dots, t_n \in T(\tau)$ and each function symbol $f \in \tau$ of arity n , $f(t_1, \dots, t_n) \in T(\tau)$.
- The set $\text{FO}(\tau)$ of τ -*formulas* consists of: (1) for $t_1, t_2 \in T(\tau)$, the *equality* $t_1 = t_2 \in \text{FO}(\tau)$; (2) for $t_1, \dots, t_n \in T(\tau)$ and each relation symbol $P \in \tau$ of arity n , $P(t_1, \dots, t_n) \in \text{FO}(\tau)$; (3) for each $\varphi \in \text{FO}(\tau)$, $\neg\varphi \in \text{FO}(\tau)$; (4) for each $\varphi, \psi \in \text{FO}(\tau)$, $(\varphi \wedge \psi) \in \text{FO}(\tau)$ and $(\varphi \vee \psi) \in \text{FO}(\tau)$; (5) for each $\varphi \in \text{FO}(\tau)$ and

each $x \in \text{VAR}$, the *existential quantification* $\exists x. \varphi \in \text{FO}(\tau)$ and the *universal quantification* $\forall x. \varphi \in \text{FO}(\tau)$.

- Formulas in $\text{FO}(\tau)$ built according to rules (1) and (2) are called *atoms* or *constraints*. Atoms and their negations are called *literals*. Formulas that are built according to rules (1) to (4) are called *quantifier-free*.
- The set of formulas in $\text{FO}(\tau)$ built without universal quantification and where negation occurs only in front of quantifier-free subformulas is called the *existential fragment*.
- An occurrence of a variable x in a formula is called *free* if it is not inside any subformula of the form $\exists x. \varphi$ or $\forall x. \varphi$. A formula without free variables is called a *sentence*. For a formula φ , we write $\varphi(x, \dots)$ to indicate that $x, \dots$ are the only free variables in φ .
- A τ -structure $\mathfrak{A} = (D, f_1^{\mathfrak{A}}, f_2^{\mathfrak{A}}, \dots, P_1^{\mathfrak{A}}, P_2^{\mathfrak{A}}, \dots)$ consists of a *domain* D and an *interpretation* for each element of the signature: each function symbol $f \in \tau$ of arity n is assigned to a function $f^{\mathfrak{A}} : D^n \rightarrow D$, and each relation symbol $P \in \tau$ of arity n is assigned to a relation $P^{\mathfrak{A}} \subseteq D^n$.
- A τ -interpretation is a tuple $\mathfrak{I} = (\mathfrak{A}, \alpha)$ of a τ -structure $\mathfrak{A}$ with domain D and a (variable) assignment $\alpha : X \rightarrow D$ for a finite $X \subseteq \text{VAR}$. For $t \in \text{T}(\tau)$ and a τ -interpretation $\mathfrak{I}$ that assigns all variables in t , $\llbracket t \rrbracket^{\mathfrak{I}} \in D$ is the *evaluation of t under $\mathfrak{I}$* which is defined according to the usual semantics. For $\varphi \in \text{FO}(\tau)$ and a τ -interpretation $\mathfrak{I}$ that assigns all variables in φ , $\llbracket \varphi \rrbracket^{\mathfrak{I}} \in \mathbb{B}$ is the *truth value of φ under $\mathfrak{I}$* which is defined according to the usual semantics. If $\llbracket \varphi \rrbracket^{\mathfrak{I}} = \text{true}$, then $\mathfrak{I}$ *satisfies* (or *is a solution of*) φ . We write $\mathfrak{I} \models \varphi$ to denote that $\mathfrak{I}$ assigns all variables in φ and $\llbracket \varphi \rrbracket^{\mathfrak{I}} = \text{true}$. If φ is a sentence, we omit the assignment in those notations by writing $\mathfrak{A}$ instead of $\mathfrak{I}$.
- Let $\varphi, \psi \in \text{FO}(\tau)$. We write $\psi \models \varphi$ if for each τ -interpretation $\mathfrak{I}$ such that $\mathfrak{I} \models \psi$ it holds $\mathfrak{I} \models \varphi$. We say φ and ψ are *equivalent* ($\varphi \equiv \psi$) if $\varphi \models \psi$ and $\psi \models \varphi$. We extend the notation to sets of formulas accordingly.
- A τ -theory $\Phi \subset \text{FO}(\tau)$ is a set of sentences such that for all $\varphi \in \text{FO}(\tau)$ such that $\Phi \models \varphi$ it holds $\varphi \in \Phi$, and $\text{false} \notin \Phi$. It is called *complete* if for all $\varphi \in \text{FO}(\tau)$ it holds either $\varphi \in \Phi$ or $\neg\varphi \in \Phi$. Let $\mathfrak{A}$ be a τ -structure, then $\{\varphi \mid \mathfrak{A} \models \varphi\}$ is a complete theory, called the *theory of $\mathfrak{A}$* .
- Let $\varphi, \psi \in \text{FO}(\tau)$ and Φ be a theory. We write $\psi \models_{\Phi} \varphi$ if for each τ -interpretation $\mathfrak{I}$ such that $\mathfrak{I} \models \Phi$ and $\mathfrak{I} \models \psi$ it holds $\mathfrak{I} \models \varphi$. We say φ and ψ are *equivalent modulo Φ* ($\varphi \equiv_{\Phi} \psi$) if $\varphi \models_{\Phi} \psi$ and $\psi \models_{\Phi} \varphi$. We extend the notation to sets of formulas accordingly.

Note that we can embed propositional logic into first-order logic by introducing relation symbols of zero arity.

We are not interested in deciding general first-order logic, but whether a given sentence is contained in a theory of interest. We consider for instance the *theory of the reals*, being the theory of the structure $(\mathbb{R}, +^{\mathbb{R}}, \cdot^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}}, <^{\mathbb{R}})$ with the usual interpretations of the functions and relations.

Definition 2.14 (Satisfiability-Modulo-Theories Problem)

The *satisfiability-modulo-theories problem (SMT)* is — given a signature τ and a τ -theory Φ — to decide whether a sentence φ in the existential fragment of $\text{FO}(\tau)$ is contained in Φ , i.e., whether there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \Phi$ and $\mathcal{I} \models \varphi$.

An *SMT solver* is a tool for solving such problems. Often, SMT solvers support *uninterpreted functions*, that is, we extend the signature of a given theory by function symbols $h_1, h_2, \dots$ and decide whether there exist functions $h_1^{\mathfrak{A}}, h_2^{\mathfrak{A}}, \dots$ such that $(\mathbb{R}, +^{\mathbb{R}}, \cdot^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}}, <^{\mathbb{R}}, h_1^{\mathfrak{A}}, h_2^{\mathfrak{A}}, \dots) \models \varphi$ in case of the theory of the reals. Further, the combination of different theories is often possible; for instance by introducing unary relations over the integers. We refer to the SMT-LIB [BFT17] specification which is the standard input language for SMT solvers for more details.

For the scope of this thesis, the following SMT theories are of interest:

- The *theory of the reals*, i.e. the theory of the structure $(\mathbb{R}, +^{\mathbb{R}}, \cdot^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}}, <^{\mathbb{R}})$.
- The *theory of linear real arithmetic*, i.e. the theory of the structure $(\mathbb{R}, +^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}}, <^{\mathbb{R}})$, which restricts the theory of the reals to linear polynomials.
- The *theory of non-linear real arithmetic*, which is incomplete as it extends the theory of the reals for division, i.e. its signature is $\tau = \{+, \cdot, /, 0, 1, <\}$. The theory is defined such that $+, \cdot, 0, 1, <$ are interpreted as $+^{\mathbb{R}}, \cdot^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}}, <^{\mathbb{R}}$ respectively, and $/$ is interpreted with a function $/* : \mathbb{R}^2 \rightarrow \mathbb{R}$ which coincides with the real division if the denominator is not equal to 0. If the denominator is equal to zero, then any satisfying interpretation fixes an arbitrary but fixed value while maintaining functional congruence (causing incompleteness of the theory).

Although this is the theory that is implemented by most SMT solvers supporting arithmetic, to avoid the peculiarities with the semantics, in this thesis, we mostly study the theory of the reals rather than the theory of non-linear real arithmetic.

Besides the SMT problem, we are also interested in deciding whether general sentences from $\text{FO}(\tau)$ are in a given theory, i.e. lifting the restriction to the existential fragment:

Definition 2.15 (Decision Problem of First-Order Theories)

The *decision problem of first-order theories* is — given a signature τ and a theory Φ — to decide whether a sentence $\varphi \in \text{FO}(\tau)$ is contained in Φ , i.e., whether there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \Phi$ and $\mathcal{I} \models \varphi$.

Related to this question is describing the values for free variables of a formula which satisfy the formula w.r.t. a given theory:

Definition 2.16 (Quantifier Elimination Problem)

The *quantifier elimination problem (QE)* is — given a signature τ and a theory Φ — to compute for a formula $\varphi \in \text{FO}(\tau)$ a quantifier-free formula $\psi \in \text{FO}(\tau)$ (whose free variables are also free variables in φ) that is equivalent to φ modulo Φ , i.e., for all structures $\mathfrak{A} \models \Phi$ and for all assignments α it holds $(\mathfrak{A}, \alpha) \models \varphi$ if and only if

$(\mathfrak{A}, \alpha) \models \psi$. We call the free variables of φ *parameters*.

Closely related to this problem is the following optimization variant:

Definition 2.17 (Optimization-Modulo-Theories Problem)

The *optimization-modulo-theories problem (OMT)* is — given a signature τ , a theory Φ , a τ -structure $\mathfrak{A}$ with domain D such that $\mathfrak{A} \models \Phi$, and a relation $<^{\mathfrak{A}}$ in $\mathfrak{A}$ that is a strict total order on D — to compute for a formula $\varphi \in \text{FO}(\tau)$ with a single free variable x the maximal value $d \in D$ such that $(\mathfrak{A}, \alpha) \models \varphi$ implies $\alpha(x) \geq d$ for all assignments α , if it exists, and $-\infty$ otherwise.

For defining algorithms solving these problems, it is useful to assume that the formulas are in some *normal form* that restricts the syntax of the input formulas, but do not restrict their expressiveness:

Definition 2.18 (Normal Forms)

- A *clause* is a disjunction of literals. A formula in *conjunctive normal form (CNF)* is a conjunction of clauses.
- A formula is in *negation normal form (NNF)* if the only subformulas that appear in a negation are atoms.
- A formula is in *prenex normal form (PNF)* if it is of the form $Q_i x_i \dots Q_n x_n. \varphi(x_1, \dots, x_n)$ where $i \in [n + 1]$, $Q_i, \dots, Q_n \in \{\forall, \exists\}$, and φ is quantifier-free. We call $Q_i x_i \dots Q_n x_n.$ the *prefix* and $\varphi(x_1, \dots, x_n)$ the *matrix*. A *quantifier block* is a set of subsequent variables occurring in the prefix with the same quantifier. The number of maximal quantifier blocks in the prefix minus one is the *number of quantifier alternations*.

We can transform each quantifier-free formula to an equivalent formula in CNF and NNF and each formula into a formula in prenex normal form, using standard transformation rules. The transformation to CNF may increase the formula's size exponentially. However, using *Tseitin's transformation* [Tse83], we can transform each quantifier-free formula into a formula in CNF by introducing additional propositional variables for each subformula; the resulting formula is satisfiable if and only if the original one is satisfiable. It is thus often assumed that a propositional formula is in CNF.

Finally, we define some related notation that we will use throughout this paper. We omit the theory, structures, and signatures in the notation as we will (mostly) use the theory of the reals. We refer to formulas of the theory of the reals as *Tarski formulas*. Further, instead of variable assignments, as in Section 2.2, we assume an ordering on the variables and use $r \in \mathbb{R}^i$ for some $i \in [n]$. We thus use, e.g., $r \models \varphi$ to indicate that $r \in \mathbb{R}^i$ satisfies (or is a solution of) φ (modulo the theory of the reals); we use $\varphi(r, x_{i+1}, \dots, x_n)$ to denote the substitution of $r_1, \dots, r_i$ for the variables $x_1, \dots, x_i$ in $\varphi(x_1, \dots, x_n)$. Given a set $R \subseteq \mathbb{R}^i$, we call a formula φ *truth-invariant* on R if and only if $\varphi(r) \in \{\text{true}, \text{false}\}$ and $\varphi(r) = \varphi(r')$ for all $r, r' \in R$.

We further admit additional relation symbols $\{=, \neq, <, >, \leq, \geq\}$ in the constraints. Usually, we assume constraints to be of the form $p \sim 0$ where p is a polynomial and $\sim$

a relation symbol; we then call $\sim$ a *sign condition* on p . We use $\text{polys}(\varphi)$ to denote the set of polynomials that occur in φ .

2.5 Satisfiability and Satisfiability-Modulo-Theories

In Section 1.1.6, we already outlined the developments in the satisfiability (SAT) and satisfiability-modulo-theories (SMT) communities. Now we introduce the algorithms on which the state-of-the-art solvers build: the CDCL algorithm for satisfiability, and its extensions for SMT, which are the CDCL(T) and MCSAT algorithms.

2.5.1 Conflict-Driven Clause Learning (CDCL)

Given a propositional formula in CNF, there are two approaches for deciding its satisfiability: (1) By *enumeration*, where we check either for each of the 2^n possible interpretations whether it satisfies the formula, or we iteratively substitute a value for some variable in the formula and backtrack if the result is not satisfiable. The latter variant may already eliminate some interpretations based on the formula's structure. (2) By *deduction*, which is based on the *resolution rule*: Let $C = (L_1 \vee \dots \vee L_k \vee L)$ and $D = (L'_1 \vee \dots \vee L'_{k'} \vee \neg L)$ be two clauses, then the *resolvent* is defined as $R = (L_1 \vee \dots \vee L_k \vee L'_1 \vee \dots \vee L'_{k'})$; $(C \wedge D) \rightarrow R$ is a tautology. By picking a variable in the formula and applying the resolution rule to each possible pair of clauses, we can eliminate the variable from the formula. By eliminating all variables, we end up with either *true* or *false* (the latter by deriving the empty clause). In the former case, we know that there exists a satisfying interpretation, and in the latter case, the formula is unsatisfiable. As during the elimination of one variable, the number of clauses grows quadratically, the whole algorithm is doubly exponential in the number of variables.

The latter idea leads to the early DP [DP60] and DPLL [DLL62] algorithm, which we omit here but refer to [Kre20, Chapter 3] for details. Modern solvers extend the DPLL algorithm to combine the enumeration and deduction approach, an approach called *conflict-driven clause learning* (CDCL) [MS96; MS99].

The CDCL algorithm works by *deciding* truth values of yet undecided variables and *propagating* these decisions to identify truth values for other variables that follow from the decision using the *unit clause rule*: A clause $L_1 \vee \dots \vee L_k \vee L$ is *unit* under an interpretation $\mathfrak{I}$ if $\mathfrak{I} \models \neg L_i$ for $i \in [k]$ and $\mathfrak{I} \not\models L$ and $\mathfrak{I} \not\models \neg L$; then, any extension of $\mathfrak{I}$ that satisfies the clause needs to satisfy L , uniquely determining the assigned value for its variable. The algorithm alternates between decisions and propagation; each decision marks the beginning of a *decision level* on which the algorithm propagates exhaustively. A special decision level 0 contains assignments based on unit clauses with a single literal and corresponding propagated assignments. The algorithm iterates until it either yields a satisfying interpretation, or some clause becomes *conflicting* (i.e. all literals are *false* under the current interpretation) during the propagation phase. It then *explains* the conflict by eliminating the variable propagated last from the conflicting clause using resolution with the propagating clause. This process continues until it obtains a clause that is *asserting*, meaning that there is a single literal which has been assigned on a higher decision level than the others; this decision level is the second highest one that occurs in the conflicting clause (or 0 if it does not exist). After *backtracking* — i.e. undoing all the decisions and propagations of that decision level and above — the clause becomes unit, and the algorithm continues with propagation.

Although this already yields a complete algorithm, *learning* the clauses that result from the explanation phase by adding them to the clause set allows to use them in further propagation phases, drastically increasing efficiency in practice. For housekeeping of memory, the algorithm removes these clauses when they do not seem helpful. Additionally, the CDCL algorithm may *restart*, that is, removing all positive decision levels but keeping all learned clauses, whenever it thinks that the current branch of the search tree is not promising; the learned clauses may guide the search process into a more promising branch.

The decisions and propagations are stored in a data structure called *trail*:

Definition 2.19 (CDCL Trail)

A *trail* M is a sequence of elements of the form

- L where L is a Boolean literal (called *decision*), or
- $C \rightarrow L$ where C is a clause and L is a Boolean literal (called *propagation*).

For any literal L , we write $L \in M$ if L or $C \rightarrow L$ (for some clause C) occurs in M . The trail M is called *consistent* if for any literal L it holds $L \notin M$ or $\neg L \notin M$.

For better readability, we denote a trail using the brackets $\llbracket \dots \rrbracket$. Given trails M and N , we use $\llbracket M, N \rrbracket$ to denote the trail resulting from the concatenation of M and N .

Throughout the algorithm, the trail is always kept consistent. We state the CDCL algorithm as a transition system. Its states are given by the following definition.

Definition 2.20 (CDCL State)

Let $\mathcal{C}$ be a set of clauses, C be a clause and M be a consistent trail. Then $\langle M, \mathcal{C} \rangle$ is a *search state*, and $\langle M, \mathcal{C} \rangle \vdash C$ is called a *conflict state*.

Note that in this context, the turnstile $\vdash$ denotes a conflict state, and does not denote an entailment.

Starting from the initial state $\langle \llbracket \rrbracket, \mathcal{C} \rangle$, we apply the following transition rules. These rules maintain consistency of the trail and ensure that the algorithm terminates.

Definition 2.21 (CDCL Transition Rules)

Let M be a consistent trail and $\mathcal{C}$ be a set of clauses.

DECIDE Let L be a literal, $L \notin M$, and $\neg L \notin M$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle \llbracket M, L \rrbracket, \mathcal{C} \rangle$$

PROPAGATE Let $C = (L \vee L_1 \vee \dots \vee L_k) \in \mathcal{C}$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, $L \notin M$, and $\neg L \notin M$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle \llbracket M, C \rightarrow L \rrbracket, \mathcal{C} \rangle$$

CONFLICT Let $C = (L_1 \vee \dots \vee L_k) \in \mathcal{C}$ (for some $k \in \mathbb{N}$) and $\neg L_i \in M$ for all $i \in [k]$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle M, \mathcal{C} \rangle \vdash C$$

EXPLAIN Let N be a trail, $C = (L_1 \vee \dots \vee L_k \vee L)$ (for some $k \in \mathbb{N}$), $D = (L'_1 \vee \dots \vee L'_{k'} \vee \neg L)$ ($k' \in \mathbb{N}$), $\neg L_1, \dots, \neg L_k, \neg L'_1, \dots, \neg L'_{k'} \in M$, and $R = (L_1 \vee \dots \vee L_k \vee L'_1 \vee \dots \vee L'_{k'})$.

$$\langle \llbracket M, D \rightarrow L, N \rrbracket, \mathcal{C} \rangle \vdash C \longrightarrow \langle \llbracket M, D \rightarrow L, N \rrbracket, \mathcal{C} \rangle \vdash R$$

BACKJUMP Let $C = (L_1 \vee \dots \vee L_k \vee L)$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, $\neg L \notin M$, $L \notin M$, and there is no $L' \in M$ such that the prefix M' of M before L' satisfies $\neg L_i \in M'$ for all $i \in [k]$. Let L' be a literal, and N be a trail.

$$\langle \llbracket M, L', N \rrbracket, \mathcal{C} \rangle \vdash C \longrightarrow \langle \llbracket M, C \rightarrow L \rrbracket, \mathcal{C} \rangle$$

LEARN Let $C \notin \mathcal{C}$.

$$\langle M, \mathcal{C} \rangle \vdash C \longrightarrow \langle M, \mathcal{C} \cup \{C\} \rangle \vdash C$$

FORGET Let $C \in \mathcal{C}$, $\mathcal{C} \setminus \{C\} \models C$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle M, \mathcal{C} \setminus \{C\} \rangle$$

RESTART

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle \llbracket \rrbracket, \mathcal{C} \rangle$$

In Algorithm 2.1, we apply the transition rules in the CDCL algorithm. At this point, we do not detail how the heuristic decisions in the lines starting with **choose** are made. However, we mention the *variable state independent decaying sum (VSIDS)* for selecting the next literal to decide, as it serves as a central heuristic of modern SAT solvers. For each literal, we track a score that increases whenever we see the literal during the explanation phase. We periodically divide these scores by a constant to focus on literals that occurred in recent conflicts. When making a decision about a literal, we choose the one with the highest score. This heuristic thus focuses on exploring the search space near recent conflicts while remaining efficient to maintain.

2.5.2 CDCL(T)

Given an SMT problem, e.g. we assume a formula in CNF where the atoms may be constraints which are interpreted according to some theory, there are two main approaches: In *eager* SMT solving, we encode a first-order formula as propositional formula which we pass to a SAT solver. The *lazy* approach uses the CDCL(T) framework [NOT06] which is an extension of CDCL for SMT problems.

The basic idea of CDCL(T) is to run the CDCL algorithm on the input formula, treating each constraint as a propositional variable; for this, we might replace each constraint by a fresh propositional variable, obtaining the *Boolean abstraction*. (1) In the *naive (or less) lazy* approach, whenever the CDCL algorithm finds a solution for the Boolean abstraction, a *theory solver* checks the consistency of that solution with the theory: It takes the literals corresponding to the Boolean solution and checks whether there exists a variable assignment that satisfies their conjunction (modulo the given theory). If yes, the input formula is satisfiable; otherwise, the theory solver returns an *infeasible subset*

Algorithm 2.1: $\text{cdcl}(\varphi)$

Input : $\varphi \in \text{PL}$.
Output : *true* if there exists an interpretation $\mathcal{I} \models \varphi$, or *false* otherwise.

```

1  $M := \square$ 
2 let  $\mathcal{C}$  be the result of Tseitin's transformation of  $\varphi$ 
3  $S := \langle M, \mathcal{C} \rangle$ 
4 while true do
5   while PROPAGATE is applicable to  $S$  do
6     apply PROPAGATE to  $S$ 
7     if exists  $C = (L_1, \dots, L_k) \in \mathcal{C}$  s.t.  $\neg L_i \in M$  for each  $i \in [k]$  then
8       apply CONFLICT to  $S$  with  $C$ 
9       // Conflict resolution
10      apply EXPLAIN to  $S$  until  $C = \emptyset$  or BACKJUMP is applicable
11       $\langle M, \mathcal{C} \rangle \vdash C := S$ 
12      if  $C = \emptyset$  then
13        return false
14      apply LEARN to  $S$  with  $C$ 
15      apply BACKJUMP to  $S$ 
16   if  $x \in M$  or  $x \notin M$  for all variables  $x$  in  $\varphi$  then
17     return true
18   choose whether to apply FORGET to  $S$  with one or more  $C$ 
19   choose whether to apply RESTART to  $S$ 
20   choose literal  $L$  such that  $L \notin M$  and  $\neg L \notin M$ 
21   apply DECIDE to  $S$  with  $L$ 

```

of its input literals, which — when its negation is interpreted as a clause — excludes the current Boolean interpretation. In the simplest case, it could return all input literals; however, if such a set is *minimal*, it may exclude more solutions to generalize the given *conflict*. Then, the CDCL algorithm continues the search, respecting the new clause. (2) In the (*full*) *lazy* approach, the theory solver continuously receives literals assigned by the CDCL algorithm and checks for consistency with the theory. For efficiency, exhaustive consistency checks may only be performed after all propagations on the current decision levels are done. When a conflict is detected, it generates a minimal infeasible subset in the same way as in the naive lazy approach. The CDCL algorithm then performs conflict resolution and backtracking using the new clause analogously to Boolean conflicts.

For stating the algorithm, we extend the CDCL transition system in the following way: The $\text{CDCL}(T)$ *trail* is defined as the CDCL trail (Definition 2.19), but decisions and propagations also include first-order literals. The $\text{CDCL}(T)$ transition system consists of the CDCL states (Definition 2.20), the CDCL transition rules (Definition 2.21) and the following transition rules:

Definition 2.22 (CDCL(T) Transition Rules)

Let Φ be a theory, M be a consistent trail, and $\mathcal{C}$ be a set of clauses.

T-CONFLICT Let $C = (L_1 \vee \dots \vee L_k)$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, and $\models_{\Phi} C$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle M, \mathcal{C} \rangle \vdash C$$

T-PROPAGATE Let $C = (L \vee L_1 \vee \dots \vee L_k)$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, $L \notin M$, $\neg L \notin M$, and $\models_{\Phi} C$.

$$\langle M, C \rangle \longrightarrow \langle \llbracket M, C \rightarrow L \rrbracket, C \rangle$$

We did not explain the T-PROPAGATE rule which allows introducing additional literals which are implied by the theory. As the CDCL(T) framework is complete without this rule, we omit it in the following presentation.

Algorithm 2.2 describes the modifications of CDCL (Algorithm 2.1) for CDCL(T). The call to `theory_solver(S)` runs the theory solver on the set of first-order literals S , and returns $\perp$ if it does not detect a conflict, and a clause C (which is the negation of an infeasible subset) if there is a conflict. As the theory solver often needs to check increasing sets of constraints, for efficiency, it should be able to work *incrementally*, that is, updating an internal state from the previous call(s) to avoid recomputing intermediate results.

Algorithm 2.2: `cdcl(t)( $\varphi$ )`

Input : $\varphi \in \text{FO}(\tau)$ for some signature τ .
Output : *true* if there exists an interpretation $\mathfrak{I} \models \varphi$, or *false* otherwise.

```

1  $M := \llbracket \rrbracket$ 
2 let  $\mathcal{C}$  be the result of Tseitin's transformation of  $\varphi$ 
3  $S := \langle M, \mathcal{C} \rangle$ 
4 while true do
5   while PROPAGATE is applicable to  $S$  do
6     apply PROPAGATE to  $S$ 
7     if exists  $C = (L_1, \dots, L_k) \in \mathcal{C}$  s.t.  $\neg L_i \in M$  for each  $i \in [k]$  then
8       apply CONFLICT to  $S$  with  $C$ 
9       // Conflict resolution
10      ... (see Algorithm 2.1 Lines 9 to 14)
11     if theory_solver( $\{L \mid L \in M\}$ ) =  $C$  then // Exhaustive checks are often only
12       performed when PROPAGATE is not applicable anymore.
13       apply T-CONFLICT to  $S$  with  $C$ 
14       // Conflict resolution
15       ... (see Algorithm 2.1 Lines 9 to 14)
16   ... (see Algorithm 2.1 Lines 15 to 20)
```

2.5.3 Model-Constructing Satisfiability Calculus (MCSAT)

Like CDCL(T), the *model-constructing satisfiability calculus (MCSAT)* [dJ13; Jov17; JBd13] is an extension of CDCL for SMT. While CDCL(T) alternates between solving of the Boolean structure and consistency checks with the underlying theory, the idea of MCSAT is to interleave the two processes. The main idea is to extend the decisions done throughout the solving process by *decisions on first-order variables (theory variables)*, i.e. assigning them to values from a given domain D . These decisions should be *consistent* with decisions and propagations on the Boolean level, that is, no literal assumed to be true should evaluate to false under the resulting assignment of theory variables and vice versa. Such decisions are used to determine truth values for literals which are not on the trail yet (*semantic propagation*), possibly triggering Boolean propagation and guiding the Boolean decisions. If there does not exist a suitable value for some theory variable (which is not yet assigned), we say there is a *theory conflict* which

is explained by an *explanation function*: Given the set of literals from the trail and assignments to theory variables, it returns a clause that generalizes the conflict (e.g. for $\llbracket x < y, x < -y, x \mapsto 1 \rrbracket$, the explanation function yields $(x < y \wedge x < -y) \rightarrow x > 0$), which is then passed to the conflict resolution from CDCL.

We thus extend the trail data structure:

Definition 2.23 (MCSAT Trail)

Let Φ be a theory and D be a set. An *MCSAT trail* M is a sequence of elements of the form

- L where L is a literal (called *decision*), or
- $C \rightarrow L$ where C is a clause and L is a literal (called *propagation*),
- $x \mapsto v$ where x is a variable and $v \in D$ is a value (called *theory decision*).

For any literal L , we write $L \in M$ if L or $C \rightarrow L$ (for some clause C) occurs in M . We define $\alpha_M : X \hookrightarrow D$ such that $\alpha_M(x) = v$ whenever $(x \mapsto v) \in M$. The trail M is called *consistent* if for any literal L it holds $L \in M$ implies $\neg L \notin M$, and $\alpha_M \models_{\Phi} L$ implies $\neg L \notin M$. The trail M is called *unit consistent* if for any variable x there exists a value $v \in D$ such that $\llbracket M, x \mapsto v \rrbracket$ is consistent.

In the following, we refer to an MCSAT trail simply as *trail* if it is clear from the context.

The MCSAT transition system consists of the CDCL states (Definition 2.20) using the MCSAT trail (Definition 2.23), the CDCL transition rules (Definition 2.21) and the following transition rules:

Definition 2.24 (MCSAT Transition Rules)

Let Φ be a theory and D be a set, M be a consistent trail, and $\mathcal{C}$ be a set of clauses.

T-DECIDE Let x be a variable, v be a value such that $\llbracket M, x \mapsto v \rrbracket$ is consistent.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle \llbracket M, x \mapsto v \rrbracket, \mathcal{C} \rangle$$

T-SEMANTIC-PROPAGATE Let L be a literal, N be a trail, $\alpha_M \models_{\Phi} L$, $L \notin M \cup N$, and $\neg L \notin M \cup N$.

$$\langle \llbracket M, x \mapsto v, N \rrbracket, \mathcal{C} \rangle \longrightarrow \langle \llbracket M, x \mapsto v, L \rightarrow L, N \rrbracket, \mathcal{C} \rangle$$

T-PROPAGATE Let $C = (L \vee L_1 \vee \dots \vee L_k)$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, $L \notin M$, $\neg L \notin M$, and $\models_{\Phi} C$.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle \llbracket M, C \rightarrow L \rrbracket, \mathcal{C} \rangle$$

T-CONFLICT Let $C = (L_1 \vee \dots \vee L_k)$ (for some $k \in \mathbb{N}$), $\neg L_i \in M$ for all $i \in [k]$, and $\models_{\Phi} C$. Note that M is not unit consistent.

$$\langle M, \mathcal{C} \rangle \longrightarrow \langle M, \mathcal{C} \rangle \vdash C$$

T-BACKJUMP-DECIDE Let $C = (L_1 \vee \dots \vee L_k \vee L)$ (for some $k \in \mathbb{N}$), $\neg L_i \notin M$ for some $i \in [k]$, $\neg L \notin M$, $L \notin M$, x be a variable, v be a value, and N be a trail.

$$\langle \llbracket M, x \mapsto v, N \rrbracket, \mathcal{C} \rangle \vdash C \longrightarrow \langle \llbracket M, L \rrbracket, \mathcal{C} \rangle$$

Similarly as in CDCL(T), we include the T-PROPAGATE as it is a common technique in SMT solving. As it is not required for completeness, we omit it from the following presentation.

Algorithm 2.3 is a possible instantiation of the MCSAT transition system, which builds upon the CDCL algorithm: (1) It can choose whether to decide a literal, or use T-DECIDE to decide on theory variables. (2) It applies T-SEMANTIC-PROPAGATE after each decision to propagate the truth values from the theory level to the Boolean level and makes them available for e.g. Boolean propagation. (3) It extends conflict resolution using the T-BACKJUMP-DECIDE rule, as we may not always obtain an asserting clause as in CDCL, as the explanations may contain multiple literals on the highest decision level (and we cannot eliminate them using Boolean resolution) as the corresponding decision is a theory decision and the literals were semantically propagated. (4) It applies T-CONFLICT whenever the trail becomes non-unit-consistent, that is, when there is no value for some variable which keeps the trail consistent. The `explain( $M$ )` function is then used to compute a clause that generalizes the conflict. In contrast to CDCL(T), that clause may contain new literals; we require that the clause evaluates to false under the Boolean and theory assignments from the trail. We apply T-SEMANTIC-PROPAGATE on all new literals before T-CONFLICT.

Example 2.4 (MCSAT Transitions)

Consider the clause set (interpreted over the theory of linear real arithmetic)

$$\mathcal{C} = \{C_1, C_2\} \text{ where } C_1 = (x > 1 \vee y = 0), C_2 = (y \neq 0 \vee x = 0).$$

Assume we start with a theory decision:

$$\begin{aligned} \langle \llbracket \rrbracket, \mathcal{C} \rangle &\xrightarrow{\text{T-DECIDE}} \langle \llbracket x \mapsto 1 \rrbracket, \mathcal{C} \rangle \\ &\xrightarrow{\text{T-SEM-PROP} \times 2} \langle \llbracket x \mapsto 1, x \leq 1 \rightarrow x \leq 1, x \neq 0 \rightarrow x \neq 0 \rrbracket, \mathcal{C} \rangle \\ &\xrightarrow{\text{PROPAGATE}} \langle \llbracket \dots, C_1 \rightarrow y = 0 \rrbracket, \mathcal{C} \rangle \xrightarrow{\text{CONFLICT}} \langle \llbracket \dots, C_1 \rightarrow y = 0 \rrbracket, \mathcal{C} \rangle \vdash C_2 \\ &\xrightarrow{\text{EXPLAIN}} \langle \llbracket \dots, C_1 \rightarrow y = 0 \rrbracket, \mathcal{C} \rangle \vdash (x > 1 \vee x = 0) \end{aligned}$$

We obtain a conflict clause and apply resolution of C_1 with C_2 using the literal $y = 0$ to obtain a new one which still evaluates to false under the current trail. BACKJUMP cannot be applied since removing $x \mapsto 1$ would cause two undefined literals in the conflicting clause. Thus, we have to use T-BACKJUMP-DECIDE which backtracks and decides one of the literals.

$$\begin{aligned} &\xrightarrow{\text{T-BACKJUMP-DECIDE}} \langle \llbracket x = 0 \rrbracket, \mathcal{C} \rangle \xrightarrow{\text{DECIDE}} \langle \llbracket x = 0, y = 0 \rrbracket, \mathcal{C} \rangle \\ &\xrightarrow{\text{T-DECIDE} \times 2} \langle \llbracket x = 0, y = 0, x \mapsto 0, y \mapsto 0 \rrbracket, \mathcal{C} \rangle \end{aligned}$$

Algorithm 2.3: $\text{mcsat}(\varphi)$

Input : $\varphi \in \text{FO}(\tau)$ for some signature τ .
Output : *true* if there exists an interpretation $\mathcal{I} \models \varphi$, or *false* otherwise.

```

1  $M := \square$ 
2 let  $\mathcal{C}$  be the result of Tseitin's transformation of  $\varphi$ 
3  $S := \langle M, \mathcal{C} \rangle$ 
4 while true do
5   apply T-SEMANTIC-PROPAGATE to  $S$  with all literals from  $\mathcal{C}$  while applicable
6   while PROPAGATE is applicable to  $S$  do
7     apply PROPAGATE to  $S$ 
8     if exists  $C = (L_1, \dots, L_k) \in \mathcal{C}$  s.t.  $\neg L_i \in M$  for each  $i \in [k]$  then
9       apply CONFLICT to  $S$  with  $C$ 
10      // Conflict resolution
11      apply EXPLAIN to  $S$  until  $C = \emptyset$  or BACKJUMP or T-BACKJUMP-DECIDE is
12      applicable
13       $\langle M, \mathcal{C} \rangle \vdash C := S$ 
14      if  $C = \emptyset$  then
15        return false
16      apply LEARN to  $S$  with  $C$ 
17      apply BACKJUMP or T-BACKJUMP-DECIDE to  $S$ 
18   if  $M$  is consistent and  $\text{dom}(\alpha_M) \subseteq \text{VAR}$  contains all variables from  $\varphi$  then
19     return true
20   if  $M$  is not unit consistent then
21      $E := \text{explain}(M)$ 
22     apply T-SEMANTIC-PROPAGATE to  $S$  with all literals from  $E$  while applicable
23     apply T-CONFLICT to  $S$  with  $E$ 
24     // Conflict resolution
25     ... (see Lines 10 to Line 15)
26   choose whether to apply FORGET to  $S$  with one or more  $C$ 
27   choose whether to apply RESTART to  $S$ 
28   choose literal  $L$  s.t.  $L \notin M$  and  $\neg L \notin M$ , or  $x \mapsto v$  s.t.  $\llbracket M, x \mapsto v \rrbracket$  is consistent
29   apply DECIDE to  $S$  with  $L$  or T-DECIDE to  $S$  with  $x \mapsto v$ 

```

The trail is consistent and assigns all variables, thus the input is satisfiable. We can also start with a Boolean decision resulting in a different trace of the algorithm:

$$\begin{aligned}
&\langle \square, \mathcal{C} \rangle \xrightarrow{\text{DECIDE} \times 2} \langle \llbracket x = 0, x > 1 \rrbracket, \mathcal{C} \rangle \\
&\quad \xrightarrow{\text{T-CONFLICT}} \langle \llbracket x = 0, x > 1 \rrbracket, \mathcal{C} \rangle \vdash (\neg(x = 0) \vee \neg(x > 1)) = E \\
&\quad \xrightarrow{\text{BACKJUMP}} \langle \llbracket x = 0, E \rightarrow \neg(x > 1) \rrbracket, \mathcal{C} \rangle \\
&\quad \xrightarrow{\text{PROPAGATE}} \langle \llbracket x = 0, E \rightarrow \neg(x > 1), C_1 \rightarrow y = 0 \rrbracket, \mathcal{C} \rangle \\
&\quad \xrightarrow{\text{T-DECIDE} \times 2} \langle \llbracket \dots, x \mapsto 0, y \mapsto 0 \rrbracket, \mathcal{C} \rangle
\end{aligned}$$

The last trace can be simplified using the T-PROPAGATE rule:

$$\begin{aligned}
&\langle \square, \mathcal{C} \rangle \xrightarrow{\text{DECIDE}} \langle \llbracket x = 0 \rrbracket, \mathcal{C} \rangle \xrightarrow{\text{T-PROPAGATE}} \langle \llbracket x = 0, E \rightarrow \neg(x > 1) \rrbracket, \mathcal{C} \rangle \\
&\quad \xrightarrow{\text{PROPAGATE}} \langle \llbracket x = 0, E \rightarrow \neg(x > 1), C_1 \rightarrow y = 0 \rrbracket, \mathcal{C} \rangle
\end{aligned}$$

Example 2.5 (Fourier-Motzkin Explanations)

Consider the clause set (interpreted over the theory of linear real arithmetic)

$$\mathcal{C} = \{C_1, C_2, C_3\} \text{ where } C_1 = (2y < z), C_2 = (y < z), C_3 = (z < x).$$

First, we propagate all unary clauses and decide some theory variables:

$$\begin{aligned} \langle \llbracket \cdot \rrbracket, \mathcal{C} \rangle &\xrightarrow{\text{PROPAGATE} \times 3} \langle \llbracket C_1 \rightarrow 2y < z, C_2 \rightarrow y < z, C_3 \rightarrow z < x \rrbracket, \mathcal{C} \rangle \\ &\xrightarrow{\text{T-DECIDE} \times 2} \langle \llbracket \dots, x \mapsto 0, y \mapsto 1 \rrbracket, \mathcal{C} \rangle \end{aligned}$$

The trail is now not unit consistent, as $2 < z \wedge z < 0$ (C_1 and C_3) and $1 < z \wedge z < 0$ (C_2 and C_3) do not admit a feasible value for z . We use Fourier-Motzkin variable elimination to eliminate z from the conflict reasons, obtaining either $(C_1 \wedge C_3) \rightarrow 2y < x$ or $(C_2 \wedge C_3) \rightarrow y < x$ as conflict clause. Both are equally suitable for explaining the conflict, we only need to proceed with one of them.

The general idea is to determine which constraints are lower and upper bounds on a given variable. By evaluating these bounds under the given assignment, we yield all conflicting bound pairs, from which we pick one to apply Fourier-Motzkin on it.

The above explanation is now used for backtracking, analogously to a clause obtained by the CONFLICT rule, as explained in the previous example.

For the termination of MCSAT, we require that all literals that are added to the trail stem from a finite set (with respect to the input formula) called *finite basis*. The Fourier-Motzkin-based explanation function from Example 2.5 satisfies this property if there is a fixed order in which the theory variables are assigned [NKÁ19, Example 1].

There are various extensions of MCSAT. The above presentation is close to [dJ13; JBd13]. In [Jov17], the framework is further generalized: Instead of only literals, any term may be placed on the trail. It also allows for propagation of theory assignments, for which it generalizes the EXPLAIN rule. The work in [Kre20] proposes to propagate restrictions on the domain of theory variables (e.g. using interval constraint propagation). Further, [Kre20] compares the expressive power of the MCSAT and CDCL(T) transition systems.

2.6 Real Algebraic Number Computations

While rational numbers can be represented exactly in a computer by storing the numerator and denominator (possibly on the heap), e.g. using **GMP-LIB** [Gt91], no such representation for real numbers exists in general. Luckily, for deciding real algebraic problems, the subset of *real algebraic numbers* is sufficient. As this set equals the set of all roots of all univariate polynomials (except 0), we identify a real algebraic number using a polynomial and encode which root of that polynomial is the represented value. For the latter, different options are in the literature [Mis93]:

Order Representation A real algebraic number $r \in (\mathbb{R} \cap \bar{\mathbb{Q}})$ is represented by a pair (p, j) where $p \in \mathbb{Q}[x]$ and $j \in [| \text{realRoots}(p(x)) |]$, and r equals the j -th root (ordered by their value, multiplicities usually not counted) of p .

Sign Representation A real algebraic number $r \in (\mathbb{R} \cap \bar{\mathbb{Q}})$ is represented by a pair $(p, (a_1, \dots, a_d))$ where $p \in \mathbb{Q}[x]$, $d = \deg_x(p)$, and $a_1, \dots, a_d \in \{-1, 0, 1\}$. The sequence $(a_1, \dots, a_d)$ encodes the signs of the derivatives of p at r ; by Thom's Lemma (Theorem 2.1), this uniquely defines a root of p .

Interval Representation A real algebraic number $r \in (\mathbb{R} \cap \bar{\mathbb{Q}})$ is represented by a pair $(p, (l, u))$ where $p \in \mathbb{Q}[x]$ and $l, u \in \mathbb{Q}$, $l < u$ such that r is the only root of p in (l, u) .

The order representation is available in `Maple`. The sign representation is described in [BPR06] and has been implemented in [Kre20], but (to the best of our knowledge) not widely applied. The interval representation is commonly used in the context of CAD and is implemented in e.g. the `z3` [Bjø10] solver, and in the `LibPoly` [JD17] library. While [Mis93] gives algorithms that implement the arithmetic operations on real algebraic numbers, in this section, we only give an overview on how the operations needed for the CAD are efficiently realized; these algorithms are extracted from the source codes of `z3` and `LibPoly`. For the bibliography, we refer to Section 1.1.4.

The interval representation has the major advantage that its operations can be implemented with a mix of numeric and algebraic computations, making it efficient in practice.

We start by a rigorous definition of the interval representation:

Definition 2.25 (Interval Representation of a Real Algebraic Number)

Let $r \in (\mathbb{R} \cap \bar{\mathbb{Q}})$, $I = (l, u) \subseteq \mathbb{R}$ be an open interval such that $l, u \in \mathbb{Q}$, and $p \in \mathbb{Q}[x] \setminus \{0\}$ be a square-free polynomial.

The tuple (p, I) is an *interval representation* of r if $p(r) = 0$, $r \in I$, and for all $r' \in \mathbb{R} \setminus \{r\}$ such that $p(r') = 0$ it holds $r' \notin I$.

The square-freeness requirement is not necessary for the interval representation to be unique, but is useful for the algorithms in this section. Further, one may even require the polynomial to be monic irreducible, which is more expensive to compute but may be more efficient for the following algorithms. In particular, a real algebraic number is represented by a unique polynomial (but a polynomial may still represent multiple real algebraic numbers).

2.6.1 Univariate Real Root Isolation

Given some univariate polynomial, we may want to compute its real roots. This process is often denoted as *real root isolation*, as we do not compute the roots explicitly, but compute intervals which isolate each root of the polynomial. Thus, the result of any real root isolation algorithm directly yields the desired interval representations of all roots.

The most common algorithm is the *bisection* [CA76] method, for which we need two ingredients: (1) Some *bound on the real roots of the polynomial*, e.g. *Cauchy's bound* or *Lagrange's bound* give an upper bound on the absolute value of all roots of a polynomial, based solely on its coefficients. This bound thus yields an interval that contains all roots of a polynomial. (2) A measure to *count the real roots of a polynomial in a given interval*. *Sturm's theorem* is suitable for exact root counting, but it is expensive to compute.

Generalizations based on *Descartes' rule of signs* yield an efficiently computable way for determining whether an interval contains no roots, exactly one root, or possibly more than one root, which is still sufficient for the bisection method (because *Vincent's theorem* guarantees that the third case does not apply for sufficiently small intervals).

The method — as given in Algorithms 2.4 and 2.5 — starts with the interval obtained by a root bound, and checks whether that interval already isolates a single real root. If this is the case, it terminates, otherwise, it splits the interval and recursively calls the method on the obtained intervals, and returns a union of the results.

Algorithm 2.4: $\text{ran_real_roots}(p)$

Input : Polynomial $p \in \mathbb{Q}[x]$.
Output : Set of rationals and interval representations of the elements ξ in $\text{realRoots}(p(x))$.
1 Replace p by its square-free factorization
2 $c_n x^n + \dots + c_1 x + c_0 := p$
3 $b := 1 + \max\{|\frac{c_{n-1}}{c_n}|, \dots, |\frac{c_0}{c_n}|\}$ // Cauchy bound
4 $R := \emptyset$
5 **if** $p(-b) = 0$ **then** $R := R \cup \{-b\}$
6 **if** $p(b) = 0$ **then** $R := R \cup \{b\}$
7 $R := R \cup \text{ran_real_roots}(p, (-b, b))$ // Algorithm 2.5
8 **return** R

Algorithm 2.5: $\text{ran_real_roots}(p, (l, u))$

Input : Polynomial $p \in \mathbb{Q}[x]$ such that p is square-free, $l, u \in \mathbb{Q}$.
Output : Set of rationals and interval representations of the elements in $\text{realRoots}(p(x)) \cap (l, u)$.
1 **if** p has no root in (l, u) **then return** $\emptyset$
2 **else if** p has exactly one root in (l, u) **then return** $\{(p, (l, u))\}$
3 **else** // p has more than one root in (l, u)
4 **choose** $v \in (l, u) \cap \mathbb{Q}$
5 $R := \text{ran_real_roots}(p, (l, v)) \cup \text{ran_real_roots}(p, (v, u))$
6 **if** $p(v) = 0$ **then** $R := R \cup \{v\}$
7 **return** R

2.6.2 Basic Operations

Algorithm 2.6 is a basic algorithm that is required for all the following operations. It refines the isolating interval around the encoded root to obtain tighter bounds. It splits the current interval using the given sample point, and checks which of the resulting intervals contains the root of the polynomial. For this task, we exploit that p is square-free, as then, the sign of the polynomial always changes when passing through a root.

Algorithm 2.6: $\text{ran_refine}((p, I), v)$

Input : Interval representation $(p, I) = (p, (l, u))$ of some $r \in (\mathbb{R} \cap \tilde{\mathbb{Q}})$, $v \in I \cap \mathbb{Q}$.
Output : Interval representation (p, I') of r such that $I' \subset I$, or $r \in \mathbb{Q}$.
1 **if** $p(v) = 0$ **then return** v
2 **else if** $\text{sgn}(p(v)) = \text{sgn}(p(l))$ **then return** $(p, (v, u))$
3 **else return** $(p, (l, v))$ // $\text{sgn}(p(v)) = \text{sgn}(p(u))$

As the interval gets smaller, the size of its representation grows if numerators and denominators are stored explicitly. `LibPoly` [JD17] thus represents the interval bounds

using *dyadic rationals* $\{\frac{a}{2^n} \mid a \in \mathbb{Z}, n \in \mathbb{N}\} \subseteq \mathbb{Q}$. While this limits the intervals that may be represented, it is sufficient for our purpose: For isolating a root, we only need to be able to split a given interval sufficiently many times. Dyadic rationals can be efficiently stored using the parameters a and n , and – as required for splitting – the mid-point $\frac{2^{n_2 - \min\{n_1, n_2\}} a_1 + 2^{n_1 - \min\{n_1, n_2\}} a_2}{2^{\max\{n_1, n_2\} + 1}}$ of an interval with dyadic endpoints $(\frac{a_1}{2^{n_1}}, \frac{a_2}{2^{n_2}})$ thus requires less space to store than the midpoint $(\frac{a_1 b_2 + a_2 b_1}{2 b_1 b_2})$ of an interval with rational endpoints $(\frac{a_1}{b_1}, \frac{a_2}{b_2})$.

Algorithm 2.7: `ran_compare((p, I), (p', I'))`

Input : Interval representation (p, I) of some $r \in (\mathbb{R} \cap \bar{\mathbb{Q}}) \setminus \mathbb{Q}$ and interval representation (p', I') of some $r' \in (\mathbb{R} \cap \bar{\mathbb{Q}}) \setminus \mathbb{Q}$.
Output : $\sim \in \{<, =, >\}$ such that $r \sim r'$.

```

1 if  $I \cap I' \neq \emptyset$  then
2    $(l, u) := I \cap I'$ 
3   if  $l \in I$  then  $(p, I) := \text{ran\_refine}((p, I), l)$  // Algorithm 2.6
4   if  $u \in I$  then  $(p, I) := \text{ran\_refine}((p, I), u)$  // Algorithm 2.6
5   if  $l \in I'$  then  $(p', I') := \text{ran\_refine}((p', I'), l)$  // Algorithm 2.6
6   if  $u \in I'$  then  $(p', I') := \text{ran\_refine}((p', I'), u)$  // Algorithm 2.6

  // Now,  $I$  and  $I'$  are either equal or disjoint.
7 if  $I = I'$  then
8   if  $p = p'$  then
9     // The polynomials and intervals are equal.
10    return =
11  else
12     $q := \text{gcd}(p, p')$  // Greatest common divisor of  $p$  and  $p'$ .
13     $(l, u) := I$ 
14    if  $\text{sgn}(q(l)) \neq \text{sgn}(q(u))$  then
15      // The GCD  $q$  has a (single) zero in the common interval.
16      return =
17    else
18      // It holds  $r \neq r'$ . We refine until the intervals become disjoint.
19      while  $I = I'$  do
20        choose  $v \in I$ 
21         $(p, I) := \text{ran\_refine}((p, I), v)$  // Algorithm 2.6
22         $(p', I') := \text{ran\_refine}((p', I'), v)$  // Algorithm 2.6

  // Now,  $I$  and  $I'$  are disjoint.
23 if  $I < I'$  then return <
24 else return >
```

Algorithm 2.7 compares two real algebraic numbers. We omit the comparison of a real algebraic number with a rational, as this case is similar but simpler. As during refinement, it may happen that an interval representation is converted to a rational, we explicitly assume that the represented value is non-rational to simplify the presentation.

Algorithm 2.8 chooses a rational sample between two algebraic numbers; again, we focus on the non-trivial case again.

2.6.3 Polynomial Evaluation

Determining the evaluation of a multivariate polynomial at a given point is a costly computation when non-rational numbers are involved. We make use of *interval arithmetic*, which are arithmetic operations on rational intervals. In this context, its relevant operations are given in the following definition.

Algorithm 2.8: `ran_sample_between`((p, I), (p', I'))

Input : Interval representation (p, I) of some $r \in (\mathbb{R} \cap \bar{\mathbb{Q}}) \setminus \mathbb{Q}$ and interval representation (p', I') of some $r' \in (\mathbb{R} \cap \bar{\mathbb{Q}}) \setminus \mathbb{Q}$ such that $r < r'$.

Output : $v \in (r, r') \cap \mathbb{Q}$.

```

1 ( $l', u'$ ) :=  $I'$ 
2 if  $l' \in I$  then ( $p, I$ ) := ran_refine(( $p, I$ ),  $l'$ )           // Algorithm 2.6
3 ( $l, u$ ) :=  $I$ 
4 if  $u \in I'$  then ( $p', I'$ ) := ran_refine(( $p', I'$ ),  $u$ )       // Algorithm 2.6
5 ( $l', u'$ ) :=  $I'$ 
6 choose  $v \in [u, l']$ 
7 return  $v$ 

```

Definition 2.26 (Interval Arithmetic)

Let $(l_1, u_1), (l_2, u_2) \subseteq \mathbb{R}$ be two open intervals, and $\circ \in \{+, -, \cdot\}$. Then

$$(l_1, u_1) \circ (l_2, u_2) = \{v_1 \circ v_2 \mid v_1 \in (l_1, u_1), v_2 \in (l_2, u_2)\},$$

that is,

$$(l_1, u_1) + (l_2, u_2) = (l_1 + l_2, u_1 + u_2)$$

$$(l_1, u_1) - (l_2, u_2) = (l_1 - u_2, u_1 - l_2)$$

$$(l_1, u_1) \cdot (l_2, u_2) = (\min\{l_1 l_2, l_1 u_2, u_1 l_2, u_1 u_2\}, \max\{l_1 l_2, l_1 u_2, u_1 l_2, u_1 u_2\}).$$

Algorithm 2.9 again assumes that the point where we evaluate the polynomial consists of non-rational coordinates only; rational values can easily be substituted beforehand. It first computes some polynomial $q(y)$ which has a root at the desired value. As the defining polynomials of the coordinates of the point may have multiple roots, $q(y)$ might have as well. We thus use interval arithmetic and refinement to compute an isolating interval around the desired value. In Algorithm 2.9, we use Algorithm 2.4 to isolate all roots of $q(y)$; in an actual implementation, we would use the techniques from the bisection algorithm (e.g. root counting) directly to isolate a single root only.

Arithmetic operations on algebraic numbers can be realized using similar techniques, as described in [Mis93], but are not required in the context of this thesis and thus omitted here.

Often, we only need to determine the *sign* of the evaluation of some polynomial at a given point, which may be more efficient in some cases: We may use refinement and interval arithmetic to determine whether the sign is negative or positive, already before we compute q . If the evaluation is close or equal to zero, we still need to compute q . However, we may use *lower (upper) bounds on the positive (negative) roots* which may allow stopping the refinement loop earlier, as we do not need the exact value, but only its sign.

2.6.4 Multivariate Real Root Isolation

An important step in the CAD is the isolation of real roots of some polynomial after substituting a partial point to obtain some univariate polynomial. While this step is trivial for rational points, it is again non-trivial and costly if non-rational numbers are involved.

Algorithm 2.9: `ran_evaluate_poly(p, S)`

Input : Polynomial $p \in \mathbb{Q}[x_1, \dots, x_n]$, sequence $S = ((p_1, I_1), \dots, (p_n, I_n))$ of interval representations of some $r^1, \dots, r^n \in (\mathbb{R} \cap \bar{\mathbb{Q}}) \setminus \mathbb{Q}$.

Output : Interval representation (p, I) of $v = p(r^1, \dots, r^n) \in (\mathbb{R} \cap \bar{\mathbb{Q}})$.

// Construct poly $q(y, x_1, \dots, x_n)$ such that y is the value of $p(r^1, \dots, r^n)$.

```

1  $q(y, x_1, \dots, x_n) := y - p(x_1, \dots, x_n)$ 
  // Eliminate variables  $x_1, \dots, x_n$  from  $q$  by resolution with the defining
  polynomials  $p_1(x_1), \dots, p_n(x_n)$ .
2 foreach  $i = n, \dots, 1$  do
  // The polynomials  $q$  and  $p_i(x_i)$  do not have a common factor as  $y$  has
  coefficient 1, thus  $\text{res}_{x_i}(q, p_i(x_i))$  does not vanish.
3    $q := \text{res}_{x_i}(q, p_i(x_i))$ 
  // The result  $q(y)$  is univariate and has a root at  $v = p(r^1, \dots, r^n)$ .
4 Replace  $q$  by its square-free factorization.
5  $R := \text{ran\_real\_roots}(q(y))$  // Algorithm 2.4
6  $I := p(I_1, \dots, I_n)$  // Interval arithmetic, Definition 2.26
  // Refine  $I$  until we refine a single root.
7 while there exist  $\mathbf{r}, \mathbf{r}' \in R, \mathbf{r} \neq \mathbf{r}'$  s.t. the values represented by  $\mathbf{r}$  and  $\mathbf{r}'$  are contained in  $I$  do
  // Algorithm 2.7
8   foreach  $i \in [n]$  do
9     choose  $v' \in I_i \cap \mathbb{Q}$ 
10     $(p_i, I_i) := \text{ran\_refine}((p_i, I_i), v')$  // Algorithm 2.6
11     $I := p(I_1, \dots, I_n)$  // Interval arithmetic, Definition 2.26
12 return  $(q, I)$ 

```

Algorithm 2.10 assumes that each coordinate of the given point is non-rational. It eliminates the variables from the input polynomial by resolution with the defining polynomials of the coordinates to obtain a univariate polynomial that has (a superset of) the desired roots.

While the idea is simple, there are two obstacles: (1) After the resolution step, the polynomial may vanish. (2) The obtained polynomial might have more roots than the desired roots.

We first examine obstacle (1). If $\bar{p} = 0$ in Line 4, some intermediate resultant had a common factor with one of the defining polynomials $p_1, \dots, p_{n-1}$. The resultant may not truly be nullified, as the defining polynomials may have multiple roots other than the algebraic numbers that we meant. Thus, we check, by evaluation of the coefficients, whether the polynomial truly nullifies in Line 7. Otherwise, we evaluate the first non-vanishing coefficient c_i and evaluate it at $r^1, \dots, r^{n-1}$. The resulting defining polynomial p_y does not have a root at 0 by assumption. In the recursive call, the resultant of p with a defining polynomial does not vanish because a coefficient of p w.r.t. x_n is y , meaning there are no common factors. The resultant of the final intermediate result $p(y, x_n)$ with p_y does not vanish because $p(y, x_n)$ is linear in y and 0 is not a root of p_y .

Obstacle (2) is about spurious roots: The resulting polynomial p in Line 15 might have more roots than $p(r^1, \dots, r^{n-1}, x_n)$, we thus check for each root whether it is a genuine one. The reasons for these spurious roots are that they witness intersections with roots of the “other” defining polynomial, intersections of complex roots, and “intersections of intersections” (see also the discussion in Section 3.2.8.3).

Regarding obstacle (1), [Kre20, p. 2.5.2] provides a deeper explanation of the issue

Algorithm 2.10: `ran_real_roots_algebraic(p, S)`

Input : Polynomial $p \in \mathbb{Q}[x_1, \dots, x_{n-1}][x_n]$, sequence $S = ((p_1, I_1), \dots, (p_{n-1}, I_{n-1}))$ of interval representations of some $r^1, \dots, r^{n-1} \in (\mathbb{R} \cap \mathbb{Q}) \setminus \mathbb{Q}$.

Output : Set of rationals and interval representations of the elements in $\text{realRoots}(p(r^1, \dots, r^{n-1}))$, or *NULL* if $p(r^1, \dots, r^{n-1}, x_n) = 0$.

// Eliminate variables $x_1, \dots, x_{n-1}$ from p by resolution with the defining polynomials $p_1(x_1), \dots, p_{n-1}(x_{n-1})$.

```

1  $\bar{p} := p$ 
2 foreach  $i = n - 1, \dots, 1$  do  $\bar{p} := \text{res}_{x_i}(\bar{p}, p_i(x_i))$ 
3 if  $\bar{p} \in \mathbb{Q} \setminus \{0\}$  then return  $\emptyset$  // A constant polynomial does not have roots.
4 else if  $\bar{p} = 0$  then
    //  $p$  may be truly nullified at  $r^1, \dots, r^{n-1}$ , or not.
    // Check whether all coefficients of  $p \in \mathbb{Q}[x_1, \dots, x_{n-1}][x_n]$  are 0.
5      $c_d x_n^d + \dots + c_1 x_n^1 + c_0 := p$ 
6      $i := d$ 
    // Check whether  $c_i(r^1, \dots, r^{n-1}) \neq 0$ .
7     while  $\text{ran\_evaluate\_poly}(c_i, ((p_1, I_1), \dots, (p_{n-1}, I_{n-1}))) = 0$  and  $i \geq 0$  do // Algorithm 2.9
         $i := i - 1$ 
8     if  $i < 0$  then return NULL // All coefficients  $c_n, \dots, c_0$  vanish at  $(r^1, \dots, r^{n-1})$ 
    //  $c_i$  is the first non-vanishing coefficient.
9      $(p_y, I_y) := \text{ran\_evaluate\_poly}(c_i, ((p_1, I_1), \dots, (p_{n-1}, I_{n-1})))$  // Algorithm 2.9
10     $q := y x_n^i + c_{i-1} x_n^{i-1} + \dots + c_1 x_n^1 + c_0 \in \mathbb{Q}[y, x_1, \dots, x_{n-1}][x_n]$ 
    // By construction, no resultant will vanish in the following recursive call.
11    return  $\text{ran\_real\_roots\_algebraic}(q(y, x_1, \dots, x_n), ((p_y, I_y), (p_1, I_1), \dots, (p_{n-1}, I_{n-1})))$ 
12 else
    //  $\bar{p}$  is univariate in  $x_n$ .
13     $R := \text{ran\_real\_roots}(\bar{p}(x_n))$  // Algorithm 2.4
    // Remove spurious roots.
14    foreach  $(p_n, I_n) \in R$  do
        // Check whether  $p(r^1, \dots, r^{n-1}, r^n) \neq 0$  (where  $r^n$  is represented by  $(p_n, I_n)$ ).
15        if  $\text{ran\_evaluate\_poly}(p, ((p_1, I_1), \dots, (p_{n-1}, I_{n-1}), (p_n, I_n))) \neq 0$  then
16             $R := R \setminus \{(p_n, I_n)\}$ 
17    return  $R$ 

```

and proposes an alternative solution. Regarding obstacle (2), [Kre20, Theorem 2.1] proposes to use Gröbner bases to mitigate the number of spurious roots (in particular the “intersections of intersections” case).

2.6.5 Note on Resultant Computations

All the computations involving real algebraic numbers and of the CAD involve the computation of resultants; we refer to Section 1.1.4 for a bibliography of the algorithms. These algorithms are generally not cheap: The degree $\deg(\text{res}_x(p, q))$ (in all variables) of the resultant of p and q is quadratic in the maximal degree $\max(\deg(p), \deg(q))$ of p and q [ED16]. The *subresultant pseudo-remainder sequence algorithm* as presented in [Duc00] is implemented in `LibPoly` and `z3`, and has a time complexity of $\mathcal{O}(d^4 \log(dc)^2)$ where $d = \max(\deg_x(p), \deg_x(q))$ and c is the largest coefficient in p and q . Although there exist more efficient algorithms, the resultant computation should be considered expensive, in particular when iterating resultant computations.

Part II

CAD-based Algorithms

Chapter 3

Cylindrical Algebraic Decomposition and Exploration-guided Algorithms

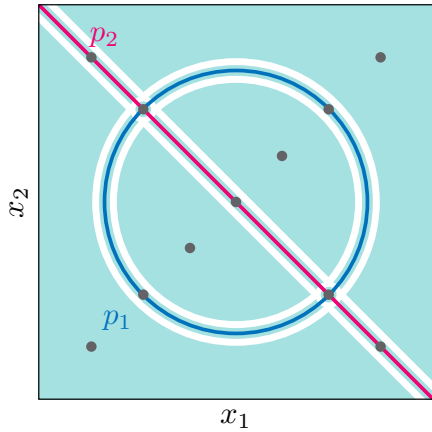
In this chapter, we introduce the cylindrical algebraic decomposition method and derived exploration-guided algorithms. While mathematical papers on this topic are usually built bottom-up, the aim of this chapter is to provide a high-level understanding of these algorithms. We thus follow a top-down approach: we first define what we aim to compute, then give abstract algorithms before elaborating on the theory behind the cylindrical algebraic decomposition. Further, we aim to develop a framework that shows the connections between the exploration-guided algorithms. For a deeper understanding of the projection, we refer to Chapter 4, and to the original CAD literature (as given in Section 1.1.2) for the rigorous mathematical details.

Origins In the following we recall prior work by others as well as by the author. The former is attributed throughout the sections. In Section 3.2, we introduce the cylindrical algebraic decomposition [Col75; McC98] and well-established related notions. While most of these contents stem from prior work, the presentation is novel and adapted to prepare for the following sections. Some definitions are taken from [NÁ⁺24]. Sections 3.2.6.1 and 3.2.6.2 are derived from parts of [NK25]. The discussion in Section 3.2.7 is a novel contribution of this thesis. Section 3.3 introduces the concept of single cell construction, where some parts are derived from [NÁ⁺24] although heavily modified. Section 3.4 introduces the cylindrical algebraic covering (CAIC) method originally introduced in [ÁDE⁺21] for existentially-quantified conjunctions only. The presentation in this thesis is a generalization that supports quantifiers, quantifier elimination, and arbitrary Boolean structure, and is an evolution of [KN23; NK25]. It re-uses parts from Section 3.3 to show the relation of CAIC and single cell construction. Section 3.4.2 is a novel contribution of this thesis and compares the NLSAT algorithm [Jd12] with CAIC. Section 3.5 is a reformulation of the NuCAD algorithm [Bro15] that re-uses parts from Section 3.3; most parts stem from [NÁ25] but were slightly modified and extended. The implicant calculation from Sections 3.6, 3.7.1, and 3.7.2 is a slightly modified version from [NK25]. Section 3.7.3 is a summary of [BNÁ⁺23] where the central theorems and examples were taken over.

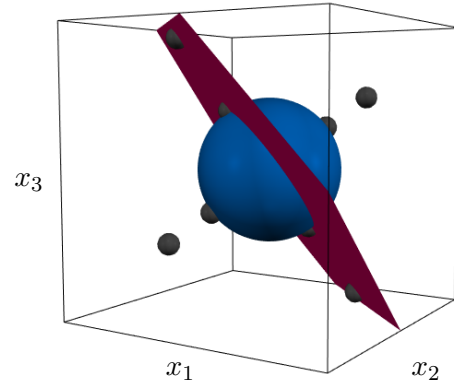
3.1 Introduction

For simplicity, let us first assume that we want to decide whether there exists a solution for a set of polynomial constraints in n variables; we will generalize the ideas to real quantifier elimination later. At first glance, it might be counter-intuitive that we can search the infinite real space $\mathbb{R}^n$ in finite time — given that already representing some irrational number like $\pi \in \mathbb{R}$ would require infinite space. The reasons why we can find such solutions if they exist (or conclude that no solutions exist) are roughly the following: (1) *Univariate* polynomials have *finitely* many roots, since the number of their roots is bounded by their degree. While multivariate polynomials can have infinitely many roots, we can at least describe them using finitely many *root functions* when viewing the polynomial as univariate polynomial in a single main variable with polynomial coefficients. We can use these root functions to decompose the real space into finitely many connected subsets which maintain certain invariance properties w.r.t. the input polynomials. (2) This decomposition is described by roots or root functions of polynomials which are or map to *real algebraic numbers*, which can be represented finitely in a computer.

Let us elaborate the first idea: the key observation is that a polynomial does not change its sign between its roots; to change its sign, we need to move a point such that its value goes through zero, meaning that we cross a root of a polynomial by moving the point. Thus, given a set of polynomials $P \subset \mathbb{Q}[x_1, \dots, x_n]$, we compute a decomposition of $\mathbb{R}^n$ by splitting the space using the roots of the polynomials in P , obtaining a finite set of connected sets (called cells) on which each polynomial in P is sign-invariant; if we choose these sets inclusion-maximal, then these cells are bounded by the surfaces of the polynomials' varieties. Within such a sign-invariant cell, a formula that is built from sign conditions on these polynomials does not change its truth value. It is thus sufficient to check a single point for each cell to determine whether there exists a solution for the input formula.



(a) The sign-invariant cells (coloured areas) and sample points for each cell.



(b) Sample points for each cell. Not visible are three sample points within the sphere (one on the hyperplane, and one on each side of the hyperplane).

Figure 3.1: Sign-invariant cells.

Example 3.1

Consider the polynomials from Example 2.1. In Figure 3.1a we depict the cells where the two polynomials are sign-invariant and a sample point for each cell. In Figure 3.1b we depict only a sample point for each cell.

The (minimal) number of cells of such a decomposition might grow exponentially in the number of variables, e.g. consider the set of polynomials $\{x_1, \dots, x_n\}$: We start with the set of cells $\{\mathbb{R}^n\}$ and iteratively split each cell polynomial-by-polynomial, obtaining 3^n cells, each representing a different sign combination of the polynomials.

The task to compute the decomposition is closely related to its representation. The cells of the decomposition as shown in the examples are bounded by roots of polynomials, which already allows for a finite representation in a computer; we call it an *algebraic* decomposition. It is unclear however how to perform operations on such a general decomposition, e.g. computing a sample point for a given cell from the decomposition. A *cylindrical* algebraic decomposition additionally maintains a certain structure of the cells, allowing for systematic exploration of the search space. The cells forming a CAD are described by a sequence of intervals $x_1 \in I_1, x_2 \in I_2(x_1), \dots, x_n \in I_n(x_1, \dots, x_{n-1})$ where the bounds of I_i depend on $x_1, \dots, x_{i-1}$ for $i \in [n]$. This structure allows for iteratively constructing a sample point in a cell by choosing a value from x_1 , substituting that value into the bounds of I_2 , choosing a value for x_2 , etc.

The following section will formally define a CAD, demonstrate its properties, and explain how it is computed. Afterwards, we proceed with the exploration-guided algorithms that build upon the CAD.

3.2 Cylindrical Algebraic Decomposition

We define a cylindrical algebraic decomposition [Col75; Col98] as the following mathematical object.

Definition 3.1 (Cylindrical Algebraic Decomposition [Col75])

Let $i \in [n]$.

- A *decomposition* of $\mathbb{R}^i$ is a finite set $D \subset 2^{\mathbb{R}^i} \setminus \{\emptyset\}$ such that each $R \in D$ is connected, $\bigcup_{R \in D} R = \mathbb{R}^i$, and either $R = R'$ or $R \cap R' = \emptyset$ for every $R, R' \in D$.
- A set $D \subset 2^{\mathbb{R}^i}$ is *algebraic* if each $R \in D$ is a semi-algebraic cell.
- A set $D_i \subset 2^{\mathbb{R}^i}$ is *cylindrical* if either $i = 1$, or $i > 1$ and $D_{i-1} = \{R \downarrow_{[i-1]} \mid R \in D_i\} \subseteq 2^{\mathbb{R}^{i-1}}$ is a cylindrical set and each $R \in D_i$ is equal to a set of the form $R \downarrow_{[i-1]} \times (-\infty, f)$, $R \downarrow_{[i-1]} \times (f, \infty)$, $R \downarrow_{[i-1]} \times (f, g)$, $R \downarrow_{[i-1]} \times f$, or $R \downarrow_{[i-1]} \times \mathbb{R}$ for some continuous functions $f, g : R \downarrow_{[i-1]} \rightarrow \mathbb{R}$ such that $f < g$.
- A *cylindrical algebraic decomposition (CAD)* of $\mathbb{R}^i$ is a decomposition of $\mathbb{R}^i$ that is algebraic and cylindrical.
- Let $R \subseteq \mathbb{R}^i$ be a cell and $f, g : R \rightarrow \mathbb{R}$ be continuous functions such that $f < g$. We call $R \times \mathbb{R}$ the *cylinder over R* , $R \times f$ a *section over R* , and $R \times (f, g)$, $R \times (-\infty, f)$, $R \times (f, \infty)$, $R \times (f, g)$, $R \times f$, or $R \times \mathbb{R}$ *sectors over R* .

The cells of a cylindrical set are described by their cylinder and continuous functions over that cylinder. In case of a CAD, the cylinders are fully split into disjoint cells, as the following corollary states.

Corollary 3.1 ([Col75])

Let $i \in [n]$ and D_i be a CAD of $\mathbb{R}^i$. Then either

- $i = 1$ and either $D_i = \mathbb{R}$, or there exists a $k \in \mathbb{N}_{>0}$ and $f_1, \dots, f_k \in \mathbb{R}$ such that $f_1 < \dots < f_k$ and $D_i = \{(-\infty, f_1), \{f_1\}, (f_1, f_2), \dots, \{f_k\}, (f_k, \infty)\}$; or
- $i > 1$ and there is a CAD D_{i-1} of $\mathbb{R}^{i-1}$ such that for each cell $R \in D_{i-1}$ it holds either $R \times \mathbb{R} \in D_i$, or there exists a $k \in \mathbb{N}_{>0}$ and continuous functions $f_1, \dots, f_k : R \rightarrow \mathbb{R}$ such that $f_1 < \dots < f_k$ and $R \times (-\infty, f_1), R \times f_1, R \times (f_1, f_2), \dots, R \times f_k, R \times (f_k, \infty) \in D_i$.

We visualize these definitions in Figure 3.2.

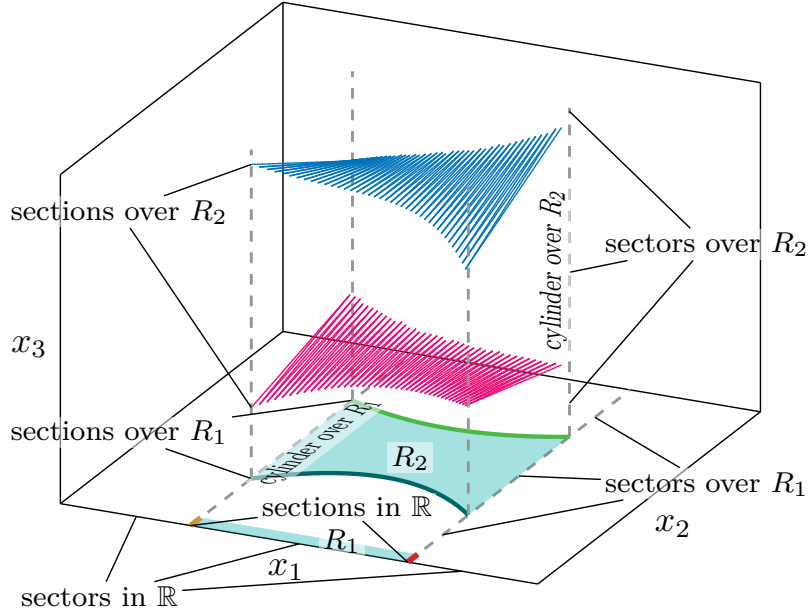


Figure 3.2: Visualization of Definition 3.1.

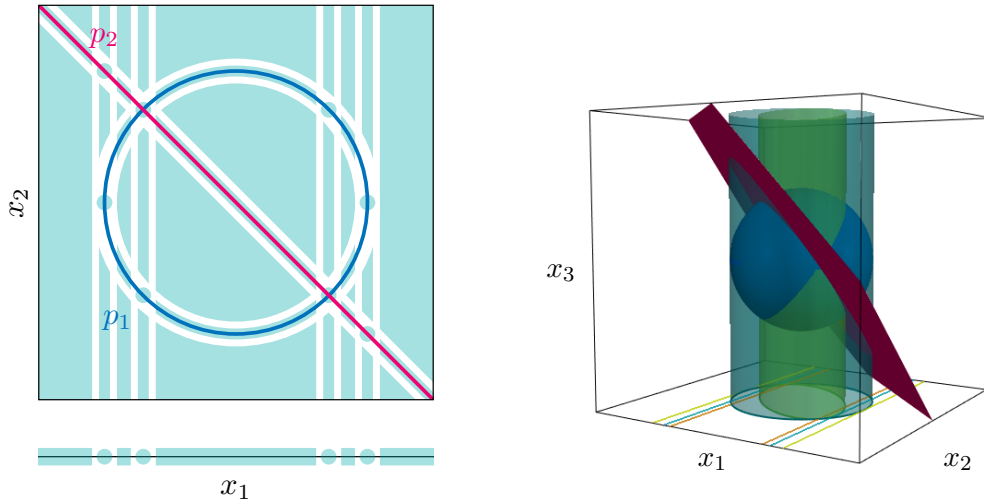
We usually do not deal with arbitrary CADs, but CADs that express properties about polynomial varieties. In particular, we are interested in sign-invariance of some input polynomials.

Definition 3.2 (Sign-invariant CAD)

Let $P \subseteq \mathbb{Q}[x_1, \dots, x_n]$ and D be a CAD of $\mathbb{R}^n$. We call D *sign-invariant for P* if P is sign-invariant in each cell $R \in D$.

Example 3.2

Consider the polynomials from Example 2.1. Figures 3.3a and 3.3b depict sign-invariant CADs.



(a) Sign-invariant CAD of $\mathbb{R}^2$. The cells of the CAD of $\mathbb{R}^2$ are depicted as the coloured areas in the 2-dimensional coordinate system. The cells of the underlying CAD of $\mathbb{R}$ are depicted on the line below. Circles denote zero-dimensional cells (i.e. points).

(b) Sign-invariant CAD of $\mathbb{R}^3$. We do not depict the cells explicitly, but the cell boundaries of the CADs of $\mathbb{R}$, $\mathbb{R}^2$, and $\mathbb{R}^3$ and the cylinder boundaries of $\mathbb{R}^2$ and $\mathbb{R}^3$. The CAD of $\mathbb{R}$ is induced by 6 points (resulting in 13 cells). The CAD of $\mathbb{R}^2$ is induced by the two circles. The CAD of $\mathbb{R}^3$ is induced by the varieties of the two original polynomials.

Figure 3.3: Sign-invariant CADs for Example 3.2.

In Figure 3.3a, the CAD D_1 of $\mathbb{R}$ consists of four points and the intervals between, below, and above them. To obtain the CAD of $\mathbb{R}^2$, each cylinder $C \times \mathbb{R}$, $C \in D_1$ is split into cells where the input polynomials are zero and the cells between, below, and above them. The cylinder and cell boundaries are depicted in white.

In Figure 3.3b, the CAD D_1 of $\mathbb{R}$ consists of six points (the endpoints at the x_1 -axis of the lines in the x_1 - x_2 -plane) and the intervals between, below, and above them. The boundaries of the cylinders in $\mathbb{R}^2$ over the cells in D_1 are depicted by the lines in the x_1 - x_2 -plane. These cylinders are split into cells by the two circles in the x_1 - x_2 -plane, forming the CAD D_2 of $\mathbb{R}^2$. The cylinder boundaries of the cylinders in $\mathbb{R}^3$ over the cells in D_2 are depicted by the transparent tubes over the two circles. These cylinders are split into cells by the variety of the input polynomials, forming the CAD D_3 of $\mathbb{R}^3$.

3.2.1 Idea of the CAD method

The recursive definition of a CAD (Definition 3.1) already hints at how we can compute a sign-invariant CAD for a set of polynomials $P_n \subset \mathbb{Q}[x_1, \dots, x_n]$: We compute a set $P_{n-1} \subset \mathbb{Q}[x_1, \dots, x_{n-1}]$ of *projection* polynomials, compute a sign-invariant CAD D_{n-1} for P_{n-1} , and *lift* D_{n-1} to a sign-invariant CAD D_n for P_n . The central property that the cells of D_{n-1} need to fulfil for the lifting to work is called *delineability* of P_n . Shortly speaking, it states that the graphs of the varieties of P_n do not “cross” over the cells of D_{n-1} .

Definition 3.3 (Delineability [Col98])

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be a cell, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}] \setminus \{0\}$. The polynomial p is called *delineable* on R if and only if there exist continuous functions $\theta_1, \dots, \theta_k : R \rightarrow \mathbb{R}$ (for $k \in \mathbb{N}$) such that

- $\theta_1 < \dots < \theta_k$;
- the real roots of the univariate polynomial $p(r, x_{i+1})$ are $\theta_1(r), \dots, \theta_k(r)$ for all $r \in R$; and
- there exist constants $m_1, \dots, m_k \in \mathbb{N}_{>0}$ such that for all $j \in [k]$, the multiplicity of the root $\theta_j(r)$ of $p(r, x_{i+1})$ is m_j for all $r \in R$.

The θ_j are called *real root functions* of p on R . The cells $R \times \theta_j$, $j \in [k]$ are called *p -sections* over R . The cells $R \times (-\infty, \theta_1)$, $R \times (\theta_k, \infty)$, and $R \times (\theta_j, \theta_{j+1})$ for $j \in [k-1]$, and in case $k = 0$ also $R \times (-\infty, \infty)$, are called *p -sectors* over R .

Let $P \subseteq \mathbb{Q}[x_1, \dots, x_{i+1}] \setminus \{0\}$ be a finite set of polynomials. P is delineable on R if the product of the polynomials in P is delineable on R . We define the real root functions of P on R as the union of the real root functions of $p \in P$ on R , and accordingly the P -sections on R and P -sectors on R ; for the empty polynomial set there are no sections and a single sector $\mathbb{R}^{i+1}$.

Using delineability, we can formally define the projection operation.

Definition 3.4 (Projection Operator)

A *CAD projection operator* is a function $\text{proj} : 2^{\mathbb{Q}[x_1, \dots, x_n]} \rightarrow 2^{\mathbb{Q}[x_1, \dots, x_{n-1}]}$ such that for any $i \in [n-1]$, any cell $R \subseteq \mathbb{R}^i$, and any set of polynomials $P \subseteq \mathbb{Q}[x_1, \dots, x_{i+1}]$ it holds that $\text{proj}(P) \subseteq \mathbb{Q}[x_1, \dots, x_i]$ and

if $\text{proj}(P)$ is sign-invariant on R then P is delineable on R .

Any projection operator that fulfils the above definition is eligible for computing a CAD. On any cell of a CAD of the projection, the input set of polynomials is delineable. This property allows us to lift that CAD to the next level, as we will see after the example.

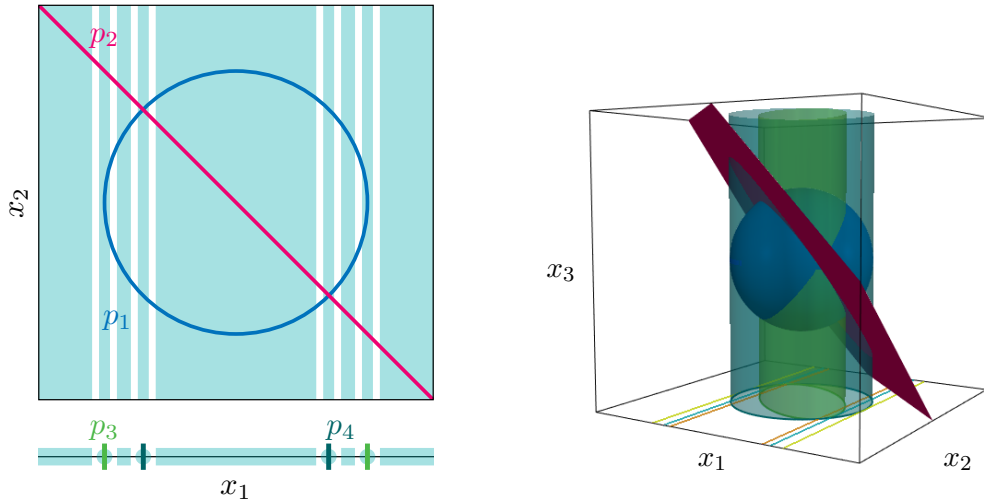
Example 3.3

Consider the polynomials from Example 2.1. Figures 3.4a and 3.4b depict the projection polynomials and the cells over which the polynomials are delineable, along with the corresponding cylinders. In the first dimension x_1 , on any strictly larger cell containing one of the cells, the polynomials are not delineable.

The projection polynomials of the two-dimensional example are $p_3 = \text{disc}_{x_2}(p_1) = 2x_1^2 - 1$ and $p_4 = \text{res}_{x_2}(p_1, p_2) = 2x_1^2 - 1$.

The projection polynomials of the three-dimensional example are $q_3 = \text{disc}_{x_3}(q_1) = -4(x_1^2 + x_2^2 - 1)$, $q_4 = \text{res}_{x_3}(q_1, q_2) = 2x_1^2 + 2x_2x_1 + 2x_2^2 - 1$, $\text{disc}_{x_2}(q_3) = -64(x_1^2 - 1)$, $\text{disc}_{x_2}(q_4) = -4(3x_1^2 - 2)$, $\text{res}_{x_2}(q_3, q_4) = 16(4x_1^4 - 4x_1^2 + 1)$.

The projection operator defines for each finite set of polynomials a sign-invariant CAD in the sense of the following definition.



(a) In the two-dimensional coordinate system, we denote the cylinders where the input polynomials are delineable. In the coordinate system below, we depict the roots of the projection polynomials and the sections and sectors defined by them.

(b) We depict the input polynomials, their projection polynomials in two variables, and the univariate projection polynomials of the latter, as well as the boundaries of the cylinders in which the polynomials are delineable.

Figure 3.4: CAD projection for Example 3.3.

Definition 3.5 (CAD induced by proj)

Let $P \subseteq \mathbb{Q}[x_1, \dots, x_i] \setminus \{0\}$, $i \in [n]$ be a finite set of polynomials. The *induced (sign-invariant) CAD D of P* is defined as follows:

- If $i = 1$, then D is the set of all P -sections and P -sectors.
- If $i \in [2..n]$, let D' be the induced (sign-invariant) CAD of $\text{proj}(P)$. Then D is the set of all P -sections and P -sectors over R' for any $R' \in D'$.

We now examine how we can compute such an induced CAD. For doing so, we need to represent the cells; let us assume for now, that we do not represent them explicitly, but only a set of sample points that witness the cells. A representation can easily be reconstructed from such a set, as we show later. Assume we computed projection polynomials $P_n \subset \mathbb{Q}[x_1, \dots, x_n], \dots, P_1 \subset \mathbb{Q}[x_1]$ — which we explain later.

For the base case $P_1 \subset \mathbb{Q}[x_1]$, we only need to decompose the real line $\mathbb{R}$: We compute the real roots of P_1 (see Section 2.6.1), then the sign-invariant CAD for P_1 consists of the real roots and the open intervals below, above and between the roots. The set of sample points S_1 thus simply contains the roots and a point for each open interval; which we call the *univariate decomposition $\text{ucad}(P_1)$ of the real line w.r.t. the roots of the polynomials in P_1* .

For lifting S_1 to S_2 , we plug in each $s \in S_1$ into P_2 and compute the univariate decomposition $\text{ucad}(\{p(s, x_2) \mid p \in P_2\})$ of the resulting univariate polynomials analogously to the base case; the set S_2 contains all extensions of s with the resulting sample points. That is, $S_2 = \{\{s\} \times \text{ucad}(\{p(s, x_2) \mid p \in P_2\}) \mid s \in S_1\}$. This process is iterated to obtain the sample points for S_n .

The scheme of the CAD algorithm is shown in Figure 3.5.

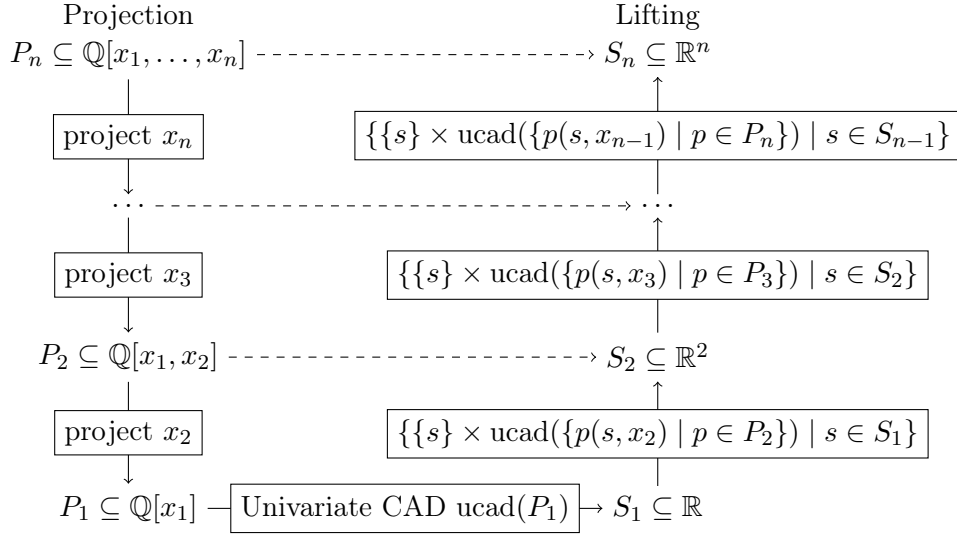
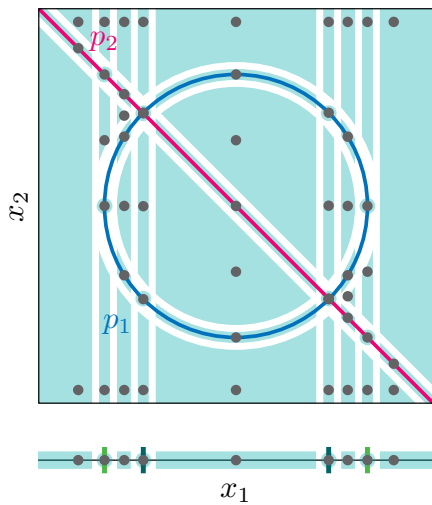


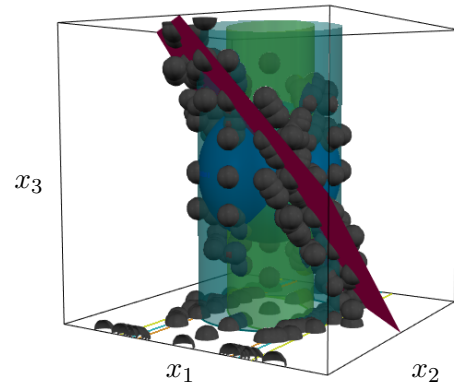
Figure 3.5: Scheme of a CAD computation. We start by the projection variable-by-variable (from top to bottom on the left) until we obtain a set of univariate polynomials, which we use to decompose the real line (from left top right at the bottom). Then, we continue with the lifting phase (from bottom to top on the right) to obtain the final set of sample points.

Example 3.4

Consider the polynomials from Example 2.1. Figures 3.6a and 3.6b depict sample points for the sign-invariant CADs.



(a) Sample points of the sign-invariant CAD of $\mathbb{R}^2$ and the underlying CAD of $\mathbb{R}$.



(b) Sample points of the sign-invariant CAD of $\mathbb{R}^3$ and the underlying CADs of $\mathbb{R}^2$ and $\mathbb{R}$.

Figure 3.6: Sample points for sign-invariant CADs for Example 3.4.

3.2.2 Representation of a CAD

For representing cells explicitly, we need to represent the functions that bound the cells of a CAD. As each (input or projection) polynomial is delineable over the underlying cell R , its variety is described by a fixed number of root functions. Thus, we can refer to the real root functions $\theta_j : R \rightarrow \mathbb{R}$ from Definition 3.3 by the polynomial and the index of the root.

Definition 3.6 (Indexed Root Expression [Bro99])

Let $i \in \mathbb{N}$, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$, and $j \in \mathbb{N}_{>0}$. The *indexed root expression* $\text{root}_{x_{i+1}}^{p,j} : \mathbb{R}^i \hookrightarrow \mathbb{R}$ maps s to the j -th real root of the univariate polynomial $p(s, x_{i+1})$ if it exists, and is not defined (*undef*) otherwise. That is for each $s \in \mathbb{R}^i$:

$$\text{root}_{x_{i+1}}^{p,j}(s) = \begin{cases} \text{undef} & \text{if } j > |\text{realRoots}(p(s, x_{i+1}))| \text{ or } p(s, x_{i+1}) = 0, \\ \xi_j & \text{where } \text{realRoots}(p(s, x_{i+1})) = \{\xi_1, \dots, \xi_k\}, \xi_1 < \dots < \xi_k \text{ and } j \in [k]. \end{cases}$$

The above is an *indexed root expression of level $i + 1$* . We use $\text{root}_{x_{i+1}}^{p,j} \cdot p = p$ and $\text{root}_{x_{i+1}}^{p,j} \cdot j = j$.

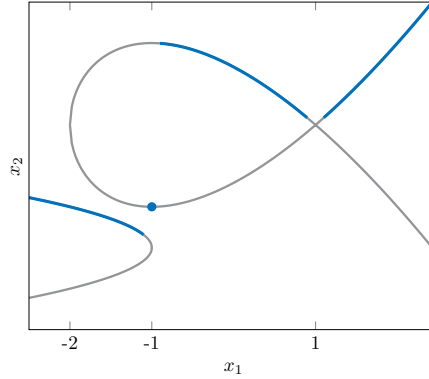


Figure 3.7: Illustration for Example 3.5. The variety of the defining polynomial p is depicted in grey, the graph of the indexed root expression $\text{root}_{x_2}^{p,2}$ in thick blue. Note that the indexed root expression refers to the index of a root, not considering multiplicities.

Example 3.5

Consider the polynomial $p = (x_2^2 - x_1^3 + 3x_1 - 2)(-(x_2 + 3)^2 - x_1 - 1)$ and the indexed root expression $\text{root}_{x_2}^{p,2}$, as depicted in Figure 3.7. Due to the number of roots of p changing, the domain of the indexed root expression is $(-\infty, 1) \cup (1, \infty)$ (as there is only one root at 1) and has a discontinuity at -1 (as there are more than two roots, but the number of roots change at -1).

An indexed root expression can be evaluated by real root isolation. Given a cell $R \subseteq \mathbb{R}^i$ where a polynomial $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ is delineable and a reasonable j , $\text{root}_{x_{i+1}}^{p,j}|_R$ is equal to a real root function of p . However, the semantics might not describe a proper root function outside R , as depending on the point, its value might not be defined or

refer to different roots of the polynomial. It is thus crucial that such an expression is evaluated in the right context, as shown by the following example.

Given the indexed root expressions, we are able to encode CAD cells. While CAD cells of $\mathbb{R}^1$ are simply expressed as intervals with real algebraic endpoints, for CAD cells in higher dimensional space, we need to use the cylindricity of the CAD for encoding their bounds: each cell can be represented by a sequence of *symbolic* intervals (i.e. an interval $I_1 \subseteq \mathbb{R}$ bounding x_1 , an interval $I_2(x_1)$ bounding x_2 that depends on x_1 , etc.) with indexed root expressions at their endpoints.

Definition 3.7 (Symbolic Interval)

A *symbolic interval* I of level i is of the form $I = (l, u)$, $I = [l, u]$, $I = [l, u)$, $I = (l, u]$, $I = (-\infty, u)$, $I = (l, \infty)$, $I = (-\infty, u]$, $I = [l, \infty)$, $I = (-\infty, \infty)$ where l and u are indexed root expressions of level i . The entry $I.l$ ($I.u$) refers to l resp. $-\infty$ (u resp. ∞). For boundaries defined by indexed root expressions, the polynomials $I.l.p$, $I.u.p$ are the *defining polynomials* of I .

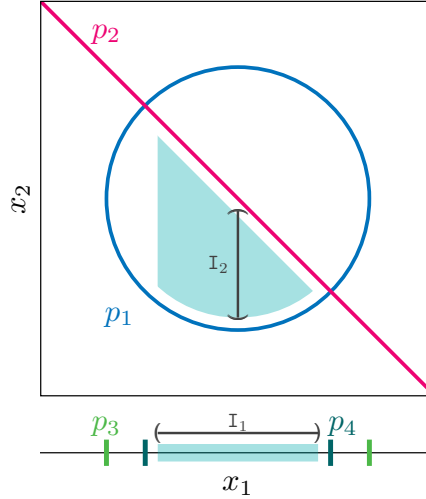


Figure 3.8: Representation of a cell using symbolic intervals as in Example 3.6.

Example 3.6

Consider the two-variable polynomials from Example 2.1 and the CAD discussed in the previous examples. Figure 3.8 depicts a cell $I_1 \times I_2$ which is described by the symbolic intervals $I_1 = (\text{root}_{x_1}^{p_4,1}, \text{root}_{x_1}^{p_4,2}) = (-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ and $I_2 = (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_2,1})$.

Further, the cylindricity of the CAD imposes a certain arrangement of its cells which allows encoding it using a tree-like data structure.

Definition 3.8 (Cylindrical Set Data Structure)

A *cylindrical set data structure* T of level $i \in [n]$ is a sequence of pairs (I, T) where I is a symbolic interval of level i and T is either $\emptyset$ or a cylindrical set data structure of level $i + 1$ (if $i < n$). In the former case, we call $(I, \emptyset)$ a *leaf*; in the latter case, we call (I, T) an *inner node*.

For a cylindrical set data structure T_i of level i , a *path in* T_i is a sequence $(I_i, \dots, I_n)$ such that $(I_i, T_{i+1}) \in T_i, (I_{i+1}, T_{i+2}) \in T_{i+1}, \dots, (I_n, \emptyset) \in T_n$.

For a cylindrical set data structure T_1 of level 1, we define $\text{expansion}(T_1) = \{I_1 \times \dots \times I_n \mid (I_1, \dots, I_n) \text{ is a path in } T_1\}$. For a cylindrical set data structure T_i of level $i > 1$ and $R \subseteq \mathbb{R}^{i-1}$, we define $R \times \text{expansion}(T_i) = \{R \times I_i \times \dots \times I_n \mid (I_i, \dots, I_n) \text{ is a path in } T_i\}$.

If we are given a cylindrical set data structure T that represents a CAD, that CAD is given by $\text{expansion}(T)$.

Example 3.7

Consider the CAD of $\mathbb{R}^2$ from the previous examples. Figure 3.9 depicts a (partial) tree that represents the CAD.

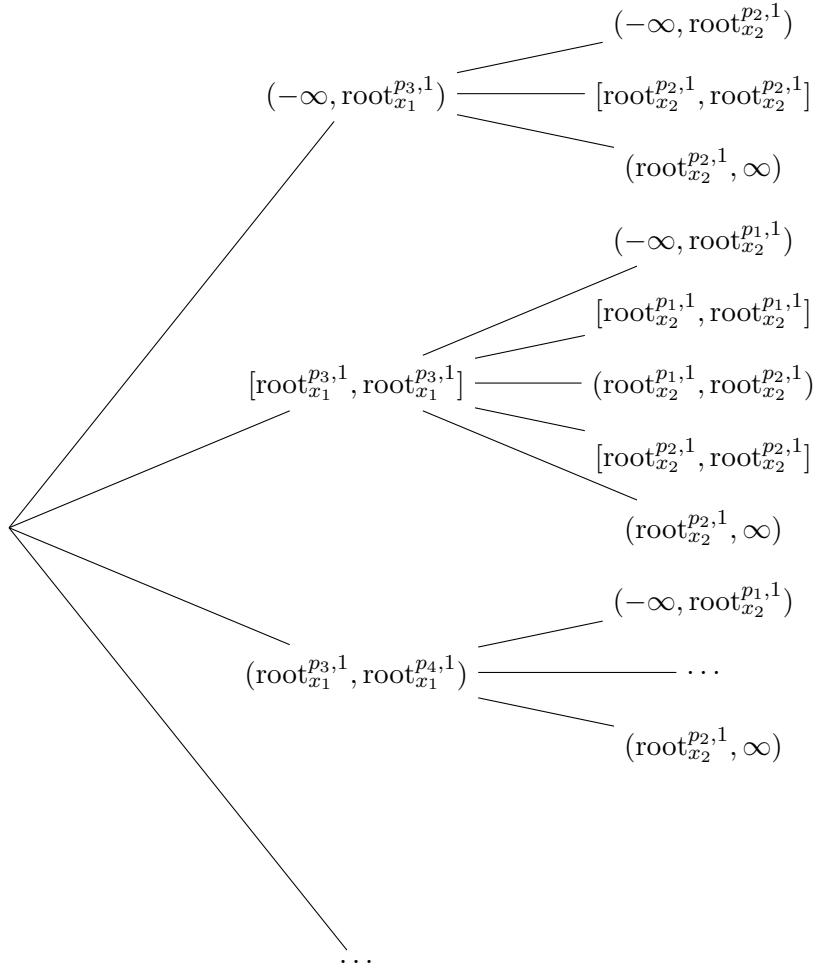


Figure 3.9: A cylindrical set data structure that represents a CAD for Example 3.7.

3.2.3 Computing a CAD

Before finally giving the algorithm of the CAD method, we need the following auxiliary definition.

Definition 3.9

Let $i \in [n]$, $s \in \mathbb{R}^i$, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ of level $i+1$, $P \subseteq \mathbb{Q}[x_1, \dots, x_{i+1}]$, and $r \in \mathbb{R}$.

- The set of indexed root expressions of p at s is defined as $\text{ire}(p, s) = \emptyset$ if $p(s, x_{i+1}) = 0$, and $\text{ire}(p, s) = \{\text{root}_{x_{i+1}}^{p,j} \mid j \in [|\text{realRoots}(p(s, x_{i+1}))|]\}$ otherwise.
- The set of indexed root expressions of P at s is defined as $\text{ire}(P, s) = \bigcup_{p \in P, \text{level}(p)=i+1} \text{ire}(p, s)$.
- The set of indexed root expressions of P at s and r is defined as $\text{ire}(P, s, r) = \{\xi \in \text{ire}(P, s) \mid r = \xi(s)\}$.

Algorithm 3.1 computes the description of a sign-invariant CAD defined by a projection operator with respect to a set of polynomials.

Algorithm 3.1: cad(P)

Input : Finite $P \subseteq \mathbb{Q}[x_1, \dots, x_n] \setminus \{0\}$.
Output : Cylindrical set data structure T such that $\text{expansion}(T)$ is a sign-invariant CAD for P .
 /* Projection Phase */

```

1  $P_n := P$ 
2 for  $i = n - 1, \dots, 1$  do
3    $P_i := \text{proj}(P_{i+1})$ 
4    $P_{i+1} := \{p \in P_{i+1} \mid \text{level}(p) = i + 1\}$ 
  /* Lifting Phase */
5  $T = \{()\}$ 
6 for  $i = 1, \dots, n$  do
7   foreach path  $I_1, \dots, I_{i-1}$  in  $T$  do                                // empty path () if  $i = 1$ 
8     choose  $s \in (I_1 \times \dots \times I_{i-1}) \cap (\mathbb{R} \cap \bar{\mathbb{Q}})^{i-1}$           // choose  $s = ()$  if  $i = 1$ 
9      $\Xi := \bigcup_{p \in P_i} \text{realRoots}(p(s, x_i))$ 
10    sort elements of  $\Xi = \{\xi_1, \dots, \xi_k\}$  such that  $\xi_1 < \dots < \xi_k$ 
11    foreach  $I \in \{(-\infty, \xi_1), [\xi_1, \xi_1], (\xi_1, \xi_2), \dots, (\xi_k, \infty)\}$  for  $k > 0$  and  $I \in \{(-\infty, \infty)\}$  for
       $k = 0$  do
12      if  $I = (-\infty, \xi_\ell)$  then  $I_i := (-\infty, u)$  with  $u \in \text{ire}(P, s, \xi_\ell)$ 
13      else if  $I = (\xi_\ell, \xi_{\ell'})$  then  $I_i := (l, u)$  with  $l \in \text{ire}(P, s, \xi_\ell), u \in \text{ire}(P, s, \xi_{\ell'})$ 
14      else if  $I = [\xi_\ell, \xi_\ell]$  then  $I_i := [b, b]$  with  $b \in \text{ire}(P, s, \xi_\ell)$ 
15      else if  $I = (\xi_\ell, \infty)$  then  $I_i := (l, \infty)$  with  $l \in \text{ire}(P, s, \xi_\ell)$ 
16      else if  $I = (-\infty, \infty)$  then  $I_i := (-\infty, \infty)$ 
17      insert path  $I_1, \dots, I_{i-1}, I_i$  into  $T$ 
18 return  $T$ 
  
```

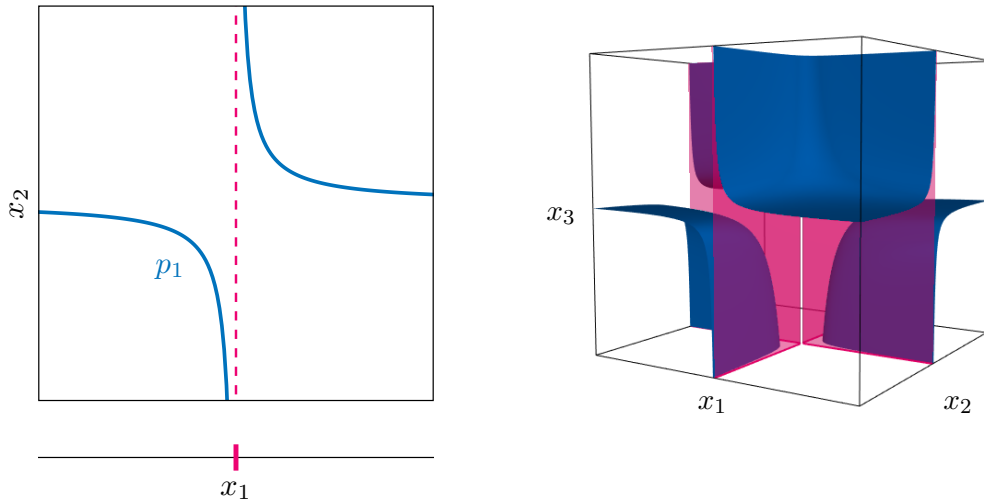
First, the projection operator is applied from level n to 1, called the *projection phase*. Starting from the top level n , we iteratively apply the projection operator to the set of polynomials in Line 3, obtaining a set of polynomials in one variable less. In Line 4, we ensure that all polynomials are sorted by their level.

The projection phase is followed by the *lifting phase* where, starting from level 1, all cells from level $i - 1$ are extended to level i by the sections and sectors above them. To achieve this, for each cell $R = I_1 \times \dots \times I_{i-1}$, the (projection and input) polynomials on level i are partially evaluated up to their last dimension using a sample $s \in R$ (chosen in Line 8). This results in univariate polynomials whose roots can be isolated (Line 9) and sorted (Line 10) to obtain (non-symbolic) intervals: point intervals at the roots of the polynomials, open intervals between them, and two open intervals below

and above all roots (Line 11) — or $(-\infty, \infty)$ if the polynomials have no roots at s . Delineability allows generalizing these intervals over R using symbolic intervals which are bounded by indexed root functions of the polynomials vanishing at the interval endpoints (Lines 12 to 16). Note that in the latter step, the choice of the bounding root function is not unique as the graphs of two polynomials may intersect over the given cell.

3.2.4 Projection Operation

The projection operation is the heart of the CAD algorithm and is crucial for its performance. Collins [Col75; Col98] introduced the first projection, which is complete but expensive. The projection operation was improved over time and is subject to research up to today; we refer to Section 1.1.2 for an overview. Throughout this thesis, we use McCallum’s projection operator [McC85; McC88; McC98] with the optimization by Brown [Bro01]. Although this operator fails when some polynomial nullifies over some point (in which case the CAD algorithm returns a failure), it has been proven to be practically efficient [VKÁ17]. Further, the projection operator is similar to Lazard’s projection [Laz94; MPP19] which is similarly efficient and complete but requires greater implementation effort.



(a) The real roots of the leading coefficient of p_1 w.r.t. x_2 depicted in $\mathbb{R}$. (b) The real roots of the leading coefficient of q_1 w.r.t. x_3 depicted in $\mathbb{R}^2$.

Figure 3.10: Leading coefficients of the polynomials from Example 3.8.

According to Definition 3.4, the projection operation must guarantee that a set of polynomials $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$ is delineable over each cell of a CAD of $\text{proj}(P) \subseteq \mathbb{Q}[x_1, \dots, x_{i-1}]$. To do so, the operator needs to catch all interesting points of the varieties of the polynomials in P :

- The *intersections* of the variety graphs of all pairs of polynomials p_1 and p_2 , which are identified by the resultants $\text{res}_{x_i}(p_1, p_2)$, see Example 2.2;
- the *self-intersections* and *endpoints* of the variety graphs of each polynomial p , which are identified by the discriminants $\text{disc}_{x_i}(p)$, see Example 2.3; and

- the *singularities* of the variety graphs of each polynomial p , which are identified by the leading coefficients $\text{ldcf}_{x_i}(p)$, see Example 3.8 below.

Example 3.8

Consider the polynomial $p_1 = x_1x_2 - 1$ and its leading coefficient $\text{ldcf}_{x_2}(p_1) = x_1$ whose variety is depicted in Figure 3.10a and the polynomial $q_1 = x_1x_2x_3 - 1$ and its leading coefficient $\text{ldcf}_{x_3}(q_1) = x_1x_2$ whose variety is depicted in Figure 3.10b.

McCallum's projection operator with Brown's optimization is then defined as follows.

Definition 3.10 (McCallum-Brown Projection [McC98; Bro01])

Let $i \in [2..n]$, $P \subseteq \mathbb{Q}[x_1, \dots, x_i] \setminus \{0\}$, and $Q = \bigcup_{p \in P} \text{factors}(p)$. Then the *McCallum-Brown projection operator* is defined as

$$\begin{aligned} \text{proj}_{\text{mc}}(P) = & \bigcup_{\substack{p \in Q, \\ \text{level}(p)=i}} \{\text{ldcf}_{x_i}(p)\} \cup \bigcup_{\substack{p \in Q, \\ \text{level}(p)=i}} \{\text{disc}_{x_i}(p)\} \\ & \cup \bigcup_{\substack{p_1, p_2 \in Q, p_1 \neq p_2, \\ \text{level}(p_1)=\text{level}(p_2)=i}} \{\text{res}_{x_i}(p_1, p_2)\} \cup \bigcup_{\substack{p \in Q, \\ \text{level}(p) < i}} \{p\}. \end{aligned}$$

Note that we first compute the irreducible factors Q of all polynomials in P before computing the projection. The reason is that the discriminant of a non-square-free polynomial and the resultant of two polynomials that share a common factor are identically zero; irreducible factors fulfil these requirements. In the literature, usually only a *square-free basis* instead of a full factorization is required, meaning that all polynomials are square-free and do not have common factors; this basis is cheaper to compute than a full factorization, and already satisfies the above requirements. However, the computation of the actual projection tends to be more efficient when using the full factorization — in particular in the context of the work presented in this thesis.

Towards proving the correctness of the McCallum-Brown projection as defined in Definition 3.10, we need to deviate from the framework defined in Definition 3.4. The theory behind McCallum's projection allows for smaller sets of projection polynomials compared to Collins' original operator. However, this theory requires proving that the CAD induced by his projection is not only sign-invariant, but also *order-invariant*. In order to lift the order-invariance property from a CAD of the projection polynomials $\text{proj}(P)$ to a CAD of P , we need to prove that P is *analytically delineable* over any order-invariant cell of $\text{proj}(P)$, which is a stronger property than delineability and defined as follows.

Definition 3.11 (Analytic Delineability [McC98])

Analytic delineability is defined as delineability in Definition 3.3, with the modification that it is only defined on connected analytic submanifolds (instead of general cells) and the real root functions are required to be analytic (instead of only continuous).

The following *lifting theorem* states that Definition 3.10 complies with Definition 3.4 when taking the above modifications into account, if no polynomial is nullified over the underlying cell.

Theorem 3.1 (Lifting Theorem for the McCallum-Brown Projection [McC98; Bro01])

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be an analytic submanifold, and $P \subseteq \mathbb{Q}[x_1, \dots, x_{i+1}] \setminus \{0\}$.

If every $p \in \text{proj}_{\text{mc}}(P)$ is order-invariant on R , then every $p \in P$ is either analytically delineable on R or is nullified on every point in R .

If additionally no $p \in P$ is nullified on any point in R , then P is analytically delineable on R and every $p \in P$ is order-invariant on every P -section over R and every P -sector over R .

Proof (Sketch). The set $\text{factors}(P)$ is the finest square-free basis of P .

Every $p \in \text{factors}(P)$ is irreducible, thus its discriminant is non-zero. By assumption, the discriminant of p is order-invariant on R , the leading coefficient of p is sign-invariant on R , and p is not nullified over any point in R . Thus, [Bro01, Theorem 3.1] yields that p is degree-invariant over R . Further, [McC98, Theorem2] yields that p is analytically delineable on R and order-invariant in each p -section over R . Additionally, p is trivially sign-invariant in each of the p -sectors over R , and thus also order-invariant on them.

The general statement for a set of polynomials is then derived in the same way as [McC98, Theorem1] is derived from [McC98, Theorem2]. $\square$

We will go into further detail about the projection operator in Chapter 4.

Towards the implementation, we need to modify Algorithm 3.1: After Line 8, we need to insert the line

if $\exists p \in P. p(s, x_i) = 0$ then return FAIL

that checks if some polynomial is nullified over the current cell (Theorem 3.1 states that a polynomial is nullified over a point in a cell if and only if it is nullified over the whole cell). The work in [McC98] proposes inserting additional *delineating polynomials* to fix some special cases where a polynomial nullifies; we do not make use of this optimization, as we will deal with these cases differently in Section 4.4.

Finally, we mention that besides further projection operators, there exists alternative projection schemes (that do not necessarily strictly comply with Definition 3.4) that handle certain cases more efficiently, for which we refer to the work cited in Section 1.1.2. Some of these optimizations will be introduced in Chapter 4.

3.2.5 Applications

Given a sign-invariant CAD of some polynomials P , we can decide the satisfiability, validity, quantifier elimination, and optimization problem of formulas built from polynomials in P . The CAD describes the sign-invariant cells of P in a way that allows for a systematic exploration of the real space $\mathbb{R}^n$.

Satisfiability of Quantifier-free Formulas Algorithm 3.2 decides the satisfiability of a quantifier-free formula φ when called using $\text{cad_sat}(\varphi, (), \text{cad}(\text{polys}(\varphi)))$. It is a recursive algorithm that takes an input formula φ , an i -dimensional sample point s , and a description of a CAD of the remaining dimensions. It first evaluates φ at s to check whether it is already satisfying, or violates the formula. If none of these cases hold, we iterate through each cell in the current cylinder: For each of them, we choose a sample point and call the algorithm recursively on the sample point and the description of the decomposition over the current cell. If the call found a satisfying sample, we terminate. If no recursive call found a satisfying sample, we conclude that s cannot be extended to satisfy φ .

We remark that although a call to $\text{cad_sat}(\varphi, s, T_i)$ conceptually deals with the formula $\varphi(s_1, \dots, s_i, x_{i+1}, \dots, x_n)$ after plugging in s into φ , we cannot compute this substitution explicitly as s might contain non-rational numbers.¹ Thus, we need to take φ and s as parameters of the algorithm, and cannot pass $\varphi(s_1, \dots, s_i, x_{i+1}, \dots, x_n)$ directly.

Further, the separation of the computation of the CAD and checking for satisfiability is only for the sake of presentation: In any practical implementation, we would interleave the checks with the lifting phase in Algorithm 3.1, potentially reducing the number of lifting steps to be performed by early termination of satisfiable formulas. The same holds for the following algorithms in this section.

Algorithm 3.2: $\text{cad_sat}(\varphi, s, T_i)$

Input : Quantifier-free Tarski formula φ in $x_1, \dots, x_n$, $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [n]$, and cylindrical set data structure T_i of level i such that $s \times \text{expansion}(T_i)$ is a sign-invariant CAD for the polynomials in $\varphi(s)$.

Output : *SAT* if $\varphi(s)$ is satisfiable, *UNSAT* otherwise.

```

1 if  $\varphi(s) = \text{true}$  then return SAT
2 else if  $\varphi(s) = \text{false}$  then return UNSAT
3 else
4   foreach  $(I_{i+1}, T_{i+1}) \in T_i$  do
5     choose  $s' \in (\{s\} \times I_{i+1}) \cap (\mathbb{R} \cap \mathbb{Q})^{i+1}$ 
6     if  $\text{cad\_sat}(\varphi, s', T_{i+1}) = \text{SAT}$  then
7       return SAT
8   return UNSAT

```

Example 3.9

Consider the formula $\varphi(x_1, x_2) = p_1 \leq 0 \wedge p_2 > 0 \wedge p_3 \geq 0 \wedge p_4 \leq 0 \wedge p_5 \leq 0$ built using the following polynomials:

$$\begin{aligned}
p_1 &= -0.006(x_1 - 2)(x_1 + 2)(x_1 - 3)(x_1 + 3)(x_1 - 4)(x_1 + 4) - x_2 \\
p_2 &= (x_1 + 2.5)^2 + (x_2 - 1.5)^2 - 0.25 & p_3 &= (x_1 - 2.5)^2 + (x_2 - 1.5)^2 - 0.25 \\
p_4 &= x_2 - 2.5 & p_5 &= x_1 \\
p_6 &= \text{disc}_{x_2}(p_2) = -4x_1^2 - 20x_1 - 24 & p_7 &= \text{disc}_{x_2}(p_3) = -4x_1^2 + 20x_1 - 24 \\
p_8 &= \text{res}_{x_2}(p_1, p_4) = 0.006x_1^6 - 6.93889 \cdot 10^{-18}x_1^5 - 0.174x_1^4 + 1.464x_1^2 + 2.22045 \cdot 10^{-16}x_1 - 0.956
\end{aligned}$$

¹Note that we can only evaluate polynomial constraints using an assignment for all variables and isolate the real roots of multivariate polynomials given an assignment for all variables except one, as elaborated in Section 2.6.

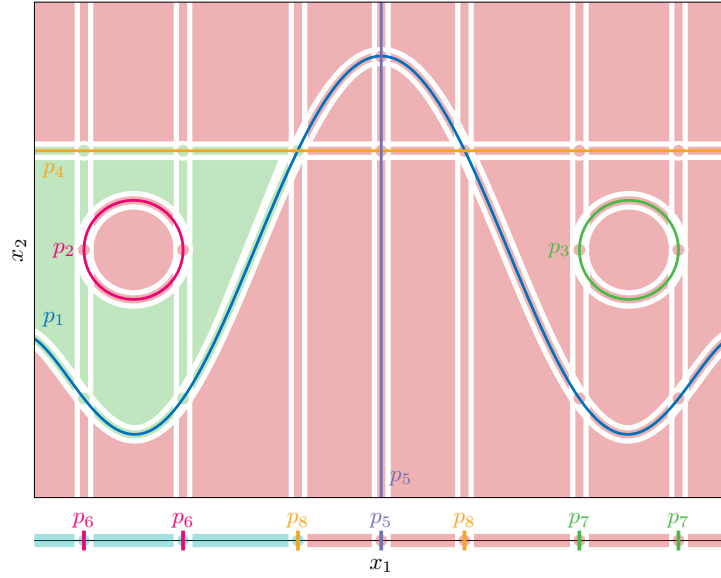


Figure 3.11: CAD illustration for Example 3.9. In the two-dimensional coordinate system, the red cells depict where the formula $\varphi = p_1 \leq 0 \wedge p_2 > 0 \wedge p_3 \geq 0 \wedge p_4 \leq 0 \wedge p_5 \leq 0$ evaluates to *false*, and the green cells where φ evaluates to *true*. In the one-dimensional coordinate system, the red cells depict where φ evaluates to *false*, and the blue cells where φ is satisfiable.

Figure 3.11 illustrates the varieties of the polynomials and their projection, as well as the CAD of $\mathbb{R}$ and $\mathbb{R}^2$.

Further, each cell is coloured according to the truth value (green for *true*, red for *false*, and blue for satisfiable) of φ on that cell, which is determined by plugging in a sample point (not depicted in the figure) from each cell into φ .

If we aim to check whether $\exists x_1. \exists x_2. \varphi$ holds, we only need to find one cell where φ evaluates to *true*. If the algorithm's heuristic is lucky, it might find a such a cell early (e.g. when starting from the leftmost cell for x_1).

Validity and Quantifier Elimination Algorithm 3.3 checks the validity of sentences and eliminates quantifiers in formulas. It works analogously to the previous algorithm: If we run the algorithm on a sentence with only existential quantifiers, the trace of the algorithm is the same. We extend the case distinction in the loop that iterates through the cells of a cylinder. For existentially quantified variables, we terminate early if the recursive call returns *true*. For universally quantified variables, we terminate early if the recursive call returns *false*. For parameter variables, we describe its bounds for the current cell as a formula if the current cell is part of the solution space (that is there are *true* cells in the decomposition over it). The latter step potentially involves indexed root expressions and uses the following encoding.

Definition 3.12 (Encoding of a Symbolic Interval)

Let $i \in [n]$ and I be a symbolic interval of level i . The *encoding* $\text{encoding}(I)$ of I is defined as

- $\text{encoding}([b, b]) = (x_i = b),$
- $\text{encoding}((l, u)) = (l < x_i \wedge x_i < u),$
- $\text{encoding}((-\infty, u)) = (x_i < u),$
- $\text{encoding}((l, \infty)) = (l < x_i),$ and
- $\text{encoding}((-\infty, \infty)) = \text{true}.$

Note that the above definition does not cover all possible forms of symbolic intervals, but the ones relevant for the CAD algorithm.

Algorithm 3.3: $\text{cad_qe}(\varphi, s, T_i)$

Input : Tarski formula $Q_j x_j \dots Q_n x_n \cdot \varphi$ in $x_1, \dots, x_n$ in prenex normal form, $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [n]$, and cylindrical set data structure T_i of level i such that $s \times \text{expansion}(T_i)$ is a sign-invariant CAD for the polynomials in $\varphi(s)$.

Output : Quantifier-free formula ψ in variables $x_{i+1}, \dots, x_{j-1}$ such that $\psi(x_{i+1}, \dots, x_{j-1}) \equiv Q_{\max(j, i+1)} x_{\max(j, i+1)} \dots Q_n x_n \cdot \varphi(s, x_{i+1}, \dots, x_n).$

```

1 if  $\varphi(s) = \text{true}$  then return true
2 else if  $\varphi(s) = \text{false}$  then return false
3 else
4    $\psi := \text{false}$ 
5   foreach  $(I_{i+1}, T_{i+1})$  in  $T_i$  do
6     choose  $s' \in (\{s\} \times I_{i+1}) \cap (\mathbb{R} \cap \bar{\mathbb{Q}})^{i+1}$ 
7      $\text{res} := \text{cad\_qe}(\varphi, s', T_{i+1})$ 
8     if  $i \geq j \wedge Q_i = \exists \wedge \text{res} = \text{true}$  then
9       return true
10    else if  $i \geq j \wedge Q_i = \forall \wedge \text{res} = \text{false}$  then
11      return false
12    else if  $i < j$  then
13       $\psi := \psi \vee (\text{encoding}(I_{i+1}) \wedge \text{res})$ 
14  if  $Q_i = \exists$  then return false
15  else if  $Q_i = \forall$  then return true
16  else return  $\psi$ 
```

Example 3.10

Consider the formula φ from Example 3.9 and assume we aim to check $\forall x_1. \exists x_2. \varphi$. Clearly, the statement does not hold, as e.g. the rightmost cylinder contains only cells where φ evaluates to *false*.

In case of quantifier elimination, the output is not a proper Tarski formula, as it might contain indexed root expressions; we call such formulas an *extended Tarski formula* [Bro99]. The theory corresponding to these formulas is certainly incomplete (we omit the formal definition here), as indexed root expressions are partial functions that are not defined where their defining polynomials have fewer roots than the given root index. However, in this context, the solution space of the formulas returned by Algorithm 3.3 is unambiguously defined as the defining polynomials are delineable on the projection of the cells they describe.

Example 3.11

Consider the formula φ from Example 3.9 and assume we aim to construct the solution formula of $\exists x_2. \varphi(x_1, x_2)$. That is, we want to encode those cells in the CAD of $\mathbb{R}$ above which there are cells on which $\varphi(x_1, x_2)$ evaluates to *true*. We thus obtain

$$\begin{aligned} \exists x_2. \varphi(x_1, x_2) \equiv & x_1 < \text{root}_{x_1}^{p_6,1} \\ & \vee x_1 = \text{root}_{x_1}^{p_6,1} \\ & \vee (\text{root}_{x_1}^{p_6,1} < x_1 \wedge x_1 < \text{root}_{x_1}^{p_6,2}) \\ & \vee x_1 = \text{root}_{x_1}^{p_6,2} \\ & \vee (\text{root}_{x_1}^{p_6,2} < x_1 \wedge x_1 < \text{root}_{x_1}^{p_8,1}) \\ & \vee x_1 = \text{root}_{x_1}^{p_8,1}. \end{aligned}$$

Assume now that we aim to construct the solution formula of $\varphi(x_1, x_2)$. Algorithm 3.3 would obtain a tree-like encoding of the cells highlighted in green Figure 3.11, that is

$$\begin{aligned} \varphi(x_1, x_2) \equiv & (x_1 < \text{root}_{x_1}^{p_6,1} \wedge (\\ & x_2 = \text{root}_{x_2}^{p_1,1} \\ & \vee (\text{root}_{x_2}^{p_1,1} < x_2 \wedge x_2 < \text{root}_{x_2}^{p_4,1}) \\ & \vee x_2 = \text{root}_{x_2}^{p_4,1})) \\ & \vee \dots \end{aligned}$$

It is possible to extend the CAD algorithm to generate proper Tarski formulas. We discuss the solution formula construction further in Section 3.2.6.

Optimization-Modulo-Theories Finally, Algorithm 3.4 demonstrates how the CAD can be used to solve OMT problems. It works analogously to Algorithm 3.2 and assumes that the objective variable is eliminated after all other variables. Thus, when it iterates through the cells in $\mathbb{R}$ ordered by their lower bounds, the first solution that is found already contains the minimal value for the objective variable x_1 .

Algorithm 3.4: $\text{cad_opt_min}(\varphi, T_1)$

Input : Tarski formula $Q_2x_2 \dots Q_nx_n. \varphi$ in $x_1, \dots, x_n$ in prenex normal form, cylindrical set data structure T_1 of level 1 such that $\text{expansion}(T_1)$ is a sign-invariant CAD for the polynomials in φ .

Output : Minimal value $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})$ for x_1 satisfying the formula (if it exists), or $s + \varepsilon$ for the infimum $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})$ of all values for x_1 satisfying the formula (if it exists), or $-\infty$ if the optimal value is unbounded, or *UNSAT*.

```

1 foreach  $(I_1, T_2)$  in  $T_1$  (in the order of their lower bounds) do
2   choose  $s' \in I_1 \cap (\mathbb{R} \cap \bar{\mathbb{Q}})$ 
3    $\text{res} := \text{cad\_qe}(\varphi, s', T_2)$  // Returns either true or false.
4   if  $\text{res} = \text{true}$  then
5     if  $I_1 = (-\infty, u)$  then return  $-\infty$ 
6     else if  $I_1 = (l, u) \vee I_1 = (l, \infty)$  then return  $l + \varepsilon$ 
7     else if  $I_1 = [b, b]$  then return  $b$ 
8 return UNSAT

```

3.2.6 Solution Formulas

Algorithm 3.3 produces a solution formula that directly encodes the CAD cells on which a given formula is true. However, this formula is quite large in practice and often larger than necessary; furthermore, the formula contains indexed root expressions, which are not part of the Tarski language (but part of the extended Tarski language). Minimizing the size of solution formulas is common in quantifier elimination; we refer to the literature in Section 1.1.3, and most prominently to [Bro99] for elaborated algorithms on this issue (in particular the *SimpleSF* algorithm described in [Bro99, Section 5.2]). Due to the complexity of this task, we omit their presentation here. Instead, in this section, we aim to present some low-effort techniques for post-processing a CAD before its encoding.

We assume that the parameter space is described by a cylindrical set data structure, where the children of each node are sorted according to their order in the cylinder, and the leaves are labelled according to the truth value of the input formula on the given cell.

Definition 3.13 (Truth-labelled Cylindrical Set Data Structures)

A *truth-labelled cylindrical set data structure* is defined like a cylindrical set data structure (Definition 3.8), but each leaf $(I, \emptyset)$ is extended to $(I, \mathbf{t})$ where $\mathbf{t} \in \{true, false\}$ is a (*truth*) *label*.

For a truth-labelled cylindrical set data structure T_i of level i and $\mathbf{t} \in \{true, false\}$, a *path to $\mathbf{t}$ in T_i* is a sequence $(I_i, \dots, I_n)$ such that $(I_i, T_{i+1}) \in T_i$, $(I_{i+1}, T_{i+2}) \in T_{i+1}, \dots, (I_n, \mathbf{t}) \in T_n$.

Such a data structure could be generated by a modification of Algorithm 3.3, where Line 13 is adapted to build the data structure accordingly, keeping all cells independent of whether they are labelled *true* and *false* (instead of encoding only the *true* cells as formula). Any path $I_1, \dots, I_k$ to a label $\mathbf{t}$ in such a data structure T encodes a cell such that $\varphi(r) \equiv \mathbf{t}$ for each $r \in I_1 \times \dots \times I_k$.

3.2.6.1 Simplification

After running the CAD algorithm, we simplify the resulting data structure by traversing it depth-first. At each node (I, T) , we apply the following steps:

1. We merge neighbouring leaves $(I'_1, \mathbf{t}'_1) \in T$ and $(I'_2, \mathbf{t}'_2) \in T$ of (I, T) . The resulting node $(I', \mathbf{t}'_1)$ is labelled with the common label and the symbolic interval is built from the lower bound of I'_1 and the upper bound of I'_2 . We iterate until we cannot merge any more.
2. If the current node (I, T) only has a single leaf as child (i.e. $T = (((-\infty, \infty), \mathbf{t}'))$), we set $(I, T) = (I, \mathbf{t}')$.

Note that by the merging described in the first step, we remove unnecessary atoms from the output formula: The two corresponding cells are neighbours in the same cylinder, allowing us to remove their bounds. In the second step, the single child corresponds to a cell without bounds, thus the value of the corresponding variable does not affect the output of the algorithm.

The construction of a formula from this data structure is straight-forward by iterating through the tree and encoding the satisfiable cells. However, we propose another optimization during this process, based on the observation that for some subtrees, there are fewer unsatisfying cells than satisfying ones. In those cases, we want to encode the unsatisfying cells instead of the satisfying ones. We encode a leaf in the obvious way, and each inner node (I, T) as follows:

1. We count the number of leaf children labelled with *true* and *false* respectively.
2. If the former number is smaller, we encode all leaf children labelled with *true* and recursively encode all inner children.
3. If the latter number is smaller, we encode all leaf children labelled with *false* and recursively encode all inner children. For the latter, we pass a flag indicating that the encoding result should describe unsatisfying cells.
4. We build the disjunction of the obtained children's encodings. Depending on which of the two cases we entered and the received flag, we negate this disjunction. Afterwards, we build the conjunction of the (possibly negated) disjunction with $\text{encoding}(I)$.

3.2.6.2 Elimination of Indexed Root Expressions

While indexed root expressions are not admitted by the Tarski language, equivalent “pure” Tarski formulas can be constructed with some effort. In the following, we discuss some possibilities for doing so, based on common techniques also used in other algorithms.

Thom's lemma (Theorem 2.1) states that the maximal set satisfying some sign condition of a univariate polynomial and all its derivatives is either empty or connected. This implies that we can describe intervals using constraints whose defining polynomials are derivatives of some polynomial.

Given a formula φ (containing indexed root expressions) that describes a single cell (which can be extracted efficiently from the cylindrical set data structure) and a point $s \in \mathbb{R}^i$ that lies in the cell, we can eliminate all indexed root expressions using a generalization of Thom's lemma to multivariate polynomials obtained by viewing them as univariate polynomials in x_i with polynomial coefficients. We may use this generalization in different ways, resulting in the following three different approaches:

- Let P be the set of defining polynomials of the indexed root expressions in φ , and $P' = \{ \frac{\partial^k}{\partial x_i^k} p(x_1, \dots, x_i) \mid p \in P, i = \text{level}(p), k \in [0.. \deg_{x_i}(p)] \}$ be the set containing all $p \in P$ and all of their partial derivatives $\frac{\partial^k}{\partial x_i^k} p$ with respect to the respective main variable x_i . If we would encode the sign conditions on the polynomials in P' that s satisfies, we would obtain some cell that is a subset of the one described by φ . To fully encode the cell described by φ , we would need to obtain all possible sign conditions on the polynomials in P' which are satisfied somewhere in the cell described by φ . For doing so, we could compute a sign-invariant CAD C of P' and identify the set of cells $C' \subseteq C$ that satisfy φ . For each cell in C' , we identify the sign conditions on the polynomials in P' (which may be identical for two different

cells). To obtain the desired encoding, we take the disjunction of the encodings of the obtained sign conditions.

- We compute the *augmented projection* of P , that is, the closure of the CAD projection and partial derivatives w.r.t. the respective main variable, as described in [Col75, p. 144]. This projection yields a CAD that is *projection definable* [Bro99, Definition 3], i.e. the CAD can be encoded using sign-conditions on the projection factors. The difference to the previous approach is the following: all derivatives are delineable in the respective lower-level cells, meaning that the resulting formula is cylindrically arranged. Thus, the resulting formula might require fewer atoms by encoding the cells in a tree-like manner.
- Using techniques from [Bro99], we can optimize the previous approach by reducing the amount of derivatives that are added to the projection set — not all derivatives are necessary for obtaining a projection-definable CAD.

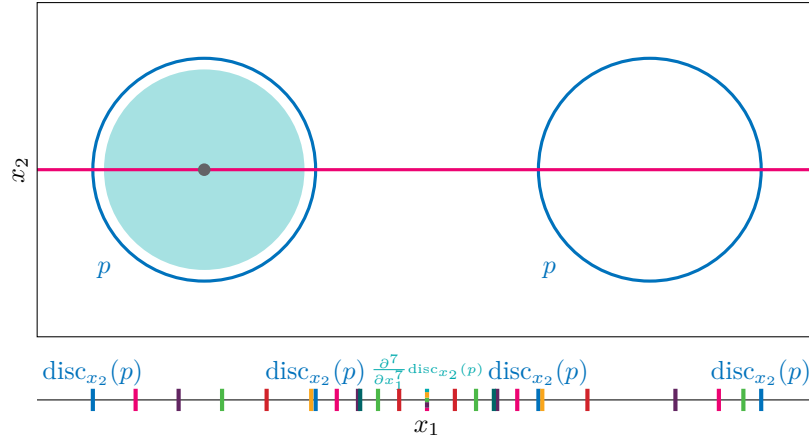


Figure 3.12: Visualization of Example 3.12.

Example 3.12

Consider the polynomial $p = (x^2 + y^2 - 1)((x - 4)^2 + (y)^2 - 1) - 0.0001$ and the sample $s = (0, 0)$, as depicted in Figure 3.12. Assume we aim to encode the highlighted cell (left circle) of the CAD defined by p and its discriminant $\text{disc}_{x_2}(p)$. The signs of the two polynomials are equal in both of the circles, we thus cannot encode the left circle using only sign conditions on p and $\text{disc}_{x_2}(p)$.

Instead, we generate all partial derivatives of p w.r.t. x_2 and all partial derivatives of $\text{disc}_{x_2}(p)$ w.r.t. x_1 , whose roots are depicted in Figure 3.12 as well. Following the first approach described above would split the left circle into multiple cells. However, note that in this example, encoding the sign condition on $\frac{\partial^7}{\partial x_1^7} \text{disc}_{x_2}(p)$ in addition to the ones of p and $\text{disc}_{x_2}(p)$ uniquely describes the desired cell.

The computation of derivatives required in the above approaches might lead to large output formulas and additional expensive CAD computations. It is thus desirable to reduce this effort. This is achieved by not encoding each cell separately, but encoding all cells in combination in order to eliminate redundancies of sign conditions on the derivatives (effectively reducing the amount of required derivatives). The work in [Bro99]

proposes efficient algorithms for this task; thus we could feed the output formula into these algorithms (either all at once or incrementally as in [Bro99, Section 6]).

Certainly, obtaining a small output formula without indexed root expression requires additional computational cost and implementation effort; further, formulas containing indexed root expressions allow for smaller encodings. Moreover, we claim that — for some applications — the CAD data structure itself is a useful for reasoning about the solution space, in particular when the variable ordering reflects the dependencies of the variables.

3.2.7 Rational Functions

Algorithm 3.5: `cad_qe_nra`(φ, s, T_i)

Input : Tarski formula $Q_j x_j \dots Q_n x_n. \varphi$ in $x_1, \dots, x_n$ in prenex normal form, $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [n]$, cylindrical set data structure T_i of level i such that $s \times \text{expansion}(T_i)$ is a sign-invariant CAD for the polynomials in $\varphi(s)$.

Output : Quantifier-free formula ψ in variables $x_{i+1}, \dots, x_{j-1}$ and quantifier-free formula $\psi_{\text{divbyzero}}$ in variables $x_{i+1}, \dots, x_n$ such that (1) for each $s' \in \mathbb{R}^{n-i}$ it holds $\psi_{\text{divbyzero}}(s, s') = \text{true}$ if and only if $\varphi(s, s') = \text{divbyzero}$ and (2) for each $s' \in \mathbb{R}^{\max(0, j-1-i)}$ such that $\psi(s, s') = \text{true}$ if and only if for $Q_{\max(j, i+1)} s''_{\max(j, i+1)} \in \mathbb{R} \dots Q_n s''_n \in \mathbb{R}$ it holds $\varphi(s, s', s''_{\max(j, i+1)}, \dots, s''_n) = \text{true}$.

// We assume that $\varphi(s)$ returns *divbyzero* if the truth value of $\varphi(s)$ cannot be determined due to a division by zero, e.g. $(x \leq 0 \wedge \frac{1}{x} > 0)(0) = \text{divbyzero}$, but $(x \leq 0 \vee \frac{1}{x} > 0)(0) = \text{true}$.

```

1 if  $\varphi(s) = \text{true}$  then return true, false
2 else if  $\varphi(s) = \text{false}$  then return false, false
3 else if  $\varphi(s) = \text{divbyzero}$  then return false, true
4 else
5    $\psi := \text{false}$ 
6    $\psi_{\text{divbyzero}} := \text{false}$ 
7   foreach  $(I_{i+1}, T_{i+1})$  in  $T_i$  do
8     choose  $s' \in (\{s\} \times I_{i+1}) \cap (\mathbb{R} \cap \bar{\mathbb{Q}})^{i+1}$ 
9      $\text{res} := \text{cad\_qe\_nra}(\varphi, s', T_{i+1})$ 
10    if  $i \geq j \wedge Q_i = \exists \wedge \text{res} = (\text{true}, \text{false})$  then
11      return true, false
12    else if  $i \geq j \wedge Q_i = \forall \wedge \text{res} = (\text{false}, \text{false})$  then
13      return false, false
14    else
15       $(\psi', \psi'_{\text{divbyzero}}) := \text{res}$ 
16       $\psi := \psi \vee (\text{encoding}(I_{i+1}) \wedge \psi')$ 
17       $\psi_{\text{divbyzero}} := \psi_{\text{divbyzero}} \vee (\text{encoding}(I_{i+1}) \wedge \psi'_{\text{divbyzero}})$ 
18  if  $Q_i = \exists$  then return false,  $\psi_{\text{divbyzero}}$ 
19  else if  $Q_i = \forall$  then return true,  $\psi_{\text{divbyzero}}$ 
20  else return  $\psi, \psi_{\text{divbyzero}}$ 

```

Although we focus on the theory of the reals in this thesis (i.e. the input formulas assumed to be Tarski formulas), the CAD may be extended for dealing with the theory of non-linear arithmetic allowing *rational functions* (i.e. p/q for some polynomials $p, q \in \mathbb{Q}[x_1, \dots, x_n]$) in the formulas, which is common in SMT solving.

The use of rational functions causes issues for the semantics of the *division-by-zero* case, which makes the theory incomplete, as described in Section 2.4. This issue is frequently discussed in every SMT-related workshop; another introduction to this issue is given

in [DEM⁺23, Chapter 6]. Most SMT solvers implement the semantics described in Section 2.4; while this is straight-forward for purely existentially quantified formulas, it leads to non-trivial adaptations for algorithms dealing with the general case. An extensive survey and a solution to the issue by defining an alternative semantic is given in [Bro25].

Here, we suggest a pragmatic approach. Whenever a division by zero causes uncertainty of the truth value of a formula, we consider this as a modelling error by the user: As the semantics of this case is not well-defined, the user should encode what should happen in this case within the input formula. Thus, if such a case occurs, we should explicitly tell the user about the error, instead of obfuscating the error by some “artificial” semantics of the division-by-zero case.

For doing so, we would compute a sign-invariant CAD for all polynomials occurring in the input formula (in case of rational functions, we consider both the numerator and denominator). We can identify the cells over which a division by zero occurs and label these cells accordingly. Algorithm 3.5 adapts Algorithm 3.3 to report relevant division-by-zero occurrences to the user: We return a tuple of formulas, where the second formula describes all cells where some division by zero occurs, and the first formula is the solution formula of the input if the values described by the second formula are interpreted as *false*.

3.2.8 Complexity and Limitations

As mentioned, the CAD method is the most common complete method in practise. Still, it is doubly exponential in worst case. Thus, one may question whether it is the ultimate choice and suitable for every situation.

3.2.8.1 Complexity of CAD

We first summarize complexity results on deciding real algebra. As outlined in Section 1.1.1, there are algorithms for real algebra which are singly exponential in the number of variables and doubly exponential in the number of quantifier alternations. For instance, the algorithm in [Gri88] has a time complexity of $\log(M)(md)^{(\mathcal{O}(n))^{4a-2}}$, where m is the number of polynomials, n is the number of variables, d is the maximal degree $\deg p$ of a polynomial p , a is the number of quantifier alternations, and M is an upper bound on the absolute value of all coefficients a polynomial. In [DH88; Wei88], a lower bound for real quantifier elimination was proven, which is doubly exponential in the number of quantifier alternations.

The CAD projection does not consider the quantifiers of an input formula; we always compute a sign-invariant CAD for the polynomials in the input formula, independently of whether the input formula is purely existentially quantified or has as many quantifier alternations as variables; the cylindrical arrangement of the cells allows for reasoning about any quantifier alternation after its construction. It is thus obvious that the CAD spends the same effort on ER problems as on real quantifier elimination problems. Collins’ original CAD algorithm for quantifier elimination has a time complexity dominated by $2^{2^{8N}}$ where N is the length of the input formula, as stated in [Col98, Theorem 18].

For a more detailed analysis, McCallum [McC85] introduced the (m, d) -property for bounding the growth during the projection phase.

Definition 3.14 (Combined Degree, (m, d) -Property [McC85])

Let $P \subseteq \mathbb{Q}[X]$.

- The *combined degree* of P is $\max_{x \in X} \deg_x(\prod_{p \in P} p)$.
- The set P has the (m, d) -*property* if there is a partition of P into m subsets, each of combined degree less than or equal to d .

Let P be a set with the (m, d) -property (e.g. P is a union of m sets of polynomials in n variables each of maximal degree d). [EBD15, Theorem 5] states that the number of cells of a sign-invariant CAD for P using McCallum's projection operator is bounded by a term dominated by $(2d)^{2^{n-1}} m^{2^{n-1}} 2^{2^{n-1}-1}$.

The work in [BD07] analyzes whether this bound is realized and show that the variable ordering has a huge impact on the number of cells produced by the CAD: (1) There is a polynomial p for which a variable ordering exists such that any sign-invariant CAD for p (w.r.t. that ordering) consists of constant number of cells (in the number of variables), while there is another variable ordering that yields a doubly exponential number of cells. (2) There is a polynomial such that any sign-invariant CAD has a doubly exponential number of cells — independently of the variable ordering. *These results show that there is a true gap between the time complexity of real algebra (and more specifically the existential theory of the reals) and the time complexity of CAD.*

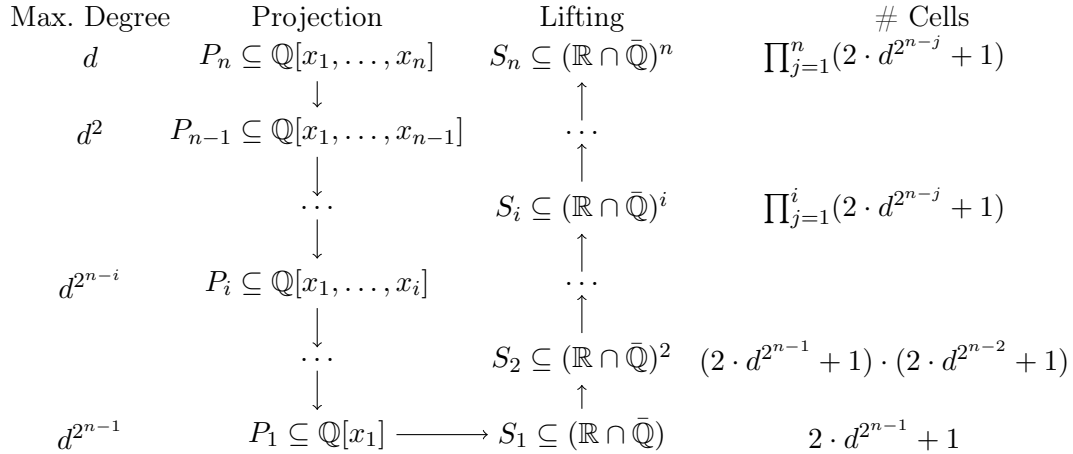
3.2.8.2 Degree Growth

Figure 3.13: Degree growth during the projection phase and number of cells of a CAD (in $\mathcal{O}$ -notation), inspired by [del24; ED16].

During the lifting phase, the number of cells that are constructed is determined by the real roots of the projection polynomials; and the number of roots are bounded by the respective degrees of the polynomials. Thus, it is intuitive to examine the degree growth of the polynomials during the projection phase to estimate the number of cells, as done in [ED16].

In each projection step, we take resultants, discriminants, and coefficients. The resultant of two polynomials of degree at most d is a polynomial of degree at most $2d^2$; similarly, the discriminant of a polynomial of degree d has degree at most $2d^2$. The coefficients

of a polynomial of degree d have degrees at most d and thus do not contribute to the degree growth. During lifting, the number of cells is bounded by twice the number of roots plus one, and the number of roots is bounded by the degree. Figure 3.13 shows the degree growth and number of cells during the CAD computation.

3.2.8.3 Equations

Assume a conjunction containing some equations, then all solutions for that formula must lie on an intersection of the varieties of the polynomials defining the equations. This case is a weakness of the CAD, as it maintains cylindricity, and thus catches more geometric properties than necessary, as shown in the following example.

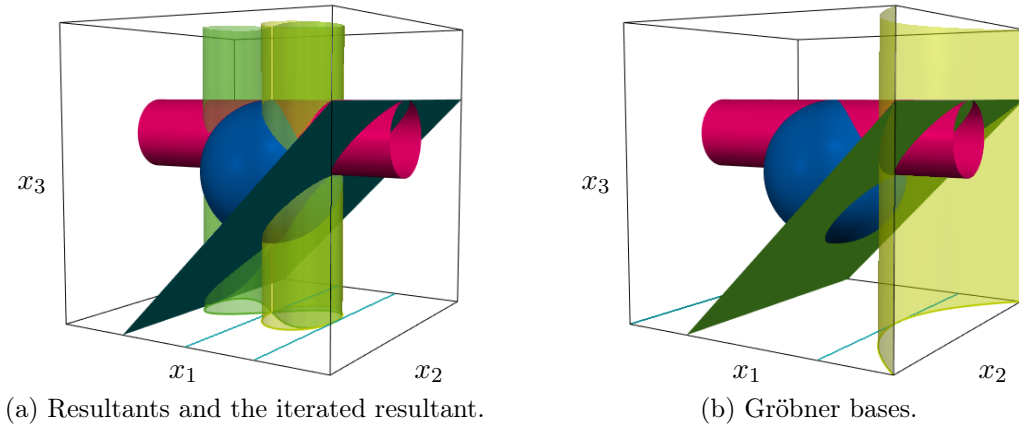


Figure 3.14: Equations in the CAD.

Example 3.13

Consider the polynomials $p_1 = x_1^2 + x_2^2 + x_3^2 - 1$, $p_2 = x_2^2 + (x_3 - 0.5)^2 - 0.25$, and $p_3 = x_1 - x_3 - 1$.

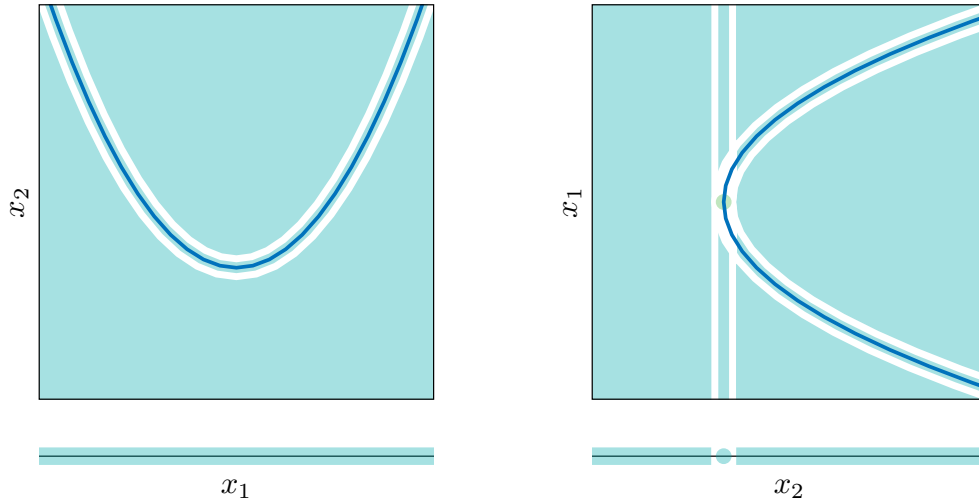
Figure 3.14a depicts p_1 (sphere), p_2 (tube), p_3 (hyperplane), $p_4 = \text{res}_{x_3}(p_1, p_2)$ (infinity shaped), $p_5 = \text{res}_{x_3}(p_1, p_3)$ (ellipses), and the iterated resultant $p_6 = \text{res}_{x_2}(p_4, p_5)$. The polynomials p_1, p_2 , and p_3 have a triple intersection at $(1, 0, 0)$. This triple intersection is witnessed by the intersection of the resultants p_4 and p_5 at $(1, 0)$, which is witnessed by the root of the iterated resultant p_6 at 1. However, p_6 has also a root at 0 which witnesses the intersection of p_4 and p_5 at $(0, 0)$. Above that point, p_1 and p_2 intersect at $(0, 0, -1)$, and p_1 and p_3 intersect at $(0, 0, 1)$ — which is not a triple intersection, but two intersections above each other. If we are only interested in solutions of $p_1 = 0 \wedge p_2 = 0 \wedge p_3 = 0$, using the CAD, we would also lift over (0) and $(0, 0)$, although there is no triple intersection of these polynomials! Figure 3.14b depicts the Gröbner basis $\{x^2 + x - 2, -4x + y^2 + 4, x - z - 1\}$ which describes the common complex roots of p_1, p_2 , and p_3 in a triangular way (whose details we do not introduce in this thesis but refer to [Buc65]). However, it can be seen that this basis does not suffer from spurious roots of the iterated resultants, but describes the solution space of $p_1 = 0 \wedge p_2 = 0 \wedge p_3 = 0$. Note that $x^2 + x - 2$ also has a root at (-2) , which witnesses an intersection of complex roots.

The example shows that Gröbner bases are better suited for equations. In fact, replacing the iterated resultant by a Gröbner basis would reduce the degree growth from doubly to singly exponential [ED16; DEM⁺23]! It thus seems that maintaining cylindricity — which requires considering “intersections of intersections” — causes the doubly exponential complexity.

Possible approaches for improving the CAD for equations are (1) when using Gröbner bases for preprocessing input formulas that contain equations (as suggested in [BH91; WBD12; HED⁺16]), (2) avoiding the use of equations by e.g. stereographic projections as in [SS83], (3) the *equational constraints* projection scheme (see [McC99; McC01; EBD15; EBD20]), and (4) the recently introduced *geometric CAD* [Che25]. We will examine (3) in Section 4.3 in more detail; while this projection scheme does not lower the degree growth during the projection, it is able to reduce the number of constraints [ED16].

3.2.9 Variable Orderings

As stated in the preceding section, the variable ordering can have a high impact on the running time of the CAD algorithm on a given input; a simple example is depicted in Figure 3.15. This importance has been proven experimentally in various works (cited in the following). Here, we aim to give an overview of existing approaches for computing “good” variable orderings for the CAD. A more in-depth introduction is given in e.g. [del24], and a literature overview is provided in Section 1.1.3.



(a) Variable ordering $x_1 \prec x_2$ yields three cells. (b) Variable ordering $x_2 \prec x_1$ yields 9 cells.

Figure 3.15: CAD projection of the same polynomial $x_1^2 - x_2$ with two different variable orderings.

For formulating the above algorithms (Algorithms 3.1 to 3.4), we assume an ordering $x_1 \prec \dots \prec x_n$ on the variables and eliminate higher variables first.

Recall that when we are eliminating quantifiers from a given formula, the variable ordering should respect the input formula’s quantifier structure: We can re-order variables within the same quantifier block, but not variables from different quantifier blocks (otherwise, the resulting formula is not equivalent to the original one). In this section, we assume that there is only a single quantifier block, that is, there are no

restrictions on the order of the variables. Lifting this restriction to the general case is straight-forward.

The variable ordering heuristics can be distinguished by how they are computed: Either the ordering is computed before the CAD projection (*static*) based solely on the set of input polynomials, or during the CAD projection (*dynamic*) based on intermediate projection results; some heuristics require the computation of (multiple) additional CAD projections (*expensive*), or use results that are computed for the CAD anyway (*inexpensive*). Some variants of heuristics fall in several categories.

Feature-based The simplest heuristics to compute variable orderings are due to Brown’s heuristic [Bro04], where we compute a set of features for each variable x with respect to the input polynomials. Brown proposed to use the maximal degree of x over all polynomials, the highest total-degree term in which x appears, and the number of terms containing x . Brown’s heuristic sorts the variables by the first feature in descending order, breaking ties with the second criterion in descending order, and then breaking ties with the third criterion in descending order. The **xai** heuristic [PdE⁺24] is similar and uses features whose importance have been identified using an explainable AI approach.

Estimation of Number of Cells The work in [DSS04] identified that, given a full CAD projection, the *sum of total degrees* (**sotd**) over all monomials in the set of input and projection polynomials correlates empirically with the number of cells of the CAD; it thus suggests choosing the variable ordering with the least number of cells. The (**g**)**mods** heuristic [dE22a] identifies a criterion on the projection polynomials inspired by a complexity analysis of the CAD. For both heuristics, there are variants which require only a single or no projection step respectively.

Variable Structure Most formulas from practical applications have some structure which are reflected by where variables occur. Typically, not all variables appear in all constraints, but constraints can be grouped into different sets sharing some sets of variables; there may be overlaps in the variables between these sets. Based on this grouping, we can often identify an (almost) “triangular” structure, where we can order the groups into a sequence such that group’s variables are contained in the preceding group’s ones; e.g. some constraints contain variables x_1, x_2, x_3 , some constraints contain x_1, x_2 and some only x_1 . The work in [LXZ⁺21] aims to retain this triangular structure throughout the projection phase.

Partial Lifting The **ndrr** heuristic [BDE⁺13b] computes the CAD projection w.r.t. all possible variable orderings and isolates the roots of the univariate polynomials, and takes the ordering with the lowest number of roots of the univariate polynomials. While the projection is expensive, the effort might be reduced during lifting. The **ncoc** heuristic [WEB⁺14] is similar, but during the lifting phase, it only considers the sectors in order to obtain the number of *open cells*. Computing these cells is computationally cheaper than the non-open ones resulting from lifting over sections, as no algebraic number computations are involved in this step (see also Section 3.2.11). It then takes the variable ordering with the lowest number of open cells.

Machine Learning Recent research on variable orderings focuses on machine learning, as it may be able to incorporate more features than human-designed heuristics, see [HEW⁺14; HEW⁺19; DE19; CZC20; FE20; dE24; del24]. However, their success heavily relies on the availability of high-quality (real-world) data.

At the time of writing, relatively simple feature-based heuristics such as **Brown** and **xai** still seem to be competitive and reasonable choices for choosing variable orderings.

3.2.10 Adjacency of Cells

As mentioned in Section 3.2.8, the number of cells of a sign-invariant CAD is up to doubly exponential in the number of variables. However, as already shown in Figures 3.6a and 3.6b, many of these cells are redundant. This is because a section always splits a whole cylinder into three, thereby possibly splitting some cells unnecessarily. Already in the early days of the CAD [Arn81; ACM84; Arn88], there was research on identifying which cells are *adjacent* in order to *cluster* them during the lifting phase: if they maintain the same invariance properties, we only need to compute a single sample point for each cluster (instead of for each cell). We refer to Section 1.1.3 for the bibliography.

Definition 3.15

Let $i \in [n]$ and $R, R' \subseteq \mathbb{R}^i$ be cells. Then R and R' are called *adjacent* if and only if $R \cup R'$ is a cell.

We are usually not interested in the adjacencies in general CADs, but rather adjacencies in the sign-invariant CAD D of $\mathbb{R}^i$ for a set of polynomials P computed by the CAD algorithm. This is because we can use the roots of the polynomials to draw conclusions about the arrangement of the cells. Identifying whether two cells $R, R' \in D$ that are contained in the same cylinder $S \times \mathbb{R}$ over a cell $S \in D'$ of the underlying CAD D' are adjacent is straightforward due to delineability. However, if R and R' stem from different cylinders (e.g. $R \subseteq S$ and $R' \subseteq S'$ for two adjacent cells $S, S' \in D'$), determining their adjacency is non-trivial.

Example 3.14

For example, consider the cells above the variety of p_1 and at the right of p_1 in Figure 3.6a. For detecting adjacency, we need to compute which root functions of p_1 meet, and which one continue to the neighbouring cylinder.

This task requires — in particular in higher dimensional problems — some computational effort and is still an open problem. A partial answer is given in [Str17].

3.2.11 Open CAD

During the computation of the CAD, as discussed in Section 2.6, lifting over sections of polynomials is particularly expensive, as they often can only be described by non-rational real algebraic numbers. Operations involving them are computationally expensive. This motivates restricting the lifting phase to sectors, where all samples can be chosen to be rational. We thus obtain a subset of the CAD, consisting of *open* cells (as they have open bounds on each level) and thus of full dimension. Such a subset of a CAD is called *open CAD* [McC93], an example is given in Figure 3.16.

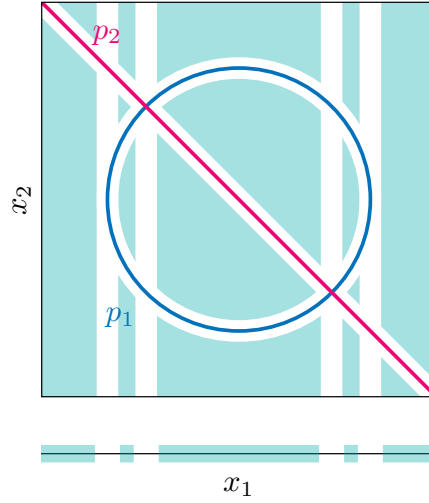


Figure 3.16: Example of an open CAD.

Such a CAD is interesting because it is sufficient for some applications. The satisfiability of formulas in negation normal form where all sign conditions on the polynomials are strict can be determined through the computation of an open CAD alone. This is because the sign conditions already eliminate the real roots of the input polynomials as potential solutions.

3.3 Cylindrical Algebraic Cells

In the last decade, there has been development of *exploration-guided* algorithms that are based on the CAD, but do not compute a full CAD projection. These algorithms switch the lifting and projection phases to perform a conflict-driven search inspired by SAT solvers — they first lift over “guessed” sample points before the projection. The first such algorithm is the NLSAT algorithm [Jd12], followed by the single cell construction [BK15; NÁS⁺24], the NuCAD algorithm [Bro15; Bro17], and the cylindrical algebraic covering method [ÁDE⁺21]; for a complete bibliography, we refer to Section 1.1.3. These algorithms are highly effective in practice, although they have the same theoretical complexity as the CAD. In the following, we will develop a common framework for these algorithms, in order to compare them, and to share projection techniques between them.

These algorithms “guess” sample points and generalize properties of the input polynomials to cells, which are computed using a partial CAD projection. This section deals with such generalizations of a sample to cells, that is, constructing a single cell instead of a whole CAD (also called *single cell construction*).

Example 3.15

Consider the formula from Example 3.9 on Page 70 again. Figure 3.17 depicts a set of cells that cover $\mathbb{R}^2$ such that φ is *truth-invariant* on each of its cells, i.e. φ invariantly evaluates to *true* or *false* on each point for each cell.

We illustrate the idea of this paradigm by sketching an idealistic algorithm that computes such a set of cells: The algorithm guesses a sample point, e.g. s ; it evaluates

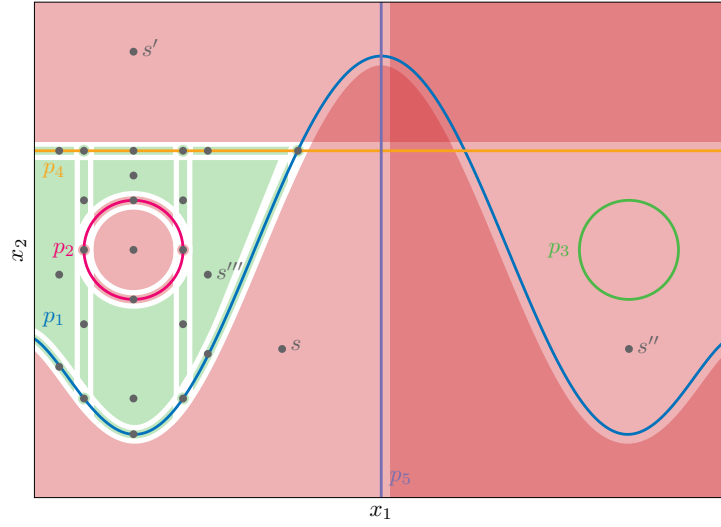


Figure 3.17: Illustration for Example 3.15. The truth-invariant cells are highlighted in red and green: The formula $\varphi = p_1 \leq 0 \wedge p_2 > 0 \wedge p_3 \geq 0 \wedge p_4 \leq 0 \wedge p_5 \leq 0$ is *false* on the red cells and *true* on the green cells.

the formula and finds that $\varphi(s) = \text{false}$, as the constraint $p_5 \leq 0$ is violated. It thus computes a sign-invariant CAD w.r.t. $\{p_5\}$, and identifies the cell that contains s , that is $(\text{root}_{x_1}^{p_5,1}, \infty) \times (-\infty, \infty)$. The algorithm proceeds with another sample point that is not contained in the generated cell, s' , similarly obtaining the cell $(-\infty, \infty) \times (\text{root}_{x_2}^{p_4,1}, \infty)$, and s'' , obtaining the cell $(-\infty, \infty) \times (-\infty, \text{root}_{x_2}^{p_1,1})$. Now assume the algorithm chooses the point s''' indicated in the figure, thus finds that $\varphi(s''') = \text{true}$. As φ is a conjunction, all constraints need to be satisfied. We thus compute a CAD w.r.t. $\{p_1, \dots, p_5\}$, obtaining the cell $(\text{root}_{x_1}^{\text{disc}_{x_2}(p_2),2}, \text{root}_{x_1}^{\text{res}_{x_2}(p_1,p_4),1}) \times (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1})$.

The algorithm could stop here if we would check whether $\exists x_1. \exists x_2. \varphi$ holds, or continue until no sample point outside the already generated cells can be found (and thus the entire space is covered by truth-invariant cells).

Note that we do not generate arbitrary truth-invariant cells: The union of the green cells at the left is a cell. However, this cell cannot be described in the “triangular” way, i.e. by an interval between some roots of the CAD projection polynomials in variables x_1 , and a symbolic interval (which depends on x_1) between root functions of the polynomials in variables x_1 and x_2 . Cells that can be described are called *locally cylindrical*, a notion we will define below.

We remark that this algorithm is a simplified version, as choosing a sample point outside the already generated cells is rather involved. The additional projection operations necessary for this step are the main differences between the algorithms presented in this chapter.

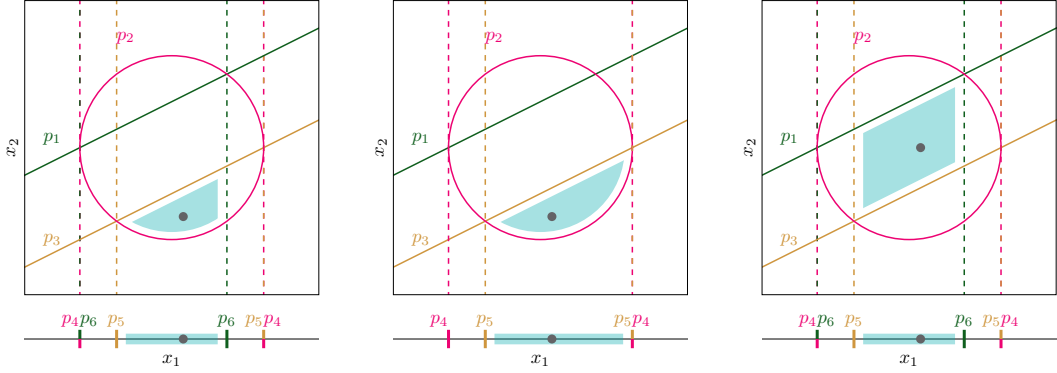
We define the notion of local cylindricity as motivated by the above example.

Definition 3.16 (Local Cylindricity [BK15])

Let $i \in [n]$. A set $R \subset \mathbb{R}^i$ is *locally cylindrical* if either

- $i = 1$ and R is equal to an interval $(-\infty, \infty)$, $(-\infty, f)$, (f, ∞) , (f, g) , or $[f, f]$ for $f, g \in \mathbb{R}$; or
- $i > 1$ and $R_{\downarrow[i-1]}$ is locally cylindrical and R is equal to a set of the form $R_{\downarrow[i-1]} \times (-\infty, \infty)$, $R_{\downarrow[i-1]} \times (-\infty, f)$, $R_{\downarrow[i-1]} \times (f, \infty)$, $R_{\downarrow[i-1]} \times (f, g)$, or $R_{\downarrow[i-1]} \times [f, f]$ for continuous functions $f, g : R_{\downarrow[i-1]} \rightarrow \mathbb{R}$ such that $f < g$.

In the following, we will examine the properties and the computation of *(locally) cylindrical (semi-)algebraic cells*.



(a) The cell around the sample point as defined by a sign-invariant CAD. (b) The cell around the sample point as defined by a locally sign-invariant CAD. (c) The cell around another sample point as defined by a locally sign-invariant CAD, which is also contained in the respective (non-locally) sign-invariant CAD.

Figure 3.18: Illustrations for Example 3.16. Locally cylindrical semi-algebraic cells on which $\{p_1, p_2, p_3\}$ is sign-invariant.

Example 3.16

Consider the polynomials

$$p_1 = 0.5x_1 + 0.5 - x_2 \quad p_2 = x_1^2 + x_2^2 - 1 \quad p_3 = 0.5x_1 - 0.5 - x_2$$

as depicted in Figure 3.18.

Figure 3.18a depicts the cell from the sign-invariant CAD for $\{p_1, p_2, p_3\}$ that contains the sample point $s = (0.125, -0.75)$. Figure 3.18b depicts another (locally cylindrical) cell which is still sign-invariant but larger than the previous one. It is still a cell of a CAD, however this CAD does not only consist of sign-invariant cells for $\{p_1, p_2, p_3\}$; we say this CAD is sign-invariant for $\{p_1, p_2, p_3\}$ w.r.t. s .

Figure 3.18c depicts a cell that contains $s' = (0.125, 0)$. Although this cell is part of a non-locally sign-invariant CAD for $\{p_1, p_2, p_3\}$, there does not exist a coarser locally sign-invariant CAD for $\{p_1, p_2, p_3\}$ w.r.t. s' defining a larger cell.

As illustrated in the above example, it is possible to compute locally cylindrical cells that are larger than the cells of a sign-invariant CAD. Still, we can compute these cells using CAD technology. We will give an algorithm later, here we formally define the property of the CAD that we require for computing locally cylindrical cells.

Definition 3.17 (Locally Sign-invariant CAD)

Let $i \in [n]$, $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s \in \mathbb{R}^i$ and D be a CAD of $\mathbb{R}^i$. We call D *locally sign-invariant for P w.r.t. s* if P is sign-invariant in the cell $R \in D$ such that $s \in R$.

Once we computed such a CAD, we can describe the desired cell using the respective indexed root expressions as symbolic interval bounds (see also Example 3.16). However, we note that this approach has some limitations, which are elaborated in the following example.

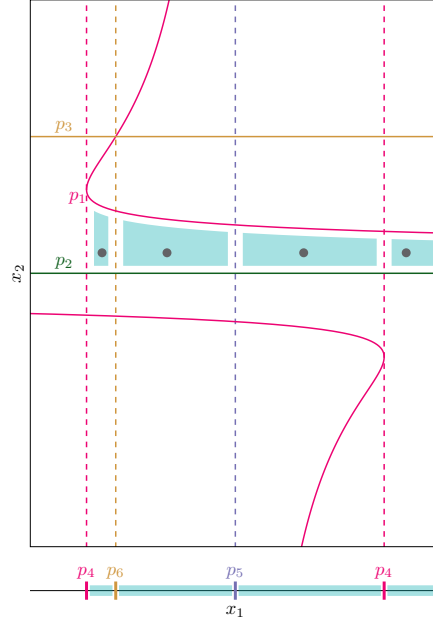


Figure 3.19: Illustration for Example 3.17. Although the root of p_2 and the second root of p_1 would describe a cylindrical cell in which all polynomials are sign-invariant, the coarsest locally sign-invariant CAD that may be computed using CAD theory splits the cell into 7 cells (here, we only depict the two-dimensional cells, not the one-dimensional ones over the sections of the projection).

Example 3.17

Consider Figure 3.18c. The cell is bounded by the root of p_3 from below and the root of p_1 from above, whereas the maximal sign-invariant cell around the depicted point is larger: it may be bounded by the root of p_3 and the first root of p_2 from below, and by the root of p_1 and the second root of p_2 from above. However, in our framework, the bounds of a cell in a given dimension need to be described by a single root of a polynomial (Limitation 1).

Now consider the polynomials

$$p_1 = x_1 x_2^3 + 16x_2^2 - 32 \quad p_2 = x_2 \quad p_3 = x_2 - 4$$

as depicted in Figure 3.19. The root of p_2 and a root of p_1 could describe a cylindrical cell that is split into 7 cells (4 sectors and 3 sections). The splits above the second root of $p_4 = \text{disc}_{x_2}(p_1)$ and above the root of $p_5 = \text{lcf}_{x_2}(p_1)$ are due to the index changes of the root function of p_1 (Limitation 2). The split above the root of

$p_6 = \text{res}_{x_2}(p_1, p_3)$ is due to the intersection of p_1 and p_3 , which is spurious, but the resultant is still relevant to guarantee local sign-invariance (Limitation 3). Although in the two-dimensional case, we can easily reason which cells of a CAD are adjacent, this is not trivial in higher dimensions. In fact, resolving these issues is closely related to computing cell adjacencies in a (sign-invariant) CAD.

In Chapter 4, we will discuss extensions to mitigate the demonstrated limitations. Limitation 1 may be addressed by describing bounds of cells by compound root functions (Section 4.7) or introducing additional polynomials as cell bounds (Section 5.4.1). Limitation 2 may be addressed by the same ideas, or mitigated by extending the notion of delineability (Section 4.8). Limitation 3 may be circumvented by additional checks which roots of the resultant are relevant (Section 4.5). However, these limitations are inherent to the CAD, and thus cannot be avoided completely.

3.3.1 Computing a Single Cell using the Levelwise Method

There are two paradigms for computing single locally sign-invariant cells: The *refinement based* approach by Brown and Košta [BK15; Bro13], and our *levelwise* approach first introduced in [NÁS⁺24]. We focus on the latter here, which was inspired by the former, and give a comparison of the two approaches afterwards.

For formalizing the algorithms, we make use of the following notion.

Definition 3.18 (Maximal Sign-invariant Cell)

Let $i \in [0..n]$, $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and $s \in \mathbb{R}^i$. Then $C(P, s) \subseteq \mathbb{R}^i$ denotes the inclusion-maximal cell such that $s \in C(P, s)$ and P is sign-invariant (or order-invariant, if we use McCallum's projection) on $C(P, s)$.

According to the definition, a set of polynomials P along with a suitable sample point s define a unique cell. We thus call the tuple (P, s) an *implicit cell* that encodes $C(P, s)$.

The levelwise approach is given by Algorithms 3.6 to 3.9. Algorithm 3.6 iterates over each level (from highest to lowest); in each step, it determines a symbolic interval describing the bounds of the cell on the current level using Algorithm 3.8, and a projection that takes the current sample point into account using Algorithm 3.9. Note that the latter may not return a result in those cases where McCallum's projection fails; in these cases, Algorithm 3.6 immediately terminates without a result. For better readability, we do not include this in the pseudocode explicitly, and do so for the other algorithms in this thesis as well.

Algorithm 3.6: `construct_cell(P, s)`

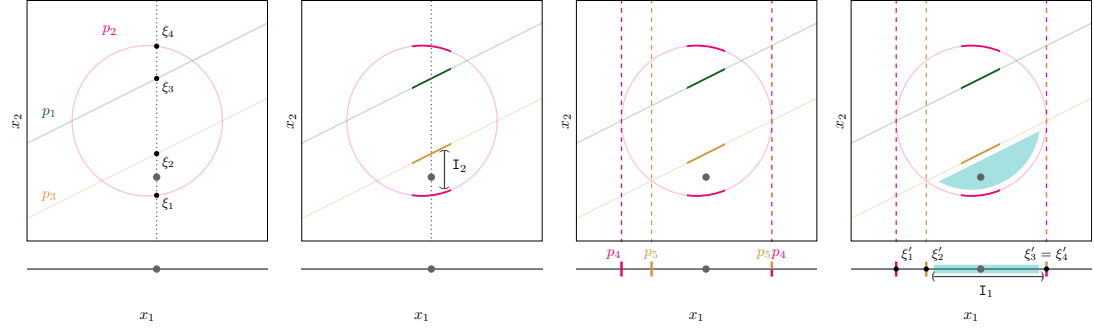
Input : $P \subseteq \mathbb{Q}[x_1, \dots, x_n]$, and $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^n$.

Output : Symbolic intervals $I_1, \dots, I_n$ of levels $1, \dots, n$ respectively such that $I_1 \times \dots \times I_n \subseteq C(P, s)$ and $s \in I_1 \times \dots \times I_n$.

```

1  $P' := \bigcup_{p \in P} \text{factors}(p)$ 
2 foreach  $i = n, \dots, 1$  do
3    $I_i := \text{choose\_interval}(P', s_{[i]})$  // Algorithm 3.8
4    $P' := \text{compute\_cell\_projection}(P', s_{[i]}, I_i)$  // Algorithm 3.9
5 return  $I_1, \dots, I_n$ 
```

As Example 3.16 suggests, a locally sign-invariant CAD might require fewer polynomials to be computed. The following example illustrates how the algorithm detects the polynomials to be left out.



(a) The large point on the one-dimensional and two-dimensional coordinate system indicates the current sample point. We isolate the real roots of the given polynomials over the one-dimensional sample point.

(b) The real roots withness root function of the polynomials whose domain is not known yet. Still, we choose to construct a cell whose second level is described by the symbolic interval I_2 .

(c) We compute the projection that catches all important points of the variety graph to obtain a sign-invariant cell.

(d) On the level below, we iterate the process: We isolate the real roots, and determine an interval describing the cell's boundary on that level.

Figure 3.20: Levelwise construction of a single cell for Example 3.18.

Example 3.18

Consider the polynomials from Example 3.16 and the point $s = (0.125, -0.75)$.

We consider the first coordinate $x_1 = s_1$ of the sample s , substitute this value into the polynomials, and isolate the real roots to find $\xi_1 = \text{root}_{x_2}^{p_2,1}$, $\xi_2 = \text{root}_{x_2}^{p_3,1}$, $\xi_3 = \text{root}_{x_2}^{p_1,1}$, and $\xi_4 = \text{root}_{x_2}^{p_2,2}$. We sort them, along with the second coordinate of s , as shown in Figure 3.20a.

Our sample lies in the interval for x_2 denoted $I_2 = (\xi_1, \xi_2)$ in Figure 3.20b. We thus know that, local to s_1 , the cell we want will be bounded from below by p_2 and from above by p_3 . We also know that the other section of p_2 and the section of p_1 are above the cell. In this figure the graphs of the polynomials are greyed out because the algorithm is not aware of them: it knows only their roots when evaluated at the sample. However, this is enough to allow the algorithm to infer behaviour around the sample, as illustrated by the thick partial lines in Figure 3.20b.

As we generalize from the sample, we must ensure that (1) ξ_1 , ξ_2 , ξ_3 and ξ_4 are well-defined and that (2) no root function crosses the symbolic interval I_2 . To achieve (1), we consider the discriminants $p_4 = \text{disc}_{x_2}(p_2)$, $\text{disc}_{x_2}(p_3)$, and $\text{disc}_{x_2}(p_1)$ (the latter two do not have real roots). For (2), we add some resultants: As ξ_1 defines the lower bound, and no other polynomial has a root below it, no root function will cross the lower bound and violate the cell's sign-invariance property. For the upper bound ξ_2 , we need to ensure that ξ_3 and ξ_4 do not cross ξ_2 , which we achieve by considering the resultants $\text{res}_{x_2}(p_1, p_3)$ (which does not have real roots) and $p_5 = \text{res}_{x_2}(p_3, p_2)$ respectively. Figure 3.20c shows how the roots of the projection

map onto the geometric features.

Analogously to x_2 , we isolate the real roots $\xi'_1 = \text{root}_{x_1}^{p_4,1}$, $\xi'_2 = \text{root}_{x_1}^{p_5,1}$, $\xi'_3 = \text{root}_{x_1}^{p_4,2}$ in x_1 . We determine the interval $I_1 = (\xi'_2, \xi'_3)$ around s_1 for x_1 and thus generate the cell in Figure 3.20d.

Finally, we could add also the resultants $\text{res}_{x_2}(p_1, p_3)$ (ensuring that ξ_2 and ξ_3 do not intersect) and $\text{res}_{x_2}(p_1, p_2)$ (ensuring that ξ_3 and ξ_4 do not intersect); by transitivity, we would achieve that ξ_2 and ξ_4 do not intersect. We will deal with these ideas in Chapter 4.

Algorithm 3.7 is used by Algorithm 3.8 and computes and sorts the real roots of the polynomials at the given sample point, and identifies the real interval for that sample point describing the diameter of the cell $C(P, s)$.

Algorithm 3.7: `get_interval`(P, s)

Input : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [n]$.

Output : Interval $I = \{r \in \mathbb{R} \mid (s_{[i-1]}, r) \in C(P, s)\}$.

- 1 $P_i := \{p \in P \mid \text{level}(p) = i\}$
 - 2 $\Xi := \bigcup_{p \in P_i} \text{realRoots}(p(s_{[i-1]}, x_i))$
 - 3 **if** $|\Xi| = 0$ **then return** $(-\infty, \infty)$
 - 4 **let** $\{\xi_1, \dots, \xi_k\} = \Xi$ **such that** $\xi_1 < \dots < \xi_k$
 - 5 **let** $I \in \{(-\infty, \xi_1), [\xi_1, \xi_1], (\xi_1, \xi_2), \dots, (\xi_k, \infty)\}$ **such that** $s_i \in I$
 - 6 **return** I
-

Algorithm 3.8 generalizes the interval above the sample point to a symbolic interval; note that the choice is not unique, as multiple polynomials could vanish at the cell's boundary.

Algorithm 3.8: `choose_interval`(P, s)

Input : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [n]$.

Output : Symbolic interval I such that $s_{[i-1]} \times I = s_{[i-1]} \times \{r \in \mathbb{R} \mid (s_{[i-1]}, r) \in C(P, s)\}$.

- 1 $I := \text{get_interval}(P, s)$ // Algorithm 3.7
 - 2 **if** $I = (-\infty, \xi_\ell)$ **then choose** $I_j := (-\infty, u)$ **s.t.** $u \in \text{ire}(P, s, \xi_\ell)$
 - 3 **else if** $I = (\xi_\ell, \xi_{\ell'})$ **then choose** $I_j := (l, u)$ **s.t.** $l \in \text{ire}(P, s, \xi_\ell), u \in \text{ire}(P, s, \xi_{\ell'})$
 - 4 **else if** $I = [\xi_\ell, \xi_\ell]$ **then choose** $I_j := [b, b]$ **s.t.** $b \in \text{ire}(P, s, \xi_\ell)$
 - 5 **else if** $I = (\xi_\ell, \infty)$ **then choose** $I_j := (l, \infty)$ **s.t.** $l \in \text{ire}(P, s, \xi_\ell)$
 - 6 **else if** $I = (-\infty, \infty)$ **then** $I_j := (-\infty, \infty)$
 - 7 **return** I
-

Once a symbolic interval has been identified, Algorithm 3.9 computes a projection that ensures that the symbolic interval describes a proper sign-invariant (or order-invariant) cell on the current level, given that the projection is sign-invariant (or order-invariant) in the underlying cell. As the projection is based on McCallum's projection operator, Line 2 checks if a polynomial is nullified and stops the algorithm's execution if this is the case. We maintain delineability of each polynomial by adding their discriminants and leading coefficients in Line 3 and ensure that no polynomial is nullified in Lines 5 and 6 (see the proof of Theorem 3.1). For ensuring non-nullification of a polynomial, we observe that if some coefficient does not vanish over some cell, then the polynomial does not nullify over that cell. Line 7 adds resultants protecting the lower bound from intersections; Line 8 adds resultants projecting the upper bound; and Line 9 adds the resultant of the lower and upper bound, ensuring connectedness of the cell (see Lemma 4.1). Finally, Line 10 factorizes the projection polynomials required for

the subsequent resultant and discriminant computations. Again, more details on the projection including proofs will be discussed in Chapter 4.

Algorithm 3.9: `compute_cell_projection(P, s, I)

---`

Input : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [n]$, and symbolic interval I of level i .
Output : $P' \subseteq \mathbb{Q}[x_1, \dots, x_{i-1}]$ such that $C(P', s_{[i-1]}) \times I \subseteq C(P, s)$.

- 1 $P_i := \{p \in P \mid \text{level}(p) = i\}$
- 2 **if** $\exists p \in P_i. p(s_{[i-1]}, x_i) = 0$ **then return** *FAIL*
- 3 $P' := (P \setminus P_i) \cup \{\text{disc}_{x_i}(p), \text{lcf}_{x_i}(p) \mid p \in P_i\}$
- 4 **foreach** $p \in P_i$ **do**
- 5 **choose** $q \in \text{coeff}_{x_i}(p)$ such that $q(s_{[i-1]}) \neq 0$
- 6 $P' := P' \cup \{q\}$
- 7 $P' := P' \cup \{\text{res}_{x_i}(I.l.p, q) \mid I.l \neq -\infty, q \in P_i \setminus \{I.l.p\}, \exists s' \leq I.l(s_i). q(s \times s') = 0\}$
- 8 $P' := P' \cup \{\text{res}_{x_i}(I.u.p, q) \mid I.u \neq \infty, q \in P_i \setminus \{I.u.p\}, \exists s' \geq I.u(s_i). q(s \times s') = 0\}$
- 9 $P' := P' \cup \{\text{res}_{x_i}(I.l.p, I.u.p) \mid I.l \neq -\infty, I.u \neq \infty, I.l.p \neq I.u.p\}$
- 10 $P' := \bigcup_{p \in P'} \text{factors}(p)$
- 11 **return** P'

3.3.2 Computing a Single Cell using the Refinement-based Method

We call the original single cell construction approach [BK15; Bro13] the *refinement-based* approach: It starts with $\mathbb{R}^n$ and iteratively refines the cell polynomial-by-polynomial to a sign-invariant cell. The refinement step considers the previously computed projection polynomials, computing as few additional polynomials as possible for refining the cell to the current polynomial. In particular, it checks whether any of the cell's boundaries change, and exploits the transitivity of root orderings as illustrated at the end of Example 3.18.

Algorithm 3.10: `construct_cell_by_refinement(P, s)

---`

Input : $P \subseteq \mathbb{Q}[x_1, \dots, x_n]$, and $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^n$.
Output : Symbolic intervals $I_1, \dots, I_n$ of levels $1, \dots, n$ respectively such that $I_1 \times \dots \times I_n \subseteq C(P, s)$.

- 1 $I_1, \dots, I_n := (-\infty, \infty), \dots, (-\infty, \infty)$
- 2 **foreach** $p \in P$ **do**
- 3 $P' := \{p\} \cup \{I_i.l.p \mid i \in [n], I_i.l \neq -\infty\} \cup \{I_i.u.p \mid i \in [n], I_i.u \neq \infty\}$
- 4 $I_1, \dots, I_n := \text{construct_cell}(P', s)$ // Algorithm 3.6
- 5 **return** $I_1, \dots, I_n$

We simplify its presentation by giving a formulation in Algorithm 3.10 that uses the levelwise approach. While the behaviour of this formulation is equivalent to the one given in [BK15], we note that the original formulation does not recompute projection polynomials (such as discriminants and coefficients) that were previously computed by bookkeeping. The algorithm starts with $\mathbb{R}^n$ as cell and iterates over the polynomials. For each polynomial, it collects the defining polynomial of the current cell and the current polynomial in Line 3, and calls the single cell construction in Line 4 to compute the refined cell.

The following example illustrates this approach and compares it to the levelwise approach.

Example 3.19

We consider the polynomials and sample point as in Example 3.18. Figure 3.21 illustrates two possible runs of Algorithm 3.10, using different orders in which the polynomials are used to refine the cell. We see that one produces a larger cell than the other. It is not obvious how to choose good orders for adding polynomials in this approach. Moreover, because the method works by refining the cell polynomial-by-polynomial, the order in which lower-level projection polynomials are considered can only be determined ad-hoc, and not according to the whole projection.

While the quality of the cell is hard to predict for the refinement-based approach, the levelwise approach gives more control over the cell; in this case, it would compute the bigger cell resulting from the second run of the refinement-based approach.

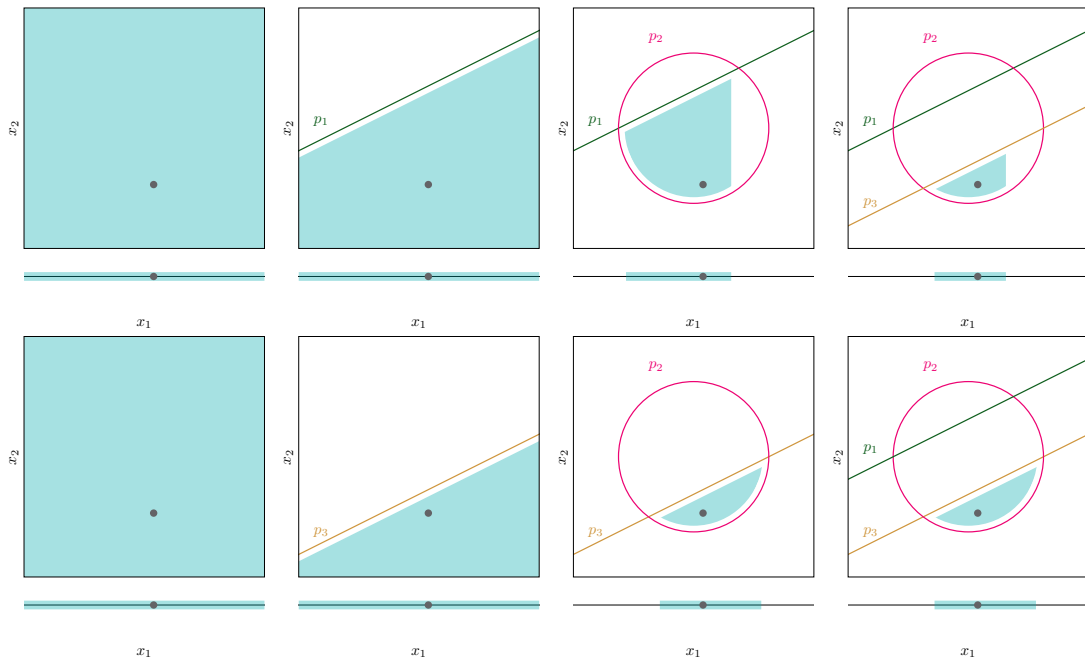


Figure 3.21: Refinement-based construction of a single cell for Example 3.19. In the upper row the original cell (the entire plane) is refined first by polynomial p_1 , then by p_2 , then by p_3 . In the lower row a different order of refinement is used: p_3 , p_2 , p_1 giving a larger cell.

3.4 Cylindrical Algebraic Covering

In Example 3.15, we sketched the idea of an algorithm that explores the whole real space. However, we did not specify to choose a sample that is outside all cells that were generated so far. For that, we need to maintain a certain global arrangement of the cells. In fact, a CAD maintains such an arrangement of locally cylindrical cells. The following (straight-forward) remark is derived from Definition 3.1 and makes this explicit.

Remark 3.1

Let $i \in [n]$, and D_i be a CAD of $\mathbb{R}^i$. Then either

- $i = 1$ and $D_1 = \{R_1, \dots, R_k\}$ such that R_j is locally cylindrical for $j \in [k]$, $\mathbb{R} = \bigcup_{j \in [k]} R_j$ and $R_j \cap R_{j'} = \emptyset$ for each $j, j' \in [k]$, $j \neq j'$; or
- $i > 1$ and there is a CAD D_{i-1} of $\mathbb{R}^{i-1}$ such that for each cell $R \in D_{i-1}$, there exist locally cylindrical cells $R_1, \dots, R_k \in D_i$ such that $R \times \mathbb{R} = \bigcup_{j \in [k]} R_j$ and $R_j \cap R_{j'} = \emptyset$ for each $j, j' \in [k]$, $j \neq j'$.

In the following, we will define an alternative arrangement of cells. We relax the notion of a (*sign-invariant*) cylindrical algebraic *decomposition* to a (*truth-invariant*) cylindrical algebraic *covering*. In a covering, we allow the cells to overlap, as long as they are still cylindrically arranged.

Definition 3.19 (Cylindrical Algebraic Covering [ÁDE⁺21])

Let $i \in [n]$.

- A *covering* of $\mathbb{R}^i$ is a finite set $D \subset 2^{\mathbb{R}^i} \setminus \{\emptyset\}$ such that each $R \in D$ is connected and $\bigcup_{R \in D} R = \mathbb{R}^i$.
- A *cylindrical algebraic covering (CAIC)* of $\mathbb{R}^i$ is a covering of $\mathbb{R}^i$ that is algebraic and cylindrical.
- Let φ be a Tarski formula with free variables $x_1, \dots, x_n$ and D be a CAIC of $\mathbb{R}^n$. We call D *truth-invariant for φ* if φ is truth-invariant in each cell $R \in D$.

Example 3.20

We consider the formula from Example 3.9 on Page 70 again. A truth-invariant CAIC for the input formula is depicted in Figure 3.22. We emphasize that, due to the cylindrical arrangement, we can represent a CAIC in a tree similar to a CAD as in Figure 3.9.

Of course, we can relate a CAIC to the notion of local cylindricity as well.

Corollary 3.2

Let $i \in [n]$, and D_i be a CAIC of $\mathbb{R}^i$. Then either

- $i = 1$ and $D_1 = \{R_1, \dots, R_k\}$ such that R_j is locally cylindrical for $j \in [k]$ and $\mathbb{R} = \bigcup_{j \in [k]} R_j$; or
- $i > 1$ and there is a CAIC D_{i-1} of $\mathbb{R}^{i-1}$ such that for each cell $R \in D_{i-1}$, there exist locally cylindrical cells $R_1, \dots, R_k \in D_i$ such that $R \times \mathbb{R} = \bigcup_{j \in [k]} R_j$.

Motivated by this corollary, we can define a covering over a given cell, which will be useful to express intermediate results during its computation.

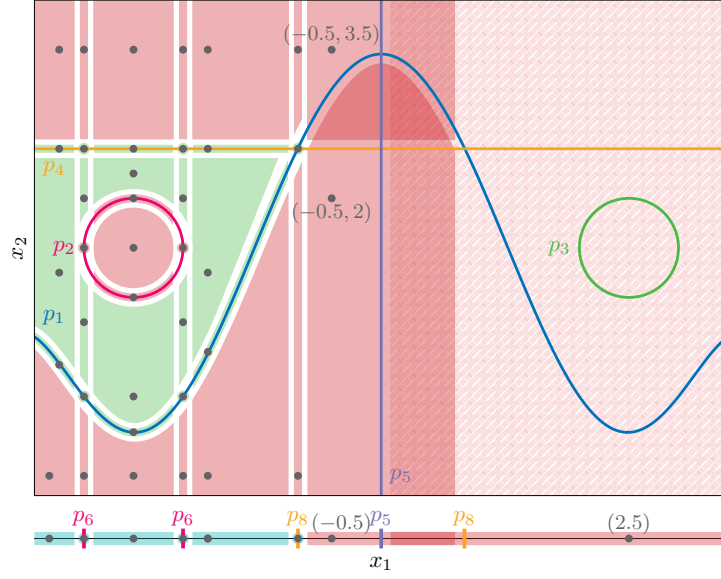


Figure 3.22: Truth-invariant cylindrical algebraic covering for Example 3.20, along with a sample point for each cell. One cell is hatched for better readability and does not have different semantics than the others. The sample point (2.5) does not have a correspondence in the two-dimensional coordinate system as the bounding polynomial of its cell contains only x_1 .

Definition 3.20

Let $i \in [0..n]$, $R \subseteq \mathbb{R}^i$, and $i' \in [i+1..n]$. A *CAIC* of $\mathbb{R}^{i'}$ over R is a finite set $D_{i'} \subseteq 2^{\mathbb{R}^{i'}} \setminus \{\emptyset\}$ such that

- either $i' = i + 1$ and $D_{i+1} = R_1, \dots, R_k$ such that R_j is locally cylindrical for $j \in [k]$ and $R \times \mathbb{R} = \bigcup_{j \in [k]} R_j$; or
- $i' > i + 1$ and there is a CAIC $D_{i'-1}$ of $\mathbb{R}^{i'-1}$ over R such that for each cell $R' \in D_{i'-1}$, there exist locally cylindrical cells $R_1, \dots, R_k \in D_{i'}$ such that $R' \times \mathbb{R} = \bigcup_{j \in [k]} R_j$.

There are two approaches for computing a CAIC: The CAIC method, and a modified version of the NLSAT algorithm. We start with the former, before we compare it with the latter.

3.4.1 Computing a CAIC using the CAIC Method

We now present the CAIC algorithm, which was originally introduced in [ÁDE⁺21] for existentially quantified conjunctions of constraints, and has been extended in [KN23; NK25] for real quantifier elimination. The following presentation is close to [NK25].

We briefly describe the intuition behind the algorithm for checking the satisfiability of existentially quantified conjunctions of constraints: The CAIC algorithm starts by picking a value s_1 for the first variable x_1 . If that value is consistent with all constraints, we continue with the next variable. Otherwise, some constraint $p \sim 0$ that is univariate in x_1 is violated, and we generalize the conflict to the implicit cell $(\{p\}, s_1)$. By isolating

the roots of p , we obtain a maximal interval on which p is sign-invariant containing s_1 which excludes conflicting values for x_1 . We iterate and try to find a value s_1 for x_1 outside all excluded intervals that is consistent with all univariate constraints in x_1 . If we succeed, then we iterate this idea, e.g. we pick a value s_2 for the next variable x_2 , etc. If at some point, there is a variable x_i for which no feasible value s_i exists, we obtained a set $\mathcal{C}$ of implicit cells that cover the line $(s_1, \dots, s_{i-1}) \times \mathbb{R}$, and the polynomials that define the implicit cells in $\mathcal{C}$ are in variables $x_1, \dots, x_i$. A specialized projection generalizes this covering to an implicit cell $(P, (s_1, \dots, s_{i-1}))$ such that the whole cylinder $C(P, (s_1, \dots, s_{i-1})) \times \mathbb{R}$ over $C(P, (s_1, \dots, s_{i-1}))$ is covered by cells from $\mathcal{C}$. We thus go back to searching a value x_{i-1} . The algorithm terminates if either a full sample $(s_1, \dots, s_n)$ satisfying all constraints is found, or no feasible value for x_1 exists (i.e. $\mathbb{R}$ is covered by implicit cells).

We now present the generalized version.

Main Algorithm Algorithm 3.11 is the main algorithm for computing a CAIC. When calling it with `calc_recurse( $\varphi$ , ())` on a quantifier-free input formula φ , it returns a data structure that describes a truth-invariant CAIC for φ . In addition to the input formula, it takes a point $s \in \mathbb{R}^i$ and checks whether the formula already evaluates to a truth value (Line 1) and generalizes s to a cell where φ is truth-invariant by calling Algorithm 3.12. Otherwise, if the formula does not evaluate to a truth value (Line 3), then it calls Algorithm 3.13 to extend s for the next dimension.

Algorithm 3.11: `calc_recurse( $\varphi$ ,  $s$ )`

Input : Quantifier-free Tarski formula φ , and $s \in (\mathbb{R} \cap \overline{\mathbb{Q}})^i$ for some $i \in [0..n]$.
Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and either (1) $\mathfrak{t} \in \{\text{true}, \text{false}\}$ such that for all $s' \in C(P, s)$ it holds $\varphi(s') \equiv \mathfrak{t}$, or (2) a truth-labelled cylindrical set data structure $\mathbf{T}$ such that for each path $\mathbf{I}_{i+1}, \dots, \mathbf{I}_n$ in $\mathbf{T}$ to a label $\mathfrak{t}$ and each $s' \in C(P, s) \times \mathbf{I}_{i+1} \times \dots \times \mathbf{I}_n$ it holds $\varphi(s') \equiv \mathfrak{t}$.

```

1 if  $\varphi[s] \equiv \text{false} \vee \varphi[s] \equiv \text{true}$  then
2   | return choose_enclosing_cell( $\varphi$ ,  $s$ )                                // Algorithm 3.12
3 else
4   | return calc_full( $\varphi$ ,  $s$ )                                           // Algorithm 3.13

```

Characterizing Truth-Invariant Cells As pointed out in Example 3.15, after guessing a sample point and evaluating the formula, we need to identify the reasons for the formula's truth value in terms of a set of polynomials that describes a truth-invariant cell in Algorithm 3.12. We formalize these reasons as *implicants*.

Definition 3.21 (Evaluation up to Level, Implicant)

Let $i \in \mathbb{N}$, $s \in \mathbb{R}^i$, and φ be a formula where (at least) the variables $x_1, \dots, x_i$ are free.

- The *evaluation of φ at s up to level i* is denoted as $\varphi[s]$ and is defined as the result of substituting s into all constraints of level at most i (and leaving the others unchanged) and simplifying the result according to Boolean semantics (e.g. $\text{false} \vee \psi$ simplifies to false for any formula ψ).
- The quantifier-free formula ψ is an *implicant of φ with respect to s* if all constraints

$p \sim 0$ in ψ have $\text{level}(p) \leq i$ and are contained in φ , $\psi[s] \equiv \text{true}$, and either $\psi \Rightarrow \varphi$ or $\psi \Rightarrow \neg\varphi$.

Example 3.21

Let $\varphi = x_1^2 > 0 \wedge (x_1 < 2 \vee x_1 > 4)$. Note that $\varphi[(1)] = \text{true}$, $\varphi[(3)] = \text{false}$ and $\varphi[(0)] = \text{false}$. We have that $x_1^2 > 0 \wedge x_1 < 2$ is an implicant of φ w.r.t. (1), and $\neg(x_1 < 2) \wedge \neg(x_1 > 4)$ is an implicant of φ w.r.t. (3). Both $\neg(x_1^2 > 0)$ and $\neg(x_1^2 > 0) \wedge \neg(x_1 > 4)$ are implicants of φ w.r.t. (0).

Let $\varphi = (x_1 < 0 \vee x_2 \leq 4) \wedge (x_1 > 2 \vee x_2 > 4)$. Note that $\varphi[(1)] \equiv \text{false}$. $\neg(x_1 < 0) \wedge \neg(x_1 > 2)$ is an implicant of φ w.r.t. (1).

Let $\varphi = x_1 < 0 \vee \forall x_2. (x_2 > 0 \rightarrow x_1 > 2)$ and assume the sample (1), then $\varphi[(1)] = \text{false}$ and $\neg(x_1 < 0) \wedge \neg(x_1 > 2)$ is an implicant for φ w.r.t. (1).

Throughout the algorithm, we represent cells as tuples (P, s) (called *implicit cell*) where $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$ and $s \in \mathbb{R}^i$ for some $i \in [0..n]$. Algorithm 3.12 wraps the evaluation of the formula and implicant computation (whose details we omit for now) to obtain, given a sample point, a corresponding set of polynomials.

Algorithm 3.12: `choose_enclosing_cell`(φ, s)

Input : Tarski formula φ , $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [n]$ such that $\varphi[s] \equiv \text{false}$ or $\varphi[s] \equiv \text{true}$.

Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $\mathbf{t} \in \{\text{true}, \text{false}\}$ such that for all $s' \in C(P, s)$ it holds $\varphi(s') \equiv \mathbf{t}$.

- 1 **choose** implicant I of φ w.r.t. s
 - 2 $P := \{p \mid (p \sim 0) \in I\}$
 - 3 $P := \bigcup_{p \in P} \text{factors}(p)$
 - 4 let $\mathbf{t} \in \{\text{true}, \text{false}\}$ such that $\varphi[s] \equiv \mathbf{t}$
 - 5 **return** $P, \mathbf{t}$ // We return $\mathbf{t}$ to simplify notation.
-

Extending the Sample Point Algorithm 3.13 chooses values $r \in \mathbb{R}$ that extend the input sample s for the next dimension in Line 3. It then recursively calls Algorithm 3.11 with the extended sample to explore the search space above (s, r) in Line 4. The call obtains an implicit cell that describes a cell around (s, r) , along with a description of a truth-invariant covering over that cell. The algorithm then stores this cell and ensures that the next sample point lies outside it in Line 3. The algorithm iterates until the whole real line is covered by the implicit cells, as checked in Line 2. Afterwards, beginning from Line 6, it aims to generate an implicit cell (using polynomials of lower levels) such that the cylinder over that cell is covered by the collected implicit cells; the resulting implicit cell is returned to be used in a covering on the level below. As the set of collected implicit cells may be redundant, Algorithm 3.14 selects a non-redundant set of implicit intervals, sorted by their lower bounds. Algorithm 3.15 finally computes the desired projection.

Example 3.22

Consider the formula $\varphi(x_1, x_2) = p_1 \leq 0 \wedge p_2 > 0 \wedge p_3 \geq 0 \wedge p_4 \leq 0 \wedge p_5 \leq 0$ and the polynomials from Example 3.9 on Page 70 again. We give an exemplary run of Algorithm 3.13 that computes the CAIC depicted in Figure 3.22. For readability, we omit the calls to `calc_recurse()`.

Algorithm 3.13: $\text{calc_full}(\varphi, s)$

Input : Tarski formula φ , $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [0..n]$.
Output : $P' \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and truth-labelled cylindrical set data structure T such that for each path $I_{i+1}, \dots, I_n$ in T to a label $\mathbf{t}$ and each $s' \in C(P, s) \times I_{i+1} \times \dots \times I_n$ it holds $\varphi(s') \equiv \mathbf{t}$.

```

1  $\mathcal{C} := \emptyset$ 
2 while  $\bigcup_{(P,s') \in \mathcal{C}} \text{get\_interval}(P, s') \neq \mathbb{R}$  do // Algorithm 3.7
3   choose  $r \in \mathbb{R}$  s.t.  $r \notin \bigcup_{(P,s') \in \mathcal{C}} \text{get\_interval}(P, s')$ 
4    $(P, T_{(P,(s,r))}) := \text{calc\_recurse}(\varphi, (s, r))$  // Algorithm 3.11
5    $\mathcal{C} := \mathcal{C} \cup \{(P, (s, r))\}$ 
6  $((P^1, s^1, I^1), \dots, (P^k, s^k, I^k)) := \text{choose\_covering}(\mathcal{C})$  // Algorithm 3.14
7  $T := \{(I^j, T_{(P^j, s^j)}) \mid j \in [k]\}$ 
8 if  $i > 0$  then
9    $P' := \text{compute\_covering\_projection}(((P^1, s^1, I^1), \dots, (P^k, s^k, I^k)))$  // Algorithm 3.15
10  return  $P', T$ 
11 else
12  return  $\emptyset, T$ 

```

$\text{calc_full}(\varphi, (2.5))$ We start by choosing the value 2.5 for x_1 .

$\text{choose_enclosing_cell}(\varphi, (2.5))$ As φ evaluates to *false* at (2.5), we obtain the implicant $p_5 > 0$ (as $p_5 \leq 0$ is violated at (2.5) and φ is a conjunction). We obtain the implicit cell $(\{p_5\}, (2.5))$. Note that the variety of p_3 is completely contained in that cell, and the algorithm thus exploits that the constraint $p_3 \geq 0$ is irrelevant for the truth of the formula.

We add the result to $\mathcal{C}$, obtaining $\mathcal{C} = \{(\{p_5\}, (2.5))\}$. The call to $\text{get_interval}(\{p_5\}, (2.5))$ yields the interval $(\text{root}_{x_1}^{p_5, 1}, \infty)$ which does not cover the whole real line. We thus choose the next sample point (-0.5) outside that interval.

$\text{calc_full}(\varphi, (-0.5))$ As φ does not evaluate yet to a truth value, we choose a value for x_2 , e.g. 2.

$\text{choose_enclosing_cell}(\varphi, (-0.5, 2))$ We obtain the implicant $p_1 > 0$, thus obtain the implicit cell $(\{p_1\}, (-0.5, 2))$.

We obtain $\mathcal{C}' = \{(\{p_1\}, (-0.5, 2))\}$. Calling $\text{get_interval}(\{p_1\}, (-0.5, 2))$ yields the interval $(-\infty, \text{root}_{x_2}^{p_1, 1}(-0.5))$. We choose 3.5 next (which is outside that interval).

$\text{choose_enclosing_cell}(\varphi, (-0.5, 3.5))$ We obtain the implicant $p_4 > 0$, thus obtain the implicit cell $(\{p_4\}, (-0.5, 3.5))$.

We obtain $\mathcal{C}' = \{(\{p_1\}, (-0.5, 2)), (\{p_4\}, (-0.5, 3.5))\}$. Calling $\text{get_interval}(\{p_4\}, (-0.5, 3.5))$ yields the interval $(\text{root}_{x_2}^{p_4, 1}(-0.5), \infty)$. Further, these two intervals $(-\infty, \text{root}_{x_2}^{p_1, 1}(-0.5)) \cup (\text{root}_{x_2}^{p_4, 1}(-0.5), \infty) = \mathbb{R}$ cover the real line, thus no further value for x_2 can be chosen. We compute the projection using $\text{compute_covering_projection}(((\{p_1\}, (-0.5, 2)), (\{p_4\}, (-0.5, 3.5))))$, which yields the implicit cell $(\{p_8\}, (-0.5))$.

We obtain $\mathcal{C} = \{(\{p_5\}, (2.5)), (\{p_8\}, (-0.5))\}$. Calling $\text{get_interval}(\{p_8\}, (-0.5))$ yields $(\text{root}_{x_1}^{p_8, 1}, \text{root}_{x_1}^{p_8, 2})$. Note that the

intervals $(\text{root}_{x_1}^{p_8,1}, \text{root}_{x_1}^{p_8,2})$ and $(\text{root}_{x_1}^{p_1,1}, \infty)$ are overlapping; we thus can omit some real root computations for the cells in the intersection of the cylinders over these intervals (which would be done in the lifting phase of CAD).

We further continue by choosing the next sample for x_1 outside $(\text{root}_{x_1}^{p_8,1}, \text{root}_{x_1}^{p_8,2}) \cup (\text{root}_{x_1}^{p_1,1}, \infty)$.

Selecting a Covering Algorithm 3.14 eliminates redundancies from a set of implicit cells, and returns a sequence of implicit cells along with symbolic intervals for each cell sorted by their lower bound over the given sample point. The ordering is useful for the projection operation which is defined in Algorithm 3.15.

Algorithm 3.14: `choose_covering( $\mathcal{C}$ )`

Input : Set $\mathcal{C}$ of implicit cells of level $i + 1$ for some $i \in [0..n - 1]$ such that $\bigcup_{(P,s) \in \mathcal{C}} \text{get_interval}(P, s) = \mathbb{R}$ and for all $(P, s), (P', s') \in \mathcal{C}$ it holds $s_{[i]} = s'_{[i]}$.

Output : A sequence of implicit cells of level $i + 1$ with symbolic intervals $((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k))$ from $\mathcal{C}$ such that

- (0) for all $j \in [k]$ it holds $\mathbf{I}^j(s_{[i]}^j) \subseteq \text{get_interval}(P^j, s^j)$,
- (1) $\bigcup_{j \in [k]} \mathbf{I}^j(s_{[i]}^j) = \mathbb{R}$,
- (2) for all $j, j' \in [k], j \neq j'$ it holds $\mathbf{I}^j(s_{[i]}^j) \not\subseteq \mathbf{I}^{j'}(s_{[i]}^{j'})$, and
- (3) for all $j \in [k-1]$ it holds $\mathbf{I}^j(s_{[i]}^j).l < \mathbf{I}^{j+1}(s_{[i]}^{j+1}).l$ or $(\mathbf{I}^j(s_{[i]}^j).l = \mathbf{I}^{j+1}(s_{[i]}^{j+1}).l = \mathbf{I}^j(s_{[i]}^j).u < \mathbf{I}^{j+1}(s_{[i]}^{j+1}).u)$.

```

1 for  $(P, s) \in \mathcal{C}$  do
2    $\mathbf{I}_{(P,s)} := \text{choose\_interval}(P, s)$  // Algorithm 3.8
3 sort elements of  $\mathcal{C}$  such that for all  $j \in [k-1]$  it holds  $\mathbf{I}^j(s_{[i]}^j).l < \mathbf{I}^{j+1}(s_{[i]}^{j+1}).l$  or
    $(\mathbf{I}^j(s_{[i]}^j).l = \mathbf{I}^{j+1}(s_{[i]}^{j+1}).l \text{ and } \mathbf{I}^j(s_{[i]}^j).u < \mathbf{I}^{j+1}(s_{[i]}^{j+1}).u)$ 
4 iterate through elements in that order and remove redundancies according to Condition (2)
5 store the result in  $((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k))$ 
6 return  $((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k))$ 

```

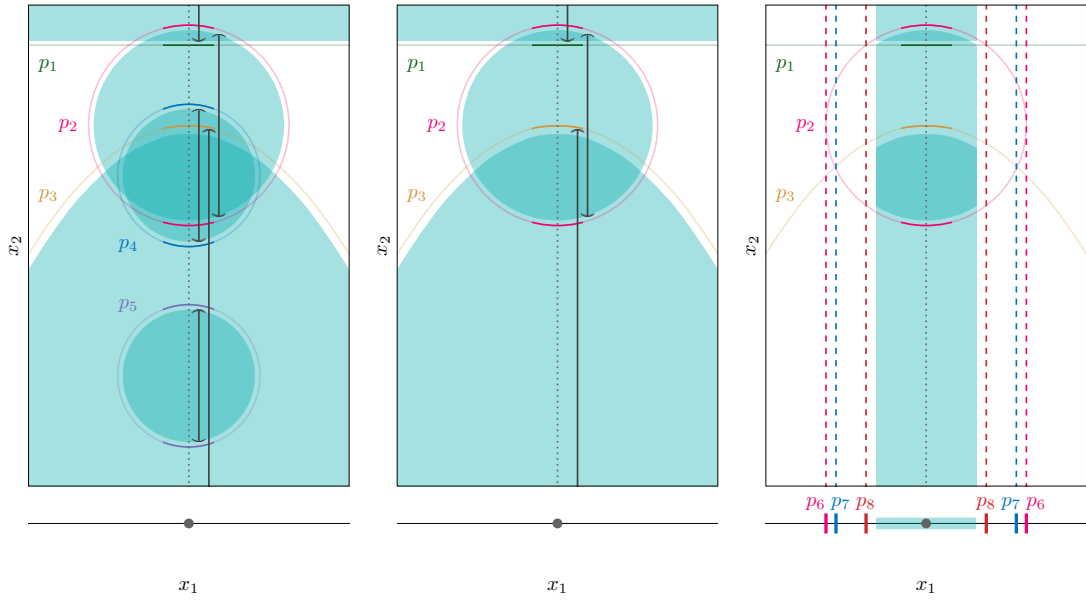
Example 3.23

Consider the polynomials

$$\begin{aligned}
 p_1 &= x_2 - 2.5 & p_2 &= x_1^2 + (x_2 - 2)^2 - 1 & p_3 &= -0.5x_1^2 + 2 - x_2 \\
 p_4 &= x_1^2 + (x_2 - 1.5)^2 - 0.5 & p_5 &= x_1^2 + (x_2 + 0.5)^2 - 0.5
 \end{aligned}$$

and the formula $p_1 > 0 \vee p_2 < 0 \vee p_3 > 0 \vee p_4 < 0 \vee p_5 < 0$. Assume that the algorithm chooses $s = (0)$ and finds implicit cells $(\{p_i\}, s^i)$ for $i \in [5]$ (all of them describe cells where φ is *true*) which cover $s \times \mathbb{R}$, as depicted in Figure 3.23a along with the corresponding symbolic intervals covering the real line above s .

Algorithm 3.14 eliminates redundancies: Both $C(\{p_4\}, s^4)$ and $C(\{p_5\}, s^5)$ can be removed from the set. $C(\{p_4\}, s^4)$ is included in $C(\{p_2\}, s^2) \cup C(\{p_3\}, s^3)$, and $C(\{p_5\}, s^5)$ is fully included in $C(\{p_3\}, s^3)$, as depicted in Figure 3.23b. Thus, the algorithm returns the sequence $((\{p_3\}, s^3, (-\infty, \text{root}_{x_2}^{p_3,1})), (\{p_2\}, s^2, (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_2,2})), (\{p_1\}, s^1, (\text{root}_{x_2}^{p_1,1}, \infty)))$.



(a) Redundant set of cells described by the polynomials' varieties. The brackets indicate the respective intervals above the given sample point. (b) Non-redundant set of covering cells. (c) Generalization of the covering by projection.

Figure 3.23: Illustration for Examples 3.23 and 3.24.

Projection of a Covering Algorithm 3.15 get a sequence of implicit cells as input and computes the projection that generalizes a sample s to a cell around s such that the given cells still cover the cylinder over that cell. To do so, it first computes the projection polynomials of each cell individually in Line 1. Then, it ensures that the boundaries of each pair of neighbouring cells remain overlapping over the generated cell by adding the resultant of the upper bound of the lower cell and the lower bound of the upper cell to the projection polynomials in Line 3. For that, it is crucial for the correctness of the projection that the cells are ordered by their lower bound and are non-redundant.

Algorithm 3.15: `compute_covering_projection` $((P^1, s^1, \mathbb{I}^1), \dots, (P^k, s^k, \mathbb{I}^k))$

Input : Sequence of implicit cells of level $i + 1$ for some $i \in [n - 1]$ with symbolic intervals $((P^1, s^1, \mathbb{I}^1), \dots, (P^k, s^k, \mathbb{I}^k))$ that complies with the output of Algorithm 3.14 and $s_{[i]}^1 = \dots = s_{[i]}^k =: s'$.

Output : $P' \subseteq \mathbb{Q}[x_1, \dots, x_i]$ such that $C(P', s') \times \mathbb{R} \subseteq \bigcup_{j \in [k]} C(P^j, s^j)$.

```

1  $P' := \bigcup_{j \in [k]} \text{compute\_cell\_projection}(P^j, s^j, \mathbb{I}^j)$  // Algorithm 3.9
2 for  $j \in [k-1]$  do
3    $P' := P' \cup \{\text{res}_{x_{i+1}}(\mathbb{I}^j.u.p, \mathbb{I}^{j+1}.l.p)\}$ 
4  $P' := \bigcup_{p \in P'} \text{factors}(p)$ 
5 return  $P'$ 

```

Example 3.24

Consider Example 3.23 again. Algorithm 3.15 gets the input sequence $((\{p_3\}, s^3, (-\infty, \text{root}_{x_2}^{p_3,1})), (\{p_2\}, s^2, (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_2,2})), (\{p_1\}, s^1, (\text{root}_{x_2}^{p_1,1}, \infty)))$.

Figure 3.23c depicts the projection consisting of p_6 , p_7 , and p_8 over which the covering encoded in the input is maintained.

In Line 1, we encode the individual cells. The discriminant $p_6 = \text{disc}_{x_2}(p_2)$ is computed in the projection of the implicit cell $(\{p_2\}, s^2)$; the projections of the other cells are empty as the discriminants are constant.

In Line 3, we ensure the covering. The resultant $p_7 = \text{res}_{x_2}(p_2, p_3)$ ensures that the bounds described by $\text{root}_{x_2}^{p_3,1}$ and $\text{root}_{x_2}^{p_2,1}$ overlap over the underlying cell, and the resultant $p_8 = \text{res}_{x_2}(p_1, p_2)$ ensures that the bounds described by $\text{root}_{x_2}^{p_2,2}$ and $\text{root}_{x_2}^{p_1,1}$ overlap over the underlying cell.

3.4.1.1 Quantifiers

Algorithm 3.13 computes a full covering describing all truth-invariant cells for a formula. Although we can use such a covering for dealing with quantifiers similarly to the CAD, we would compute more than we need: In Algorithm 3.13, if the current variable is existentially quantified, we only need to find a single cell in which the input formula evaluates to *true*; if we find such a cell, we can terminate the loop and simply return a projection of that cell instead of the projection of a covering; otherwise, we project the covering of cells, on which the formula evaluates to *false*, to prove that there does not exist such a cell. Analogously, if the current variable is universally quantified, we only need to compute the projection of a cell where the formula evaluates to *false* in case we find one, and otherwise the projection of cells where the formula is *true*. Free variables can still be handled as before.

Algorithms 3.16 and 3.17 implement the new idea. Algorithm 3.16 replaces the main algorithm Algorithm 3.11; it assumes the input formula to be in prenex normal form and calls Algorithm 3.13 if the next variable is a parameter (i.e. is free) to describe its solution space, and calls Algorithm 3.17 for quantified variables.

Algorithm 3.16: $\text{calc_recurse}(\varphi, s)$

Input : Tarski formula φ in prenex normal form, and $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [0..n]$.
Output : see Algorithm 3.11

```

1 if  $\varphi[s] \equiv \text{false} \vee \varphi[s] \equiv \text{true}$  then
2   return choose_enclosing_cell( $\varphi, s$ )                                // Algorithm 3.12
3 else
4   if  $\varphi = \exists x_{i+1}. \psi$  then
5     return calc_quantifier( $\exists, \psi, s$ )                                // Algorithm 3.17
6   else if  $\varphi = \forall x_{i+1}. \psi$  then
7     return calc_quantifier( $\forall, \psi, s$ )                                // Algorithm 3.17
8   else
9     return calc_full( $\varphi, s$ )                                          // Algorithm 3.13

```

Algorithm 3.17 replaces Algorithm 3.13 implementing the above observations and is mostly analogous to it. It maintains a set $\mathcal{C}$ of cells covering the real line over the given sample point s . In the $\exists$ -case, these cells violate the input formula, and in the $\forall$ -case, these cells satisfy the input formula. In Line 2, it checks whether $\mathcal{C}$ already covers the real line. If yes, as before, it computes a base cell that generalizes s (Line 14)

using $\mathcal{C}$, along with a flag that corresponds to the given quantifier (Line 12). If not, it chooses an extension (s, r) of the sample point (Line 3) not covered by $\mathcal{C}$ and calls Algorithm 3.16 (Line 4) to obtain an implicit cell containing (s, r) on which the input formula is truth-invariant. Now, if the obtained cell satisfies (violates) the input formula in the $\exists$ -case ($\forall$ -case), it returns a projection of that cell and immediately returns (Lines 5 to 8). Otherwise, the obtained cell is added to $\mathcal{C}$ Line 10.

Implicitly, the algorithm computes a partial covering of $\mathbb{R}^n$ over the cell it returns. However, for reasons of brevity, Algorithm 3.17 does only return a flag indicating whether the returned cell satisfies or violates the input formula, and does not return a description of the partial covering (i.e. an encoding of the cells which witness whether the quantifier is fulfilled or not).

Algorithm 3.17: $\text{calc_quantifier}(Q, \varphi, s)$

Input : $Q \in \{\exists, \forall\}$, Tarski formula φ , and $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [0..n]$.

Output : $P' \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and $\mathbf{t} \in \{\text{true}, \text{false}\}$ such that for all $s' \in C(P', s)$ it holds $\varphi(s') \equiv \mathbf{t}$.

```

1  $\mathcal{C} := \emptyset$ 
2 while  $\bigcup_{(P, s') \in \mathcal{C}} \text{get\_interval}(P, s') \neq \mathbb{R}$  do // Algorithm 3.7
3   choose  $r \in \mathbb{R}$  s.t.  $r \notin \bigcup_{(P, s') \in \mathcal{C}} \text{get\_interval}(P, s')$ 
4    $(P, \mathbf{t}) := \text{calc\_recurse}(\varphi, (s, r))$  // Algorithm 3.16
5   if  $(Q = \exists \wedge \mathbf{t} = \text{true}) \vee (Q = \forall \wedge \mathbf{t} = \text{false})$  then
6      $I := \text{choose\_interval}(P, s)$  // Algorithm 3.8
7      $P' := \text{compute\_cell\_projection}(P, s, I)$  // Algorithm 3.9
8     return  $P', \mathbf{t}$ 
9   else
10     $\mathcal{C} := \mathcal{C} \cup \{(P, (s, r))\}$ 
11  $((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k)) := \text{choose\_covering}(\mathcal{C})$  // Algorithm 3.14
12 if  $Q = \exists$  then  $\mathbf{t} := \text{false}$  else  $\mathbf{t} := \text{true}$ 
13 if  $i > 0$  then
14    $P' := \text{compute\_covering\_projection}(((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k)))$  // Algorithm 3.15
15   return  $P', \mathbf{t}$ 
16 else
17   return  $\emptyset, \mathbf{t}$ 

```

3.4.2 Computing a CAIC using the (Simplified) NLSAT Algorithm

The NLSAT algorithm [Jd12] is an instantiation of the MCSAT framework [dJ13] (see Section 2.5.3 for details) for checking quantifier-free Tarski formulas for satisfiability. It interleaves the Boolean reasoning with the theory reasoning — in particular, it guesses values for theory variables similarly to the CAIC algorithm. Further, conflicts that occur are resolved similarly using a reduced CAD projection which already takes the given sample point into account. However, the original version does not incorporate any of the various insights from the work on single cell construction and cylindrical algebraic coverings. We thus start by introducing a modified version of the conflict resolution (called *explanations*) that uses these paradigms.

Due to the consistency requirement on the trail, conflicts occur when the current sample point cannot be extended for the next variable. In that case, some constraints on the trail cannot be satisfied with a single sample point. After plugging in the partial sample point $s \in \mathbb{R}^i$ (for some $i \in [0..n]$), the constraints define conflicting intervals for the next variable, and the union of these intervals cover the whole real line. Speaking with the

terminology from this chapter, each constraint $p \sim 0$ defines an implicit cell $(\{p\}, (s, r))$ (for some $r \in \mathbb{R}$) where $p \sim 0$ is violated, and the union of these implicit cells cover $s \times \mathbb{R}$. We now need to generate an explanation that generalizes s ; for doing so, we can apply the covering projection. On the result, we apply the single cell construction algorithm (Algorithm 3.6) to obtain symbolic intervals $I_1, \dots, I_i$ describing some unsatisfiable cell, in order to add a lemma $\neg \text{encoding}(I_1) \vee \dots \vee \neg \text{encoding}(I_i)$ to the clause database, guiding the future search of NLSAT. Algorithm 3.18 formalizes the explanation function for the NLSAT algorithm.

Algorithm 3.18: nlsat_explain(C, s)

Input : Set C of constraints such that for all $(p \sim 0) \in C$ it holds $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ and $\text{level}(p) = i + 1$ for some $i \in [0..n - 1]$, $s \in (\mathbb{R} \cap \mathbb{Q})^i$ such that $s \not\models \exists x_{i+1}. \bigwedge_{c \in C} c$.

Output : Explanation lemma ψ .

```

1  $\mathcal{C} := \emptyset$ 
2 foreach  $(p \sim 0) \in C$  do
3   sort elements of  $\text{realRoots}(p(s, x_{i+1})) = \{\xi_1, \dots, \xi_k\}$  such that  $\xi_1 < \dots < \xi_k$ 
4   foreach  $I \in \{(-\infty, \xi_1), [\xi_1, \xi_1], (\xi_1, \xi_2), \dots, (\xi_k, \infty)\}$  for  $k > 0$  and  $I \in \{(-\infty, \infty)\}$  for  $k = 0$ 
5     do
6       choose  $r \in I$ 
7       if  $p(s, r) \not\sim 0$  then
8          $\mathcal{C} := \mathcal{C} \cup \{(\{p\}, (s, r))\}$ 
9          $c^{\{p\}, (s, r)} := p \sim 0$ 
9  $((P^1, s^1, I^1), \dots, (P^k, s^k, I^k)) := \text{choose\_covering}(C)$  // Algorithm 3.14
10  $P' := \text{compute\_covering\_projection}(((P^1, s^1, I^1), \dots, (P^k, s^k, I^k)))$  // Algorithm 3.15
11  $(I_1, \dots, I_i) := \text{construct\_cell}(P', s)$  // Algorithm 3.6
12 return  $\bigwedge_{j \in [k]} c^{(P^j, s^j)} \rightarrow \neg(\bigwedge_{j \in [i]} \text{encoding}(I_j))$ 

```

The lemmas generated by Algorithm 3.18 may contain indexed root expressions. As these indexed root expressions may be part of a future conflict, the explanation backend also needs to be able to handle those inputs; as the adaptation is rather technical, we omit its presentation for the sake of brevity and refer to Section 4.9.

Instead, we draw attention to the fact that the NLSAT algorithm implicitly generates a cylindrical algebraic covering. Its shape, however, may be hidden in the technicalities of the MCSAT framework due to the application of Boolean reasoning and resolution on the obtained explanation functions.

We now present an algorithm that vaguely resembles the behaviour of NLSAT on conjunctions of constraints, and extends this behaviour to quantified problems. The main idea is to re-use already generated cells, which is done in NLSAT by applying the single cell construction and storing a formula that describes the cell and its semantics. In NLSAT, Boolean resolution will be applied to these formulas, effectively building their intersection. Thus, in our algorithm, we use sets of symbolic intervals for describing the resulting cells, and define their semantics as the “intersection” of its elements:

Definition 3.22 (Sets of Symbolic Intervals)

Let $i \in [n]$, J be a set of symbolic intervals of level i , and $R \subseteq \mathbb{R}^{i-1}$. We define $R \times \text{expansion}(J) = \bigcap_{I \in J} (R \times I)$.

Algorithm 3.19 is analogous to Algorithm 3.13 with the main difference that it does not output an implicit cell, but a sequence $(J_1, \dots, J_i)$ of sets of symbolic intervals of level

i respectively, representing the cell $\text{expansion}(J_1) \times \dots \times \text{expansion}(J_i)$. On the one hand, the explicit description of a cylindrical cell allows reusing it in another branch of the search tree where the sample point changed. On the other hand, the sets of symbolic intervals allow representing intersections of cylindrical cells.

Like in Algorithm 3.13, Algorithm 3.19 collects cells in the set $\mathcal{C}$, which cover the real line over the given sample point, in a loop starting at Line 2. In Line 4, it obtains such a cell, described as a sequence of sets of symbolic intervals $(J_1, \dots, J_{i+1})$. To build the implicit cell, it uses polynomials bounding the cell on the top level $i + 1$ in Line 5. The formula is truth-invariant on the implicit cell as long as we intersect its underlying cell with the first i levels $(J_1, \dots, J_i)$, which is why the algorithm stores this description as well.

After computing the covering projection in Line 12 that generalizes the given sample point s to a cell whose cylinder is covered by $\mathcal{C}$, the algorithm uses the single cell construction to obtain an explicit description of that cell in Line 13. To maintain correctness, the algorithm intersects the obtained cell with the underlying cells of the cells covering the cylinder by building the union of the cell descriptions in Line 14.

Algorithm 3.19: nlsat_full(φ, s)

Input : Tarski formula φ , and $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [0..n]$.

Output : Sets $J_1, \dots, J_i$ of symbolic intervals of levels $1, \dots, i$ respectively, and truth-labelled cylindrical set data structure $\mathbf{T}$ such that for each path $\mathbf{I}_{i+1}, \dots, \mathbf{I}_{i'}$ in $\mathbf{T}$ to a label $\mathbf{t}$ and for each $s' \in \text{expansion}(J_1) \times \dots \times \text{expansion}(J_i) \times \mathbf{I}_{i+1} \times \dots \times \mathbf{I}_{i'}$ it holds that $\varphi(s') \equiv \mathbf{t}$.

```

1  $\mathcal{C} := \emptyset$ 
2 while  $\bigcup_{(P,s') \in \mathcal{C}} \text{get\_interval}(P, s') \neq \mathbb{R}$  do // Algorithm 3.7
3   choose  $r \in \mathbb{R}$  s.t.  $r \notin \bigcup_{(P,s') \in \mathcal{C}} \text{get\_interval}(P, s')$ 
4    $((J_1, \dots, J_{i+1}), \mathbf{T}) := \text{nlsat\_recurse}(\varphi, (s, r))$  // Algorithm 3.20
5    $P := \{\mathbf{I}_{i+1}.l.p \mid \mathbf{I}_{i+1} \in J_{i+1}, \mathbf{I}_{i+1}.l \neq -\infty\} \cup \{\mathbf{I}_{i+1}.u.p \mid \mathbf{I}_{i+1} \in J_{i+1}, \mathbf{I}_{i+1}.u \neq \infty\}$ 
6    $((J_1)_{(P,(s,r))}, \dots, (J_{i+1})_{(P,(s,r))}) := (J_1, \dots, J_{i+1})$ 
7    $\mathbf{T}_{(P,(s,r))} := \mathbf{T}$ 
8    $\mathcal{C} := \mathcal{C} \cup \{(P, (s, r))\}$ 
9  $((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k)) := \text{choose\_covering}(\mathcal{C})$  // Algorithm 3.14
10  $\mathbf{T} := \{(\mathbf{I}^j, \mathbf{T}_{(P^j, s^j)}) \mid j \in [k]\}$ 
11 if  $i > 0$  then
12    $P' := \text{compute\_covering\_projection}(((P^1, s^1, \mathbf{I}^1), \dots, (P^k, s^k, \mathbf{I}^k)))$  // Algorithm 3.15
13    $(\mathbf{I}_1, \dots, \mathbf{I}_i) := \text{construct\_cell}(P', s)$  // Algorithm 3.6
14   foreach  $i' \in [i]$  do  $J_{i'} := \{\mathbf{I}_{i'}\} \cup \bigcup_{j \in [k]} (J_{i'})_{(P^j, s^j)}$ 
15   return  $(J_1, \dots, J_i), \mathbf{T}$ 
16 else
17   return  $\emptyset, \mathbf{T}$ 

```

Algorithm 3.20 is analogous to Algorithm 3.11. Again, the main difference is the type of data structures describing the cells, which is why we need to use the single cell construction in Lines 5 and 10. Furthermore, it maintains a global variable D_φ which is persistent across all calls to this function. We store all constructed cells in this set, and check at the beginning in Line 1 whether we already generated the required cell and return this instead of re-computing the cell. The crucial point of this adaptation is that we can check the membership of a sample point in a previously generated cell (Line 1) — which is not possible for implicit cells.

To summarize, NLSAT and CAIC roughly differ in the following two aspects. Firstly,

Algorithm 3.20: `nlsat_recurse( $\varphi, s$ )`

Input : Tarski formula φ in prenex normal form, and $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [0..n]$.
Output : Sets $J_1, \dots, J_i$ of symbolic intervals of levels $1, \dots, i$ respectively, and either (1) $\mathbf{t} \in \{\text{true}, \text{false}\}$ such that for all $s' \in \text{expansion}(J_1) \times \dots \times \text{expansion}(J_i)$ it holds $\varphi(s') \equiv \mathbf{t}$, or (2) truth-labelled cylindrical set data structure T such that for each path $I_{i+1}, \dots, I_{i'}$ in T to a label $\mathbf{t}$ and for each $s' \in \text{expansion}(J_1) \times \dots \times \text{expansion}(J_i) \times I_{i+1} \times \dots \times I_{i'}$ it holds that $\varphi(s') \equiv \mathbf{t}$.
// We assume that $\mathcal{D}_\varphi$ is a global variable, i.e. it is shared between calls to this function.

```

1 if there is  $((J_1, \dots, J_i), T) \in \mathcal{D}_\varphi$  s.t.  $s \in \text{expansion}(J_1) \times \dots \times \text{expansion}(J_i)$  then
2   return  $(J_1, \dots, J_i), T$ 
3 if  $\varphi[s] \equiv \text{false}$  then
4    $P := \text{choose\_enclosing\_cell}(\varphi, s)$  // Algorithm 3.12
5    $I_1, \dots, I_i := \text{construct\_cell}(P, s)$  // Algorithm 3.6
6    $\mathcal{D}_\varphi := \mathcal{D}_\varphi \cup \{((\{I_1\}, \dots, \{I_i\}), \text{false})\}$ 
7   return  $(\{I_1\}, \dots, \{I_i\}), \text{false}$ 
8 else if  $\varphi[s] \equiv \text{true}$  then
9    $P := \text{choose\_enclosing\_cell}(\varphi, s)$  // Algorithm 3.12
10   $I_1, \dots, I_i := \text{construct\_cell}(P, s)$  // Algorithm 3.6
11   $\mathcal{D}_\varphi := \mathcal{D}_\varphi \cup \{((\{I_1\}, \dots, \{I_i\}), \text{true})\}$ 
12  return  $(\{I_1\}, \dots, \{I_i\}), \text{true}$ 
13 else // it holds  $i < n$ 
14    $(J_1, \dots, J_i), T := \text{nlsat\_full}(\psi, s)$  // Algorithm 3.19
15    $\mathcal{D}_\varphi := \mathcal{D}_\varphi \cup \{((\{I_1\}, \dots, \{I_i\}), \text{true})\}$ 
16   return  $(J_1, \dots, J_i), T$ 

```

NLSAT eagerly computes explicit representations of cells for re-using them in other branches of the search tree, while CAIC avoids such computations at the cost of having only single-use cells. Secondly, NLSAT views each bound on the cell as separate literal, and implicitly stores their relations in the Boolean structure; while the literals are considered “out-of-context” when forming a covering, the relations are restored during Boolean resolution. Algorithms 3.19 and 3.20 resemble these two steps explicitly. CAIC only uses “well-defined” truth-invariant cells.

We remark that Algorithm 3.17 for handling quantifiers can be adapted similarly. Furthermore, other mixtures of the CAIC and NLSAT algorithm are conceivable, in particular, extensions of MCSAT than can mimic the CAIC algorithm. We leave this for future work.

3.5 Non-uniform CAD

The CAD and CAIC maintain a cylindrical arrangement of the cells. Maintaining a global cylindrical arrangement requires cutting the cells such that they can be arranged into cylinders. The *Non-uniform CAD (NuCAD)* maintains an alternative arrangement of the cells, where each cell is still locally cylindrical, but which does not maintain a global cylindrical arrangement. However, we still partition the space into cells which are arranged cylindrically (that is, the cells form a cylindrical decomposition), but these cells might be split into cylindrically arranged cells recursively. We thus do not have a single global cylindrical arrangement of the cells, but a *non-uniform* cylindrical arrangement:

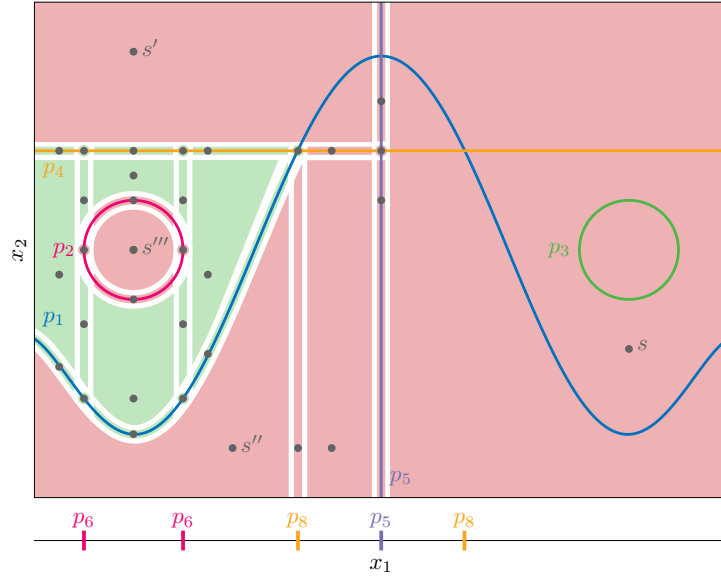


Figure 3.24: Truth-invariant non-uniform CAD for Example 3.25, along with a sample point for each cell.

Definition 3.23 (Non-uniform CAD [Bro15])

Let $i \in [n]$, $R \subseteq \mathbb{R}^i$ be an algebraic cell.

A *non-uniform cylindrical algebraic decomposition (NuCAD)* of R is a decomposition D of R that is algebraic and consists of locally cylindrical cells such that there exist a partition $P_1, \dots, P_k \subseteq D$ of D (that is $P_1 \cup \dots \cup P_k = D$ is a disjoint union) such that $\{\bigcup_{C \in P_j} C \mid j \in [k]\}$ is cylindrical, and for each $j \in [k]$ the set P_j consists either of a single element or is a NuCAD of $\bigcup_{C \in P_j} C$.

Example 3.25

We consider the formula from Example 3.9 on Page 70 again. A truth-invariant NuCAD for the input formula is depicted in Figure 3.24. Figure 3.25 displays some cells and their partition from Definition 3.23. The blue cells are recursively decomposed into a NuCAD, while the green and red cells are not.

We store a NuCAD in a tree-like data structure (similar to the cylindrical set data structure), but its leaves may contain a NuCAD data structure recursively, for representing how a cell is partitioned into groups of smaller cells. This way, the data structure keeps some information about adjacencies of cells encoded in the NuCAD.

Definition 3.24 (Labelled NuCAD Data Structure)

A *labelled NuCAD data structure* of level $i \in [n]$ w.r.t. global level $i' \in [i..n]$ is defined like a truth-labelled cylindrical set data structure (Definition 3.8 on page 64 and Definition 3.13 on page 74), but each leaf $(I, \emptyset)$ is extended to either $(I, \mathfrak{t})$ where $\mathfrak{t} \in \{\text{true}, \text{false}\}$, or (I, T) where T is a labelled NuCAD data structure of level 1 w.r.t. global level i' .

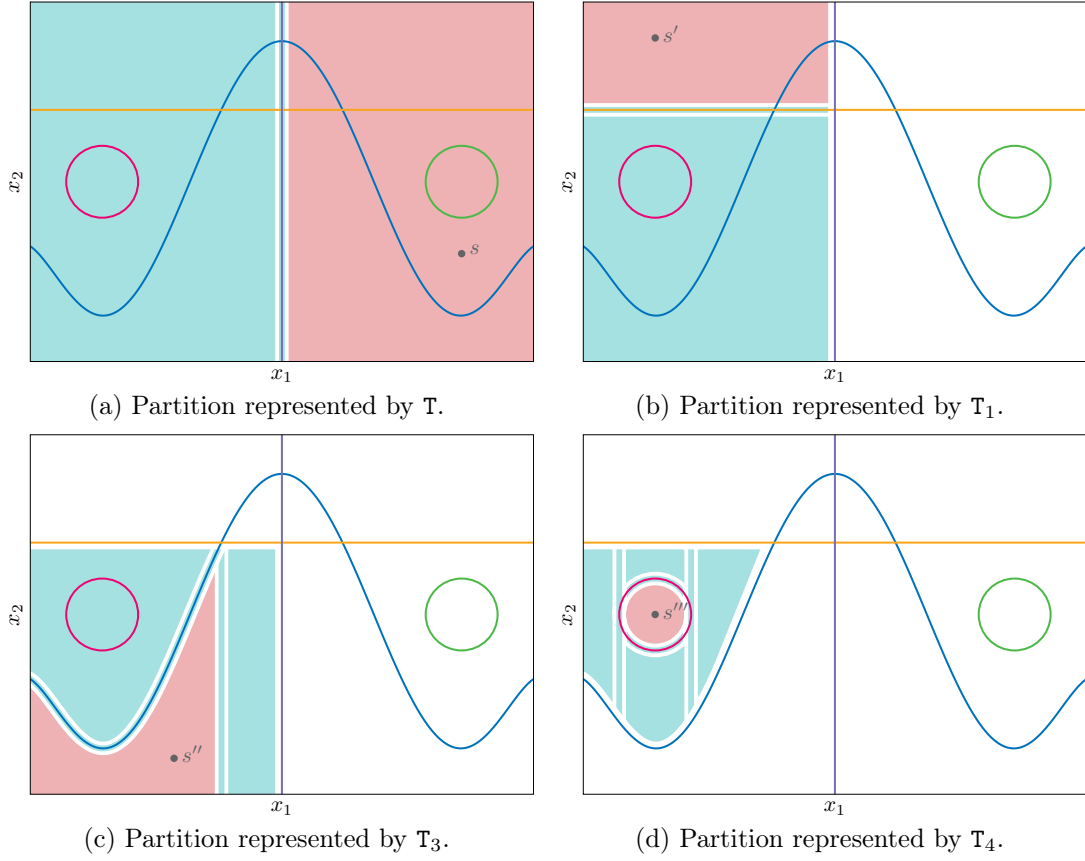


Figure 3.25: Some partitions of cells in the NuCAD from Figures 3.24 and 3.26.

For a labelled NuCAD data structure T_i of level i w.r.t. global level i' and $\mathbf{t} \in \{\text{true}, \text{false}\}$ or a labelled NuCAD data structure $\mathbf{t}$ of level 1 w.r.t. global level i' , a *path to $\mathbf{t}$ in T_i* is a sequence $(I_i, \dots, I_n)$ such that $(I_i, T_{i+1}) \in T_i$, $(I_{i+1}, T_{i+2}) \in T_{i+1}, \dots, (I_n, \mathbf{t}) \in T_n$.

Example 3.26

The NuCAD depicted in Figure 3.24 is represented by the tree partially depicted in Figures 3.25 and 3.26.

Analogously to a CAIC over a cell (Definition 3.20 on page 94; that is, part of the space is covered by cylindrically arranged cells whose projection is the given cell), we define a NuCAD over a cell. This notion allows us to define combinations of CAIC and NuCAD, and is also relevant for the algorithm in the following subsection.

Definition 3.25

Let $i \in [0..n]$, $i' \in [i+1..n]$, and $R \subseteq \mathbb{R}^i$, $R' \subseteq \mathbb{R}^{i'}$ be algebraic cells. A *NuCAD of R' over R* is a NuCAD D of R such that $R'' \downarrow_{[i]} = R' \downarrow_{[i]} = R$ for all $R'' \in D$.

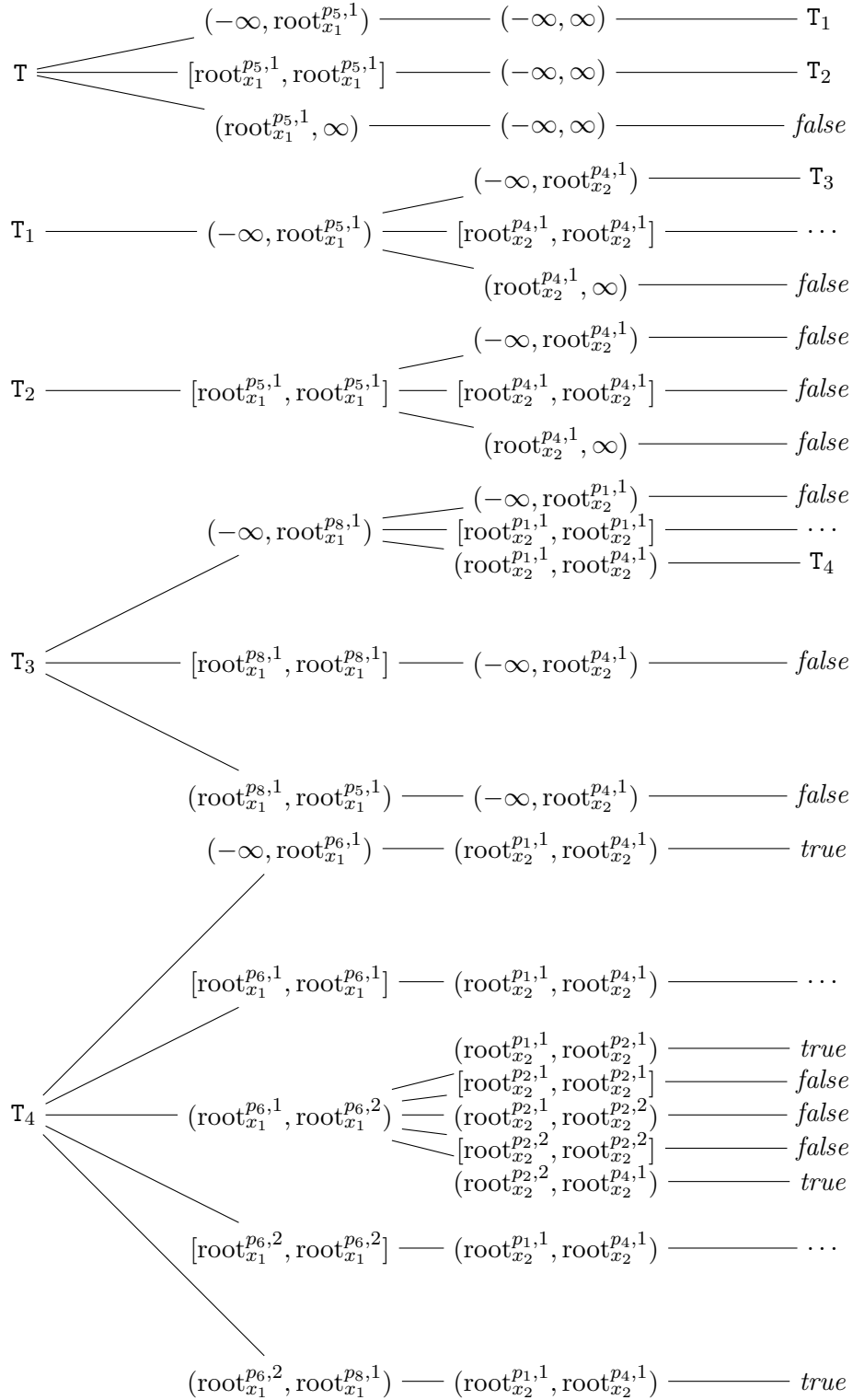


Figure 3.26: NuCAD data structure that represents the NuCAD in Figure 3.24.

3.5.1 Computing a NuCAD

We introduce an extension of the NuCAD algorithm, originally formalized by Brown [Bro15], to compute an *open NuCAD* of $\mathbb{R}^n$, where only the cells of full dimension are considered (as in the open CAD). Our extension computes a full NuCAD, including the cells of lower dimension.

We first give a brief overview of the algorithm for checking the satisfiability of existentially quantified conjunctions of constraints: The method starts by guessing some sample point. If all input constraints are satisfied, it terminates with a positive answer. Otherwise, it takes a constraint which is violated at the current sample. It then generates a single cell around the sample where the defining polynomial is sign-invariant (and thus where the input is false). This cell is excluded from further search. Now, the unexplored parts may be described by locally cylindrical cells; e.g. for a two-dimensional cell, we take the cells below, above, left and right of the cell. The algorithm then recursively decomposes these cells. It selects a cell and guesses another sample point (now within the given cell), determines a violated constraint (or terminates), and again constructs a cell where the constraint is false. For the latter, it takes not only the defining polynomial of the constraint as input, but also the polynomials that define the boundaries of the given cell. The resulting cell is thus a subset of the given cell. Analogously to before, the algorithm recursively decomposes the remaining unexplored parts. The base case of the recursion is reached when some constructed truth-invariant cell already covers the whole of the input cell of the respective call. If the algorithm did not find a satisfying solution, the whole of $\mathbb{R}^n$ is covered by cells where the input is violated.

We now present the generalized version in Algorithm 3.21 which we explain in the following.

Generating a Truth-Invariant Cell Assume that we want to generate a NuCAD of some input cell $R = I_1 \times \dots \times I_n \subseteq \mathbb{R}^n$ (initially $(-\infty, \infty) \times \dots \times (-\infty, \infty)$, or a cell from a recursive call) for a quantifier-free formula φ using Algorithm 3.21. We start by picking a sample point contained in R in Line 1, and in Line 2, we compute an implicant for φ w.r.t. s (as in the CAIC algorithm) and obtain a corresponding implicit cell. In the loop starting at Line 4, we apply the single cell construction algorithm on the obtained implicit cell. If $R \neq \mathbb{R}^n$, then we want to compute a cell that is a subset of R , we thus insert the defining polynomials of $I_1, \dots, I_n$ to P in Line 3 before constructing the single cell. The result $I'_1 \times \dots \times I'_n$ is a cell where the formula is invariantly true or false.

Splitting the Cell By construction, the obtained cell $R' = I'_1 \times \dots \times I'_n$ is a subset of R , and φ is truth-invariant on R' . What remains is to explore the remaining parts $R \setminus R'$. We do so by splitting R using R' into multiple cylindrical cells in Lines 10 to 18: We observe that a point is outside the cell if it violates one bound. We thus iterate through each level i and encode that the first $i - 1$ levels are in R' ; on the i -th level, we leave R' (we are either below or above the cell, if possible) but stay in R ; and on the levels starting from $i + 1$, we remain in R , but do not make assumptions about the bounds on R' , as we already left R' on the i -th level.

Recursive Calls After this process, we have a set Q of unexplored disjoint cells which are locally cylindrical, i.e. they are described by symbolic intervals for each

Algorithm 3.21: `nucad_full( $\varphi, (I_1, \dots, I_n)$ )`

Input : Quantifier-free Tarski formula φ and symbolic intervals $(I_1, \dots, I_n)$.
Output : NuCAD data structure T of level 1 w.r.t. global level n representing a truth-invariant NuCAD of $I_1 \times \dots \times I_n$ for φ .

```

/* Sampling and Cell Construction */
1 choose  $r \in (\mathbb{R} \cap \mathbb{Q})^n$  s.t.  $r \in I_1 \times \dots \times I_n$ 
2  $P, t := \text{choose\_enclosing\_cell}(\varphi, r)$  // Algorithm 3.12
3  $P := P \cup \{I_{j,l,p} \mid j \in [1..n], I_{j,l} \neq -\infty\} \cup \{I_{j,u,p} \mid j \in [1..n], I_{j,u} \neq \infty\}$ 
4 foreach  $j = n, \dots, 1$  do
5    $I'_j := \text{choose\_interval}(P, r_{[j]})$  // Algorithm 3.8
6    $P := \text{compute\_cell\_projection}(P, r_{[j]}, I'_j)$  // Algorithm 3.9
7  $T := \emptyset$ 
8 insert path  $I'_1, \dots, I'_n$  to  $t$  into  $T$ 
/* Splitting the Input Cell */
9  $Q := \emptyset$ 
10 foreach  $j = 1, \dots, n$  do
11   if  $I'_{j,l} \neq I_{j,l}$  then
12      $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, (I_{j,l}, I'_{j,l}), I_{j+1}, \dots, I_n)\}$ 
13     if  $I'_j$  is a sector then
14        $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, [I'_{j,l}, I'_{j,l}], I_{j+1}, \dots, I_n)\}$ 
15   if  $I'_{j,u} \neq I_{j,u}$  then
16      $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, (I'_{j,u}, I_{j,u}), I_{j+1}, \dots, I_n)\}$ 
17     if  $I'_j$  is a sector then
18        $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, [I'_{j,u}, I'_{j,u}], I_{j+1}, \dots, I_n)\}$ 
/* Recursive Calls */
19 foreach  $(I'_1, \dots, I'_n) \in Q$  do
20    $T' := \text{nucad\_full}(\varphi, (I'_1, \dots, I'_n))$ 
21   insert path  $I'_1, \dots, I'_n$  to  $T'$  into  $T$ 
22 return  $T$ 

```

variable. Thus, we call the NuCAD algorithm recursively on each cell in Q in Line 20 to explore the remaining parts of R . If we find that $R = R'$ ($I_1 = I'_1, \dots, I_n = I'_n$), the algorithm returns without any recursive calls. This is the case in particular when the whole formula is already truth-invariant in R . At latest, this is the case when R is contained in the sign-invariant CAD w.r.t. the set of polynomials in φ that would be computed using the employed projection operator.

Example 3.27

Consider the formula $\varphi(x_1, x_2) = p_1 \leq 0 \wedge p_2 > 0 \wedge p_3 \geq 0 \wedge p_4 \leq 0 \wedge p_5 \leq 0$ and the polynomials from Example 3.9 on Page 70 again. We give an exemplary run of Algorithm 3.21 that computes the NuCAD as depicted in Figures 3.24 and 3.26.

`nucad_full( $\varphi, ((-\infty, \infty), (-\infty, \infty))$ )` We choose $s = (2.5, 0.5)$ as a sample point. The formula φ evaluates to *false* at s , we obtain the implicant $p_5 > 0$, and thus obtain $(\{p_5\}, s)$ as an implicit cell. By using single cell construction, we obtain the cell described by $I'_1 \times I'_2 = (\text{root}_{x_1}^{p_5, 1}, \infty) \times (-\infty, \infty)$. Accordingly, we split $(-\infty, \infty) \times (-\infty, \infty)$ into $(-\infty, \text{root}_{x_1}^{p_5, 1}) \times (-\infty, \infty)$, $[\text{root}_{x_1}^{p_5, 1}, \text{root}_{x_1}^{p_5, 1}] \times (-\infty, \infty)$, and $I'_1 \times I'_2$, as depicted in Figure 3.25a.

`nucad_full( $\varphi, ((-\infty, \text{root}_{x_1}^{p_5, 1}), (-\infty, \infty))$ )` We choose the sample point $s' = (-2.5, 3.5)$, and obtain the implicit cell $(\{p_4\}, s')$ where φ is truth-

invariant. As we aim to compute a truth-invariant cell that is a subset of $(-\infty, \text{root}_{x_1}^{p_5,1}) \times (-\infty, \infty)$, we run the single cell construction on $(\{p_4, p_5\}, s')$. We obtain $I'_1 \times I'_2 = (-\infty, \text{root}_{x_1}^{p_5,1}) \times (\text{root}_{x_2}^{p_4,1}, \infty)$, and split $(-\infty, \infty) \times (-\infty, \infty)$ into $(-\infty, \text{root}_{x_1}^{p_5,1}) \times (-\infty, \text{root}_{x_2}^{p_4,1})$, $(-\infty, \text{root}_{x_1}^{p_5,1}) \times [\text{root}_{x_2}^{p_4,1}, \text{root}_{x_2}^{p_4,1}]$, and $I'_1 \times I'_2$, as depicted in Figure 3.25b.

`nucad_full`($\varphi, ((-\infty, \text{root}_{x_1}^{p_5,1}), (-\infty, \text{root}_{x_2}^{p_4,1}))$) We choose $s'' = (-1.5, -0.5)$, and obtain $(\{p_1\}, s'')$ from the implicant. We run single cell construction on $(\{p_1, p_4, p_5\}, s'')$ and obtain $(-\infty, \text{root}_{x_1}^{p_8,1}) \times (-\infty, \text{root}_{x_2}^{p_1,1})$ (where p_8 is the resultant of p_1 and p_4), and split the input cell into the cells as indicated in Figure 3.25c and the following recursive calls:

```

nucad_full( $\varphi, ([\text{root}_{x_1}^{p_8,1}, \text{root}_{x_1}^{p_8,1}], (-\infty, \text{root}_{x_2}^{p_4,1}))$ ) ...
nucad_full( $\varphi, ((\text{root}_{x_1}^{p_8,1}, \text{root}_{x_1}^{p_5,1}), (-\infty, \text{root}_{x_2}^{p_4,1}))$ ) ...
nucad_full( $\varphi, ((-\infty, \text{root}_{x_1}^{p_8,1}), [\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_1,1}])$ ) ...
nucad_full( $\varphi, ((-\infty, \text{root}_{x_1}^{p_8,1}), (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}))$ ) We
  choose  $s''' = (-2.5, 1.5)$ , and obtain  $(\{p_1, p_2, p_4\}, s''')$  from the
  implicant. We run single cell construction on  $(\{p_1, p_2, p_4\}, s''')$ 
  and obtain  $(\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,2}) \times (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_2,2})$  (where  $p_6$ 
  is the resultant of  $p_2$ ), and split the input cell into the cells as
  indicated in Figure 3.25d and the following recursive calls:
  nucad_full( $((-\infty, \text{root}_{x_1}^{p_6,1}), (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}))$ ) ...
  nucad_full( $([\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,1}], (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}))$ ) ...
  nucad_full( $([\text{root}_{x_1}^{p_6,2}, \text{root}_{x_1}^{p_6,2}], (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}))$ ) ...
  nucad_full( $((\text{root}_{x_1}^{p_6,2}, \text{root}_{x_1}^{p_8,1}), (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}))$ ) ...
  nucad_full( $((\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,2}), (\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_2,1}))$ ) ...
  nucad_full( $((\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,2}), [\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_2,1}])$ ) ...
  nucad_full( $((\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,2}), [\text{root}_{x_2}^{p_2,2}, \text{root}_{x_2}^{p_2,2}])$ ) ...
  nucad_full( $((\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_6,2}), (\text{root}_{x_2}^{p_2,2}, \text{root}_{x_2}^{p_4,1}))$ ) ...
nucad_full( $\varphi, ((-\infty, \text{root}_{x_1}^{p_5,1}), [\text{root}_{x_2}^{p_4,1}, \text{root}_{x_2}^{p_4,1}])$ ) ...
nucad_full( $\varphi, ([\text{root}_{x_1}^{p_5,1}, \text{root}_{x_1}^{p_5,1}], (-\infty, \infty))$ ) ...

```

3.5.2 Quantifiers

Algorithm 3.21 computes a NuCAD of $\mathbb{R}^n$. The arrangement of its cells does not admit reasoning about quantifier alternations as for the CAD. The work in [Bro17] extends the NuCAD algorithm for handling quantifiers by refining a NuCAD of $\mathbb{R}^n$ such that reasoning about quantifier blocks is possible. Borrowing ideas from the CaIC algorithm, we present an alternative extension which directly computes a formula adapted to the quantifier structure, making the refinement step in [Bro17] obsolete and potentially reduces the effort for its computation as, depending on the quantifiers, we only need to compute parts of the NuCAD.

For handling quantifier alternations, we use the notion of a NuCAD over a cell (Definition 3.25). We group the variables by the quantifier blocks, and compute a NuCAD

block-by-block: Assuming the input

$$\forall x_{k+1} \dots \forall x_{k+k'} \exists x_{k+k'+1} \dots \exists x_{k+k'+k''}. \varphi(x_1, \dots, x_{k+k'+k''})$$

we compute a (truth-invariant) NuCAD D of the first k levels; and for each cell of D , we compute a (truth-invariant) NuCAD D' of the next k' levels; and for each cell of D' , we compute a (truth-invariant) NuCAD D'' of the next k'' levels. This structure allows to recursively determine the truth value of each cell by analysing the truth values of the cells of the NuCAD over that cell.

Algorithm 3.22: `nucad_parameter`($\varphi, s, (I_{i+1}, \dots, I_{i'})$)

Input : Tarski formula $\varphi, s \in (\mathbb{R} \cap \mathbb{Q})^i$ for an $i \in [0..n]$, and symbolic intervals $(I_{i+1}, \dots, I_{i'})$ for an $i' \in [i+1..n]$ such that $x_1, \dots, x_{i'}$ are free in φ .

Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and NuCAD data structure T of level i w.r.t. global level i' representing a truth-invariant NuCAD of $C(P, s) \times I_{i+1} \times \dots \times I_{i'}$ over $C(P, s)$ for φ .

/ Sampling and Cell Construction */*

```

1 choose  $r \in (\mathbb{R} \cap \mathbb{Q})^{i'}$  s.t.  $r \in s \times I_{i+1} \times \dots \times I_{i'}$ 
2  $P, t := \text{nucad\_recurse}(\varphi, r)$  // Algorithm 3.23
3  $P := P \cup \{I_j.l.p \mid j \in [i+1..i'], I_j.l \neq -\infty\} \cup \{I_j.u.p \mid j \in [i+1..i'], I_j.u \neq \infty\}$ 
4 foreach  $j = i+1, \dots, i'$  do
5    $I'_j := \text{choose\_interval}(P, r_{[j]})$  // Algorithm 3.8
6    $P := \text{compute\_cell\_projection}(P, r_{[j]}, I'_j)$  // Algorithm 3.9
7  $T := \emptyset$ 
8 insert path  $I'_{i+1}, \dots, I'_{i'}$  to  $t$  into  $T$ 
/* Splitting the Input Cell */
9  $Q := \emptyset$ 
10 foreach  $j = i+1, \dots, i'$  do
11   if  $I'_j.l \neq I_j.l$  then
12      $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, (I_j.l, I'_j.l), I_{j+1}, \dots, I_{i'})\}$ 
13     if  $I'_j$  is a sector then
14        $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, [I'_j.l, I'_j.l], I_{j+1}, \dots, I_{i'})\}$ 
15   if  $I'_j.u \neq I_j.u$  then
16      $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, (I'_j.u, I_j.u), I_{j+1}, \dots, I_{i'})\}$ 
17     if  $I'_j$  is a sector then
18        $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, [I'_j.u, I'_j.u], I_{j+1}, \dots, I_{i'})\}$ 
/* Recursive Calls */
19 foreach  $(I'_{i+1}, \dots, I'_{i'}) \in Q$  do
20    $P', T' := \text{nucad\_parameter}(\varphi, s, (I'_{i+1}, \dots, I'_{i'}))$ 
21    $P := P \cup P'$ 
22   insert path  $I'_{i+1}, \dots, I'_{i'}$  to  $T'$  into  $T$ 
23 return  $P, T$ 

```

Algorithm 3.22 computes a NuCAD over some cell; the changes compared to Algorithm 3.21 are highlighted in grey. If we call Algorithm 3.22 with a sample point $s \in \mathbb{R}^i$, $i' < n$, and symbolic intervals $((-\infty, \infty), \dots, (-\infty, \infty))$, it will compute a NuCAD over some cell around s of $\mathbb{R}^{i'}$. For doing so, it fixes the first i levels of the guessed sample point in Line 1, applies the single cell construction only from level i' to level $i+1$ in Line 4, applies the splitting accordingly in Lines 10 to 18, and — most importantly for the correctness — collects the polynomials that characterize the NuCAD. This set consists of the polynomials of level i and below that are left over from the single cell construction, and the projections obtained from the recursive calls to NuCAD (see Line 21). Although we do not compute a NuCAD of $\mathbb{R}^n$ but of $\mathbb{R}^{i'}$, the call to Algorithm 3.23 in Line 2 obtains a truth-invariant cell for φ even if we are only given a partial sample point.

Algorithm 3.23 is similar to Algorithm 3.11 from the CAIC algorithm. It decides whether to call Algorithm 3.12 or call the NuCAD algorithm based on Algorithm 3.22. For handling formulas with quantifiers (in prenex normal form), it “structures” the NuCAD such that we construct a NuCAD for each quantifier block separately.

Algorithm 3.23: `nucad_recurse( $\varphi, s$ )`

Input : Tarski formula φ in prenex normal form, and $s \in (\mathbb{R} \cap \mathbb{Q})^i$ for some $i \in [0..n]$.
Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and either (1) $\mathbf{t} \in \{true, false\}$ such that for all $s' \in C(P, s)$ it holds $\varphi(s') \equiv \mathbf{t}$, or (2) NuCAD data structure T representing a truth-invariant NuCAD of $C(P, s) \times \mathbb{R}^{i'-i}$ over $C(P, s)$ for φ for some $i' \in [i+1..n]$ such that $x_1, \dots, x_{i'}$ are free in φ .

```

1 if  $\varphi[s] \equiv false \vee \varphi[s] \equiv true$  then
2   | return choose_enclosing_cell( $\varphi, s$ )                                // Algorithm 3.12
3 else                                                                    // it holds  $i < n$ 
4   | if  $\varphi = \exists x_{i+1} \dots \exists x_{i'}. \psi$  for some  $i' \in [i+1..n]$  then
5   |   | return nucad_quantifier( $\exists, \psi, s, (-\infty, \infty)^{i'}$ )          // Algorithm 3.24
6   | else if  $\varphi = \forall x_{i+1} \dots \forall x_{i'}. \psi$  for some  $i' \in [i+1..n]$  then
7   |   | return nucad_quantifier( $\forall, \psi, s, (-\infty, \infty)^{i'}$ )          // Algorithm 3.24
8   | else
9   |   | let  $i' \in [i+1..n]$  such that  $x_{i+1}, \dots, x_{i'}$  are free in  $\varphi$ 
10  |   | return nucad_parameter( $\psi, s, (-\infty, \infty)^{i'}$ )              // Algorithm 3.22

```

We could use Algorithm 3.22 for quantifier blocks: replace all calls in Algorithm 3.23 to Algorithm 3.24 by calls to Algorithm 3.22 and, for the exists (forall) quantifier blocks, return the label *true* if and only if at least one (all) cells are labelled *true*.

However, similarly to the CAIC algorithm, we can terminate the NuCAD construction earlier, depending on the quantifier: If the given variables are existentially quantified, we can stop when we find a cell where the input formula is *true*, and return a projection of that cell (instead of the NuCAD). Analogously, if the given variables are universally quantified, we stop when we find a cell where the input formula is *false*. Algorithm 3.24 implements this idea and differs from Algorithm 3.22 only in three aspects. If a call to Algorithm 3.23 yields a desired cell, we call the single cell construction and return the result in Line 7. If a recursive call to itself yields such a cell, we return that cell in Line 25, omitting the intermediate results from the NuCAD computation. Further, it does not maintain a NuCAD data structure for the sake of brevity.

3.5.3 Solution Formulas

For constructing a solution formula, we need to encode the NuCAD. As for the CAD and the CAIC, we may encode the cells that are labelled with *true*; note that we do not need to encode all the cells in the nested data structure, but only the *outermost* (i.e. the bounds in the nodes closest to the leaves) ones.

However, also a NuCAD data structure may be simplified, similarly as described in Section 3.2.6. In fact, these techniques can be applied always to its outermost cylindrical tree data structures. Additionally, if it turns out that a cell has been split but all resulting cells carry the same label, we can undo the split by removing the subtree below the given cell.

Algorithm 3.24: `nucad_quantifier`($Q, \varphi, s, (I_{i+1}, \dots, I_{i'})$)

Input : $Q \in \{\exists, \forall\}$, Tarski formula φ , $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [0..n]$, and symbolic intervals $(I_{i+1}, \dots, I_{i'})$ for an $i' \in [i+1..n]$.

Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and $\mathbf{t} \in \{\text{true}, \text{false}\}$ such that for all $s' \in C(P, s)$ it holds $\exists x_{i+1} \dots \exists x_{i'}. x_{i+1} \in I_{i+1} \wedge \dots \wedge x_{i'} \in I_{i'} \wedge \varphi(s') \equiv \mathbf{t}$ (if $Q = \exists$) or $\forall x_{i+1} \dots \forall x_{i'}. (x_{i+1} \in I_{i+1} \wedge \dots \wedge x_{i'} \in I_{i'}) \rightarrow \varphi(s') \equiv \mathbf{t}$ (if $Q = \forall$).

/ Sampling and Cell Construction */*

```

1 choose  $r \in (\mathbb{R} \cap \bar{\mathbb{Q}})^{i'}$  s.t.  $r \in s \times I_{i+1} \times \dots \times I_{i'}$ 
2  $P, \mathbf{t} := \text{nucad\_recurse}(\varphi, r)$  // Algorithm 3.23
3 if  $(Q = \exists \wedge \mathbf{t} = \text{true}) \vee Q = \forall \wedge \mathbf{t} = \text{false}$  then
4   foreach  $j = i', \dots, i+1$  do //  $j$  is decreasing
5      $I'_j := \text{choose\_interval}(P, r_{[j]})$  // Algorithm 3.8
6      $P := \text{compute\_cell\_projection}(P, r_{[j]}, I'_j)$  // Algorithm 3.9
7   return  $P, \mathbf{t}$ 
8  $P := P \cup \{I_j.l.p \mid j \in [i+1..i'], I_j.l \neq -\infty\} \cup \{I_j.u.p \mid j \in [i+1..i'], I_j.u \neq \infty\}$ 
9 foreach  $j = i', \dots, i+1$  do
10    $I'_j := \text{choose\_interval}(P, r_{[j]})$  // Algorithm 3.8
11    $P := \text{compute\_cell\_projection}(P, r_{[j]}, I'_j)$  // Algorithm 3.9

```

/ Splitting the Input Cell */*

```

12  $Q := \emptyset$ 
13 foreach  $j = i+1, \dots, i'$  do
14   if  $I'_j.l \neq I_j.l$  then
15      $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, (I_j.l, I'_j.l), I_{j+1}, \dots, I_{i'})\}$ 
16     if  $I'_j$  is a sector then
17        $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, [I'_j.l, I'_j.l], I_{j+1}, \dots, I_{i'})\}$ 
18   if  $I'_j.u \neq I_j.u$  then
19      $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, (I'_j.u, I_j.u), I_{j+1}, \dots, I_{i'})\}$ 
20     if  $I'_j$  is a sector then
21        $Q := Q \cup \{(I'_{i+1}, \dots, I'_{j-1}, [I'_j.u, I'_j.u], I_{j+1}, \dots, I_{i'})\}$ 

```

/ Recursive Calls */*

```

22 foreach  $(I'_{i+1}, \dots, I'_{i'}) \in Q$  do
23    $P', \mathbf{t} := \text{nucad\_quantifier}(Q, \varphi, s, (I'_{i+1}, \dots, I'_{i'}))$ 
24   if  $(Q = \exists \wedge \mathbf{t} = \text{true}) \vee Q = \forall \wedge \mathbf{t} = \text{false}$  then
25     return  $P', \mathbf{t}$ 
26    $P := P \cup P'$ 

```

27 **if** $Q = \exists$ **then** $\mathbf{t} := \text{false}$ **else** $\mathbf{t} := \text{true}$

28 **return** $P, \mathbf{t}$

Example 3.28

Consider Figure 3.24 on page 105 and Figure 3.26 on page 107. The cell $[\text{root}_{x_1}^{p_{5,1}}, \text{root}_{x_1}^{p_{5,1}}] \times (-\infty, \infty)$ is split into multiple cells which carry the same label (as NuCAD requires different implicants to ensure the truth value), thus these cells can be merged in the post-processing of the NuCAD. Similarly, the children of $(-\infty, \text{root}_{x_1}^{p_{8,1}}) \times [\text{root}_{x_2}^{p_{4,1}}, \text{root}_{x_2}^{p_{4,1}}]$ and $(-\infty, \text{root}_{x_1}^{p_{8,1}}) \times [\text{root}_{x_2}^{p_{1,1}}, \text{root}_{x_2}^{p_{1,1}}]$ can be merged, respectively.

Note that during the preprocessing, some outermost bounds on variables may be omitted. Thus, after the preprocessing, the outermost bound on a variable changes.

Further, the technique of the second half of Section 3.2.6, which obtains smaller solution formulas by taking the negation of the encoding of the cells labelled with *false*, can be adapted also for a NuCAD data structure.

3.5.4 Improved Splitting

The loop in Lines 10-18 of Algorithm 3.21 splits the input cell $(I_{i+1}, \dots, I_{i'})$ into the generated cell $(I'_{i+1}, \dots, I'_{i'})$ and the neighbouring sections and sectors. If the generated cell is a sector, then we split the space below and above into a section and sector each, and explore them separately. However, in many cases, handling them separately causes unnecessary effort, as their truth value may be the same — see e.g. the first and second child of **T** in Figure 3.26. In particular, exploring the sections is computationally heavy, as the points sampled from a section are likely non-rational (as discussed e.g. in Section 3.2.11).

Algorithm 3.25: Modification of `nucad_full()`.

```

1 if  $I'_j.l \neq I_j.l$  then
2   let ? be ( if  $I_j.l$  is strict and [ otherwise
3   if  $I'_j.l$  is weak then
4      $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, ?I_j.l, I'_j.l), I_{j+1}, \dots, I_n)\}$ 
5   else
6      $Q := Q \cup \{(I'_1, \dots, I'_{j-1}, ?I_j.l, I'_j.l], I_{j+1}, \dots, I_n)\}$ 
7 if  $I'_j.u \neq I_j.u$  then
  // Analogous.
```

By using half-closed symbolic intervals, we can avoid such unnecessary splitting (or defer splitting in the corresponding recursive call to cases where the truth value indeed changes). For doing so, we adapt Algorithm 3.21 (and Algorithms 3.22 and 3.24 analogously) as follows:

- We allow closed bounds (i.e. $[l, u)$, $(l, u]$, $[l, u]$) for the symbolic intervals of the input cell and the generated cell.
- When adding the defining polynomials of the input cell to the polynomial set in Line 3 of Algorithm 3.21, we flag those that stem from closed bounds. For these, we do not require sign-invariance in the required cell, but only *semi-sign-invariance* (it holds $p \leq 0$ or $p \geq 0$ on the required cell), which is further defined in Section 4.6.
- During the computation of the cell, we use the techniques described in Section 4.6 which is able to derive half-closed cell bounds if possible, based on the mentioned flags and other properties.
- We replace the body of the **for**-loop in Lines 10-18 of Algorithm 3.21 by Algorithm 3.25.
- Some cells may turn out to be empty when choosing a sample point; these cells need to be omitted.

Towards the second item, Section 4.9 introduces a more rigorous way to achieve the desired cell.

3.6 Computing Implicants

The CAIC and NuCAD algorithms as presented above rely on Algorithm 3.12 to compute an implicant that generalizes the reason why a formula simplifies to either

true or *false* at a given sample point, as defined in Definition 3.21. In the following, we propose different variants for computing such an implicant. In all of these variants, the choice of the implicant is generally not unique. We always define the set of all implicants that can be computed in the described way. In an implementation, we could either collect all possible implicants and decide afterwards which one to take, or, to reduce the computational effort, compute a single good implicant.

We remark that the algorithms presented in this section are kept simple for illustrating the ideas. We assume that the formula is in prenex normal form and its matrix φ is in negation normal form.

3.6.1 Evaluation Only

We omit Boolean reasoning and simply evaluate the formula using the given sample point. Let $s \in \mathbb{R}^i$ such that $\varphi[s] = \text{false}$. We compute the set of implicants recursively:

$$\begin{aligned} \text{implicants}_s(c) &= \{\} \text{ if } c \text{ is a constraint and } c(s) \neq \text{false} \\ \text{implicants}_s(c) &= \{\{\neg c\}\} \text{ if } c \text{ is a constraint and } c(s) = \text{false} \\ \text{implicants}_s(\psi_1 \wedge \psi_2) &= \text{implicants}_s(\psi_1) \cup \text{implicants}_s(\psi_2) \\ \text{implicants}_s(\psi_1 \vee \psi_2) &= \text{implicants}_s(\psi_1) \times \text{implicants}_s(\psi_2) \end{aligned}$$

The case $\varphi[s] = \text{true}$ is analogous, as we basically switch the cases $\psi_1 \wedge \psi_2$ and $\psi_1 \vee \psi_2$. We interpret the resulting sets of constraints as conjunction.

3.6.2 Boolean Propagation

The above approach misses important Boolean information:

Example 3.29

Consider $\varphi_1 = (x_1 < 0 \vee x_2 = 0) \wedge (x_1 < 1 \vee x_2 \neq 0)$, which is unsatisfiable over $x_1 = 2$, but our partial evaluation does not detect that the resulting formula is logically equivalent to *false*: As the set of implicants for $x_2 = 0$ and $x_2 \neq 0$ is empty, the sets of implicants of all subformulas and the formula are empty as well.

We thus incorporate Boolean reasoning that resembles the Boolean propagation implemented within SAT solvers.

Consider Algorithms 3.26 and 3.28. Let $s \in \mathbb{R}^i$. If $\text{implicants}(\varphi, (\varphi), s)$ (respectively $\text{implicants}(\varphi, (\neg\varphi), s)$) is non-empty, then $\varphi[s] \equiv \text{false}$ ($\varphi[s] \equiv \text{true}$), and the algorithm already computes all possible implicants. The algorithm takes a formula φ , a sequence of decisions D (its entries are subformulas of φ or their negation) and a sample point s as input. The algorithm maintains for each subformula two sets $T(\psi)$ and $F(\psi)$ of *reasons* (which are sets of formulas maintained continuously through the algorithm) implying the subformula evaluates to *true* or *false*, respectively, given the decisions in D and the sample s . For every decision $d \in D$, its set of reasons $T(d)$ contains only the empty set, which represents an “unconditional” reason. Every subformula ψ and its negation $\neg\psi$ share the same sets (e.g. $T(\psi)$ is the same as $F(\neg\psi)$). Note that the `evaluate` subroutine corresponds to the evaluation only approach. The algorithm yields an implicant whenever it finds a conflict, that is, some subformula has both reasons to evaluate to *true* and to *false*, contradicting the assumption that

φ (respectively, $\neg\varphi$) holds. We note that this scheme can be extended to arbitrary Boolean operators such as exclusive-or and negation.

Example 3.30

Consider φ_1 and the sample $x_1 = 2$ from Example 3.29 again. Using the sample, we conclude $F(x_1 < 0) = \{\{x_1 < 0\}\}$ and $F(x_1 < 1) = \{\{x_1 < 1\}\}$. Additionally, we decide $T(\varphi_1) = \{\emptyset\}$ and propagate $T(x_1 < 0 \vee x_2 = 0) = \{\emptyset\}$ and $T(x_1 < 1 \vee x_2 \neq 0) = \{\emptyset\}$. By propagation via the disjunctions, we conclude that $T(x_2 = 0) = F(x_2 \neq 0) = \{\{x_1 < 0\}\}$ and $T(x_2 \neq 0) = F(x_2 = 0) = \{\{x_1 < 1\}\}$. Thus, $T(x_2 = 0) \times F(x_2 = 0) = \{\{x_1 < 0, x_1 < 1\}\}$ is the set of implicants.

Algorithm 3.26: `implicants( $\varphi, D, s$ )`

```

1 foreach subformula  $\psi$  of  $\varphi$  do  $T(\psi) := \emptyset$ 
2 foreach subformula  $\psi$  of  $\varphi$  do  $F(\psi) := \emptyset$ 
3 foreach  $d$  in  $D$  do  $T(d) := \{\emptyset\}$ 
4 foreach atom  $c$  in  $\varphi$  do
5   if  $c(s) = \text{true}$  then  $T(c) := \{\{c\}\}$ 
6   else if  $c(s) = \text{false}$  then  $F(c) := \{\{\neg c\}\}$ 
7 while  $T$  or  $F$  changed do
8   foreach subformula  $\psi$  of  $\varphi$  do
9     evaluate( $\psi$ )
10    propagate( $\psi$ )
11 return  $\bigcup_{\psi \in \varphi} T(\psi) \times F(\psi)$ 

```

Algorithm 3.27: `explore( $\varphi, D, s$ )`

```

1  $I_D := \text{implicants}(\varphi, D, s)$ 
2 if  $I_D \neq \emptyset$  then return  $I_D$ 
3 choose subformula  $\psi$  of  $\varphi$  s.t.  $T(\psi) = F(\psi) = \emptyset$ 
   at the end of the implicants call
4 if no such  $\psi$  exists then return  $\emptyset$ 
5  $I_{(D, \psi)} := \text{explore}(\varphi, (D, \psi), s)$ 
6  $I_{(D, \neg\psi)} := \text{explore}(\varphi, (D, \neg\psi), s)$ 
7 return  $I_{(D, \psi)} \times I_{(D, \neg\psi)}$ 

```

Algorithm 3.28: `evaluate and propagate`

// The syntax " $A \sqcup= B$ " is a shortcut for " $A = A \sqcup B$ ".

```

1 def evaluate( $\psi_1 \wedge \psi_2$ ):
2    $T(\psi_1 \wedge \psi_2) \sqcup= T(\psi_1) \times T(\psi_2)$ 
3    $F(\psi_1 \wedge \psi_2) \sqcup= F(\psi_1) \cup F(\psi_2)$ 
4 def evaluate( $\psi_1 \vee \psi_2$ ):
5    $T(\psi_1 \vee \psi_2) \sqcup= T(\psi_1) \cup T(\psi_2)$ 
6    $F(\psi_1 \vee \psi_2) \sqcup= F(\psi_1) \times F(\psi_2)$ 
7 def propagate( $\psi_1 \wedge \psi_2$ ):
8    $T(\psi_1) \sqcup= T(\psi_1 \wedge \psi_2)$ 
9    $T(\psi_2) \sqcup= T(\psi_1 \wedge \psi_2)$ 
10   $F(\psi_1) \sqcup= F(\psi_1 \wedge \psi_2) \times T(\psi_2)$ 
11   $F(\psi_2) \sqcup= F(\psi_1 \wedge \psi_2) \times T(\psi_1)$ 
12 def propagate( $\psi_1 \vee \psi_2$ ):
13   $T(\psi_1) \sqcup= T(\psi_1 \vee \psi_2) \times F(\psi_2)$ 
14   $T(\psi_2) \sqcup= T(\psi_1 \vee \psi_2) \times F(\psi_1)$ 
15   $F(\psi_1) \sqcup= F(\psi_1 \vee \psi_2)$ 
16   $F(\psi_2) \sqcup= F(\psi_1 \vee \psi_2)$ 

```

3.6.3 Boolean Exploration

The previous approach is able to detect some Boolean conflicts by propagation. However, there are formulas which are not satisfiable by the Boolean structure already, which can only be detected by exploration.

Example 3.31

Consider φ_1 from Example 3.29 and $\varphi_2 = ((x_3 = 0 \vee \varphi_1) \wedge (x_3 \neq 0 \vee \varphi_1))$ and the sample $x_1 = 2$. This formula is clearly unsatisfiable for the given sample $x_1 = 2$. However, although we would evaluate the literals ($F(x_1 < 0) = \{\{x_1 < 0\}\}$ and $F(x_1 < 1) = \{\{x_1 < 1\}\}$) and decide $T(\varphi_2) = \{\emptyset\}$, we would not be able to propagate further. We thus need to check the cases $x_3 = 0$ and $x_3 \neq 0$.

Consider Algorithm 3.27. Let $s \in \mathbb{R}^i$. If `explore`($\varphi, (\varphi), s$) (`explore`($\varphi, (\neg\varphi), s$)) is non-empty, then $\varphi[s] \equiv \text{false}$ ($\varphi[s] \equiv \text{true}$), and the algorithm again computes

all possible implicants. The algorithm naively assumes formulas to evaluate to *true* and *false*, calls recursively, and if both choices lead to a conflict, then it combines the obtained implicants. We note that also a partial exploration of the search space is possible.

Example 3.32

Assume we call **explore** on φ_2 from Example 3.31, $D = (\varphi_2)$ and $s = (2)$ (i.e. $x_1 = 2$). The implicants call would return the empty set, we thus choose a subformula, for example $\psi := x_3 = 0$ thus recursively call **explore** on φ_2 and $D = (\varphi, x_3 = 0)$ and on φ_2 and $D = (\varphi, x_3 \neq 0)$. The first call would set the reasons of $x_3 = 0$ to $\emptyset$, propagate these reasons and detect an immediate conflict from this decision, thus return the implicant $\{x_1 < 0, x_1 < 1\}$; similarly, the second call would return $\{x_1 < 0, x_1 < 1\}$. Thus the overall call would return $\{x_1 < 0, x_1 < 1\} \times \{x_1 < 0, x_1 < 1\} = \{x_1 < 0, x_1 < 1\}$ as implicant.

3.6.4 Inprocessing

The implicant could be further simplified using preprocessing techniques, such as Gröbner bases ([HED⁺16] predicts whether preprocessing using Gröbner bases benefits a CAD computation) or techniques described in [BV20]. We do not need bookkeeping of relations of input and output constraints, as we do not need to return infeasible subset as in the CDCL(T) framework.

3.7 Variants of the CALC Algorithm

In this section, we re-visit the CALC algorithm; we will see a couple of observations and modifications, which are also applicable to the NLSAT and NuCAD algorithms.

3.7.1 Variable Ordering

In the CALC algorithm as presented above, we explore all branches with the same (global) variable ordering, for the simplicity of the presentation. However, this is not necessary: we can explore different branches using different variable orderings, and can combine the results in the same way as before. To be more precise, we could modify e.g. Algorithm 3.16 by inserting after Line 3 a step where we choose which variable is assigned next (or, equivalently, rename the remaining variables in the formula). Of course, we would require some additional bookkeeping about which branch used which variable, in particular for generating solution formulas.

As stated in Section 3.2.9, the variable ordering has a significant impact on the number of cells. In the covering algorithm, after choosing some sample $s \in \mathbb{R}^i$, we may already simplify the formula φ using s (using the *evaluation up to level i* approach) to rule out polynomials to occur in an implicant in the current branch of the algorithm. This knowledge may be used to refine the variable ordering according to some heuristic.

3.7.2 Quantifier Structure

Algorithm 3.16 assumes the input to be in prenex normal form. In the following, we lift this restriction to better exploit the quantifier structure by switching the quantifier

order and solving independent subformulas separately. Potentially, we can rule out parts of the formula as being irrelevant in the current branch. To some degree, this is already facilitated by choosing implicants; however, implicants only consider the Boolean structure and completely ignore the quantifiers.

Example 3.33

Consider the formula $\varphi = \forall x \forall y. (\psi_1(x, y) \wedge \psi_2(x, y))$, which is logically equivalent to $\forall x \forall y. \psi_1(x, y) \wedge \forall x \forall y. \psi_2(x, y)$. We can check $\forall x \forall y. \psi_1(x, y)$ and $\forall x \forall y. \psi_2(x, y)$ separately and combine the results accordingly. For each subformula, we can even choose a different variable ordering.

We will first examine how the input formula may be rewritten to reflect its quantifier structure, and then extend the algorithm for handling formulas which are not in prenex normal form.

3.7.2.1 Input Transformation

We proceed as follows: Assume that we transformed the input formula to negation normal form, that is we pushed the negations into the formula using the double negation rule and De Morgan's rules such that only the atoms occur negatively in the formula. Afterwards, we push the quantifiers as far as possible into the formula using the following rules:

$$Qx. Qy. \varphi(x, y) \equiv Qy. Qx. \varphi(x, y) \text{ for } Q \in \{\exists, \forall\} \quad (\text{Swapping Quantifiers})$$

$$Qx. \varphi \equiv \varphi \text{ for } Q \in \{\exists, \forall\} \quad (\text{Null Quantification})$$

$$\forall x. (\varphi(x) \wedge \psi(x)) \equiv \forall x. \varphi(x) \wedge \forall x. \psi(x) \quad (\text{Distribution over Conjunction})$$

$$\exists x. (\varphi(x) \vee \psi(x)) \equiv \exists x. \varphi(x) \vee \exists x. \psi(x) \quad (\text{Distribution over Disjunction})$$

$$Qx. (\varphi(x) \circ \psi) \equiv (Qx. \varphi(x)) \circ \psi \text{ for } Q \in \{\exists, \forall\}, \circ \in \{\wedge, \vee\} \\ (\text{Prenex Law for \{Con, Dis\}junction})$$

where x does not occur in φ and ψ unless explicitly indicated in parentheses.

Note that we can apply the Equation (Swapping Quantifiers) for formulas in prenex normal form as well. In this case, it is rather clear how to make use of this rule — we just choose a typical variable ordering that works well for the CAD. In the general case, the order in which we apply Equation (Swapping Quantifiers) and the other rules might affect the result. Thus, the variable ordering has an influence on how we can distribute the quantifiers.

Example 3.34

Consider the formula $\varphi = \forall x \forall y. (\psi_1(x, y) \wedge \psi_2(y))$. When keeping the variable ordering, we can transform the formula to $\forall x. (\forall y. \psi_1(x, y) \wedge \forall y. \psi_2(y))$, although $\psi_2(y)$ does not depend on x . When switching the quantifiers, we can transform the formula to $\forall y \forall x. \psi_1(x, y) \wedge \forall y. \psi_2(y)$.

We could transform the formula iteratively after each assignment of a variable in the covering algorithm by plugging it in to all constraints and propagating the truth values:

Example 3.35

Consider the formula $\varphi = \forall x \forall y. [(x > 0 \wedge \psi_1(x, y) \wedge \psi_2(x, y)) \vee (x \leq 1 \wedge \psi_3(x, y))]$. Due to the disjunction, we cannot pull any quantifier into the formula.

Now assume we picked $x = 2$, thus the second case $\vartheta = (x \leq 1 \wedge \psi_3(x, y))$ of φ simplifies to *false*. An implicant for ϑ w.r.t. 2 is $x > 1$. Thus, for exploring the search space below $x = 2$, we can replace ϑ by $x > 1$ and obtain the simplified formula

$$\begin{aligned} & \forall x \forall y. [(x > 0 \wedge \psi_1(x, y) \wedge \psi_2(x, y)) \vee x > 1] \\ & \equiv \forall x. [(x > 0 \wedge \forall y. \psi_1(x, y) \wedge \forall y. \psi_2(x, y)) \vee x > 1] \end{aligned}$$

which allows us to split the problem into multiple ones. As the formula is in negation normal form, any implicant for the simplified formula (w.r.t. a sample that extends $x = 1$) is also an implicant for the original one.

3.7.2.2 Adaptation of the Algorithm**Algorithm 3.29: calc_recurse(φ, s)**

Input : Tarski formula $\varphi, s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$ for some $i \in [0..n]$.

Output : $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, and quantifier-free Tarski formula ϑ such that for each $s' \in C(P, s)$ it holds $\varphi(s') \equiv \vartheta(s')$.

```

1 if  $\varphi[s] \equiv \text{false} \vee \varphi[s] \equiv \text{true}$  then
2   return choose_enclosing_cell( $\varphi, s$ )                                // Algorithm 3.12
3 else                                                                    // it holds  $i < n$ 
4   choose a transformation of  $\varphi$  based on  $s$  according to Section 3.7.2.1 and store it to  $\varphi$ 
5   if  $\varphi = \exists x_{i+1}. \psi$  then
6     return calc_quantifier( $\exists, \psi, s$ )                                // Algorithm 3.17
7   else if  $\varphi = \forall x_{i+1}. \psi$  then
8     return calc_quantifier( $\forall, \psi, s$ )                                // Algorithm 3.17
9   else if  $\varphi = \bigvee_{j=1, \dots, k} \psi_j$  and we choose to split then
10     $\mathcal{C} := \emptyset$ 
11    choose an ordering of  $\psi_1, \dots, \psi_k$ 
12    foreach  $j = 1, \dots, k$  do
13       $(P, \vartheta_{(P,s)}) := \text{calc\_recurse}(\psi_j, s)$ 
14      if  $\vartheta_{(P,s)} = \text{true}$  then return  $(P, \text{true})$ 
15      else  $\mathcal{C} := \mathcal{C} \cup \{(P, s)\}$ 
16    return  $\bigcup_{(P,s) \in \mathcal{C}} P, \bigvee_{(P,s) \in \mathcal{C}} \vartheta_{(P,s)}$ 
17   else if  $\varphi = \bigwedge_{j=1, \dots, k} \psi_j$  and we choose to split then
18     $\mathcal{C} := \emptyset$ 
19    choose an ordering of  $\psi_1, \dots, \psi_k$ 
20    foreach  $j = 1, \dots, k$  do
21       $(P, \vartheta_{(P,s)}) := \text{calc\_recurse}(\psi_j, s)$ 
22      if  $\vartheta_{(P,s)} = \text{false}$  then return  $(P, \text{false})$ 
23      else  $\mathcal{C} := \mathcal{C} \cup \{(P, s)\}$ 
24    return  $\bigcup_{(P,s) \in \mathcal{C}} P, \bigwedge_{(P,s) \in \mathcal{C}} \vartheta_{(P,s)}$ 
25   else
26     // We assume that Algorithm 3.13 returns an encoding  $\vartheta$  of the covering.
27      $P, \vartheta := \text{calc\_full}(\varphi, s)$                                 // Algorithm 3.13
28     if all leaves in  $\vartheta$  are true then  $\vartheta := \text{true}$ 
29     else if all leaves in  $\vartheta$  are false then  $\vartheta := \text{false}$ 
30     return  $P, \vartheta$ 

```

Algorithm 3.29 is a replacement for Algorithm 3.16 and works almost the same. For reasons of simplicity, it generates an encoding of the computed covering directly, and assumes Algorithm 3.13 to do so as well. It accepts arbitrary Tarski formulas, and checks first whether the formula evaluates to a value under the current sample point in Line 1. Note that the implicant calculation in the call to `choose_enclosing_cell`(φ , s) is defined for arbitrary Tarski formulas in Definition 3.21.

If none of these cases hold, it transforms the input formula as described in Section 3.7.2.1 (Line 4). Then, we do a case distinction on the formula's structure: If the formula is a quantified formula (Lines 5 and 7), we call the algorithm handling the respective quantifier. If the formula is a conjunction or disjunction (Lines 9 and 17), we can choose to call Algorithm 3.29 recursively on the individual subformulas. Analogously to the exists (for all) case, for disjunctions (conjunctions), we return early once one of the recursive calls returns a *true* (*false*) cell; otherwise, we collect the remaining cells and build their intersection by merging the corresponding returned implicit cells. In case of a non-quantified formula and we do not split, we call Algorithm 3.13 to describe the parameter space.

We emphasize that the splitting mechanism overlaps with the implicant calculation. In an efficient implementation, this would be interleaved with the calculation of implicants, possibly considering the whole Boolean structure of φ at once to traverse the search tree non-chronologically. We could compute valid combinations of recursive calls and choose the best combination according to some metric.

3.7.3 Closed Cells in CAIC

Lifting over sections is particularly expensive. The CAIC algorithm already omits such lifting operations when two neighbouring intervals overlap. In Section 3.2.11, we discussed that existentially quantified formulas in negation normal form with only strict constraints do not have a solution on the varieties of their defining polynomials, and thus we do not need to lift over sections to check for satisfiability. We now transfer this idea to the CAIC algorithm: As the resulting covering consists of cells that stem from different sets of polynomials, we can apply this observation also if only some constraints are strict! For doing so, we mark cells which stem from strict constraints as *closed*, and inherit this flag throughout the algorithm accordingly.

We illustrate the idea in the following example.

Example 3.36

Consider $p_1 = -x_1^2 - x_2 + 1$, $p_2 = x_1^2 - x_2 - 1$, $p_3 = x_1 + 0.5$, and the formula $\varphi = \exists x_1 \exists x_2. p_1 < 0 \wedge p_2 > 0 \wedge p_3 \geq 0$. Figure 3.27a depicts the varieties of the formula's defining polynomials. Assuming the CAIC algorithm always chooses the smallest possible sample for x_1 , the depicted non-satisfying cells are generated before the solution $(1.5, 0)$ is found.

Figure 3.27b illustrates how we may reduce the amount of generated cells: Assume we picked $x_1 = 0.25$, then $p_1 < 0$ and $p_2 > 0$ are violated in the intervals $(-\infty, \text{root}_{x_2}^{p_1, 1}]$ and $[\text{root}_{x_2}^{p_2, 1}, \infty)$ respectively as the relation symbol is strict, and thus they cannot be satisfied on the varieties of their defining polynomials. Intuitively, this means that these intervals still overlap at $x_1 = 1$, and thus the cylinder over (1) does not need to be explored separately.

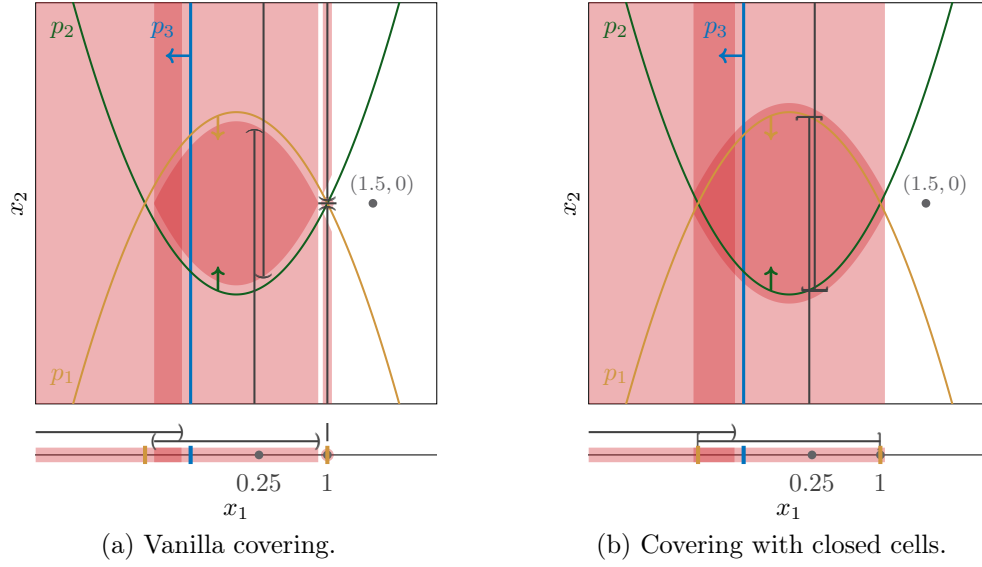


Figure 3.27: Two variants of the covering from Example 3.36. We depict the varieties of each defining polynomial, along with arrows indicating the violating areas for each polynomial. We use bracket-bounded thick black lines (open intervals) and short line segments (point intervals) to indicate the intervals determined during the CAIC algorithm.

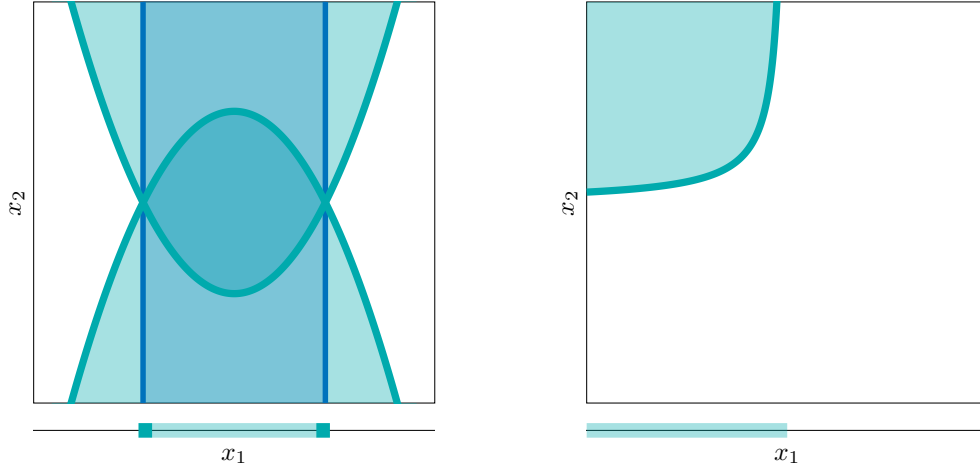
The actual justification is not based on intervals; in particular, we cannot draw similar conclusions if one of the constraints would be weak, although the resulting intervals would still cover the cylinder over (1) — we will see another technique in Section 4.6 that handles those cases. Instead, the reasoning here is that if $p_1 < 0$ or $p_2 > 0$ is not satisfied in some cell respectively, then the same holds in the closure of that cell. Further, if the cylinder over $(-1, 1)$ is covered by closed cells, then also its closure $[-1, 1]$ is covered by them; applied to the example, we obtain that the formula is not satisfied by any extension of a point in $[-1, 1]$.

The statement in the last sentence of the example is justified by the following theorem.

Theorem 3.2 ([BNÁ⁺23])

Assume $i, k \in \mathbb{N}_{>0}$, closed cells $R_1, \dots, R_k \subseteq \mathbb{R}^i$, and a cell $R \subseteq \mathbb{R}^{i-1}$ such that $R \times \mathbb{R} \subseteq \bigcup_{j=1}^k R_j$. Then $\text{cl}(R) \times \mathbb{R} \subseteq \bigcup_{j=1}^k R_j$.

Proof. Assume for contradiction that there exists an $r \in \text{bound}(R) \times \mathbb{R} \subseteq \mathbb{R}^i$ such that $r \notin \bigcup_{j=1}^k R_j$. Then $r \notin R_j$, i.e. $r \in \mathbb{R}^i \setminus R_j$ for all $j \in [k]$. As the sets R_j are closed, their complements $\mathbb{R}^i \setminus R_j$ are open. By definition of an open cell, for each $j \in [k]$ there exist $\varepsilon_j > 0$ such that $B_{\varepsilon_j}(r) \subseteq \mathbb{R}^i \setminus R_j$. Let ε be the smallest such ε_j of all $j \in [k]$. Then $B_\varepsilon(r) \subseteq \mathbb{R}^i \setminus R_j$ for all $j \in [k]$. As $r \in \text{bound}(R) \times \mathbb{R} = \text{bound}(R \times \mathbb{R})$, by the definition of $B_\varepsilon(r)$ there exist an $r' \in B_\varepsilon(r) \cap (R \times \mathbb{R})$, in other words, $r' \notin R_j$ for all $j \in [k]$, which is a contradiction to the assumption that $R \times \mathbb{R} \subseteq \bigcup_{j=1}^k R_j$. $\square$



(a) Closed cells covering a cylinder also cover its boundaries.

(b) A closed set with open projection.

Figure 3.28: Illustrations of cell projections.

Figure 3.28a illustrates the above theorem. Although it might seem straight-forward at the first sight, but there are some special cases that make it less trivial, e.g. that the projection of a closed cell might not be closed as shown in Figure 3.28b.

We thus modify the CAIC algorithm as follows:

- We store a *flag* along with each implicit cell, indicating that we mean its closure.
- We adapt Algorithm 3.12 to set the flag accordingly: If the formula φ is *false* at s , then we set the flag if *all* constraints in the implicant carry a *strict* relation symbol — as the formula is also false in the cell's closure. Dually, if the formula φ is *true* at s , then we set the flag if all constraints in the implicant carry a *weak* relation symbol.
- We adapt Line 2 of Algorithm 3.13 to check *additionally* whether the closure of cells carrying a flag cover the whole real line; if this is the case, we exit the while-loop.
- We relax the output criterion of Algorithm 3.14 for sets of flagged cells accordingly: We replace (0) by *for all $j \in [k]$ it holds $\text{cl}(\mathcal{I}^j(s_{[i]}^j)) \subseteq \text{cl}(\text{get_interval}(P^j, s^j))$* , and (1) by $\bigcup_{j \in [k]} \text{cl}(\mathcal{I}^j(s_{[i]}^j)) = \mathbb{R}$,
- For sets of flagged cells, Algorithm 3.15 returns a flagged cell (e.g. we apply Theorem 3.2).

We justify the correctness of the latter step as follows: Given $i \in [n]$ and sequence of implicit cells of level $i + 1$ with symbolic intervals $((P^1, s^1, \mathcal{I}^1), \dots, (P^k, s^k, \mathcal{I}^k))$ (where the s^j share the first $i - 1$ dimensions $s_{[i-1]}$) that fulfil the modified criterion of Algorithm 3.14, we need to compute a set $P' \subseteq \mathbb{Q}[x_1, \dots, x_{i-1}]$ and such that $\text{cl}(C(P', s_{[i-1]})) \times \mathbb{R} \subseteq \bigcup_{j \in [k]} \text{cl}(C(P^j, s^j))$. We claim that the covering projection in Algorithm 3.15 complies with this requirement: By assumption, $s_{[i-1]} \times \mathbb{R} \subseteq \bigcup_{j \in [k]} \text{cl}(C(P^j, s^j))$. The projection yields that $C(P', s_{[i-1]}) \times \mathbb{R} \subseteq \bigcup_{j \in [k]} \text{cl}(C(P^j, s^j))$; this is easy to see, as the projection guarantees that the bounding root functions either stay in the same order, or are equal over the whole underlying cell (see Lemma 4.1). The application of Theorem 3.2 yields the claim.

Chapter 4

Proof System for Cylindrical Algebraic Decomposition

The projection of exploration-guided algorithms based on the CAD as described in Chapter 3 is tailored towards a given (partial) sample point. We already gave basic projection operations, however, they do not exploit the full potential of local projection. There exist many special cases where further projection polynomials may be left out or replaced by simpler ones; often, there exists a *choice* of different ways to obtain a cell that fulfils the desired properties.

To express the different possibilities and separate them from the algorithms and the heuristics that decide for a choice, we introduce a *proof system* of *properties of a cell* that are led back to properties of some *underlying cell*. The proof system is independent of a specific algorithm and thus may be used by CAIC, NuCAD, or NLSAT. These algorithms can be seen as high level *proof strategy* assembling a proof by applying rules on sets of properties, while the proof system ensures correctness of the computed results.

Beyond this motivation, the proof system may also be a step towards proof-producing algorithms for real algebra: The trace of rule applications done throughout a run of some algorithm may serve as a high-level proof skeleton, which either may be filled with formally verified proofs of the rules themselves, or verified proofs of particular applications of some rule generated by another method (which may be more efficient than the CAD).

Origins The basic proof system from Sections 4.1 and 4.2 as well as the equational constraint optimization in Section 4.3 were already introduced in [NÁ⁺24]. The extension of the proof system to coverings in Section 4.2.9 was introduced in [NK25]. The rule for completeness in Section 4.4 is a novel contribution of this thesis. Local delineability from Section 4.5 was introduced in [NÁ24]; examples and basic definitions were taken over, but main results were reformulated using the proof system. The reader may benefit from also reading the prior publication. The weak bounds adaptation in Section 4.6 was briefly mentioned in [NÁ24], but is elaborated here. The compound cell bounds in Section 4.7 are a novel contribution of this thesis. Results based on projective delineability in Section 4.8 are already published in [NMÁ⁺25]; some examples and illustrations were taken over. However, the statements in this thesis are different to fit into the framework of the proof system. The reader may benefit from also reading the

prior publication. Section 4.9 about indexed root expressions are a novel contribution of this thesis.

4.1 Introduction

Before we go into the formal presentation, we outline the idea using a single cell construction example.

Example 4.1

Recall Example 3.18 and Figure 3.20 where we sought to construct a cell $R \subseteq \mathbb{R}^2$ around $s = (s_1, s_2)$ such that the sign-invariance of p_1, p_2, p_3 hold on R . We started by eliminating x_2 : we isolated real roots of $p_j(s_1, x_2)$ to define $\xi_1, \dots, \xi_4$, sorted them, and observed that $\xi_1(s_1) < s_2 < \xi_2(s_1)$, thus we determined that in the second level the cell was represented by $I_2 = (\xi_1, \xi_2)$.

This root ordering and the representation should now be generalized such that the underlying cell R_1 is restricted by specifying certain properties ensuring ξ_1 and ξ_2 remain as cell boundaries: they must remain well-defined over R_1 ; no further root of p_1, p_2, p_3 should appear that intersects I_2 over R_1 ; and the other roots, ξ_3 and ξ_4 , should remain outside the symbolic interval I_2 over R_1 . For the latter, we extend to a partial ordering $\prec$ with $\xi_2 \prec \xi_3$ and $\xi_2 \prec \xi_4$.

To conclude that p_1, p_2, p_3 are sign-invariant in R , the proof system shows the need to maintain the properties that: the sample s is included in R ; I_2 describes the cell's boundaries on the current level; the underlying cell R_1 is a connected analytic submanifold; and p_1, p_2, p_3 are delineable. Furthermore, we maintain that the partial ordering $\prec$ is maintained over R_1 .

To maintain these properties the proof system concludes that we must also prove order-invariance and sign-invariance of some coefficients, resultants, and discriminants of the input polynomials. After simplification, we find that on level 1 the polynomials $p_4 = \text{disc}_{x_2}(p_2)$ and $p_5 = \text{res}_{x_2}(p_3, p_2)$ must be ensured sign-invariant. This leads to a representation $I_1 = (\xi'_2, \xi'_3)$ and then since all polynomials are univariate and their zeros are algebraic numbers, no further projection or proof steps are necessary. The constructed cell is described by $R = I_1 \times I_2$.

The graph of the proof constructed is shown in Figure 4.1, in which the sign-invariance properties $\text{sgn_inv}(p_1)$, $\text{sgn_inv}(p_2)$, $\text{sgn_inv}(p_3)$ lead to $R = R_{\downarrow[1]} \times I_2$ and $R_{\downarrow[1]} = I_1$. The exact proof rules used in that figure will be defined in the rest of this section.

Note that in the example there were multiple choices for $\prec$. Choosing $\xi_2 \prec \xi_3$ and $\xi_3 \prec \xi_4$ would have been a valid choice, but that leads to a smaller cell (the one depicted in the top right of Figure 3.21). We will discuss ordering heuristics to make this choice in Section 5.1.

Our system is for constructing cells. We will define *properties* of the cell that is being constructed and *rules of inference* to infer that a property holds for a cell given some other properties and conditions. As indicated in the previous example, there may be different rules that can be used to prove a property.

$$\begin{aligned}
& \text{sgn_inv}(p_1)(R) \dashv \text{sample}(s)(R), \text{repr}(\mathbf{I}_2, s_1)(R), \text{ir_ord}(\prec, s_1)(R), \text{an_del}(p_1)(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \text{sgn_inv}(p_2)(R) \dashv \text{sample}(s)(R), \text{repr}(\mathbf{I}_2, s_1)(R), \text{ir_ord}(\prec, s_1)(R), \text{an_del}(p_2)(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \text{sgn_inv}(p_3)(R) \dashv \text{sample}(s)(R), \text{repr}(\mathbf{I}_2, s_1)(R), \text{ir_ord}(\prec, s_1)(R), \text{an_del}(p_3)(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \quad \text{sample}(s)(R) \dashv \text{repr}(\mathbf{I}_2, s_1)(R), \text{sample}(s_1)(R) \\
& \text{repr}(\mathbf{I}_2, s_1)(R) \dashv R = R \downarrow_{[1]} \times \mathbf{I}_2, \text{an_del}(p_2)(R), \text{an_del}(p_3)(R), \text{sample}(s_1)(R) \\
& \text{ir_ord}(\prec, s_1)(R) \dashv \text{an_del}(p_1)(R), \text{an_del}(p_2)(R), \text{an_del}(p_3)(R), \text{ord_inv}(\text{res}_{x_2}(p_3, p_1))(R), \\
& \quad \text{ord_inv}(\text{res}_{x_2}(p_3, p_2))(R), \text{an_sub}(1)(R), \text{connected}(1)(R), \text{sample}(s_1)(R) \\
& \text{an_del}(p_1)(R) \dashv \text{non_null}(p_1)(R), \text{ord_inv}(\text{disc}_{x_2}(p_1))(R), \text{sgn_inv}(\text{lcf}_{x_2}(p_1))(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \text{an_del}(p_2)(R) \dashv \text{non_null}(p_2)(R), \text{ord_inv}(\text{disc}_{x_2}(p_2))(R), \text{sgn_inv}(\text{lcf}_{x_2}(p_2))(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \text{an_del}(p_3)(R) \dashv \text{non_null}(p_3)(R), \text{ord_inv}(\text{disc}_{x_2}(p_3))(R), \text{sgn_inv}(\text{lcf}_{x_2}(p_3))(R), \\
& \quad \text{an_sub}(1)(R), \text{connected}(1)(R) \\
& \text{non_null}(p_1)(R) \dashv \text{true} \\
& \text{non_null}(p_2)(R) \dashv \text{true} \\
& \text{non_null}(p_3)(R) \dashv \text{true} \\
& \text{ord_inv}(\text{disc}_{x_2}(p_1))(R) \dashv \text{true} \\
& \text{ord_inv}(\text{disc}_{x_2}(p_2))(R) \dashv \text{sgn_inv}(p_4)(R), \text{sample}(s_1)(R) \\
& \text{ord_inv}(\text{disc}_{x_2}(p_3))(R) \dashv \text{true} \\
& \quad \text{sgn_inv}(p_4)(R) \dashv \text{repr}(\mathbf{I}_1)(R) \\
& \text{ord_inv}(\text{res}_{x_2}(p_3, p_1))(R) \dashv \text{true} \\
& \text{ord_inv}(\text{res}_{x_2}(p_3, p_2))(R) \dashv \text{sgn_inv}(p_5)(R), \text{sample}(s_1)(R) \\
& \quad \text{sgn_inv}(p_5)(R) \dashv \text{repr}(\mathbf{I}_1)(R) \\
& \quad \text{sgn_inv}(\text{lcf}_{x_2}(p_1))(R) \dashv \text{true} \\
& \quad \text{sgn_inv}(\text{lcf}_{x_2}(p_2))(R) \dashv \text{true} \\
& \quad \text{sgn_inv}(\text{lcf}_{x_2}(p_3))(R) \dashv \text{true} \\
& \quad \text{an_sub}(1)(R) \dashv \text{repr}(\mathbf{I}_1)(R) \\
& \quad \text{connected}(1)(R) \dashv \text{true} \\
& \quad \text{sample}(s_1)(R) \dashv \text{repr}(\mathbf{I}_1)(R) \\
& \quad \text{repr}(\mathbf{I}_1)(R) \dashv R \downarrow_{[1]} = \mathbf{I}_1
\end{aligned}$$

Figure 4.1: Proof for Example 4.1 (slightly truncated, read in columns). $Q \dashv P_1, \dots, P_k$ means that Q is derived from $P_1, \dots, P_k$.

Definition 4.1 (Property)

Let $i \in \mathbb{N}$. A *property of level i* is a function $q : \{R \mid R \subseteq \mathbb{R}^i\} \rightarrow \mathbb{B}$. A property q of level i *holds* on a set $R \subseteq \mathbb{R}^i$ if $q(R) = 1$. The set Prop denotes the set of all properties of any level. For a set $Q \subseteq \text{Prop}$, we denote by $Q|_i$ the set of properties of level i and by $Q|_{[0..i]}$ the set of properties of level at most i .

Note that the concept of levels of polynomials has been extended to properties in the sense that the satisfaction of a property of level i cannot be influenced by values of the variables $x_{i+1}, \dots, x_n$.

We denote a *rule of inference* in the form $A_1, \dots, A_k \vdash Q$, where $A_1, \dots, A_k$ are the antecedents and Q is the consequent. The set of proof rules that we define in the following sections ensures that the antecedents are smaller than the consequent with respect to an ordering $\triangleleft$ on Prop .

For convenience, we allow lifting properties from lower levels to higher levels as follows.

Definition 4.2

We extend every property q of level i to $q : \{R \mid R \subseteq \mathbb{R}^j, j \geq i\} \rightarrow \mathbb{B}$ such that $q(R) = q(R \downarrow_{[i]})$ for any $R \subseteq \mathbb{R}^j, j \geq i$. Accordingly, we introduce the following rule

of inference:

$$R \subseteq \mathbb{R}^{i+1}, q \in \text{Prop}|_i, q(R \downarrow_{[i]}) \quad \vdash q(R)$$

So, for example, this rule allows us to say that because the property of sign-invariance holds for $x^2 - 2$ in the one-level cell $-1 < x < 1$, sign-invariance holds for $x^2 - 2$ in the interior of the unit circle, which is a two-level cell that projects down to $-1 < x < 1$.

4.1.1 Application of Proof Rules

We will modify the algorithms in Chapter 3 by replacing the sets of polynomials P by sets of properties Q . One of these properties is $\text{sgn_inv}(p)$ where $p \in \mathbb{Q}[x_1, \dots, x_n]$, stating that p is sign-invariant on the constructed cell; we thus can easily replace P by these properties.

We will later introduce algorithms which replace Algorithms 3.9 and 3.15 which implement the projection operation based on the proof system. These algorithms use Algorithm 4.1 for applying the proof rules. However, some rules require global decisions for its instantiation; e.g. some sample point s around which the cell is to be constructed, some symbolic interval I which describes the cell's bounds on the current level, or some ordering $\prec$ on some roots. We will thus generalize Algorithms 3.8 and 3.14 to compute a *representation* (see Chapter 5) which contains all the information necessary for the instantiation of the rules.

Algorithm 4.1 takes a set of properties, takes the largest property w.r.t. the ordering $\triangleleft$, checks which rules are applicable for that rule using the given representation, and instantiates the rule. It may happen that some property is left over as there is no suitable proof rule — this might be due to incompleteness of the proof system, or because there is a lower-level property which needs a lower-level representation instantiating corresponding proof rules; the algorithm simply skips these properties and returns them to the outer algorithm.

Algorithm 4.1: $\text{apply}(Q, \text{representation})$

Input : Finite $Q \subseteq \text{Prop}|_{[i]}$ for some $i \in [n]$, some tuple *representation* for instantiating the proof rules.

Output : $Q' \subseteq \text{Prop}|_{[i]}$ such that for all $R \subseteq \mathbb{R}^i$ it holds $(\forall q' \in Q'. q'(R)) \Rightarrow (\forall q \in Q. q(R))$, and for each $q \in Q \setminus Q'$ and $q' \in Q' \setminus Q$ it holds $q' \triangleleft q$.

```

1  $S = \emptyset$  // set of skipped properties
2 foreach  $q \in Q \setminus S$  where  $q$  is the (unique) greatest element with respect to  $\triangleleft$  do
3   let  $\text{choices} \subseteq 2^{\text{Prop}}$  such that each  $Q' \in \text{choices}$  is a set of properties from a proof rule which
   can be applied according to representation to derive  $q$  (see explanatory text above for
   details)
4   if  $\text{choices} = \emptyset$  then
5      $S = S \cup \{q\}$ 
6   else if  $\exists Q' \in \text{choices}. Q' \subseteq Q$  then
7      $Q := Q \setminus \{q\}$ 
8   else
9     choose  $Q' \in \text{choices}$ 
10     $Q := (Q \setminus \{q\}) \cup Q'$ 

```

4.2 McCallum's Projection

We start with the basic McCallum's projection to cover the techniques in the projection defined in Algorithms 3.9 and 3.15, before moving on to more elaborated proof rules.

4.2.1 Basic Cell Properties

We first define the basic properties on polynomials and cells that follow from the original work on McCallum CAD projection. Throughout, i will refer to the level of the given property and s refers to *algebraic* points. It is important to recall that an i -level property is a function that maps subsets of $\mathbb{R}^i$ to Boolean values. Consider for example the first element of Property 4.1: “ $sample(s)$ ” for $s \in \mathbb{R}^i$ is a property, meaning that $sample$ is a function that maps a point (and a level i that is given implicitly by the point) to a function mapping subsets of $\mathbb{R}^i$ to Booleans. Thus, $sample(s)$ is not a true/false value. Rather, it is the function $sample(s)$ applied to a given subset of $\mathbb{R}^i$ that yields a true/false value. Note that for the fifth and sixth items in Property 4.1 the level i must be given explicitly, while with the others the level is implicit in other arguments.

Property 4.1

Let $i \in \mathbb{N}$, and $R \subseteq \mathbb{R}^i$.

1. Let $s \in \mathbb{R}^i$ be a sample point. The property $sample(s)$ holds on R if and only if $s \in R$.
2. Let $p \in \mathbb{Q}[x_1, \dots, x_i]$ be a polynomial of level i . The property $ord_inv(p)$ holds on R if and only if p is order-invariant on R .
3. Let $p \in \mathbb{Q}[x_1, \dots, x_i]$ be a polynomial of level i . The property $sgn_inv(p)$ holds on R if and only if p is sign-invariant on R .
4. Let $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ be a polynomial of level $i+1$. The property $non_null(p)$ holds on R if and only if p is not nullified on any point in R .
5. The property $an_sub(i)$ holds on R if and only if R is an analytic submanifold.
6. The property $connected(i)$ holds on R if and only if R is connected.
7. Let $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ be a polynomial of level $i+1$. The property $an_del(p)$ holds on R if and only if R is a connected analytic submanifold, p is analytically delineable on R , and p is order-invariant in all p -sections over R .

Note that for some properties such as $sgn_inv(p)$, if they hold on $R \subseteq \mathbb{R}^i$, then they also hold on all subsets of R . For others such as $an_sub(i)$ or $connected(i)$, this is not the case: a subset of R is not necessarily an analytic submanifold nor connected. This is one of the reasons why the proof rules below cannot cover all subsets where a given property holds, but they do cover sufficiently many subsets for our purposes.

For the remainder of this chapter we will present proof rules for these properties and prove their correctness. For the ease of navigation we give references to the rules which apply to prove each property (and do similarly after introducing further properties in later subsections).

References

<i>sample(s)</i> Rule 4.12	<i>non_null(p)</i> Rule 4.2
<i>ord_inv(p)</i> Rule 4.3, Rule 4.4, Rule 4.6, Rule 4.18, Rule 4.20, Rule 4.39	<i>an_sub(i)</i> Rule 4.8, Rule 4.35
<i>sgn_inv(p)</i> Rule 4.3, Rule 4.4, Rule 4.7, Rule 4.10, Rule 4.17, Rule 4.19, Rule 4.26, Rule 4.43	<i>connected(i)</i> Rule 4.11, Rule 4.15, Rule 4.32, Rule 4.46
	<i>an_del(p)</i> Rule 4.1, Rule 4.29, Rule 4.40, Rule 4.41

In the following rule definitions we list assumptions before the actual inference rule. These assumptions are formally part of the inference rule, but we keep them separated to improve readability.

We start by defining rules that relate to McCallum's projection.

We already explained the role of the discriminant and the leading coefficient for delineability, which is formalized in the following rule. Later, we give an alternative rule in Section 4.8.

Rule 4.1

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$. Assume that p is irreducible.

$$\begin{aligned} & \text{an_sub}(i)(R), \text{connected}(i)(R), \text{non_null}(p)(R), \\ & \text{ord_inv}(\text{disc}_{x_{i+1}}(p))(R), \text{sgn_inv}(\text{lcf}_{x_{i+1}}(p))(R) \quad \vdash \text{an_del}(p)(R) \end{aligned}$$

Note that throughout this chapter, when proving the correctness of a mapping, we implicitly use the assumptions from the mapping's definition in the proofs.

Proof. Follows immediately from [Bro01, Theorem 3.1] and [McC98, Theorem 2]. $\square$

Rule 4.2

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$. Assume that p is irreducible, and $p = c_m \cdot x_{i+1}^m + \dots + c_1 \cdot x_{i+1} + c_0$ such that $c_m, \dots, c_0 \in \mathbb{Q}[x_1, \dots, x_i]$.

$$\begin{aligned} & \text{sample}(s)(R), \text{deg}_{x_{i+1}}(p) > 1, \text{disc}_{x_{i+1}}(p)(s) \neq 0, \\ & \text{sgn_inv}(\text{disc}_{x_{i+1}}(p))(R) \quad \vdash \text{non_null}(p)(R) \\ & \text{sample}(s)(R), \exists j \in [0..m]. (c_j(s) \neq 0 \wedge \text{sgn_inv}(c_j)(R)) \quad \vdash \text{non_null}(p)(R) \end{aligned}$$

Note that the condition $\text{deg}_{x_{i+1}}(p) > 1 \wedge \text{disc}_{x_{i+1}}(p)(s) \neq 0$ already implies that $p(s, x_{i+1}) \neq 0$, as does $c_j(s) \neq 0$. In practice, it is good to delay choosing which rule to use because we may observe that one of the non-zero c_j 's is already required to be sign-invariant from some other part of the projection process (e.g. leading coefficients are often added to the projection). Moreover, the second case is trivial if the c_j is constant.

Proof. The first rule follows from the definition of the discriminant by the Sylvester matrix.

For the second rule, observe that p is nullified on any point $r \in R$ if and only if $c_j(r) = 0$ for all $j \in [m]$. Then, the statement follows immediately. See also [BK15, Lemma 5]. $\square$

We present proof rules that simplify sign-invariance and order-invariance.

Rule 4.3

Let $p \in \mathbb{Q}$.

$$\vdash \text{ord_inv}(p)(\mathbb{R}^0)$$

$$\vdash \text{sgn_inv}(p)(\mathbb{R}^0)$$

Proof. Trivial. $\square$

Rule 4.4

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is reducible, and $\text{factors}(p) = \{q_1, \dots, q_k\}$.

$$\text{ord_inv}(q_1)(R), \dots, \text{ord_inv}(q_k)(R) \quad \vdash \text{ord_inv}(p)(R)$$

$$\text{sgn_inv}(q_1)(R), \dots, \text{sgn_inv}(q_k)(R) \quad \vdash \text{sgn_inv}(p)(R)$$

Proof. Note that if p is reducible, then $p \notin \text{factors}(p)$. Both statements follow by [McC85, Lemma 3.2.2]. $\square$

Rule 4.5

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is reducible, $\text{factors}(p) = \{q_1, \dots, q_k\}$, and $j \in [k]$.

$$\text{sample}(s)(R), q_j(s) = 0, \text{sgn_inv}(q_j)(R) \quad \vdash \text{sgn_inv}(p)(R)$$

Note it makes sense to choose j such that $\text{level}(q_j) < i$ if possible (in this case p is nullified). $\square$

Proof. If a factor of p is invariantly 0 on R , then p is itself invariantly 0 on R . $\square$

Order-invariance is a central notion in McCallum's projection theory; a CAD is often referred to as *order-invariant* instead of *sign-invariant*. However, the following clarifies that order-invariance is just sign-invariance plus order-invariance of the discriminant (which is a lower-level property).

Rule 4.6

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is irreducible.

$$\begin{array}{l} p(s) \neq 0, \text{ sample}(s)(R), \text{ sgn_inv}(p)(R) \vdash \text{ord_inv}(p)(R) \\ p(s) = 0, \text{ sample}(s)(R), \text{ an_sub}(i-1)(R), \text{ connected}(i)(R), \\ \text{sgn_inv}(p)(R), \text{ an_del}(p)(R) \vdash \text{ord_inv}(p)(R) \end{array}$$

Proof. In the first case, from $p(s) \neq 0$ and the sign-invariance of p on R follows $p(r) \neq 0$ for all $r \in R$. Thus, p is order-invariant on R as an immediate consequence of the definition of order-invariance.

In the second case, from $p(s) = 0$ and the sign-invariance of p on R follows $p(r) = 0$ for all $r \in R$. As p is analytically delineable on $R \downarrow_{[i-1]}$, p is order-invariant in each p -section on $R \downarrow_{[i-1]}$, R is a p -section on $R \downarrow_{[i-1]}$, thus the order-invariance of p on R follows immediately. $\square$

4.2.2 Sign Invariance for Polynomials Without Roots

For polynomials without roots over the current sample, we must ensure that no further root appears over the underlying cell. In this case, it is sufficient to make the polynomial delineable. Figure 4.2 illustrates some cases.

Rule 4.7

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is irreducible, and $\text{realRoots}(p(s, x_i)) = \emptyset$.

$$\text{sample}(s)(R), \text{ an_del}(p)(R) \vdash \text{sgn_inv}(p)(R)$$

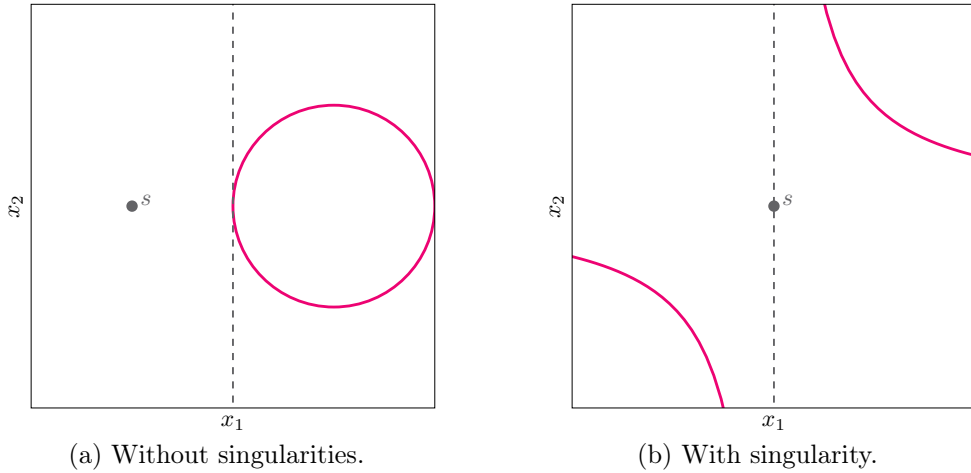


Figure 4.2: Examples of the two cases of polynomials without roots at the projection s_1 of the sample $s = (s_1, s_2)$. Dashed lines denote cylinder boundaries over the projected cell.

Proof. By assumption p is (analytically) delineable on $R_{\downarrow[i-1]}$, thus by Definition 3.3 the number of roots of p is constant over $R_{\downarrow[i-1]}$. As $\text{realRoots}(p(s, x_{i+1})) = \emptyset$ and $s \in R_{\downarrow[i-1]}$, p does not have any roots over $R_{\downarrow[i-1]}$, thus p is sign-invariant on $R_{\downarrow[i-1]} \times \mathbb{R} \supset R$. $\square$

4.2.3 Cell Boundary Representations

We are aiming to generate the description of a sign-invariant cell for the input polynomials. In the language of our proof system: at every level we identify a symbolic interval upon which we prove sign-invariance of the polynomials of that level. To do this, we take their real roots over the current sample into account.

We determine whether we are in a section or sector (i.e. whether a polynomial has a root exactly at the current sample point or not) and pick indexed root expressions to be symbolic descriptions of the cell boundaries that generalize a concrete interval around the sample point: either the root at the current sample or the largest root below and the smallest root above. Note that the choice of such boundaries might not be unique in the cases where two root functions cross over the underlying sample. We will discuss more general choices of the symbolic intervals in Section 5.1.

Our next property is designed to state that a designated representation describes the cell boundaries on the current level.

To do so, we define how we use indexed root expressions to refer to real root functions of a polynomial. An indexed root expression has discontinuities where the number or multiplicity of the roots of its defining polynomial changes (see Figure 3.7). Therefore, a root function of a polynomial is only uniquely determined by an indexed root expression in combination with a given sample. In the following, we define the unique real root function determined this way:

Definition 4.3 (Real Root Function)

Let $i \in \mathbb{N}$, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$, $R \subseteq \mathbb{R}^i$, and $\theta : R \rightarrow \mathbb{R}$ continuous. We call θ a *real root function* of p on R if

- $p(r, \theta(r)) = 0$ for all $r \in R$,
- for any continuous $\theta' : R \rightarrow \mathbb{R}$ such that $p(r, \theta'(r)) = 0$ for all $r \in R$, if there exists some $r \in R$ such that $\theta(r) = \theta'(r)$, then $\theta' = \theta$.

Let $r \in \mathbb{R}^i$ and $\xi \in \text{ire}(p, r)$. Then $\theta_{\xi, r} : R' \rightarrow \mathbb{R}$ is the real root function of p on some R' such that $\xi(r) = \theta_{\xi, r}(r)$ and R' is maximal.

Property 4.2

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i .

The property $\text{repr}(I, s)$ holds on R if and only if $I.l \in \text{ire}(I.l.p, s)$ (if $I.l \neq -\infty$), $I.u \in \text{ire}(I.u.p, s)$ (if $I.u \neq \infty$), and one of the following holds:

- $I = (l, u)$, $\text{dom}(\theta_{l,s}) \cap \text{dom}(\theta_{u,s}) \supseteq R_{\downarrow[i-1]}$ and $R = R_{\downarrow[i-1]} \times (\theta_{l,s}, \theta_{u,s})$; or
- $I = (-\infty, u)$, $\text{dom}(\theta_{u,s}) \supseteq R_{\downarrow[i-1]}$ and $R = R_{\downarrow[i-1]} \times (-\infty, \theta_{u,s})$; or

- $I = (l, \infty)$, $\text{dom}(\theta_{l,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \downarrow_{[i-1]} \times (\theta_{l,s}, \infty)$; or
 - $I = (-\infty, \infty)$ and $R = R \downarrow_{[i-1]} \times (-\infty, \infty)$; or
 - $I = [l, u]$, $\text{dom}(\theta_{l,s}) \cap \text{dom}(\theta_{u,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \downarrow_{[i-1]} \times [\theta_{l,s}, \theta_{u,s}]$; or
 - another analogous case for $I = (-\infty, u]$, $I = [l, \infty)$, $I = (l, u]$, $I = [l, u)$.
- (for the real root functions $\theta_{l,s}$ and $\theta_{u,s}$ according to Definition 4.3).

References

repr(I, s) Rule 4.13

Since the cell's boundaries are described by real root functions, we can prove that it is an analytic submanifold.

Rule 4.8

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i .

$$\begin{array}{ll} \text{repr}(I, s)(R), \text{ an_sub}(i-1)(R) & \vdash \text{an_sub}(i)(R) \end{array}$$

Proof. As the endpoints of I are described by analytic functions, the statement follows immediately from [McC85, Theorem 2.2.3 and Theorem 2.2.4]. $\square$

In this section, we will only use symbolic intervals that describe sections and sectors, that is if $I = [l, u]$ then $l = u$.

4.2.4 Root Orderings

We define a partial ordering on the indexed root expressions of the given set of polynomials, which we will (mainly) use to ensure that none of their roots crosses the cell's boundary on the current level.

We first give a general definition of indexed root orderings.

Definition 4.4 (Indexed Root Ordering)

Let $i \in \mathbb{N}$, and Ξ be a set of indexed root expressions of level $i+1$.

We define the following relations w.r.t. a tuple of relations $(\approx, \prec) \subseteq \Xi \times \Xi$:

$$\begin{aligned} =_{\approx, \prec} &= \{(\xi, \xi') \mid \xi \approx^{rts} \xi'\} \\ <_{\approx, \prec} &= \{(\xi, \xi') \mid \text{there ex. } \sim_1, \dots, \sim_{k-1} \in \{\prec, \approx\}, \xi_1, \dots, \xi_k \text{ s.t.} \\ &\quad \xi = \xi_1 \sim_1 \dots \sim_{k-1} \xi_k = \xi' \text{ and } \prec \in \{\sim_1, \dots, \sim_{k-1}\}\} \end{aligned}$$

An *indexed root ordering* on Ξ is a tuple of relations $(\approx, \prec)$ such that $\approx, \prec \subseteq \Xi \times \Xi$, $=_{\approx, \prec}$ is an equivalence relation on Ξ , and $<_{\approx, \prec}$ is a strict partial order on Ξ . Indexed root orderings of this form are also called *indexed root ordering of level $i+1$* , and we define $\text{dom}(\approx, \prec) = \text{dom}(\approx) \cup \text{dom}(\prec)$.

Let $s \in \mathbb{R}^i$. An indexed root ordering $(\approx, \prec)$ of level $i + 1$ *matches* s if and only if $\xi \in \text{ire}(\xi.p, s)$ for all $\xi \in \text{dom}(\approx, \prec)$, $\xi(s) = \xi'(s)$ for all $(\xi, \xi') \in \approx$, and $\xi(s) < \xi'(s)$ for all $(\xi, \xi') \in \prec$.

In our algorithm we will pick an indexed root ordering such that, in the sector case, roots lower than the interval's lower bound remain lower, and roots greater than the interval's upper bound remain greater. In the section case, the lower and upper bounds in that condition both refer to a single bound. We do not give an explicit definition here yet, as it will be part of the rules defined below.

The motivation for introducing the general concept of indexed root orderings is to allow for choices between different root orderings. We will discuss them in Chapter 5.

Example 4.2

Given indexed root expressions $\Xi = \{\xi_1, \dots, \xi_5\}$ of level $i + 1$ and a sample $s \in \mathbb{R}^i$ such that $\xi_1(s) < \dots < \xi_5(s)$ and $I = (\xi_1, \xi_2)$; then indexed root orderings $(\approx, \prec)$ on Ξ with $\prec = \{(\xi_2, \xi_3), (\xi_2, \xi_4), (\xi_2, \xi_5)\}$, $\prec = \{(\xi_2, \xi_3), (\xi_3, \xi_4), (\xi_2, \xi_5)\}$, or $\prec = \{(\xi_2, \xi_3), (\xi_3, \xi_4), (\xi_4, \xi_5)\}$ allow us to derive sign-invariance of the corresponding polynomials with roots above the sample s , if the $(i+1)$ st dimension is described by I .

Given indexed root expressions $\Xi = \{\xi_1, \dots, \xi_5\}$ of level $i + 1$ and a sample $s \in \mathbb{R}^i$ such that $\xi_1(s) < \xi_2(s) < \xi_3(s) = \xi_4(s) < \xi_5(s)$ and $I = (\xi_1, \xi_2)$; then indexed root ordering $(\approx, \prec)$ on Ξ with $\approx = \{(\xi_3, \xi_4)\}$ and $\prec = \{(\xi_2, \xi_3), (\xi_4, \xi_5)\}$, allow us to derive sign-invariance of the corresponding polynomials with roots above the sample s , if the $(i+1)$ st dimension is described by I .

We need to maintain that an indexed root ordering holds over the underlying cell. First, we will define this property formally.

Property 4.3

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $(\approx, \prec)$ be an indexed root ordering of level $i + 1$. The property $\text{ir_ord}((\approx, \prec), s)$ holds on R if and only if $(\approx, \prec)$ matches s , for all $\xi \in \text{dom}(\approx, \prec)$ it holds $R \subseteq \text{dom}(\theta_{\xi, s})$, and for all $\xi, \xi' \in \text{dom}(\approx, \prec)$ it holds that

$$\xi <_{\approx, \prec} \xi' \Rightarrow \theta_{\xi, s}(r) < \theta_{\xi', s}(r) \text{ for all } r \in R$$

and

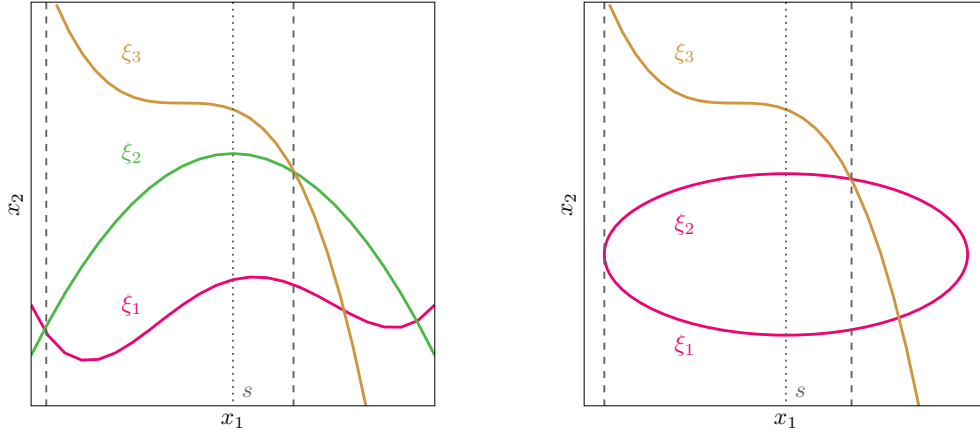
$$\xi \approx_{\approx, \prec} \xi' \Rightarrow \theta_{\xi, s}(r) = \theta_{\xi', s}(r) \text{ for all } r \in R$$

(for the real root functions $\theta_{\xi, s}$, $\theta_{\xi', s}$ according to Definition 4.3).

References

$\text{ir_ord}((\approx, \prec), s)$ Rule 4.9

Note that the semantics of an indexed root expression ξ is only defined in combination with the sample s , as this uniquely determines the real root function $\theta_{\xi, s}$; thus, we add s to the signature of the property. To clarify this further: the sample s is only used to identify the referred real root functions and is not necessarily contained in the



(a) Assume $\prec = \{(\xi_1, \xi_2), (\xi_2, \xi_3)\}$. All three real root functions are defined over the underlying cell. Only the order-invariance of two resultants are necessary to maintain the ordering of the root functions, e.g. $\text{res}_{x_2}(\xi_1.p, \xi_2.p)$ and $\text{res}_{x_2}(\xi_2.p, \xi_3.p)$. We conclude the ordering between ξ_1 and ξ_3 by transitivity.

(b) Assume $\prec = \{(\xi_1, \xi_2), (\xi_1, \xi_3)\}$. The lower bound on x_1 is due to the delineability of the common defining polynomial of ξ_1 and ξ_2 . The ordering $\xi_1 \prec \xi_2$ is maintained by $\text{disc}_{x_2}(\xi_1.p)$. The ordering $\xi_1 \prec \xi_3$ by the resultant $\text{res}_{x_2}(\xi_1.p, \xi_3.p)$. Note that latter causes the upper bound on x_1 to be smaller than necessary, as $\xi_3.p$ intersects $\theta_{\xi_2, s}$ “before” it intersects $\theta_{\xi_1, s}$.

Figure 4.3: Two examples for the property $\text{ir_ord}((\emptyset, \prec), s)$.

constructed cell by definition of the above property (although in the following proof rules, this will be required).

The property is maintained by delineability and adding resultants for the defining polynomials of a pair of indexed root expressions to the projection. We essentially prove that the ordering of root functions ensured by the order-invariance of resultants and discriminants is transitive. Figure 4.3 shows an example where transitivity is maintained for three root functions.

Lemma 4.1

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be a connected analytic submanifold and $p_1, p_2 \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ irreducible and coprime such that $\text{level}(p_1) = \text{level}(p_2) = i + 1$, $\theta_1, \theta_2 : R \rightarrow \mathbb{R}$ be real root functions of p_1 and p_2 respectively, and $\sim \in \{=, <, >\}$.

If p_1, p_2 are not nullified on any point in R and $\text{res}_{x_{i+1}}(p_1, p_2)$ is order-invariant on R and p_1 and p_2 are projectively delineable on R , then $\theta_1(r) \sim \theta_2(r) \iff \theta_1(r') \sim \theta_2(r')$ for all $r, r' \in R$.

Proof. The proof is given in [NÁS⁺24, Appendix A, Proof of Theorem A.1]. $\square$

Rule 4.9

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $(\approx, \prec)$ be an indexed root ordering of level $i + 1$.

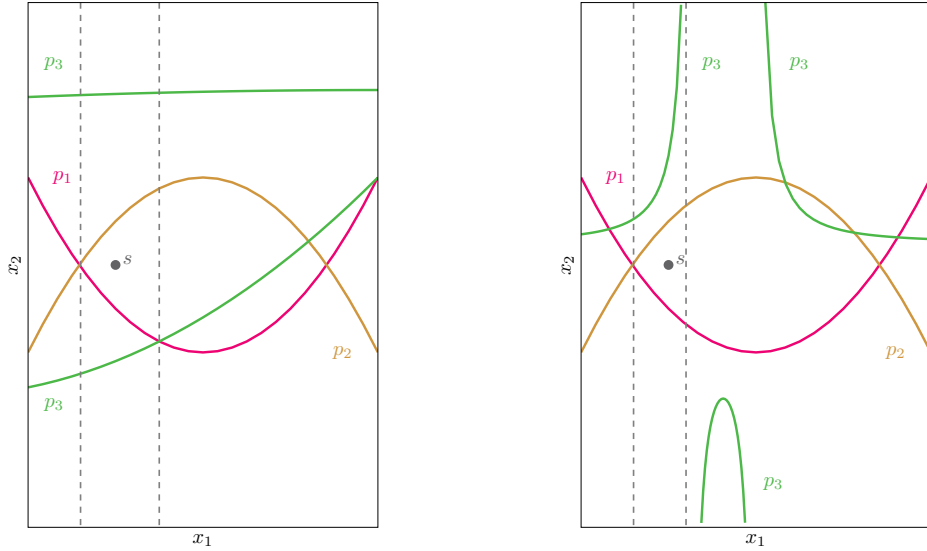
Assume that $\xi.p$ is irreducible for all $\xi \in \text{dom}(\approx, <)$, and that $(\approx, <)$ matches s .

$$\begin{aligned} & \text{sample}(s)(R), \text{an_sub}(i)(R), \text{connected}(i)(R), \\ & \forall \xi \in \text{dom}(\approx, <). \text{an_del}(\xi.p)(R), \\ & \forall (\xi, \xi') \in (\approx \cup <). \text{ord_inv}(\text{res}_{x_{i+1}}(\xi.p, \xi'.p))(R) \quad \vdash \text{ir_ord}((\approx, <), s)(R) \end{aligned}$$

Proof. First consider $\xi \sim \xi'$. Either $\xi.p = \xi'.p$, then $\theta_{\xi,s} = \theta_{\xi',s}$ by definition of delineability (Definition 3.3), or $\text{res}_{x_{i+1}}(\xi.p, \xi'.p)$ is order-invariant on R , then $\theta_{\xi,s} = \theta_{\xi',s}$ holds by Lemma 4.1. Similarly, for $\xi < \xi'$, it holds $\text{res}_{x_{i+1}}(\xi.p, \xi'.p)$ is order-invariant on R , and thus $\theta_{\xi,s} < \theta_{\xi',s}$ on R holds by Lemma 4.1.

Now consider $\xi(<_{\approx, <} \cup \approx_{\approx, <})\xi'$, then there exist indexed root expressions $\xi_1, \dots, \xi_k$ such that $\xi = \xi_1 \sim_1 \dots \sim_{k-1} \xi_k = \xi'$ with $\sim_1, \dots, \sim_{k-1} \in \{\approx, <\}$. We show that either $\theta_{\xi,s} = \theta_{\xi',s}$ on R or $\theta_{\xi,s} < \theta_{\xi',s}$ on R holds by induction. The base case for $k = 2$ is proven by the preceding paragraph. Assume that the statement holds for ξ_1 and ξ_j with $j < k$ (induction hypothesis), i.e. either $\theta_{\xi_1,s} = \theta_{\xi_j,s}$ or $\theta_{\xi_1,s} < \theta_{\xi_j,s}$ on R . Observe that $\theta_{\xi_j,s} = \theta_{\xi_{j+1},s}$ or $\theta_{\xi_j,s} < \theta_{\xi_{j+1},s}$ on R by the preceding paragraph. It follows by transitivity that $\theta_{\xi_1,s} = \theta_{\xi_{j+1},s}$ or $\theta_{\xi_1,s} < \theta_{\xi_{j+1},s}$ on R . $\square$

Now we are ready to define the rule for ensuring the sign-invariance of a polynomial p , given a representation $\mathbf{I}$ and an indexed root ordering $(\approx, <)$. The latter will be used to ensure that no real root function of p crosses the cell's boundaries. The cases are depicted in Figure 4.4.



(a) The indexed root ordering ensures that real root functions remain outside the cell. As p_3 is delineable, this is sufficient to prove sign invariance.

(b) We note that if p_3 has singularities, the cell might be made smaller than required for ensuring sign-invariance due to the strong notion of delineability. Relaxing this notion is done in Section 4.8.

Figure 4.4: Two examples for maintaining the sign-invariance of a polynomial p_3 in a cell (here defined by the enclosed cell between the graphs of p_1 and p_2).

Rule 4.10

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, I be a symbolic interval of level i , and $(\approx, \prec)$ be an indexed root ordering of level i .

Assume that p is irreducible, $\text{ire}(p, s) \neq \emptyset$, $\xi.p$ is irreducible for all $\xi \in \text{dom}(\approx, \prec)$, $(\approx, \prec)$ matches s , and for all $\xi \in \text{ire}(p, s)$ it holds either $\xi <_{\approx, \prec} I.l$, $\xi =_{\approx, \prec} I.l$ (if $I.l \neq -\infty$), $I.u <_{\approx, \prec} \xi$, or $I.u =_{\approx, \prec} \xi$ (if $I.u \neq \infty$).

$$\text{sample}(s)(R), \text{repr}(I, s)(R), \text{ir_ord}((\approx, \prec), s)(R), \text{an_del}(p)(R) \vdash \text{sgn_inv}(p)(R)$$

Proof. As p is delineable on $R \downarrow_{[i-1]}$, the variety of p on $R \downarrow_{[i-1]}$ is described by the real root functions $\theta_{\xi, s}$, $\xi \in \text{ire}(p, s)$. As $\text{repr}(I, s)$ holds on R , it follows that if $l \neq -\infty$, then $\theta_{l, s}$ exists on $R \downarrow_{[i-1]}$, and if $u \neq \infty$, then $\theta_{u, s}$ exist on $R \downarrow_{[i-1]}$.

As $s \in R \downarrow_{[i-1]}$, $(\approx, \prec)$ matches s , the conditions on $(\approx, \prec)$, and $\text{ir_ord}((\approx, \prec), s)$ holds on $R \downarrow_{[i-1]}$, for each $\xi \in \text{ire}(p, s)$ it holds that $\theta_{\xi, s} < \theta_{l, s}$ on R , or $\theta_{u, s} < \theta_{\xi, s}$ on R , or $\theta_{\xi, s} = \theta_{l, s}$ on R , or $\theta_{\xi, s} = \theta_{u, s}$ on R . $\square$

4.2.5 Connectedness

For maintaining properties (in particular the order-invariance of some polynomials) on higher levels, we need to maintain connectedness. Besides some preconditions, i.e. that the cell's boundaries are described by two well-defined real root functions, we need to ensure that the lower and upper bound of a sector do not cross as illustrated in Figure 4.5.

Rule 4.11

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, I be a symbolic interval of level i , and $(\approx, \prec)$ be an indexed root ordering of level i . Assume that $(\approx, \prec)$ matches s .

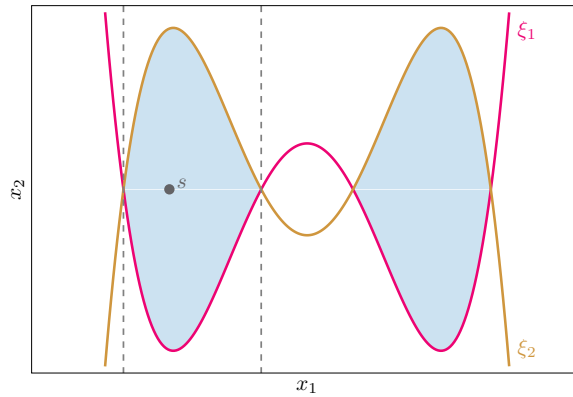


Figure 4.5: Let $I = (\xi_1, \xi_2)$, then $\text{repr}(I, s_1)$ holds in the whole blue area, thus forming a non-connected subset of $\mathbb{R}^2$. By adding $\text{res}_{x_2}(\xi_1.p, \xi_2.p)$ to the projection, the subset is made connected by restricting the underlying cell as indicated by the dashed vertical lines.

Let $Q := \text{connected}(i-1)(R) \wedge \text{repr}(\mathbf{I}, s)(R) \wedge \text{sample}(s)(R)$.

$$\begin{array}{ll}
 & \vdash \text{connected}(0)(\mathbb{R}^0) \\
 Q, \mathbf{I} = (l, u), l \neq -\infty, u \neq \infty, l <_{\approx, <} u, \text{ir_ord}((\approx, <), s)(R) & \vdash \text{connected}(i)(R) \\
 Q, \mathbf{I} = (l, u), l = -\infty \vee u = \infty & \vdash \text{connected}(i)(R) \\
 Q, \mathbf{I} = [b, b] & \vdash \text{connected}(i)(R)
 \end{array}$$

Proof. We only consider the case $\mathbf{I} = (l, u), l \neq -\infty, u \neq \infty, l.p \neq u.p$, as the other cases are similar but simpler.

As $\text{repr}(\mathbf{I}, s)$ holds, $\theta_{l,s}$ and $\theta_{u,s}$ are well-defined on $R_{\downarrow[i-1]}$. As $s \in R$, $(\approx, <)$ matches s , $l <_{\approx, <} u$, and $\text{ir_ord}((\approx, <), s)$ holds on $R_{\downarrow[i-1]}$, it holds $\theta_{l,s} < \theta_{u,s}$ on $R_{\downarrow[i-1]}$. The property $\text{repr}(\mathbf{I}, s)$ holds on R and $R_{\downarrow[i-1]}$ is connected, thus R is connected (noting that empty sets are connected). $\square$

4.2.6 Generalization of the Sample Point

We define a mapping for generalizing the sample to the cell on the current level.

Rule 4.12

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $\mathbf{I}$ be a symbolic interval of level i . Assume that $s_i \in \{s_{[i-1]}\} \times \mathbf{I}$.

$$\begin{array}{ll}
 & \vdash \text{sample}(())(\mathbb{R}^0) \\
 \text{sample}(s_{[i-1]})(R), \text{repr}(\mathbf{I}, s_{[i-1]})(R) & \vdash \text{sample}(s)(R)
 \end{array}$$

Proof. Follows immediately from the definitions. $\square$

4.2.7 Describing Cells

Now, we define a property that states that the cell is described by its representation.

Property 4.4

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, and $\mathbf{I}$ be a symbolic interval of level i .

The property $\text{holds}(\mathbf{I})$ holds on R if and only if $R = R_{\downarrow[i-1]} \times \mathbf{I}$.

This property is the only one that cannot be mapped to a set of other properties, i.e., properties of this kind are the assumptions in our proof system.

Given $\text{holds}(\mathbf{I})$, we can maintain $\text{repr}(\mathbf{I}, s)$. To do so, we need to ensure that each indexed root expression describing the cell's boundaries always refers to the same root on the underlying cell by making their defining polynomials delineable (such that the referred real root function is defined and the number of roots below the referred root function is constant over the underlying cell).

Rule 4.13

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i . Assume that $I.l \in \text{ire}(I.l.p, s)$ (if $I.l \neq -\infty$), $I.u \in \text{ire}(I.u.p, s)$ (if $I.u \neq \infty$) respectively $I.b \in \text{ire}(I.b.p, s)$.

$$\begin{array}{ll} \text{sample}(s)(R), \text{ holds}(I)(R), I = [b, b], \text{ an_del}(b.p)(R) & \vdash \text{repr}(I, s)(R) \\ \text{sample}(s)(R), \text{ holds}(I)(R), I = (l, u), \\ l = -\infty \vee \text{ an_del}(l.p)(R), u = \infty \vee \text{ an_del}(u.p)(R) & \vdash \text{repr}(I, s)(R) \end{array}$$

Proof. The graph of any delineable polynomial p over R is described by real root functions $\theta_1 < \dots < \theta_k$ on R . It follows that for any indexed root expression ξ with $\xi.p = p$ and $\xi.k \leq k$ it holds $\text{dom}(\theta_{\xi,s}) \supseteq R$ and $\theta_{\xi,s} = \theta_{\xi,k} = \xi$ on R .

Now, the result follows immediately from the definition of $\text{holds}(I)$ and the previous statement applied on the cell's boundaries. $\square$

4.2.7.1 Single Cell Projection

Algorithm 4.2 replaces Algorithm 3.9 to use the proof system: Instead of a set of polynomials, it takes a set Q of properties and a symbolic interval I such that all properties in Q are satisfied in the cell $R \times I$ over the cell R which is induced by the returned set Q' of properties. If Q consists of properties $\text{sgn_inv}(p)$ (meaning that p is sign-invariant on the constructed cell $R \times I$) for $p \in P$, the call to Algorithm 4.2 is equivalent to calling Algorithm 3.9 on P .

Algorithm 4.2: compute_cell_projection($Q, s, I, (\approx, <)$)

Input : $Q \subseteq \text{Prop}|_{[i]}$ for some $i \in [n]$, $s \in (\mathbb{R} \cap \bar{\mathbb{Q}})^i$, symbolic interval I of level i , indexed root ordering $(\approx, <)$ of level i .

Output : $Q' \subseteq \text{Prop}|_{[i-1]}$ such that for any $R \subseteq \mathbb{R}^{i-1}$ it holds $(\forall q' \in Q'. q'(R)) \Rightarrow (\forall q \in Q. q(R \times I))$; or *FAIL*.

```

1  $Q := \text{apply}(Q, (s, I, (\approx, <)))$  // Algorithm 4.1
2 if  $Q|_{\{i\}} \neq \{\text{holds}(I)\}$  then
3   return FAIL
4 return  $(Q \setminus \{\text{holds}(I)\}, s)$ 

```

The algorithm takes a symbolic interval I and an indexed root ordering $(\approx, <)$ and passes them to Algorithm 4.1 to instantiate the proof rules. It leads all properties on the current level back to $\text{holds}(I)$ which is not lead back to other properties, but seen as assumption (see Example 4.1). As the proof system — as presented in this section — is incomplete, it might happen that a property of the form $\text{non_null}(p)$ for some nullified polynomial remains on the current level; in this case, the algorithm fails.

It remains the question of how I and $(\approx, <)$ are determined: The short answer is, we replace the call to Algorithm 3.8 by

choose symbolic interval I and indexed root ordering $(\approx, <)$ according to Q .

However, the process is almost the same: The proof system is designed such that after a call to Algorithm 4.2, the highest properties on the level $i - 1$ below are of the form

$sgn_inv(p)$ for irreducible polynomials p and $connected(i)$; we thus pick I and $(\approx, <)$ such that they fulfil the requirements of Rules 4.10 and 4.11. We thus isolate and sort the roots of all these polynomials, determine I according to the given sample point, choose some ordering $(\approx, <)$ that protects I from the other roots, and extend the ordering to ensure connectedness if necessary. We elaborate the choices in Chapter 5.

4.2.8 Projection Consistency and Non-empty Intervals

The cells that may be constructed using the proof system introduced so far may lead to non-connected cells, as shown in Figure 4.5. This example violates also another important property that is required for certain applications, which we call *projection consistency*, which means that the description of the underlying cell $I_1 \times \dots \times I_{i-1}$ is the projection $(I_1 \times \dots \times I_i) \downarrow_{[i-1]}$ of the cell $I_1 \times \dots \times I_i$. This property is not required by all applications, but at least by the CAIC and NuCAD algorithm.

Defining a property for projection consistency in the framework of our proof system is not possible. Instead, we introduce a property $non_empty(I, s)$ that states that some symbolic interval I is non-empty over the cell being constructed (if the cell is empty, the property trivially holds). If this property holds for all symbolic intervals describing a cell, this implies projection consistency. Thus, if we need projection consistency, we modify Algorithm 4.2 by inserting

$$Q := Q \cup \{non_empty(I, s)\}$$

before the first line.

Property 4.5

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and I be a symbolic interval of level $i + 1$.

The property $non_empty(I, s)$ holds on R if and only if $I.l \in \text{ire}(I.l.p, s)$ (if $I.l \neq -\infty$), $I.u \in \text{ire}(I.u.p, s)$ (if $I.u \neq \infty$), and one of the following hold:

- $I = (l, u)$, $\text{dom}(\theta_{l,s}) \cap \text{dom}(\theta_{u,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \times (\theta_{l,s}, \theta_{u,s}) \downarrow_{[i-1]}$; or
- $I = (-\infty, u)$, $\text{dom}(\theta_{u,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \times (-\infty, \theta_{u,s}) \downarrow_{[i-1]}$; or
- $I = (l, \infty)$, $\text{dom}(\theta_{l,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \times (\theta_{l,s}, \infty) \downarrow_{[i-1]}$; or
- $I = (-\infty, \infty)$ and $R = R \times (-\infty, \infty) \downarrow_{[i-1]}$; or
- $I = [l, u]$, $\text{dom}(\theta_{l,s}) \cap \text{dom}(\theta_{u,s}) \supseteq R \downarrow_{[i-1]}$ and $R = R \times [\theta_{l,s}, \theta_{u,s}] \downarrow_{[i-1]}$; or
- another analogous case for $I = (-\infty, u]$, $I = [l, \infty)$, $I = (l, u]$, $I = [l, u)$.

(for the real root functions $\theta_{l,s}$ and $\theta_{u,s}$ according to Definition 4.3).

References

$non_empty(I, s)$ Rule 4.14, Rule 4.33, Rule 4.44

This property is proven like the connectedness rule:

Rule 4.14

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, I be a symbolic interval of level $i + 1$, and $(\approx, \prec)$ be an indexed root ordering of level $i + 1$. Assume that $(\approx, \prec)$ matches s .

$$\begin{array}{ll}
I = (l, u), l \neq -\infty, u \neq \infty, l <_{\approx, \prec} u, \text{sample}(s)(R), \\
\text{an_del}(l.p)(R), \text{an_del}(u.p)(R), \text{ir_ord}((\approx, \prec), s)(R) & \vdash \text{non_empty}(I, s)(R) \\
I = (l, u), l = -\infty, u \neq \infty, \text{an_del}(u.p)(R) & \vdash \text{non_empty}(I, s)(R) \\
I = (l, u), l \neq -\infty, u = \infty, \text{an_del}(l.p)(R) & \vdash \text{non_empty}(I, s)(R) \\
I = (l, u), l = -\infty, u = \infty & \vdash \text{non_empty}(I, s)(R) \\
I = [b, b], \text{an_del}(b.p)(R) & \vdash \text{non_empty}(I, s)(R)
\end{array}$$

Proof. We only consider the case $I = (l, u), l \neq -\infty, u \neq \infty, l.p \neq u.p$, as the other cases are similar but simpler.

As $l.p$ and $u.p$ are delineable on R , $\theta_{l,s}$ and $\theta_{u,s}$ are well-defined on R . As $s \in R$, $(\approx, \prec)$ matches s , $l <_{\approx, \prec} u$, and $\text{ir_ord}((\approx, \prec), s)$ holds on R , it holds $\theta_{l,s} < \theta_{u,s}$ on R . Thus, $(\theta_{l,s}(r), \theta_{u,s}(r))$ is non-empty for each $r \in R$, which implies the desired property. $\square$

To simplify the set of proof rules, the following property replaces Rule 4.11 using the new property:

Rule 4.15

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i .

$$\text{connected}(i-1)(R), \text{repr}(I, s)(R), \text{non_empty}(I, s)(R) \vdash \text{connected}(i)(R)$$

Proof. We only consider the case $I = (l, u), l \neq -\infty, u \neq \infty, l.p \neq u.p$, as the other cases are similar but simpler.

The property $\text{repr}(I, s)$ holds on R , $R \downarrow_{[i-1]}$ is connected, and $\text{non_empty}(I, s)$ holds on $R \downarrow_{[i-1]}$, thus R is connected (noting that empty sets are connected). $\square$

4.2.9 Covering of Cylinders

For using the proof system in the CAIC algorithm, we define a property that holds on R if and only if a set of symbolic intervals covers the whole real line if we substitute any point $s \in R$, as illustrated in Figure 4.6.

Property 4.6

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $\mathbf{C} = (I_1, \dots, I_k)$ be a sequence of symbolic intervals of level $i + 1$.

The property $\text{cov}(\mathbf{C}, s)$ holds on R if and only if there exist $R'_1, \dots, R'_k \subseteq \mathbb{R}^{i+1}$ such that $R'_j \downarrow_{[i]} = R$ and $\text{repr}(I_j, s)$ holds on R'_j for every $j \in [k]$, and $R = \bigcup_{j \in [k]} R'_j$.

References

$cov(\mathbf{C}, s)$ Rule 4.16, Rule 4.34

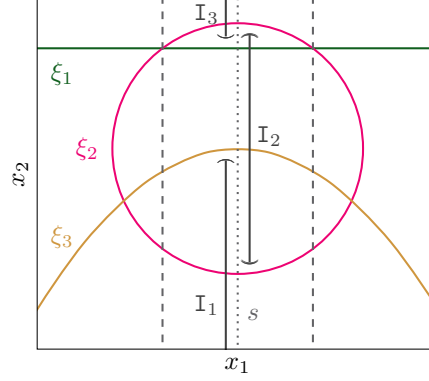


Figure 4.6: Covering of symbolic intervals.

This property can be proven using the following rule. It assumes that the intervals are ordered by their lower bounds and are not redundant. We then use an indexed root ordering which ensures that the bounds of neighbouring intervals overlap.

Rule 4.16

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $\mathbf{C} = (I_1, \dots, I_k)$ be a sequence of symbolic intervals of level $i + 1$, and $(\approx, <)$ be an indexed root ordering of level $i + 1$.

Assume that $\mathbf{C}$ fulfils the following conditions:

- $I_1.l = -\infty$ and $I_k.u = \infty$,
- $I_j.l(s) < I_{j+1}.l(s) \vee (I_j.l(s) = I_{j+1}.l(s) \wedge (I_j \text{ section} \wedge I_{j+1} \text{ sector}))$ for $j \in [k-1]$,
- $I_j.u(s) < I_{j+1}.u(s) \vee (I_j.u(s) = I_{j+1}.u(s) \wedge (I_j \text{ sector} \wedge I_{j+1} \text{ section}))$ for $j \in [k-1]$, and
- $I_{j+1}.l(s) < I_j.u(s) \vee (I_{j+1}.l(s) = I_j.u(s) \wedge (I_{j+1} \text{ section} \vee I_j \text{ section}))$ for $j \in [k-1]$.

Assume that $(\approx, <)$ matches s , and for $j \in [k-1]$ it holds $I_{j+1}.l \approx_{\approx, <} I_j.u$ or $I_{j+1}.l <_{\approx, <} I_j.u$.

$$\begin{aligned} & \text{sample}(s)(R), \text{ir_ord}((\approx, <), s)(R), \\ & \forall j \in [k-1]. (\text{an_del}(I_j.u.p)(R) \wedge \text{an_del}(I_{j+1}.l.p)(R)) \quad \vdash cov(\mathbf{C}, s) \end{aligned}$$

Proof. We ensure that all polynomials defining the lower and upper bounds of the symbolic intervals are analytically delineable on R which contains the current sample point, that means that all their root functions that exist over R are well-defined on R .

By assumption, it holds $s \times I_1 \cup \dots \cup s \times I_k = s \times \mathbb{R}$. More specifically, the two neighbouring intervals I_j and I_{j+1} overlap or are equal, but do not contain each other. Thus, by ensuring $\text{ir_ord}((\approx, <), s)$ on R , for any $s' \in R$ and $r \in \mathbb{R}$, the point (s', r) fulfils $\text{repr}(I_j, s)$ for a $j \in [k]$. $\square$

Note that we do not need to maintain the $non_empty(I, s)$ property for each interval to ensure that the cylinder is covered; the ordering on the roots ensures that for each point in the cylinder, there is *some* non-empty interval that includes that point.

4.2.9.1 Covering Projection

Algorithm 4.3 replaces Algorithm 3.15 and embeds the proof system. The structure of both algorithms is the same: We first process each cell separately, and then characterize the covering of these cells. Both take a sequence of tuples which describe cells that cover the real line above some sample point; the new algorithm additionally takes some indexed root ordering which will be used both for describing the cells forming the individual cells and the covering.

Algorithm 4.3: $compute_covering_projection(((Q^1, s^1, I^1), \dots, (Q^k, s^k, I^k)), (\approx, <))$

Input : Sequence of tuples $((Q^1, s^1, I^1), \dots, (Q^k, s^k, I^k))$ where $Q^j \subseteq Prop|_{i+1}$, $s^j \in (\mathbb{R} \cap \mathbb{Q})^{i+1}$, and symbolic interval I^j of level $i+1$ for some $i \in [0..n-1]$ such that the sequence complies with the output of Algorithm 3.14 (e.g. $s' = s^1_{[i]} = \dots = s^k_{[i]}$); and an indexed root ordering $(\approx, <)$ of level $i+1$.

Output : $Q' \subseteq Prop|_{[i]}$ such that for any $R \subseteq \mathbb{R}^i$ it holds $(\forall q' \in Q'. q'(R)) \Rightarrow \exists j \in [k]. (\forall R' \in Ext(R). (repr(I^j, s^j)(R') \Rightarrow (\forall q \in Q^j. q(R'))))$; or *FAIL*.

```

1 for  $j \in [k]$  do
2    $Q^j := apply(Q^j, (s^j, I^j, (\approx, <)))$  // Algorithm 4.1
3   if  $Q^j|_i$  contains other property than  $repr(I^j, s^j)$  then
4     return FAIL
5  $Q := \bigcup_{j \in [k]} Q^j$ 
6  $Q := Q \cup \{cov((I^1, \dots, I^k), s')\} \setminus \{repr(I^j, s^j) \mid j \in [k]\}$ 
7  $Q := apply(Q, (s', (\approx, <)))$  // Algorithm 4.1
8 return  $(Q, s')$ 

```

For characterizing the cells individually, it applies the rules such that $repr(I^j, s^j)$ is the only remaining property on the highest level, which states that the bounds of I^j are well-defined over some cell around s^j (we pass this information to Algorithm 4.1 although not explicitly stated in the algorithm). It then builds the union of the collected underlying properties and adds the $cov((I^1, \dots, I^k), s')$ property, which states that $R \times I^1, \dots, R \times I^k$ cover the cylinder $R \times \mathbb{R}$ of any cell R that satisfies the property. Finally, we apply proof rules again.

The application of the algorithm may be justified by the following “meta” rule:

Theorem 4.1

Let $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $I_1, \dots, I_k$ be symbolic intervals of level $i+1$, $Q_1, \dots, Q_k \subseteq Prop|_{[i]}$, $Q'_1, \dots, Q'_k \subseteq Prop|_{[i-1]}$ such that for each $j \in [k]$ it holds

$$repr(I_j, s)(R), \forall q \in Q'_j. q(R) \vdash \forall q \in Q_j. q(R).$$

Then

$$\begin{aligned} cov((I_1, \dots, I_k), s)(R), \forall q \in (Q'_1 \cup \dots \cup Q'_k). q(R) \\ \vdash \forall s' \in R \times \mathbb{R}. \exists R' \in Ext(R). (s' \in R' \wedge \exists j \in [k]. \forall q \in Q_j. q(R')). \end{aligned}$$

4.2.10 Summary and Discussion

4.2.10.1 Graph of Properties

We observe that the rules of inference defined in Rules 4.1 to 4.13 are cycle-free in the sense that all properties in the antecedents are smaller than the consequent property according to some ordering $\triangleleft$, which can be read off Figure 4.7, which visualizes the properties introduced in this section and how they relate to each other as established by the proof rules.

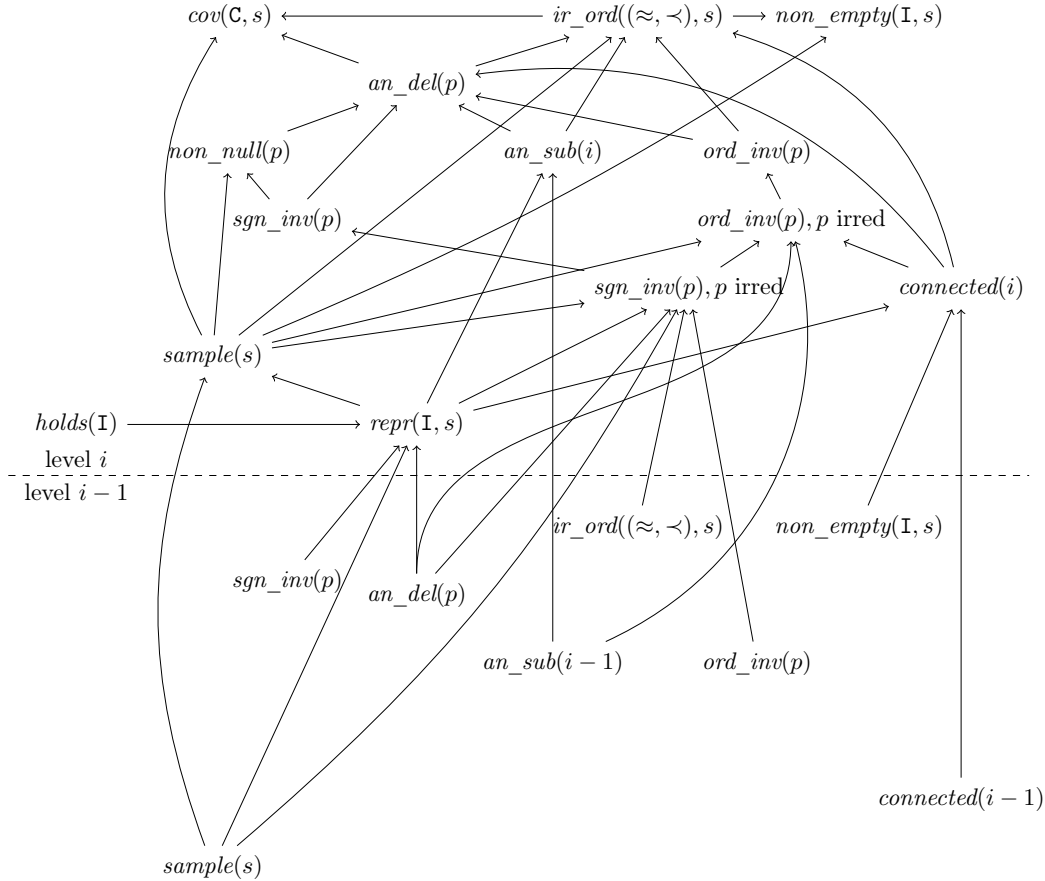


Figure 4.7: Proof rule relationships; *irred* stands for *irreducible*.

4.2.10.2 Non-connected “Cell” Descriptions

Recall that we aim to compute a single cell on which the input polynomials are sign-invariant, but McCallum’s CAD projection theory uses the stronger property of order-invariance. It has been observed before that this can allow for small optimizations at the top layer. For example, when using equational constraint projection we must take discriminants if the projection is in a middle layer (to ensure order-invariance is provided which is a hypothesis of the next lifting) but can avoid these at the top layer as we never need to lift over that [EBD20].

The visualization of our proof rules in Figure 4.7 alerted us to another such optimization, which if enacted has some strange consequences. Note that a cell being connected at level i is required for it to be order-invariant, but not sign-invariant. Thus, we do not

need to ensure connectivity at the top level, i.e. we could avoid taking resultants of the upper and lower bound polynomials to satisfy Rule 4.11. Without connectedness, it would not be accurate to describe what we are constructing as a cell, rather it is a semi-algebraic set (see also Figure 4.5). However, it is still describing a portion of space on whose points the respective polynomials are all sign-invariant. Thus, in the context of the NLSAT search (see Section 3.4.2), the set is still describing a portion of space on whose points the constraints are all unsatisfiable for the same reason. Thus, its negation is still a valid explanation clause for the further search.

We note that the resultants saved by this optimization may still have to be computed later, if the cell is used in further propagation. However, this will not always be the case and so often this optimization may save computation. Discovery of this optimization illustrates the advantages of the proof system presentation.

4.2.10.3 Non-connected “Cells” as Part of a Covering

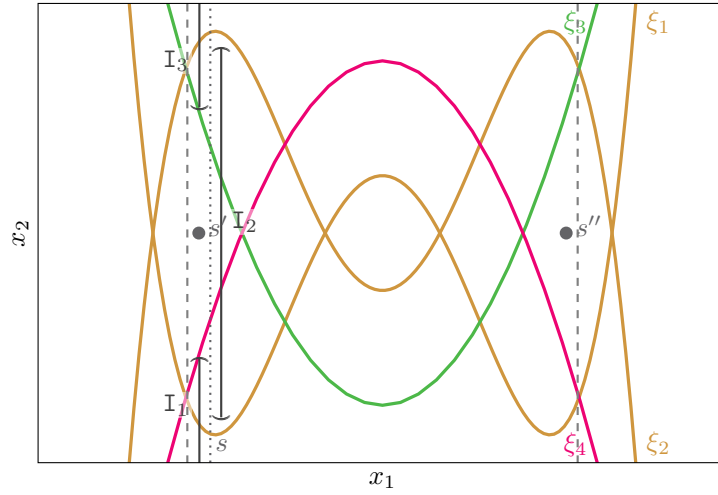


Figure 4.8: Non-connected cells are also possible for coverings if not explicitly assured: Here, three cells form a covering over the sample s . There are two samples s' and s'' which are contained in different connected components of the subset defined by (ξ_1, ξ_2) . Although the cells $(-\infty, \xi_4)$, (ξ_1, ξ_2) , and (ξ_3, ∞) cover the whole cylinder of some connected base cell, some higher-level properties which are satisfied at s' may not be satisfied at s'' .

We emphasize that the phenomenon of non-connectedness is also relevant in case we compute a covering: Figure 4.8 illustrates an example which shows that *connected*(i) is not implied by some covering, and thus we always need to apply Rule 4.11 for each cell individually, even if we form a covering of cells.

4.2.10.4 Factorization of Polynomials

To ensure that the output of the CAD algorithm is correct we must compute a *square-free basis* of the current set of polynomials P (i.e. a set of square-free polynomials without common factors which define the same varieties as P) before the application of a projection operator.

The approach of the rules presented above is to fully factorize each polynomial, resulting in what is called the *finest* square-free basis. This is a fairly standard choice made in CAD implementations as the effort of computing a full factorization pays off compared to the heavy resultant, discriminant, and real root isolation computations, which are all simpler for smaller polynomials.

When building a full CAD, the choice of a square-free basis does not affect the decomposition computed, just the time taken to compute it. However, for the single cell construction, this makes a difference also in the size of the resulting cell, specifically the cell can be larger if we factor. This can easily be observed by considering Example 3.18. Here, polynomial $p_1 \cdot p_2 \cdot p_3$ is already square-free. Using this directly in the single cell construction algorithm without factorization as a whole would simply result in computing the discriminant of the polynomial (and some coefficients): the discriminant must have as factors all the cross resultants of p_1, p_2, p_3 by definition. So in this case, no improvement over a full projection is achieved, and we would find the smaller cell from Figure 3.18a. However, if we factor to consider the set $\{p_1, p_2, p_3\}$ instead then we can obtain the larger cell from Example 3.18.

4.2.10.5 The Limits of Factorization

We note that this described gain in size of the computed cell from factorization does not mean that we build “optimal” cells for all inputs where there is a geometric separation. Figure 4.9a shows an example with two circles and a linear polynomial for which the constructed cell we ideally build is the inside of the circle defined by p_1 . If we were originally presented with $p_1 \cdot p_2$, then factorization would allow the single cell construction to ignore the intersections of p_2 and p_3 to construct the entire inner circle. But consider the similar example from Figure 4.9b, in which we have perturbed the problem to consider the irreducible polynomial $p_1 \cdot p_2 + 1$. Graphically, the two inputs seem to be similar, and the delineation of the roots are identical. But in the second

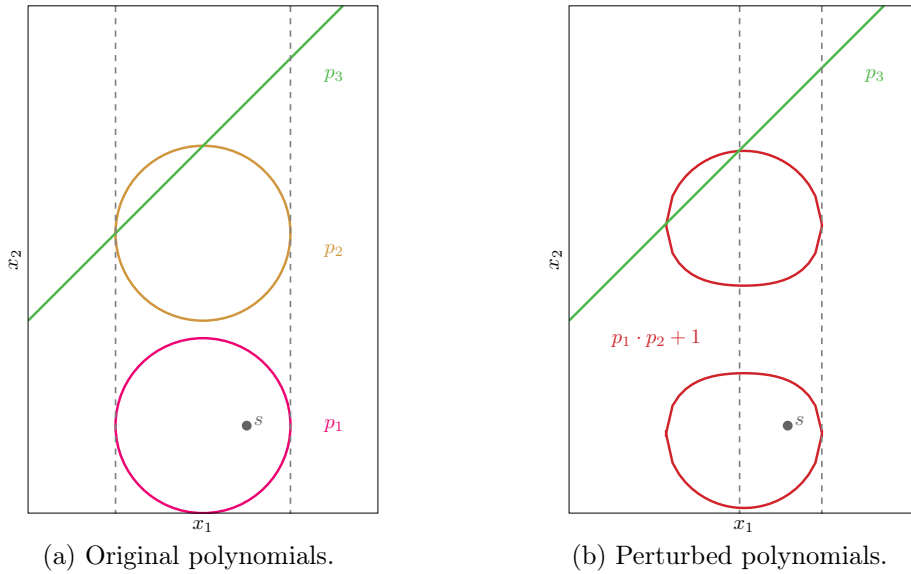


Figure 4.9: By perturbing a reducible polynomial making it irreducible, the problem gets harder.

case the single cell construction cannot treat the two ovals separately: we must consider the irrelevant intersection and thus build a smaller cell.

This shows some limits of the single cell construction by way of a well-observed truth in computer algebra: inputs which appear similar to human beings (i.e. when viewed geometrically) are not always equally hard algebraically.

4.3 Equational Constraint Projection

As discussed in Section 3.2.8.3, the CAD has weaknesses when reasoning about equalities in the input. An optimization to CAD which tailors it to the logical structure of the problem is the theory of equational constraints. A polynomial equation is an *equational constraint* if it is logically implied by the truth of the input formula. If the input to CAD has an equational constraint then we may perform fewer projection and lifting operations to achieve a truth-invariant CAD. This optimization dates back to [McC99] with the recent paper [EBD20] summarizing the state of the art.

In the context of building a single cell, we have even greater scope to use equational constraint savings. Here, any time we are generalizing in the section case, we can apply the reduced projection and lifting. I.e., if a polynomial is zero at the sample point then it must also be zero throughout the cell we are generalizing the sample to, and so for the purposes of this cell, it is an equational constraint. In this case we need only make the section defining polynomial delineable. All other constraints can be made sign-invariant simply by including resultants with the section defining polynomial (but not other projection polynomials).

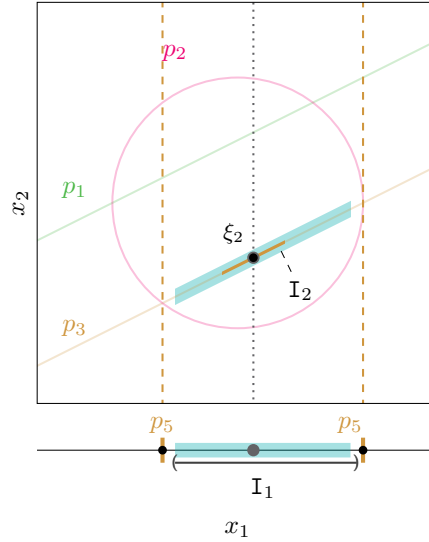


Figure 4.10: Illustration for Example 4.3. The polynomial p_3 is treated as equational constraint as its root describes the symbolic interval $I_2 = [\xi_2, \xi_2]$.

Example 4.3

Consider the polynomials from Example 3.16 on page 86 and the point $s = (0.125, -0.4375)$ as illustrated in Figure 4.10. The sample lies on the first root ξ_2 of p_3 , and thus, p_3 is invariantly zero on the cell being constructed, and we may

apply the equational constraint projection. That is, we maintain that p_3 is delineable (which comes for free in this example), and add the resultant of p_3 with p_1 (constant) and with p_2 (which is p_5) respectively.

We state the corresponding proof rule as follows.

Rule 4.17

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, and $\mathbf{I}$ be a symbolic interval of level i . Assume that p is irreducible, and $\mathbf{I} = [b, b]$.

Let $Q := \text{an_sub}(i-1)(R) \wedge \text{connected}(i-1)(R) \wedge \text{repr}(\mathbf{I}, s)(R) \wedge \text{an_del}(b.p)(R)$.

$$\begin{array}{ll} Q, b.p = p & \vdash \text{sgn_inv}(p)(R) \\ Q, b.p \neq p, \text{ord_inv}(\text{res}_{x_i}(b.p, p))(R) & \vdash \text{sgn_inv}(p)(R) \end{array}$$

Proof. As $\text{repr}(\mathbf{I}, s)$ holds on R , it holds $b.p(r) = 0$ for all $r \in R$.

In the first case, if $p = b.p$, p is trivially sign-invariant on R .

In the second case, if $p \neq b.p$, by the order-invariance of $\text{res}_{x_i}(b.p, p)$ on $R \downarrow_{[i-1]}$, [McC99, Theorem 2.2] yields that p is sign-invariant on R . $\square$

For the proof system, in case we encounter a section, the rule allows for an alternative projection to the use of indexed root orderings for maintaining the sign-invariance of polynomials. This allows (1) for avoiding the computation of the discriminants and leading coefficients of polynomials (in case it is not added for maintaining their delineability or order-invariance), and (2) the rule is applicable for polynomials that are nullified at the current sample point.

In case a polynomial is required to be made order-invariant (instead of only sign-invariant), the following rule may be used to omit its leading coefficient, as it replaces maintaining its delineability directly by the order-invariance of its discriminant:

Rule 4.18

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, and $\mathbf{I}$ be a symbolic interval of level i . Assume that p is irreducible, and $\mathbf{I} = [b, b]$.

Let $Q := \text{an_sub}(i-1)(R) \wedge \text{connected}(i-1)(R) \wedge \text{repr}(\mathbf{I}, s)(R) \wedge \text{an_del}(b.p)(R)$.

$$Q, b.p \neq p, \text{ord_inv}(\text{res}_{x_i}(b.p, p))(R), \text{ord_inv}(\text{disc}_{x_i}(p))(R) \vdash \text{ord_inv}(p)(R)$$

Proof. As $\text{repr}(\mathbf{I}, s)$ holds on R , it holds $b.p(r) = 0$ for all $r \in R$.

If $p \neq b.p$, by the order-invariance of $\text{res}_{x_i}(b.p, p)$ on $R \downarrow_{[i-1]}$ and of $\text{disc}_{x_i}(p)$ on $R \downarrow_{[i-1]}$, [McC01, Theorem 2.1] yields that p is order-invariant on R . $\square$

Note that for the application of this rule, Algorithm 4.2 needs to be adapted to postpone the application of a proof rule to this property to the point where the symbolic interval describing the cell on the current level has been chosen.

Regarding the choice of $\mathbf{I}$ and $(\approx, <)$ before calling Algorithm 4.2, the application of Rule 4.17 can simply be enforced by not considering the respective polynomial in the ordering $(\approx, <)$.

4.4 Sign- and Order-invariance for Nullified Polynomials

The proof system presented so far is incomplete, as it fails on nullified polynomials.

A common solution for fixing the incompleteness of McCallum's projection is to use Lazard's projection and lifting [Laz94; MPP19; BM20; Nai21] (see Section 1.1.2). It is expected that the resulting projection is the same as McCallum if no nullification occurs. Although our proof system could be extended for Lazard's projection, Lazard's theory relies on a different lifting scheme, which would require extensive modifications to the proof system and the ideas in the following sections that build on it.

In this section, we add rules that make the proof system complete — certainly in a less efficient way than Lazard's theory allows, but the cases where no nullifications occur remain unchanged.

If a polynomial that is sign-invariant is nullified, then we can simply use the following rule to handle its nullification:

Rule 4.19

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is irreducible, and $\text{coeff}_{x_i}(p) = \{q_1, \dots, q_k\}$.

$$p(s, x_i) = 0, \text{ sample}(s)(R), \text{ sgn_inv}(q_1)(R), \dots, \text{sgn_inv}(q_k)(R) \vdash \text{sgn_inv}(p)(R)$$

Proof. Follows immediately from the fact that p is nullified on $R \downarrow_{[i-1]}$ if and only if $q_j(r) = 0$ for all $r \in R \downarrow_{[i-1]}$. $\square$

However, if we aim to show sign-invariance for some input polynomials, we may need to maintain order-invariance for some projection polynomials p : Rule 4.6 leads $\text{ord_inv}(p)$ back to $\text{an_del}(p)$, which in turn is lead back by Rule 4.1 to $\text{non_null}(p)$.

The following rule shows order-invariance for nullified polynomials without requiring its delineability. It applies the definition of the order of a polynomial at some point in an almost straight-forward way, and is thus rather inefficient:

Rule 4.20

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, and $k = \text{ord}_s(p)$. Assume that p is irreducible.

$$\begin{aligned} & p(s_{[i-1]}, x_i) = 0, \text{ sample}(s)(R), \text{ connected}(i)(R), \\ & \exists \alpha \in \mathbb{N}^n. \left(\sum_{i \in [n]} \alpha_i = k \wedge \left(\frac{\partial^k}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p \right)(s) \neq 0 \wedge \text{sgn_inv} \left(\frac{\partial^k}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p \right)(R) \right), \\ & \forall \alpha \in \mathbb{N}^n. \left(\sum_{i \in [n]} \alpha_i = k - 1 \rightarrow \text{sgn_inv} \left(\frac{\partial^{k-1}}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p \right)(R) \right) \\ & \vdash \text{ord_inv}(p)(R) \end{aligned}$$

Proof. Remind the definition of the order of a polynomial p at r :

$$\text{ord}_r(p) = \min(\{k \in \mathbb{N} \mid \exists \alpha \in \mathbb{N}^n. (\sum_{i \in [n]} \alpha_i = k \wedge (\frac{\partial^k}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p)(r) \neq 0)\} \cup \{\infty\}).$$

The second row ensures that $\text{ord}_r(p) \leq k$ for all $r \in R$, which follows immediately from the definition of order invariance.

The third row ensures that $\text{ord}_r(p) \geq k$ for all $r \in R$. By assumption, it holds for all $k' < k$ and $\alpha \in \mathbb{N}^n$, $\sum_{i \in [n]} \alpha_i = k'$ that $(\frac{\partial^{k'}}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p)(s) = 0$. The rule ensures that this holds for $k - 1$ and all $r \in R$, that is for all $r \in R$, it holds $(\frac{\partial^{k-1}}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p)(r) = 0$. It remains to show that for all $k' < k - 1$ and for all $r \in R$, it holds $(\frac{\partial^{k'}}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p)(r) = 0$, which follows from the mean value theorem:

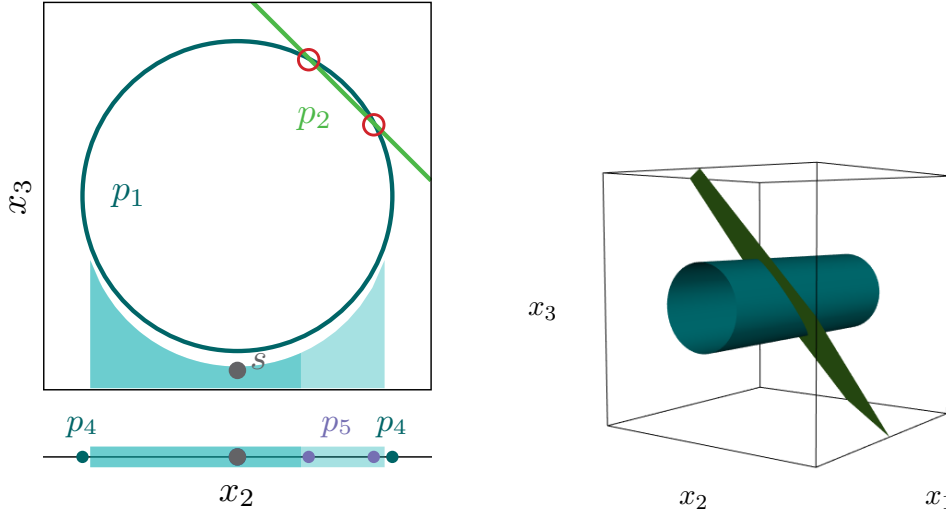
Assume towards a contradiction that there is some $r' \in R$ and a partial derivative of order $k - 2$ for some α , $f := (\frac{\partial^{k-2}}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}} p)$, such that $f(r') \neq 0$. Assume a path (a continuous function) $\pi : [0, 1] \rightarrow R$. Then by the mean value theorem, the first derivative of $\pi \circ f$ is non-zero at some point $t \in [0, 1]$. Thus, there is some partial derivative of p of order $k - 1$ for some α' such that $(\frac{\partial^{k-1}}{\partial x_1^{\alpha'_1} \dots \partial x_n^{\alpha'_n}} p)(\pi^{-1}(t)) \neq 0$, which is a contradiction to the assumption. We inductively apply this argument on partial derivatives of order $k - 3, \dots, 0$ to obtain the desired statement. $\square$

4.5 Merging Adjacent Cells using Local Delineability

An observation in Section 4.2.10.4 is that some intersections of root functions seem irrelevant for the cell, but the algebraic tools do not detect this. Motivated by this, we aim to introduce checks for relevance of intersections (which are witnessed by a root of some resultant), and thus merge adjacent cells by ignoring root functions (which witness irrelevant intersections) when determining the symbolic interval describing the bounds on some variable. Before we formally define what we mean by this, we illustrate the ideas by some single cell construction examples.

Example 4.4

Consider the polynomials $p_1 = x_2^2 + x_3^2 - 1$ and $p_2 = -1.25 + x_2 + x_3$ and the sample $s = (0, -1.125)$, whose real roots are depicted in Figure 4.11a (note that we do not use x_1 here yet, the example is two-dimensional). The single cell algorithm would produce a cell where the x_3 -dimension is bounded from above by the *first* root of p_1 . We make sure it is well-defined by adding the discriminant p_4 of p_1 . To maintain that no root of p_2 crosses this upper bound over the underlying cell, we add the resultant p_5 of p_1 and p_2 to the projection; speaking in terms of properties, we maintain the ordering $\text{root}_{x_2}^{p_1,1} < \text{root}_{x_2}^{p_2,1}$ on the one-dimensional cell. Now, for the x_2 -dimension, the cell will be bounded from above by a root of p_5 . However, both roots of the resultant mark points where p_2 intersects the *second* root of p_1 , thus these intersections would not affect sign-invariance of the constructed cell — they are *irrelevant* to the constructed cell. In this example, we thus could ignore the roots of p_5 , yielding a much larger cell. Computationally, we need to check for every root $s_2 \in \mathbb{R}$ of p_5 whether the first root of $p_1(s_2, x_3)$ is smaller than the first root



(a) The intersections of the real roots of p_1 and p_2 (marked with red circles) are irrelevant for constructing the cell around s . Thus, instead of the smaller cell (shaded in darker colour) bounded from above by a root of the resultant p_5 in the x_2 -dimension, we could obtain the wider cell (shaded in lighter colour) bounded from above by the discriminant p_4 in the x_2 -dimension.

(b) The example extended by another dimension. The two-dimensional version is the cut at $x_1 = -1.25$ of this example.

Figure 4.11: First example from Example 4.4.

of $p_2(s_2, x_3)$ — the computations refer to the correct roots as p_2 is delineable on $\mathbb{R}$ and p_1 is delineable on an interval containing the sample point and the two roots of the resultant (and thus the roots can be indexed consistently over that interval).

In fact, to describe the same set of points, the single cell would need to compute 5 cells (all upper bounded by the first root of p_1 in the x_3 -dimension, and in the x_2 -dimension bounded by the point intervals defined by the roots of p_5 , and the three open intervals between the roots of p_4 and p_5 respectively). Thus, by ignoring roots of the resultant, we *merge* adjacent cells.

Now consider the modified polynomials $p_1 = x_2^2 + x_3^2 - 1$ and $p_2 = x_1 + x_2 + x_3$ and the sample $s = (-1.25, 0, -1.125)$. The circle p_1 becomes a cylinder aligned with the x_1 -axis, and the line gets a plane which is not parallel to any axis. Five cuts of this example and its projection are depicted in Figures 4.11b and 4.12.

Consider the third cut at $x_1 = -1.25$ in Figure 4.12, where the sample point lies. In fact, the situation is the same as in Figure 4.11a, thus we can determine irrelevant intersections analogously to obtain a larger cell. However, note that we cannot ignore the corresponding roots of the resultant p_5 completely: consider the cut at 0, where the intersection became relevant, i.e. if we would not bound the cell in the x_1 -dimension properly, then the cell is not sign-invariant any more. The intuitive reason is that at $x_1 = -1$, the second root of the resultant p_5 switches from an intersection of p_2 with the *second* root of p_1 to an intersection with the *first* root of p_1 , which describes the upper bound in the x_3 -dimension. This point is caught by the intersection of the resultant p_5 with the discriminant p_4 of p_1 , i.e. we need

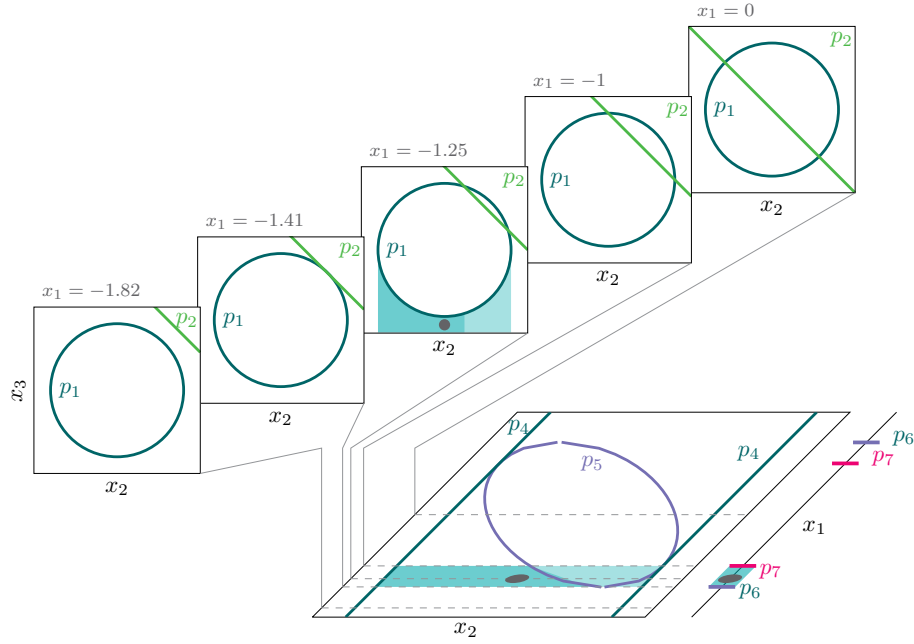


Figure 4.12: First example from Example 4.4, extended by another dimension. The five coordinate systems at the top left depict x_1 -cuts. The two-dimensional coordinate system at the bottom depicts the CAD-projection in the x_1 - x_2 -plane, and the one-dimensional coordinate system depicts the CAD projection on the x_1 -line. On all coordinate systems: the smaller cell is shaded in darker colour, the larger cell is shaded in lighter colour, the point is the sample.

to bound the cell in x_1 -dimension by the root of the resultant p_7 of p_4 and p_5 . For similar reasons, we need to take the discriminant p_6 of the resultant p_5 into account in general.

The reason why we catch everything “bad” that can happen is: The sections of the resultant p_5 of p_1 and p_2 are well-ordered between the roots of the discriminant p_4 of p_1 — i.e. the varieties of p_5 and p_4 can be described by continuous functions $\theta_1 < \theta_2 < \theta_3 < \theta_4$ on the interval between the corresponding roots of p_6 and p_7 . Thus, the sections θ_2 and θ_3 mark the points where the *same* roots of p_1 and p_2 intersect. These intersections are relevant at the current sample point (which we can check computationally as described above) if and only if these intersections are relevant everywhere above these sections. We will later define a property called *local delineability* that generalizes the well-ordering of the sections of the resultants and discriminants.

Example 4.5

Consider the same set of polynomials, now with the sample $s = (0, 0, -1.125)$.

As depicted in Figure 4.13, only a single intersection of p_1 and p_2 is irrelevant to the cell (we maintain the ordering $\text{root}_{x_2}^{p_1,1} < \text{root}_{x_2}^{p_2,1}$). Still, we can merge three adjacent cells (which are in the same cylinder). As the second root of the resultant p_5 witnesses the non-irrelevant intersection, it defines the upper bound in the x_2 -dimension of the cell.

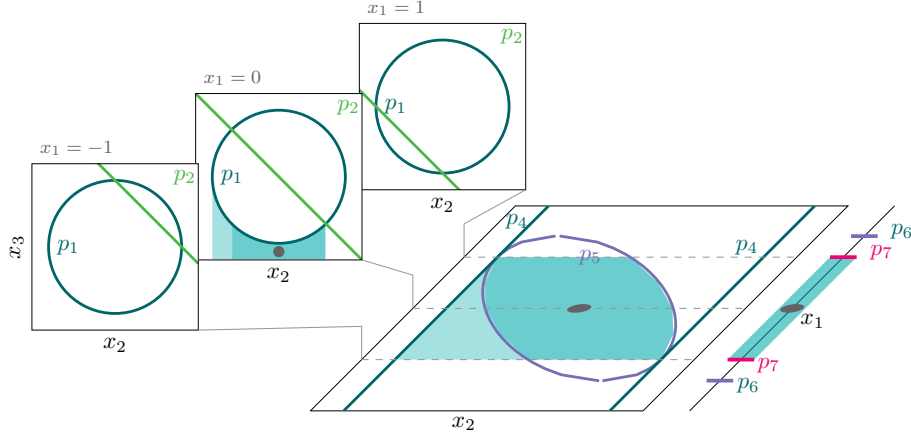


Figure 4.13: First example, different sample point as in Example 4.5.

Before we formally define what local delineability is, we examine a second example hinting an even more general observation:

Example 4.6

Consider the sample point $(-0.5, 0, -1.125)$, the set of polynomials $\{p_1, p_2, p_3\}$ and its projection polynomials:

$$\begin{array}{lll} p_1 = x_2^2 + x_3^2 - 1 & p_2 = x_3 + 0.75 & p_3 = x_1 - x_3 \\ p_4 = \text{disc}_{x_3}(p_1) & p_5 = \text{res}_{x_3}(p_1, p_3) & p_6 = \text{res}_{x_3}(p_1, p_2) \\ p_7 = \text{disc}_{x_2}(p_5) & p_8 = \text{res}_{x_2}(p_5, p_6) & p_9 = \text{res}_{x_2}(p_6, p_4) \end{array}$$

As in the previous examples, we depict some cuts and the projection in Figure 4.14. Observe that the original cell is bounded by the iterated resultant p_8 , i.e. over its roots, the intersections of p_3 with p_1 and p_2 with p_1 intersect. For the cell to be constructed around the given sample point, this intersection is not relevant to our cell^a, as p_3 intersects with the *second* root of p_1 , while the *first* root of p_1 defines the upper bound in the x_3 -dimension (that is, we maintain the ordering $\text{root}_{x_2}^{p_1,1} < \text{root}_{x_2}^{p_2,1}$ and $\text{root}_{x_2}^{p_1,1} < \text{root}_{x_2}^{p_3,1}$ on the two-dimensional cell). We can thus extend the cell beyond the root of p_8 .

Now observe that the varieties of the resultant p_5 of p_1 and p_2 and the discriminant p_4 of p_1 in the larger underlying two-dimensional cell is not described by well-ordered continuous functions — but it is over some larger cell (in the same cylinder over the one-dimensional cell). In this case, to maintain the mentioned *local delineability* property, we only need the discriminant p_7 of p_5 , the resultant of p_6 and p_4 (which is trivial), and the resultant p_9 of p_5 with p_4 — we do not even need to compute the iterated resultant p_8 at all! In contrast to the previous example, the number of computed projection polynomials is reduced.

^aIf we construct a cell at $(0.5, 0, -1.125)$, this intersection of intersections *would be* relevant to the cell.

We developed ideas for an adaptation of the single cell construction for detecting and generalizing irrelevant roots. We will now translate them into our proof system. In Section 4.2, we lead the $ir_ord((\approx, <), s)$ (where $(\approx, <)$ is of level i) property back

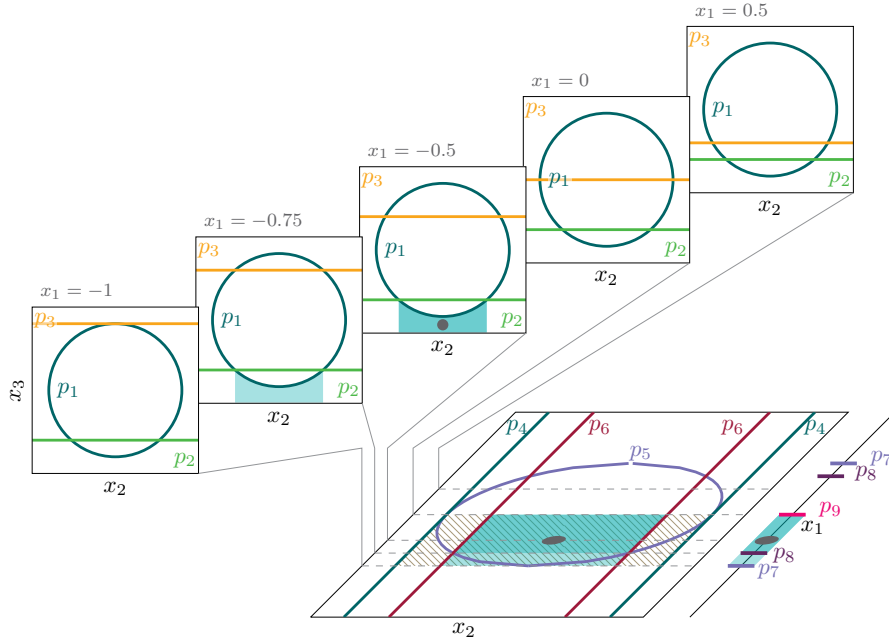


Figure 4.14: Second example from Example 4.6. In the hatched cell, the roots of p_4 and p_5 are well-ordered.

to the order-invariance of a resultant, which was lead back to the sign-invariance of some polynomials on the current level plus some lower-level properties. However, as we want to reason about the resultant's root directly (i.e. allow some roots being inside the underlying cell), we will introduce rules that allow to lead back $ir_ord((\approx, \prec), s)$ to some property $ir_ord((\approx', \prec'), s_{[i-1]})$ (where $(\approx', \prec')$ is of level $i - 1$). This allows to process each pair of root functions in $\approx \cup \prec$ separately, and check for relevant intersections of their defining polynomials.

Processing a pair of root functions involves the following steps: (1) We check over which roots of the resultant of the defining polynomials the root functions intersect. (2) We ensure that the respective resultant is *locally delineable* in the underlying cell, that is, its root functions do not intersect in the underlying cell. This establishes a correspondence between the intersections of the root functions and the resultant's sections. (3) We merge the adjacent sectors with a resultant's section when the root functions do not intersect above the section (i.e. the test from step (1) fails).

To elaborate step (1): Assume two root functions θ_1 and θ_2 of two polynomials p_1 and p_2 , both of level $i + 1$. Remind their representation as indexed root expressions: If p_1 and p_2 are delineable over some cell, the index of their roots is invariant, i.e. the indexed root will refer to the same root function. We thus determine the interval I such that p_1 and p_2 are delineable on $s_{[i-1]} \times I$ by computing the required coefficients and discriminants. We now consider each root $s' \in \mathbb{R}$ of the resultant $\text{res}_{x_i}(p_1, p_2)$ over the current sample $s_{[i-1]} \in \mathbb{R}^{i-1}$ to determine whether θ_1 and θ_2 intersect above $(s_{[i-1]}, s')$: If $s' \in I$, we evaluate θ_1 and θ_2 at $(s_{[i-1]}, s')$ and check whether the result is equal. If $s' \notin I$, we cannot check for intersections of the root functions, thus we assume s' is relevant. We will formalize this in Rule 4.22.

In step (2), we will determine an *ordering of the resultant's roots* that needs to hold on the underlying cell $R' \subseteq \mathbb{R}^{i-1}$ such that the resultant is locally delineable on a

part of the cylinder $R' \times \mathbb{R}$ around the current sample point. We will formalize this in Definition 4.5 and Rule 4.21.

In step (3), we determine the set R : It will be composed of some adjacent sections and sectors $R_1, \dots, R_k \subseteq \mathbb{R}^i$ on R' of the resultant such that θ_1 and θ_2 are in the same order on $R_1, \dots, R_k$ (and thus on R). To determine these sections and sectors, we use the information computed in step (1); the local delineability in step (2) ensures that these cells are *arranged cylindrically*, i.e. the cells that we merge are adjacent cells in the same cylinder $R' \times \mathbb{R}$. Thus, we obtain a cylindrical cell R after the merge.

4.5.1 Local Delineability

We formalize the property described in step (2) above:

Definition 4.5 (Local Delineability)

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$ be a cell, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$, and $R' \subseteq \mathbb{R}^{i+1}$ connected such that $R' \downarrow_{[i]} = R$. The polynomial p is called *locally R' -delineable* if and only if there exists a cell $R'' \supseteq R'$ such that $R'' \downarrow_{[i]} = R$ and there exist continuous functions $\theta_1, \dots, \theta_k : R \rightarrow \mathbb{R}$ (for $k \in \mathbb{N}$) such that

- $(r, \theta_j(r)) \in R''$ for all $r \in R$ and all $j \in [k]$;
- $\theta_j < \theta_{j+1}$ on R for all $j \in [k - 1]$;
- the set of real roots of the univariate polynomial $p(r, x_{i+1})$ that are contained in R'' is $\{\theta_j(r) \mid j \in [k]\}$ for all $r \in R$; and
- there exist constants $m_1, \dots, m_k \in \mathbb{N}_{>0}$ such that for all $j \in [k]$ and all $r \in R$, the multiplicity of the root $\theta_j(r)$ of $p(r, x_{i+1})$ is m_j .

A set of polynomials $P \subset \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(\prod_{p \in P} p) = i + 1$ is called *locally R' -delineable* if and only if $\prod_{p \in P, \text{level}(p)=i+1} p$ is *locally R' -delineable*.

Note that if a polynomial is delineable on R , then it is locally $R \times (-\infty, \infty)$ -delineable.

Figure 4.15 illustrates various examples of local delineability. In particular, Figure 4.15e discusses the limits of local delineability, and suggests a relaxation which is formalized in the following definition. However, the more complex notion might require more computational effort and is harder to translate into an algorithm.

Definition 4.6 (Sample-local Delineability)

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$ be a cell, $s \in R$, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$, and $R' \in \text{Ext}(R)$ be a cell in $\mathbb{R}^{i+1}$. The polynomial p is called *s -locally R' -delineable on R* if and only if there exists a cell $R'' \in \text{Ext}(R)$, $R'' \supseteq R'$ and there exist finitely many continuous partial functions $\theta_1, \dots, \theta_k : R \hookrightarrow \mathbb{R}$ (for $k \geq 0$) such that

- $s \in \text{dom}(\theta_j)$ for all $j \in [k]$;
- $\theta_j(r) \in R''$ for all $r \in \text{dom}(\theta_j)$ and all $j \in [k]$;
- $\theta_j < \theta_{j+1}$ on $\text{dom}(\theta_j) \cap \text{dom}(\theta_{j+1}) \subseteq R$ for all $j \in [k - 1]$;

- the set of real roots of the univariate polynomial $p(r, x_{i+1})$ that are contained in R'' is $\{\theta_j(r) \mid j \in [k], r \in \text{dom}(\theta_j)\}$ for all $r \in R$; and
- there exist constants $m_1, \dots, m_k \in \mathbb{N}_{>0}$ such that for all $j \in [k]$ and all $r \in \text{dom}(\theta_j)$, the multiplicity of the root $\theta_j(r)$ of $p(r, x_{i+1})$ is m_j .

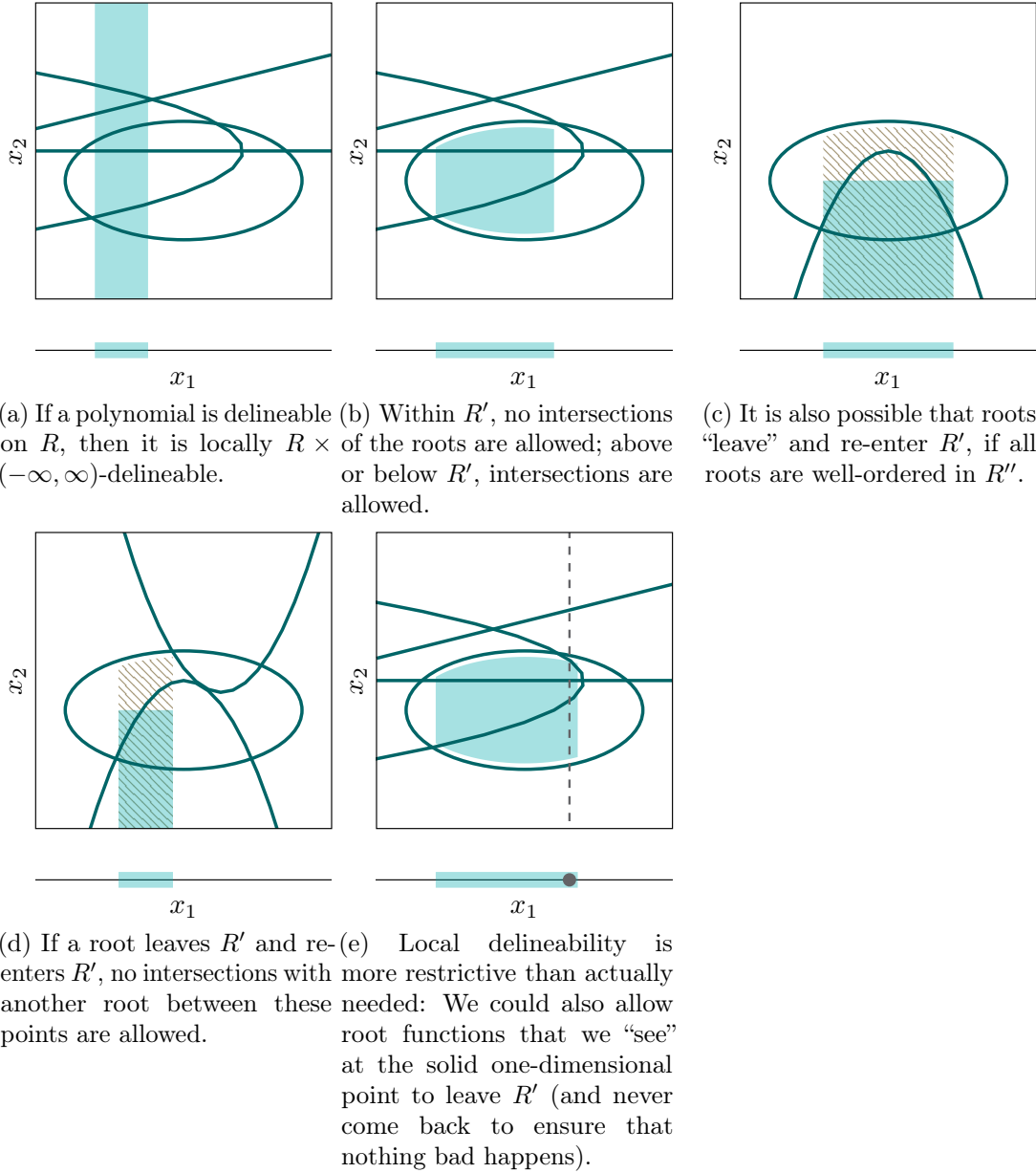


Figure 4.15: Examples of local delineability. The shaded area in the one-dimensional coordinate system depicts R from Definition 4.5, the shaded area in the two-dimensional one depicts R' , the hatched area depicts R'' if it differs from R' .

As in the case of delineability, we define the analytic case.

Definition 4.7 (Analytic Local Delineability)

Analytic local delineability is defined as local delineability, with the modification that it is only defined on connected analytic submanifolds (instead of general cells) and the functions $\theta_1, \dots, \theta_k$ are required to be analytic (instead of only continuous).

Property 4.7

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, and $P \subset \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(\prod_{p \in P} p) = i + 1$, I be a symbolic interval of level $i + 1$, and $s \in \mathbb{R}^i$.

The property $\text{an_loc_del}(P, I, s)$ holds on R if and only if P is analytically locally R' -delineable on some connected superset R'' of R where $R' \in \text{Ext}(R'')$ such that $\text{repr}(I, s)(R')$ holds, and each $p \in P$ is order-invariant in $(\text{dom}(\theta_j) \times \theta_j) \cap R'$ for each $j \in [k]$ where the θ_j are the witnessing functions for analytic delineability of P .

References

$\text{an_loc_del}(P, I, s)$ Rule 4.21, Rule 4.31

We are now able to formulate the rule for local delineability. Observe that order-invariance is just sign-invariance plus some restriction on the underlying cell; we thus simply reason about root functions on the current level, and restrict the lower-level cell to maintain order-invariance on the sections described on the current level.

Rule 4.21

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(\prod_{p \in P} p) = i + 1$, I be a symbolic interval of level $i + 1$, and $(\approx, <)$ be an indexed root ordering of level $i + 1$. Let $Q = \{q \in \text{factors}(P) \mid \text{ire}(q, s) \subseteq \text{dom}(\approx, <)\}$ and $\text{ire}(Q, s) = \{\xi_1, \dots, \xi_k\}$ s.t. $\xi_1(s) \leq \dots \leq \xi_k(s)$.

Assume that $(\approx, <)$ matches s , and that I and $(\approx, <)$ fulfil that there exist $\ell, \ell' \in [k]$ (ℓ' might be smaller than ℓ meaning that we select the empty range) such that:

1. For all $j \in [\ell.. \ell' - 1]$ it holds $\xi_j(=\approx, < \cup < \approx, <) \xi_{j+1}$ on R .
2. For all $j \in [1.. \ell - 1]$ it holds
 - if $I = (I.l, I.u)$ with $I.l \neq -\infty$, then $\xi_j(=\approx, < \cup < \approx, <) I.l$ on R ;
 - if $I = [I.l, I.u]$, then $\xi_j < \approx, < I.l$ on R .
3. For all $j \in [\ell' + 1.. k]$ it holds
 - if $I = (I.l, I.u)$ with $I.u \neq \infty$, then $I.u(=\approx, < \cup < \approx, <) \xi_j$ on R ;
 - if $I = [I.l, I.u]$, then $I.u < \approx, < \xi_j$ on R .
4. For all $j \in [1.. \ell - 1]$ it holds $\xi_j(=\approx, < \cup < \approx, <) \xi_\ell$ on R .
5. For all $j \in [\ell' + 1.. k]$ it holds $\xi_{\ell'}(=\approx, < \cup < \approx, <) \xi_j$ on R .

$$\begin{aligned}
& \text{sample}(s)(R), \text{connected}(i)(R), \text{an_sub}(i)(R), \\
& \text{ir_ord}((\approx, \prec), s)(R), \text{repr}(\mathbf{I}, s)(R), \\
& \forall q \in \text{factors}(P) \setminus Q. \text{ord_inv}(q)(R), \\
& \forall q \in Q. \text{an_del}(q)(R) \quad \vdash \text{an_loc_del}(P, \mathbf{I}, s)(R)
\end{aligned}$$

Note that if we choose $\ell = 1$ and $\ell' = k$, then this yields delineability of P on R (see Example 4.6). If $\ell' < \ell$, then this yields sign-invariance of P on R .

The factors which are not contained in Q are the ones which are not considered in the indexed root ordering $(\approx, \prec)$, which means that we either maintain their sign-invariance using the equational constraints projection rule (we thus simply ensure their sign-invariance to leave the further reasoning for Rule 4.6 and Rule 4.17), or Rule 4.20 if some factor nullifies.

Proof. Assume that R fulfils the antecedents of the theorem.

Let $\theta'_1, \dots, \theta'_{k'} : R \rightarrow \mathbb{R}$ be the real root functions describing the graph of $\prod_{p \in P} p$ on R . By construction, for every $j' \in [k']$ there exists $j \in [k]$ such that $\theta_j := \theta_{\xi_j, s} = \theta'_{j'}$ on R . Now, either a factor is required to be sign-invariant on R (e.g. it is a lower-level factor or not covered by $(\approx, \prec)$ by choice) or is contained in Q (thus covered by $(\approx, \prec)$). Factors of the first kind are not relevant for local delineability.

By analysis of each case, the real root functions of P defined at s are in the shape required by Definition 4.5: $\theta_\ell, \dots, \theta_{\ell'}$ are the root functions matching Definition 4.5 contained in some $R'' \subseteq \mathbb{R}^{i+1}$ such that $R'' \downarrow_{[i]} = R$. Condition 1 ensures that they do not intersect. The functions θ_ℓ and $\theta_{\ell'}$ bound R'' ; Conditions 4 and 5 ensure that all other root functions $\theta_1, \dots, \theta_{\ell-1}, \theta_{\ell'+1}, \dots, \theta_k$ ($\theta_1, \dots, \theta_k$ if $\ell > \ell'$) do not enter R'' . The set $R' \in \text{Ext}(R)$ such that $\text{repr}(\mathbf{I}, s)$ holds on R' is described by the root functions corresponding to $\mathbf{I}$; Condition 2 and 3 ensure that $R' \subseteq R''$. Thus, the requirements from Definition 4.5 are fulfilled, and P is analytically locally R' -delineable.

The factors $q \notin Q$ are all order-invariant on R by assumption. The factors $q \in Q$ are order-invariant in each of their respective sections as $\text{an_del}(q)$ holds on R . $\square$

4.5.2 Root Orderings

The above rule only gives the conditions that need to hold for local delineability. Now, we formalize which roots are irrelevant, and what needs to hold in the underlying projection. In the examples above, we hinted that we can only distinguish relevant from irrelevant intersections of root functions where the defining polynomials are delineable, as we can only compute values of indexed roots; thus we define:

Definition 4.8

Let $i \in \mathbb{N}$, $P \subseteq \mathbb{Q}[x_1, \dots, x_{i+1}]$, and $s \in \mathbb{R}^i$.

For $p \in P$, the *delineating polynomials of P w.r.t. s* is the set $\text{proj_polys}(p)$ of polynomials of level i that need to be sign-invariant to maintain the property $\text{an_del}(p)$ (i.e. the discriminant, leading coefficient and some non-zero coefficient for non-nullification).

Let

$$\begin{aligned}\rho_l &= \max\{\rho \in \text{realRoots}(\cup_{p \in P, q \in \text{proj_polys}(p)} q(s_{[i-1]})) \mid \rho \leq s_i\}, \\ \rho_u &= \min\{\rho \in \text{realRoots}(\cup_{p \in P, q \in \text{proj_polys}(p)} q(s_{[i-1]})) \mid \rho \geq s_i\}.\end{aligned}$$

The *delineable interval* of P w.r.t. s is defined as $\text{del_int}(P, s) = (\rho_l, \rho_u)$ if $\rho_l \neq \rho_u$ and $\text{del_int}(P, s) = [\rho_l, \rho_u]$ otherwise.

Rule 4.22

Let $i \in \mathbb{N}$, $s \in \mathbb{R}^i$, $(\approx, \prec)$ be an indexed root ordering of level $i + 1$, I be a symbolic interval of level i , and $(\approx', \prec')$ be an indexed root ordering of level i .

Assume that $\xi.p$ is irreducible for all $\xi \in \text{dom}(\approx, \prec)$, $(\approx, \prec)$ matches s , and $(\approx', \prec')$ matches $s_{[i-1]}$.

Assume that for every $(\xi_1, \xi_2) \in \prec$, $\xi_1.p \neq \xi_2.p$, the following holds: Let $P = \{p \in \text{factors}(\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p)) \mid \text{ire}(p, s) \subseteq \text{dom}(\approx', \prec')\}$ and $\text{ire}(P, s_{[i-1]}) = \{\xi'_1, \dots, \xi'_k\}$ s.t. $\xi'_1(s_{[i-1]}) \leq \dots \leq \xi'_k(s_{[i-1]})$. For every $j \in [k]$, if

- $\xi'_j(s_{[i-1]}) \notin \text{del_int}(\{\xi_1.p, \xi_2.p\}, s)$, or
- $s' := \xi'_j(s_{[i-1]}) \in \text{del_int}(\{\xi_1.p, \xi_2.p\}, s)$ and $\xi_1(s_{[i-1]}, s') = \xi_2(s_{[i-1]}, s')$,

then it holds

- if $I = (I.l, I.u)$, then $I.l \neq -\infty$ and $\xi'_j(=\approx', \prec' \cup <\approx', \prec')I.l$ or $I.u \neq \infty$ and $I.u(=\approx', \prec' \cup <\approx', \prec')\xi'_j$;
- if $I = [I.l, I.u]$, then $\xi'_j <\approx', \prec' I.l$ or $I.u <\approx', \prec' \xi'_j$.

Assume that for every $(\xi_1, \xi_2) \in \approx$, $\xi_1.p \neq \xi_2.p$, the following holds: Let $P = \{p \in \text{factors}(\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p)) \mid \text{ire}(p, s) \subseteq \text{dom}(\approx', \prec')\}$ and $\text{ire}(P, s_{[i-1]}) = \{\xi'_1, \dots, \xi'_k\}$ s.t. $\xi'_1(s_{[i-1]}) \leq \dots \leq \xi'_k(s_{[i-1]})$. For every $j \in [k]$, if $s_i = \xi'_j(s_{[i-1]})$, then it holds $\xi'_j \approx', \prec' I.l = I.u$.

For each $\xi_1, \xi_2 \in \text{dom}(\approx, \prec)$, let $Q_{\xi_1, \xi_2} := \{\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p), \text{proj_polys}(\xi_1.p), \text{proj_polys}(\xi_2.p)\}$.

$$\begin{aligned}& \text{sample}(s)(R), \text{connected}(i)(R), \\& \text{ir_ord}((\approx', \prec'), s_{[i-1]})(R), \text{repr}(I, s_{[i-1]})(R), \\& \forall \xi \in \text{dom}(\approx, \prec). \text{an_del}(\xi.p)(R), \\& \forall (\xi_1, \xi_2) \in (\approx \cup \prec). \xi_1.p \neq \xi_2.p \rightarrow \text{an_loc_del}(Q_{\xi_1, \xi_2}, I, s)(R) \vdash \text{ir_ord}((\approx, \prec), s)(R)\end{aligned}$$

Note that the case $\xi_1(s) = \xi_2(s)$ is a degenerate case (the resultant is zero at s due to the intersection). Note furthermore that for the case $\xi_1(s) < \xi_2(s)$, checking the second bullet point requires lifting over each (possibly non-rational) root of the resultant, which might be computationally expensive. We emphasize that in an actual implementation, no complete check needs to be implemented. We could e.g. restrict ourselves to rational sample points, or employ other efficient incomplete methods.

Proof. Assume that R fulfils the antecedents of the rule.

Let $(\xi_1, \xi_2) \in \prec$ such that $\xi_1.p \neq \xi_2.p$. We show that $ir_ord((\emptyset, \{(\xi_1, \xi_2)\}), s)$ holds on R . The generalization is then analogous to the proof of Rule 4.9.

To simplify notation, let $\theta_1 = \theta_{\xi_1, s}$, $\theta_2 = \theta_{\xi_2, s}$, $\theta'_j = \theta_{\xi'_j, s}$ for $j \in [k]$, $p_1 = \xi_1.p$, and $p_2 = \xi_2.p$.

Assume for contradiction that there exists $t \in R$ such that $\theta_1(t) \not\prec \theta_2(t)$. As $\theta_1(s) < \theta_2(s)$, there exists some $t' \in R$ such that $\theta_1(t') = \theta_2(t')$ which is a zero of $\text{res}_{x_{i+1}}(p_1, p_2)$, thus there exists $j^* \in [k]$ s.t. $\theta'_{j^*}(t'_{[i-1]}) = t'_i$ (i.e. $t' \in R_{\downarrow[i-1]} \times \theta'_{j^*}$).

Let $R' \supseteq R$, $R'_{\downarrow[i-1]} = R_{\downarrow[i-1]}$ be maximal such that $P = \{\text{res}_{x_{i+1}}(p_1, p_2), \text{proj_polys}(p_1), \text{proj_polys}(p_2)\}$ is analytically R' -locally delineable on $R_{\downarrow[i-1]}$, and p_1 and p_2 are analytically delineable on R' , and P is order-invariant in each of its sections on $R_{\downarrow[i-1]}$ that intersect with R' . Note that as we included the proj_polys sets into the set of polynomials to be locally delineable, for each $j \in [k]$ it holds that $(R_{\downarrow[i-1]} \times \theta'_j) \cap R \neq \emptyset$ implies $R_{\downarrow[i-1]} \times \theta'_j \subseteq R'$.

Now let $R'_1, \dots, R'_{k'} \subseteq R'$ be the maximal sign-invariant cells of $\text{res}_{x_{i+1}}(p_1, p_2)$ in R' (which are arranged cylindrically, that is for all $j \in [k']$ it holds $R'_{j \downarrow[i-1]} = R_{\downarrow[i-1]}$); $\text{res}_{x_{i+1}}(p_1, p_2)$ is order-invariant in each of these cells by assumption. By Lemma 4.1, for each $j \in [k']$ there exists $\sim_j$ such that $\theta_1 \sim_j \theta_2$ on R'_j .

Towards the contradiction, we conclude that θ'_{j^*} is one of the sections contained in R' (i.e. there exists $j \in [k']$ such that $R_{\downarrow[i-1]} \times \theta'_{j^*} = R'_j$). By construction, it holds $(\{s_{[i-1]}\} \times \text{del_int}(\{p_1, p_2\}, s)) \cap R'_j = \{(s_{[i-1]}, s')\}$ for some $s' \in R'$ such that $s' = \theta'_{j^*}(s_{[i-1]})$. By requirement of the theorem, θ'_{j^*} is required to “remain outside” I on $R_{\downarrow[i-1]}$, that is $(R_{\downarrow[i-1]} \times \theta'_{j^*}) \cap R = \emptyset$. This is a contradiction to the assumption which implied $t' \in (R_{\downarrow[i-1]} \times \theta'_{j^*}) \cap R$. $\square$

4.5.3 Adaptation of the Algorithm and Heuristic Choices

The presented proof rules lead $ir_ord((\approx, \prec), s)$ back to lower-level root ordering $(\approx', \prec')$, without going via the $sgn_inv(p)$ property. Thus, determining $(\approx', \prec')$ is certainly less straight-forward than as described in Section 4.2.7.1.

The idea is the following: For each pair in $\approx \cup \prec$, we compute and factorize the resultant. We then iterate through its roots and check for its relevance. We store all roots along with a flag indicating its relevance, and the associated polynomial pair; we also flag the projection polynomials (discriminants and coefficients) maintaining the delineability of the polynomial pair with the pair. After we obtained the set of roots from all such pairs and other polynomials whose sign-invariance is to be maintained, we are able to sort them, and choose a symbolic interval I : Now, if there is an irrelevant root, we can choose that this root is contained in I (of course, we do not have to). Afterwards, we can choose an ordering $(\approx', \prec')$ that satisfies the requirements from the above rules: Roughly, that is, the roots outside the interval remain outside the interval, and the roots within the interval that are flagged with the same polynomial pair need to stay in the same order.

Note that the set of computed resultants is in general not comparable (none is a subset of the other) with the one computed in the vanilla approach: For maintaining local delineability, we may compute resultants between polynomials with roots inside I that we would not compute otherwise. We thus emphasize that we do not need to choose I to be maximal, but could limit ourselves to cases as described in Example 4.6 which

strictly reduce the amount of projection polynomials.

4.5.4 Future Work

The rules presented in this section only check for relevance of roots of *resultants*. It is unclear whether similar ideas may be transferred to roots of *discriminants*, in order to address the limitations described in Section 4.2.10.5. This seems at least highly challenging, because at the roots of a discriminant, polynomials might have self-intersections and “turning points”, and thus, discriminants are a central tool to reason about well-defined root functions. The latter is crucial to generalize from a single root to a larger cell.

4.6 Weak Orderings and Closing Interval Bounds

So far, we did only consider root functions θ_1, θ_2 that are either strictly ordered or equal on some underlying cell in the previous sections. We will now consider root functions that are ordered less-or-equal.

To motivate this, assume we want to construct a cell where some constraint $p \leq 0$ holds. Rule 4.10 tells us that we can construct a cell around some sample s such that $p(s) \leq 0$ where p is sign-invariant by requiring that every root of p remains outside the cell that we describe. However, this yields a smaller cell guaranteeing the stronger property $p < 0$. Similarly, as in the previous section, we could merge this cell with the adjacent cells where $p = 0$ holds (again, as long as we cut these cells such that they are in the same cylinder).

This merger might not only produce bigger cells, it also avoids lifting over roots of polynomials, which is particularly expensive as non-rational number computations cannot be avoided in this case.

To describe such cells, we use symbolic intervals that are closed or half-closed. For constructing such a cell, we will (1) generalize the rules from the previous section for half-closed intervals and weak orderings, and (2) define how we determine a symbolic interval that describes the cell as general as possible.

4.6.1 Semi Sign Invariance

Semi sign invariance is the property that allows a polynomial to be *greater-or-equal* or *less-or-equal* to zero, instead of only strictly greater, strictly less, or equal to zero. We define the property as well as some straight-forward rules.

Property 4.8

Let $i \in \mathbb{N}$, and $R \subseteq \mathbb{R}^i$.

Let $p \in \mathbb{Q}[x_1, \dots, x_i]$ be a polynomial of level i . The property *semi_sgn_inv*(p) holds on R if and only if $p \leq 0$, or $p \geq 0$ on R .

References

semi_sgn_inv(p) Rule 4.23, Rule 4.24, Rule 4.25, Rule 4.27

Rule 4.23

Let $p \in \mathbb{Q}$.

$$\vdash \text{semi_sgn_inv}(p)(\mathbb{R}^0)$$

Rule 4.24

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is reducible, and $\text{factors}(p) = \{q_1, \dots, q_k\}$.

$$\text{semi_sgn_inv}(q_1)(R), \dots, \text{semi_sgn_inv}(q_k)(R) \quad \vdash \text{semi_sgn_inv}(p)(R)$$

Rule 4.25

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$. Assume that p is irreducible.

$$\text{sgn_inv}(p)(R) \quad \vdash \text{semi_sgn_inv}(p)(R)$$

Any meaningful implementation would apply the previous rule only in the case $p(s) = 0$.

4.6.2 Root Orderings

To maintain semi sign invariance, we need to generalize indexed root orderings for weak orderings that allow roots to intersect without crossing:

Definition 4.9 (Weak Indexed Root Ordering)

Let $i \in \mathbb{N}$, and Ξ be a set of indexed root expressions of level $i + 1$.

We define the following relations w.r.t. a tuple of relations $(\preceq, \prec)$, $\preceq, \prec \subseteq \Xi \times \Xi$:

$$\begin{aligned} =_{\preceq, \prec} &= \{(\xi, \xi') \mid \xi \preceq^t \xi' \wedge \xi' \preceq^t \xi\} \\ <_{\preceq, \prec} &= \{(\xi, \xi') \mid \text{there ex. } \sim_1, \dots, \sim_{k-1} \in \{\prec, \preceq\}, \xi_1, \dots, \xi_k \text{ s.t.} \\ &\quad \xi = \xi_1 \sim_1 \dots \sim_{k-1} \xi_k = \xi' \text{ and } \prec \in \{\sim_1, \dots, \sim_{k-1}\}\} \\ \leq_{\preceq, \prec} &= \{(\xi, \xi') \mid \xi(\preceq \cup \prec)^t \xi'\} \cup \{(\xi, \xi) \mid \xi \in \text{dom}(\preceq, \prec)\} \end{aligned}$$

A *weak indexed root ordering* on Ξ is a tuple of relations $(\preceq, \prec)$ such that $\preceq, \prec \subseteq \Xi \times \Xi$, $=_{\preceq, \prec}$ is an equivalence relation on Ξ , $\leq_{\preceq, \prec}$ is a partial order on Ξ , and $<_{\preceq, \prec}$ is a strict partial order on Ξ . Indexed root orderings of this form are also called *weak indexed root ordering of level $i + 1$* , and we define $\text{dom}(\preceq, \prec) = \text{dom}(\preceq) \cup \text{dom}(\prec)$. Let $s \in \mathbb{R}^i$. A weak indexed root ordering $(\preceq, \prec)$ of level $i + 1$ *matches* s if and only if $\xi \in \text{ire}(\xi.p, s)$ for all $\xi \in \text{dom}(\preceq, \prec)$, $\xi(s) = \xi'(s)$ for all $(\xi, \xi') \in =_{\preceq, \prec}$, $\xi(s) \leq \xi'(s)$ for all $(\xi, \xi') \in \leq_{\preceq, \prec}$, and $\xi(s) < \xi'(s)$ for all $(\xi, \xi') \in <_{\preceq, \prec}$.

Property 4.9

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $(\preceq, \prec)$ be a weak indexed root ordering of level $i + 1$. The property $ir_ord((\preceq, \prec), s)$ holds on R if and only if $(\preceq, \prec)$ matches s , for all $\xi \in \text{dom}(\preceq, \prec)$ it holds $R \subseteq \text{dom}(\theta_{\xi, s})$, and for all $\xi, \xi' \in \text{dom}(\preceq, \prec)$ it holds that

$$\xi \leq_{\preceq, \prec} \xi' \Rightarrow \theta_{\xi, s}(r) \leq \theta_{\xi', s}(r) \text{ for all } r \in R$$

and

$$\xi <_{\preceq, \prec} \xi' \Rightarrow \theta_{\xi, s}(r) < \theta_{\xi', s}(r) \text{ for all } r \in R$$

and

$$\xi =_{\preceq, \prec} \xi' \Rightarrow \theta_{\xi, s}(r) = \theta_{\xi', s}(r) \text{ for all } r \in R$$

(for the real root functions $\theta_{\xi, s}$, $\theta_{\xi', s}$ according to Definition 4.3).

References

$ir_ord((\preceq, \prec), s)$ Rule 4.28

For convenience, we introduce the following notation:

Definition 4.10

Let $i \in \mathbb{N}$, I be a symbolic interval of level i , $(\preceq, \prec)$ be an indexed root ordering on Ξ of level $i - 1$, and $\xi \in \Xi$.

The root ξ is *strictly outside* I w.r.t. $(\preceq, \prec)$ if and only if the following hold:

► If $I = (l, u)$, then either

$$l \neq -\infty \text{ and } \xi \leq_{\preceq, \prec} l, \text{ or } u \neq \infty \text{ and } u \leq_{\preceq, \prec} \xi.$$

► If $I = [l, u]$, then either

$$l \neq -\infty \text{ and } \xi <_{\preceq, \prec} l, \text{ or } u \neq \infty \text{ and } u <_{\preceq, \prec} \xi.$$

► The other cases are analogous.

The root ξ is *weakly outside* I w.r.t. $(\preceq, \prec)$ if and only if the following holds:

$$l \neq -\infty \text{ and } \xi \leq_{\preceq, \prec} l, \text{ or } u \neq \infty \text{ and } u \leq_{\preceq, \prec} \xi.$$

4.6.3 Maintaining (Semi-)Sign-Invariance

We are now able to define rules which allow proving semi sign invariance using weak indexed root orderings. We first rephrase Rule 4.10:

Rule 4.26

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, I be a symbolic interval of level i , and $(\preceq, \prec)$ be a weak indexed root ordering of level i .

Assume that p is irreducible, $\text{ire}(p, s) \neq \emptyset$, $\xi.p$ is irreducible for all $\xi \in \text{dom}(\preceq, \prec)$, $(\preceq, \prec)$ matches s , and for all $\xi \in \text{ire}(p, s)$ it holds that ξ is strictly outside $\mathbf{I}$ w.r.t. $(\preceq, \prec)$.

$$\begin{aligned} & \text{sample}(s)(R), \text{repr}(\mathbf{I}, s_{[i-1]})(R), \\ & \text{ir_ord}((\preceq, \prec), s)(R), \text{an_del}(p)(R) \quad \vdash \text{sgn_inv}(p)(R) \end{aligned}$$

This can be generalized to semi sign invariance in a straight-forward way:

Rule 4.27

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, $\mathbf{I}$ be a symbolic interval of level i , and $(\preceq, \prec)$ be a weak indexed root ordering of level i .

Assume that p is irreducible, $\text{ire}(p, s) \neq \emptyset$, $\xi.p$ is irreducible for all $\xi \in \text{dom}(\preceq, \prec)$, $(\preceq, \prec)$ matches s , and for all $\xi \in \text{ire}(p, s)$ it holds that ξ is weakly outside $\mathbf{I}$ w.r.t. $(\preceq, \prec)$.

$$\begin{aligned} & \text{sample}(s)(R), \text{repr}(\mathbf{I}, s_{[i-1]})(R), \\ & \text{ir_ord}((\preceq, \prec), s)(R), \text{an_del}(p)(R) \quad \vdash \text{semi_sgn_inv}(p)(R) \end{aligned}$$

4.6.4 Maintaining Root Orderings

Finally, we get to the part where the main magic happens: We handle weak orderings of two root functions when reducing them to a lower-level ordering. We thus extend Rule 4.22 as follows:

Rule 4.28

Let $i \in \mathbb{N}$, $s \in \mathbb{R}^i$, $(\preceq, \prec)$ be a weak indexed root ordering of level $i + 1$, $\mathbf{I}$ be a symbolic interval of level i , and $(\preceq', \prec')$ be a weak indexed root ordering of level i . Assume that $\xi.p$ is irreducible for all $\xi \in \text{dom}(\preceq, \prec)$, $(\preceq, \prec)$ matches s , and $(\preceq', \prec')$ matches $s_{[i-1]}$.

Assume that for every $(\xi_1, \xi_2) \in (\preceq \cup \prec)$, $\xi_1.p \neq \xi_2.p$, the following holds: Let $P = \{p \in \text{factors}(\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p)) \mid \text{ire}(p, s) \subseteq \text{dom}(\preceq', \prec')\}$ and $\text{ire}(P, s_{[i-1]}) = \{\xi'_1, \dots, \xi'_k\}$ s.t. $\xi'_1(s_{[i-1]}) \leq \dots \leq \xi'_k(s_{[i-1]})$. For every $j \in [k]$, if

- $\xi'_j(s_{[i-1]}) \notin \text{del_int}(\{\xi_1.p, \xi_2.p\}, s)$, or
- $s' := \xi'_j(s_{[i-1]}) \in \text{del_int}(\{\xi_1.p, \xi_2.p\}, s)$ and $\xi_1(s_{[i-1]}, s') = \xi_2(s_{[i-1]}, s')$,

it holds either

- ξ'_j is strictly outside $\mathbf{I}$ w.r.t. $(\preceq', \prec')$ if $\xi_1 \prec \xi_2$, or
- ξ'_j is weakly outside $\mathbf{I}$ w.r.t. $(\preceq', \prec')$ otherwise.

For each $\xi_1, \xi_2 \in \text{dom}(\preceq, \prec)$, let $Q_{\xi_1, \xi_2} := \{\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p), \text{proj_polys}(\xi_1.p), \text{proj_polys}(\xi_2.p)\}$.

$$\begin{aligned}
& \text{sample}(s)(R), \text{ connected}(i)(R), \\
& \text{ir_ord}((\preceq', \prec'), s_{[i-1]})(R), \text{ repr}(\mathbf{I}, s_{[i-1]})(R), \\
& \forall \xi \in \text{dom}(\preceq, \prec). \text{ an_del}(\xi.p)(R), \\
& \forall (\xi_1, \xi_2) \in (\preceq \cup \prec). \xi_1.p \neq \xi_2.p \rightarrow \text{ an_loc_del}(Q_{\xi_1, \xi_2}, \mathbf{I}, s)(R) \vdash \text{ ir_ord}((\preceq, \prec), s)(R)
\end{aligned}$$

Proof. Analogous to Rule 4.22. □

Compared to Rule 4.22, the above rule takes into account whether two root functions described by ξ_1 and ξ_2 are required to be strictly ordered ($\xi_1 \prec \xi_2$) or are allowed to intersect but not cross ($\xi_1 \preceq \xi_2$). In the former case, any root of the resultant of their defining polynomials is required to remain strictly outside the interval on the level below, whereas in the latter case, such roots are allowed to intersect the root function that defines a bound on that interval.

4.6.5 Adaptation of the Algorithm and Heuristic Choices

We need to alter the way we choose the symbolic interval, similarly to the previous section: Throughout the algorithm, we maintain sets of polynomials that need to be made (semi-)sign-invariant or order-invariant and orderings of root functions respectively. These properties define *critical roots* at which their satisfaction might change, which are flagged as *relevant* (root must not be inside the interval), *weakly relevant* (root might intersect with a closed bound of the interval) or *irrelevant* (root might be inside the cell, see previous section). After we computed and classified these roots according to e.g. Rules 4.27 and 4.28, we are able to pick an admissible symbolic interval.

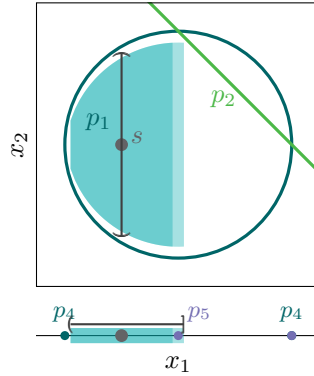


Figure 4.16: The cell from Example 4.7 where two polynomials are sign-invariant can be made right-closed. The darker cell depicts the cell in traditional single cell construction, and the lighter cell depicts the bigger cell if we allow closed interval bounds.

Example 4.7

Consider the polynomials $p_1 = x_1^2 + x_2^2 - 1$ and $p_2 = -1 + x_1 + x_2$, and assume we construct the sign-invariant cell around $s = (-0.5, 0)$. Figure 4.16 depicts this example where a cell is constructed which is sign-invariant for p_1 and p_2 . As the interval for x_2 has open bounds defined by p_1 , the intersection point of p_1 and p_2 will

be outside that interval, even if we extend the underlying cell in that direction. Thus, the root of the resultant p_5 is weakly relevant, and we can close the upper bound on x_1 . We note that we cannot close the other bound for the reason that the indexed roots of the defining polynomials for the interval of x_2 need to be well-defined on the underlying cell.

4.6.6 Maintaining Delineability on Cylindrical Cells

The described approach is not fully supported by the proof system yet: As we allow closed bounds of the symbolic intervals, the set they describe is not an analytic submanifold. As in particular the rule for maintaining delineability requires an analytic submanifold, the application of the proof rules run into a dead end. However, this is just a mathematical subtlety as we show in the following.

To address the issue, instead of an analytic submanifold, we require an *analytically-bounded cylindrical cell*, which is a locally cylindrical cell bounded by analytic functions.

Property 4.10

Let $i \in \mathbb{N}$, and $R \subseteq \mathbb{R}^i$.

The property $an_cyl_cell(i)$ holds on R if and only if R is a *analytically-bounded cylindrical cell*, that is,

- $i = 1$ and R is equal to an interval $(-\infty, \infty)$, $(-\infty, f)$, (f, ∞) , (f, g) , $(-\infty, f]$, $[f, \infty)$, $[f, g)$, $(f, g]$, or $[f, g]$ for $f, g \in \mathbb{R}$; or
- $i > 1$ and $R \downarrow_{[i-1]}$ is locally cylindrical and R is equal to a set of the form $R \downarrow_{[i-1]} \times (-\infty, \infty)$, $R \downarrow_{[i-1]} \times (-\infty, f)$, $R \downarrow_{[i-1]} \times (f, \infty)$, $R \downarrow_{[i-1]} \times (f, g)$, $R \downarrow_{[i-1]} \times (-\infty, f]$, $R \downarrow_{[i-1]} \times [f, \infty)$, $R \downarrow_{[i-1]} \times [f, g)$, $R \downarrow_{[i-1]} \times (f, g]$, or $R \downarrow_{[i-1]} \times [f, g]$ for analytic functions $f, g : S \rightarrow \mathbb{R}$ such that $f < g$ where $S \supseteq R \downarrow_{[i-1]}$ is an analytically-bounded cylindrical cell where all bounds are open except for point intervals.

References

$an_cyl_cell(i)$ Rule 4.30

Analytic delineability is only defined on analytic submanifolds. We thus modify the definition of the $an_del(p)$ property to hold on a cylindrical cell if there exist (possibly infinitely many) analytic submanifolds that cover the cell. One of these analytic submanifolds is the interior of the cylindrical cell, the remaining ones are sufficiently many neighbourhoods of points on the boundary of the cell, which also contain points outside the given cell.

Property 4.11

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ be a polynomial of level $i + 1$.

The property $an_del(p)$ holds on R if and only if $S = \text{int}(R)$ is a connected analytic submanifold, and there is some set $\mathcal{S}$ of connected analytic submanifolds of the same dimension as S such that $S \subseteq R \subseteq \text{cl}(S)$, $R \subseteq S \cup \bigcup_{S' \in \mathcal{S}} S'$, for each $S' \in \mathcal{S}$ it holds $S \cap S' \neq \emptyset$, and for each $S' \in \mathcal{S} \cup \{S\}$ it holds that p is analytically delineable on S'

and p is order-invariant in all p -sections over S' .

The property may seem odd on first sight. However, note that as the analytic submanifolds overlap and the given polynomial is analytically delineable on each of them, the witnessing root functions must coincide on all analytic submanifolds. Further, we can define a meaningful notion of an analytic function on unions of analytic submanifolds. By a straight-forward generalization of Lemma 4.1, all the proof rules can be adapted towards the new notion. However, for reasons of readability, we do not apply this step in the proofs explicitly.

The proof of the following rule shows that we can find such sets of analytic submanifolds (in particular the neighbourhoods of points on the boundary) due to the way polynomial roots behave.

Rule 4.29

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$. Assume that p is irreducible.

$$\begin{aligned} & \text{an_cyl_cell}(i)(R), \text{connected}(i)(R), \text{non_null}(p)(R), \\ & \text{ord_inv}(\text{disc}_{x_{i+1}}(p))(R), \text{sgn_inv}(\text{lclcf}_{x_{i+1}}(p))(R) \quad \vdash \text{an_del}(p)(R) \end{aligned}$$

Proof. Let $S \subseteq R$ be the connected analytic submanifold that we obtain by changing all closed bounds on R (except the “point” intervals) to open bounds using [McC85, Theorem 2.2.3 and Theorem 2.2.4] (for which we need the local cylindricity of R). Let $T = T_{\downarrow[i-1]} \times \mathbb{R}$ such that $T_{\downarrow[i-1]} \supseteq R_{\downarrow[i-1]} \supseteq S_{\downarrow[i-1]}$ be the connected analytic submanifold that we obtain by taking the intersection of the locally cylindrical cells which are the domains of the analytic functions bounding R (note that the intersection is locally cylindrical and thus an analytic submanifold). Observe that T has the same dimension as S .

S is already the required “main” analytic submanifold. For constructing the (possibly infinite) set $\mathcal{S}$, we use the following two observations.

- O1 If some polynomial $q \in \mathbb{Q}[x_1, \dots, x_i]$ is non-zero at some point $r \in \mathbb{R}^i$, then there is some neighbourhood $N \subseteq \mathbb{R}^i$ of r on which q is non-zero.

Proof: This follows by the continuity of $q : \mathbb{R}^i \rightarrow \mathbb{R}$.

- O2 If some polynomial $q \in \mathbb{Q}[x_1, \dots, x_i]$ is zero at some point $r \in \text{cl}(S)$ and zero on S , then there is some connected analytic submanifold $N \subseteq T$ of the same dimension as S containing r on which q is zero.

Proof: By [McC85, Theorem 2.2.1 and the following remark], as T is an analytic submanifold of dimension $s \in [0..i]$, for every $r \in T$, there exists a neighbourhood N of r , an open $V \subseteq \mathbb{R}^i$ and a coordinate system $\Phi : N \rightarrow V$ about r such that $\Phi(T \cap N) = \{r \in \mathbb{R}^i \mid r_{s+1} = \dots = r_i = 0\} \cap V$. Further, $q : \mathbb{R}^i \rightarrow \mathbb{R}$ is an analytic function, thus also the composition $q \circ (\Phi_1^{-1}, \dots, \Phi_s^{-1}) : W \rightarrow \mathbb{R}$ where $W = (\Phi_1, \dots, \Phi_s)(T \cap N) \subseteq \mathbb{R}^i$ is an analytic function by [McC85, Theorem 2.2.2]. By assumption, $S \cap N \neq \emptyset$ is an analytic submanifold where q is zero. Thus, $(\Phi_1, \dots, \Phi_s)(S \cap N) \subseteq W$ is an open set where $q \circ (\Phi_1^{-1}, \dots, \Phi_s^{-1})$ is zero. By the

following claim (or alternatively by the identity theorem), $q \circ (\Phi_1^{-1}, \dots, \Phi_s^{-1})$ is zero on W . Thus, q must be zero on $T \cap N = (\Phi_1^{-1}, \dots, \Phi_s^{-1})(W)$.

Claim: Let $O \subseteq \mathbb{R}^i$ be open, $f : O \rightarrow \mathbb{R}$ be analytic, and $P \subseteq O$ be open. If f is zero on P , then f is zero on O .

Proof: All partial derivatives of any order of f exist and f is continuous. As f is zero on P , there is a point in P where all partial derivatives of f are zero. Thus, all partial derivatives of f are zero on O , and in particular, f is zero on O .

We now build the set $\mathcal{S}$. For any $r \in R \cap \text{cl}(S)$, we determine a connected analytic submanifold S' such that p is not nullified on any point in S' , $\text{ldcf}_{x_{i+1}}(p)$ is sign-invariant on S' , and $\text{disc}_{x_{i+1}}(p)$ is order-invariant on S' as follows.

- The polynomial p is non-nullified on R by assumption. There is some neighbourhood N of r where p is non-nullified by applying O1 to the coefficients of p . The set $N \cap T$ is a connected analytic submanifold of the same dimension as T .
- If the sign of $\text{ldcf}_{x_{i+1}}(p)$ is non-zero on R , there is some neighbourhood N where $\text{ldcf}_{x_{i+1}}(p)$ is non-zero by O1. The set $N \cap T$ is a connected analytic submanifold of the same dimension as T .
If the sign of $\text{ldcf}_{x_{i+1}}(p)$ is zero on R , there is some connected analytic submanifold $N \subseteq T$ of the same dimension as T where $\text{ldcf}_{x_{i+1}}(p)$ is zero by O2.
- Let t be the order of $\text{disc}_{x_{i+1}}(p)$ on R . All partial derivatives of order less than t are zero throughout R , thus by O2, there is some connected analytic submanifold N where they are zero. Further, there is some partial derivative of order t which is non-zero at r ; thus, by O1, there is some partial derivative of order t which is non-zero in some neighbourhood N , and $N \cap T$ is a connected analytic submanifold.

It is obvious that we can choose a connected analytic submanifold $S' \subseteq T$ of the same dimension as S which fulfils each of the above items.

Finally, by connectedness, irreducibility of p , [Bro01, Theorem 3.1] and [McC98, Theorem 2], for each $S' \in \mathcal{S} \cup \{S\}$, p is analytically delineable on S' and p is order-invariant in all p -sections over S' . $\square$

Rule 4.30

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and $\mathbf{I}$ be a symbolic interval of level i .

$$\begin{aligned} & \vdash \text{an_cyl_cell}(0)(\mathbb{R}^0) \\ \text{repr}(\mathbf{I}, s)(R), \text{an_cyl_cell}(i-1)(R), \text{connected}(i)(R), \\ \text{non_empty}(\mathbf{I}, s)(R), \text{an_del}(\mathbf{I}.l)(R), \text{an_del}(\mathbf{I}.u)(R) & \vdash \text{an_cyl_cell}(i)(R) \end{aligned}$$

Proof. The property immediately follows from the definition of $\text{repr}(\mathbf{I}, s)(R)$. Here, we briefly justify that f and g are analytic and defined on some cylindrical cell $S \supseteq R \downarrow_{[i-1]}$ which is open-bounded (except for sections). The functions f and g are witnessed by $\mathbf{I}.l$ and $\mathbf{I}.u$ respectively, which are analytic as their defining polynomials are analytically delineable on some set $S' \supseteq R \downarrow_{[i-1]}$. It is easy to see that one may construct some cylindrical cell $S \subseteq S'$ and $S \supseteq R \downarrow_{[i-1]}$. $\square$

4.6.7 Local Delineability

It follows an adaptation of Rule 4.21 which exploits the possibility of two roots being weakly ordered.

Rule 4.31

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(\prod_{p \in P} p) = i + 1$, I be a symbolic interval of level $i + 1$, and $(\preceq, \prec)$ be a weak indexed root ordering of level $i + 1$. Let $Q = \{q \in \text{factors}(P) \mid \text{ire}(q, s) \subseteq \text{dom}(\preceq, \prec)\}$ and $\text{ire}(Q, s) = \{\xi_1, \dots, \xi_k\}$ s.t. $\xi_1(s) \leq \dots \leq \xi_k(s)$.

Assume that $(\preceq, \prec)$ matches s , and that I and $(\preceq, \prec)$ fulfil that there exist $\ell, \ell' \in [k]$ (ℓ' might be smaller than ℓ meaning that we select the empty range) such that:

1. For all $j \in [\ell.. \ell' - 1]$ it holds $\xi_j(=\preceq, \prec \cup <\preceq, \prec) \xi_{j+1}$.
2. For all $j \in [1..k] \setminus [\ell.. \ell']$ it holds
 - ξ_j is weakly or strictly outside I w.r.t. $(\preceq, \prec)$,
 - $\xi_j(=\preceq, \prec \cup <\preceq, \prec) I.l$ if $\xi_{j'}(s) = I.l(s)$ for a $j' \in [k]$ and $I.l$ is closed, and
 - $I.u(=\preceq, \prec \cup <\preceq, \prec) \xi_j$ if $\xi_{j'}(s) = I.u(s)$ for a $j' \in [k]$ and $I.u$ is closed.
3. For all $j \in [1.. \ell - 1]$ it holds $\xi_j(=\preceq, \prec \cup <\preceq, \prec) \xi_\ell$.
4. For all $j \in [\ell' + 1..k]$ it holds $\xi_{\ell'}(=\preceq, \prec \cup <\preceq, \prec) \xi_j$.

$$\begin{aligned}
 & \text{sample}(s)(R), \text{ connected}(i)(R), \text{ an_cyl_cell}(i)(R), \\
 & \text{ ir_ord}((\preceq, \prec), s)(R), \text{ repr}(I, s)(R), \\
 & \forall q \in \text{factors}(P) \setminus Q. \text{ ord_inv}(q)(R), \\
 & \forall q \in Q. \text{ an_del}(q)(R) \vdash \text{ an_loc_del}(P, I, s)(R)
 \end{aligned}$$

Proof. Assume that R fulfils the antecedents of the rule.

The proof is analogous to Rule 4.21, except that some root functions of Q are allowed to intersect the bounds of I . Condition (2) replaces Conditions (2) and (3) of Rule 4.21 (as we defined the “weakly/strictly outside” notion). The second and third bullet points of Condition (2) ensure that if the cell boundary of I is closed and thus intersects R , the root functions of Q that intersect the bounds of I are either equal or disjoint (which is required for local delineability). $\square$

4.6.8 Connectedness

For proving connectedness, we can make use of weak orderings as well when the symbolic interval is not open:

Rule 4.32

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, I be a symbolic interval of level i , and $(\preceq, \prec)$ be an indexed root ordering of level i . Assume that $(\preceq, \prec)$ matches s .

Let $Q := \text{connected}(i-1)(R) \wedge \text{repr}(\mathbf{I}, s)(R) \wedge \text{sample}(s)(R)$.

	$\vdash \text{connected}(0)(\mathbb{R}^0)$
$Q, \mathbf{I}.l \neq -\infty, \mathbf{I}.u \neq \infty, \mathbf{I}.l \leq_{\preceq, \prec} \mathbf{I}.u, \text{ir_ord}((\preceq, \prec), s)(R)$	$\vdash \text{connected}(i)(R)$
$Q, \mathbf{I}.l = -\infty \vee \mathbf{I}.u = \infty$	$\vdash \text{connected}(i)(R)$
$Q, \mathbf{I} = [b, b]$	$\vdash \text{connected}(i)(R)$

4.6.9 Non-emptiness

Similarly, we can adapt the rules for non-emptiness. Note the subtle difference to Rule 4.32.

Rule 4.33

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $\mathbf{I}$ be a symbolic interval of level i , and $(\preceq, \prec)$ be an indexed root ordering of level i . Assume that $(\preceq, \prec)$ matches s .

$\mathbf{I}.l \neq -\infty, \mathbf{I}.u \neq \infty, \mathbf{I}.l <_{\preceq, \prec} \mathbf{I}.u, \text{sample}(s)(R),$ $\text{an_del}(\mathbf{I}.l.p)(R), \text{an_del}(\mathbf{I}.u.p)(R), \text{ir_ord}((\preceq, \prec), s)(R)$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$
$\mathbf{I} = [l, u], \mathbf{I}.l \leq_{\preceq, \prec} \mathbf{I}.u, \text{sample}(s)(R),$ $\text{an_del}(\mathbf{I}.l.p)(R), \text{an_del}(\mathbf{I}.u.p)(R), \text{ir_ord}((\preceq, \prec), s)(R)$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$
$\mathbf{I}.l = -\infty, \mathbf{I}.u \neq \infty, \text{an_del}(\mathbf{I}.u.p)(R)$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$
$\mathbf{I}.l \neq -\infty, \mathbf{I}.u = \infty, \text{an_del}(\mathbf{I}.l.p)(R)$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$
$\mathbf{I}.l = -\infty, \mathbf{I}.u = \infty$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$
$\mathbf{I} = [b, b], \text{an_del}(\mathbf{I}.b.p)(R)$	$\vdash \text{non_empty}(\mathbf{I}, s)(R)$

4.6.10 Coverings

Finally, we relax Rule 4.16 using weak orderings when the intervals forming the covering have closed bounds:

Rule 4.34

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $\mathbf{C} = (\mathbf{I}_1, \dots, \mathbf{I}_k)$ be a sequence of symbolic intervals of level i , and $(\preceq, \prec)$ be an indexed root ordering of level i .

Assume that $\mathbf{C}$ fulfils the following conditions:

- $\mathbf{I}_1.l = -\infty$ and $\mathbf{I}_k.u = \infty$,
- $\mathbf{I}_j.l(s) < \mathbf{I}_{j+1}.l(s) \vee (\mathbf{I}_j.l(s) = \mathbf{I}_{j+1}.l(s) \wedge (\mathbf{I}_j.l \text{ closed} \vee \mathbf{I}_{j+1}.l \text{ open}))$ for $j \in [k-1]$,
- $\mathbf{I}_j.u(s) < \mathbf{I}_{j+1}.u(s) \vee (\mathbf{I}_j.u(s) = \mathbf{I}_{j+1}.u(s) \wedge (\mathbf{I}_j.u \text{ open} \vee \mathbf{I}_{j+1}.u \text{ closed}))$ for $j \in [k-1]$, and
- $\mathbf{I}_{j+1}.l(s) < \mathbf{I}_j.u(s) \vee (\mathbf{I}_{j+1}.l(s) = \mathbf{I}_j.u(s) \wedge (\mathbf{I}_{j+1}.l \text{ closed} \vee \mathbf{I}_j.u \text{ closed}))$ for $j \in [k-1]$.

Assume that $(\preceq, \prec)$ matches s , and for $j \in [k-1]$ it holds either $I_{j+1}.l <_{\preceq, \prec} I_j.u$ if $I_j.u$ and $I_{j+1}.l$ are open or $I_{j+1}.l \leq_{\preceq, \prec} I_j.u$ otherwise.

$$\begin{aligned} & \text{sample}(s)(R), \text{ connected}(i)(R), \text{ ir_ord}((\preceq, \prec), s)(R), \\ & \forall j \in [k-1]. (\text{an_del}(I_j.u.p)(R) \wedge \text{an_del}(I_{j+1}.l.p)(R)) \quad \vdash \text{cov}(\mathbf{C}, s)(R) \end{aligned}$$

Proof. Analogous to Rule 4.16. □

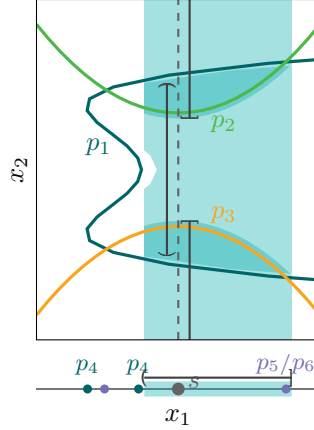


Figure 4.17: Illustration for Example 4.8 where the cylinder over the one dimensional cell is covered by cells where one of the constraints $p_1 \leq 0$, $p_2 < 0$, or $p_3 < 0$ is violated. The symbolic intervals for x_2 and x_1 are depicted. In this case, the interval for x_1 is half-open.

Example 4.8

Figure 4.17 depicts the polynomials $p_1 = x_1 - (5(x_2 - 0.25)(x_2 + 0.25)(x_2 - 0.75)(x_2 + 0.75) - 0.5)$, $p_2 = x_2 - (0.5x_1^2 + 0.5)$ and $p_3 = x_2 - (-0.5x_1^2 - 0.5)$ and the cells where $p_1 \leq 0$, $p_2 < 0$ and $p_3 < 0$ are not satisfied respectively. The symbolic interval's bounds for x_2 for the latter two are closed, and the bounds for the first interval are open.

We characterize each cell separately. In this case, we only need to add the discriminant p_4 of p_1 to maintain its delineability; for the other two cells, we do not need to add anything, as the respective polynomials are trivially delineable on $\mathbb{R}$.

Now, we characterize that the three cells cover the cylinder $R \times \mathbb{R}$ over the cell $R \subseteq \mathbb{R}$ that we are to construct. To do so, we need to maintain that $\text{root}_{x_2}^{p_1,1} \leq \text{root}_{x_2}^{p_2,1}$ on R and $\text{root}_{x_2}^{p_2,1} \leq \text{root}_{x_2}^{p_1,2}$ on R , i.e. that the bounds of the symbolic intervals for x_2 overlap on R . Note that we only need a weak ordering here as the bounds of the lowest and highest intervals are closed.

We construct the common underlying cell: The interval for x_1 is lower bounded by a root of the discriminant p_4 , thus the bound is open. The upper bound stems from the resultants p_5 of p_1 and p_2 and p_6 of p_1 and p_3 . This root is weakly relevant, as both intersections above this point are contained in a non-satisfying cell (we require only a weak ordering).

Note that technically, we need to require local delineability of p_5 and p_6 here; however, this is trivial in the univariate case.

4.6.10.1 Comparison to Closed Cells in CAIC

Applying this theorem to the CAIC algorithm yields a similar method as described in Section 3.7.3, where strict constraints in the input to the covering methods are exploited: Every (implicitly represented) cell that is constructed carries a flag indicating whether the input formula is violated in its closure as well. The main theorem says that if all cells covering some cylinder $R \times \mathbb{R}$ over some common cell R carry that flag, then the formula is also violated in the closure of R (i.e. that cell also carries the flag). The justification for this approach does not involve CAD theory; it simply uses the fact that the projection of a union of closed sets covering some cylinder is also closed.

Although these approaches seem similar, their justification is quite different, resulting also in different algorithms: The *ordering-based* approach from this section is able to close interval bounds defined by certain roots of resultants, while the *set-based* approach from Section 3.7.3 does not distinguish on the projection polynomials. However, the set-based approach can only be applied if all cells covering some cylinders are closed, while the ordering-based approach is able to mix closed and open cells (i.e. the set-based approach fails on Example 4.8): In the set-based approach, we observe that inheriting flags rarely happens after only a few projection steps, while the ordering-based approach might be able to merge cells on lower levels as well. In fact, the ordering-based approach is able to merge cells even if there are no strict constraints in the input.

4.6.11 Future Work

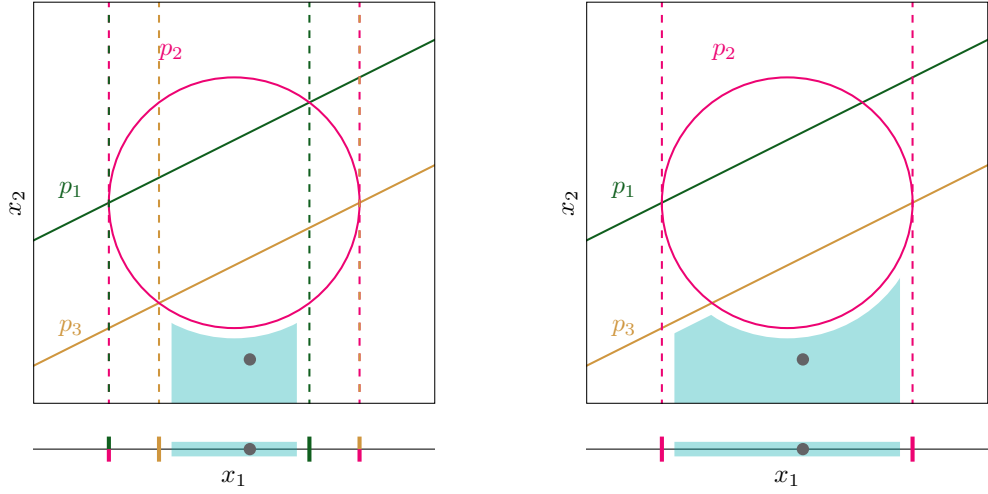
Similarly as in the previous section, so far, we apply weak ordering only to roots of resultants. It is desirable to extend the work to roots of discriminants and (leading) coefficients. Although in the previous section, we claimed that filtering roots of discriminants might be impossible, we have higher hopes here, as we only need to decide whether we can close a bound of some symbolic interval. The work in Section 3.7.3 is another reason for this hope: Although the reasoning is quite different, merging that approach into the framework of this section is desirable (which is of course not straight-forward).

4.7 Compound Cell Bounds

So far, we described the cell by a single root function per bound. In this section, we will use multiple root functions to bound the cell. We motivate this extension by some examples.

Example 4.9

Consider the polynomials from Example 3.16 on page 86 and the point $s = (0.125, -1.25)$. Figure 4.18a depicts the cell resulting from the single cell construction. Figure 4.18b depicts the cell we would obtain if we would take all root functions above s as upper bound on the cell: We do not use the indexed root ordering to ensure that all roots stay above $\text{root}_{x_2}^{p_2,1}$, but ensure that x_2 is in the “intersection” of $(-\infty, \text{root}_{x_2}^{p_1,1})$, $(-\infty, \text{root}_{x_2}^{p_2,1})$, $(-\infty, \text{root}_{x_2}^{p_3,1})$, and $(-\infty, \text{root}_{x_2}^{p_2,2})$, which we sloppily denote here as $(-\infty, \min\{\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1}, \text{root}_{x_2}^{p_2,2}\})$ (later, we introduce a slightly extended notation).



(a) Cell described by the interval $(-\infty, \text{root}_{x_2}^{p_2,1})$ for x_2 and the ordering $\prec = \{(\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1}), (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_1,1}), (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_2,2})\}$.
 (b) Cell described by the interval $(-\infty, \min\{\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1}, \text{root}_{x_2}^{p_2,2}\})$ for x_2 and the ordering $\prec = \emptyset$.

Figure 4.18: Single cell construction from Example 4.9 without and with compound bounds.

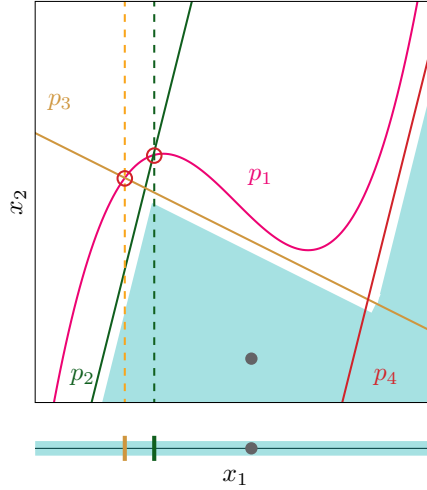


Figure 4.19: The cell from Example 4.10 is bounded by a piecewise linear function defined by the root functions of p_2 , p_3 , and p_4 . The two circles denote intersections of these polynomials with p_1 which are irrelevant in the sense of Section 4.5.

Example 4.10

Now consider the example depicted in Figure 4.19, where x_2 is bounded by $\text{root}_{x_2}^{p_2,1}$, $\text{root}_{x_2}^{p_3,1}$, and $\text{root}_{x_2}^{p_4,1}$. The example shows two challenges: First, the compound function cannot be represented by a minimum of the root function's values, but as $\max\{\min\{\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1}\}, \text{root}_{x_2}^{p_4,1}\}$ (again, with a sloppy notation). Second, to keep p_1 outside the cell, we maintain $\prec = \{(\max\{\min\{\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1}\}, \text{root}_{x_2}^{p_4,1}\}, \text{root}_{x_2}^{p_1,1})\}$; however, the intersection of p_1 with p_2 and p_3 respectively would make the cell smaller than necessary. We thus

need to apply the techniques from Section 4.5 to ignore these intersections.

4.7.1 Adapting Symbolic Intervals and Compound Bounds

We now define the compound bounds of a symbolic interval:

Definition 4.11 (Compound Root Function)

Let $i \in \mathbb{N}$, let B be a set of sets of indexed root expressions of level i . A *compound max-min function* $\max_min_B : \mathbb{R}^i \hookrightarrow \mathbb{R}$ of level i is defined as

$$\max_min_B(s) = \begin{cases} \text{undef} & \text{if ex. } b \in B' \in B \text{ s.t. } b(s) = \text{undef} \\ \max\{\min\{b(s) \mid b \in B'\} \mid B' \in B\} & \text{otherwise.} \end{cases}$$

A *compound min-max function* $\min_max_B : \mathbb{R}^i \hookrightarrow \mathbb{R}$ of level i is defined as

$$\min_max_B(s) = \begin{cases} \text{undef} & \text{if ex. } b \in B' \in B \text{ s.t. } b(s) = \text{undef} \\ \min\{\max\{b(s) \mid b \in B'\} \mid B' \in B\} & \text{otherwise.} \end{cases}$$

Definition 4.12 (Adaptation of Symbolic Interval)

We adapt the definition of a symbolic interval I of level i such that $I.l$ can be a compound max-min function of level i , and $I.u$ can be a compound min-max function of level i .

Note that $\max_min_B < x$ can be expressed as the formula

$$\bigwedge_{B' \in B} \bigvee_{b \in B'} b < x$$

and $x < \min_max_B$ can be expressed as the formula

$$\bigwedge_{B' \in B} \bigvee_{b \in B'} x < b$$

Property 4.12

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i .

We extend the property $\text{repr}(I, s)$ in the following sense: If $I.l = \max_min_B$ respectively $I.u = \min_max_B$, we require $\forall B' \in B \forall \xi \in B' . \xi \in \text{ire}(\xi.p, s)$, and

- $I = (\max_min_{B_l}, \min_max_{B_u})$, $(\bigcap_{B' \in B_l} \bigcap_{l \in B'} \text{dom}(\theta_{l,s})) \cap (\bigcap_{B' \in B_u} \bigcap_{u \in B'} \text{dom}(\theta_{u,s})) \supseteq R \downarrow_{[i-1]}$ and $R = R \downarrow_{[i-1]} \times I$; or
- another analogous case for $I = [\max_min_{B_l}, \min_max_{B_u}]$, $I = (-\infty, \min_max_{B_u}]$, $I = [\max_min_{B_l}, \infty)$, $I = (-\infty, \min_max_{B_u})$,

$$\mathbf{I} = (\max_min_{B_l}, \infty), \quad \mathbf{I} = (\max_min_{B_l}, \min_max_{B_u}], \quad \mathbf{I} = [\max_min_{B_l}, \min_max_{B_u}).$$

Property 4.13

We extend the property *holds*($\mathbf{I}$) in the obvious way.

Rule 4.35

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and $\mathbf{I} = (\max_min_{B_l}, \min_max_{B_u})$ be a symbolic interval of level i for some appropriate B_l, B_u .

$$\text{repr}(\mathbf{I}, s)(R), \text{ an_sub}(i-1)(R) \quad \vdash \text{ an_sub}(i)(R)$$

Proof. The endpoints of $\mathbf{I}$ are described by continuous functions of the form $\max_min_{B_l}$ or $\min_max_{B_u}$ (as *repr*($\mathbf{I}, s$)(R) holds). The statement thus follows immediately from [McC85, Theorem 2.2.4] which only requires continuous functions. $\square$

Definition 4.13 (Adaptation of Indexed Root Orderings)

We adapt the definition of a weak indexed root ordering ($\preceq, \prec$) of level i such that it also involves compound max-min and compound min-max functions of level i .

Let Ξ be a set of indexed roots of level i , $\Xi_{\max_min}$ be a set of compound max-min functions of level i , $\Xi_{\min_max}$ be a set of compound min-max functions of level i , then $(\preceq, \prec)$ is an indexed root ordering on Ξ if $\preceq$ and $\prec$ are subsets of

$$\begin{aligned} & (\Xi \times (\Xi \cup \Xi_{\max_min} \cup \Xi_{\min_max})) \\ & \cup ((\Xi \cup \Xi_{\max_min} \cup \Xi_{\min_max}) \times \Xi) \\ & \cup \Xi_{\max_min} \times \Xi_{\min_max}. \end{aligned}$$

We slightly extend the notion of $\leq_{\preceq, \prec}$, $<_{\preceq, \prec}$ and $=_{\preceq, \prec}$ by extending these relations such that:

$$\begin{aligned} \xi \leq_{\preceq, \prec} \max_min_B & \iff \{\xi\} \in B \\ \xi =_{\preceq, \prec} \max_min_B & \iff \{\{\xi\}\} = B \\ \min_max_B \leq_{\preceq, \prec} \xi & \iff \{\xi\} \in B \\ \min_max_B =_{\preceq, \prec} \xi & \iff \{\{\xi\}\} = B \end{aligned}$$

We extend Property 4.9 as follows:

Property 4.14

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $(\preceq, \prec)$ be a weak indexed root ordering of level $i+1$. The property *ir_ord*($(\preceq, \prec), s$) holds on R if and only if $(\preceq, \prec)$ matches s , for all indexed root expressions $\xi \in \text{dom}(\preceq, \prec)$ it holds $R \subseteq \text{dom}(\theta_{\xi, s})$, for each pair of indexed root expressions $\xi, \xi' \in \text{dom}(\preceq, \prec)$ the conditions from Rule 4.28 hold, for each $\max_min_B \in \text{dom}(\preceq, \prec)$ and each $\min_max_B \in \text{dom}(\preceq, \prec)$ it holds for each $\xi \in B' \in B$ that $R \subseteq \text{dom}(\theta_{\xi, s})$, and for $\min_max_B, \max_min_B, \xi \in \text{dom}(\preceq, \prec)$ it

holds that

$$\begin{aligned} \max_min_B \leq_{\preceq, \prec} \xi &\Rightarrow \max_{B' \in B} \min_{\xi' \in B'} \theta_{\xi', s}(r) \leq \theta_{\xi, s}(r) \text{ for all } r \in R \\ \xi \leq_{\preceq, \prec} \min_max_B &\Rightarrow \theta_{\xi, s}(r) \leq \min_{B' \in B} \max_{\xi' \in B'} \theta_{\xi', s}(r) \text{ for all } r \in R \end{aligned}$$

(the further cases are analogous).

References

ir_ord(($\preceq$, $\prec$), s) Rule 4.36, Rule 4.37

4.7.2 Straight-Forward Proof Rules

We first give a straight-forward proof rule that may be used for Example 4.9, using the standard rules from Section 4.2.

Rule 4.36

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $(\preceq, \prec)$ be an indexed root ordering of level $i + 1$. We define $\Xi_\xi = \{\xi\}$, $\Xi_{\min_max_B} = \{\xi \in B' \in B\}$, $\Xi_{\max_min_B} = \{\xi \in B' \in B\}$, and $\Xi = \bigcup_{\xi \in \text{dom}(\preceq, \prec)} \Xi_\xi \cup \bigcup_{\min_max_B \in \text{dom}(\preceq, \prec)} \Xi_{\min_max_B} \cup \bigcup_{\max_min_B \in \text{dom}(\preceq, \prec)} \Xi_{\max_min_B}$. Assume that $\xi.p$ is irreducible for all $\xi \in \Xi$, and that $(\preceq, \prec)$ matches s .

$$\begin{aligned} &sample(s)(R), an_sub(i)(R), connected(i)(R), \forall \xi \in \Xi. an_del(\xi.p)(R), \\ &\forall (f, f') \in (\preceq \cup \prec), \xi \in \Xi_f, \xi' \in \Xi_{f'}. ord_inv(res_{x_{i+1}}(\xi.p, \xi'.p))(R) \\ &\quad \vdash ir_ord((\preceq, \prec), s)(R) \end{aligned}$$

Now assume that we choose the symbolic interval such that the point-wise minimum of all roots above the current sample point defines the upper bound, and the point-wise maximum of all roots below the sample point defines the lower bound. Then, the role of the indexed root ordering is to maintain connectedness, that is, ensuring that the lower bound is below the upper bound. By Rule 4.36, this means that we “only” compute the resultants of all polynomials with roots below the sample point with all polynomials with roots above the sample point — at the cost of increased effort during the lifting. Of course, we could choose a symbolic interval that contains fewer bounds (and use the indexed root ordering for maintaining sign-invariance of the other polynomials); the approach yields additional flexibility regarding the resultants to be computed.

4.7.3 Using Local Delineability

We now give the rule that enables us to construct the cell from Example 4.10:

Rule 4.37

Let $i \in \mathbb{N}$, $s \in \mathbb{R}^i$, $(\preceq, \prec)$ be a weak indexed root ordering of level $i + 1$, I be a symbolic interval of level i , and $(\preceq', \prec')$ be a weak indexed root ordering of level i . Assume the conditions in Rule 4.28 hold.

Assume that for every $(\xi_1, \max_min_B) \in (\preceq \cup \prec)$, $\xi_2 \in B' \in B$, $\xi_1.p \neq \xi_2.p$, the following holds: Let $P = \{p \in \text{factors}(\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p)) \mid \text{ire}(p, s) \subseteq \text{dom}(\preceq', \prec')\}$ and $\text{ire}(P, s_{[i-1]}) = \{\xi'_1, \dots, \xi'_k\}$ s.t. $\xi'_1(s_{[i-1]}) \leq \dots \leq \xi'_k(s_{[i-1]})$. For every $j \in [k]$, if

- $\xi'_j(s_{[i-1]}) \notin \text{del_int}(\{\xi_1.p\} \cup \{p \mid B' \in B\}, s)$, or
- $s' := \xi'_j(s_{[i-1]}) \in \text{del_int}(\{\xi_1.p\} \cup \{p \mid B' \in B\}, s)$ and $\xi_1(s_{[i-1]}, s') = \xi_2(s_{[i-1]}, s') = \min_{B' \in B} \max_{\xi \in B'} \xi(s_{[i-1]}, s')$,

it holds either

- ξ'_j is strictly outside I w.r.t. $(\preceq', \prec')$ if $\xi_1 \prec \max_min_B$, or
- ξ'_j is weakly outside I w.r.t. $(\preceq', \prec')$ otherwise.

Assume that the analogous statements for the other cases hold.

For each $\xi_1, \xi_2 \in \text{dom}(\preceq, \prec)$, let $Q_{\xi_1, \xi_2} := \{\text{res}_{x_{i+1}}(\xi_1.p, \xi_2.p), \text{proj_polys}(\xi_1.p), \text{proj_polys}(\xi_2.p)\}$.

$$\begin{aligned} & \text{sample}(s)(R), \text{connected}(i)(R), \\ & \text{ir_ord}((\preceq', \prec'), s_{[i-1]})(R), \text{repr}(I, s_{[i-1]})(R), \\ & \forall \xi \in \text{dom}(\preceq, \prec). \text{an_del}(\xi.p)(R), \\ & \forall (\xi_1, \xi_2) \in (\preceq \cup \prec). \xi_1.p \neq \xi_2.p \rightarrow \text{an_loc_del}(Q_{\xi_1, \xi_2}, I, s)(R) \vdash \text{ir_ord}((\preceq, \prec), s)(R) \end{aligned}$$

Note that this rule is rather general and allows any polynomial to define the bounds of the symbolic interval. Thus, the checks are quite expensive, as at each resultant root, we need to check which root function defines the boundary. However, in some special cases, more efficient checks are possible: If the bound is piece-wise linear and described by bivariate polynomials, we can easily compute the intersections between the defining polynomials and can thus determine an interval for each polynomial where it describes the current bound. We will examine this in Section 5.4.2.

4.8 Projective Delineability

So far, we always added the discriminant and leading coefficient to the projection in order to guarantee delineability of a polynomial. In this section, we will formalize the role of the leading coefficient (as already illustrated in Example 3.8) in order to identify the cases when we can omit it. To that end, we introduce a new theory, including the notion of *projective delineability* which is achieved by the order-invariance of the discriminant.

Example 4.11

Consider the polynomials from Example 3.16 on page 86 and $p_4 = -x_1x_2 - 0.75$, as depicted in Figure 4.20a, along with a sample point and a cell as constructed using the proof rules introduced in Section 4.2: The algorithm adds coefficients and discriminants to guarantee delineability of these polynomials; in this case, we add $p_5 = \text{disc}_{x_2}(p_2)$ and $p_6 = \text{ldcf}_{x_2}(p_4)$ (the others are trivial). Further, as the cell is described by $I_2 = (\text{root}_{x_2}^{p_2,1}, \text{root}_{x_2}^{p_3,1})$ on level 2 we aim to maintain $\text{root}_{x_2}^{p_4,1} < \text{root}_{x_2}^{p_2,1}$ on R , $\text{root}_{x_2}^{p_2,1} < \text{root}_{x_2}^{p_3,1}$ on R , $\text{root}_{x_2}^{p_3,1} < \text{root}_{x_2}^{p_1,1}$ on R , and $\text{root}_{x_2}^{p_3,1} < \text{root}_{x_2}^{p_2,2}$

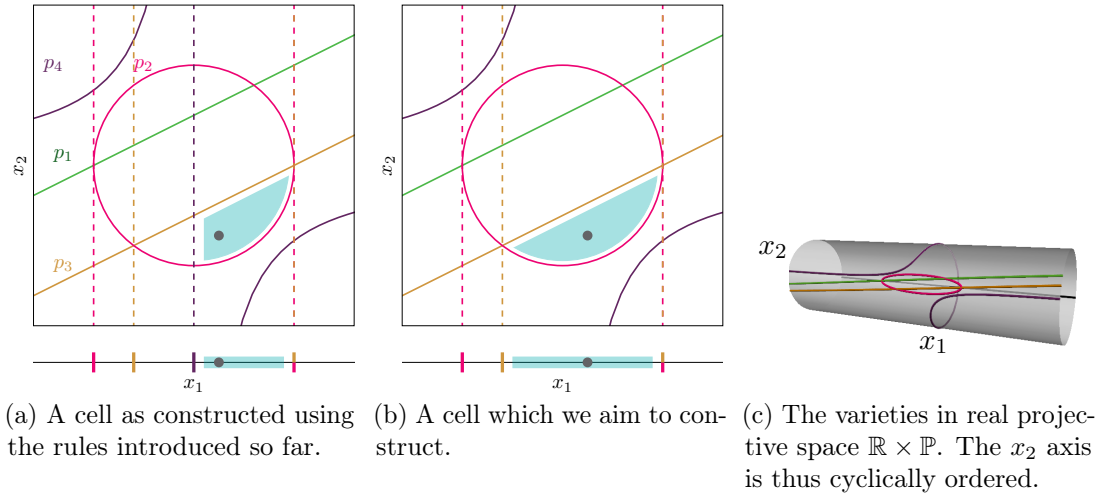


Figure 4.20: Illustration for Example 4.11.

on R . We thus add the corresponding resultants, of which only $p_7 = \text{res}_{x_2}(p_2, p_3)$ is non-trivial. On level 1, we determine $I_1 = (\text{root}_{x_1}^{p_6,1}, \text{root}_{x_1}^{p_5,2})$ as describing the interval.

The singularity of p_4 (witnessed by its leading coefficient) splits the cell, although crossing that boundary does not change the sign of any input polynomial. Figure 4.20b shows the cell that we aim to construct instead: if we detect that the singularity of p_4 does not affect the cell, we omit the leading coefficient of p_4 and build the enlarged version.

For this reasoning, we view the roots of the polynomials in the projective real line, as depicted in Figure 4.20c, where intuitively $-\infty$ and $+\infty$ are identified with the same point ∞ . Above the singularity of p_4 , the two distinct root functions of p_4 coincide at the point ∞ , and thus can be described as a unique real projective root function of p_4 . The order-invariance of the discriminant of p_4 ensures that the variety of p_4 can be described using such projective root functions. Now, by adding the resultant of p_2 and p_4 , we ensure that the mentioned root function does not intersect the circle, and thus does not enter the cell. By projective delineability of p_4 , we know that there are no other roots, and we thus can omit the leading coefficient from our projection polynomials.

4.8.1 Projective Roots

We assume the reader to be familiar with the notions related to the real projective line $\mathbb{P}$ introduced in Section 2.3. For in-depth details on the theory introduced in this subsection, we refer to [MNM⁺24].

We define the projective roots of a (univariate) polynomial. These are defined as the roots of its homogenization:

Definition 4.14 ([MNM⁺24, Definitions 2, 3 and 5])

- A *binary form of degree* $d \in \mathbb{N}$ is a function $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ of the form

$$g(x, y) = \sum_{k=0}^d c_k x^k y^{d-k}, \quad c_0, \dots, c_d \in \mathbb{R}.$$

The set of binary forms of degree d is denoted by $\mathbb{R}[x, y]_d$.

- The *pull-back of a binary forms of degree* $d \in \mathbb{N}$ is the map $\iota^{*d} : \mathbb{R}[x, y]_d \rightarrow \mathbb{R}[x]_{\leq d} : g \mapsto g(x, 1)$ (where $\mathbb{R}[x]_{\leq d}$ is the set of univariate polynomial functions of degree less or equal to d). The map is invertible. The *homogenization w.r.t. degree* d is $H^d = (\iota^{*d})^{-1}$.
- Let $p \in \mathbb{Q}[x, y]$ be a binary form of degree d . Then $(a : b) \in \mathbb{P}$ is a *projective root of* p if $p(a : b) = 0$.
- Let $p \in \mathbb{Q}[x]$, and $d \in \mathbb{N}$. Then $(a : b) \in \mathbb{P}$ is a projective root of p w.r.t. d if it is a projective root of $H^d(p)$.

Lemma 4.2 ([MNM⁺24, Lemmas 2 and 3])

- Let $(a : b) \in \mathbb{P}$ be a projective root of the binary form $p \in \mathbb{Q}[x, y]$ of degree d . Then $\varphi^{-1}(a : b)$ is either a real root of $\iota^{*d}(p)$ of multiplicity k and the root $(a : b)$ of p is of the same multiplicity k ; or the root $(a : b) = \infty$ of p is of multiplicity $d - \deg(\iota^{*d}(p))$.
- Let $p \in \mathbb{Q}[x]$, and $d \in \mathbb{N}$. Then $(a : b) \in \mathbb{P}$ is a projective root of p w.r.t. d of multiplicity k if and only if it is a projective root of $H^d(p)$ of multiplicity k .

The set of projective roots of a univariate polynomial p (with respect to d) thus splits into the real roots and the root at infinity: $(a : b) \in \mathbb{P}$ is a projective root of p of multiplicity k w.r.t. d if and only if either $b \neq 0$ and $\frac{a}{b}$ is a real root p of multiplicity k or $b = 0$ and $k = d - \deg(p)$. The set of the projective roots of p w.r.t. d is thus determined by computing $\text{realRoots}(p)$ and adding ∞ if $d > \deg(p)$.

We now transfer the notion to multivariate polynomials:

Definition 4.15 ([MNM⁺24, Definition 10])

Let $p \in \mathbb{Q}[x_1, \dots, x_i]$. We define the set

$$\text{projRoots}(p) =$$

$$\{(s_1, \dots, s_{i-1}, (a, b)) \in \mathbb{R}^{i-1} \times \mathbb{P} \mid H^{\deg_{x_i}(p)}(p)(s_1, \dots, s_{i-1}, (a, b)) = 0\}.$$

We say for $(s_1, \dots, s_{i-1}, (a, b)) \in \text{projRoots}(p)$ that (a, b) is a *projective root of* p over $(s_1, \dots, s_{i-1})$.

4.8.2 Projective Delineability

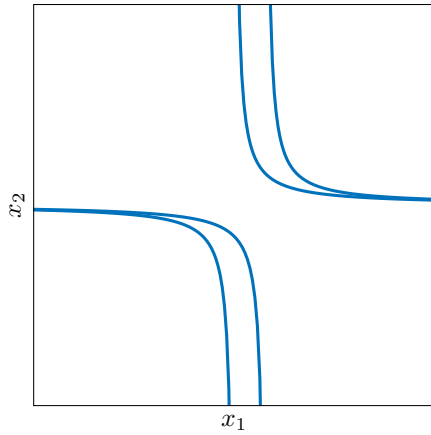
We are now ready to formalize the notion of projective delineability.

Definition 4.16 (Projective Delineability [MNM⁺24, Definition 11])

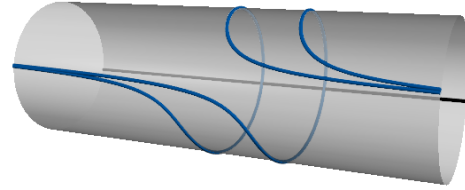
Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be a cell, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}] \setminus \{0\}$. The polynomial p is called *projectively delineable* on R if and only if there exist continuous functions $\theta_1, \dots, \theta_k : R \rightarrow \mathbb{P}$ (for $k \in \mathbb{N}$) such that

- for any point in R , the values of $\theta_1, \dots, \theta_k$ are distinct;
- the projective roots of the univariate polynomial $p(r, x_{i+1})$ w.r.t. $\deg_{x_{i+1}}(p)$ are $\theta_1(r), \dots, \theta_k(r)$ for all $r \in R$; and
- there exist constants $m_1, \dots, m_k \in \mathbb{N}_{>0}$ such that for all $r \in R$ and all $j \in [k]$, the multiplicity of the root $\theta_j(r)$ of $p(r, x_{i+1})$ w.r.t. $\deg_{x_{i+1}}(p)$ is m_j .

The θ_j are called *projective root functions* of p on R . The cells $R \times \theta_j$, $j \in [k]$ are called *projective p -sections* over R .



(a) Variety of a polynomial in Euclidean space.



(b) Variety in projective space. The black line marks the point at ∞ .

Figure 4.21: The polynomial $p = (x_1x_2 - 1)((x_1 - 1)x_2 - 1)$ is projectively delineable on $(-\infty, \infty)$.

In particular, the first condition means that the function values as points around the unit circle maintain a cyclic ordering. Figure 4.21 illustrates an example of a polynomial that is projectively delineable. Note that we do not have projective sectors here, although this would be possible. The reason is that they might go through infinity, having a chain of consequences if being used for the proof system; we thus postpone this step to potential future work.

Definition 4.17 (Analytic Projective Delineability)

Analytic projective delineability is defined as projective delineability, with the modification that it is only defined on connected analytic submanifolds (instead of general cells) and the projective root functions are required to be analytic (instead of only continuous).

The central theorem of this section states that order-invariance of the discriminant plus non-nullification is enough to guarantee projective delineability, no need to maintain the sign-invariance of the leading coefficient.

Note that the theorem requires the underlying cell to be simply connected (Section 2.1.6), which is stronger than connectedness. This assumption is always met for locally cylindrical cells, since they are homeomorphic to open cubes of Euclidean spaces [BPR06, Proposition 5.3] (which are simply connected). Thus, this restriction is rather technical.

Theorem 4.2

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be a simply connected analytic submanifold, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ of level $i + 1$. Assume that p is irreducible. If p is not nullified on any point in R and $\text{disc}_{x_{i+1}}(p)$ is order-invariant on R , then p is analytically projectively delineable on R and p is order-invariant in each projective p -section on R .

Proof. This is in immediate consequence of [MNM⁺24, Theorem 2]. □

We now state the (rather technical) properties and proof rules:

Property 4.15

Let $i \in \mathbb{N}$, and $R \subseteq \mathbb{R}^i$.

The property *simply_connected*(i) holds on R if and only if R is simply connected.

References

simply_connected(i) Rule 4.45

We note that on subsets of $\mathbb{R}^n$, simple connectedness implies connectedness.

Property 4.16

Let $i \in \mathbb{N}$, and $R \subseteq \mathbb{R}^i$.

Let $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ be a polynomial of level $i + 1$. The property *an_proj_del*(p) holds on R if and only if R is a simply connected analytic submanifold, p is analytically projectively delineable on R , and p is order-invariant in all projective p -sections over R .

References

an_proj_del(p) Rule 4.38

Rule 4.38

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$. Assume that p is

irreducible.

$$an_sub(i)(R), \text{ simply_connected}(i)(R), \text{ non_null}(p)(R), \\ ord_inv(\text{disc}_{x_{i+1}}(p))(R) \vdash an_proj_del(p)(R)$$

Rule 4.39

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$.

$$p(s) = 0, \text{ sample}(s)(R), an_sub(i-1)(R), \text{ connected}(i)(R), \\ \text{sgn_inv}(p)(R), an_proj_del(p)(R) \vdash ord_inv(p)(R)$$

Rule 4.40

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$.

$$an_proj_del(p)(R), \text{sgn_inv}(\text{ldcf}_{x_{i+1}}(p))(R), \text{ldcf}_{x_{i+1}}(p)(s) \neq 0 \quad \vdash an_del(p)(R) \\ an_proj_del(p)(R), \text{sgn_inv}(\text{ldcf}_{x_{i+1}}(p))(R), \text{ldcf}_{x_{i+1}}(p)(s) = 0, \\ \text{connected}(i)(R) \quad \vdash an_del(p)(R)$$

Proof. This is [MNM⁺24, Corollary 1]. $\square$

The definition of projective delineability is equivalent to delineability over cells where the polynomial does not have any roots. In those cases, we can guarantee delineability without sign-invariance of the leading coefficient.

Rule 4.41

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$.

$$an_proj_del(p)(R), \text{ sample}(s)(R), \text{ldcf}_{x_{i+1}}(p)(s) \neq 0, \\ \text{realRoots}(p(s, x_{i+1})) = \emptyset \quad \vdash an_del(p)(R)$$

Proof. Let $\theta_1, \dots, \theta_k$ be the projective p -sections over R . If $\text{ldcf}_{x_{i+1}}(p)(s) \neq 0$, then it holds $\deg_{x_{i+1}}(p(s, x_{i+1})) = \deg_{x_{i+1}}(p)$, thus the projective roots $p(s, x_{i+1})$ are all real. Thus, $\{\theta_1(s), \dots, \theta_k(s)\} = \text{realRoots}(p(s, x_{i+1})) = \emptyset$, and it follows $k = 0$. It follows that $p(r, x_{i+1}) \neq 0$ for all $r \in R$, and thus p is delineable on R . $\square$

4.8.3 Projective Root Orderings

For now, we are able to guarantee projective delineability; however, we are not able to use this property yet through our proof system. In this subsection, we will adapt the concept of indexed root orderings to reason about projective root functions. We will identify cases, where reasoning about them is sufficient to guarantee sign-invariance of some polynomial.

For doing so, we extend some familiar technical definitions for projective root functions.

Definition 4.18 (Indexed Projective Root Expressions)

Let $i \in \mathbb{N}$, and $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$. An *indexed projective root expression* is either an *indexed root expression* or an expression of the form $\text{root}_{x_{i+1}}^{p,0} : \mathbb{R}^i \rightarrow \{\infty, \text{undef}\}$ is the projective root ∞ of p over $s \in \mathbb{R}^i$ if $\deg_{x_{i+1}}(p(s)) < \deg_{x_{i+1}}(p)$, and *undef* otherwise.

We define $\text{pire}(p, s)$, $\text{pire}(P, s)$, $\text{pire}(P, s, \xi)$ analogously to Definition 3.9 and including $\text{root}_{x_{i+1}}^{p,0}$ where defined.

Definition 4.19 (Projective Root Function)

Let $i \in \mathbb{N}$, $p \in \mathbb{Q}[x_1, \dots, x_{i+1}]$, $\text{level}(p) = i + 1$, $R \subseteq \mathbb{R}^i$, and $\theta : R \rightarrow \mathbb{P}$ continuous. We call θ a *projective root function* of p on R if

- $p(r, \theta(r)) = 0$ for all $r \in R$ and $\theta(r) \in \mathbb{R}$,
- $\deg_{x_{i+1}}(p(r, x_{i+1})) < \deg_{x_{i+1}}(p)$ for all $r \in R$ and $\theta(r) = \infty$,
- for all $r' \in R$ and $\theta' : R \rightarrow \mathbb{P}$ such that θ' fulfils the previous condition and $p(r', \theta'(r')) = 0$, it holds $\theta' = \theta$.

Let $r \in \mathbb{R}^i$ and $\xi \in \text{pire}(p, r)$. Then $\theta_{\xi, r} : R' \rightarrow \mathbb{P}$ is the projective root function of p on some R' such that $\xi(r) = \theta_{\xi, r}(r)$ and R' is maximal.

We now define projective root orderings. As we look at functions in real projective space, we define a rather peculiar property that maintains that some root function does not intersect some other root function “from below” or “from above”. This property is formalized by the existence of other functions “in between” these functions with respect to some cyclic ordering that is maintained at the sample point we are looking at.

Definition 4.20 (Indexed Projective Root Orderings)

Let $i \in \mathbb{N}$, and Ξ be a set of indexed projective root expressions of level $i + 1$. An *indexed projective root ordering on Ξ* is a reflexive relation $\preceq \subseteq \Xi \times \Xi$. Indexed projective root orderings of this form are also called *indexed projective root ordering of level $i + 1$* , and we define $\text{dom}(\preceq) = \{\xi, \xi' \mid (\xi, \xi') \in \preceq\}$.

We define the following relation w.r.t $\preceq$ and s :

$$\begin{aligned} \xi \leq_s^{\preceq} \xi' &\iff \exists \xi_1, \dots, \xi_k \in \text{dom}(\preceq). \xi = \xi_1 \wedge \xi' = \xi_k \wedge \xi_1 \preceq \dots \preceq \xi_k \\ &\quad \wedge \forall 1 \leq j < j' < j'' \leq k. (\xi_j(s) \neq \xi_{j'}(s) \neq \xi_{j''}(s) \rightarrow [\xi_j(s), \xi_{j'}(s), \xi_{j''}(s)]) \end{aligned}$$

Note that we use a single relation $\preceq$ with less-or-equal semantics instead of a tuple. This is for reasons of simplicity of the proofs.

Property 4.17

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, $\preceq$ be an indexed projective root ordering of level $i + 1$. The property $\text{ipr_ord}(\preceq, s)$ holds on R if and only if R is connected, for all $\xi \in \text{dom}(\preceq)$

it holds $R \subseteq \text{dom}(\theta_{\xi,s})$, and for all $\xi, \xi' \in \text{dom}(\preceq)$ it holds that

$$\begin{aligned} \xi \leq_{\preceq}^s \xi' \wedge \xi(s) \neq \xi'(s) &\Rightarrow \exists \theta_1, \dots, \theta_k : R \rightarrow \mathbb{P}. \theta_1, \dots, \theta_k \text{ continuous} \\ &\wedge \theta_1 = \theta_{\xi,s}|_R \wedge \theta_k = \theta_{\xi',s}|_R \\ &\wedge [\theta_1(s), \dots, \theta_k(s)] \wedge \forall j \in [k-1]. \forall r \in R. \theta_j(r) \neq \theta_{j+1}(r) \end{aligned}$$

and

$$\xi \leq_{\preceq}^s \xi' \wedge \xi(s) = \xi'(s) \Rightarrow \forall r \in R. \theta_{\xi,s}(r) = \theta_{\xi',s}(r)$$

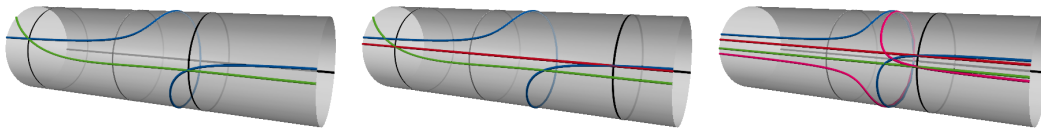
(for the projective root functions $\theta_{\xi,s}, \theta_{\xi',s}$ according to Definition 4.19).

References

ipr_ord($\preceq, s$) Rule 4.42

Example 4.12

Consider Figure 4.22, which depicts three examples. Figure 4.22a illustrates the simple case of two projective root functions which fulfil the ordering as long as they do not intersect. Figure 4.22b illustrates the simple transitive case, where an additional root function is “between” the two root functions. The root ordering does not imply that a cyclic ordering is maintained throughout the whole cylinder. The root ordering determines which root functions are not allowed to intersect. The cyclic ordering at the given sample point however tells us that ξ_4 will not intersect ξ_2 “from above”, but only from “below” when moving from the sample point. Figure 4.22b illustrates a more complex case. Again, from the cyclic ordering at the sample point, we know that ξ_1 will not intersect ξ_4 from below (when moving from the sample point), but only from above.



(a) $\preceq = \{(\xi_2, \xi_4)\}$ (b) $\preceq = \{(\xi_2, \xi_3), (\xi_3, \xi_4)\}$ (c) $\{(\xi_1, \xi_2), (\xi_2, \xi_3), (\xi_3, \xi_4)\}$

Figure 4.22: Illustrations of *ipr_ord*($\preceq, s$) from Example 4.12. ξ_1 is magenta, ξ_2 is green, ξ_3 is red, ξ_4 is blue. The black lines denote the cylinder boundaries. The grey lines denote the sample point s .

We now aim to give a proof rule for the property. Similarly to the vanilla case (Lemma 4.1), the resultant catches the intersections of root functions:

Lemma 4.3

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$ be a connected analytic submanifold and $p_1, p_2 \in \mathbb{Q}[x_1, \dots, x_{i+1}]$ irreducible and coprime such that $\text{level}(p_1) = \text{level}(p_2) = i + 1$, $\theta_1, \theta_2 : R \rightarrow \mathbb{P}$ be real projective root functions of p_1 and p_2 respectively, and $\sim \in \{=, \neq\}$.

If $\text{res}_{x_{i+1}}(p_1, p_2)$ is order-invariant on R and p_1 and p_2 are analytically projectively delineable on R , then $\theta_1(r) \sim \theta_2(r) \iff \theta_1(r') \sim \theta_2(r')$ for all $r, r' \in R$.

Proof. This is in immediate consequence of [MNM⁺24, Theorem 3]. $\square$

Rule 4.42

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $\preceq$ be an indexed projective root ordering of level $i + 1$. Assume that $\xi.p$ is irreducible for all $\xi \in \text{dom}(\preceq)$.

$$\begin{aligned} & \text{sample}(s)(R), \text{an_sub}(i)(R), \text{connected}(i)(R), \\ & \forall \xi \in \text{dom}(\preceq). \text{an_proj_del}(\xi.p)(R), \\ & \forall (\xi, \xi') \in \preceq. \text{ord_inv}(\text{res}_{x_{i+1}}(\xi.p, \xi'.p))(R) \vdash \text{ipr_ord}(\preceq, s)(R) \end{aligned}$$

Proof. We consider $\xi, \xi' \in \text{dom}(\preceq)$ such that $\xi \leq_{\preceq}^s \xi'$.

We first show the case $\xi(s) = \xi'(s)$.

- If $\xi \preceq \xi'$, by projective delineability of $\xi.p$ and $\xi'.p$ and the order invariance of $\text{res}_{x_{i+1}}(\xi.p, \xi'.p)$, it holds by Lemma 4.3 that $\theta_{\xi, s}(r) = \theta_{\xi', s}(r)$ for all $r \in R$.
- If $\xi \preceq^{rt} \xi'$, let $\xi = \xi_1, \dots, \xi_k = \xi'$ be the witnesses for $\xi \leq_{\preceq}^s \xi'$. We apply the previous argument inductively on $\xi_1, \dots, \xi_k$ to obtain that $\theta_{\xi_1, s}(r) = \dots = \theta_{\xi_k, s}(r)$ for all $r \in R$.

We now show the case $\xi(s) \neq \xi'(s)$.

- If $\xi \preceq \xi'$, by projective delineability of $\xi.p$ and $\xi'.p$ and the order invariance of $\text{res}_{x_{i+1}}(\xi.p, \xi'.p)$, it holds by Lemma 4.3 that $\theta_{\xi, s}(r) \neq \theta_{\xi', s}(r)$ for all $r \in R$.
- If $\xi \preceq^{rt} \xi'$, let $\xi = \xi_1, \dots, \xi_k = \xi'$ be the witnesses for $\xi \leq_{\preceq}^s \xi'$. By assumption there exist $\theta_1, \dots, \theta_{k'} \subseteq \{\theta_{\xi_1, s}, \dots, \theta_{\xi_k, s}\}$ such that $\theta_1 = \theta_{\xi, s}|_R$, $\theta_k = \theta_{\xi', s}|_R$, and $[\theta_1(s), \dots, \theta_k(s)]$. We apply the previous argument and the first case inductively to obtain that $\theta_{\xi_j, s}(r) \neq \dots \neq \theta_{\xi_{j+1}, s}(r)$ for all $j \in [k - 1]$ and $r \in R$.

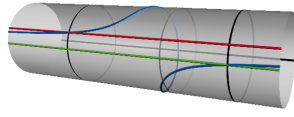
$\square$

For maintaining the sign-invariance property of a polynomial, we need to ensure that each of its projective roots do not intersect the bounds of the cell's symbolic interval of the current level:

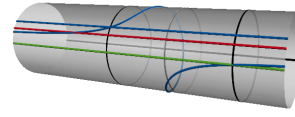
Example 4.13

Consider a projective root that is below the symbolic interval at the current sample point (as depicted in blue in Figures 4.23a to 4.23f). Protecting the lower bound from that root is analogous to the case of regular delineability. However, we now also need to prevent it to cross the upper bound. Projective delineability allows omitting the leading coefficient if the interval is bounded in both directions (otherwise we need the leading coefficient for preventing crossing ∞).

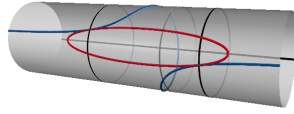
We first consider the simple case where we compute the resultant of the polynomial



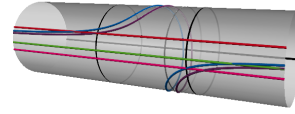
(a) Resultant with bounds: We build the resultant of the blue polynomial with each polynomial defining a bound.



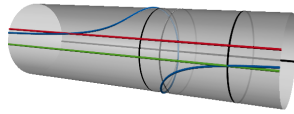
(b) Resultants computed anyway: The blue polynomial has roots both below and above the cell, we would add these resultants anyway.



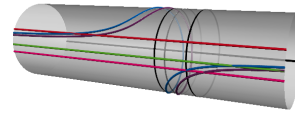
(c) Cell defined by single polynomial: If both cell bounds are described by the same polynomial, the resultant protects the cell from below and above.



(d) Exploiting transitivity: By adding the resultants of the blue with the purple and magenta polynomials respectively, its root is trapped between the other. Additionally, we add the resultant of the purple with the red one, and of the magenta with the green.



(e) Adding the leading coefficient: We may simply add the leading coefficient of the blue polynomial, avoiding its intersection with the point at ∞ .



(f) Combining the previous two ideas: like the former, the blue root is trapped between the magenta and purple, however, instead of the resultant of the purple and red polynomials, we add the leading coefficient of the purple.

Figure 4.23: Projective root “orderings” from Example 4.13 that guarantee sign-invariance. The grey line indicates the first coordinate of the sample point. We aim to describe the cell bounded by the green and red polynomials. The black lines indicate the resulting cell boundaries on the first dimension.

with the polynomials defining the lower and upper bound are computed: e.g. we can omit the leading coefficient of the blue polynomial in Figure 4.23a (which we would add for its delineability in the classical setting) by additionally adding the resultant of the blue and the red polynomial (which is not added in the classical setting). The trade-off may not be attractive; but if the polynomial has a root below and above the cell at the current sample point (e.g. Figure 4.23b), or if both bounds are defined by the same polynomial (e.g. Figure 4.23c), this would not require additional resultants.

We may also use the extended notion of root orderings, which gives us three options:

- We compute a chain of resultants that maintain a cyclic ordering of the given root,

the upper bound and some roots in between in the cyclic sense (see Figure 4.23d, by adding the resultants of the blue with the purple and magenta polynomials respectively, its root is trapped between the other; additionally, we add the resultant of the purple with the red one, and of the magenta with the green).

- We add the leading coefficient of the defining polynomial to the projection, avoiding the intersection of the root with ∞ , thus making it delineable (see Figure 4.23e).
- We mix the two approaches, e.g. for some polynomial in the chain maintaining the cyclic ordering, we add its leading coefficient to avoid crossing ∞ (in Figure 4.23f, like in the first point, the blue root is trapped between the magenta and purple, however, instead of the resultant of the purple and red polynomials, we add the leading coefficient of the purple).

These options are all encoded in the following rule:

Rule 4.43

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$, I be a symbolic interval of level i , $\preceq$ be an indexed projective root ordering of level i .

$$\begin{aligned}
 & I.l \neq -\infty, I.u \neq \infty, \\
 & \forall \xi \in \text{pire}(p, s). (\xi \leq_{\preceq}^s I.l \vee (\exists \xi'. \xi \leq_{\preceq}^s \xi' \wedge I.l(s) < \xi(s) \leq \xi'(s) \wedge \text{an_del}(\xi'.p)(R))), \\
 & \forall \xi \in \text{pire}(p, s). (I.u \leq_{\preceq}^s \xi \vee (\exists \xi'. \xi' \leq_{\preceq}^s \xi \wedge \xi'(s) \leq \xi(s) < I.u(s) \wedge \text{an_del}(\xi'.p)(R))), \\
 & \text{sample}(s)(R), \text{repr}(I, s)(R), \\
 & \text{ipr_ord}(\preceq, s)(R), \text{an_proj_del}(p)(R) \vdash \text{sgn_inv}(p)(R)
 \end{aligned}$$

In the case $I.l = -\infty$ or $I.u = \infty$, we fall back to Rule 4.10, i.e. use normal delineability.

Proof. As p is analytically projectively delineable on $R_{\downarrow[i-1]}$ and $s \in R_{\downarrow[i-1]}$, its variety on $R_{\downarrow[i-1]}$ is described by the projective root functions $\theta_{\xi, s}$, $\xi \in \text{pire}(p, s)$. We show that for all such ξ and all $r \in R_{\downarrow[i-1]}$ it holds $\theta_{\xi, s}(r) \notin R$.

Let $\xi \in \text{pire}(p, s)$. We consider only the case where $I.u(s) \neq \xi(s)$, $\xi(s) \neq I.l(s)$, and $I.l(s) \neq I.u(s)$, as the other cases are similar.

We show that $\text{ipr_ord}(\{(I.u, \xi)\}, s)$ and $\text{ipr_ord}(\{(\xi, I.u)\}, s)$ hold on R . Then, as $\text{repr}(I, s)(R)$ holds and by connectedness of $R_{\downarrow[i-1]}$, the continuous functions that witness the both orderings witness that $\theta_{\xi, s}(r) \notin R$ for all $r \in R_{\downarrow[i-1]}$.

We focus on showing that $\text{ipr_ord}(\{(I.u, \xi)\}, s)$ holds. If $I.u \leq_{\preceq}^s \xi$, then this follows immediately. Otherwise, there is a ξ' such that $\xi \leq_{\preceq}^s \xi'$, $\xi'(s) < \xi(s) < I.u(s)$ and $\xi'.p$ is analytically delineable on R . By induction, it holds $\text{ipr_ord}(\{(\xi', \xi)\}, s)$; thus there are also functions witnessing this ordering. By adding the functions $\theta_{\xi, s}|_R$ and $\theta^* : R \rightarrow \mathbb{P}, r \mapsto \infty$ (reminding that $\xi'.p$ is analytically delineable on R), we obtain a set of functions that witness that $\text{ipr_ord}(\{(I.u, \xi)\}, s)$ holds on R . $\square$

4.8.4 Non-emptiness and (Simple) Connectedness

We give the (straight-forward) rules for guaranteeing non-emptiness of symbolic intervals, connectedness and simple connectedness.

Rule 4.44

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, I be a symbolic interval of level $i + 1$, and $\preceq$ be an indexed projective root ordering of level $i + 1$.

$$I = (l, u), \quad l \neq -\infty, \quad u \neq \infty, \quad l \leq_s^s u, \quad \text{sample}(s)(R), \\ \text{an_del}(l.p)(R), \quad \text{an_del}(u.p)(R), \quad \text{ipr_ord}(\preceq, s)(R) \vdash \text{non_empty}(I, s)(R)$$

Proof. Analogous to the proof of Rule 4.14. Note that defining polynomials of I are delineable on R , which ensures that $R \subseteq \mathbb{R}^i$ (i.e. it does not contain any point in $\mathbb{R}^{i-1} \times \infty$). $\square$

Rule 4.45

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^{i-1}$, and I be a symbolic interval of level i .

$$\text{simply_connected}(i-1)(R), \quad \text{repr}(I, s)(R), \quad \text{sample}(s)(R), \\ \text{non_empty}(I, s)(R) \vdash \text{simply_connected}(i)(R)$$

Proof. The resulting cell is homeomorphic to a ball, which is simply connected. $\square$

Rule 4.46

Let $i \in \mathbb{N}_{>0}$ and $R \subseteq \mathbb{R}^i$.

$$\text{simply_connected}(i)(R) \vdash \text{connected}(i)(R)$$

Proof. On subsets of $\mathbb{R}^n$, simple connectedness implies connectedness. $\square$

4.8.5 Discussion and Future Work

In our modification, our aim is — compared to the classical setting (using real root functions) — to avoid additional resultants but omit leading coefficients whenever possible. We may thus transform any indexed root ordering (referring to real root functions) to a projective root ordering: for each root θ above (below) the cell, we either add $\theta \prec \infty$ (effectively adding a leading coefficient) or $\theta \prec \theta'$ for some root θ' below (above) the cell (effectively adding a resultant with a polynomial that has roots on the “other” side). As we do not want to add additional resultants, we do the latter only if the corresponding resultant had been added in the non-projective case.

This section shows how to make use of projective delineability to maintain certain root orderings which do not require adding leading coefficients to the projection.

However, we may still need to add coefficients to the projection to prevent polynomials from nullification over the underlying cell (according to Rule 4.2). Although we are free

to choose a simpler coefficient (w.r.t. degree) than the leading coefficient, it is desirable to find more cases where we can guarantee non-nullification of a polynomial without the need of adding a coefficient.

Furthermore, as soon as some root defines the boundary of the cell, it is not allowed to “go through infinity” as we did not define what a *projective sector* that includes the point at infinity is. Further, we would need additional conditions for maintaining the sign-invariance of polynomials on projective sectors (e.g. $xy - 1$ changes its sign over the projective section over $(0, \infty)$, but $xy^2 - 1$ does not). Further, for some algorithms, we would need to evaluate the indexed projective root expressions describing the cell boundaries for arbitrary sample points, which might be computationally heavy; however, the CAIC algorithm does not require such evaluations, as it generalizes a covering over a fixed sample point, making it suitable for dealing with projective sectors.

With projective delineability, we characterized the role of the leading coefficient. This potentially allows identifying more cases where it may be left out by additional computational checks in the style of Section 4.5. Similarly, we may use the new tool to identify which cell boundaries may be closed, extending the work in Section 4.6.

4.9 Handling Indexed Root Expressions

The algorithms that use our proof system are required to handle indexed root expressions: either they use them as intermediate representation, or even include them in the output to describe solution formulas of quantifier elimination problems.

The first subsection is on reasoning about the truth of constraints involving indexed root expressions, and the second extends the discussion from Section 3.2.6.2 to the proof system.

4.9.1 Indexed Root Expressions as Input

For the NuCAD and CAIC algorithms, we need to pass constraints that involve indexed root expressions as input to the proof system. Further, it may be useful for the user of a quantifier elimination tool to use the same tool for reasoning about solution formulas. We thus introduce rules that neatly integrate them. To make the differences to regular constraints explicit, we introduce properties and rules for them as well.

Property 4.18

Let $i \in \mathbb{N}$, $R \subseteq \mathbb{R}^i$, and $s \in \mathbb{R}^i$.

1. Let $p \in \mathbb{Q}[x_1, \dots, x_i]$ be a polynomial of level i and $\sim \in \{=, \neq, <, >, \leq, \geq\}$. The property $\text{truth_inv}(p \sim 0, s)$ holds on R if and only if $(p(r) \sim 0) = (p(s) \sim 0)$ for all $r \in R$.
2. Let ξ be an indexed root expression of level i and $\sim \in \{=, \neq, <, >, \leq, \geq\}$. The property $\text{truth_inv}(\xi \sim x_i, s)$ holds on R if and only if $(\xi(r_{[i-1]}) \sim r_i) = (\xi(s_{[i-1]}) \sim s_i)$ for all $r \in R$.

References $truth_inv(p \sim 0, s)$ Rule 4.47 $truth_inv(\xi \sim x_i, s)$ Rule 4.48**Rule 4.47**Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and $p \in \mathbb{Q}[x_1, \dots, x_i]$, $\text{level}(p) = i$.

$$\begin{array}{ll}
\sim \in \{=, \neq, <, >, \leq, \geq\}, \text{sgn_inv}(p)(R) & \vdash truth_inv(p \sim 0, s)(R) \\
\sim \in \{\leq, \geq\}, p(s) \sim 0, \text{semi_sgn_inv}(p)(R) & \vdash truth_inv(p \sim 0, s)(R) \\
\sim \in \{<, >\}, p(s) \not\sim 0, \text{semi_sgn_inv}(p)(R) & \vdash truth_inv(p \sim 0, s)(R)
\end{array}$$

The rule for constraints is straight forward. For constraints involving indexed roots, we could introduce a simple rule that ensures sign-invariance and delineability of the defining polynomial. However, this might be more restrictive than actually needed; instead, we introduce the following rules that use indexed root orderings directly:

Rule 4.48

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, ξ be an indexed root expression of level i , I be a symbolic interval of level i , and $(\preceq, \prec)$ be a weak indexed root ordering of level i .

Assume that $(\preceq, \prec)$ matches $s_{[i-1]}$.

Let $Q := repr(I, s)(R) \wedge ir_ord((\preceq, \prec), s)(R)$.

$$\begin{array}{ll}
Q, \xi(s_{[i-1]}) = s_i, I = [b, b], \xi =_{(\preceq, \prec)} b & \vdash truth_inv(\xi = x_i, s)(R) \\
Q, \xi(s_{[i-1]}) \neq s_i, \xi \text{ is strictly outside } I \text{ w.r.t. } (\preceq, \prec) & \vdash truth_inv(\xi = x_i, s)(R) \\
Q, \xi(s_{[i-1]}) < s_i, I = (l, u), \xi \leq_{(\preceq, \prec)} l & \vdash truth_inv(\xi < x_i, s)(R) \\
Q, \xi(s_{[i-1]}) < s_i, I = [l, u], \xi <_{(\preceq, \prec)} l & \vdash truth_inv(\xi < x_i, s)(R) \\
Q, \xi(s_{[i-1]}) \geq s_i, I = (l, u), \xi \geq_{(\preceq, \prec)} u & \vdash truth_inv(\xi < x_i, s)(R) \\
Q, \xi(s_{[i-1]}) \geq s_i, I = [l, u], \xi \geq_{(\preceq, \prec)} u & \vdash truth_inv(\xi < x_i, s)(R) \\
Q, \xi(s_{[i-1]}) \leq s_i, I = (l, u), \xi \leq_{(\preceq, \prec)} l & \vdash truth_inv(\xi \leq x_i, s)(R) \\
Q, \xi(s_{[i-1]}) \leq s_i, I = [l, u], \xi \leq_{(\preceq, \prec)} l & \vdash truth_inv(\xi \leq x_i, s)(R) \\
Q, \xi(s_{[i-1]}) > s_i, I = (l, u), \xi \geq_{(\preceq, \prec)} u & \vdash truth_inv(\xi \leq x_i, s)(R) \\
Q, \xi(s_{[i-1]}) > s_i, I = [l, u], \xi >_{(\preceq, \prec)} u & \vdash truth_inv(\xi \leq x_i, s)(R)
\end{array}$$

The rules for $truth_inv(\xi \neq x_i, s)(R)$, $truth_inv(\xi > x_i, s)(R)$, and $truth_inv(\xi \geq x_i, s)(R)$ are dual.

4.9.2 Replace Indexed Root Expressions using Thom's Lemma

For some applications like quantifier elimination and NLSAT, it might be desirable to describe the constructed cells without indexed root expressions but with pure Tarski formulas. Thom's Lemma (Theorem 2.1) gives a way to describe a connected cell using sign conditions on the derivatives.

For doing so, we will put the two approaches described in the first two bullet points in Section 3.2.6.2 into relation of our proof system, although it does not fit into its structure.

Theorem 4.3

Let $i \in \mathbb{N}_{>0}$, $R \subseteq \mathbb{R}^i$, $s \in \mathbb{R}^i$, and I be a symbolic interval of level i . For simplicity of notation, assume that $I.l \neq -\infty$ and $I.u \neq \infty$.

$$\begin{aligned}
 & s \in s_{[i-1]} \times I, \text{ sample}(s)(R), \text{ connected}(i)(R), \\
 & \text{an_del}(I.l.p)(R \downarrow_{[i-1]}), \text{an_del}(I.u.p)(R \downarrow_{[i-1]}) \\
 & \forall k \in [0.. \deg_{x_i}(I.l.p)]. \text{sgn_inv}\left(\frac{\partial^k}{\partial x_i^k} I.l.p\right)(R), \\
 & \forall k \in [0.. \deg_{x_i}(I.u.p)]. \text{sgn_inv}\left(\frac{\partial^k}{\partial x_i^k} I.u.p\right)(R), \\
 & \vdash \exists R' \supseteq R. \text{repr}(I, s)(R')
 \end{aligned}$$

Proof. Let ξ be a root function of level i and $R_\xi \subseteq \mathbb{R}^i$. Assume that $\xi.p$ is analytically delineable on $R_{\xi \downarrow_{[i-1]}}$, thus the variety of $\xi.p$ in R_ξ is described by root functions on $R_{\xi \downarrow_{[i-1]}}$. Assume that additionally all partial derivatives of $\xi.p$ w.r.t. x_i are sign-invariant on R_ξ , thus by Thom's Lemma (Theorem 2.1), R_ξ is connected. As $\xi.p$ is sign-invariant on R_ξ , either $R_{\xi \downarrow_{[i-1]}} \times \xi = R_\xi$ or $R_{\xi \downarrow_{[i-1]}} \times \xi \cap R_\xi = \emptyset$. Further, assume that R_ξ is maximal and $s \in R_\xi$ in order to make it unique.

Now observe that $R \subseteq R_{I.l} \cap R_{I.u}$. By the properties of $R_{I.l}$, $R_{I.u}$ and $s \in s_{[i-1]} \times I$ and connectedness of R , it holds that (shortly speaking) $R \downarrow_{[i-1]} \times I.l$ is below R , and $R \downarrow_{[i-1]} \times I.u$ is above R . Thus, there exists some $R' \supset R$ such that $\text{repr}(I, s)(R')$. $\square$

Using this “rule”, we can resemble the approaches in Section 3.2.6.2: Instead of using application of Rule 4.13, we use Theorem 4.3 to obtain a smaller cell. Besides computing the underlying cell $R \downarrow_{[i-1]}$, we need to maintain $\text{sample}(s)(R) \wedge \forall k \in [0.. \deg_{x_i}(I.l.p)]. \text{sgn_inv}\left(\frac{\partial^k}{\partial x_i^k} I.l.p\right)(R) \wedge \forall k \in [0.. \deg_{x_i}(I.u.p)]. \text{sgn_inv}\left(\frac{\partial^k}{\partial x_i^k} I.u.p\right)(R)$. We may encode the i -th level of R in one of the following ways:

- We encode the sign-conditions on the derivatives that hold at s in a straight-forward way.
- We adapt the indexed root ordering and symbolic interval on the current level to also include the partial derivatives. Then, we only need to encode the sign-conditions on the defining polynomials of the new symbolic interval.

Part III

Practical Aspects

Chapter 5

Projection Heuristics for the Proof System

In Chapter 4, we defined a proof system that ensures the correctness of projections for exploration-guided algorithms. Often, there are multiple possibilities to compute a projection; the proof system does not make a choice which one to take. Instead, it provides the set of choices to the user.

Our algorithm has two points where a choice needs to be made: (1) when choosing a *rule of inference* to apply in Algorithm 4.1 Line 9; and (2) when choosing the symbolic interval(s) and indexed root ordering that are passed to Algorithm 4.1, Algorithm 4.2 (cell projection), and Algorithm 4.3 (covering projection) — which we call *representation*.

For the first choice, the rules of inference may define multiple possible property sets for a property that each imply that property, and so we have the freedom to choose which one to use. We aim to pick the “best” such set. One could compute each of these sets individually and compare them, or, to avoid superfluous heavy computations of resultants or discriminants, prefer sets which seem easier to compute. The latter approach, making the choice heuristically (with respect to e.g. estimated computational effort, or degrees of polynomials, or number of polynomials / real roots), is the approach taken by our implementation. For instance, in the implementation of Rule 4.2, we always pick a coefficient rather than computing a discriminant. Note that even if sets involving resultants or discriminants are avoided where possible, the algorithm should first check whether one such set is already maintained before computing any new set.

For the second choice, we elaborate what we mean by a *representation*. As already discussed, there might be multiple choices for the cell description I on the current level, the choice of cells that cover a cylinder, and the indexed root ordering $(\approx, <)$ for fulfilling the requirements for Rules 4.10, 4.11, and 4.16 (when restricting to the basic set of rules). Additionally, if I is a section, we can make use of the equational constraints projection in Rule 4.17. A representation determines all these parameters.

We focus on representations for instantiating the basic rules in Section 4.2 to illustrate the ideas. The extension to the other rules is straight-forward, but highly technical.

Origins Section 5.1 is based on [NÁS⁺24]. Sections 5.2 and 5.3 are novel contributions of this thesis. Section 5.4 is a summary of [PNÁ25] which is based on the Master’s respectively Bachelor’s theses of Promies [Pro22] and Wagner [Wag23], and Section 5.5

is a summary of the Bachelor's thesis of Perkampus [Per24].

5.1 Basic Heuristics for Cells

We are given some set of polynomials $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$ and a sample point $s \in \mathbb{R}^i$ for some $i \in \mathbb{N}_{>0}$, and aim to compute a symbolic interval $\mathbf{I}$ and an indexed root ordering $(\approx, \sim)$ that maintains sign-invariance of each $p \in P$ according to Rule 4.10 and connectedness or non-emptiness of the cell according to Rules 4.11 and 4.14 if required. We thus need to provide an implementation analogous to Algorithm 3.8 but for the proof system setting.

For the symbolic interval, we minimize the degrees in the main variable of the defining polynomials.

Definition 5.1 (Choice of the Symbolic Interval)

Let $i \in \mathbb{N}_{>0}$, $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$ be a set of irreducible polynomials of level i , and $s \in \mathbb{R}^i$.

Let $\Xi = \text{ire}(P, s_{[i-1]})$. We define the sets Ξ_{lo} and Ξ_{up} of the greatest lower respectively lowest upper bounds of s_i as follows:

$$\begin{aligned}\Xi_{lo} &= \{\xi \in \Xi \mid \xi(s_{[i-1]}) \leq s_i \wedge \forall \xi' \in \Xi. (\xi'(s_{[i-1]}) \leq s_i \Rightarrow \xi'(s_{[i-1]}) \leq \xi(s_{[i-1]}))\}, \\ \Xi_{up} &= \{\xi \in \Xi \mid s_i \leq \xi(s_{[i-1]}) \wedge \forall \xi' \in \Xi. (s_i \leq \xi'(s_{[i-1]}) \Rightarrow \xi(s_{[i-1]}) \leq \xi'(s_{[i-1]}))\}.\end{aligned}$$

We pick $\xi_{lo} \in \Xi_{lo}$ such that $\deg_{x_i}(\xi_{lo}.p)$ is minimal and $\xi_{up} \in \Xi_{up}$ such that $\deg_{x_i}(\xi_{up}.p)$ is minimal. Additionally, if $\Xi_{lo} = \Xi_{up}$, then we require $\xi_{lo} = \xi_{up}$.

We define the *lowest degree interval* $\mathbf{I}_{\text{ldg}}$ of s with respect to P as

$$\begin{aligned}\mathbf{I}_{\text{ldg}} &= [\xi_{lo}, \xi_{up}] && \text{if } \xi_{lo} = \xi_{up}, \\ \mathbf{I}_{\text{ldg}} &= (\xi_{lo}, \xi_{up}) && \text{otherwise.}\end{aligned}$$

For the indexed root ordering, there are two possibilities: aim to make the underlying cell as big as possible; or aim to avoid heavy resultant computations. In theory, we could compute the results for all (or all promising) possible indexed root orderings and pick the best one. However, as this is infeasible in practice, we define below several alternative heuristics with different rationales.

To achieve this, we employ a somewhat idealistic view on the problem. Recall that an indexed root ordering is ensured by making resultants order-invariant, which is often a stronger ordering on the roots than required by the picked indexed root ordering. We ignore this fact and base our heuristics on the set of indexed roots without considering common defining polynomials between them (for now). This is further discussed below.

We start with the following observation: as the given derivation rules always require delineability for all polynomials whose sign-invariance is proven using an indexed root ordering (Rule 4.10), a fixed ordering of all real root functions defined at $s_{[i-1]}$ of such polynomials is guaranteed anyway. Thus, we can restrict the heuristics for the choice of $(\approx, \prec)$ by computing the ordering on the set $\tilde{\Xi}$ containing for each polynomial only the closest lower and upper roots to s_i , and extending an ordering $\tilde{\preceq}$ of $\tilde{\Xi}$ to an ordering $\preceq$ on Ξ . This is formalized in the following definition.

Definition 5.2

Let i, P, s, Ξ be as in Definition 5.1. Then

$$\begin{aligned}\tilde{\Xi} = & \{ \xi \in \Xi \mid \xi(s_{[i-1]}) \leq s_i \wedge \forall \xi' \in \Xi \setminus \{\xi\}. (\xi'.p = \xi.p \Rightarrow \\ & \neg(\xi(s_{[i-1]}) < \xi'(s_{[i-1]}) \leq s_i)) \} \\ & \cup \{ \xi \in \Xi \mid s_i \leq \xi(s_{[i-1]}) \wedge \forall \xi' \in \Xi \setminus \{\xi\}. (\xi'.p = \xi.p \Rightarrow \\ & \neg(s_i \leq \xi'(s_{[i-1]}) < \xi(s_{[i-1]}))) \}.\end{aligned}$$

For any indexed root ordering $(\tilde{\approx}, \tilde{<})$ on $\tilde{\Xi}$ matching $s_{[i-1]}$, we define the ordering $(\approx, <)$ on Ξ matching $s_{[i-1]}$ as

$$\begin{aligned}\approx &= \tilde{\approx} \\ < &= \tilde{<} \cup \{ (\xi, \xi') \mid \xi, \xi' \in \Xi \text{ s.t. } \xi.p = \xi'.p \wedge \xi(s_{[i-1]}) < \xi'(s_{[i-1]}) \}.\end{aligned}$$

In the following definitions we give different heuristics to choose an indexed root ordering $(\tilde{\approx}, \tilde{<})$ which in each case can be extended to a corresponding ordering $(\approx, <)$. We focus on the requirements of Rule 4.10, as satisfying the requirements of Rules 4.11 and 4.16 is as easy as adding the pair $(I.l, I.u)$ to $<$.

First, in the section case, the below EQUATIONAL CONSTRAINT heuristic enforces simply the application of the equational constraint rule to all polynomials by leaving the indexed root ordering empty.

Definition 5.3 (EQUATIONAL CONSTRAINT Representation)

Let $i, P, s, \Xi, \xi_{lo}, \xi_{up}, I_{\text{deg}}$ be as in Definition 5.1. If $\xi_{lo} = \xi_{up}$, we define the EQUATIONAL CONSTRAINT representation as the tuple $(I_{\text{deg}}, (\emptyset, \emptyset))$.

Next, the BIGGEST CELL heuristic defines the weakest ordering on the indexed roots according to Rule 4.10 and thus defines the biggest possible underlying cell (under the assumption that this ordering can be realized perfectly, i.e. the resultants have roots only below the crossing of a polynomial's root with a cell boundary) as visualized in Figure 5.1a. Note that for the section case, the BIGGEST CELL heuristic is the EQUATIONAL CONSTRAINT heuristic plus some discriminants and coefficients; thus, the application of BIGGEST CELL only makes sense in the sector case.

Definition 5.4 (BIGGEST CELL Representation)

Let $i, P, s, \Xi, \xi_{lo}, \xi_{up}, I_{\text{deg}}$ be as in Definition 5.1.

We define the indexed root ordering $(\approx, <)$ on Ξ according to

$$\begin{aligned}\tilde{\approx} &= \{ (\xi, \xi_{lo}) \mid \xi \in \tilde{\Xi} \setminus \{\xi_{lo}\} \text{ s.t. } \xi(s_{[i-1]}) = \xi_{lo}(s_{[i-1]}) \} \\ &\cup \{ (\xi_{up}, \xi) \mid \xi \in \tilde{\Xi} \setminus \{\xi_{up}\} \text{ s.t. } \xi_{up}(s_{[i-1]}) = \xi(s_{[i-1]}) \} \\ \tilde{<} &= \{ (\xi, \xi_{lo}) \mid \xi \in \tilde{\Xi} \setminus \{\xi_{lo}\} \text{ s.t. } \xi(s_{[i-1]}) < \xi_{lo}(s_{[i-1]}) \} \\ &\cup \{ (\xi_{up}, \xi) \mid \xi \in \tilde{\Xi} \setminus \{\xi_{up}\} \text{ s.t. } \xi_{up}(s_{[i-1]}) < \xi(s_{[i-1]}) \}\end{aligned}$$

and the BIGGEST CELL representation as the tuple $(I_{\text{deg}}, (\approx, <))$.

The below **LOWEST DEGREE BARRIERS** heuristic minimizes the degrees of the defining polynomials (locally per level), and thus also the degree of the computed resultants (under the above assumption) as visualized in Figure 5.1b. Furthermore, it enforces the equational constraints rule whenever possible.

This heuristic has two motivations. First, that the polynomial degrees grow doubly exponentially during the CAD projection. Second, that the running time of the resultant computation depends quadratically on the degree in the main variable of the input polynomials.

Definition 5.5 (LOWEST DEGREE BARRIERS Representation)

Let $i, P, s, \Xi, \xi_{lo}, \xi_{up}, I_{\text{lddeg}}$ be as in Definition 5.1.

We assume an injection $o : \Xi \rightarrow \mathbb{N}$ that orders the elements of Ξ such that $o(I_{\text{lddeg}}.b) = 0$ respectively $o(I_{\text{lddeg}}.l) = 0$, $o(I_{\text{lddeg}}.u) = 1$ (if both are indexed root expressions), or $o(I_{\text{lddeg}}.l) = 0$ (if $I_{\text{lddeg}}.u = \infty$), or $o(I_{\text{lddeg}}.u) = 0$ (if $I_{\text{lddeg}}.l = -\infty$).

Let $\Xi' \subseteq \Xi$. For $\xi \in \Xi'$, we define the *barrier* of ξ w.r.t. Ξ' , that is a root in Ξ' between $\xi(s_{[i-1]})$ and s_i with minimal degree in the main variable:

If $\xi(s_{[i-1]}) \leq s_i$, then

$$\text{br}_{\Xi'}(\xi) = \arg \min_{\xi' \in \Xi' \text{ s.t. } \xi(s_{[i-1]}) \leq \xi'(s_{[i-1]}) \leq s_i} (\deg_{x_i}(\xi'.p), s_i - \xi'(s_{[i-1]}), o(\xi'));$$

if $s_i \leq \xi(s_{[i-1]})$, then

$$\text{br}_{\Xi'}(\xi) = \arg \min_{\xi' \in \Xi' \text{ s.t. } s_i \leq \xi'(s_{[i-1]}) \leq \xi(s_{[i-1]})} (\deg_{x_i}(\xi'.p), \xi'(s_{[i-1]}) - s_i, o(\xi')).$$

If $\xi_{lo} \neq \xi_{up}$, we define the indexed root ordering $(\approx, \prec)$ on Ξ according to

$$\begin{aligned} \tilde{\approx} &= \{(\xi, \text{br}_{\Xi}(\xi)) \mid \xi \in \tilde{\Xi} \text{ s.t. } \xi(s_{[i-1]}) = \text{br}_{\Xi}(\xi)(s_{[i-1]}) < s_i \text{ and } \xi \neq \text{br}_{\Xi}(\xi)\}, \\ &\cup \{(\text{br}_{\Xi}(\xi), \xi) \mid \xi \in \tilde{\Xi} \text{ s.t. } s_i < \text{br}_{\Xi}(\xi)(s_{[i-1]}) = \xi(s_{[i-1]}) \text{ and } \xi \neq \text{br}_{\Xi}(\xi)\} \\ \tilde{\prec} &= \{(\xi, \text{br}_{\Xi}(\xi)) \mid \xi \in \tilde{\Xi} \text{ s.t. } \xi(s_{[i-1]}) < \text{br}_{\Xi}(\xi)(s_{[i-1]}) < s_i\}, \\ &\cup \{(\text{br}_{\Xi}(\xi), \xi) \mid \xi \in \tilde{\Xi} \text{ s.t. } s_i < \text{br}_{\Xi}(\xi)(s_{[i-1]}) < \xi(s_{[i-1]})\} \end{aligned}$$

and the **LOWEST DEGREE BARRIERS representation** as the tuple $(I_{\text{lddeg}}, (\approx, \prec))$.

If $\xi_{lo} = \xi_{up}$, we exclude the polynomials with roots around s_i which do not qualify as a barrier for some other roots from the indexed root ordering (thus enforcing the application of the equational constraint rule). To do so, for a set of polynomials $P' \subseteq P$, we define the set $\tilde{\Xi}|_{P'} = \{\xi \mid \xi \in \tilde{\Xi} \text{ s.t. } \xi.p \in P'\}$ and let $P_{\text{eq}} \subseteq P$ be the result of the following fixed point computation (i.e. $P_{\text{eq}} = P_j = P_{j+1}$ for some j) which contains the polynomials for which we apply the equational constraints rule:

$$\begin{aligned} P_0 &= \emptyset \\ P_{j+1} &= P_j \cup \{p \in P \mid p \neq I_{\text{lddeg}}.b.p \\ &\quad \wedge \exists \xi \in \tilde{\Xi} \setminus \tilde{\Xi}|_{P_j}. (\xi.p = p \wedge \text{br}_{\tilde{\Xi} \setminus \tilde{\Xi}|_{P_j}}(\xi) = I_{\text{lddeg}}.b \wedge \\ &\quad \nexists \xi' \in \tilde{\Xi} \setminus \tilde{\Xi}|_{P_j}. \text{br}_{\tilde{\Xi} \setminus \tilde{\Xi}|_{P_j}}(\xi') = \xi)\}. \end{aligned}$$

We define the indexed root ordering $(\approx, \prec)$ on $\Xi \setminus \Xi|_{P_{\text{eq}}}$ according the definition of $(\tilde{\approx}, \tilde{\prec})$ as above but only considering the roots $\tilde{\Xi} \setminus \tilde{\Xi}|_{P_{\text{eq}}}$ and the **LOWEST DEGREE BARRIERS representation** as the tuple $(I_{\text{lddeg}}, (\approx, \prec))$.

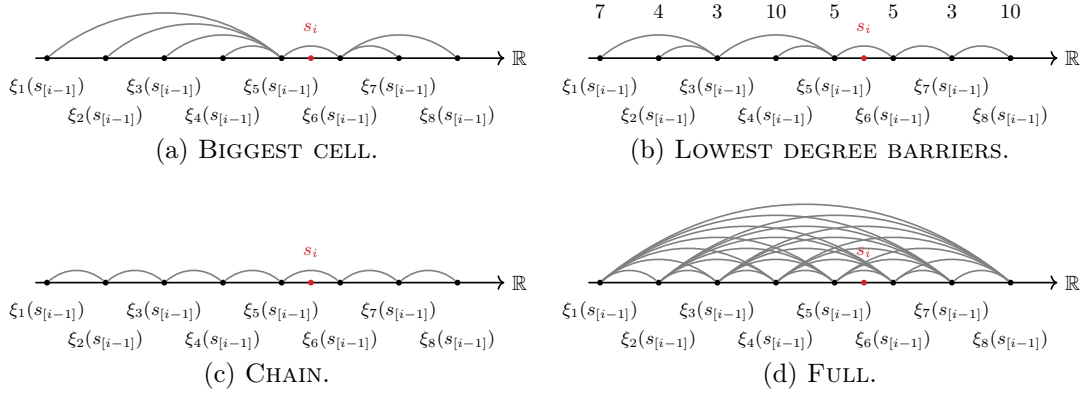


Figure 5.1: Visualization of indexed root ordering heuristics.

The below CHAIN heuristic fixes the total ordering on the roots as visualized in Figure 5.1c.

Definition 5.6 (CHAIN Representation)

Let $i, P, s, \Xi, \xi_{lo}, \xi_{up}, I_{deg}$ be as in Definition 5.1.

Let $\{\xi_1, \dots, \xi_k\} = \Xi$ s.t. $\xi_j(s_{[i-1]}) \leq \xi_{j+1}(s_{[i-1]})$ for all $j \in [k-1]$.

We define the indexed root ordering $(\approx, <)$ on Ξ according to

$$\begin{aligned} \approx &= \{(\xi_j, \xi_{j+1}) \mid j \in [k-1] \text{ s.t. } \xi_j(s_{[i-1]}) = \xi_{j+1}(s_{[i-1]})\} \\ < &= \{(\xi_j, \xi_{j+1}) \mid j \in [k-1] \text{ s.t. } \xi_j(s_{[i-1]}) < \xi_{j+1}(s_{[i-1]})\} \end{aligned}$$

and the *CHAIN representation* as the tuple $(I_{deg}, (\approx, <))$.

Finally, the FULL heuristic fixes the same total ordering as it is the unique transitive closure of the CHAIN heuristic as visualized in Figure 5.1d. Note that we include this heuristic only for illustrative purposes: it resembles the cells constructed naively by making a full projection without adapting to the behaviour at the sample.

Definition 5.7 (FULL Representation)

Let $i, P, s, \Xi, \xi_{lo}, \xi_{up}, I_{deg}$ as in Definition 5.1.

Let $\{\xi_1, \dots, \xi_k\} = \Xi$ s.t. $\xi_j(s_{[i-1]}) \leq \xi_{j+1}(s_{[i-1]})$ for all $j \in [k-1]$.

We define the indexed root ordering $(\approx, <)$ on Ξ as

$$\begin{aligned} \approx &= \{(\xi_j, \xi_{j'}) \mid j, j' \in [k] \text{ s.t. } j < j' \text{ and } \xi_j(s_{[i-1]}) = \xi_{j'}(s_{[i-1]})\} \\ < &= \{(\xi_j, \xi_{j'}) \mid j, j' \in [k] \text{ s.t. } j < j' \text{ and } \xi_j(s_{[i-1]}) < \xi_{j'}(s_{[i-1]})\} \end{aligned}$$

and the *FULL representation* as the tuple $(I_{deg}, (\approx, <))$.

5.1.1 Comparison of the Heuristics

The heuristics are designed to optimize desirable characteristics. We acknowledge their heuristic nature, and in particular that the intuition is made under the idealistic view that any indexed root ordering $(\approx, <)$ can be perfectly realized: That is, the resultants

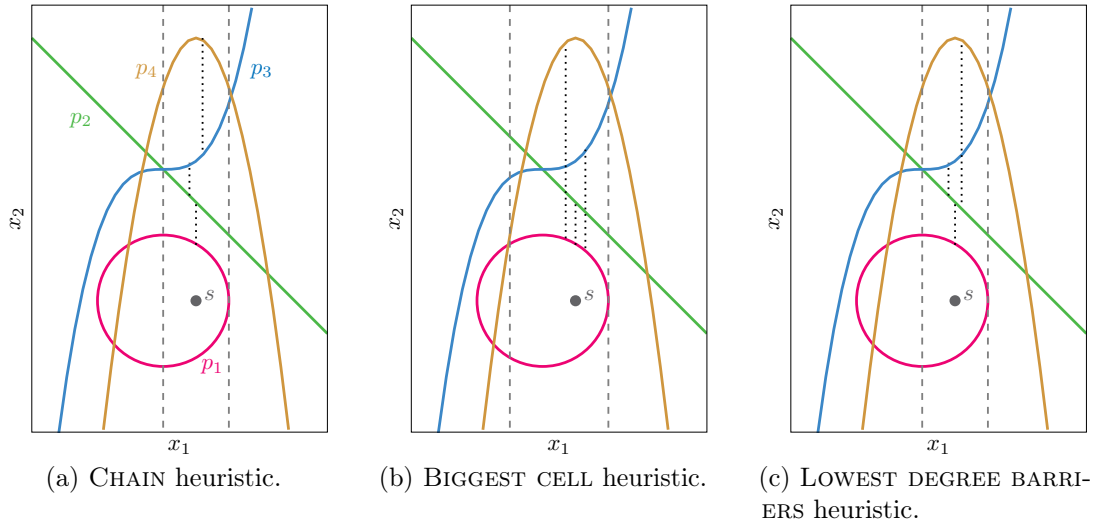


Figure 5.2: Root ordering heuristics; the dotted lines indicate for which pairs of polynomials the resultant is added to the projection.

calculated only have roots which indicate a crossing of two root functions considered in $(\approx, <)$ when in reality they also have “spurious” roots which do not have relevance for the problem at hand.

FULL vs CHAIN Heuristic As mentioned above, both the FULL and the CHAIN heuristic fix the same ordering on the real root functions over any cell containing the current sample as depicted in Figure 5.2a.

Obviously, the CHAIN heuristic is more efficient here as it uses a strict subset of the work done by the FULL heuristic. Unlike the FULL heuristic, the CHAIN heuristic takes the underlying sample into account, and thus the resulting projection is only valid for a single cell, i.e. *locally delineable*. The FULL heuristic is independent of the sample and makes the set of polynomials fully delineable.

CHAIN vs BIGGEST CELL Heuristic The CHAIN heuristic still fixes a stronger ordering on the root functions than necessary. As defined above, the BIGGEST CELL heuristic is the minimal requirement on the ordering to maintain sign-invariance on the constructed cell. This way, we hope that the size of the underlying cell is maximized, as in Figure 5.2b. Note that while the CHAIN heuristic only takes the sample in one dimension less into account, the BIGGEST CELL heuristic also considers the highest dimension of the sample.

BIGGEST CELL vs LOWEST DEGREE BARRIERS Heuristic The LOWEST DEGREE BARRIERS heuristic minimizes the degrees of the resultants, as illustrated in Figure 5.2c. The rationale here is that resultant computations are heavy and their complexity depends on the degrees of the input polynomials, and that polynomials with lower degrees have fewer roots. Thus reducing degrees may reduce the case of resultants having real roots that do not actually correspond to relevant points. Note that despite the naming, the LOWEST DEGREE BARRIERS heuristic could in theory lead to bigger cells than the BIGGEST CELL heuristic in certain cases.

Equational Constraint Projection The equational constraint projection allows us to leave out discriminants and coefficients of all polynomials except the section-defining polynomial by only adding resultants of all polynomials with the defining polynomial. Thus, we expect this rule to be more efficient than the presented heuristics in most cases. However, we added the possibility for the application of heuristics for cases where the section-defining polynomial has high degree and thus computing resultants with that polynomial is desirable to be avoided.

5.1.2 Idealized View versus Reality

For the LOWEST DEGREE BARRIERS heuristic, this might lead to the computation of a redundant set of resultants regarding the minimal requirement on the indexed root ordering. In Figure 5.3, the relation between the first roots of p_2 and p_3 (depicted in red) is superfluous, but its corresponding resultant is added because all roots are considered individually and not their connection via the defining polynomials.

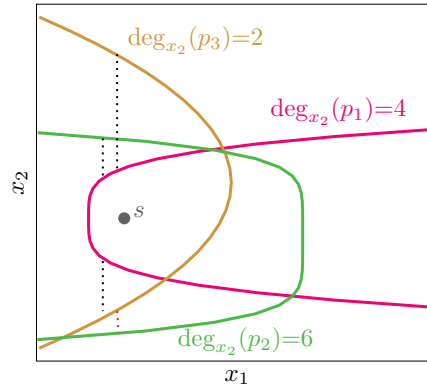


Figure 5.3: Redundancies in the LOWEST DEGREE BARRIERS heuristic.

5.1.3 Levelwise versus Refinement-based Approach

The refinement-based approach to single cell construction (see Section 3.3.2) saves resultants compared to full CAD projection in the same way as the levelwise variant, by exploiting the transitivity of the induced ordering on real root functions. Complexity-wise, the approaches are the same, as both add up to two resultants per polynomial on a level.

However, the influence on the ordering of constraints on the refinement-based variant means that the quality of the constructed cell varies. In the worst case, the resulting ordering corresponds to the CHAIN heuristic: in Figure 5.2a, this is achieved for the example when the polynomials are merged in the ordering p_4, p_3, p_2, p_1 . Then, the upper cell boundary is updated in every step to a lower boundary. In the best case, the BIGGEST CELL heuristic is achieved: for our example in Figure 5.2b, this is when the polynomial p_1 defining the sector's boundary is merged first.

For the section case, it should also be noted that the levelwise approach can always apply the equational constraints rule as the cell description is known before the projection. The refinement-based approach only starts applying the equational constraints rule when the cell collapses to a section; until then it adds discriminants of all polynomials, which may not actually be needed. The illustrating examples may be used to explain

this observation. Consider Figure 5.2a but with the sample s moved to the upper root of p_1 . When merging polynomials in the ordering p_4, p_3, p_2, p_1 , the polynomials p_4, p_3, p_2 are merged as in the sector case until the cell collapses to a section when merging p_1 . When merging p_1 first, then the section case is identified directly and the reduced projection applied when merging p_4, p_3, p_2 , meaning the discriminants and coefficients for those polynomials need not be added.

5.2 Basic Heuristics for Coverings

We are given a set $\mathcal{C}$ of implicit cells of the form (P, s) , with $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s \in \mathbb{R}^i$ for some $i \in \mathbb{N}_{>0}$ and $s' := s_{[i-1]}$, such that $\mathcal{C}$ covers the real line $s' \times \mathbb{R}$ above s' . We aim to compute some sequence $(P^1, s^1), \dots, (P^k, s^k)$ of elements from $\mathcal{C}$ which is non-redundant (in the sense of Algorithm 3.14), along with symbolic intervals $\mathbb{I}^1, \dots, \mathbb{I}^k$ that cover the real line $s' \times \mathbb{R}$ above s' and some indexed root ordering $(\approx, \sim)$ which maintains the covering according to Rule 4.16 and sign-invariance of each $p \in P^j$ in $\mathbb{I}^j$ according to Rule 4.10 for each $j \in [k]$ (and connectedness or non-emptiness of the cells described by $\mathbb{I}^j$ according to Rules 4.11 and 4.14 if required). We thus need to provide an analogue to Algorithm 3.14 for the proof system setting.

Towards the first step — the selection of the sequence, we follow two basic approaches:

GREEDY MIN CELLS This is the heuristic described in [ÁDE⁺21]: We determine the intervals over s' , sort them by lower bound in ascending order, breaking ties with the upper bound in descending order. Then, we take the first interval, erase the following intervals with a lower upper bound, and iterate to the next interval. After the process, we obtain a non-redundant covering.

GREEDY MIN DEGREE We sort the elements of $\mathcal{C}$ by the maximum total degree of its polynomials in descending order. Then, we iterate through the elements and remove an element if the result still covers the whole cylinder $s' \times \mathbb{R}$. Finally, we sort the cells by the lower bound of their intervals over s' .

For the second step, the straight-forward approach is to choose some minimal set of implicit cells according to Rule 4.16, compute representations for (P^j, s^j) separately as described in Section 5.1, taking the union of the resulting orderings, and extend the ordering to maintain the covering according to Rule 4.16:

Definition 5.8 (X-COMPOSED Y-COVERING Representation)

Let $X \in \{\text{BIGGEST CELL}, \text{LOWEST DEGREE BARRIERS}, \text{CHAIN}\}$ and $Y \in \{\text{GREEDY MIN CELLS}, \text{GREEDY MIN DEGREE}\}$.

Let $i \in \mathbb{N}_{>0}$, $k \in \mathbb{N}_{>0}$, $\mathcal{C}$ be a set of implicit cells, and $((P^1, s^1), \dots, (P^k, s^k))$ be a sequence from elements of $\mathcal{C}$ meeting the requirements in the output of Algorithm 3.14 and such that $P^j \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s^j \in \mathbb{R}^i$ for each $j \in [k]$, $s_{[i-1]}^j = s_{[i-1]}^{j'} =: s'$ for $j, j' \in [k]$.

Assume that $((P^1, s^1), \dots, (P^k, s^k))$ has been constructed from $\mathcal{C}$ using heuristic Y . Let $(\mathbb{I}^j, (\approx^j, \prec^j))$ be the X representation of s^j with respect to P^j .

We define the indexed root ordering $(\approx, \prec)$ as

$$\begin{aligned}\approx &= \bigcup_{j \in [k]} \approx^k \cup \{(\mathbf{I}_{j+1}.l, \mathbf{I}_j.u) \mid j \in [k-1], \mathbf{I}_{j+1}.l(s') = \mathbf{I}_j.u(s')\} \\ \prec &= \bigcup_{j \in [k]} \prec^k \cup \{(\mathbf{I}_{j+1}.l, \mathbf{I}_j.u) \mid j \in [k-1], \mathbf{I}_{j+1}.l(s') \neq \mathbf{I}_j.u(s')\}\end{aligned}$$

and the *X-COMPOSED Y-COVERING representation* as the tuple $((\mathbf{I}^1, \dots, \mathbf{I}^k), (\approx, \prec))$.

This heuristic treats the cells separately; however, if the cells share common polynomials, we may obtain redundancies in the ordering (e.g. a subset of the indexed ordering would already guarantee the desired properties). We thus propose the following heuristic, which is similar to the CHAIN representation for cells:

Definition 5.9 (CHAIN Y-COVERING Representation)

Let $Y \in \{\text{GREEDY MIN CELLS}, \text{GREEDY MIN DEGREE}\}$.

Let $i \in \mathbb{N}_{>0}$, $k \in \mathbb{N}_{>0}$, $\mathcal{C}$ be a set of implicit cells, and $((P^1, s^1), \dots, (P^k, s^k))$ be a sequence from elements of $\mathcal{C}$ meeting the requirements in the output of Algorithm 3.14 and such that $P^j \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s^j \in \mathbb{R}^i$ for each $j \in [k]$, $s_{[i-1]}^j = s_{[i-1]}^{j'} =: s'$ for $j, j' \in [k]$.

Assume that $((P^1, s^1), \dots, (P^k, s^k))$ has been constructed from $\mathcal{C}$ using heuristic Y . Let the Ξ^j be defined as in Definition 5.1, $\mathbf{I}_{\text{deg}}^j$ be the lowest degree interval of s^j with respect to P^j , and the $\tilde{\Xi}^j$ be defined as in Definition 5.2.

Let $\Xi = \bigcup_{j \in [k]} \Xi^j$ and $\tilde{\Xi} = \bigcup_{j \in [k]} \tilde{\Xi}^j$. Analogously to Definition 5.2, for any indexed root ordering $(\approx, \prec)$ on $\tilde{\Xi}$, we define the indexed root ordering $(\approx, \prec)$ on Ξ .

Let $\{\xi_1, \dots, \xi_k\} = \tilde{\Xi}$ s.t. $\xi_j(s') \leq \xi_{j+1}(s')$ for all $j \in [k-1]$.

We define the indexed root ordering $(\approx, \prec)$ on Ξ according to

$$\begin{aligned}\approx &= \{(\xi_j, \xi_{j+1}) \mid j \in [k-1], \xi_j(s') = \xi_{j+1}(s')\} \\ \prec &= \{(\xi_j, \xi_{j+1}) \mid j \in [k-1], \xi_j(s') \neq \xi_{j+1}(s')\}\end{aligned}$$

and the *CHAIN Y-COVERING representation* as the tuple $((\mathbf{I}_{\text{deg}}^1, \dots, \mathbf{I}_{\text{deg}}^k), (\approx, \prec))$.

Finally, for comparison purposes and analogous to Definition 5.7, we define the representation that adds all resultants:

Definition 5.10 (FULL Y-COVERING Representation)

Let $Y \in \{\text{GREEDY MIN CELLS}, \text{GREEDY MIN DEGREE}\}$.

Let $i \in \mathbb{N}_{>0}$, $k \in \mathbb{N}_{>0}$, $\mathcal{C}$ be a set of implicit cells, and $((P^1, s^1), \dots, (P^k, s^k))$ be a sequence from elements of $\mathcal{C}$ meeting the requirements in the output of Algorithm 3.14 and such that $P^j \subseteq \mathbb{Q}[x_1, \dots, x_i]$, $s^j \in \mathbb{R}^i$ for each $j \in [k]$, $s_{[i-1]}^j = s_{[i-1]}^{j'} =: s'$ for $j, j' \in [k]$.

Assume that $((P^1, s^1), \dots, (P^k, s^k))$ has been constructed from $\mathcal{C}$ using heuristic Y . Let the Ξ^j be defined as in Definition 5.1 and $\mathbf{I}_{\text{deg}}^j$ be the lowest degree interval of s^j with respect to P^j .

Let $\Xi = \bigcup_{j \in [k]} \Xi^j$. Let $\{\xi_1, \dots, \xi_k\} = \Xi$ s.t. $\xi_j(s') \leq \xi_{j+1}(s')$ for all $j \in [k-1]$. We define the indexed root ordering $(\approx, \prec)$ on Ξ as

$$\begin{aligned} \approx &= \{(\xi_j, \xi_{j'}) \mid j < j' \text{ and } \xi_j(s') = \xi_{j'}(s')\} \\ \prec &= \{(\xi_j, \xi_{j'}) \mid j < j' \text{ and } \xi_j(s') \neq \xi_{j'}(s')\} \end{aligned}$$

and the *FULL Y-COVERING* representation as the tuple $((\mathbf{I}_{\text{ldeg}}^1, \dots, \mathbf{I}_{\text{ldeg}}^k), (\approx, \prec))$.

5.3 Compound Bounds Heuristic for Cells and Coverings

In Section 4.7, we introduced the possibility of using multiple root functions to bound the cell.

Inspired by Example 4.9, we introduce the following heuristic, which simply takes all roots as bounds:

Definition 5.11 (ALL COMPOUND Representation)

Let $i \in \mathbb{N}_{>0}$, $P \subseteq \mathbb{Q}[x_1, \dots, x_i]$ be a set of irreducible polynomials of level i , $s \in \mathbb{R}^i$, and $\Xi = \text{ire}(P, s_{[i-1]})$.

If there exists some $\xi \in \Xi$ such that $\xi(s_{[i-1]}) = s_i$, the ALL COMPOUND representation is defined as Definition 5.3.

Otherwise, let

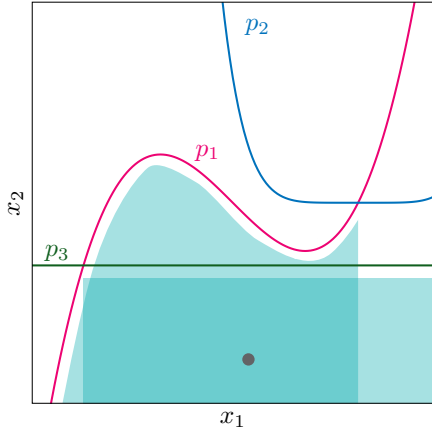
$$\begin{aligned} \Xi_{lo} &= \{\xi \in \Xi \mid \xi(s_{[i-1]}) < s_i\}, \\ \Xi_{up} &= \{\xi \in \Xi \mid s_i < \xi(s_{[i-1]})\}, \\ \mathbf{I} &= (\max_min_{\{\{\xi\} \mid \xi \in \Xi_{lo}\}}, \min_max_{\{\{\xi\} \mid \xi \in \Xi_{up}\}}), \\ \prec &= \{(\max_min_{\{\{\xi\} \mid \xi \in \Xi_{lo}\}}, \min_max_{\{\{\xi\} \mid \xi \in \Xi_{up}\}})\}, \end{aligned}$$

and the ALL COMPOUND representation is defined as $(\mathbf{I}, (\emptyset, \prec))$.

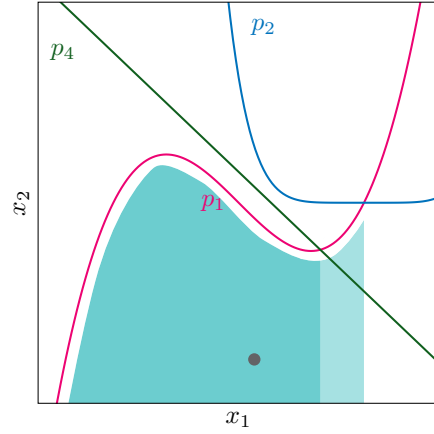
For representing coverings, we can extend Definition 5.8 for the ALL COMPOUND representation in a straight-forward way.

5.4 Under-Approximation of Cells

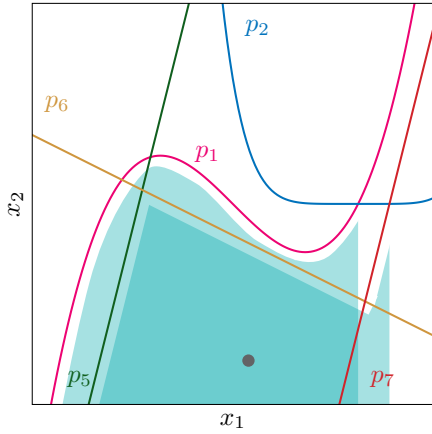
The complexity of the computed projection is caused by the doubly exponential degree growth of the computed resultant and discriminants. We thus propose the idea to insert *roots of additional polynomials* with low degree into the symbolic intervals and indexed root orderings such that they are in between two roots of polynomials of high degree. By transitivity, we still obtain an indexed root ordering which guarantees the desired properties, but it is computationally easier to prove, as the required resultant computations are replaced by simpler ones, at the cost of losing the termination guarantee of the overall algorithm. This is inspired by an intuition from proving theorems by hand: A common technique is to find sufficiently tight bounds on terms which are easier to handle later on.



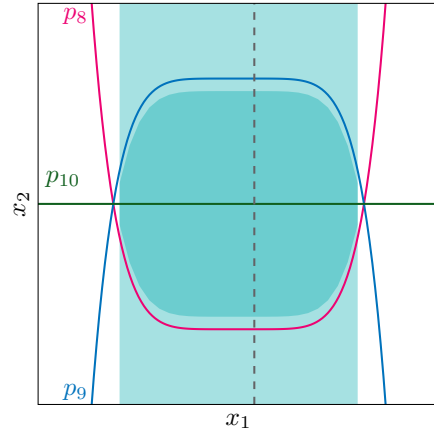
(a) An approximating cell bounded by a constant function.



(b) An additional root between two original roots, derived using Taylor series.



(c) An approximating cell bounded by a piecewise linear function.



(d) Simplified ordering guaranteeing a covering using a constant function.

Figure 5.4: Illustrations for Example 5.1. Examples for under-approximations of cells and coverings. In the first three illustrations, we depict the cell without and with approximation.

Example 5.1

We consider three single cell examples and one covering example. The original polynomials for the first three examples are p_1, p_2 and the sample point $s = (0.125, -1.25)$; the cells of the last example are described by p_8 and $(0.125, 1)$, p_9 and $(0.125, -1)$.

$$p_1 = (x_1 - 1)x_1(x_1 - 1) - x_2$$

$$p_2 = x_1^6 - x_2$$

$$p_8 = x^6 - 1 - x_2$$

$$p_9 = -x^6 + 1 - x_2$$

Figure 5.4a is originally described by the interval $I_2 = (-\infty, \text{root}_{x_2}^{p_1,1})$ and the ordering $\prec = \{(\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_2,1})\}$. By taking a point between s and $\text{root}_{x_2}^{p_1,1}(s_1)$, we can derive a valid constant bound on the cell, e.g. $x_2 < -0.5$. We thus obtain $I'_2 = (-\infty, \text{root}_{x_2}^{p_3,1})$ and $\prec' = \{(\text{root}_{x_2}^{p_3,1}, \text{root}_{x_2}^{p_1,1}), (\text{root}_{x_2}^{p_3,1}, \text{root}_{x_2}^{p_2,1})\}$ where $p_3 = x_2 + 0.5$. The advantage here is that the bound is efficient to compute with, at the cost of a potentially high approximation error (although in this example, the bounds

on x_1 are less tight due to the approximation). Further, we avoid computing the resultant $\text{res}_{x_2}(p_1, p_2)$ (which is of degree 6, but in general could be of degree up to $3 \cdot 6$); instead, we compute two resultants of degree 3 and 6 which are easy to compute, as we simply plug in the bounding constant for x_2 .

Figure 5.4b illustrates that we may also insert additional roots in the ordering only, without modifying the interval. In this case, we take the Taylor series expansion of p_1 at $x_1 = 0.125$ and shift it a bit to obtain p_4 . The new cell is described by $I'_2 = (-\infty, \text{root}_{x_2}^{p_1,1})$ and $\prec' = \{(\text{root}_{x_2}^{p_1,1}, \text{root}_{x_2}^{p_4,1}), (\text{root}_{x_2}^{p_4,1}, \text{root}_{x_2}^{p_2,1})\}$. Again, we replace the resultant $\text{res}_{x_2}(p_1, p_2)$ by two easy-to-compute resultants, as one of the input polynomials is linear.

Figure 5.4c tries to describe the original bound more accurately using a piecewise linear function, described by p_5 , p_6 , and p_7 . For doing so, we evaluate the bound $\text{root}_{x_2}^{p_1,1}$ for different values at the left and right of s to obtain the support points. The resulting cell is described by $I'_2 = (-\infty, B)$ and $\prec' = \{(B, \text{root}_{x_2}^{p_1,1}), (B, \text{root}_{x_2}^{p_2,1})\}$ where $B = \min_ \max_{\{\{\text{root}_{x_2}^{p_5,1}\}, \{\text{root}_{x_2}^{p_6,1}, \text{root}_{x_2}^{p_7,1}\}\}}$. We thus need six easy-to-compute resultants.

Figure 5.4d shows that we can also adapt the ideas for coverings. Originally, the cells covering the cylinders are described by $I_2^1 = (-\infty, \text{root}_{x_2}^{p_9,1})$ and $I_2^2 = (\text{root}_{x_2}^{p_8,1}, \infty)$, which is generalized using $\prec = \{(\text{root}_{x_2}^{p_8,1}, \text{root}_{x_2}^{p_9,1})\}$. By sampling some point between the two root functions, we can replace the ordering by $\prec' = \{(\text{root}_{x_2}^{p_8,1}, \text{root}_{x_2}^{p_{10},1}), (\text{root}_{x_2}^{p_{10},1}, \text{root}_{x_2}^{p_9,1})\}$. Again, the two resultants are easier to compute.

As the possibilities are quite diverse, we focus on two variants which appeared to be most promising in our experiments: The simple approach using constant bounds as in Figure 5.4a, and the approach with piecewise-linear bounds as in Figure 5.4c.

5.4.1 Constant Bounds

In the simple variant, we determine the symbolic interval as in Definition 5.1. In case of a sector, we choose a point between the given sample point and the obtained lower and upper bound respectively. We use these points for determining a new symbolic interval with constant bounds. We define the indexed root ordering as in the BIGGEST CELL heuristic (Definition 5.4). However, for efficiency and termination, two details are important: (1) The points between the sample point and the exact bounds should be *efficient to compute with*. We thus apply techniques that obtain a *simple* rational sample point (i.e. a small bit-width), such as choosing a *mediant* using the *Stern-Brocot tree* [Ste58; Bro61]. (2) It may not pay off to approximate each polynomial, but only ones with a certain degree. Further, for ensuring termination, we need to limit the number of polynomials added. In our experiments, it turned out best to increase the threshold of the degree above which polynomials are approximated with the number of cells. More details are described in [PNÁ25].

Definition 5.12 (CONSTANT APPROXIMATION Representation)

Let i , P , s , Ξ , I_{deg} be as in Definition 5.1.

If there exists some $\xi \in \Xi$ such that $\xi(s_{[i-1]}) = s_i$, the CONSTANT APPROXIMATION representation is defined as Definition 5.3.

Otherwise, let $t = a \cdot C + b$ where $a, b \in \mathbb{Q}$ are constants and $C \in \mathbb{N}$ is some value that increases throughout the run of the algorithm. If $\mathbf{I}_{\text{deg}}.l \neq -\infty$ and $\deg_{x_i}(\mathbf{I}_{\text{deg}}.l.p) \geq t$, let $s' \in \mathbb{Q}$ be a simple rational (e.g. a mediant) such that $\mathbf{I}_{\text{deg}}.l(s_{[i-1]}) < s' < s_i$, and we define $\mathbf{I}.l = \text{root}_{x_i}^{x_i-s', 1}$ (and $\mathbf{I}.l = -\infty$ otherwise). Analogously, we choose $\mathbf{I}.u$.

We define the indexed root ordering $(\approx, <)$ on Ξ as the BIGGEST CELL (Definition 5.4) and the *CONSTANT APPROXIMATION representation* as the tuple $(\mathbf{I}, (\approx, <))$.

In the context of single cell construction and NLSAT, we choose C to be the number of previously generated cells where at least one bound was approximated, and $a = \frac{1}{5}$ and $b = 3$.

5.4.2 Piecewise-linear Bounds

In order to reflect the shape of the approximated bound better, we generate multiple support points around the given sample point. For reasons of practical feasibility, we fix the first $i - 2$ dimensions of the given sample point s , thus reduce the choice of these support points to two dimensions. For that, we need to (1) identify the first dimension of these points, evaluate the bound and determine the corresponding support points; (2) compute a compound root function according to Section 4.7; and additionally (3) apply the techniques for merging adjacent cells described in Sections 4.5 and 4.7 during the projection to generalize the computed bound, as motivated already in Example 4.10.

There are some (technical) challenges: Towards (1), we need to decide how many support points we compute and how to distribute them. These support points should be rational and simple (see Section 5.4.1). As we need to guarantee that the new bound is between s_i and the exact bound, we need to choose s_{i-1} as support point. However, if s_{i-1} is non-rational, the polynomial defining the line between this and the neighbouring support point is not linear; therefore, in this case, we fall back to the constant bounds approach from the previous section. As we need to evaluate the exact bound, we identify an interval I around s_{i-1} such that the polynomial defining the approximated bound is delineable in $s_{[i-2]} \times I$. We can thus choose $c \in \mathbb{N}_{>0}$ simple rational values in I to obtain $c - 1$ pieces describing the piecewise-linear bound, evaluate the exact bound at these points and choose some simple rational value below or above it. In our implementation, we choose $c \in \{3, 5, 7\}$; although we may choose c dynamically depending on the evaluation results, we stick to this simple approach for computational efficiency. Regarding (2), it is not straight-forward to transform the representation by support points to the min-max representation from Section 4.7, which has extensively been studied in [XvD14; XVD⁺16]. We apply the simplified technique as described in [Wag23, Section 3.4]. The challenges for (3) are already discussed in Sections 4.5 and 4.7.

Definition 5.13 (PIECEWISE-LINEAR APPROXIMATION Representation)

Let $i \in \mathbb{N}$, $i \geq 2$, P , s , Ξ , $\mathbf{I}_{\text{deg}}$ be as in Definition 5.1.

If there exists some $\xi \in \Xi$ such that $\xi(s_{[i-1]}) = s_i$, the PIECEWISE-LINEAR APPROXIMATION representation is defined as Definition 5.3. If $s_{i-1} \notin \mathbb{Q}$, the PIECEWISE-LINEAR APPROXIMATION representation is defined as Definition 5.12.

Otherwise, let $t = a \cdot C + b$ where $a, b \in \mathbb{Q}$ are constants and $C \in \mathbb{N}$ is some value

that increases throughout the run of the algorithm. Given some polynomial $p \in P$, let $I_p \subseteq \mathbb{R}$ be the maximal interval that contains s_{i-1} and either does not contain any root of $\text{realRoots}(\text{disc}_{x_i}(p)(s_{[i-2]})) \cup \text{realRoots}(\text{ldcf}_{x_i}(p)(s_{[i-2]}))$, or is a point equal to one of them.

If $I_{\text{deg}.l} \neq -\infty$ and $\deg_{x_i}(I_{\text{deg}.l}.p) \geq t$, let $\tilde{s}^1, \dots, \tilde{s}^{c-1} \in I_{\text{deg}.l}.p \cap \mathbb{Q}$ for some $c \in \mathbb{N}_{>0}$ be simple rationals. Let $\tilde{s}'^1, \dots, \tilde{s}'^{c-1}$ be simple rationals such that $I_{\text{deg}.l}(s_{[i-2]}, \tilde{s}^j) < \tilde{s}'^j$ for $j \in [c-1]$ and $s' \in \mathbb{Q}$ be a simple rational such that $I_{\text{deg}.l}(s_{[i-1]}) < s' < s_i$. Let $I.l$ be the max-min representation built from defining polynomials in x_{i-1} and x_i whose roots connect the support points $(\tilde{s}^1, \tilde{s}'^1), \dots, (\tilde{s}^{c-1}, \tilde{s}'^{c-1}), (s_{i-1}, s')$. If $I_{\text{deg}.l} = -\infty$, let $I.l = -\infty$. Analogously, we choose $I.u$.

We define the indexed root ordering $(\approx, \prec)$ on Ξ as the BIGGEST CELL (Definition 5.4) and the *PIECEWISE-LINEAR APPROXIMATION representation* as the tuple $(I, (\approx, \prec))$.

5.5 Combinatorial Optimization Approach for Single Cells

The discussion in Section 5.1 suggests that there is no simple greedy heuristic that minimizes the degree of the resultants. We now state the choice of the indexed root ordering for single cell construction as a *combinatorial optimization* problem. To that end, we briefly summarize the results from [Per24]. We start by giving the optimization problems formally and giving their complexity, before we discuss the practical aspects.

The general problem is stated as follows:

Definition 5.14 (OPTIMAL GENERAL ORDERING)

Let $i \in \mathbb{N}_{>0}$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_i]$ of level i , $\Xi \subseteq \text{ire}(P, s_{[i-1]})$, and a cost function $c_{\text{res}} : P \times P \rightarrow \mathbb{R}_{\geq 0}$.

Compute $\xi_\ell, \xi_u \in \Xi$ and a relation $\preceq \subseteq \Xi^2$ such that $\preceq^{rt}$ is an ordering on Ξ and $\xi_\ell(s_{[i-1]}) \leq s_i \leq \xi_u(s_{[i-1]})$, and for all $\xi \in \Xi$ it holds $\xi(s_{[i-1]}) \leq s_i \Rightarrow \xi \preceq^{rt} \xi_\ell$ and $s_i \leq \xi(s_{[i-1]}) \Rightarrow \xi_u \preceq^{rt} \xi$ such that $\sum_{p, p' \in P \text{ s.t. } \exists (\xi, \xi') \in \preceq. (\xi.p = p \wedge \xi'.p = p')} c_{\text{res}}(p, p')$ is minimal.

For simplifying the notation, we use a single relation $\preceq$ which can easily be transformed into an (extended) indexed root ordering.

The OPTIMAL GENERAL ORDERING problem is an instance of the STEINER TREE problem, and thus certainly in NP.

Theorem 5.1

OPTIMAL GENERAL ORDERING is NP-hard.

It turns out that OPTIMAL GENERAL ORDERING is NP-hard by reduction from BOOLEAN SATISFIABILITY (SAT); the reduction is given in [Per24].

However, the problem is more general than we need; in Definition 5.2, we already pointed out that we only need to consider the closest roots of each polynomial below and above the given sample point. We thus can simplify the problem as follows:

Definition 5.15 (OPTIMAL ORDERING)

Let $i \in \mathbb{N}_{>0}$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_i]$ of level i , $\Xi \subseteq \text{ire}(P, s_{[i-1]})$ such that

- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $\xi(s_{[i-1]}) < s_i$ and
- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $s_i < \xi(s_{[i-1]})$,

and a cost function $c_{\text{res}} : P \times P \rightarrow \mathbb{R}_{\geq 0}$.

Compute $\xi_\ell, \xi_u \in \Xi$ and a relation $\preceq \subseteq \Xi^2$ such that $\preceq^{rt}$ is an ordering on Ξ and $\xi_\ell(s_{[i-1]}) \leq s_i \leq \xi_u(s_{[i-1]})$, and for all $\xi \in \Xi$ it holds $\xi(s_{[i-1]}) \leq s_i \Rightarrow \xi \preceq^{rt} \xi_\ell$ and $s_i \leq \xi(s_{[i-1]}) \Rightarrow \xi_u \preceq^{rt} \xi$ such that $\sum_{p, p' \in P \text{ s.t. } \exists (\xi, \xi') \in \preceq. (\xi.p = p \wedge \xi'.p = p')} c_{\text{res}}(p, p')$ is minimal.

For this problem, we do not know whether there exists a polynomial-time algorithm yet. However, if we additionally assume that all roots are disjoint, we obtain a polynomial-time algorithm:

Definition 5.16 (OPTIMAL STRICT ORDERING)

Let $i \in \mathbb{N}_{>0}$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_i]$ of level i , $\Xi \subseteq \text{ire}(P, s_{[i-1]})$ such that

- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $\xi(s_{[i-1]}) < s_i$ and
- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $s_i < \xi(s_{[i-1]})$ and
- for all $\xi, \xi' \in \Xi$ it holds $\xi(s_{[i-1]}) \neq \xi'(s_{[i-1]})$,

and a cost function $c_{\text{res}} : P \times P \rightarrow \mathbb{R}_{\geq 0}$.

Compute $\xi_\ell, \xi_u \in \Xi$ and a relation $\preceq \subseteq \Xi^2$ such that $\preceq^{rt}$ is an ordering on Ξ and $\xi_\ell(s_{[i-1]}) \leq s_i \leq \xi_u(s_{[i-1]})$, and for all $\xi \in \Xi$ it holds $\xi(s_{[i-1]}) \leq s_i \Rightarrow \xi \preceq^{rt} \xi_\ell$ and $s_i \leq \xi(s_{[i-1]}) \Rightarrow \xi_u \preceq^{rt} \xi$ such that $\sum_{p, p' \in P \text{ s.t. } \exists (\xi, \xi') \in \preceq. (\xi.p = p \wedge \xi'.p = p')} c_{\text{res}}(p, p')$ is minimal.

We can reduce this problem to MINIMUM-WEIGHT EDGE COVER: A *graph* is a tuple (V, E) of a set V of *vertices* and set $E \subseteq \{\{u, v\} \mid u, v \in V\}$ of *edges*. Let $G = (V, E)$ be a graph. An *edge cover* of G is a subset $E' \subseteq E$ of edges such that each vertex $v \in V$ is incident to at least one edge $e \in E'$, e.g. $v \in e \in E'$.

Definition 5.17 (MINIMUM-WEIGHT EDGE COVER)

Let $G = (V, E)$ be a graph, and $w : E \rightarrow \mathbb{R}_{\geq 0}$ be a weight function. Compute an edge cover $E' \subseteq E$ of G such that $w(E') = \sum_{e \in E'} w(e)$ is minimal.

A polynomial time algorithm is obtained by reduction to MINIMUM-WEIGHT PERFECT MATCHING, as given in [Sch03].

Theorem 5.2

OPTIMAL STRICT ORDERING is in P by a polynomial-time reduction to MINIMUM-WEIGHT EDGE COVER.

The reduction is given in [Per24], and allows for a straight-forward implementation using any algorithm that solves MINIMUM-WEIGHT PERFECT MATCHING.

However, in practice, we need to solve OPTIMAL ORDERING; for now, we may use one of the following “hacks” that use the algorithm for OPTIMAL STRICT ORDERING:

- For each pair of roots $\xi, \xi' \in \Xi$ such that $\xi(s_{[i-1]}) = \xi'(s_{[i-1]})$, we assume fictitiously either $\xi(s_{[i-1]}) < \xi'(s_{[i-1]})$ or $\xi'(s_{[i-1]}) < \xi(s_{[i-1]})$, and call the algorithm OPTIMAL STRICT ORDERING on all resulting combinations to determine the optimal one. Unfortunately, this introduces an exponential blow-up.
- We just try one of the above options, avoiding the exponential blow-up, at the cost of a non-optimal solution. For now, this is our preferred option.

Inspired by Definition 5.5, we want to optimize when to apply the equational constraints projection to ensure sign-invariance of some polynomial. Surprisingly, the extended problem is NP-hard:

Definition 5.18 (OPTIMAL STRICT ORDERING WITH EC)

Let $i \in \mathbb{N}_{>0}$, $s \in \mathbb{R}^i$, $P \subset \mathbb{Q}[x_1, \dots, x_i]$ of level i , $\Xi \subseteq \text{ire}(P, s_{[i-1]})$ such that

- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $\xi(s_{[i-1]}) < s_i$ and
- for each $p \in P$ there is at most one $\xi \in \Xi$ s.t. $\xi.p = p$ and $s_i < \xi(s_{[i-1]})$ and
- for all $\xi, \xi' \in \Xi$ it holds $\xi(s_{[i-1]}) \neq \xi'(s_{[i-1]})$,
- $s_i = \xi(s_{[i-1]})$ for an $\xi \in \Xi$,

and a cost functions $c_{\text{res}} : P \times P \rightarrow \mathbb{R}_{\geq 0}$ and $c_{\text{disc}} : P \rightarrow \mathbb{R}_{\geq 0}$.

Compute $\xi_b \in \Xi$, $P_{\text{eq}} \subseteq P$ and a relation $\preceq \subseteq \{\xi \in \Xi \mid \xi.p \notin P_{\text{eq}}\}^2$ such that $\preceq^{rt}$ is an ordering on Ξ , and $\xi_b(s_{[i-1]}) = s_i$, and for all $\xi \in \Xi$ it holds either $(\xi(s_{[i-1]}) \leq s_i \Rightarrow \xi \preceq^{rt} \xi_b$ and $s_i \leq \xi(s_{[i-1]}) \Rightarrow \xi_b \preceq^{rt} \xi)$ or $\xi.p \in P_{\text{eq}}$ such that $\sum_{p, p' \in P \text{ s.t. } \exists (\xi, \xi') \in \preceq. (\xi.p = p \wedge \xi'.p = p')} c_{\text{res}}(p, p') + \sum_{p \in P_{\text{eq}} \text{ s.t. } \xi_b.p \neq p} c_{\text{res}}(\xi_b.p, p) + \sum_{p \in P \setminus P_{\text{eq}}} c_{\text{disc}}(p)$ is minimal.

Theorem 5.3

OPTIMAL STRICT ORDERING WITH EC is NP-hard.

The NP-hardness is shown by reduction from BOOLEAN SATISFIABILITY (SAT) as given in [Per24].

At this point, we leave the choice of the cost function $c_{\text{res}} : P \times P \rightarrow \mathbb{R}_{\geq 0}$, which estimates the cost of the resultant of two polynomials, open. Assume e.g. we take the product of the (total) degree of the two input polynomials to estimate the (total) degree of the resultant, constant 1 to minimize the number of resultants, or some estimate for the number of monomials. We will elaborate on some choices in Chapter 7 where we describe our implementation.

Definition 5.19 (C-OPTIMAL CELL Representation)

Let C denote a cost function.

The *C-OPTIMAL CELL representation* is defined as the result of solving the corresponding OPTIMAL ORDERING problem using the cost function specified by C .

Chapter 6

Variants of Cylindrical Algebraic Covering

The exploration-guided algorithms presented in Chapter 3 define a scheme for exploring the search space. To some degree, they allow heuristics to steer the order in which its parts are visited, e.g. by choosing sample points. Still, deeper modifications allow for greater flexibility, think e.g. of replacing the recursion by a loop and a queue, which allows to “jump” between branches of the search tree. Although we do not fully exploit all possibilities in the scope of this thesis, we present two interesting variants of the CAIC algorithm: An incremental version that is suitable for CDCL(T) and thus changes the way how the Boolean and theory level interact, and a variant that defers lifting over non-rational numbers.

Origins The idea of Section 6.1 stems from the Bachelor’s thesis of Kroll [Kro20]. Section 6.2 is a summary of the Bachelor’s thesis of Kolbin [Kol25].

6.1 Incremental CAIC for CDCL(T)

For SMT solving, we may embed the CAIC algorithm as theory solver into CDCL(T), which is implemented by almost all SMT solvers. The theory solver is only given a conjunction of constraints and is asked to find a solution if it exists, which simplifies the CAIC algorithm. However, the theory solver is called throughout the solving process, each time with an updated set of constraints. For efficiency, we thus aim to maintain a state of the CAIC algorithm which stores the intermediate results from the previous calls. We update this state incrementally when a new constraint is added or some constraint is removed, following ideas in [Kro20].

We adapt the algorithm as follows: We maintain a sequence $(\mathcal{C}_1, \dots, \mathcal{C}_n)$ of sets $\mathcal{C}_i$ of implicit cells (defined by polynomials of level i), a solution s which is initialized to $(0, \dots, 0)$, and a flag f which is *SAT* or *UNSAT*. We update this state as follows:

- If $f = \text{SAT}$ and a constraint $p \sim 0$ of level i is *added*, we determine all implicit cells $(\{p\}, (s_{[i-1]}, s^1)), \dots, (\{p\}, (s_{[i-1]}, s^k))$ on which $p \sim 0$ evaluates to *false*. If no implicit cell contains s , we add them to $\mathcal{C}_i$ and are done.

Otherwise, we run the CAIC algorithm for existentially quantified variables on the set of received constraints, initializing the implicit cells $\mathcal{C}_1, \dots, \mathcal{C}_i$ on levels $1, \dots, i$

respectively, and choosing the sample $(s_1, \dots, s_{[i-1]})$. If the algorithm terminates with a solution, we set s to the solution, and $\mathcal{C}_1, \dots, \mathcal{C}_n$ to the algorithm's last state. If the algorithm concludes unsatisfiability, we set $f = \text{UNSAT}$, and erase s and $\mathcal{C}_1, \dots, \mathcal{C}_n$. Moreover, to compute an infeasible subset, during the run of the CAIC algorithm, we store for each implicit cell its *origins*, i.e. the set of input constraints that we used to derive it.

- If $f = \text{SAT}$ and a constraint $p \sim 0$ of level i is *removed*, we remove all implicit cells from $\mathcal{C}_1, \dots, \mathcal{C}_n$ whose set of origins contains $p \sim 0$.
- If $f = \text{UNSAT}$ and a constraint $p \sim 0$ of level i is *added*, we do nothing.
- If $f = \text{UNSAT}$ and a constraint $p \sim 0$ of level i is *removed*, we rerun the CAIC algorithm on the set of received constraints.

This approach avoids recalculations as long as constraints are added. However, when we remove constraints, we may need to recalculate parts of the covering tree (as we delete such information once we have projected a cylinder covered by implicit cells). Avoiding this, however, would require storing the whole covering tree in memory, which is practically infeasible. However, we update the internal state only after a decision level in CDCL is fully propagated, reducing also the amount of recalculations.

6.2 Exploration Heuristic for CAIC

In the computation tree of the CAIC, some branches may be harder to compute than other branches. In many applications, we do not need to generate a full covering of $\mathbb{R}^n$, for example, when checking the satisfiability of some formula, the algorithm may terminate as soon as it finds a solution of the input formula. It may thus be desirable to postpone branches of the computation tree which are expensive to compute, in the hope that we do not need to compute it at all.

A criterion for expensive branches is inspired by the open CAD (Section 3.2.11): Exploring the search space above a section is expensive, as we are often forced to choose a non-rational sample point, which makes evaluating a polynomial and the real root isolation costly.

We thus modify the CAIC algorithm as follows:

- Whenever we decide to postpone the exploration of a subtree of the computation tree, we define an implicit cell that describes an unexplored part of the search space. For this, we use the bounds of the cells corresponding to the neighbouring parts of the search space. Assume e.g. that the current sample is s , and we already explored two branches which result in implicit cells with symbolic intervals $(P^1, (s, s^1), (-\infty, \text{root}_{x_i}^{p_1, k_1}))$ and $(P^2, (s, s^2), (\text{root}_{x_i}^{p_2, k_2}, \infty))$, and we decide that we postpone the exploration of the search space between $\text{root}_{x_i}^{p_1, k_1}(s)$ and $\text{root}_{x_i}^{p_2, k_2}(s)$. We thus choose a sample s^3 between the two points, and use $(\{p_1, p_2\}, (s, s^3), (\text{root}_{x_i}^{p_1, k_1}, \text{root}_{x_i}^{p_2, k_2}))$ to describe the unexplored part. There are various possible criteria for postponing parts of the search space this way. Our criterion of interest is to postpone whenever two neighbouring intervals cover everything but a single point; in the above example when $\text{root}_{x_i}^{p_1, k_1}(s) = \text{root}_{x_i}^{p_2, k_2}(s)$ holds. Then,

s^3 potentially is non-rational, leading to expensive computations. The resulting strategy is to explore everything except the point intervals, which are postponed to later.

- By omitting parts of the search space, some cylinders are partially explored. We need to compute a projection of such cylinders to continue the search on the levels below. For that, we simply use the usual covering projection with the set of explored and unexplored cells as input.
- As we may need to go into the unexplored cells (or partially explored cylinders) later in the search process, we need to revisit the path leading to them. We thus need to store the subtrees of the search tree that contain unexplored cells. However, as before, we can still forget the subtree of the explored cells. Thus, there is a trade-off between leaving parts of the search space unexplored and the memory usage.
- At a later stage, we may decide to revisit unexplored parts of the search space. In that case, we replace the description of the unexplored cell by one or more descriptions of (explored or partially explored) cells. Recursively, we need to recompute the projection of the corresponding cylinder. As this projection may also be smaller or larger than the old projection, we need to fill the gaps on the levels below, by either introducing new unexplored cells, or exploring the corresponding parts of the search space.

The Bachelor's thesis of Kolbin [Kol25] describes an instantiation of these ideas for postponing the exploration of sections.

Chapter 7

Implementation and Experiments

Throughout this thesis, we presented a wide range of algorithms, projection rules and heuristics. In this chapter, we aim to evaluate all of these aspects to give an overview about which of them perform best on the available practical benchmarks sets and possibly give directions for future research. We first describe our implementation in Section 7.1 and the available benchmark sets and the state-of-the-art in Section 7.2. We compare the different algorithms based on our implementation with respect to different aspects in Section 7.3, and evaluate the different optimizations and heuristics of the proof system in Section 7.4.

Origins Although we also evaluate results from already published work, all experiments in this chapter were re-done for better comparability. Partially, even the implementation differs. Thus, the experimental results may differ from the corresponding papers.

Acknowledgements Computing resources were granted by RWTH Aachen University under project `rwth1560`.

7.1 Implementation

All the algorithms presented in this thesis are implemented in the SMT solver **SMT-RAT** [CKJ⁺15]. The solver consists of a collection of modules (e.g. preprocessing, SAT solver, or cylindrical algebraic decomposition, etc) and strategies which compose instantiations of these modules (with different settings) to a solver. Based on the input formula, the strategy may use different modules for solving the formula. This modular design not only enables the customization of the solver, but also the evaluation of different algorithms and heuristics.

SMT-RAT provides a SAT solving module based on **MiniSat** [ES04] adapted for CDCL(T) [CKJ⁺15]. Theory solvers for CDCL(T) relevant for NRA are *simplex*, *interval constraint propagation*, *virtual substitution*, *cylindrical algebraic decomposition* [KÁ20; Kre20], and *subtropical satisfiability* [NÁ23]. These modules are called in that order (e.g., the next module is invoked only if an incomplete method could not find an answer); they work incrementally and potentially share information about the solution space. Similarly, **SMT-RAT** provides a module extending **MiniSat** for MCSAT [Kre20]. Available explanation backends are based on *Fourier-Motzkin*, *virtual substitution* [ÁNK17],

interval constraint propagation [Kre20], *refinement-based single-cell construction* [BK15], and *NLSAT's model-based projection* [Jd12]. The explanation methods are called in that order (e.g., the next one is called whenever some incomplete method could not explain the conflict), are non-incremental and do not share information. It supports various heuristics for variable assignment, including the feature-based **xai** ordering [PdE⁺24] and an extension of the VSIDS for theory variables as described in [NKÁ19]. For algebraic computations, **SMT-RAT** uses **LibPoly** [JD17] (for most CAD operations) and **CoCoALib** [AB10] (primarily for factorization).

In the scope of this thesis, we extended **SMT-RAT** by the following components:

- A library implementing the *proof system* from Chapter 4 and the heuristics described in Chapter 5,
- an *explanation backend for MCSAT* using single cell construction and the proof system,
- a *CDCL(T)-based CAIC theory solver* using the proof system, and
- a library providing implementations of the *CAIC* and *NuCAD* algorithms using the proof system, supporting quantifiers and quantifier elimination, and using the implicant computations from Section 3.6 instead of a SAT solver.

Although the default strategy of **SMT-RAT** applies basic preprocessing on quantifier-free input formulas before the main algorithms are called, for most of our experiments, we do not apply preprocessing.

7.1.1 Proof System

The proof system implementation consists of two main parts: The application of the rules on a set of properties, and the heuristics that determines the representation (e.g. the intervals and indexed root orderings) for given sets of properties. Towards the first, we consider three sets of proof rules:

DEL The basic proof rules given in Sections 4.2 to 4.4 and 4.7.

LoDEL Like **DEL**, but using additionally the rules from Sections 4.5 to 4.7.

PRDEL Like **DEL**, but using additionally the rules from Section 4.8.

We implement the following variants for describing a cell, which rely on the set of proof rules given in the brackets:

BC (DEL) Uses the **EQUATIONAL CONSTRAINT** (Definition 5.3) and **BIGGEST CELL** representations (Definition 5.4).

LDB (DEL) Uses the **LOWEST DEGREE BARRIERS** (Definition 5.5) representation.

CHAIN (DEL) Uses the **EQUATIONAL CONSTRAINT** (Definition 5.3) and the **CHAIN** (Definition 5.6) representation.

FULL (DEL) Uses the **FULL** (Definition 5.7) representation.

COMPOUND (DEL) Uses the ALL COMPOUND (Definition 5.11) representation.

APXCONST (DEL) Uses the CONSTANT APPROXIMATION (Definition 5.12) representation with $a = 1/5$ and $b = 3$.

APXPWL^c (LODEL) Uses the PIECEWISE-LINEAR APPROXIMATION (Definition 5.13) representation with $a = 1/5$, $b = 3$, and $c \in \{3, 5, 7\}$.

OPT^c (DEL) Uses the c -OPTIMAL CELL representation (Definition 5.19), where c is some cost function. Let $p, q \in \mathbb{Q}[X][x]$ where X is a set of variables and $x \notin X$ a variable. We define the following cost functions:

DEGREE As the degree of the resultant is bounded by $2 \max(\deg(p), \deg(q))^2$ [ED16], we use $c(p, q) = 2 \max(\deg(p), \deg(q))^2$.

SOTD The work in [DSS04] identifies the sum of total degrees of a set of polynomials as predictor for the number of cells in a CAD. We thus define $c(p, q) = \text{tdeg}(p) + \text{tdeg}(q)$.

NUMVAR We aim to predict the number of variables of the resultant and define $c(p, q)$ to be the number of variables from X that occur in p and q .

NUMRES To minimize the number of resultants used, we simply set $c(p, q) = 1$.

BCLDALL (LODEL) We implement a full check for irrelevant roots from Section 4.5 and in particular Rule 4.22: For each root of a resultant that is inside the delineable interval of the two originating polynomials, we isolate their roots to determine whether there is an irrelevant intersection as specified by the root ordering to be maintained.

BCLDINDEP (LODEL) As BCLDALL, but we only check the roots of resultants which are irreducible. Further, we only make use of local delineability if all roots of a given resultant are not relevant; otherwise, we just require the order-invariance of that resultant. These are exactly the cases analogous to Example 4.6 where we do not need to compute iterative resultants involving the given resultant.

BCLDRAT (LODEL) As BCLDALL, but we only check whether a root is irrelevant if it is rational and the underlying sample point does not contain a non-rational number. We thus focus on easy-to-compute checks, as root isolation of polynomials above non-rational samples is computationally expensive.

BCLDBOUNDS (LODEL) We do not check for irrelevant roots, but check whether a bound of a symbolic interval can be closed using weak root orderings as described in Section 4.6. We restrict to cases where no two root functions intersect over the current sample, as in those cases, there will be a section on the level below. In those cases, we apply the EQUATIONAL CONSTRAINT (Definition 5.3) representation.

BcPd (PRDEL) We use projective delineability (Section 4.8) to reduce the number of leading coefficients. We compute the BIGGEST CELL representation as if every polynomial would be delineable, and check which leading coefficients are required: We omit leading coefficients of those polynomials that have a root on both sides of the cell, and of those for which we computed a resultant with a polynomial that has roots on the “other” side (see also the discussion in Section 4.8.5).

LDBPd (PrDEL) Same as BCPd, but uses the LOWEST DEGREE BARRIERS representation.

We implement the following variants for describing a covering:

BCCov (DEL) Uses the BIGGEST-CELL-COMPOSED GREEDY-MIN-CELLS-COVERING (Definitions 5.3, 5.4, and 5.8) representation.

LDBCov (DEL) Uses the LOWEST-DEGREE-BARRIERS-COMPOSED GREEDY-MIN-CELLS-COVERING (Definitions 5.5 and 5.8) representation. In the implementation, we greedily avoid inserting unnecessary resultants by checking whether the order of two root functions is already guaranteed by transitivity before adding the corresponding pair to the relation.

CHAINCov (DEL) Uses the CHAIN GREEDY-MIN-CELLS-COVERING (Definition 5.9) representation.

FULLCov (DEL) Uses the FULL GREEDY-MIN-CELLS-COVERING (Definition 5.10) representation.

COMPOUNDCov (DEL) Uses the ALL-COMPOUND-COMPOSED GREEDY-MIN-CELLS-COVERING (Definitions 5.8 and 5.11) representation.

BCLD*Cov, $*$ $\in \{\text{ALL}, \text{INDEP}, \text{RAT}, \text{BOUNDS}\}$ (LoDEL) Analogous to the cell variants.

***PdCov, $*$ $\in \{\text{Bc}, \text{LDB}\}$ (PrDEL)** Analogous to the cell variants.

7.1.2 MCSAT

Our MCSAT implementation supports various tuning options; here, we mention the most important ones.

Variable Decisions A crucial choice is the order in which the variables are decided. Note that each theory literal has an associated Boolean variable.

THFEATURE This heuristic first decides all original Boolean variables according to an (arbitrary) fixed ordering, and theory variables according to an ordering computed at the beginning of the algorithm which is never changed. The ordering of the theory variables is the `xai` ordering taken from [PdE⁺24].

VSIDS A heuristic which extends the VSIDS from SAT solving by tracking activities for the theory variables and takes the (Boolean or theory) variable with the highest activity whenever a decision needs to be made. The heuristic and its technical implications are described in [NKÁ19].

When assigning a theory variable, we choose the value according to the following scheme: If possible, we take 0. Otherwise, if possible, we choose an integer below all infeasible values. Otherwise, if possible, we choose an integer above all infeasible values. Otherwise, we choose a sample point between existing feasible intervals; again, we prefer integers or nice rational numbers if possible, as choosing algebraic numbers leads to expensive computations.

Explanation Variants We implement Algorithm 3.18 with support for the different proof system variants described in Section 7.1.1. Further, besides the plain MCSAT variant, the MCSAT-Comb variant calls *Fourier-Motzkin*, *virtual substitution*, and *interval constraint propagation* before calling Algorithm 3.18.

Naming Convention To name the different variants, we write the used heuristics as parameter in diamond brackets, e.g. MCSAT<THFEATURE,BC,BcCOV> is the MCSAT variant using THFEATURE for variable orderings, BC for describing cells, and BcCOV for describing coverings. If we leave out a parameter, we assume THFEATURE, BC, and BcCOV as default respectively.

7.1.3 CAIC and NuCAD

Our CAIC and NuCAD implementations share large parts.

Variable Decisions As our implementation does not support dynamic variable orderings, we only consider the THFEATURE ordering. To adapt this ordering for quantified formulas in prenex normal form, we sort each quantifier block separately. The values for variables to be decided are analogous to the MCSAT implementation.

Implicants Experience from [NKÁ19], where disabling Boolean decisions in our MCSAT implementation performed best, led to the assumption that Boolean reasoning might make unfavourable decisions for non-linear arithmetic problems. Usually, the algebraic part is harder to solve than the Boolean structure (if it is not too complex). To investigate the impact of the “algebraic” complexity on the running time, we implement three variants for computing implicants (Section 3.6) in a (naive) straight-forward way: We compute all possible implicants and choose the best implicant afterwards (see below).

NOOP We take all polynomials from the formula as implicant.

EVAL Implementation of the evaluation-only variant.

PROP Implementation of the Boolean-propagation variant. In a preprocessing step, we add clauses like $\neg(p < 0) \vee \neg(p > 0)$ to facilitate Boolean propagations: Without this clause, if $p < 0$ would be assumed to be true, although we would conclude that $p \geq 0$ cannot hold by Boolean reasoning, we would not be able to do so for $p > 0$ causing additional effort in the theory solving.

Although we transform the input formula to prenex normal form, we do not eliminate Boolean operators such as exclusive-or and the like, but extend the implicant computation to support these operators. After we compute a set of implicants using one of the above variants, we take the implicant with the minimal sum of total degrees, i.e. the sum of total degrees of all monomials of all polynomials in the implicant. The work in [DSS04] suggests that this predicts the size and computation time of a CAD. We break ties using the size of the implicant.

Algorithm Variants We denote the plain CAIC algorithm from Section 3.4 by CAIC, and the NuCAD algorithm from Section 3.5 by NuCAD. Additionally, we denote the incremental theory solver for CDCL(T) based on CAIC from Section 6.1 by CAIC-Inc.

Naming Convention `CA1C<PROP,BC,BCCOV>` is the `CA1C` variant using `PROP` for the implicant selection, `BC` for describing cells, and `BCCOV` for describing coverings (the first two parameters are not applicable for `CA1C-Inc`). If we leave out a parameter, we assume `PROP`, `THFEATURE`, `BC`, and `BCCOV` as default respectively.

7.1.4 Omitted Variants from this Evaluation

This thesis’ evaluation gives an overview about the basic techniques. For reasons of conciseness, we do not include an evaluation of the variants presented in Sections 3.7.3 and 6.2.

7.2 Benchmarks and State of the Art

For the evaluation of our algorithms, we compare their running times and other metrics to observe their behaviour on a *benchmark set* of formulas (*instances*). Ideally, such a benchmark set is representative for all the formulas the algorithm would ever be applied on. The author of this thesis conjectures that solvers need to specialize on certain formulas in order to exploit their “structural” information – an intuition given by the *no free lunch theorems for search and optimization* [WM95; WM97] (although the author does not know how to formalize this intuition). As a consequence, we can only conclude about the practical usability of an algorithm by observing its performance on formulas that stem from practical applications or are handcrafted by humans – randomly generated instances are less meaningful (although they share some structural information as they are generated according to some scheme and are thus not “truly” random).

Fortunately, the SMT community maintains a repository of benchmark sets called *SMT-LIB*. From its 2024 release [PSB⁺24], we use the `QF_NRA` benchmark set containing 12 154 quantifier-free non-linear real arithmetic formulas and the `NRA` benchmark set containing 3819 quantified non-linear real arithmetic formulas. The former consists of 16 *families*, consisting of instances that usually share common characteristics as they either stem from the same application or encode the same problem with different parameters; about half of the instances stem from a single family, which clearly indicates a bias in the benchmark set. The latter benchmark set consists almost only of a single family. For quantifier elimination, no such community-maintained collection exists; the only ready-to-use collection `QE` of 78 handcrafted instances is provided by the *CAD example bank* from Bath [Wil13] (which we converted to the SMT-LIB format). These instances encode mathematical problems, but do not include further (industrial) applications.

Clearly, these benchmark sets are not quite representative, harbouring the danger of “overfitting” the solver heuristics to them, especially due to the annual SMT competitions. On the other hand, as long as the availability of diverse benchmarks is sparse, there is no better approach.

7.2.1 Characteristics of the Benchmarks

We analyze the structure of the instances in the `QF_NRA` benchmark set, omitting `NRA` and `QE` due to their limited diversity. For this reason, most of our evaluation is based on `QF_NRA`. For our analysis, we extract various features from the benchmark set using

Table 7.1: The maximum, mean, and median values of the features. The indented features are correlated with the preceding feature.

	max	mean	25%	median	75%
# nodes	664211	3519.06	47	80	1039.25
# real vars	140547	564.52	3	3	31
# polynomials	248725	1118.17	7	9	99
# monomials	544901	2459.41	17	31	276
Σ degrees	248725	1140.47	11	17	128
Σ total degrees	266461	1339.81	13	21	151.50
depth	158	15.81	6	8	14
# bool vars	1689	15.43	0	0	0
max degree	94	4.77	2	3	6
max total degree	95	5.87	2	4	7
Σ tot. deg. mon.	573191	3212.34	28	80	422

pySMT [GM15]; for 314 instances, this failed due to the limited scalability of pySMT. The most important features are:

- number of nodes of the directed acyclic graph (DAG) that represents the formula (*# nodes*),
- depth of the formula’s DAG ignoring constraints (*depth*),
- number of Boolean/real-valued variables (*# bool vars*, *# real vars*),
- number of polynomials/monomials (*# polynomials*, *# monomials*),
- sum/maximum of the (total) degrees of the polynomials (*Σ degrees*, *max degrees*, *Σ total degrees*, *max total degrees*),
- sum of total degrees of the monomials (*Σ tot. deg. mon.*).

Most of these features are correlated with the number of nodes (Pearson coefficient > 0.95), except the depth, number of Boolean variables, the maximum of the (total) degrees of the polynomials, and the sum of total degrees of the monomials. The maximum of the degrees of the polynomials and maximum of the total degrees of the polynomials are correlated as well. We thus group the features as given in Table 7.1. We depict the distribution of the important features in Figure 7.1.

We clearly see that the vast majority of the instances is relatively small, only for about 1000 instances, the feature values are growing. This holds in particular for the degree-based features, which we examine further, as these features seem particularly relevant in the context of CAD: Figure 7.2a compares the maximal degree with the maximal total degree. These numbers are relatively close on many instances, thus only a single variable has a high degree while the others have relatively small degrees. Figure 7.2b compares the sum of degrees of all polynomials with the maximal degree of all polynomials. On some instances with small sum of degrees, the maximal degree is relatively high, suggesting that most polynomials have low degrees and only few polynomials have high degrees. On instances with high sum of degrees, the maximal degree is low, suggesting that these instances consist of many low-degree polynomials.

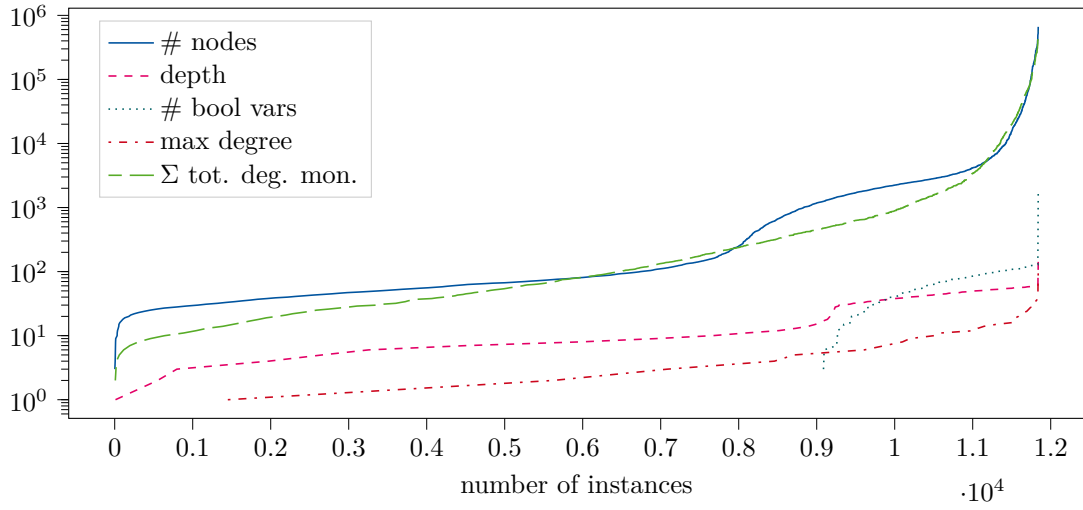
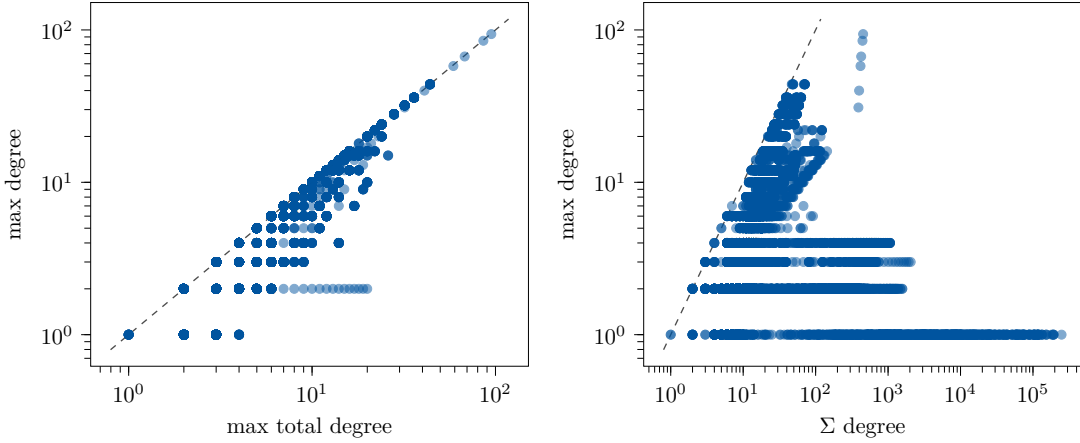


Figure 7.1: Survival plot of key features. Each line represents the number of instances (x-axis) with a feature value less than or equal to the corresponding value on the y-axis.



(a) Maximal degree and maximal total degree.

(b) Maximal degree and sum of degrees.

Figure 7.2: Comparison of degrees. Each point in the scatter plot represents an instance.

We conclude that most of the instances have a favourable structure, and few instances seem quite hard to solve. Further, [dE24] observed that the set is imbalanced, as a lot of instances are similar; we do not propose any balancing here, but should keep this fact in mind when evaluating our algorithms.

7.2.2 Performance of State-of-the-Art Tools

For further analysis, we give a quick evaluation of the state-of-the-art tools, along with the best version of our tool **SMT-RAT** (which uses pre-processing, subtropical satisfiability as in [NÁ23], and MCSAT for quantifier-free formulas; and CAIC without pre-processing for quantified formulas), on the benchmark sets:

- **QF_NRA**: **z3** (version 4.13.4), **cvc5** (version 1.2.0), **yices2** (version 2.6.4), **SMT-RAT**; as **z3** is a portfolio solver (which the others are not), we also consider **z3-non-port** which runs its NLSAT implementation with the default configuration;

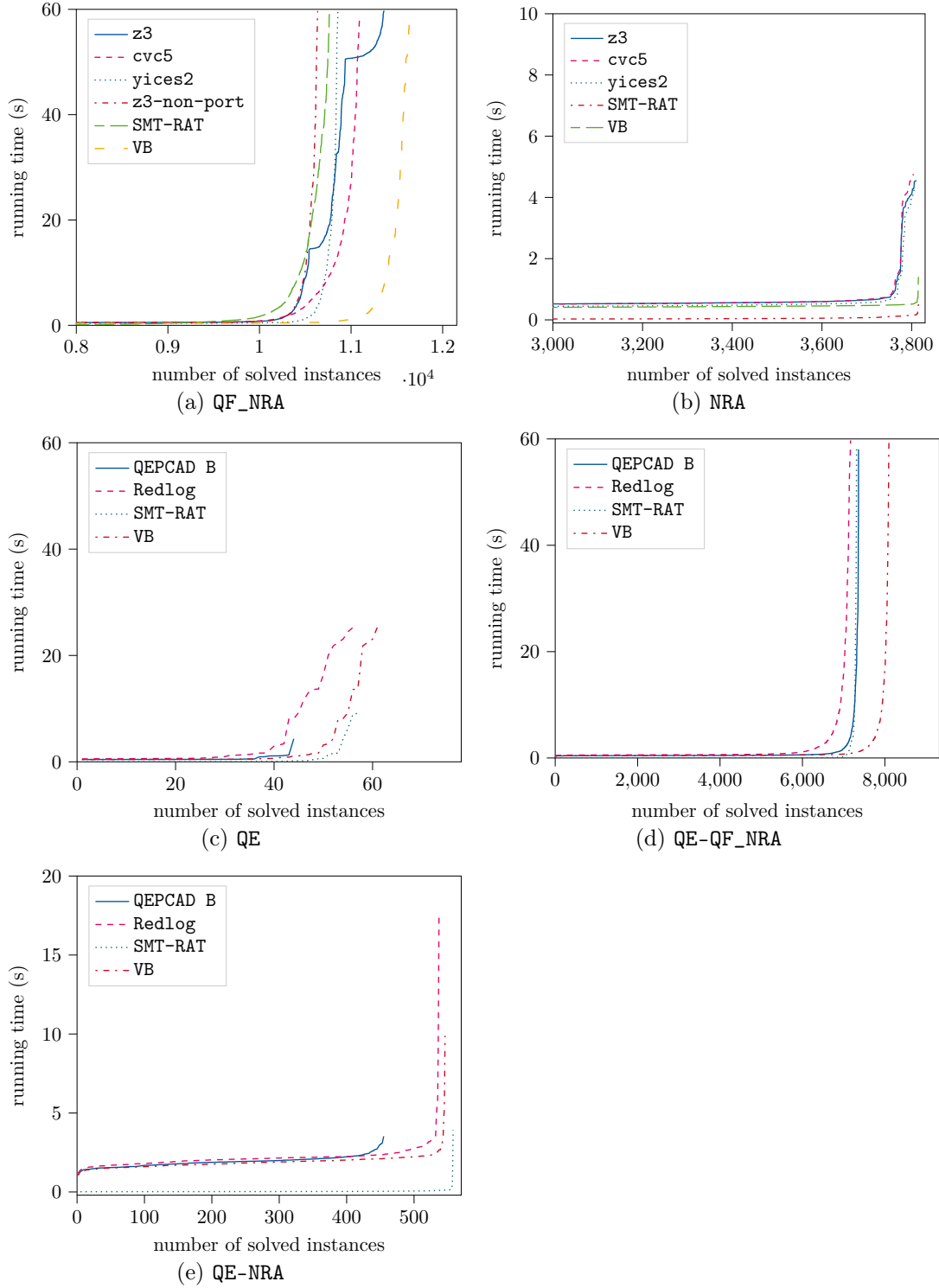


Figure 7.3: Performance profiles of solvers on the benchmark sets. A point on a line determines how many instances (x -axis) were solved by the solver in the given time (y -axis).

- **NRA**: `z3`, `cvc5`, `yicesQS` (commit 4661444), **SMT-RAT**;
- **QE**: **QEPCAD B** through **Tarski** (version 1.28), **Redlog** (commit 6658), **SMT-RAT**
- **QE-QF_NRA**, where we run a quantifier elimination tool to describe the solution formula in all variables of **QF_NRA**: **QEPCAD B**, **Redlog**, **SMT-RAT**;
- **QE-NRA**, where we run a quantifier elimination tool to describe the solution formula in all free variables of **NRA**: **QEPCAD B**, **Redlog**, **SMT-RAT**.

We use `pySMT` for translating the **SMT-LIB** input format to **QEPCAD B** and **Redlog**, which fails in some cases due to non-supported syntax (mostly rational functions); we thus restrict the comparison to **QE-QF_NRA** and **QE-NRA** when necessary. We also verify the correctness of our implementation wherever possible: On **QF_NRA** and **NRA**, for most instances, the expected result is known; we compare our output accordingly. Regarding quantifier elimination, we use **QEPCAD B** to check the equivalence of the original formula with the solution formula generated by our tool; other tools are not suitable, as only **QEPCAD B** (respectively **Tarski**) is able to reason about indexed root expressions, which are contained in the output of our tool.

We run all experiments in this chapter on Intel®Xeon®8468 Sapphire CPUs with 2.1 GHz per core with a time limit of 60 seconds and memory limit of 4 GB per instance.

Instructions for running the experiments and raw results are available at <https://doi.org/10.5281/zenodo.16082393>.

Figure 7.3 depicts the results of the solvers on the different benchmark sets. We added a *virtual best* (**VB**) solver that is a virtual portfolio of all non-**SMT-RAT** solvers that always chooses the fastest solver for a given instance.

Regarding **QF_NRA**, we observe that the solvers complement each other to some extent, given that **VB** solves the most instances. Further, **z3** is particularly successful due to the portfolio approach, compared to the results of **z3-non-port**. The slope of the lines suggests that a solver either solves some instance in short time or fails to solve it at all. We may conclude from these observations that the solvers do “lucky” guesses by tailored heuristics and avoid heavy algebraic computations. Only for `cvc5`, the slope is slightly flatter, possibly due to the linearization techniques applied by the solver.

Regarding the benchmarks involving quantifiers, we observe that there is a need for interesting benchmarks, particularly the **NRA** benchmarks are mostly trivial.

7.2.2.1 Analysis of Features on **QF_NRA**

One might be curious how the features determine the running time of **VB** on an instance. Figure 7.4 depicts the two most striking features: The number of nodes seems to have some impact; starting from a certain number of nodes, the running times vary significantly. The maximal degree does not have a meaningful impact; the instances that cannot be solved have a low degree. The fact that all high-degree instances are solved almost instantly means that the given instances are easy to solve for other reasons.

7.2.2.2 Solver Behaviour

To support our claims about the difficulty of the benchmark set, we analyze the behaviour of **SMT-RAT**, `cvc5`, and `z3` on **QF_NRA**.

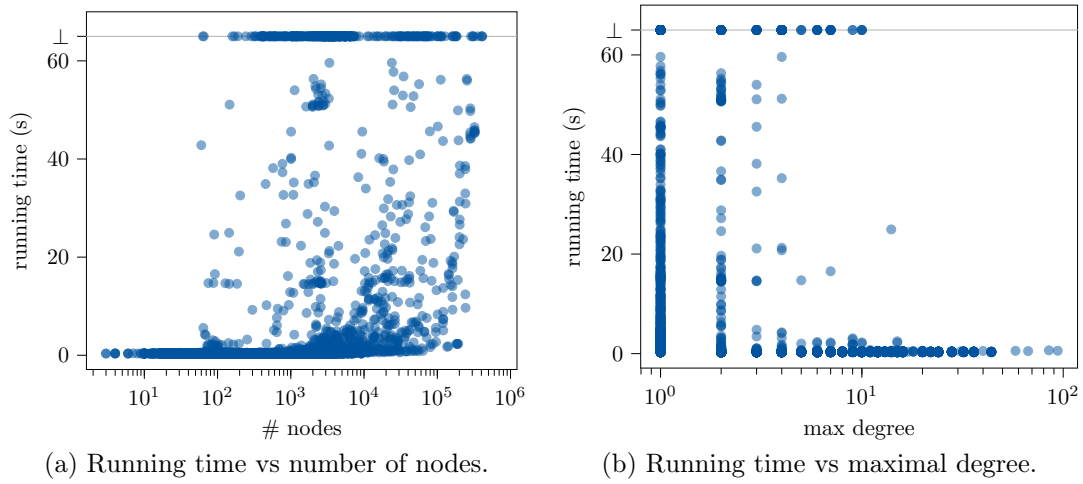


Figure 7.4: Scatter plots of the running time vs some features. Each point represents an instance; points on the $\perp$ line represent a time out of VB on that instance.

For **SMT-RAT**, of the 10 765 instances it solved, 2867 (27%) were solved during pre-processing, 1455 (14%) were solved by the subtropical method, 5855 (54%) were solved using MCSAT but without a call to a CAD-based explanation method, and only 588 (5%) were solved using CAD-based methods.

Similarly, for **cvc5**, of the 11 088 instances it solved, 9086 (82%) were solved without CAD, and only 2002 (18%) were solved using a CAD-based method. For **z3**, of the 11 360 instances it solved, 4757 (42%) were solved without CAD, 6603 (58%) were solved using a CAD-based method, and only 2635 (23%) called the CAD more than 10 times (see also [Kre20, Section 2.6.1]).

We conclude that the use of CAD is relatively low, as most of the instances can already be solved by pre-processing, Boolean reasoning, or relatively efficient incomplete methods.

7.3 Evaluation of Algorithms

7.3.1 Algorithms for Quantifier-free Formulas

We compare the **MCSAT**, **MCSAT-Comb**, **MCSAT<VSIDS>**, **CA1C**, **CA1C-Inc**, and **NuCAD** algorithms in **SMT-RAT** on the **QF_NRA** benchmark set in Figure 7.5a, noting that we do not use pre-processing, and that all algorithms use a comparable variable ordering (except **MCSAT<VSIDS>** which is included for comparison purposes as it is the one used by **SMT-RAT**).

Clearly, **MCSAT** and **MCSAT-Comb** outperform the other solvers. **MCSAT<VSIDS>** performs slightly worse — however, when restricting to instances containing Boolean variables, it solves 85 instances more than **MCSAT**. As **MCSAT** and **CA1C** are more successful than **CA1C-Inc**, it seems that **CDCL(T)** is less well-suited for NRA problems due to the missing theory guidance used by the other approaches. **NuCAD** is far behind **CA1C**, which we will analyze below on the quantifier elimination results. **MCSAT** outplays **CA1C**, although **CA1C** solves 102 instances not solved by **MCSAT** (the other way round is 300); we thus analyze their different behaviour in the following sections.

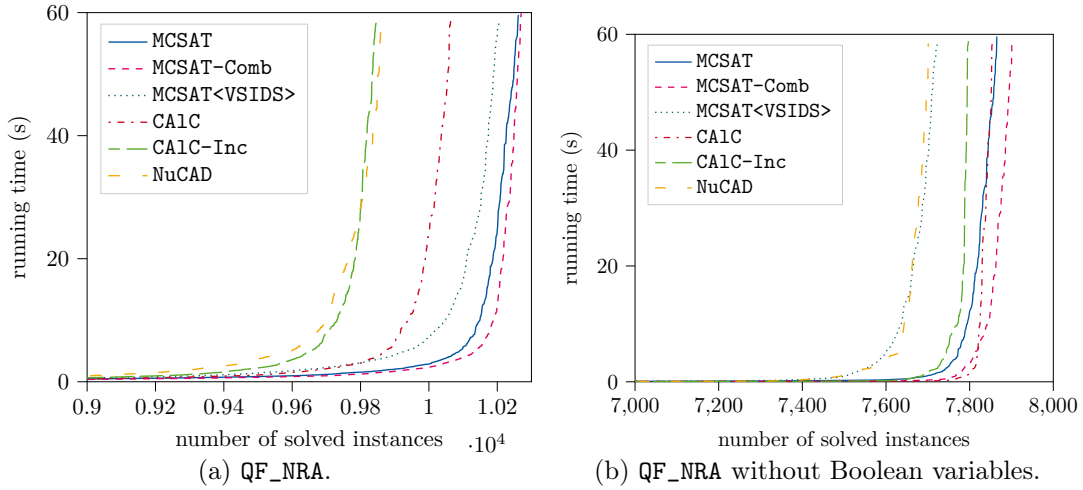


Figure 7.5: Performance profiles of algorithms.

The picture changes when ignoring instances that contain Boolean variables, as depicted in Figure 7.5b. The theory-guidance gets more important, thus CA1C and MCSAT perform similar, while MCSAT<VSIDS> falls behind. However, as this picture may simply be produced by an imbalance of the present benchmark set, we continue our analysis on the whole set.

7.3.1.1 Boolean and Algebraic Reasoning

Table 7.2: Results of CA1C variants on QF_NRA.

	CA1C<NOOP>	CA1C<EVAL>	CA1C
solved	7991	8750	10064
sat	4981	5106	5151
unsat	3010	3644	4913
timeout	3768	2993	1670
memout	395	411	420

The different algorithms have their strengths and weaknesses. Certainly, Boolean reasoning on the input formula is a crucial ingredient for an efficient algorithm: We compare different variants of the CA1C algorithm in Table 7.2, which vary in the implicant computation: CA1C<NOOP> takes all polynomials from the input formula, CA1C<EVAL> takes a minimal set of polynomials that determines the truth-value of the input formula (given the current sample point), and CA1C uses Boolean propagation. We see a clear difference in the performance of the algorithms. In [NK25], we provide an exhaustive evaluation of the choice of the implicants; the main take-away is that Boolean reasoning greatly reduces the effort for CAD-based algorithms.

The main differences between CA1C-Inc, CA1C, and MCSAT are the interleaving of Boolean reasoning with the theory and the re-use of information. A measure for the success of an algorithm with respect to these two aspects is the number of calls to the covering projection (e.g. Algorithm 3.15); for MCSAT, this is the number of calls to the explanation function. Figure 7.6a clearly shows that CA1C requires fewer projection

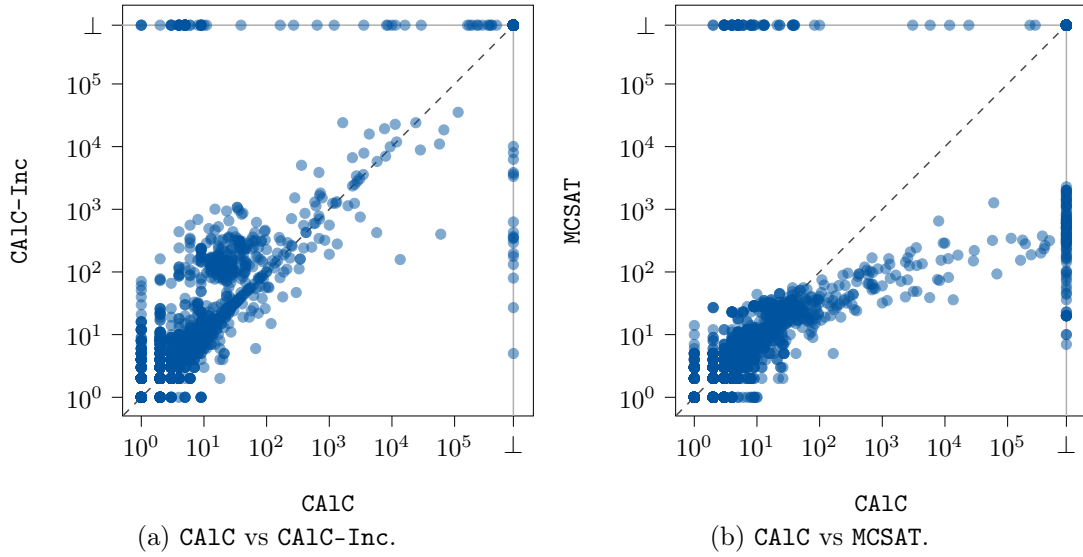


Figure 7.6: Comparison of number of covering projection calls on `QF_NRA` without Boolean variables. $\perp$ indicates a timeout of the respective solver on a given instance.

Table 7.3: Features of solved instances, split by instances solved by both, none, or one solver exclusively.

	instances # solved	# nodes			max degree		
		mean	median	max	mean	median	max
both	9962	415.68	67	6417	5.18	3	58
only CA1C	102	482.89	94	4768	8.60	7	94
only MCSAT	300	1956.34	2095	9206	2.12	2	20
none	1790	20608.07	3234.50	664211	1.88	1	16

steps than `CA1C-Inc`, which may be explained by the theory guidance of `CA1C`. Moreover, Figure 7.6b shows a drastic reduction of projection step in `MCSAT` compared to `CA1C`. As due to the similar variable ordering of the two algorithms, we conjecture that `MCSAT`'s advantage stems from its eager construction of cells, allowing their re-use in future reasoning steps.

However, the eager projection by `MCSAT` comes with some cost, as the comparison in Table 7.3 shows: Although the instances solved exclusively by `MCSAT` are larger than the ones of `CA1C`, `CA1C` seems to have an advantage on instances with larger maximal degree. Thus, `CA1C` may be able to rule out expensive computations better.

7.3.1.2 Time Spent on Algebraic Computations

We further examine where the solver spends time when solving an instance, in particular, we are interested in the algebraic computations, as they are particularly expensive.

To get an impression about the impact of the efficiency of the projection, we examine Table 7.4: The distance of `CA1C<NOOP, FULLCOV>` (which essentially computes a whole CAD of the input) to `CA1C<FULLCOV>` (which roughly computes a CAD of the relevant parts of the input, based on the Boolean structure) is unequally larger than the one from the latter to `CA1C`. We conclude that on the given benchmark set, the Boolean

Table 7.4: Results of CA1C variants on QF_NRA.

	CA1C<NOOP,FULLCov>	CA1C<FULLCov>	CA1C
solved	7909	9981	10064
sat	4944	5112	5151
unsat	2965	4869	4913
timeout	3780	1680	1670
memout	465	493	420

reasoning is more important than the projection steps; however, the latter still has a significant impact.

Table 7.5: Total time spent on parts of the solver (sum over all instances). Each pair of columns considers the instances solved by both solvers. The percentages are with respect to the total running time. CA1C<N,F> is CA1C<NOOP,FULLCov>.

	CA1C	MCSAT	CA1C	CA1C<N,F>	CA1C	NuCAD
total	5115.44	2924.61	398.50	3879.15	3198.98	6478.83
	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
algebraic	530.82	878.57	119.58	2014.57	438.03	2268.33
	10.38%	30.04%	30.01%	51.93%	13.69%	35.01%
resultant	0.49	1.63	0.06	2.85	0.69	4.45
	0.01%	0.06%	0.01%	0.07%	0.02%	0.07%
discriminant	25.72	73.24	11.31	14.13	12.38	25.42
	0.50%	2.50%	2.84%	0.36%	0.39%	0.39%
factorization	61.28	62.33	55.77	66.81	87.45	73.30
	1.20%	2.13%	13.99%	1.72%	2.73%	1.13%
root isolation	229.21	606.16	47.78	1503.49	258.88	1992.92
	4.48%	20.73%	11.99%	38.76%	8.09%	30.76%
evaluation	193.50	131.56	1.88	368.83	68.60	154.40
	3.78%	4.50%	0.47%	9.51%	2.14%	2.38%
non-algebraic	4584.62	2046.04	278.92	1864.57	2760.95	4210.50
	89.62%	69.96%	69.99%	48.07%	86.31%	64.99%

Table 7.5 compares where solvers spend their time. Among all solvers, real root isolation is the most dominant operation, while resultant and discriminant computations only take fractions of the time. This is partly explained by the frequency in which this operation is called (particularly MCSAT), but also the number and degree of the (projection) polynomials have an impact (see CA1C<NOOP,FULLCov>). Interestingly even for CA1C<NOOP,FULLCov>, the discriminant computations are more expensive than the resultant computations.

Comparing CA1C and MCSAT, CA1C spends less time on the algebraic part (despite the higher number of projection operations), which may be explained by its focus on avoiding expensive computations at the cost of the efficiency of the combinatorial (Boolean) reasoning. MCSAT spends most time on real root isolation, as it inserts cells described by indexed root functions as clauses which are considered throughout the whole solving process.

The comparison of CA1C and CA1C<NOOP,FULLCov> confirm that the exploration-guided techniques greatly reduce the effort spent on algebraic computations.

Table 7.6: Resource outs during algebraic computations.

	CA1C	MCSAT	CA1C<NoOp, FULLCov>	NuCAD
timeout+memout	2090	1896	4244	2297
during algebraic	743	943	1090	1031
during resultant	23	32	21	23
during discriminant	201	348	156	226
during factorization	287	326	287	283
during root isolation	173	185	390	448
during evaluation	54	51	200	48

Finally, the failure of NuCAD compared to CA1C on QF_NRA may be explained by the significantly higher effort on the root isolation: The CA1C algorithm is able to avoid lifting over sections as it computes a covering. NuCAD decomposes the space, and due to the nature of the CAD, this means that all sections need to be explored. Lifting over sections is particularly expensive, as the associated real root isolation operations involve real algebraic numbers.

The discussion may suggest that algebraic computations are not an issue for the given algorithms. However, the above statistics only consider *solved* instances on which the solvers did not time out. However, as algebraic computations are doubly exponential, their behaviour may be erratic. Table 7.6 shows that the number of instances on which the time or memory limit were exceeded during an algebraic computation is significant.

7.3.2 Algorithms for Quantified Formulas

We briefly describe how CA1C and NuCAD perform on the quantified benchmarks QE, QE-NRA, and QE-QF_NRA.

Table 7.7: Top: Solved instances of the given solver on the respective benchmark set. Bottom: Sum of atoms in the solution formulas and sum of number of leaves and nodes in the computed tree over all instances solved by both solvers.

	QE		QE-NRA		QE-QF_NRA	
	CA1C	NuCAD	CA1C	NuCAD	CA1C	NuCAD
solved	56	58	3816	3813	9260	9173
sat	50	52	4	3	4352	4446
unsat	6	6	3812	3810	4908	4727
timeout	22	20	3	6	2483	2544
memout	0	0	0	0	411	437
Σ atoms	3613	2471	117	88	229 276	181 327
# leaves	6504	3447	196	122	376 670	261 391
# nodes	7869	7207	291	478	494 725	879 858

Table 7.7 gives the overall results. It appears that CA1C is superior to NuCAD in terms of number of solved instances (except on QE), but NuCAD computes smaller solution formulas (measured by the number of atoms). The latter is surprising, given the weak performance of NuCAD for satisfiability checking on QF_NRA. We see that NuCAD is able to

describe the solution space with fewer cells (e.g. the number of leaves in its computation tree), while the computation tree is remarkably larger in general (e.g. the number of nodes).

Table 7.8: Features of solved instances on **QE-QF_NRA**, split by instances solved by both, none, or one solver exclusively.

	instances # solved	# nodes			max degree		
		mean	median	max	mean	median	max
both	9028	430.08	65	6417	5.13	3	44
only CA1C	232	833.12	306.50	4768	9.85	3	94
only NuCAD	145	1565.12	2094	3703	3.80	2	16
none	2749	13591.37	1427	664211	2.64	2	22

As Table 7.8 shows, **NuCAD** seems to work better on large formulas, but worse on formulas with polynomials of higher degree. The former may be due to better combinatorial reasoning: while **CA1C** splits cells into finer cells to maintain global cylindricity, **NuCAD** may compute a coarser decomposition, as suggested by the lower number of leaves in its computation tree (see Table 7.7). The latter may be due to the expensive real root isolation operations called more frequently by **NuCAD** (see previous section).

Table 7.9: Top: Solved instances of the given solver on the subset of the respective benchmark set which are suitable as input for **QEPCAD B** and **Redlog**. Bottom: Sum of atoms in the solution formulas over all instances solved by all solvers.

	QE				QE-NRA				QE-QF_NRA			
	NuC	CA1	QEP	Red	NuC	CA1	QEP	Red	NuC	CA1	QEP	Red
solved	58	56	44	57	556	558	455	537	6974	7045	7360	7167
sat	52	50	38	51	2	3	3	2	4151	4187	4758	4493
unsat	6	6	6	6	554	555	452	535	2823	2858	2602	2674
unknown	0	0	27	0	0	0	103	0	0	0	1666	0
timeout	20	22	7	21	4	2	2	23	1968	1928	297	2156
memout	0	0	0	0	0	0	0	0	381	350	0	0
Σ atoms	626	724	120	11 209	0	0	0	0	75 076	83 951	17 095	523 080

Table 7.9 shows that **NuCAD** and **CA1C** solve slightly more instances on **QE** and **QE-NRA** than the other solvers (noting that the unknown results from **QEPCAD B** stem from its incomplete projection operator), but less on **QE-QF_NRA**. Still, the satisfiability-checking variant of **CA1C** solves 7846 on the corresponding subset of **QF_NRA**. On **QE-QF_NRA**, the algorithms need to iterate over the whole solution space of the input, on which the exploration-guided **NuCAD** and **CA1C** algorithms may be less-suited compared to **QEPCAD B** and **Redlog** which consider the input formula globally. For the same reasons, we may conclude that **NuCAD** and **CA1C** may be more efficient on quantified formulas; however, the sparsity of the benchmark sets does not allow for a final conclusion. Another explanation is that **NuCAD** and **CA1C** return an encoding of cylindrical cells describing the solution space, **QEPCAD B** and **Redlog** may return an arbitrary solution formula (even simply returning the input formula). The former may be of more use than the latter, as it admits e.g. a straight-forward generation of sample points.

Comparing the size of the output formulas, NuCAD and CA1C are superior to Redlog, but inferior to QEPCAD B. The latter is to be expected, as QEPCAD B employs various techniques to minimize the size of the solution formula.

A combination of NuCAD or CA1C with the techniques from QEPCAD B may increase the applicability on formulas with quantifiers.

For more detailed evaluations, we refer to [NK25; NÁ25].

7.4 Evaluation of Proof System

The proof system admits great flexibility for the projection operations of exploration-guided algorithms. In the following, we evaluate the different techniques using MCSAT, as it is our most mature implementation.

Most of these techniques were already evaluated in corresponding publications; we refer to them for more insights, as we focus on an overall comparison here. The results of the following evaluation may differ from previous ones due to many incremental (but impactful) changes to the solver heuristics which also shift how different techniques and heuristics compare to each other. The most notable change is the use of the THFEATURE heuristic instead of the VSIDS heuristic.

7.4.1 Basic Heuristics

We start by evaluating the basic heuristics, that are the BIGGEST CELL (MCSAT<Bc, BcCov>), the LOWEST DEGREE BARRIERS (MCSAT<LDB, LDBCov>), and the CHAIN (MCSAT<CHAIN, CHAINCov>) representations. Additionally, we consider the ALL COMPOUND (MCSAT<COMPOUND, COMPOUNDCov>) representation. For demonstration purposes, we also include the FULL (MCSAT<FULL, FULLCov>) representation.

Table 7.10: Basic heuristics. Total number of *solved* instances by solver, the number of instances solved by the baseline but not the solver (*lost*), and the number of instances solved by the solver but not the baseline (*gained*). The baseline is marked with *.

	solved	lost	gained
MCSAT<Bc, BcCov>*	10262	0	0
MCSAT<LDB, LDBCov>	10258	8	4
MCSAT<CHAIN, CHAINCov>	10253	15	6
MCSAT<COMPOUND, COMPOUNDCov>	10239	58	35
MCSAT<FULL, FULLCov>	10228	39	5
MCSAT<VSIDS, Bc, BcCov>*	10206	0	0
MCSAT<VSIDS, CHAIN, CHAINCov>	10200	58	52
MCSAT<VSIDS, LDB, LDBCov>	10196	11	1
MCSAT<VSIDS, COMPOUND, COMPOUNDCov>	10187	132	113
MCSAT<VSIDS, FULL, FULLCov>	10176	90	60

The results are compared in Table 7.10. We observe that the differences between the basic heuristics are minor, a slight complementarity of MCSAT<COMPOUND, COMPOUNDCov> to the basic heuristics (looking at the number of lost and gained instances), and a surprisingly high number of solved instances of MCSAT<FULL, FULLCov>.

Switching the variable ordering to VSIDS (which was previously used in our implementation), the differences are bigger, as shown in Table 7.10: $\text{MCSAT}\langle\text{Bc}, \text{BcCov}\rangle$ and $\text{MCSAT}\langle\text{CHAIN}, \text{CHAINCov}\rangle$ solve a similar total number of instances, but seem to be quite complementary. The complementarity of $\text{MCSAT}\langle\text{COMPOUND}, \text{COMPOUNDCov}\rangle$ also increases. Still, $\text{MCSAT}\langle\text{FULL}, \text{FULLCov}\rangle$ solves surprisingly many instances (remind that it almost computes a full CAD projection).

We thus conclude that the impact of the representation for a cell or covering is lower than expected; this may be explained by our observation that e.g. discriminant computations take more time than resultant computations (indicating that discriminants are more expensive than resultants also for real root isolation). Although the solvers may have erratic behaviour (that is, small changes in the decisions made throughout the solving and projection phase have a relatively high impact) we see that some heuristics have significant complementary behaviour: In particular for the VSIDS ordering, which presumably entails a more expensive projection than the feature based one, the choice of the representation has a greater impact.

For a more detailed evaluation, we refer to [NÁS⁺24].

7.4.2 Combinatorial Optimization Heuristics for Cells

Motivated by the observation from the basic heuristics, we hope to minimize the cost (measured e.g. by the degree) of the resultants using combinatorial optimization, hopefully resolving the above complementarities.

Table 7.11: Optimization heuristics.

	solved	lost	gained
$\text{MCSAT}\langle\text{Bc}, \text{BcCov}\rangle^*$	10262	0	0
$\text{MCSAT}\langle\text{OPT}^{\text{DEGREE}}, \text{BcCov}\rangle$	10256	12	6
$\text{MCSAT}\langle\text{OPT}^{\text{SOTD}}, \text{BcCov}\rangle$	10254	14	6
$\text{MCSAT}\langle\text{OPT}^{\text{NUMRES}}, \text{BcCov}\rangle$	10253	14	5
$\text{MCSAT}\langle\text{OPT}^{\text{NUMVAR}}, \text{BcCov}\rangle$	10251	16	5
$\text{MCSAT}\langle\text{VSIDS}, \text{Bc}, \text{BcCov}\rangle^*$	10206	0	0
$\text{MCSAT}\langle\text{VSIDS}, \text{OPT}^{\text{DEGREE}}, \text{BcCov}\rangle$	10199	47	40

Table 7.11 shows that this approach did not succeed, as in the upper part using THFEATURE , the differences are marginal, and in the lower part using VSIDS, the complementarities still exist.

We analyze whether the non-optimal “hack” plays a role for these results: When restricting to the 7258 instances where single cell construction is applied, the median number of its applications is 8; on 1355, it is applied more than 100 times. Restricting the analysis based on VSIDS to the instances with at most 100 applications, we lose 5 instances and gain 1, which effectively means that the two variants behave similar. We cannot conclude whether the approach would pay off with an always-optimal version.

Furthermore, the approach does not incorporate compound cell bounds yet, which is the most complementary variant in the above evaluation.

For a more detailed evaluation, we refer to [Per24].

7.4.3 Under-Approximation of Cells

The under-approximation of cells should reduce the effort during the cell construction and produce cells that are more efficient to compute with.

Table 7.12: Approximation heuristics.

	solved	lost	gained
MCSAT<APXCONST, BCCOV>	10268	5	11
MCSAT<Bc, BCCOV>*	10262	0	0
MCSAT<APXPWL ³ , BCCOV>	10214	58	10
MCSAT<APXPWL ⁵ , BCCOV>	10194	76	8
MCSAT<APXPWL ⁷ , BCCOV>	10176	95	9
MCSAT<VSIDS, APXCONST, BCCOV>	10242	48	84
MCSAT<VSIDS, Bc, BCCOV>*	10206	0	0

The upper part of Table 7.12 shows that the CONSTANT APPROXIMATION (MCSAT<APXCONST, BCCOV>) representation in fact yields a slight performance gain, while the PIECEWISE-LINEAR APPROXIMATION (MCSAT<APXPWL*, BCCOV>) representation loses many instances. The more accurate description by the latter seemingly does not pay off, presumably due to the more involved computation: We need more real root isolation steps to compute the support points, and use the techniques for local delineability for generalizing the bound.

The lower part of Table 7.12 shows that in a setting where the projection is more expensive, the CONSTANT APPROXIMATION yields a significant amount of instances. Here, the approximation is applied on 1919 instances at least for one cell (of which 1439 are solved); however, only on 313 instances, the approximation is applied on more than 10 cells. Still, the approximation reduces effort during projection and lifting: Over the instances solved by both solvers, the approximation reduces the time spent on resultant computations from 30 seconds (which is — as observed above — only a minor part of the algebraic computations) to 5 seconds, and the time required for real root isolation from 777 seconds to 615 seconds.

For a more detailed evaluation, we refer to [Pro22; Wag23; PNÁ25].

7.4.4 Local Delineability

Table 7.13: Local delineability.

	solved	lost	gained
MCSAT<BCLDBOUNDS, BCLDBOUNDSCOV>	10309	11	62
MCSAT<Bc, BCCOV>*	10258	0	0
MCSAT<BCLDRAT, BCLDRATCOV>	10255	9	6
MCSAT<BCLDINDEP, BCLDINDEPCOV>	10172	90	4
MCSAT<BCLDALL, BCLDALLCOV>	10161	109	12

Table 7.13 shows that MCSAT<BCLDALL, BCLDALLCOV>, which exhaustively checks for relevant roots, MCSAT<BCLDINDEP, BCLDINDEPCOV>, which detects only independent

resultants, and $\text{MCSAT}\langle\text{BcLdRAT}, \text{BcLdRATCov}\rangle$, which checks only rational roots for relevance, perform worse than the baseline. Table 7.14 shows that for $\text{MCSAT}\langle\text{BcLdALL}, \text{BcLdALLCov}\rangle$ and $\text{MCSAT}\langle\text{BcLdINDEP}, \text{BcLdINDEPCov}\rangle$, the effort for checking the relevance of roots is expensive, and for $\text{MCSAT}\langle\text{BcLdALL}, \text{BcLdALLCov}\rangle$, $\text{MCSAT}\langle\text{BcLdINDEP}, \text{BcLdINDEPCov}\rangle$ and $\text{MCSAT}\langle\text{BcLdRAT}, \text{BcLdRATCov}\rangle$, only few non-relevant roots to be included in the interval are identified.

Table 7.14: Share of non-relevant roots contained in an interval (first column) and share of the time spent on checking their relevance (second column) over all 10 140 instances solved by all variants, and the number of instances on which the variant timed out during checks for relevance (third column).

	roots in interval	time	time out
$\text{MCSAT}\langle\text{BcLdBOUNDS}, \text{BcLdBOUNDS}\text{Cov}\rangle$	7%	0%	0
$\text{MCSAT}\langle\text{BcLdRAT}, \text{BcLdRAT}\text{Cov}\rangle$	1%	0%	0
$\text{MCSAT}\langle\text{BcLdINDEP}, \text{BcLdINDEP}\text{Cov}\rangle$	0%	8%	161
$\text{MCSAT}\langle\text{BcLdALL}, \text{BcLdALL}\text{Cov}\rangle$	2%	11%	223

The situation for $\text{MCSAT}\langle\text{BcLdBOUNDS}, \text{BcLdBOUNDS}\text{Cov}\rangle$ is different: The checks are cheap (no additional real root isolations are required) and detect a good portion of non-relevant roots to be included in the interval, effectively avoiding lifting over them in the following solving process. These savings are mostly on the top level (i.e. the level of the input constraints of a call to the explanation backend) and the level below, where 14% respectively 12% of the roots are included, although on the lower levels, still 1% to 5% of the roots are included each.

The latter optimization is also successful for the CA1C and NuCAD algorithm on QE-QF_NRA, as depicted in Table 7.15. The share of non-relevant roots contained in an interval (on the instances solved by both solvers) is 26% for $\text{CA1C}\langle\text{BcLdBOUNDS}, \text{BcLdBOUNDS}\text{Cov}\rangle$ and only 0.4% for $\text{NuCAD}\langle\text{BcLdBOUNDS}\rangle$.

Table 7.15: Local delineability on QE-QF_NRA.

	solved	lost	gained
$\text{CA1C}\langle\text{BcLdBOUNDS}, \text{BcLdBOUNDS}\text{Cov}\rangle$	9559	2	301
$\text{CA1C}\langle\text{Bc}, \text{Bc}\text{Cov}\rangle^*$	9260	0	0
$\text{NuCAD}\langle\text{BcLdBOUNDS}\rangle$	9253	12	92
$\text{NuCAD}\langle\text{Bc}\rangle^*$	9173	0	0

For a more detailed evaluation, we refer to [NÁ24].

7.4.5 Projective Delineability

Table 7.16 shows that the variants using projective delineability do not greatly affect the number of problems that are tractable within the time limit. The VSIDS variants (which presumably have a more expensive projection) are moderately complementary to the baseline variants.

The results may suggest that the optimization from projective delineability is only applied in very rare cases, but this is not the case, considering the instances solved

Table 7.16: Projective delineability.

	solved	lost	gained
MCSAT<Bc, BcCov>*	10258	0	0
MCSAT<BcPd, BcPdCov>	10251	16	9
MCSAT<Ldb, LdbCov>*	10258	0	0
MCSAT<LdbPd, LdbPdCov>	10251	15	8
MCSAT<VSIDS, Bc, BcCov>*	10196	0	0
MCSAT<VSIDS, BcPd, BcPdCov>	10187	63	54
MCSAT<VSIDS, Ldb, LdbCov>*	10196	0	0
MCSAT<VSIDS, LdbPd, LdbPdCov>	10182	67	53

by MCSAT<VSIDS, BcPd, BcPdCov>: In total, the leading coefficient can be omitted for 359 270 polynomials, while for 967 784 polynomials, the optimization cannot be applied as the cell is unbounded in some direction, and for 5 279 polynomials, the optimization cannot be applied as it does not have a root on both sides of the bounds, or we did not find an appropriate resultant that may replace the leading coefficient. The optimization is thus applied in a substantial 27% of the cases.

There are multiple explanations why this does not translate in practical efficiency gains: First, to ensure non-nullification of a polynomial, we need to add any non-vanishing coefficient. This is often the leading coefficient, but using projective delineability, we are able to choose a simpler coefficient (e.g. in terms of degree). Second, the leading coefficients do not dominate the degree growth during the projection phase, but rather — as observed above — the discriminants.

For a more detailed evaluation, we refer to [NMÁ⁺25].

7.5 Summary

Our conclusions are a little sobering: It seems that the reasons for state-of-the-art solvers succeeding on a “non-trivial” input are more due to lucky guesses of some heuristic than efficient usage of the CAD. Further, the solvers are highly complementary (z3 even uses a sequential portfolio), supporting the claim that there does not exist some universal heuristic yet.

However, the generalizability of these and the following conclusions is limited by the availability of benchmarks: As shown, even the relatively large QF_NRA benchmark set lacks diversity; for quantified benchmarks, there only exist few publicly available benchmarks. However, we think that only real world benchmarks allow drawing meaningful conclusions.

We find that the different exploration-guided algorithms MCSAT, CAIC, and NuCAD, are complementary to some degree, as they show significantly different behaviour. The reasons are not fully explained yet. Due to their algorithmic similarities, a tight integration of their ideas into a single framework seems desirable.

Although exploration-guided algorithms originally stem from SMT solving, the CAIC and NuCAD algorithms demonstrate that these concepts can also be applied to quantifier

elimination, resulting in highly effective algorithms. A combination with existing algorithms from quantifier elimination may improve the size of the generated solution formulas.

Exploration-guided algorithms spend significantly more time on real root isolation than on computing the projection polynomials on the instances they solve. There are various possible explanations, among them: (1) Real root isolation is used more frequently and suffers more from degree explosion of projection polynomials. (2) There is a survivorship bias, the picture would be different on unsolved instances; however, without solving them, we cannot collect the data.

Nevertheless, our improvements to the proof system (which determines the projection polynomials to be computed) mostly do not lead to a larger number of solved instances; although better projection polynomials should lead to lower cost for real root isolation as well, these savings are apparently not sufficient to have a noticeable impact. Still, some of them show complementary behaviour if the projection is expensive enough, that is, they solve different instances than the baseline. It is not clear whether such behaviour is just by accident, or there exists a good heuristic for exploiting it. Still, two techniques brought significant improvements: the constant under-approximation scheme ($\text{MCSAT}\langle\text{VSIDS}, \text{APXCONST}, \text{BCOV}\rangle$), and closing interval bounds using local delineability ($\text{MCSAT}\langle\text{BCLDBOUNDS}, \text{BCLDBOUNDSCOV}\rangle$, $\text{CALC}\langle\text{BCLDBOUNDS}, \text{BCLDBOUNDSCOV}\rangle$). Both of these techniques reduce the cost for real root isolation.

Future research directions should thus focus on avoiding the expensive CAD machinery by further improving the algorithms, improving heuristics such as variable orderings, and incorporating incomplete techniques more tightly. These may reduce the effort for real root isolation or the degree of projection polynomials.

Chapter 8

Conclusion

We conclude on the contributions of this thesis. Note that in Section 1.2 and at the beginning of the chapters, we already attributed the author’s contributions, and which of them are already contained in published works. We thus focus on the high-level results of this thesis here.

The contributions of this thesis can be grouped into two clusters: Improvements and extensions to the exploration-guided algorithms NLSAT, CAIC, and NuCAD; and the development of a proof system used by exploration-guided algorithms for projections as well as improvements to the latter.

Improvements to Exploration-guided Algorithms This work unified the presentation of the exploration-guided algorithms NLSAT, CAIC, and NuCAD, making their common ideas and building blocks explicit. This does not only improve the understanding of these algorithms, but also allows a straight-forward transfer of improvements between them. We thus improved the projection operation of both the NLSAT and CAIC algorithm.

Further, we transferred ideas from SMT solving to quantifier elimination by extending the CAIC algorithm for quantifier elimination, and modified the NuCAD algorithm for quantifier elimination by transferring these ideas. We are convinced that quantifier elimination tools may further benefit from SMT solving ideas.

We also show that — mainly for the CAIC method — heuristic modifications to the algorithm, changing the order in which the search space is explored, are possible.

Our evaluation shows that the presented exploration-guided algorithms are complementary to some degree, meaning that each algorithm has its strengths and weaknesses. Our work lays the foundation of the combination of the algorithms to close the gap to the virtual best portfolio. Furthermore, the evaluation shows that quantifier elimination tools may benefit by combining their techniques with exploration-guided methods.

Proof System and Improvements to the Projection As exploration-guided algorithms explain the relevant conflicts only, they are able to reduce the size of the CAD projection based on given sample points. These savings are also enabled by many optimizations which are applicable in certain cases; furthermore, there often is not a single way to reduce the projection, but multiple options. This increases the complexity (in the sense of its description and implementation) of the projection operation. To tackle this challenge, we reformulated the projection operation used by

the exploration-guided algorithms as a proof system. We designed a set of properties and rules splitting the projection up into smaller steps, justifying the presence of each projection polynomial by a chain of rule applications. This makes the proof system modular and extensible, but retains its maintainability. Its modularity allows its application in CAIC, NLSAT and NuCAD on the one hand; but also provides an interface for heuristics guiding the projection operation separating heuristic decisions from the correctness on the other hand. The proof system thus also increases the trust in our implementation, and enables the generation of (high-level) certificates for real algebra problems as a first step.

The proof system enabled the development of optimizations of the CAD projection: We added rules for dealing with nullifications (without relying on Lazard) and developed the notion of local delineability which allows “omitting” irrelevant roots during projection. It further led to the new notion of projective delineability which makes the role of leading coefficients explicit and thus not only contributes to the understanding of the CAD, but allows for potential savings in the projection.

We further investigated various heuristics for instantiating the proof rules. Among them are basic schemes, a combinatorial optimization approach, and two approximation techniques.

During our experimental evaluation, it turned out that many of the theoretical contributions and heuristic improvements did not translate to practical gains on the available benchmark sets. Still, we could show that the resulting projection variants are complementary to some degree. Further, two optimizations significantly improved the practical running times: An approximation technique and the use of local delineability for reducing lifting over non-rational numbers.

8.1 Future Work

The evaluation shows the limitations of state-of-the-art SMT solvers on real algebra problems. However, our work shows the potential of existing techniques, and we believe that their exploitation may lead to practical gains.

Following our evaluation, the greatest gains may be achieved by improving the algorithms rather than the projection operation, as the efficient use of algebraic operations outside the projection operation seems to be the bottleneck. We can think of various areas for improvements of the former: (1) In our CAIC and NuCAD implementations, the variable orderings are static according to properties of the input polynomials. Further, the VSIDS heuristic in our MCSAT implementation performs worse than the static ordering. Still, we think that a dynamic ordering that respects the polynomials’ properties has potential, in particular in the context of CAIC. (2) As the methods have strengths and weaknesses, a combination of CAIC, NLSAT, and NuCAD seems desirable. (3) Connected to that, the integration with Boolean reasoning is crucial. To that end, the MCSAT transition system could be further generalized to also capture the CAIC and NuCAD algorithms. (4) Previous work shows that integration of incomplete techniques (like linearization or interval constraint propagation) with the CAD yields practical improvements. However, there is no deep integration of the exploration-guided methods with incomplete techniques yet. Such deep integration may be achieved by an extension of the transition system as in (3).

Regarding the projection operation, we observed that some heuristics are complementary. These gaps may be closed by more sophisticated heuristics, e.g. based on data-based analysis as recently done for CAD variable orderings.

We provide a proof system for the projection operation. However, our implementation does not expose the application of the rules as certificate yet for various reasons: The standard output formats for certificates were not mature enough during the time of the author's work on this thesis. And more crucially, filling the gaps to obtain a certificate suitable for a theorem prover is a lot of work: Even for checking the correct instantiation of the proof system, formalizations of the algebraic computations are required; and proving the proof rules themselves requires major effort. However, we think that a certificate proving the correct instantiation of the proof system is an achievable goal that already provides additional value.

Regarding the ideas proposed above, we think that quantifier elimination should be a first-class citizen during their development. Further, to increase the usability of real algebra tools, additional interfaces specifically for real algebra tools could allow deeper interaction with the underlying data structures or could pass hints about the problem's structure, hopefully increasing the tool's efficiency. Further, as real algebra problems are usually hard to solve, semi-automated tuning of the tool's heuristic towards a class of input problems may increase their tractability.

Bibliography

- [AB10] John Abbott and Anna M. Bigatti. “CoCoALib: A C++ Library for Computations in Commutative Algebra... and Beyond”. In: *Mathematical Software (ICMS 2010)* 2010. Lecture Notes in Computer Science vol. 6327. DOI: 10.1007/978-3-642-15582-6_15.
- [ÁAB⁺16] Erika Ábrahám, John Abbott, Bernd Becker, Anna M. Bigatti, Martin Brain, Bruno Buchberger, Alessandro Cimatti, James H. Davenport, Matthew England, Pascal Fontaine, Stephen Forrest, Alberto Griggio, Daniel Kroening, Werner M. Seiler, and Thomas Sturm. “SC²: Satisfiability Checking Meets Symbolic Computation”. In: *Intelligent Computer Mathematics (CICM 2016)* 2016. Lecture Notes in Computer Science vol. 9791. DOI: 10.1007/978-3-319-42547-4_3.
- [ÁDE⁺21] Erika Ábrahám, James H. Davenport, Matthew England, and Gereon Kremer. “Deciding the consistency of non-linear real arithmetic constraints with a conflict driven search using cylindrical algebraic coverings”. In: *Journal of Logical and Algebraic Methods in Programming* 119 (2021). DOI: 10.1016/j.jlamp.2020.100633.
- [ÁNK17] Erika Ábrahám, Jasper Nalbach, and Gereon Kremer. “Embedding the Virtual Substitution Method in the Model Constructing Satisfiability Calculus Framework (work-in-progress paper)”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2017)* 2017. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-1974/EAb.pdf>.
- [ÁNP23] Erika Ábrahám, Jasper Nalbach, and Valentin Promies. “Automated Exercise Generation for Satisfiability Checking”. In: *Formal Methods Teaching (FMTea 2023)* 2023. Lecture Notes in Computer Science vol. 13962. DOI: 10.1007/978-3-031-27534-0_1.
- [ABS94] A Akritas, A Bocharov, and A Strzeboński. “Implementation of real root isolation algorithms in Mathematica”. In: *International conference on interval and computer-algebraic methods in science and engineering (interval’94)* 1994. URL: <https://faculty.e-ce.uth.gr/akritas/articles/41.pdf>.
- [AS05] A. G. Akritas and A. W. Strzeboński. “A Comparative Study of Two Real Root Isolation Methods”. In: *Nonlinear Analysis: Modelling and Control* 10.4 (2005). DOI: 10.15388/NA.2005.10.4.15110.
- [Ali20] Fabian Alieff. “Simplex heuristics in SMT solving”. Bachelor’s thesis. RWTH Aachen University, 2020. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/alieff_bachelor.pdf.

- [ACG00] Alessandro Armando, Claudio Castellini, and Enrico Giunchiglia. “SAT-Based Procedures for Temporal Reasoning”. In: *Recent Advances in AI Planning (ECP 1999)* 2000. Lecture Notes in Computer Science. DOI: 10.1007/10720246_8.
- [AG93] Alessandro Armando and Enrico Giunchiglia. “Embedding complex decision procedures inside an interactive theorem prover”. In: *Annals of Mathematics and Artificial Intelligence* 8.3 (1993). DOI: 10.1007/BF01530803.
- [Arn88] Dennis S. Arnon. “A cluster-based cylindrical algebraic decomposition algorithm”. In: *Journal of Symbolic Computation* 5.1 (1988). DOI: 10.1016/S0747-7171(88)80012-9.
- [ACM84] Dennis S. Arnon, George E. Collins, and Scott McCallum. “Cylindrical Algebraic Decomposition II: An Adjacency Algorithm for the Plane”. In: *SIAM Journal on Computing* 13.4 (1984). DOI: 10.1137/0213055.
- [Arn81] Dennis Soulé Arnon. “Algorithms for the geometry of semi-algebraic sets”. PhD thesis. University of Wisconsin-Madison, 1981. URL: <https://dl.acm.org/doi/book/10.5555/910085>.
- [ABC⁺02] Gilles Audemard, Piergiorgio Bertoli, Alessandro Cimatti, Artur Kornilowicz, and Roberto Sebastiani. “A SAT Based Approach for Solving Formulas over Boolean and Linear Mathematical Propositions”. In: *Automated Deduction (CADE-18)* 2002. Lecture Notes in Computer Science. DOI: 10.1007/3-540-45620-1_17.
- [AS09] Gilles Audemard and Laurent Simon. “Predicting learnt clauses quality in modern SAT solvers”. In: *Artificial Intelligence (IJCAI 2009)* 2009. DOI: 10.5555/1661445.1661509.
- [Bär22] Philipp Bär. “Exploiting strict constraints in the computations of cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2022. URL: <https://ths.rwth-aachen.de/wp-content/uploads/sites/4/bachelorarbeit-philipp-baer.pdf>.
- [BNÁ⁺23] Philipp Bär, Jasper Nalbach, Erika Ábrahám, and Christopher W. Brown. “Exploiting Strict Constraints in the Cylindrical Algebraic Covering”. In: *Satisfiability Modulo Theories (SMT 2023)* 2023. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-3429/paper13.pdf>.
- [BBB⁺22] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. “cvc5: A Versatile and Industrial-Strength SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)* 2022. Lecture Notes in Computer Science. DOI: 10.1007/978-3-030-99524-9_24.
- [BRK⁺22] Haniel Barbosa, Andrew Reynolds, Gereon Kremer, Hanna Lachnitt, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Arjun Viswanathan, Scott Viteri, Yoni Zohar, Cesare Tinelli, and Clark Barrett. “Flexible Proof Production in an Industrial-Strength SMT Solver”. In: *Automated Reasoning (IJCAR 2022)* 2022. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-10769-6_3.

- [BFT17] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The SMT-LIB standard: Version 2.6*. Tech. rep. Department of Computer Science, The University of Iowa, 2017.
- [BDL98] Clark W. Barrett, David L. Dill, and Jeremy R. Levitt. “A decision procedure for bit-vector arithmetic”. In: *Design Automation Conference (DAC 1998)* 1998. DOI: 10.1145/277044.277186.
- [BDS02] Clark W. Barrett, David L. Dill, and Aaron Stump. “Checking Satisfiability of First-Order Formulas by Incremental Translation to SAT”. In: *Computer Aided Verification (CAV 2002)* 2002. Lecture Notes in Computer Science. DOI: 10.1007/3-540-45657-0_18.
- [BDH⁺22] Max Barth, Daniel Dietsch, Matthias Heizmann, and Andreas Podelski. *Ultimate eliminator at SMT-COMP 2022*. Tech. rep. 2022.
- [BPR96] Saugata Basu, Richard Pollack, and Marie-Françoise Roy. “On the combinatorial and algebraic complexity of quantifier elimination”. In: *Journal of the ACM* 43.6 (1996). DOI: 10.1145/235809.235813.
- [BPR06] Saugata Basu, Richard Pollack, and Marie-Françoise Roy. *Algorithms in Real Algebraic Geometry*. 2nd ed. Algorithms and computation in mathematics volume 10. Berlin: Springer, 2006. ISBN: 978-3-540-33098-1. DOI: 10.1007/3-540-33099-2.
- [BKR84] Michael Ben-Or, Dexter Kozen, and John Reif. “The complexity of elementary algebra and geometry”. In: *Theory of Computing (STOC 1984)* 1984. DOI: 10.1145/800057.808712.
- [BG06] Frédéric Benhamou and Laurent Granvilliers. “Continuous and Interval Constraints”. In: *Handbook of Constraint Programming*. Ed. by Francesca Rossi, Peter van Beek, and Toby Walsh. Vol. 2. Foundations of Artificial Intelligence. Elsevier, 2006. DOI: 10.1016/S1574-6526(06)80020-9.
- [BC04] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development*. Ed. by Wilfried Brauer, Grzegorz Rozenberg, and Arto Salomaa. Texts in Theoretical Computer Science An EATCS Series. Berlin, Heidelberg: Springer, 2004. ISBN: 978-3-642-05880-6 978-3-662-07964-5. DOI: 10.1007/978-3-662-07964-5.
- [BFT11] Frédéric Besson, Pascal Fontaine, and Laurent Théry. “A Flexible Proof Format for SMT: a Proposal”. In: *Proof eXchange for Theorem Proving (PxTP 2011)* 2011. URL: <https://inria.hal.science/hal-00642544>.
- [Bie08] Armin Biere. “PicoSAT Essentials”. In: *Journal on Satisfiability, Boolean Modeling and Computation* 4.2-4 (2008). DOI: 10.3233/SAT190039.
- [Bie09] Armin Biere, ed. *Handbook of Satisfiability*. Frontiers in artificial intelligence and applications v. 185. Amsterdam, The Netherlands ; Washington, DC: IOS Press, 2009. ISBN: 978-1-58603-929-5.
- [Bie10] Armin Biere. *Lingeling, Plingeling, PicoSAT and PrecoSAT at SAT Race 2010*. Tech. rep. Institut for Formal Models and Verification, Johannes Kepler University, 2010. DOI: 10.35011/FMVTR.2010-1.

- [BFF⁺20] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020*. Tech. rep. 2020. URL: <https://cca.informatik.uni-freiburg.de/papers/BiereFazekasFleuryHeisinger-SAT-Competition-2020-solvers.pdf>.
- [Bjø10] Nikolaj Bjørner. “Linear Quantifier Elimination as an Abstract Decision Procedure”. In: *Automated Reasoning (IJCAR 2010)* 2010. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-14203-1_27.
- [BJ15] Nikolaj Bjørner and Mikolas Janota. “Playing with Quantified Satisfaction”. In: *Logic for Programming, Artificial Intelligence and Reasoning (LPAR-20)* 2015. Vol. 35. DOI: 10.29007/vv21.
- [BN24] Nikolaj Bjørner and Lev Nachmanson. “Arithmetic Solving in Z3”. In: *Computer Aided Verification* 2024. DOI: 10.1007/978-3-031-65627-9_2.
- [BGT81] Robert G. Bland, Donald Goldfarb, and Michael J. Todd. “The Ellipsoid Method: A Survey”. In: *Operations Research* 29.6 (1981). DOI: 10.1287/opre.29.6.1039.
- [BGS17] Maria Paola Bonacina, Stéphane Graham-Lengrand, and Natarajan Shankar. “Satisfiability Modulo Theories and Assignments”. In: *Automated Deduction (CADE-26)* 2017. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-63046-5_4.
- [BGV23] Maria Paola Bonacina, Stéphane Graham-Lengrand, and Christophe Vauthier. “QSMA: A New Algorithm for Quantified Satisfiability Modulo Theory and Assignment”. In: *Automated Deduction (CADE-29)* 2023. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-38499-8_5.
- [BGV25] Maria Paola Bonacina, Stéphane Graham-Lengrand, and Christophe Vauthier. “The QSMA Algorithm for Quantifiers in SMT”. In: *Journal of Automated Reasoning* 69.2 (2025). DOI: 10.1007/s10817-025-09727-8.
- [BCD⁺09] Thomas Bouton, Diego Caminha B. de Oliveira, David Déharbe, and Pascal Fontaine. “veriT: An Open, Trustable and Efficient SMT-Solver”. In: *Automated Deduction (CADE-22)* 2009. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-02959-2_12.
- [BDE⁺13a] Russell Bradford, James H. Davenport, Matthew England, Scott McCallum, and David Wilson. “Cylindrical algebraic decompositions for boolean combinations”. In: *Symbolic and Algebraic Computation (ISSAC 2013)* 2013. DOI: 10.1145/2465506.2465516.
- [BDE⁺13b] Russell Bradford, James H. Davenport, Matthew England, and David Wilson. “Optimising Problem Formulation for Cylindrical Algebraic Decomposition”. In: *Intelligent Computer Mathematics (CICM 2013)* 2013. Lecture Notes in Computer Science vol. 7961. DOI: 10.1007/978-3-642-39320-4_2.
- [BKK⁺19] Franz Brauße, Konstantin Korovin, Margarita Korovina, and Norbert Müller. “A CDCL-Style Calculus for Solving Non-linear Constraints”. In: *Frontiers of Combining Systems* 2019. DOI: 10.1007/978-3-030-29007-8_8.

- [Bro61] Achille Brocot. “Calcul des rouages par approximation: nouvelle méthode”. In: *Revue chronométrique. Journal des horlogers, scientifique et pratique* 3 (1861). URL: <https://gallica.bnf.fr/ark:/12148/bpt6k1661912>.
- [Bro99] Christopher W. Brown. “Solution formula construction for truth invariant CAD’s”. PhD thesis. University of Delaware, 1999. URL: <https://dl.acm.org/doi/10.5555/929495>.
- [Bro01] Christopher W. Brown. “Improved Projection for Cylindrical Algebraic Decomposition”. In: *Journal of Symbolic Computation* 32.5 (2001). DOI: 10.1006/jSCO.2001.0463.
- [Bro03] Christopher W. Brown. “QEPCAD B: A program for computing with semi-algebraic sets using CADs”. In: *ACM SIGSAM Bulletin* 37.4 (2003). DOI: 10.1145/968708.968710.
- [Bro04] Christopher W. Brown. *Companion to the tutorial “Cylindrical algebraic decomposition”, presented at ISSAC ’04*. 2004. URL: <https://www.usna.edu/Users/cs/wcbrown/research/ISSAC04/handout.pdf>.
- [Bro09] Christopher W. Brown. “Fast simplifications for Tarski formulas”. In: *Symbolic and Algebraic Computation (ISSAC 2009)* 2009. DOI: 10.1145/1576702.1576714.
- [Bro12] Christopher W. Brown. “Fast simplifications for Tarski formulas based on monomial inequalities”. In: *Journal of Symbolic Computation. International Symposium on Symbolic and Algebraic Computation (ISSAC ’09)* 47.7 (2012). DOI: 10.1016/j.jSC.2011.12.012.
- [Bro13] Christopher W. Brown. “Constructing a single open cell in a cylindrical algebraic decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2013)* 2013. DOI: 10.1145/2465506.2465952.
- [Bro15] Christopher W. Brown. “Open Non-uniform Cylindrical Algebraic Decompositions”. In: *Symbolic and Algebraic Computation (ISSAC 2015)* 2015. DOI: 10.1145/2755996.2756654.
- [Bro17] Christopher W. Brown. “Projection and Quantifier Elimination Using Non-uniform Cylindrical Algebraic Decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2017)* 2017. DOI: 10.1145/3087604.3087651.
- [Bro25] Christopher W. Brown. “Semantics of division for polynomial solvers”. In: *Journal of Symbolic Computation* 131 (2025). DOI: 10.1016/j.jSC.2025.102453.
- [BD07] Christopher W. Brown and James H. Davenport. “The complexity of quantifier elimination and cylindrical algebraic decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2007)* 2007. DOI: 10.1145/1277548.1277557.
- [BK15] Christopher W. Brown and Marek Košta. “Constructing a single cell in cylindrical algebraic decomposition”. In: *Journal of Symbolic Computation* 70 (2015). DOI: 10.1016/j.jSC.2014.09.024.
- [BM20] Christopher W. Brown and Scott McCallum. “Enhancements to Lazard’s Method for Cylindrical Algebraic Decomposition”. In: *Computer Algebra in Scientific Computing (CASC 2020)* 2020. Lecture Notes in Computer Science. DOI: 10.1007/978-3-030-60026-6_8.

- [BS10] Christopher W. Brown and Adam Strzeboński. “Black-box/white-box simplification and applications to quantifier elimination”. In: *Symbolic and Algebraic Computation (ISSAC 2010)* 2010. DOI: 10.1145/1837934.1837953.
- [BV20] Christopher W. Brown and Fernando Vale-Enriquez. “From simplification to a partial theory solver for non-linear real polynomial constraints”. In: *Journal of Symbolic Computation* 100 (2020). DOI: 10.1016/j.jsc.2019.07.020.
- [Bro78] W. S. Brown. “The Subresultant PRS Algorithm”. In: *ACM Transactions on Mathematical Software* 4.3 (1978). DOI: 10.1145/355791.355795.
- [BT71] W. S. Brown and J. F. Traub. “On Euclid’s Algorithm and the Theory of Subresultants”. In: *Journal of the ACM* 18.4 (1971). DOI: 10.1145/321662.321665.
- [BB08] Robert Brummayer and Armin Biere. “Lemmas on demand for the extensional theory of arrays”. In: *Satisfiability Modulo Theories (SMT 2008) and Bit-Precise Reasoning (BPR 2008)* 2008. DOI: 10.1145/1512464.1512467.
- [Buc65] Bruno Buchberger. “Ein Algorithmus zum Auffinden der Basiselemente des Restklassenringes nach einem nulldimensionalen Polynomideal”. PhD thesis. University of Innsbruck, 1965.
- [BH91] Bruno Buchberger and Hoon Hong. *Speeding-up quantifier elimination by Gröbner bases*. Tech. rep. 91-06. RISC, Johannes Kepler University, 1991. URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=8700eabc4aca0f3cda9f830f2634632760ea6d51>.
- [Bur98] Stanley N. Burris. *Logic for Mathematics and Computer Science*. Upper Saddle River, N.J.: Prentice Hall, 1998. ISBN: 978-0-13-285974-5.
- [Can93] J. Canny. “Improved Algorithms for Sign Determination and Existential Quantifier Elimination”. In: *The Computer Journal* 36.5 (1993). DOI: 10.1093/comjnl/36.5.409.
- [Can88] John Canny. “Some algebraic and geometric computations in PSPACE”. In: *Theory of Computing (STOC 1988)* 1988. DOI: 10.1145/62212.62257.
- [CMX⁺09] Changbo Chen, Marc Moreno Maza, Bican Xia, and Lu Yang. “Computing cylindrical algebraic decomposition via triangular decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2009)* 2009. DOI: 10.1145/1576702.1576718.
- [CZC20] Changbo Chen, Zhangpeng Zhu, and Haoyu Chi. “Variable Ordering Selection for Cylindrical Algebraic Decomposition with Artificial Neural Networks”. In: *Mathematical Software (ICMS 2020)* 2020. Lecture Notes in Computer Science vol. 12097. DOI: 10.1007/978-3-030-52200-1_28.
- [Che25] Rizeng Chen. *A Geometric Approach to Cylindrical Algebraic Decomposition*. 2025. DOI: 10.48550/arXiv.2311.10515.
- [Chu36a] Alonzo Church. “A note on the Entscheidungsproblem”. In: *The Journal of Symbolic Logic* 1.1 (1936). DOI: 10.2307/2269326.
- [Chu36b] Alonzo Church. “An Unsolvable Problem of Elementary Number Theory”. In: *American Journal of Mathematics* 58.2 (1936). DOI: 10.2307/2268571.

- [CGI⁺18] Alessandro Cimatti, Alberto Griggio, Ahmed Irfan, Marco Roveri, and Roberto Sebastiani. “Incremental Linearization for Satisfiability and Verification Modulo Nonlinear Arithmetic and Transcendental Functions”. In: *ACM Transactions on Computational Logic* 19.3 (2018). DOI: 10.1145/3230639.
- [CGL⁺22] Alessandro Cimatti, Alberto Griggio, Enrico Lipparini, and Roberto Sebastiani. “Handling Polynomial and Transcendental Functions in SMT via Unconstrained Optimisation and Topological Degree Test”. In: *Automated Technology for Verification and Analysis (ATVA 2022)* 2022. DOI: 10.1007/978-3-031-19992-9_9.
- [CGS⁺13] Alessandro Cimatti, Alberto Griggio, Bastiaan Joost Schaafsma, and Roberto Sebastiani. “The MathSAT5 SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2013)* 2013. DOI: 10.1007/978-3-642-36742-7_7.
- [Col71] George E Collins. “The Calculation of Multivariate Polynomial Resultants”. In: *Journal of the ACM* 18.4 (1971).
- [Col67] George E. Collins. “Subresultants and Reduced Polynomial Remainder Sequences”. In: *Journal of the ACM* 14.1 (1967). DOI: 10.1145/321371.321381.
- [Col75] George E. Collins. “Quantifier elimination for real closed fields by cylindrical algebraic decomposition”. In: *Automata Theory and Formal Languages* 1975. Lecture Notes in Computer Science. DOI: 10.1007/3-540-07407-4_17.
- [Col85] George E. Collins. “The SAC-2 computer algebra system”. In: *Computer Algebra (EUROCAL 1985)* 1985. Lecture Notes in Computer Science. DOI: 10.1007/3-540-15984-3_235.
- [Col98] George E. Collins. “Quantifier Elimination for Real Closed Fields by Cylindrical Algebraic Decomposition”. In: *Quantifier Elimination and Cylindrical Algebraic Decomposition* 1998. Texts and Monographs in Symbolic Computation. DOI: 10.1007/978-3-7091-9459-1_4.
- [CA76] George E. Collins and Alkiviadis G. Akritas. “Polynomial real root isolation using Descarte’s rule of signs”. In: *Symbolic and Algebraic Computation (SYMSAC 1976)* 1976. DOI: 10.1145/800205.806346.
- [CH91] George E. Collins and Hoon Hong. “Partial Cylindrical Algebraic Decomposition for quantifier elimination”. In: *Journal of Symbolic Computation* 12.3 (1991). DOI: 10.1016/S0747-7171(08)80152-6.
- [Coo71] Stephen A. Cook. “The complexity of theorem-proving procedures”. In: *Theory of Computing (STOC 1971)* 1971. DOI: 10.1145/800157.805047.
- [CÁ11] Florian Corzilius and Erika Ábrahám. “Virtual Substitution for SMT-Solving”. In: *Fundamentals of Computation Theory (FCT 2011)* 2011. DOI: 10.1007/978-3-642-22953-4_31.

- [CKJ⁺15] Florian Corzilius, Gereon Kremer, Sebastian Junges, Stefan Schupp, and Erika Ábrahám. “SMT-RAT: An Open Source C++ Toolbox for Strategic and Parallel SMT Solving”. In: *Theory and Applications of Satisfiability Testing (SAT 2015)* 2015. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-24318-4_26.
- [CR88] M. Coste and M. F. Roy. “Thom’s lemma, the coding of real algebraic numbers and the computation of the topology of semi-algebraic sets”. In: *Journal of Symbolic Computation* 5.1 (1988). DOI: 10.1016/S0747-7171(88)80008-7.
- [Cot10] Scott Cotton. “Natural Domain SMT: A Preliminary Assessment”. In: *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)* 2010. DOI: 10.1007/978-3-642-15297-9_8.
- [Cov23] Kristian Cović. “Heuristical variable ordering in the cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2023. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/thesis_covic.pdf.
- [CLO15] David A. Cox, John Little, and Donal O’Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Undergraduate Texts in Mathematics. Cham: Springer, 2015. ISBN: 978-3-319-16720-6 978-3-319-16721-3. DOI: 10.1007/978-3-319-16721-3.
- [Dan90] George B. Dantzig. “Origins of the simplex method”. In: *A history of scientific computing*. Ed. by Stephen G. Nash. New York, NY, USA: Association for Computing Machinery, June 1990. DOI: 10.1145/87252.88081.
- [Dan51] George Bernard Dantzig. “Linear Programming”. In: *Problems for the Numerical Analysis of the Future* 1951.
- [DST93] J. H. Davenport, Y. Siret, and E. Tournier. *Computer algebra (2nd ed.): systems and algorithms for algebraic computation*. USA: Academic Press Professional, Inc., June 1993. ISBN: 978-0-12-204232-4.
- [DEM⁺23] James H. Davenport, Matthew England, Scott McCallum, and Ali K. Uncu. *Iterated Resultants and Rational Functions in Real Quantifier Elimination*. 2023. DOI: 10.48550/arXiv.2312.16210.
- [DH88] James H. Davenport and Joos Heintz. “Real quantifier elimination is doubly exponential”. In: *Journal of Symbolic Computation* 5.1 (1988). DOI: 10.1016/S0747-7171(88)80004-X.
- [DLL62] Martin Davis, George Logemann, and Donald Loveland. “A machine program for theorem-proving”. In: *Communications of the ACM* 5.7 (1962). DOI: 10.1145/368273.368557.
- [DP60] Martin Davis and Hilary Putnam. “A Computing Procedure for Quantification Theory”. In: *Journal of the ACM* 7.3 (1960). DOI: 10.1145/321033.321034.
- [dB07] Leonardo de Moura and Nikolaj Bjørner. “Efficient E-Matching for SMT Solvers”. In: *Automated Deduction (CADE-21)* 2007. Lecture Notes in Computer Science. DOI: 10.1007/978-3-540-73595-3_13.

- [dB08] Leonardo de Moura and Nikolaj Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)* 2008. Lecture Notes in Computer Science vol. 4963. DOI: 10.1007/978-3-540-78800-3_24.
- [dJ13] Leonardo de Moura and Dejan Jovanović. “A Model-Constructing Satisfiability Calculus”. In: *Verification, Model Checking, and Abstract Interpretation (VMCAI 2013)* 2013. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-35873-9_1.
- [dU21] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction (CADE-28)* 2021. Lecture Notes in Computer Science. DOI: 10.1007/978-3-030-79876-5_37.
- [del24] Tereso del Río. “Heuristics and Machine Learning to Improve Symbolic Computation Algorithms: Speeding Up Cylindrical Algebraic Decomposition”. PhD thesis. Coventry University, 2024. URL: <https://pureportal.coventry.ac.uk/en/studentTheses/heuristics-and-machine-learning-to-improve-symbolic-computation-a>.
- [dE22a] Tereso del Río and Matthew England. “New Heuristic to Choose a Cylindrical Algebraic Decomposition Variable Ordering Motivated by Complexity Analysis”. In: *Computer Algebra in Scientific Computing (CASC 2022)* 2022. Lecture Notes in Computer Science vol. 13366. DOI: 10.1007/978-3-031-14788-3_17.
- [dE22b] Tereso del Río and Matthew England. “The benefits of clustering in cylindrical algebraic decomposition”. In: *Proceedings of the EACA* (2022). URL: <https://matthewengland.coventry.domains/Publications/dRE22a.pdf>.
- [dE24] Tereso del Río and Matthew England. “Lessons on Datasets and Paradigms in Machine Learning for Symbolic Computation: A Case Study on CAD”. In: *Mathematics in Computer Science* 18.3 (2024). DOI: 10.1007/s11786-024-00591-0.
- [Des37] René Descartes. *La Géométrie*. 1637.
- [Dol00] Andreas Dolzmann. “Algorithmic Strategies for Applicable Real Quantifier Elimination”. PhD thesis. Universität Passau, 2000.
- [DSS04] Andreas Dolzmann, Andreas Seidl, and Thomas Sturm. “Efficient projection orders for CAD”. In: *Symbolic and Algebraic Computation (ISSAC 2004)* 2004. DOI: 10.1145/1005285.1005303.
- [DS97a] ANDREAS Dolzmann and THOMAS Sturm. “Simplification of Quantifier-free Formulae over Ordered Fields”. In: *Journal of Symbolic Computation* 24.2 (1997). DOI: 10.1006/jSCO.1997.0123.
- [DS97b] Andreas Dolzmann and Thomas Sturm. “REDLOG: computer algebra meets computer logic”. In: *ACM SIGSAM Bulletin* 31.2 (1997). DOI: 10.1145/261320.261324.
- [DW00] Andreas Dolzmann and Volker Weispfenning. “Local quantifier elimination”. In: *Symbolic and Algebraic Computation (ISSAC 2000)* 2000. DOI: 10.1145/345542.345589.

- [DE19] Florescu Dorian and Matthew England. “Algorithmically Generating New Algebraic Features of Polynomial Systems for Machine Learning”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2019)* 2019. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-2460/paper4.pdf>.
- [Duc00] Lionel Ducos. “Optimizations of the subresultant algorithm”. In: *Journal of Pure and Applied Algebra* 145.2 (2000). DOI: 10.1016/S0022-4049(98)00081-4.
- [Dut14] Bruno Dutertre. “Yices 2.2”. In: *Computer Aided Verification (CAV 2014)* 2014. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-08867-9_49.
- [Dd06] Bruno Dutertre and Leonardo de Moura. “A Fast Linear-Arithmetic Solver for DPLL(T)”. In: *Computer Aided Verification (CAV 2006)* 2006. Lecture Notes in Computer Science. DOI: 10.1007/11817963_11.
- [ES04] Niklas Eén and Niklas Sörensson. “An Extensible SAT-solver”. In: *Theory and Applications of Satisfiability Testing (SAT 2004)* 2004. DOI: 10.1007/978-3-540-24605-3_37.
- [Eme10] Pavel Emeliyanenko. “A complete modular resultant algorithm targeted for realization on graphics hardware”. In: *Parallel and Symbolic Computation (PASCO 2010)* 2010. DOI: 10.1145/1837210.1837219.
- [EMT08] Ioannis Z. Emiris, Bernard Mourrain, and Elias P. Tsigaridas. “Real Algebraic Numbers: Complexity Analysis and Experimentation”. In: *Reliable Implementation of Real Number Algorithms: Theory and Practice* 2008. Lecture Notes in Computer Science. DOI: 10.1007/978-3-540-85521-7_4.
- [EP05] Ioannis Z. Emiris and Victor Y. Pan. “Improved algorithms for computing determinants and resultants”. In: *Journal of Complexity*. Foundations of Computational Mathematics Conference 2002 21.1 (2005). DOI: 10.1016/j.jco.2004.03.003.
- [EBC⁺14] Matthew England, Russell Bradford, Changbo Chen, James H. Davenport, Marc Moreno Maza, and David Wilson. “Problem Formulation for Truth-Table Invariant Cylindrical Algebraic Decomposition by Incremental Triangular Decomposition”. In: *Intelligent Computer Mathematics (CICM 2014)* 2014. DOI: 10.1007/978-3-319-08434-3_5.
- [EBD15] Matthew England, Russell Bradford, and James H. Davenport. “Improving the Use of Equational Constraints in Cylindrical Algebraic Decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2015)* 2015. DOI: 10.1145/2755996.2756678.
- [EBD20] Matthew England, Russell Bradford, and James H. Davenport. “Cylindrical algebraic decomposition with equational constraints”. In: *Journal of Symbolic Computation* 100 (2020). DOI: 10.1016/j.jsc.2019.07.019.
- [ED16] Matthew England and James H. Davenport. “The Complexity of Cylindrical Algebraic Decomposition with Respect to Polynomial Degree”. In: *Computer Algebra in Scientific Computing (CASC 2016)* 2016. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-45641-6_12.

- [FOR⁺01] Jean-Christophe Filliâtre, Sam Owre, Harald Rue*B, and Natarajan Shankar. “ICS: Integrated Canonizer and Solver?” In: *Computer Aided Verification (CAV 2001)* 2001. Lecture Notes in Computer Science. DOI: 10.1007/3-540-44585-4_22.
- [FJO⁺03] Cormac Flanagan, Rajeev Joshi, Xinming Ou, and James B. Saxe. “Theorem Proving Using Lazy Proof Explication”. In: *Computer Aided Verification (CAV 2003)* 2003. Lecture Notes in Computer Science. DOI: 10.1007/978-3-540-45069-6_34.
- [FE20] Dorian Florescu and Matthew England. “A Machine Learning Based Software Pipeline to Pick the Variable Ordering for Algorithms with Polynomial Inputs”. In: *Mathematical Software (ICMS 2020)* 2020. Lecture Notes in Computer Science vol. 12097. DOI: 10.1007/978-3-030-52200-1_30.
- [FE24] Dorian Florescu and Matthew England. “Constrained Neural Networks for Interpretable Heuristic Creation to Optimise Computer Algebra Systems”. In: *Mathematical Software (ICMS 2024)* 2024. DOI: 10.1007/978-3-031-64529-7_19.
- [FOS⁺17] Pascal Fontaine, Mizuhito Ogawa, Thomas Sturm, and Xuan Tung Vu. “Subtropical Satisfiability”. In: *Frontiers of Combining Systems (FroCoS 2017)* 2017. DOI: 10.1007/978-3-319-66167-4_11.
- [Fou25] Jean Baptiste Joseph Fourier. “Sur le Calcul des conditions d’inégalité”. In: *Nouveau Bulletin des Sciences par la Société philomathique de Paris* 99 (1825).
- [Fou26] Jean Baptiste Joseph Fourier. “Solution d’une question particuliere du calcul des inégalités”. In: *Nouveau Bulletin des Sciences par la Société philomathique de Paris* (1826).
- [FHI⁺21] Nils Froleyks, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda. “SAT Competition 2020”. In: *Artificial Intelligence* 301 (2021). DOI: 10.1016/j.artint.2021.103572.
- [FIS16] Ryoya Fukasaku, Hidenao Iwane, and Yosuke Sato. “CGSQE/SyNRAC: a real quantifier elimination package based on the computation of comprehensive Gröbner systems”. In: *ACM Communications in Computer Algebra* 50.3 (2016). DOI: 10.1145/3015306.3015313.
- [GHN⁺04] Harald Ganzinger, George Hagen, Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. “DPLL(T): Fast Decision Procedures”. In: *Computer Aided Verification (CAV 2004)* 2004. Lecture Notes in Computer Science. DOI: 10.1007/978-3-540-27813-9_14.
- [GKC13] Sicun Gao, Soonho Kong, and Edmund M. Clarke. “dReal: An SMT Solver for Nonlinear Theories over the Reals”. In: *Automated Deduction (CADE-24)* 2013. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-38574-2_14.
- [GM15] Marco Gario and Andrea Micheli. “PySMT: a solver-agnostic library for fast prototyping of SMT-based algorithms”. In: *Satisfiability Modulo Theories (SMT 2015)* 2015. CEUR Workshop Proceedings. URL: <https://andrea.micheli.website/papers/pysmt.pdf>.

- [Gd09] Yeting Ge and Leonardo de Moura. “Complete Instantiation for Quantified Formulas in Satisfiability Modulo Theories”. In: *Computer Aided Verification (CAV 2009)* 2009. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-02658-4_25.
- [GBE⁺14] Jürgen Giesl, Marc Brockschmidt, Fabian Emmes, Florian Frohn, Carsten Fuhs, Carsten Otto, Martin Plücker, Peter Schneider-Kamp, Thomas Ströder, Stephanie Swiderski, and René Thiemann. “Proving Termination of Programs Automatically with AProVE”. In: *Automated Reasoning (IJCAR 2014)* 2014. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-08587-6_13.
- [Göd31] Kurt Gödel. “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I”. In: *Monatshefte für Mathematik und Physik* 38.1 (1931). DOI: 10.1007/BF01700692.
- [GJD20] Stéphane Graham-Lengrand, Dejan Jovanović, and Bruno Dutertre. “Solving Bitvectors with MCSAT: Explanations from Bits and Pieces”. In: *Automated Reasoning (IJCAR 2020)* 2020. Lecture Notes in Computer Science vol. 12166. DOI: 10.1007/978-3-030-51074-9_7.
- [Gt91] Torbjörn Granlund and the GMP development team. *GNU Multiple Precision Arithmetic Library*. 1991. URL: <https://gmplib.org/>.
- [Gri88] D. Yu Grigor’ev. “Complexity of deciding Tarski algebra”. In: *Journal of Symbolic Computation* 5.1-2 (1988). DOI: 10.1016/S0747-7171(88)80006-3.
- [GV88] D. Yu. Grigor’ev and N.N. Vorobjov. “Solving systems of polynomial inequalities in subexponential time”. In: *Journal of Symbolic Computation* 5.1-2 (1988). DOI: 10.1016/S0747-7171(88)80005-1.
- [Har22] Max Harder. “Generating coverings using virtual substitution for explanations in mcSAT”. Bachelor’s thesis. RWTH Aachen University, 2022. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/Bachelorarbeit_MaxHarder.pdf.
- [Hat01] Allen Hatcher. *Algebraic Topology*. New York: Cambridge University Press, 2001. ISBN: 978-0-521-79540-1. URL: <https://pi.math.cornell.edu/~hatcher/AT/ATpage.html>.
- [HS17] Jonathan D. Hauenstein and Andrew J. Sommese. “What is numerical algebraic geometry?” In: *Journal of Symbolic Computation* 79 (2017). DOI: 10.1016/j.jsc.2016.07.015.
- [Hei20] Daniel Alexander Heinen. “Purging spurious samples in the cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2020. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/heinen_bachelor.pdf.
- [HRS90] Joos Heintz, Marie-Françoise Roy, and Pablo Solernó. “Sur la complexité du principe de Tarski-Seidenberg”. In: *Bulletin de la Société mathématique de France* 118.1 (1990). DOI: 10.24033/bsmf.2138.
- [Hil22] Kai Hilgers. “An FMplex-inspired simplex heuristics”. Bachelor’s thesis. RWTH Aachen University, 2022. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/hilgers_bachelor.pdf.

- [Hon90] H. Hong. “An improvement of the projection operator in cylindrical algebraic decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 1990)* 1990. DOI: 10.1145/96877.96943.
- [Hon91] Hoon Hong. *Comparison of several decision algorithms for the existential theory of the reals*. Citeseer, 1991. URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=bf258184cd1ad6b9a7428ea3f2b9710b4b076732>.
- [Hon92] Hoon Hong. “Simple solution formula construction in cylindrical algebraic decomposition based quantifier elimination”. In: *Symbolic and Algebraic Computation (ISSAC 1992)* 1992. DOI: 10.1145/143242.143306.
- [HED⁺16] Zongyan Huang, Matthew England, James H. Davenport, and Lawrence C. Paulson. “Using Machine Learning to Decide When to Precondition Cylindrical Algebraic Decomposition with Groebner Bases”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2016)* 2016. DOI: 10.1109/SYNASC.2016.020.
- [HEW⁺14] Zongyan Huang, Matthew England, David Wilson, James H. Davenport, Lawrence C. Paulson, and James Bridge. “Applying Machine Learning to the Problem of Choosing a Heuristic to Select the Variable Ordering for Cylindrical Algebraic Decomposition”. In: *Intelligent Computer Mathematics (CICM 2014)* 2014. Lecture Notes in Computer Science vol. 8543. DOI: 10.1007/978-3-319-08434-3_8.
- [HEW⁺19] Zongyan Huang, Matthew England, David J. Wilson, James Bridge, James H. Davenport, and Lawrence C. Paulson. “Using Machine Learning to Improve Cylindrical Algebraic Decomposition”. In: *Mathematics in Computer Science* 13.4 (2019). DOI: 10.1007/s11786-019-00394-8.
- [IA17] Hidenao Iwane and Hirokazu Anai. “Formula Simplification for Real Quantifier Elimination Using Geometric Invariance”. In: *Symbolic and Algebraic Computation (ISSAC 2017)* 2017. DOI: 10.1145/3087604.3087627.
- [IYA14] Hidenao Iwane, Hitoshi Yanami, and Hirokazu Anai. “SyNRAC: A Toolbox for Solving Real Algebraic Constraints”. In: *Mathematical Software (ICMS 2014)* 2014. Lecture Notes in Computer Science. DOI: 10.1007/978-3-662-44199-2_78.
- [IYA⁺09] Hidenao Iwane, Hitoshi Yanami, Hirokazu Anai, and Kazuhiro Yokoyama. “An effective implementation of a symbolic-numeric cylindrical algebraic decomposition for quantifier elimination”. In: *Symbolic Numeric Computation (SNC 2009)* 2009. DOI: 10.1145/1577190.1577203.
- [IYA⁺13] Hidenao Iwane, Hitoshi Yanami, Hirokazu Anai, and Kazuhiro Yokoyama. “An effective implementation of symbolic-numeric cylindrical algebraic decomposition for quantifier elimination”. In: *Theoretical Computer Science*. Symbolic-Numerical Algorithms 479 (2013). DOI: 10.1016/j.tcs.2012.10.020.
- [Jov17] Dejan Jovanović. “Solving Nonlinear Integer Arithmetic with MCSAT”. In: *Verification, Model Checking, and Abstract Interpretation (VMCAI 2017)* 2017. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-52234-0_18.

- [JBd13] Dejan Jovanović, Clark Barrett, and Leonardo de Moura. “The design and implementation of the model constructing satisfiability calculus”. In: *Formal Methods in Computer-Aided Design (FMCAD 2013)* 2013. DOI: 10.1109/FMCAD.2013.7027033.
- [Jd12] Dejan Jovanović and Leonardo de Moura. “Solving Non-linear Arithmetic”. In: *Automated Reasoning (IJCAR 2012)* 2012. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-31365-3_27.
- [JD17] Dejan Jovanović and Bruno Dutertre. “LibPoly: A Library for Reasoning about Polynomials”. In: *Satisfiability Modulo Theories (SMT 2017)* 2017. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-1889/paper3.pdf>.
- [JD21] Dejan Jovanović and Bruno Dutertre. “Interpolation and Model Checking for Nonlinear Arithmetic”. In: *Computer Aided Verification (CAV 2021)* 2021. Lecture Notes in Computer Science. DOI: 10.1007/978-3-030-81688-9_13.
- [JÁG⁺12] Sebastian Junges, Erika Ábrahám, Jürgen Giesl, and Dipl Inform Ulrich Loup. “On Gröbner bases in SMT-compliant decision procedures”. Bachelor’s thesis. RWTH Aachen University, 2012. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/teaching/theses/junges_bachelor.pdf.
- [Kal03] Erich Kaltofen. “Polynomial factorization: a success story”. In: *Symbolic and Algebraic Computation (ISSAC 2003)* 2003. DOI: 10.1145/860854.860857.
- [Kan60] L. V. Kantorovich. “Mathematical Methods of Organizing and Planning Production”. In: *Management Science* 6.4 (1960). DOI: 10.1287/mnsc.6.4.366.
- [Khwh20] Muhammad ibn Musa al-Khwarizmi. *al-Kitāb al-Mukhtaṣar fī Ḥisāb al-Jabr wal-Muqābalah (The Compendious Book on Calculation by Completion and Balancing)*. 820.
- [KBD13] Tim King, Clark Barrett, and Bruno Dutertre. “Simplex with sum of infeasibilities for SMT”. In: *Formal Methods in Computer-Aided Design (FMCAD 2013)* 2013. DOI: 10.1109/FMCAD.2013.6679409.
- [Kob21] Paul Kobialka. “Connecting simplex and Fourier-Motzkin into a novel quantifier elimination method for linear real algebra”. Master’s Thesis. RWTH Aachen University, 2021. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/kobialka_master.pdf.
- [Kol25] Alexej Kolbin. “Traversal heuristics for cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2025. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/thesis_Kolbin.pdf.
- [KKS14] Konstantin Korovin, Marek Košta, and Thomas Sturm. “Towards Conflict-Driven Learning for Virtual Substitution”. In: *Computer Algebra in Scientific Computing (CASC 2014)* 2014. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-10515-4_19.

- [KTV09] Konstantin Korovin, Nestan Tsiskaridze, and Andrei Voronkov. “Conflict Resolution”. In: *Principles and Practice of Constraint Programming (CP 2009)* 2009. DOI: 10.1007/978-3-642-04244-7_41.
- [Kos23] Katherine Kosaian. “Formally Verifying Algorithms for Real Quantifier Elimination”. PhD thesis. Carnegie Mellon University, 2023. URL: <http://reports-archive.adm.cs.cmu.edu/anon/2023/CMU-CS-23-130.pdf>.
- [Koš16] Marek Košta. “New concepts for real quantifier elimination by virtual substitution”. doctoralThesis. Universität des Saarlandes, 2016. DOI: 10.22028/D291-26679.
- [KS15] Marek Košta and Thomas Sturm. *A Generalized Framework for Virtual Substitution*. 2015. DOI: 10.48550/arXiv.1501.05826.
- [KV13] Laura Kovács and Andrei Voronkov. “First-Order Theorem Proving and Vampire”. In: *Computer Aided Verification (CAV 2013)* 2013. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-39799-8_1.
- [Kre13] Gereon Kremer. “Isolating real roots using adaptable-precision interval arithmetic”. Master’s Thesis. RWTH Aachen University, 2013. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/teaching/theses/kremer_master.pdf.
- [Kre20] Gereon Kremer. “Cylindrical Algebraic Decomposition for Nonlinear Arithmetic Problems”. PhD thesis. RWTH Aachen University, 2020. DOI: 10.18154/RWTH-2020-05913.
- [KÁ20] Gereon Kremer and Erika Ábrahám. “Fully incremental cylindrical algebraic decomposition”. In: *Journal of Symbolic Computation* 100 (2020). DOI: 10.1016/j.jsc.2019.07.018.
- [KÁE⁺21] Gereon Kremer, Erika Ábrahám, Matthew England, and James H. Davenport. “On the Implementation of Cylindrical Algebraic Coverings for Satisfiability Modulo Theories Solving”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2021)* 2021. DOI: 10.1109/SYNASC54541.2021.00018.
- [KN23] Gereon Kremer and Jasper Nalbach. “Cylindrical Algebraic Coverings for Quantifiers”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2022)* 2023. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-3458/paper1.pdf>.
- [KS16] Daniel Kroening and Ofer Strichman. *Decision Procedures*. Texts in Theoretical Computer Science. An EATCS Series. Berlin, Heidelberg: Springer, 2016. ISBN: 978-3-662-50496-3 978-3-662-50497-0. DOI: 10.1007/978-3-662-50497-0.
- [Kro20] Philip Kroll. “Efficient data structures for cylindrical algebraic coverings”. Bachelor’s thesis. RWTH Aachen University, 2020. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/kroll_bachelor.pdf.
- [Kro23] Philip Kroll. “Implementation of cylindrical algebraic coverings for quantifier elimination”. Master’s Thesis. RWTH Aachen University, 2023. URL: <https://ths.rwth-aachen.de/wp-content/uploads/sites/4/MasterPhilipKroll.pdf>.

- [Laz94] D. Lazard. “An Improved Projection for Cylindrical Algebraic Decomposition”. In: *Algebraic Geometry and its Applications: Collections of Papers from Shreeram S. Abhyankar’s 60th Birthday Conference*. Ed. by Chandrajit L. Bajaj. New York, NY: Springer, 1994. DOI: 10.1007/978-1-4612-2628-4_29.
- [LR07] Daniel Lazard and Fabrice Rouillier. “Solving parametric polynomial systems”. In: *Journal of Symbolic Computation* 42.6 (2007). DOI: 10.1016/j.jsc.2007.01.007.
- [LS21] Huu Phuoc Le and Mohab Safey El Din. “Faster One Block Quantifier Elimination for Regular Polynomial Systems of Equations”. In: *Symbolic and Algebraic Computation (ISSAC 2021)* 2021. DOI: 10.1145/3452143.3465546.
- [LXZ⁺21] Haokun Li, Bican Xia, Huiying Zhang, and Tao Zheng. “Choosing the Variable Ordering for Cylindrical Algebraic Decomposition via Exploiting Chordal Structure”. In: *Symbolic and Algebraic Computation (ISSAC 2021)* 2021. DOI: 10.1145/3452143.3465520.
- [LXZ23] Haokun Li, Bican Xia, and Tianqi Zhao. *Local Search for Solving Satisfiability of Polynomial Formulas*. 2023. DOI: 10.48550/arXiv.2303.09072.
- [Lip24] Enrico Lipparini. “Satisfiability modulo Nonlinear Arithmetic and Transcendental Functions via Numerical and Topological methods”. PhD thesis. University of Genova, 2024. URL: <https://iris.unige.it/handle/11567/1224015>.
- [Loo82a] R. Loos. “Computing in Algebraic Extensions”. In: *Computer Algebra: Symbolic and Algebraic Computation*. Ed. by Bruno Buchberger, George Edwin Collins, and Rüdiger Loos. Vienna: Springer, 1982. DOI: 10.1007/978-3-7091-3406-1_12.
- [Loo82b] R. Loos. “Generalized Polynomial Remainder Sequences”. In: *Computer Algebra: Symbolic and Algebraic Computation*. Ed. by Bruno Buchberger, George Edwin Collins, and Rüdiger Loos. Vienna: Springer, 1982. DOI: 10.1007/978-3-7091-3406-1_9.
- [LW93] Rüdiger Loos and Volker Weispfenning. “Applying linear quantifier elimination”. In: *The Computer Journal* 36.5 (1993).
- [LSC⁺13] Ulrich Loup, Karsten Scheibler, Florian Corzilius, Erika Ábrahám, and Bernd Becker. “A Symbiosis of Interval Constraint Propagation and Cylindrical Algebraic Decomposition”. In: *Automated Deduction (CADE-24)* 2013. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-38574-2_13.
- [Mah07] Assia Mahboubi. “Implementing the cylindrical algebraic decomposition within the Coq system”. In: *Mathematical Structures in Computer Science* 17.1 (2007). DOI: 10.1017/S096012950600586X.
- [MNA⁺02] Moez Mahfoudh, Peter Niebert, Eugene Asarin, and Oded Maler. “A satisfiability checker for difference logic”. In: *Proceedings of SAT 2* (2002). URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=061644829078be20f9e993f8861a738d1837ca69>.

- [Map24] Maple. *resultant*. 2024. URL: <https://www.maplesoft.com/support/help/Maple/view.aspx?path=resultant>.
- [MFK⁺16] Alexandre Maréchal, Alexis Fouilhé, Tim King, David Monniaux, and Michael Périn. “Polyhedral Approximation of Multivariate Polynomials Using Handelman’s Theorem”. In: *Verification, Model Checking, and Abstract Interpretation (VMCAI 2016)* 2016. Lecture Notes in Computer Science. DOI: 10.1007/978-3-662-49122-5_8.
- [MS96] J.P. Marques-Silva and K.A. Sakallah. “GRASP-A new search algorithm for satisfiability”. In: *Computer Aided Design (ICCAD 1996)* 1996. DOI: 10.1109/ICCAD.1996.569607.
- [MS99] J.P. Marques-Silva and K.A. Sakallah. “GRASP: a search algorithm for propositional satisfiability”. In: *IEEE Transactions on Computers* 48.5 (1999). DOI: 10.1109/12.769433.
- [McC93] S. McCallum. “Solving Polynomial Strict Inequalities Using Cylindrical Algebraic Decomposition”. In: *The Computer Journal* 36.5 (1993). DOI: 10.1093/comjnl/36.5.432.
- [McC85] Scott McCallum. “An improved projection operation for cylindrical algebraic decomposition”. PhD thesis. University of Wisconsin-Madison, 1985.
- [McC88] Scott McCallum. “An improved projection operation for cylindrical algebraic decomposition of three-dimensional space”. In: *Journal of Symbolic Computation* 5.1-2 (1988). DOI: 10.1016/S0747-7171(88)80010-5.
- [McC98] Scott McCallum. “An Improved Projection Operation for Cylindrical Algebraic Decomposition”. In: *Quantifier Elimination and Cylindrical Algebraic Decomposition* 1998. Texts and Monographs in Symbolic Computation. DOI: 10.1007/978-3-7091-9459-1_12.
- [McC99] Scott McCallum. “On projection in CAD-based quantifier elimination with equational constraint”. In: *Symbolic and Algebraic Computation (ISSAC 1999)* 1999. DOI: 10.1145/309831.309892.
- [McC01] Scott McCallum. “On propagation of equational constraints in CAD-based quantifier elimination”. In: *Symbolic and Algebraic Computation (ISSAC 2001)* 2001. DOI: 10.1145/384101.384132.
- [MB09] Scott McCallum and Christopher W. Brown. “On delineability of varieties in CAD-based quantifier elimination with two equational constraints”. In: *Symbolic and Algebraic Computation (ISSAC 2009)* 2009. DOI: 10.1145/1576702.1576715.
- [MC02] Scott McCallum and George E. Collins. “Local Box Adjacency Algorithms for Cylindrical Algebraic Decompositions”. In: *Journal of Symbolic Computation* 33.3 (2002). DOI: 10.1006/jsc.2001.0499.
- [MH16] Scott McCallum and Hoon Hong. “On using Lazard’s projection in CAD construction”. In: *Journal of Symbolic Computation* 72 (2016). DOI: 10.1016/j.jsc.2015.02.001.
- [MPP19] Scott McCallum, Adam Parusiński, and Laurentiu Paunescu. “Validity proof of Lazard’s method for CAD construction”. In: *Journal of Symbolic Computation* 92 (2019). DOI: 10.1016/j.jsc.2017.12.002.

- [MH05] Sean McLaughlin and John Harrison. “A Proof-Producing Decision Procedure for Real Arithmetic”. In: *Automated Deduction (CADE-20)* 2005. Lecture Notes in Computer Science. DOI: 10.1007/11532231_22.
- [MKS09] Kenneth L. McMillan, Andreas Kuehlmann, and Mooly Sagiv. “Generalizing DPLL to Richer Logics”. In: *Computer Aided Verification (CAV 2009)* 2009. Lecture Notes in Computer Science. DOI: 10.1007/978-3-642-02658-4_35.
- [MNM⁺24] Lucas Michel, Jasper Nalbach, Pierre Mathonet, Naïm Zénaïdi, Christopher W. Brown, Erika Ábrahám, James H. Davenport, and Matthew England. “On Projective Delineability”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2024)* 2024. DOI: 10.1109/SYNASC65383.2024.00015.
- [MP90] B. Mishra and P. Pedersen. “Arithmetic with real algebraic numbers is in NC”. In: *Symbolic and Algebraic Computation (ISSAC 1990)* 1990. DOI: 10.1145/96877.96909.
- [Mis93] Bhubaneswar Mishra. *Algorithmic Algebra*. New York, NY: Springer, 1993. ISBN: 978-1-4612-4344-1. DOI: 10.1007/978-1-4612-4344-1.
- [MT16] Michael Monagan and Baris Tuncer. “Using Sparse Interpolation in Hensel Lifting”. In: *Computer Algebra in Scientific Computing (CASC 2016)* 2016. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-45641-6_25.
- [MT18] Michael Monagan and Baris Tuncer. “Factoring Multivariate Polynomials with Many Factors and Huge Coefficients”. In: *Computer Algebra in Scientific Computing (CASC 2018)* 2018. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-99639-4_22.
- [MMZ⁺01] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. “Chaff: engineering an efficient SAT solver”. In: *Design Automation Conference (DAC 2001)* 2001. DOI: 10.1145/378239.379017.
- [Mot36] Theodore Samuel Motzkin. “Beiträge zur Theorie der linearen Ungleichungen”. PhD thesis. Universität Basel, 1936.
- [Nai21] Akshar S. Nair. “Curtains in Cylindrical Algebraic Decomposition”. PhD thesis. University of Bath, 2021.
- [Nal17] Jasper Nalbach. “Embedding the virtual substitution in the MCSAT framework”. Bachelor’s thesis. RWTH Aachen University, 2017. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/teaching/theses/nalbach_bachelor.pdf.
- [Nal20] Jasper Nalbach. “A novel adaption of the Simplex algorithm for linear real arithmetic”. Master’s Thesis. RWTH Aachen University, 2020. DOI: 10.18154/RWTH-2021-04303.
- [NÁ23] Jasper Nalbach and Erika Ábrahám. “Subtropical Satisfiability for SMT Solving”. In: *NASA Formal Methods (NFM 2023)* 2023. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-33170-1_26.

- [NÁ24] Jasper Nalbach and Erika Ábrahám. “Merging Adjacent Cells During Single Cell Construction”. In: *Computer Algebra in Scientific Computing (CASC 2024)* 2024. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-69070-9_15.
- [NÁ25] Jasper Nalbach and Erika Ábrahám. “A Variant of Non-uniform Cylindrical Algebraic Decomposition for Real Quantifier Elimination”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025)* (accepted, in print) 2025. DOI: 10.48550/arXiv.2508.00505.
- [NÁK21] Jasper Nalbach, Erika Ábrahám, and Gereon Kremer. “Extending the Fundamental Theorem of Linear Programming for Strict Inequalities”. In: *Symbolic and Algebraic Computation (ISSAC 2021)* 2021. DOI: 10.1145/3452143.3465538.
- [NÁS⁺24] Jasper Nalbach, Erika Ábrahám, Philippe Specht, Christopher W. Brown, James H. Davenport, and Matthew England. “Levelwise construction of a single cylindrical algebraic cell”. In: *Journal of Symbolic Computation* 123 (2024). DOI: 10.1016/j.jsc.2023.102288.
- [NK25] Jasper Nalbach and Gereon Kremer. “Extensions of the Cylindrical Algebraic Covering Method for Quantifiers”. In: *Mathematics in Computer Science (accepted, in print)* (2025). DOI: 10.48550/arXiv.2411.03070.
- [NKÁ19] Jasper Nalbach, Gereon Kremer, and Erika Ábrahám. “On variable orderings in MCSAT for non-linear real arithmetic”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2019)* 2019. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-2460/paper5.pdf>.
- [NMÁ⁺25] Jasper Nalbach, Lucas Michel, Erika Ábrahám, Christopher Brown, James H. Davenport, Matthew England, Pierre Mathonet, and Naïm Zénaïdi. “Projective Delineability for Single Cell Construction”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025)* (accepted, in print) 2025. DOI: 10.48550/arXiv.2508.00512.
- [NPÁ⁺23] Jasper Nalbach, Valentin Promies, Erika Ábrahám, and Paul Kobialka. “FMplex: A Novel Method for Solving Linear Real Arithmetic Problems”. In: *Games, Automata, Logics, and Formal Verification (GandALF 2023)* 2023. Vol. 390. DOI: 10.4204/EPTCS.390.2.
- [NO79] Greg Nelson and Derek C. Oppen. “Simplification by Cooperating Decision Procedures”. In: *ACM Transactions on Programming Languages and Systems* 1.2 (1979). DOI: 10.1145/357073.357079.
- [NPR⁺21] Aina Niemetz, Mathias Preiner, Andrew Reynolds, Clark Barrett, and Cesare Tinelli. “Syntax-Guided Quantifier Instantiation”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)* 2021. Lecture Notes in Computer Science vol. 12652. DOI: 10.1007/978-3-030-72013-1_8.
- [NO05] Robert Nieuwenhuis and Albert Oliveras. “DPLL(T) with Exhaustive Theory Propagation and Its Application to Difference Logic”. In: *Computer Aided Verification (CAV 2005)* 2005. Lecture Notes in Computer Science. DOI: 10.1007/11513988_33.

- [NO06] Robert Nieuwenhuis and Albert Oliveras. “On SAT Modulo Theories and Optimization Problems”. In: *Theory and Applications of Satisfiability Testing (SAT 2006)* 2006. Lecture Notes in Computer Science. DOI: 10.1007/11814948_18.
- [NOT06] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. “Solving SAT and SAT Modulo Theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)”. In: *Journal of the ACM* 53.6 (2006). DOI: 10.1145/1217856.1217859.
- [Pau94] Lawrence C. Paulson, ed. *Isabelle*. Vol. 828. Lecture Notes in Computer Science. Berlin/Heidelberg: Springer, 1994. ISBN: 978-3-540-58244-1. DOI: 10.1007/BFb0030541.
- [Per24] Carsten Peter Perkampus. “Speed up single cell construction via combinatorial optimization”. Bachelor’s thesis. RWTH Aachen University, 2024. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/thesis_perkampus.pdf.
- [PdE⁺24] Lynn Pickering, Tereso del Río, Matthew England, and Kelly Cohen. “Explainable AI Insights for Symbolic Computation: A case study on selecting the variable ordering for cylindrical algebraic decomposition”. In: *Journal of Symbolic Computation* 123 (2024). DOI: 10.1016/j.jsc.2023.102276.
- [Pla17] André Platzer. “A Complete Uniform Substitution Calculus for Differential Dynamic Logic”. In: *Journal of Automated Reasoning* 59.2 (2017). DOI: 10.1007/s10817-016-9385-1.
- [Pon21] André Leon Poncelet. “Conflict generalization for quadratic real-arithmetic constraints in SMT-RAT”. Bachelor’s thesis. RWTH Aachen University, 2021. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/poncelet_bachelor.pdf.
- [PSB⁺24] Mathias Preiner, Hans-Jörg Schurr, Clark Barrett, Pascal Fontaine, Aina Niemetz, and Cesare Tinelli. *SMT-LIB release 2024 (non-incremental benchmarks)*. 2024. DOI: 10.5281/zenodo.11061097.
- [Pre30] Mojzesz Presburger. “Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchen die Addition als einzige Operation hervortritt”. In: *Sprawozdanie z I Kongresu Matematyków Krajów Słowiańskich (Comptes-rendus du Ier Congrès des Mathématiciens des Pays Slaves)* 1930.
- [Pro22] Valentin Promies. “Underapproximating cell bounds in MCSAT using low-degree polynomials”. Master’s Thesis. RWTH Aachen University, 2022. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/master_valentin_promies.pdf.
- [PÁ24] Valentin Promies and Erika Ábrahám. “A Divide-and-Conquer Approach to Variable Elimination in Linear Real Arithmetic”. In: *Formal Methods (FM 2024)* 2024. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-71162-6_7.

- [PNÁ25] Valentin Promies, Jasper Nalbach, and Erika Ábrahám. “More is Less: Adding Polynomials for Faster Explanations in NLSAT”. In: *Automated Deduction (CADE-30)* 2025. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-99984-0_7.
- [PNÁ⁺25] Valentin Promies, Jasper Nalbach, Erika Ábrahám, and Paul Kobialka. “FMplex: Exploring a Bridge between Fourier-Motzkin and Simplex”. In: *Logical Methods in Computer Science* Volume 21, Issue 2 (2025). DOI: 10.46298/lmcs-21(2:6)2025.
- [Ren88] James Renegar. *A faster PSPACE algorithm for deciding the existential theory of the reals*. Tech. rep. Cornell University Operations Research and Industrial Engineering, 1988. URL: <https://ecommons.cornell.edu/bitstream/handle/1813/8675/TR000792.pdf?sequence=1>.
- [Ren92a] James Renegar. “On the computational complexity and geometry of the first-order theory of the reals. Part I: Introduction. Preliminaries. The geometry of semi-algebraic sets. The decision problem for the existential theory of the reals”. In: *Journal of Symbolic Computation* 13.3 (1992). DOI: 10.1016/S0747-7171(10)80003-3.
- [Ren92b] James Renegar. “On the computational complexity and geometry of the first-order theory of the reals. Part II: The general decision problem. Preliminaries for quantifier elimination”. In: *Journal of Symbolic Computation* 13.3 (1992). DOI: 10.1016/S0747-7171(10)80004-5.
- [Ren92c] James Renegar. “On the computational complexity and geometry of the first-order theory of the reals. Part III: Quantifier elimination”. In: *Journal of Symbolic Computation* 13.3 (1992). DOI: 10.1016/S0747-7171(10)80005-7.
- [Ros36] John Barkley Rosser. “Extensions of some Theorems of Gödel and Church”. In: *Journal of Symbolic Logic* 1.3 (1936). DOI: 10.2307/2269028.
- [RS21] Gergely Röst and AmirHosein Sadeghimanesh. “Exotic Bifurcations in Three Connected Populations with Allee Effect”. In: *International Journal of Bifurcation and Chaos* 31.13 (2021). DOI: 10.1142/S0218127421502023.
- [RZ04] Fabrice Rouillier and Paul Zimmermann. “Efficient isolation of polynomial’s real roots”. In: *Journal of Computational and Applied Mathematics*. Proceedings of the International Conference on Linear Algebra and Arithmetic 2001 162.1 (2004). DOI: 10.1016/j.cam.2003.08.015.
- [Rub74] Cyrenus M Rubald. “Algorithms for polynomials over a real algebraic number field”. PhD thesis. University of Wisconsin-Madison, Department of Computer Sciences, 1974.
- [SM16] Michael Sagraloff and Kurt Mehlhorn. “Computing real roots of real polynomials”. In: *Journal of Symbolic Computation* 73 (2016). DOI: 10.1016/j.jsc.2015.03.004.
- [SCM⁺21] Matias Scharager, Katherine Cordwell, Stefan Mitsch, and André Platzer. “Verified Quadratic Virtual Substitution for Real Arithmetic”. In: *Formal Methods (FM 2021)* 2021. Lecture Notes in Computer Science. DOI: 10.1007/978-3-030-90870-6_11.

- [Sch03] A. Schrijver. *Combinatorial optimization: polyhedra and efficiency*. Algorithms and combinatorics 24. Berlin ; New York: Springer, 2003. ISBN: 978-3-540-44389-6. URL: <https://link.springer.com/book/9783540443896>.
- [SFB⁺21] Hans-Jörg Schurr, Mathias Fleury, Haniel Barbosa, and Pascal Fontaine. “Alethe: Towards a Generic SMT Proof Format (extended abstract)”. In: *Proof eXchange for Theorem Proving (PxTP 2021)* 2021. Vol. 336. DOI: 10.4204/EPTCS.336.6.
- [SS83] Jacob T Schwartz and Micha Sharir. “On the “piano movers” problem. II. General techniques for computing topological properties of real algebraic manifolds”. In: *Advances in Applied Mathematics* 4.3 (1983). DOI: 10.1016/0196-8858(83)90014-3.
- [Seb07] Roberto Sebastiani. “Lazy Satisfiability Modulo Theories”. In: *Journal on Satisfiability, Boolean Modeling and Computation* 3.3-4 (2007). DOI: 10.3233/SAT190034.
- [ST15] Roberto Sebastiani and Patrick Trentin. “OptiMathSAT: A Tool for Optimization Modulo Theories”. In: *Computer Aided Verification (CAV 2015)* 2015. Lecture Notes in Computer Science vol. 9206. DOI: 10.1007/978-3-319-21690-4_27.
- [Sei54] Abraham Seidenberg. “A new decision method for elementary algebra”. In: *Annals of Mathematics* (1954). DOI: 10.2307/1969640. JSTOR: 1969640.
- [SS03] Andreas Seidl and Thomas Sturm. “A generic projection operator for partial cylindrical algebraic decomposition”. In: *Symbolic and Algebraic Computation (ISSAC 2003)* 2003. DOI: 10.1145/860854.860903.
- [Sho78] Robert E. Shostak. “An algorithm for reasoning about equality”. In: *Communications of the ACM* 21.7 (1978). DOI: 10.1145/359545.359570.
- [Spe20] Philippe Specht. “A level-wise variant of single cell construction in cylindrical algebraic decomposition”. Bachelor’s thesis. RWTH Aachen University, 2020. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/teaching/theses/specht_bachelor.pdf.
- [Spe23] Philippe Specht. “Improving incremental linearization for satisfiability modulo non-linear real arithmetic checking”. Master’s Thesis. RWTH Aachen University, 2023. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/master_specht.pdf.
- [Ste22] Svenja Stein. “An incremental adaption of the FMplex method for solving linear real algebraic formulas”. Bachelor’s thesis. RWTH Aachen University, 2022. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/Bachelor_Stein_Svenja.pdf.
- [Ste58] Moritz Stern. “Ueber eine zahlentheoretische Funktion.” In: *Journal für die reine und angewandte Mathematik (Crelles Journal)* 1858.55 (1858). DOI: 10.1515/crll.1858.55.193.
- [Str00] Adam Strzeboński. “Solving Systems of Strict Polynomial Inequalities”. In: *Journal of Symbolic Computation* 29.3 (2000). DOI: 10.1006/jSCO.1999.0327.

- [Str14a] Adam Strzeboński. “Cylindrical algebraic decomposition using local projections”. In: *Symbolic and Algebraic Computation (ISSAC 2014)* 2014. DOI: 10.1145/2608628.2608633.
- [Str14b] Adam Strzeboński. *Divide-And-Conquer Computation of Cylindrical Algebraic Decomposition*. 2014. URL: <http://arxiv.org/abs/1402.0622>.
- [Str17] Adam Strzeboński. “CAD Adjacency Computation Using Validated Numerics”. In: *Symbolic and Algebraic Computation (ISSAC 2017)* 2017. DOI: 10.1145/3087604.3087641.
- [Stu29] Jacques Charles François Sturm. “Mémoire sur la résolution des équations numériques”. In: *Bulletin des Sciences de Férussac* 11 (1829).
- [Stu99] Thomas Sturm. “Real Quantifier Elimination in Geometry”. PhD thesis. Universität Passau, 1999.
- [Stu15] Thomas Sturm. “Subtropical Real Root Finding”. In: *Symbolic and Algebraic Computation (ISSAC 2015)* 2015. DOI: 10.1145/2755996.2756677.
- [Stu17] Thomas Sturm. “A Survey of Some Methods for Real Quantifier Elimination, Decision, and Satisfiability and Their Applications”. In: *Mathematics in Computer Science* 11.3 (2017). DOI: 10.1007/s11786-017-0319-z.
- [Tar51] Alfred Tarski. “A Decision Method For Elementary Algebra And Geometry”. In: 1951. DOI: 10.1007/978-3-7091-9459-1_3.
- [Tin02] Cesare Tinelli. “A DPLL-Based Calculus for Ground Satisfiability Modulo Theories”. In: *Logics in Artificial Intelligence (JELIA 2002)* 2002. DOI: 10.1007/3-540-45757-7_26.
- [Tra84] B.A. Trakhtenbrot. “A Survey of Russian Approaches to Perebor (Brute-Force Searches) Algorithms”. In: *Annals of the History of Computing* 6.4 (1984). DOI: 10.1109/MAHC.1984.10036.
- [Tse83] G. S. Tseitin. “On the Complexity of Derivation in Propositional Calculus”. In: *Automation of Reasoning: 2: Classical Papers on Computational Logic 1967–1970*. Ed. by Jörg H. Siekmann and Graham Wrightson. Berlin, Heidelberg: Springer, 1983. DOI: 10.1007/978-3-642-81955-1_28.
- [TE06] Elias P. Tsigaridas and Ioannis Z. Emiris. “Univariate Polynomial Real Root Isolation: Continued Fractions Revisited”. In: *Algorithms (ESA 2006)* 2006. DOI: 10.1007/11841036_72.
- [TVO16] Vu Xuan Tung, To Van Khanh, and Mizuhito Ogawa. “raSAT: An SMT Solver for Polynomial Constraints”. In: *Automated Reasoning (IJCAR 2016)* 2016. Lecture Notes in Computer Science. DOI: 10.1007/978-3-319-40229-1_16.
- [Tur37] A. M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* s2-42.1 (1937). DOI: 10.1112/plms/s2-42.1.230.
- [Unc22] Ali Uncu. *Bruno Buchberger - 15 Minutes Academics Advice*. 2022. URL: <https://www.youtube.com/watch?v=qZhYMeBBwqM>.
- [VB18] Fernando Vale-Enriquez and Christopher W. Brown. “Polynomial Constraints and Unsat Cores in Tarski”. In: *Mathematical Software (ICMS 2018)* 2018. Lecture Notes in Computer Science vol. 10931. DOI: 10.1007/978-3-319-96418-8_55.

- [vR85] Cornelis van de Panne and Farhood Rahnema. “The first algorithm for linear programming: An analysis of Kantorovich’s method”. In: *Economics of Planning* 19.2 (1985). DOI: 10.1007/BF00348824.
- [VKÁ17] Tarik Viehmann, Gereon Kremer, and Erika Ábrahám. “Comparing Different Projection Operators in the Cylindrical Algebraic Decomposition for SMT Solving”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2017)* 2017. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-1974/RP2.pdf>.
- [Vin36] Alexandre Joseph Hidulphe Vincent. “Sur la resolution des équations numériques”. In: *Journal de Mathématiques Pures et Appliquées* 1 (1836). URL: <https://gallica.bnf.fr/ark:/12148/bpt6k57787134/f5.item.r=Agence%20Rol.langEN>.
- [Wag23] Paul Tristan Wagner. “Piecewise linear under-approximation of cell boundaries in MCSAT”. Bachelor’s thesis. RWTH Aachen University, 2023. URL: https://ths.rwth-aachen.de/wp-content/uploads/sites/4/bachelor_thesis_paul_wagner.pdf.
- [Wan78] Paul S. Wang. “An improved multivariate polynomial factoring algorithm”. In: *Mathematics of Computation* 32.144 (1978). DOI: 10.1090/S0025-5718-1978-0568284-3.
- [WCD⁺19] Tjark Weber, Sylvain Conchon, David Déharbe, Matthias Heizmann, Aina Niemetz, and Giles Reger. “The SMT Competition 2015–2018”. In: *Journal on Satisfiability, Boolean Modeling and Computation* 11.1 (2019). DOI: 10.3233/SAT190123.
- [Wei88] Volker Weispfenning. “The complexity of linear problems in fields”. In: *Journal of Symbolic Computation* 5.1-2 (1988). DOI: 10.1016/S0747-7171(88)80003-8.
- [Wei92] Volker Weispfenning. “Comprehensive Gröbner bases”. In: *Journal of Symbolic Computation* 14.1 (1992). DOI: 10.1016/0747-7171(92)90023-W.
- [Wei94] Volker Weispfenning. “Quantifier elimination for real algebra - the cubic case”. In: *Symbolic and Algebraic Computation (ISSAC 1994)* 1994. DOI: 10.1145/190347.190425.
- [Wei97] Volker Weispfenning. “Quantifier Elimination for Real Algebra — The Quadratic Case and Beyond”. In: *Applicable Algebra in Engineering, Communication and Computing* 8.2 (1997). URL: <https://doi.org/10.1007/s002000050055>.
- [Wei24] Eric W. Weisstein. *Resultant*. Text. 2024. URL: <https://mathworld.wolfram.com/>.
- [WHH14] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. “DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs”. In: *Theory and Applications of Satisfiability Testing (SAT 2014)* 2014. DOI: 10.1007/978-3-319-09284-3_31.
- [WR10] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*. Cambridge University Press, 1910.

- [Wil13] David Wilson. *Real geometry and connectedness via triangular description: CAD example bank*. 2013. URL: <https://researchdata.bath.ac.uk/69/>.
- [WEB⁺14] David Wilson, Matthew England, Russell Bradford, and James H. Davenport. “Using the Distribution of Cells by Dimension in a Cylindrical Algebraic Decomposition”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2014)* 2014. DOI: 10.1109/SYNASC.2014.15.
- [WBD12] David J. Wilson, Russell J. Bradford, and James H. Davenport. “Speeding Up Cylindrical Algebraic Decomposition by Gröbner Bases”. In: *Intelligent Computer Mathematics (CICM 2012)* 2012. Lecture Notes in Computer Science vol. 7362. DOI: 10.1007/978-3-642-31374-5_19.
- [WM97] D.H. Wolpert and W.G. Macready. “No free lunch theorems for optimization”. In: *IEEE Transactions on Evolutionary Computation* 1.1 (1997). DOI: 10.1109/4235.585893.
- [WM95] David H. Wolpert and William G. Macready. *No free lunch theorems for search*. Tech. rep. The Santa Fe Institute, 1995. URL: <https://www.santafe.edu/research/results/working-papers/no-free-lunch-theorems-for-search>.
- [XVD⁺16] Jun Xu, Ton J. J. Van Den Boom, Bart De Schutter, and Xiong-Lin Luo. “Minimal Conjunctive Normal Expression of Continuous Piecewise Affine Functions”. In: *IEEE Transactions on Automatic Control* 61.5 (2016). DOI: 10.1109/TAC.2015.2465212.
- [XvD14] Jun Xu, Ton J.J. van den Boom, and Bart De Schutter. “Irredundant lattice piecewise affine representations and their applications in explicit model predictive control”. In: *Decision and Control (CDC 2014)* 2014. DOI: 10.1109/CDC.2014.7040078.
- [Yun74] David Y. Yun. “The Hensel Lemma in Algebraic Manipulation”. PhD thesis. Massachusetts Institute of Technology, 1974. URL: <https://apps.dtic.mil/sti/pdfs/ADA002737.pdf>.

List of Publications

This work is partly based on the following peer-reviewed papers that have already been published. Please refer to Section 1.2.1 for a summary of the author’s contributions to the relevant publications.

- [ÁNP23] Erika Ábrahám, Jasper Nalbach, and Valentin Promies. “Automated Exercise Generation for Satisfiability Checking”. In: *Formal Methods Teaching (FMTea 2023)* 2023. Lecture Notes in Computer Science vol. 13962. DOI: 10.1007/978-3-031-27534-0_1.
- [BNÁ⁺23] Philipp Bär, Jasper Nalbach, Erika Ábrahám, and Christopher W. Brown. “Exploiting Strict Constraints in the Cylindrical Algebraic Covering”. In: *Satisfiability Modulo Theories (SMT 2023)* 2023. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-3429/paper13.pdf>.
- [KN23] Gereon Kremer and Jasper Nalbach. “Cylindrical Algebraic Coverings for Quantifiers”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2022)* 2023. CEUR Workshop Proceedings. URL: <https://ceur-ws.org/Vol-3458/paper1.pdf>.
- [MNM⁺24] Lucas Michel, Jasper Nalbach, Pierre Mathonet, Naïm Zénaïdi, Christopher W. Brown, Erika Ábrahám, James H. Davenport, and Matthew England. “On Projective Delineability”. In: *Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2024)* 2024. DOI: 10.1109/SYNASC65383.2024.00015.
- [NÁ23] Jasper Nalbach and Erika Ábrahám. “Subtropical Satisfiability for SMT Solving”. In: *NASA Formal Methods (NFM 2023)* 2023. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-33170-1_26.
- [NÁ24] Jasper Nalbach and Erika Ábrahám. “Merging Adjacent Cells During Single Cell Construction”. In: *Computer Algebra in Scientific Computing (CASC 2024)* 2024. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-69070-9_15.
- [NÁ25] Jasper Nalbach and Erika Ábrahám. “A Variant of Non-uniform Cylindrical Algebraic Decomposition for Real Quantifier Elimination”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025) (accepted, in print)* 2025. DOI: 10.48550/arXiv.2508.00505.
- [NÁK21] Jasper Nalbach, Erika Ábrahám, and Gereon Kremer. “Extending the Fundamental Theorem of Linear Programming for Strict Inequalities”. In: *Symbolic and Algebraic Computation (ISSAC 2021)* 2021. DOI: 10.1145/3452143.3465538.

- [NÁS⁺24] Jasper Nalbach, Erika Ábrahám, Philippe Specht, Christopher W. Brown, James H. Davenport, and Matthew England. “Levelwise construction of a single cylindrical algebraic cell”. In: *Journal of Symbolic Computation* 123 (2024). DOI: 10.1016/j.jsc.2023.102288.
- [NK25] Jasper Nalbach and Gereon Kremer. “Extensions of the Cylindrical Algebraic Covering Method for Quantifiers”. In: *Mathematics in Computer Science (accepted, in print)* (2025). DOI: 10.48550/arXiv.2411.03070.
- [NMÁ⁺25] Jasper Nalbach, Lucas Michel, Erika Ábrahám, Christopher Brown, James H. Davenport, Matthew England, Pierre Mathonet, and Naïm Zénaïdi. “Projective Delineability for Single Cell Construction”. In: *Satisfiability Checking and Symbolic Computation (SC-square 2025) (accepted, in print)* 2025. DOI: 10.48550/arXiv.2508.00512.
- [NPÁ⁺23] Jasper Nalbach, Valentin Promies, Erika Ábrahám, and Paul Kobiálka. “FMplex: A Novel Method for Solving Linear Real Arithmetic Problems”. In: *Games, Automata, Logics, and Formal Verification (GandALF 2023)* 2023. Vol. 390. DOI: 10.4204/EPTCS.390.2.
- [PNÁ25] Valentin Promies, Jasper Nalbach, and Erika Ábrahám. “More is Less: Adding Polynomials for Faster Explanations in NLSAT”. In: *Automated Deduction (CADE-30)* 2025. Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-99984-0_7.
- [PNÁ⁺25] Valentin Promies, Jasper Nalbach, Erika Ábrahám, and Paul Kobiálka. “FMplex: Exploring a Bridge between Fourier-Motzkin and Simplex”. In: *Logical Methods in Computer Science* Volume 21, Issue 2 (2025). DOI: 10.46298/lmcs-21(2:6)2025.